Image Processing

Department of Computer Science & Engineering, KUET

Dr. Sk. Masudul Ahsan's Lectures (Lec 01 ? Lec 10)

Comprehensive Course Slide Deck with Embedded Navigation & Table of Contents

Table of Contents

1 Lec_01_IPnCG intro p. 3

2 Lec_02_formation_acqusition p. 35

3 Lec_02_Pixel Neighborhood p. 63

4 Lec_03_Point Operations p. 97

5 Lec_04_Spatial Filtering_convlution p. 176

6 Lec_05_Spatial Filtering_Enhancement p. 221

7 Lec_06_Dft_1D p. 272

8 Lec_06_DFT_2D p. 310

9 Lec_06_DFT_properties p. 363

10 Lec_07_Filtering in FD p. 429

11 Lec_08_Restoration_Basics_spatial p. 477

12 Lec_09_Restoration_FD p. 540

13 Lec_10_Segmentaion_threshold_edge p. 602

CSE 4101

COMPUTER GRAPHICS AND IMAGE PROCESSING

IMAGE PROCESSING

Introduction

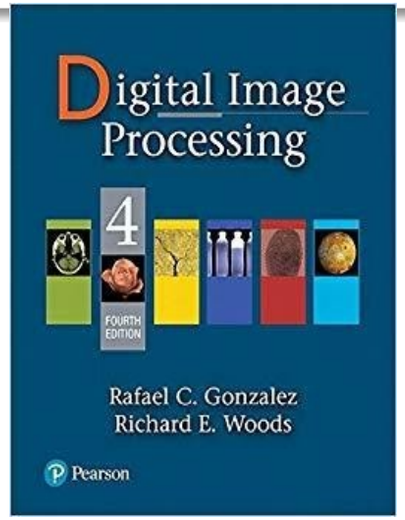
Dr. Sk. Md. Masudul Ahsan

Professor, Dept of CSE, KUET

Syllabus

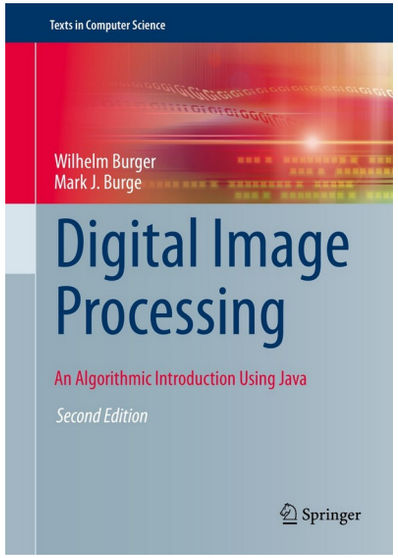
- Image Acquisition
- Point Operations
  - Basics, Histogram processing
- Spatial Operation
  - Convolution, Smoothing, Sharpening
- Frequency Domain and the DFT
  - Image Operations in Freq. Domain
- Image Restoration

Reference books



- **Digital Image Processing, 4th ed**

  Rafael C. Gonzalez & Richard E. Woods,



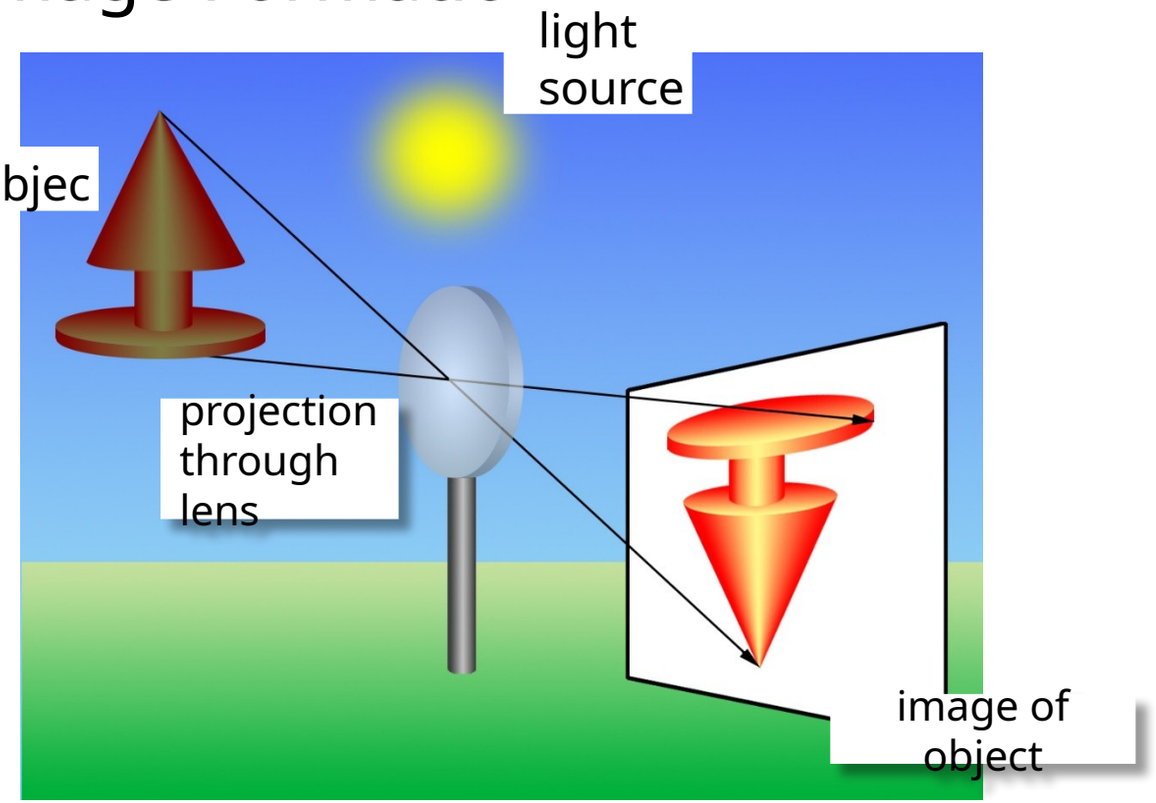
- **Digital Image Processing** An Algorithmic Introduction Using Java

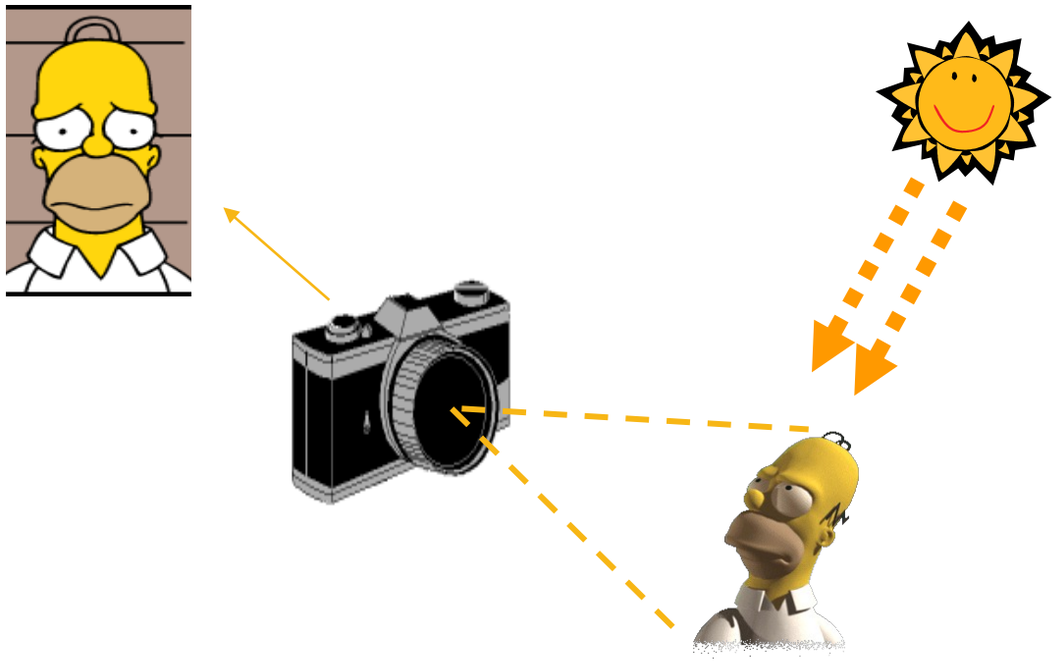
  Wilhelm Burger, Mark J. Burge

What is an image? - Analog Image

- An image is a projection of a 3D scene into a 2D *projection plane*.

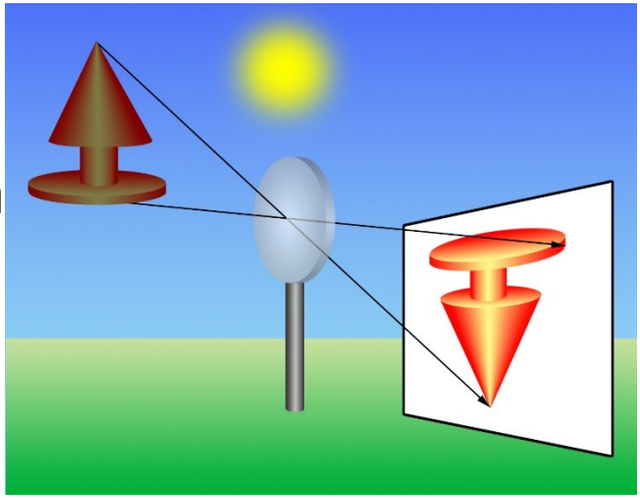
- Image Formation





What is an image? - Analog Image

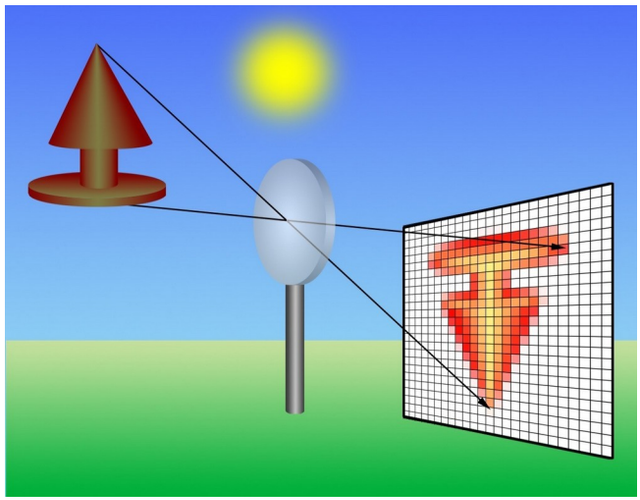
- A 2D light intensity **function** f(x,y), from R^2 to R where:
  - x and y are spatial coordinates
  - f(x,y) - the value of f at any point (x, y)
    - proportional to the brightness or gray value of the image at th



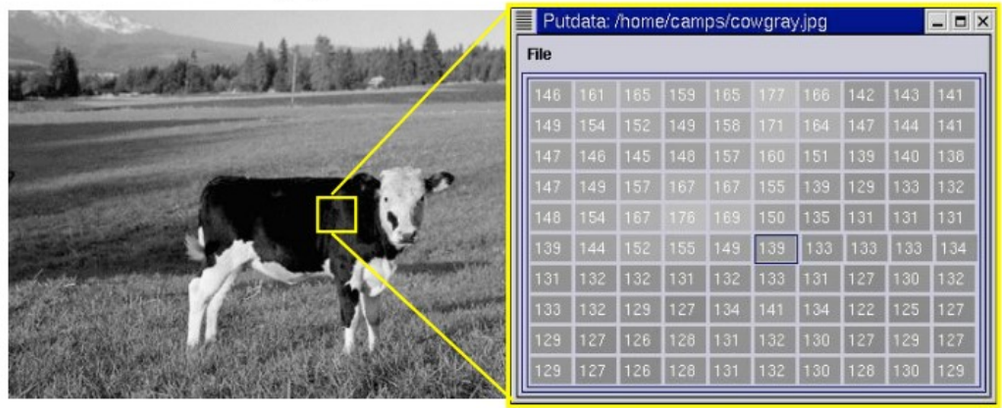
- Realistically, we define with a finite range:
  - f: $[a,b]$x$[c,d]$ → $[0,1]$
  - 0 black; 1 white; in-between gray

Digital Image

- Spatial and grayscale values of f(x,y) are made discrete by -
  - Sampling and Quantization
  - **regularly spaced grid** in x and y directions
  - Intensities are sampled to
    - 8 bits (256 values) per point for grayscale,
- Stored as a 2D arrays of gray-level values.
- The array elements are called **pixels** and identified by their x, y coordinates.



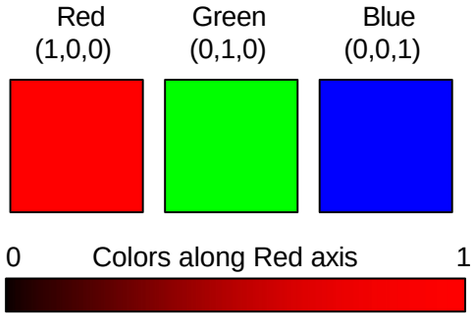
$$f(x,y) = \begin{bmatrix} f(0,0) & f(0,1) & \cdots & f(0,N-1) \\ f(1,0) & f(1,1) & \cdots & f(1,N-1) \\ \vdots & \vdots & \ddots & \vdots \\ f(M,-10) & f(M,-11) & \cdots & f(M,-1N-1) \end{bmatrix}$$

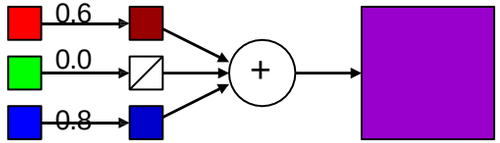


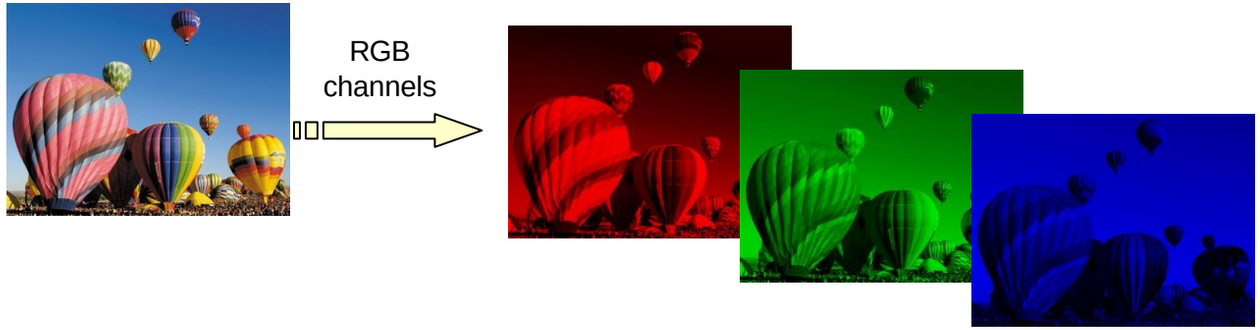
What is an image?

- A color image is just three functions pasted together. We can write this as a “vector-valued” function:

$$f(x,y) = \begin{bmatrix} r(x,y) \\ g(x,y) \\ b(x,y) \end{bmatrix}$$





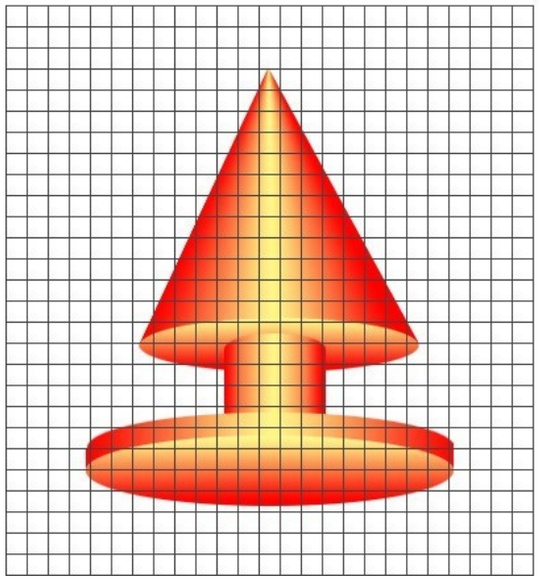


- 3x8 bits per point

Digital Image

- Sampling and Quantization

pixel grid



real image

sampled

quantized

sampled & quantized

Image Types

Three types of images:

- Binary images

  $g(x,y) \in \{0 , 1\}$

- Gray-scale images

  $g(x,y) \in C$

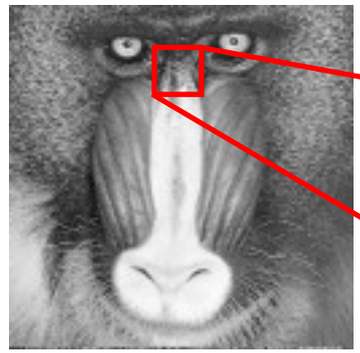
  typically c={0,…,255}

- Color Images

  three channels:

$g_R(x,y) \in C \quad g_G(x,y) \in C \quad g_B(x,y) \in C$

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

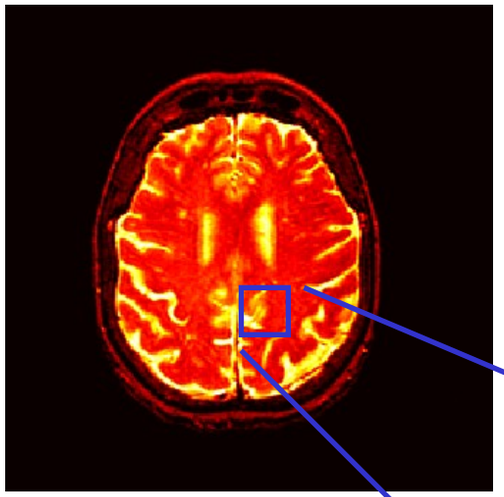


$$\begin{bmatrix} 10 & 10 & 16 & 28 \\ 9 & 6 & 26 & 37 \\ 15 & 25 & 13 & 22 \\ 32 & 15 & 87 & 39 \end{bmatrix}$$

$$\begin{bmatrix} 10 & 10 & 16 & 28 \\ 9 & \begin{bmatrix} 65 & 70 & 56 & 43 \\ 32 & \begin{bmatrix} 99 & 70 & 56 & 78 \\ 60 & 90 & 96 & 67 \\ 85 & 85 & 43 & 92 \\ 32 & 65 & 87 & 99 \end{bmatrix} \\ 21 & 54 \end{bmatrix} \\ 15 & \\ 32 & \end{bmatrix}$$

Digital Image Types : Index Image

- Each pixel contains index number pointing to a color in a color lookup table (palette)
  - Pixel stores an index instead of direct RGB values
  - Reduces storage requirement
- Used in:
  - GIF images
  - Web graphics



$$\begin{bmatrix} 1 & 4 & 9 \\ 6 & 4 & 7 \\ 6 & 5 & 2 \end{bmatrix}$$

Index value

Color Table

Index No.	Red component	Green component	Blue component
1	0.1	0.5	0.3
2	1.0	0.0	0.0
3	0.0	1.0	0.0
4	0.5	0.5	0.5
5	0.2	0.8	0.9
…	…	…	…

Types of Digital Images

Multispectral and Hyperspectral Images

These images capture information beyond visible RGB channels.

Multispectral

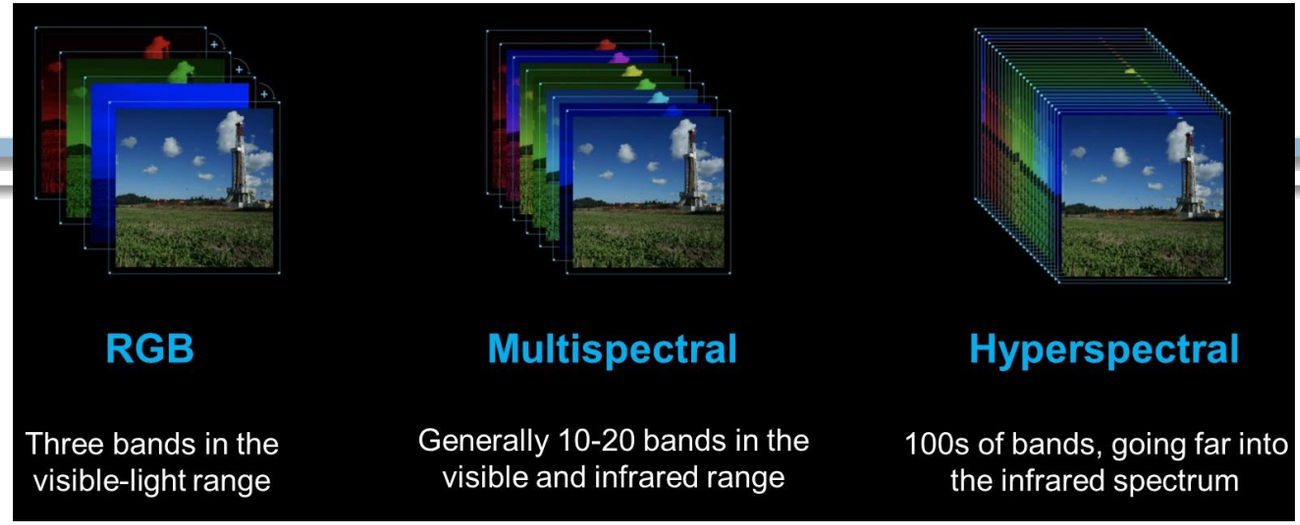
- Few spectral bands
- Example: satellite imaging

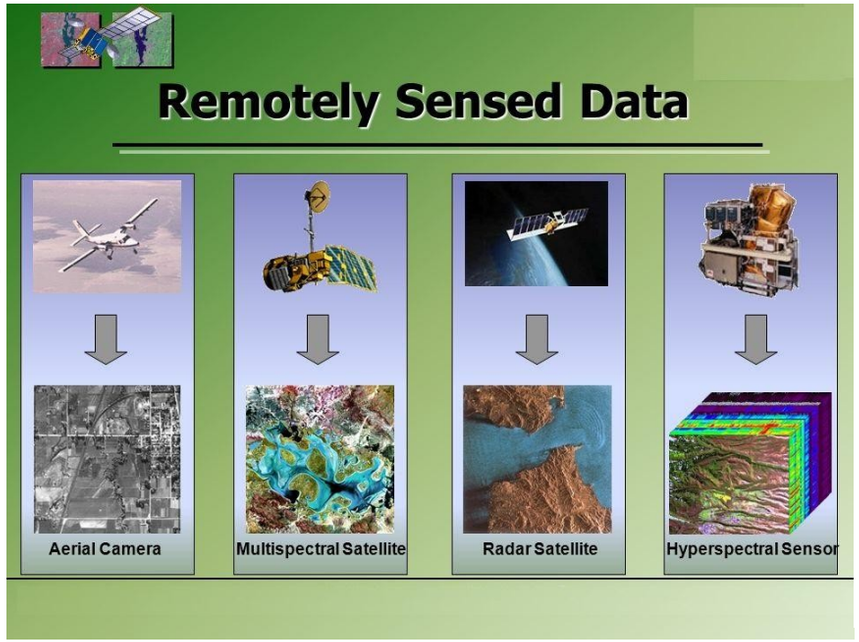
Hyperspectral

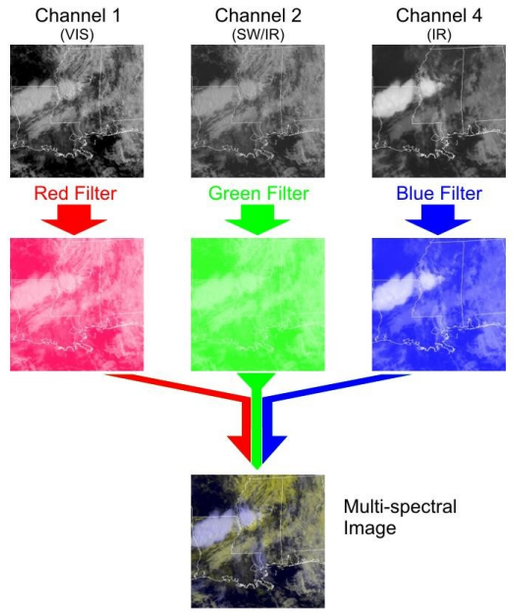
- Hundreds of narrow spectral bands
- Provides detailed material information

Applications:

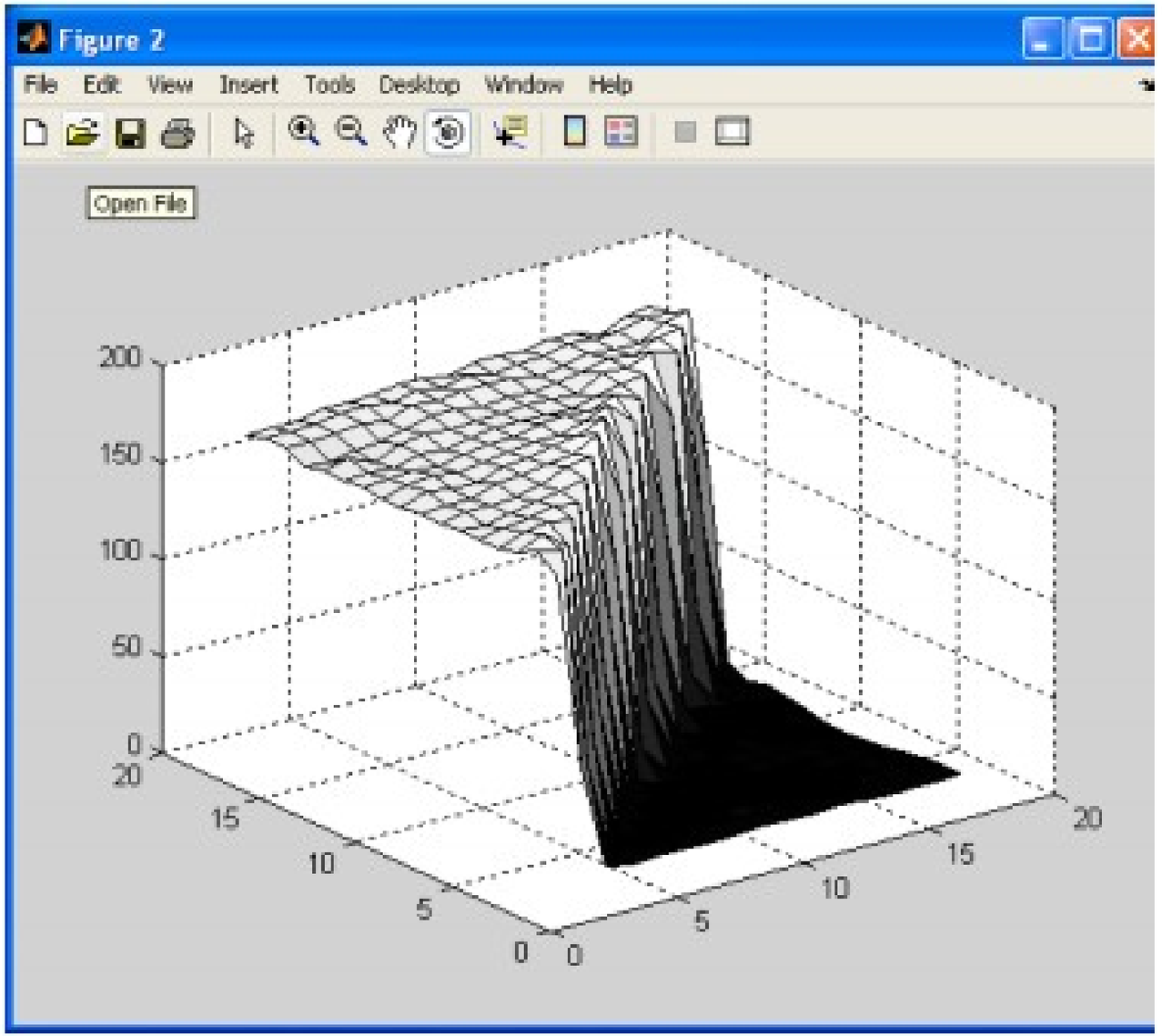
- Agriculture
- Mineral detection
- Environmental monitoring







Images as Surfaces



What is Digital Image Processing?

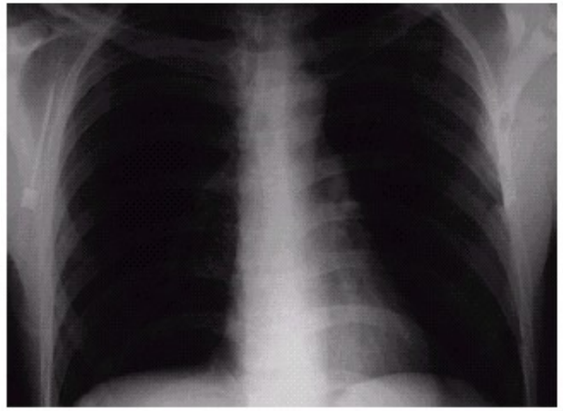
- Focuses on two major tasks
  - Improvement of pictorial information for human interpretation
  - Processing of image data for storage, transmission and representation for autonomous machine perception
- Some argument about where image processing ends and fields such as image analysis and computer vision start

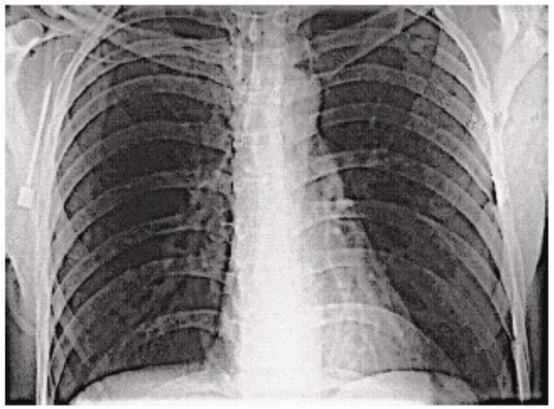
Why we need Digital Image Processing ?

- To save images
- To improve the image using a mathematical process.
- To assist in image analysis.
- To synthesize
- To create a visual system for computers.

Image Processing

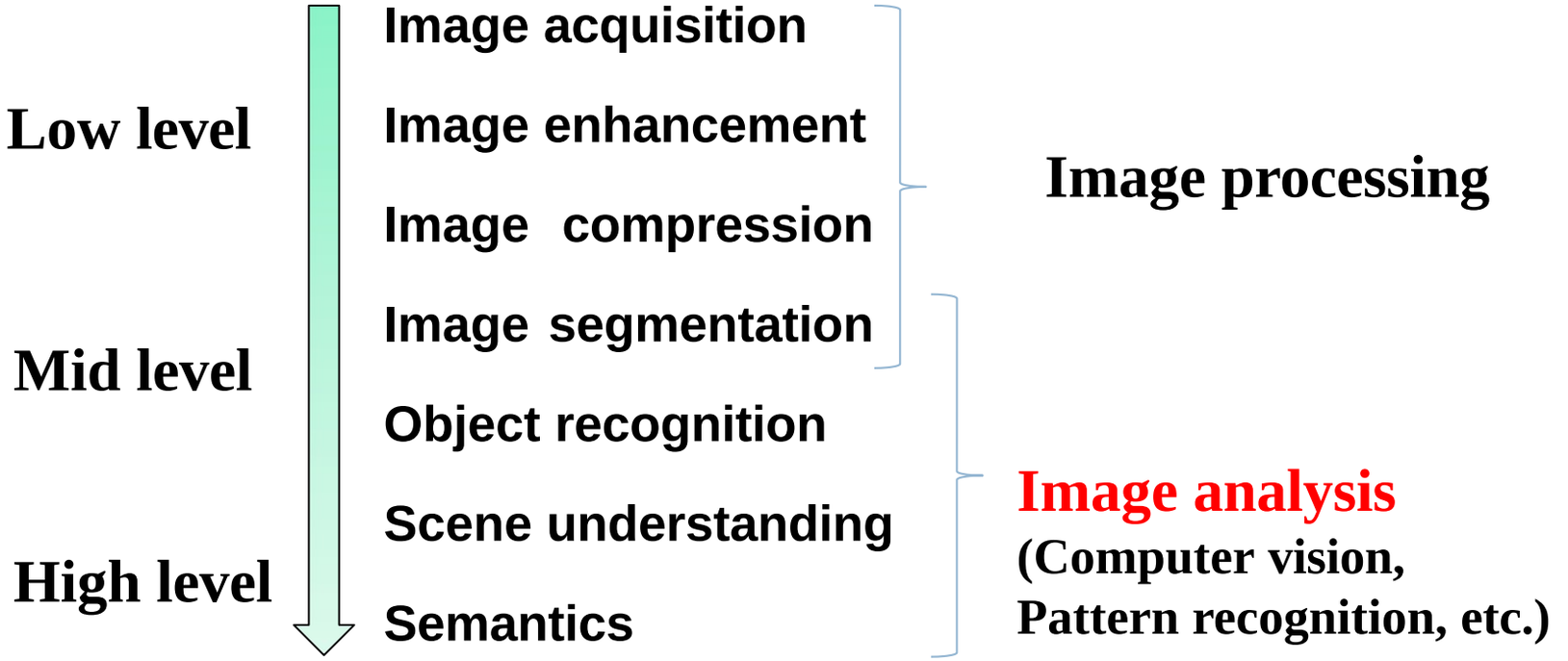
- Image Enhancement



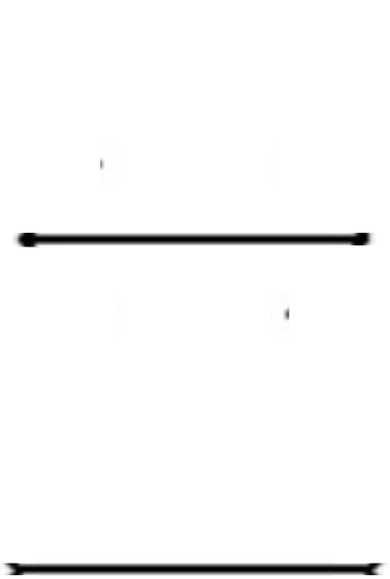


- Segmentation

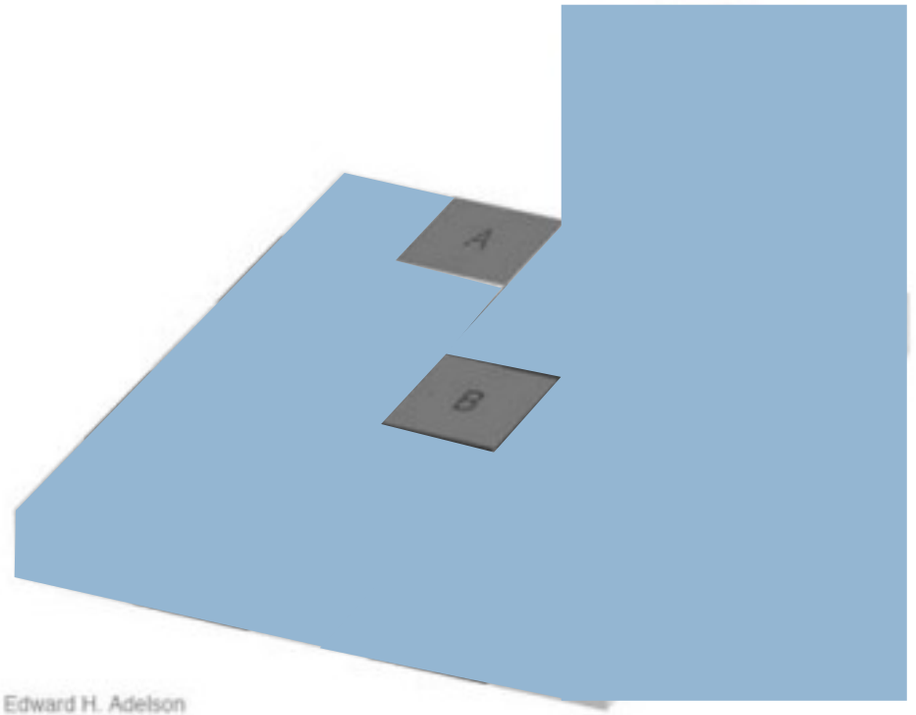
Image Processing Image Analysis



Vision is really hard



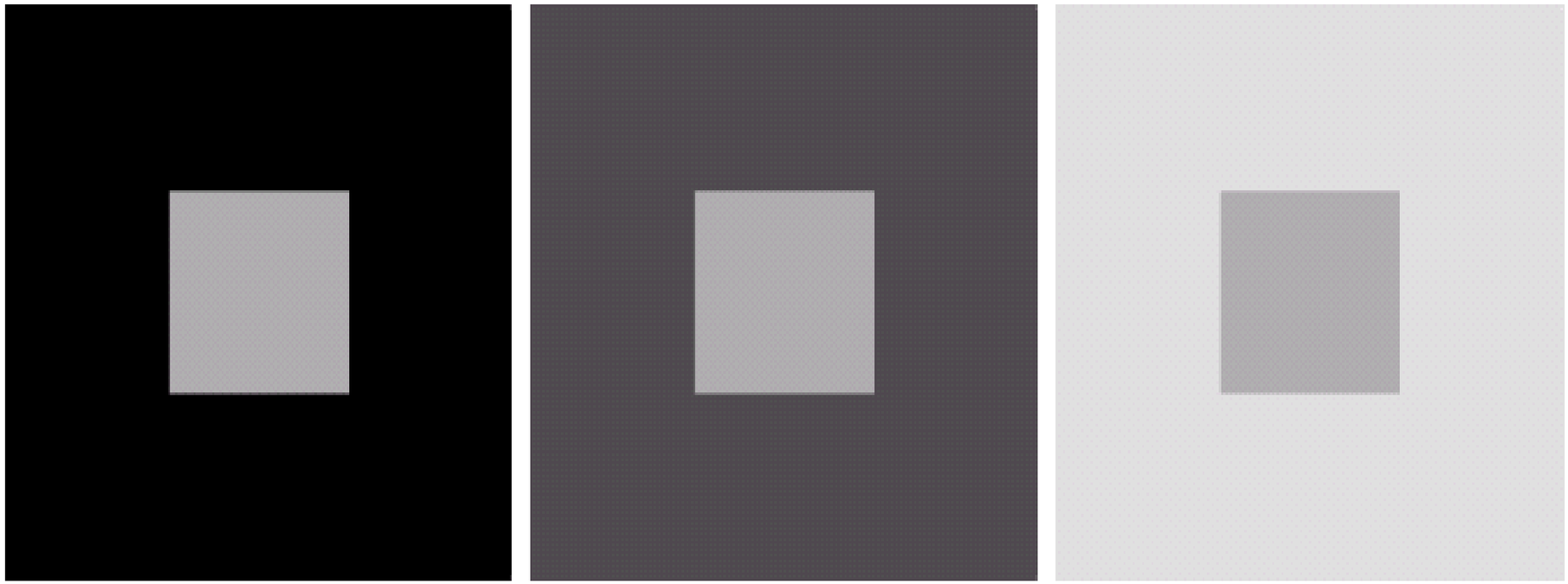
(a)



(b)

Vision is hard

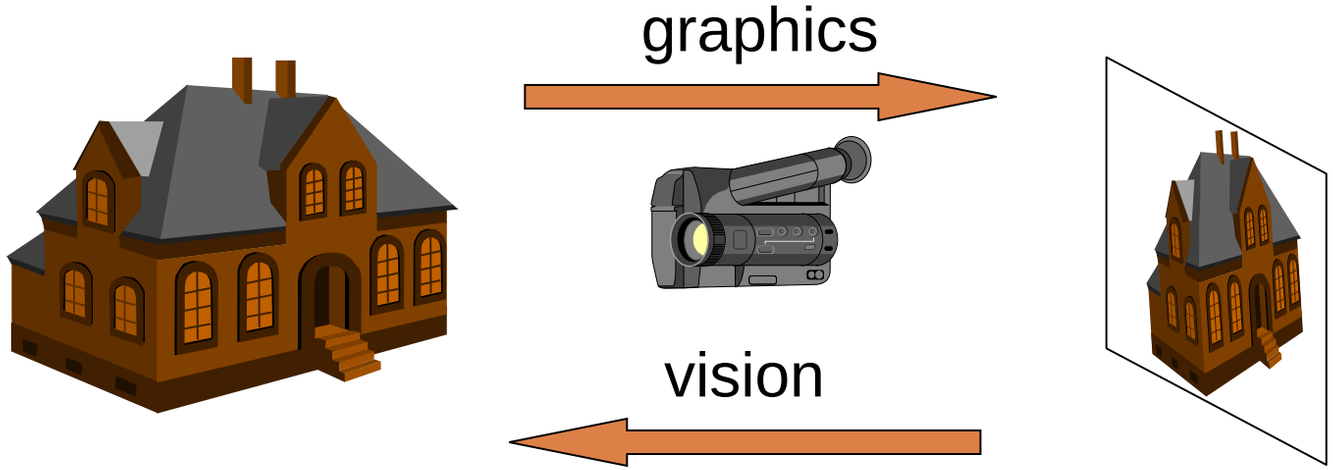
- Perceived Intensity is Not a Simple Function of the Actual Intensity – Simultaneous Contrast



a b c

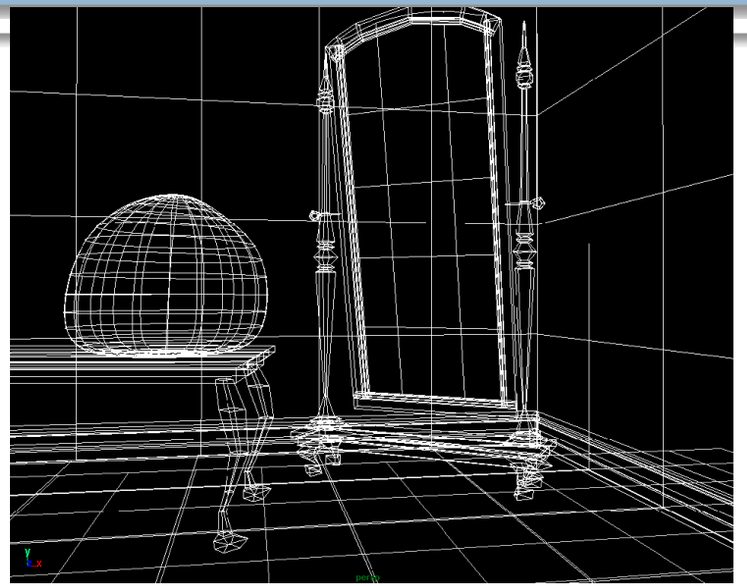
FIGURE 2.8 Examples of simultaneous contrast. All the inner squares have the same intensity, but they appear progressively darker as the background becomes lighter.

Vision/Image Analysis and Graphics

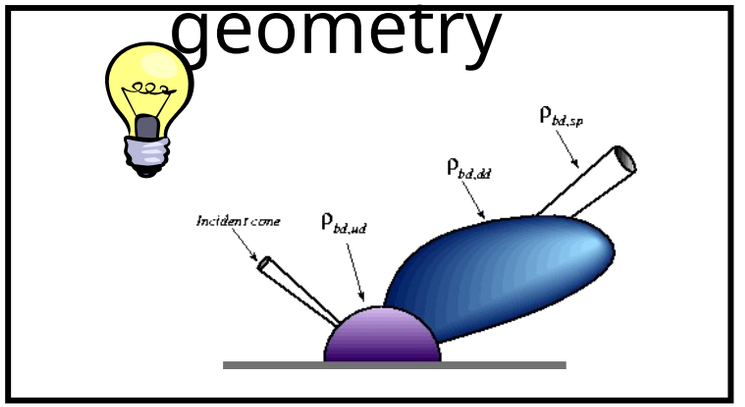


Inverse problems: analysis and synthesis.

What is computer graphics? (3D->2D)



3D geometry



physi

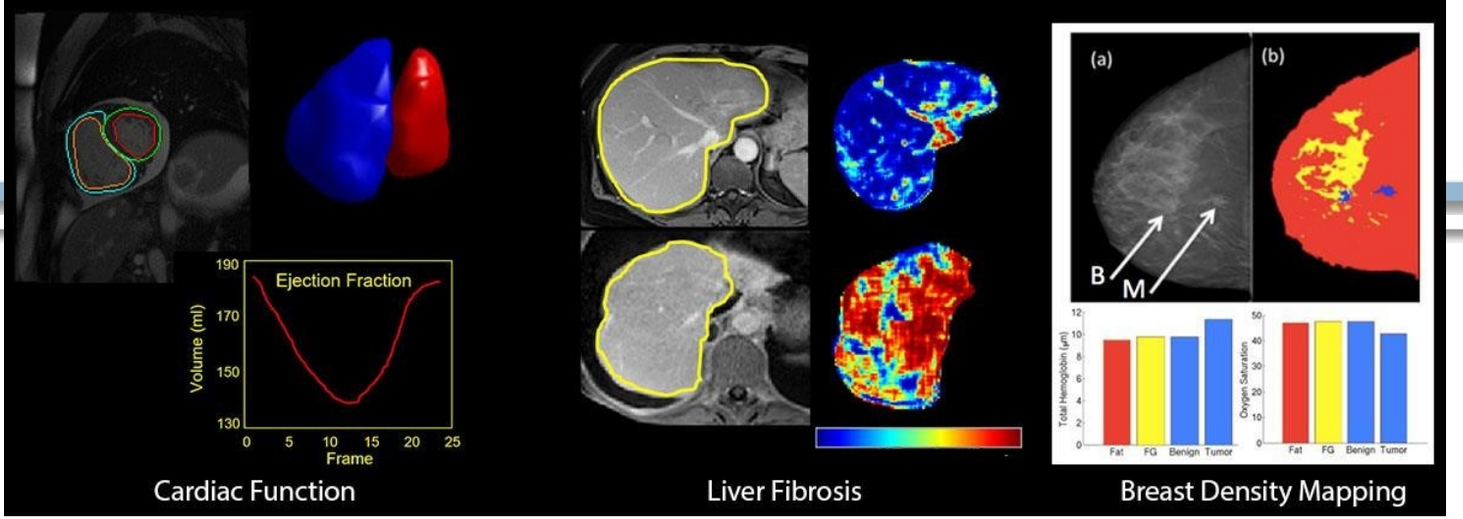
projecti on

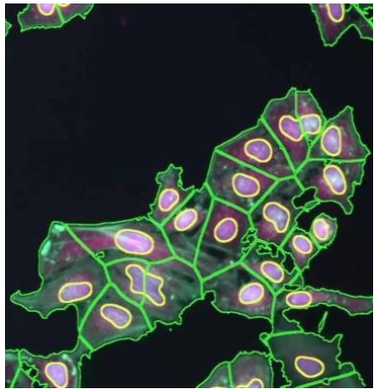
Simulation

GRAPHICS

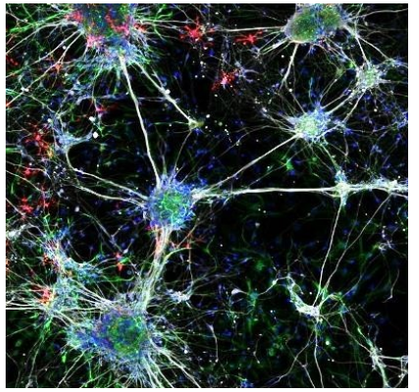
DIP

- Applications:
  - Medical imaging
  - Face recognition
  - Autonomous vehicles
  - Remote sensing
  - Surveillance systems
  - Augmented reality



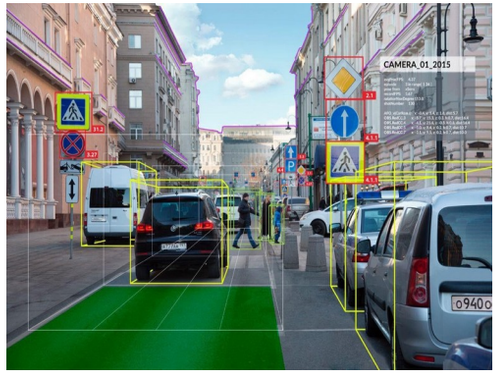


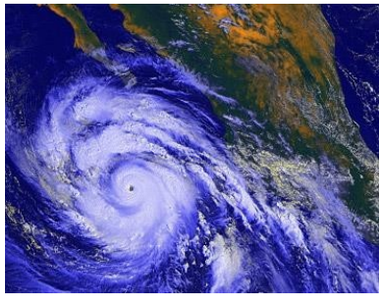
biolog



microsco

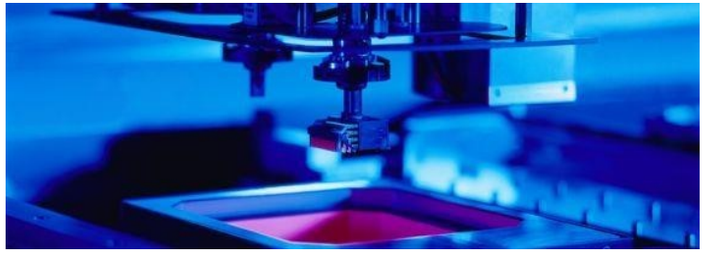
handwritten / optical character recognition





meteorolo
gy

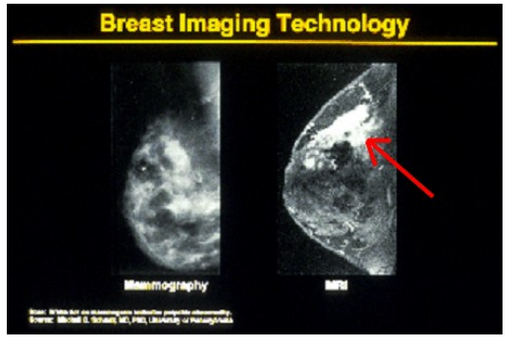
astrono
my



industry
processes

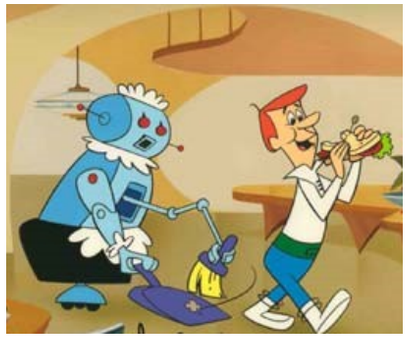
Image Processing and Computer Vision Applications

Safety



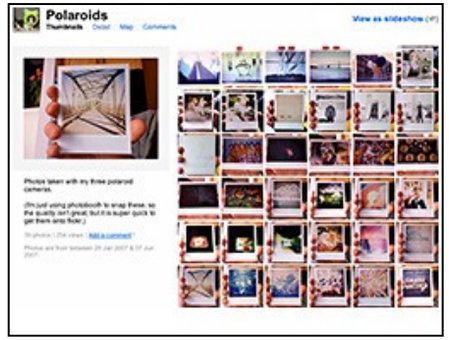
Health

Security



Comfort

Fun



Access

Industry and Applications

- Arial Photography
  - Image Enhancement
  - Missile Guidance
  - Geological Mapping
- Robotics
  - Autonomous Vehicles
- Security and Safety
  - Biometry verification (face, iris)
  - Surveillance (fences, swimming pools)
- Military
  - Tracking and localizing
  - Detection
  - Missile guidance
- Traffic and Road Monitoring
  - Traffic monitoring
  - Adaptive traffic lights

Cruise Missiles

Key Processes in Image Analysis

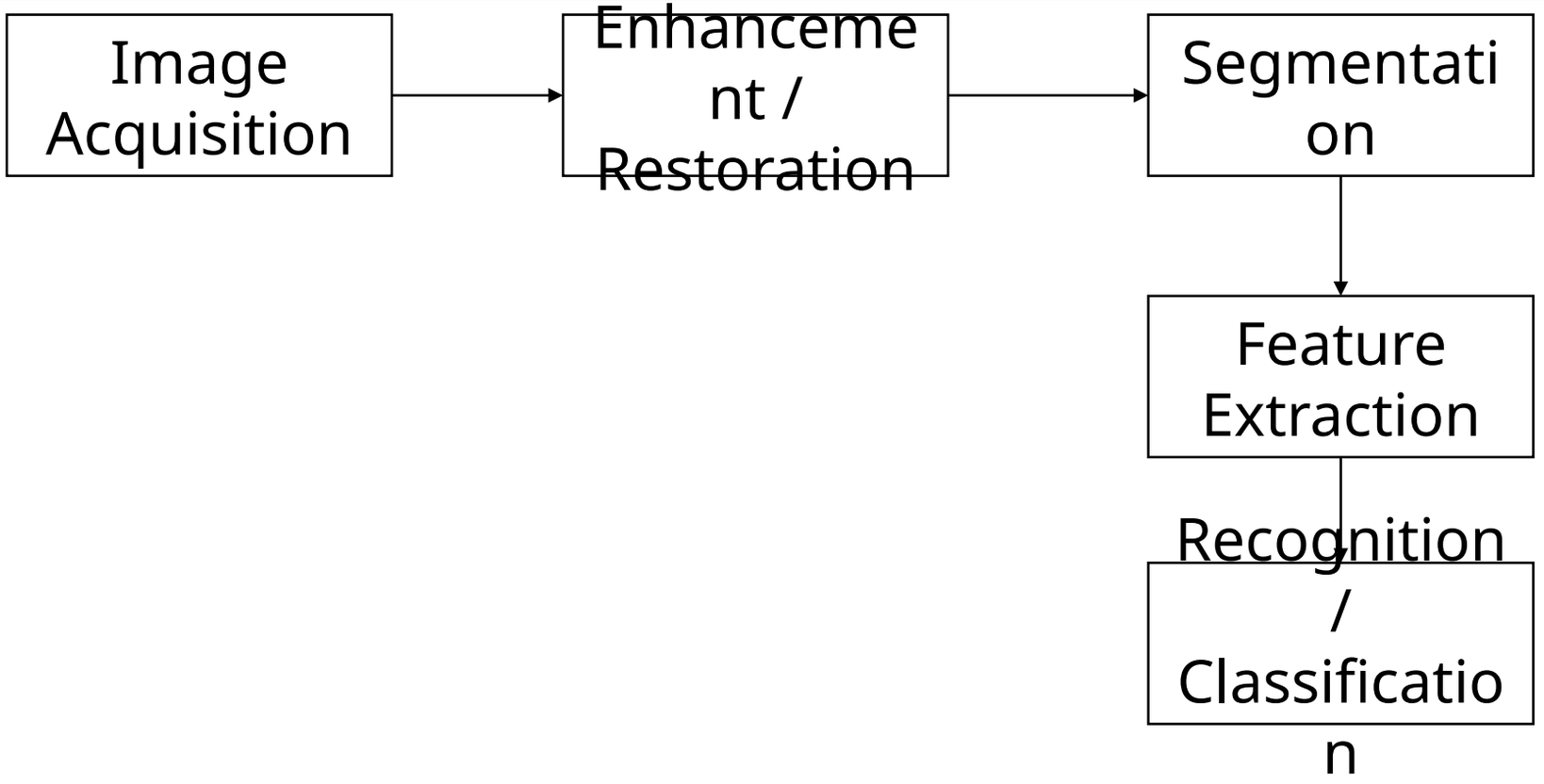


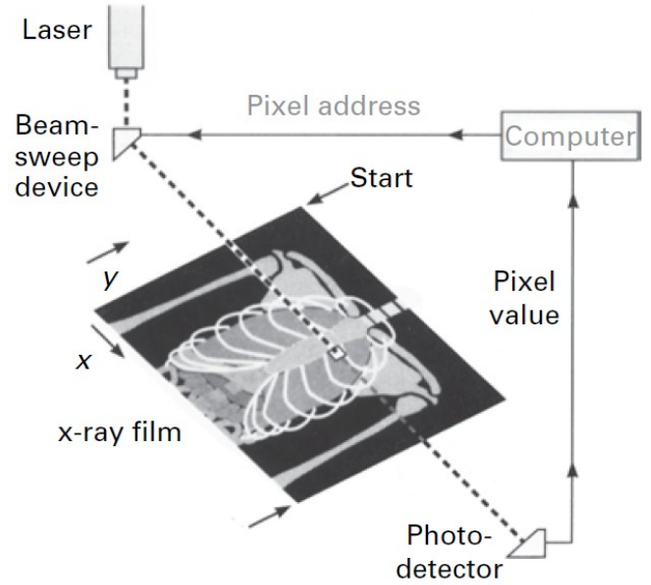
Image Acquisition

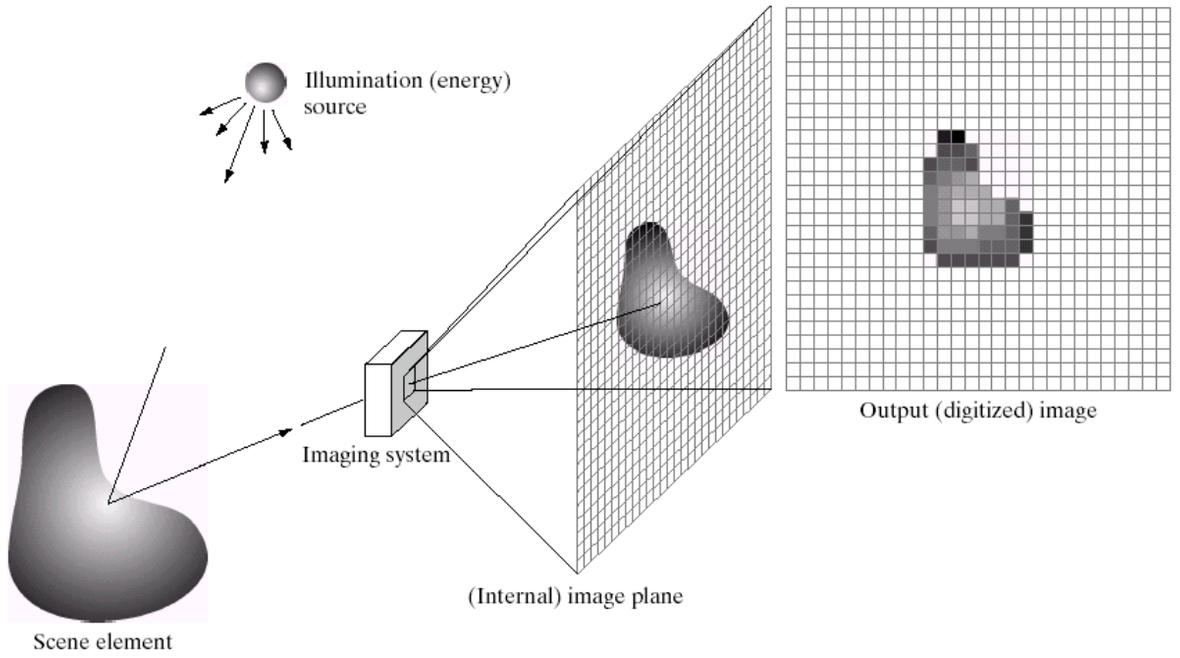
- Capture images from different modality devices.
  - Camera
  - X-ray/CT Scanner
  - MRI Scanner, Ultrasound Device
- **Output:** Digital image: f(x,y)

  where

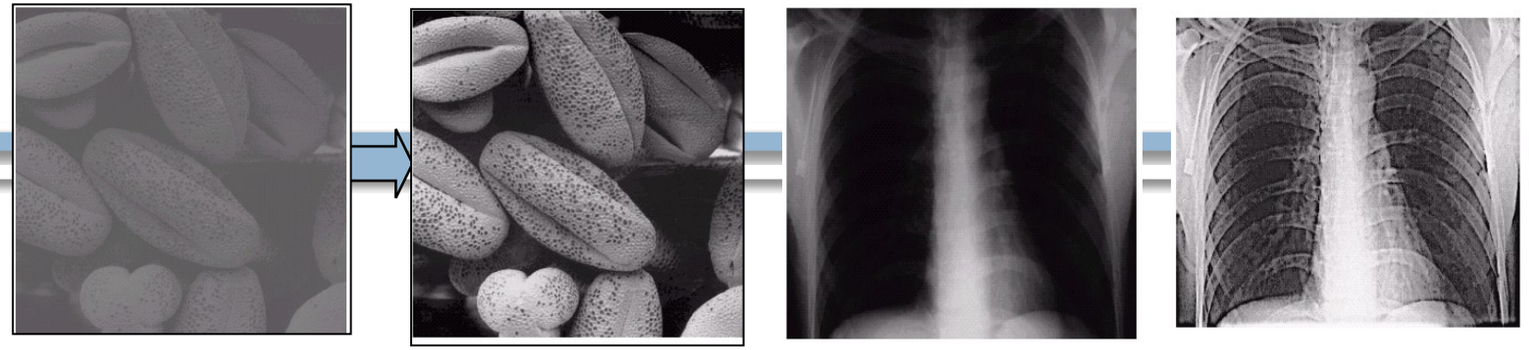
  (x,y) = pixel coordinates

  f(x,y) = intensity value
- **Challenges**
  - Noise
  - Motion artifacts
  - Low contrast
  - Limited resolution

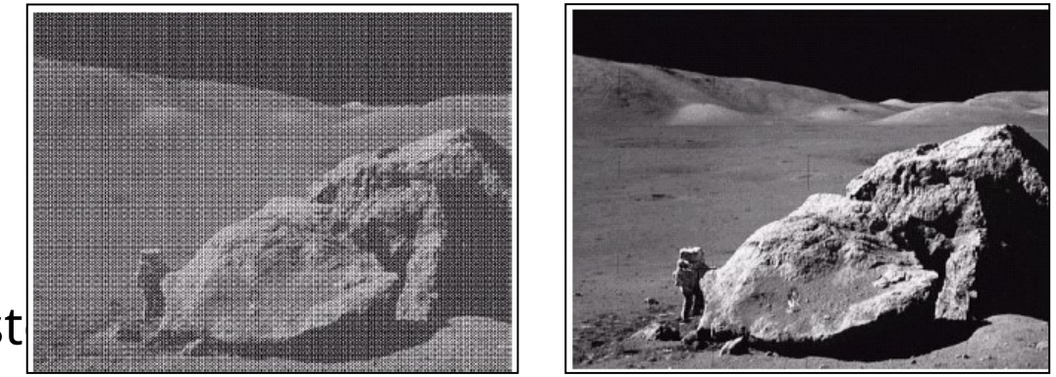


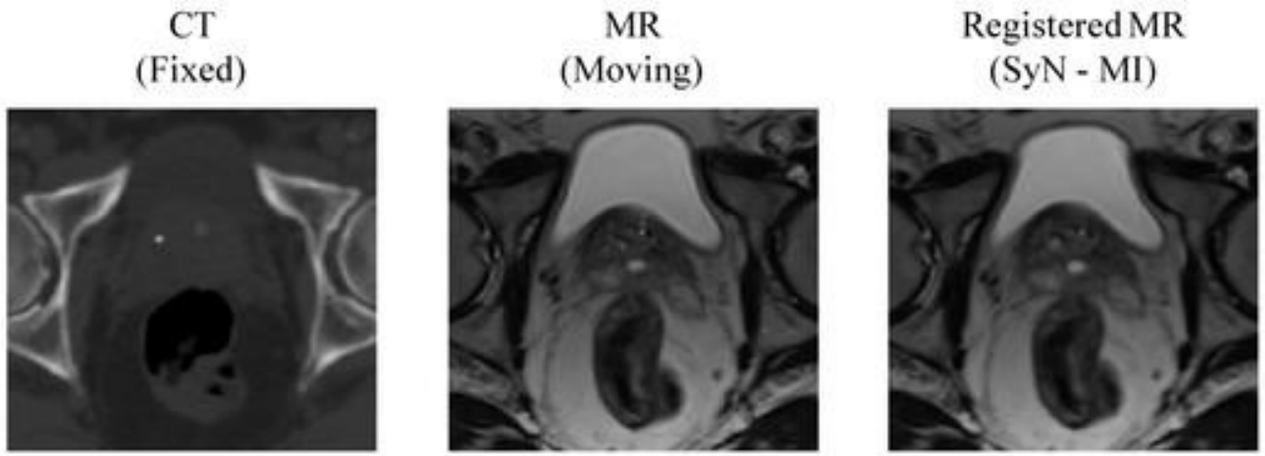


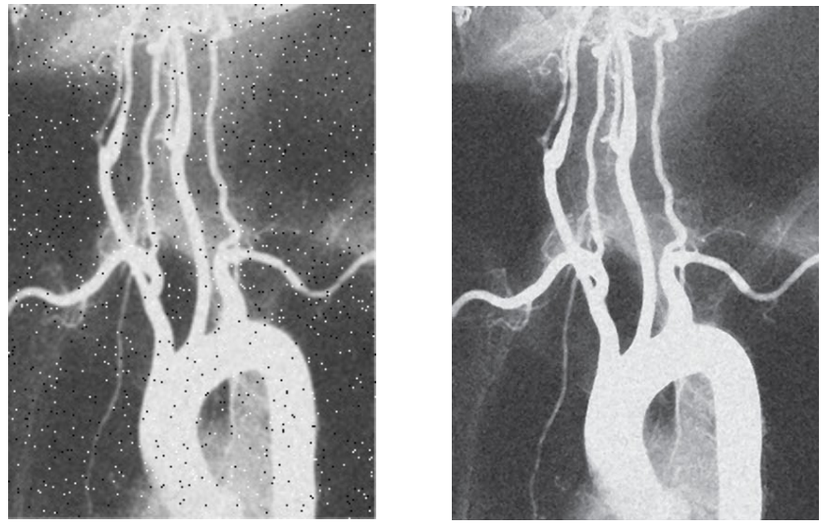
Enhancement



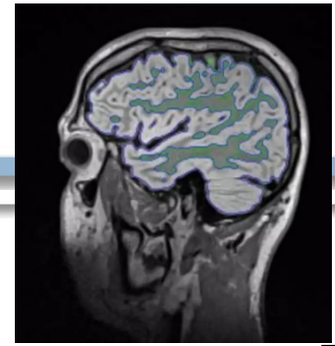
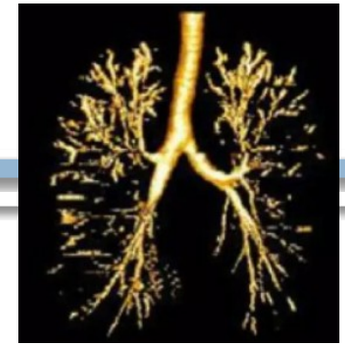
- Improve visual quality.
  - **Contrast Stretching**
  - **Histogram Equalization**
  - **Noise Remove**
  - **Image Registration**
    - Align multiple images into the same coordinate syst



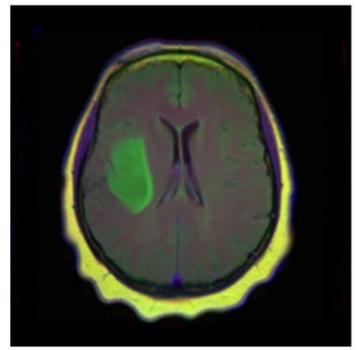


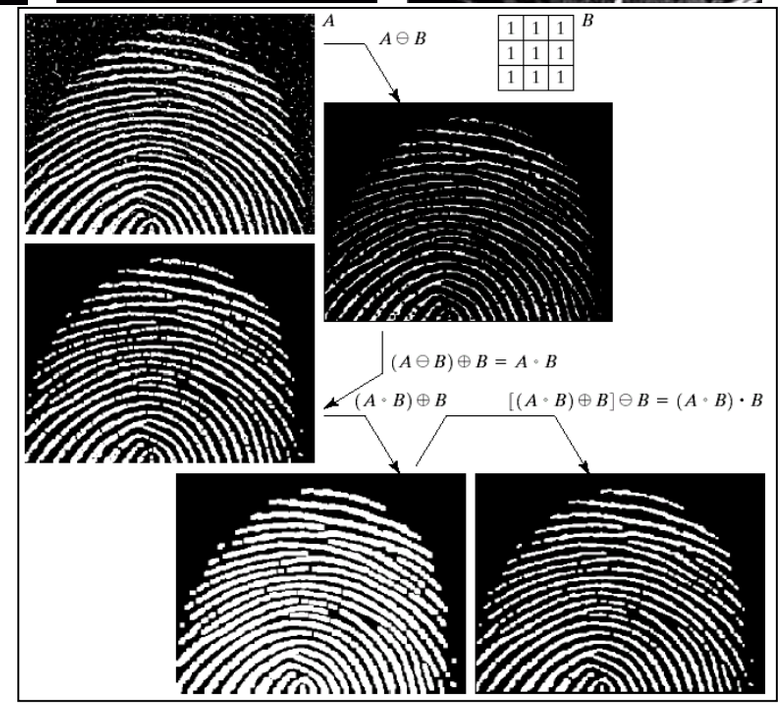


Segmentation / Processing

- Partitions an image into meaningful (anatomical or constituent) regions of interest
  - Delimiting vessels, organs, bone, brain structures.

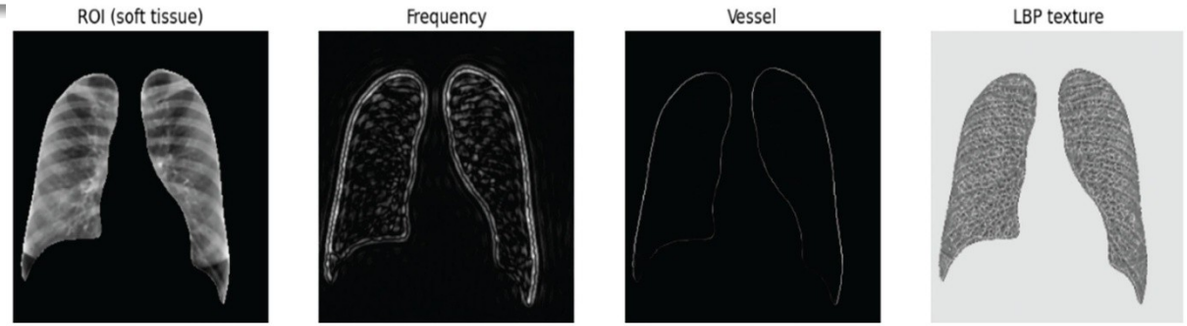


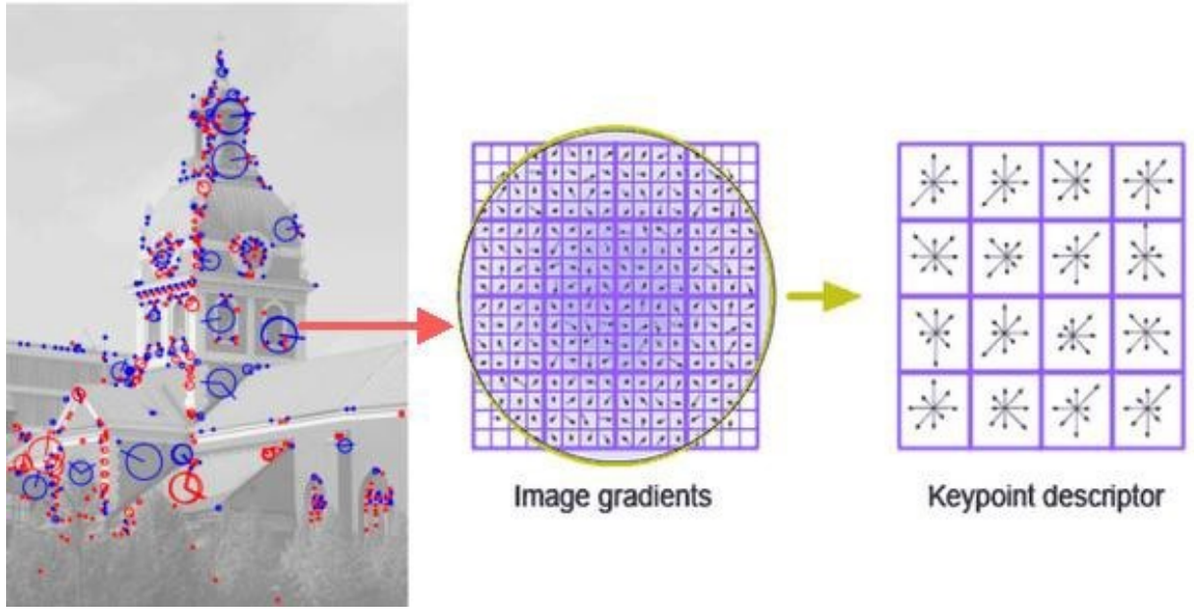


- **Methods**
  - Thresholding
  - Edge Detection
  - Region Growing
  - Deep Learning (U-Net)

Feature Extraction

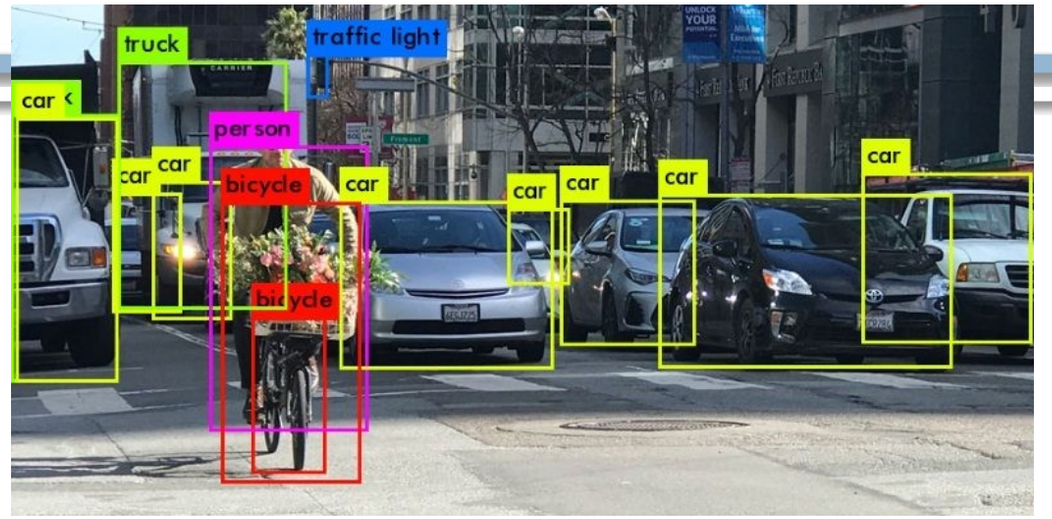
- Convert images into numerical descriptors.
- **Common Features**
  - **Intensity Features:** Mean, Variance, Entropy
  - **Shape Features:** Area, Perimeter
  - **Texture Features:** GLCM, LBP
- Tumor Feature Vector $F = [f_1, f_2, f_3, \ldots, f_n]$





Classification / Recognition

- **Input -** Feature vector
- **Output –** Object/ disease type /severity
  - Normal, Benign, Malignant
- **Traditional Methods**
  - k-NN, SVM
  - Random Forest
- **Deep Learning**
  - CNN
  - Vision Transformer



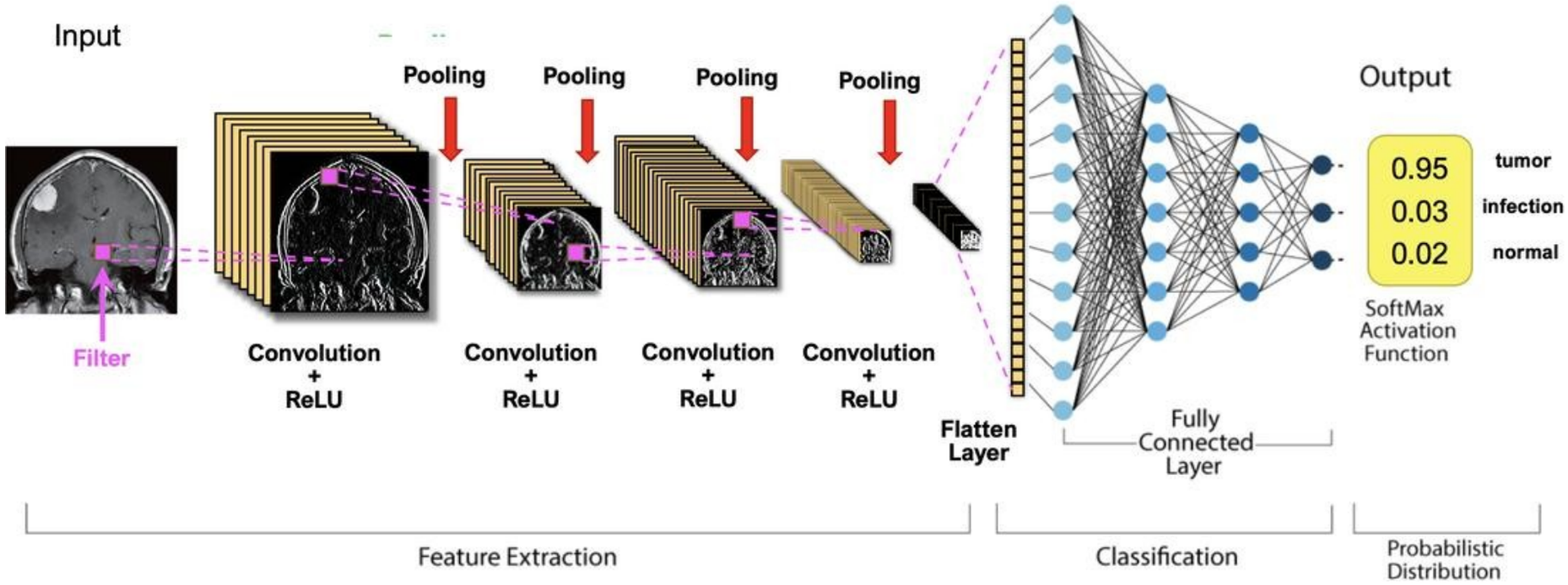
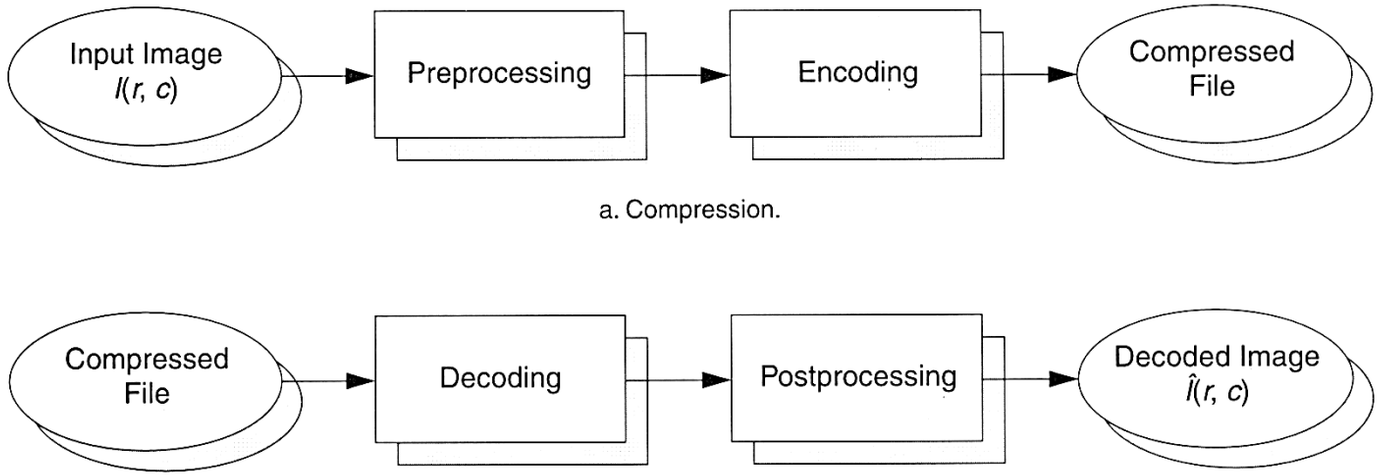
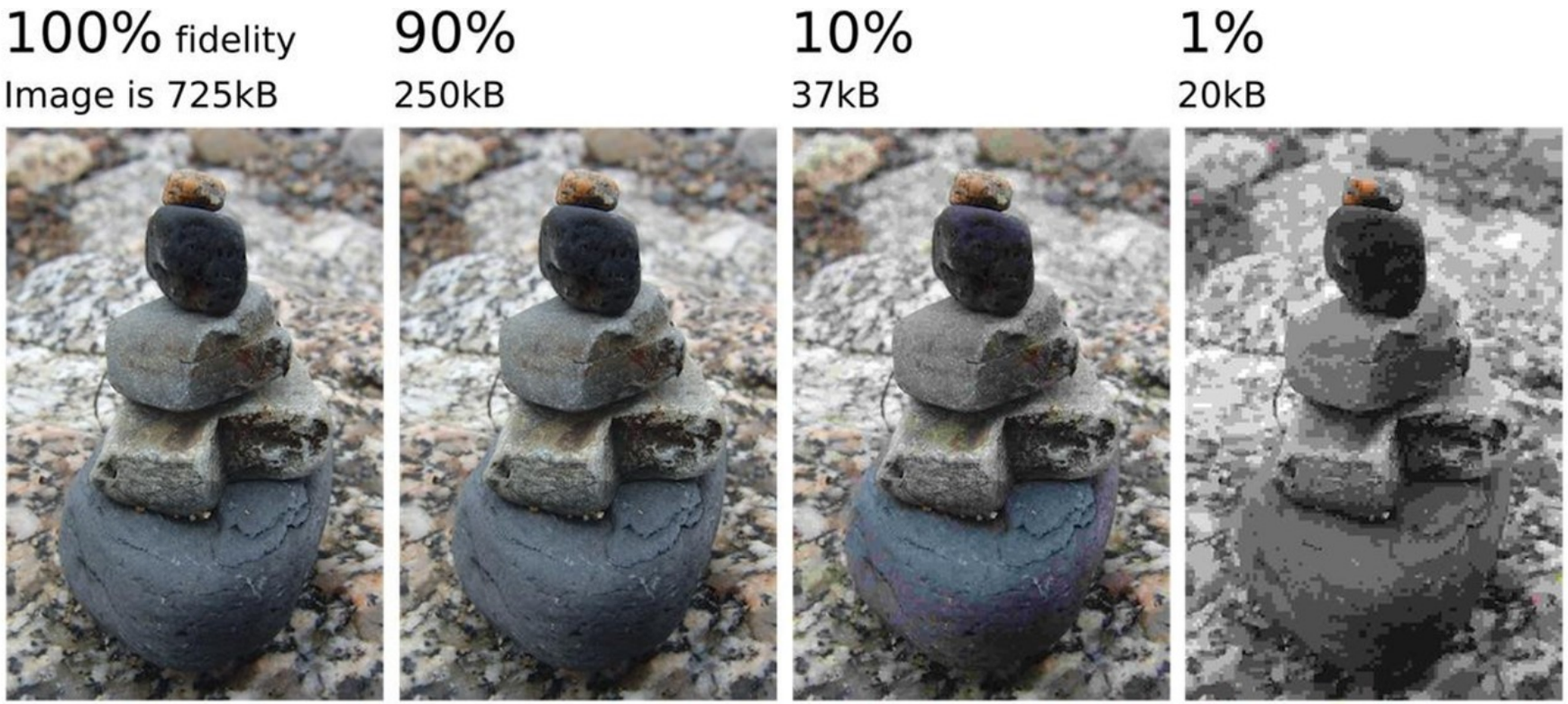


Image Processing

- Image Compression

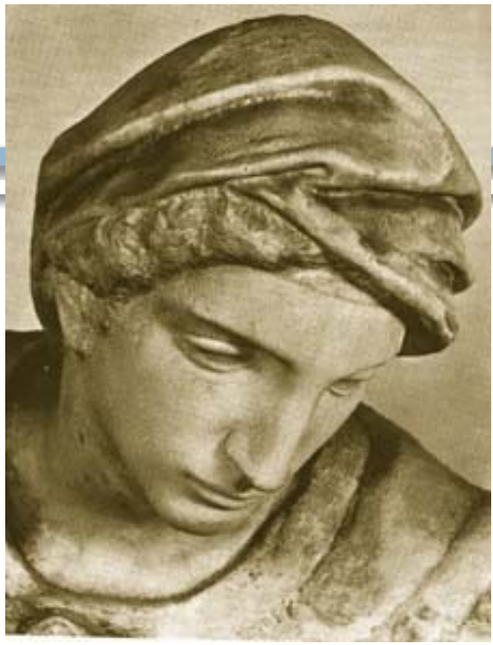


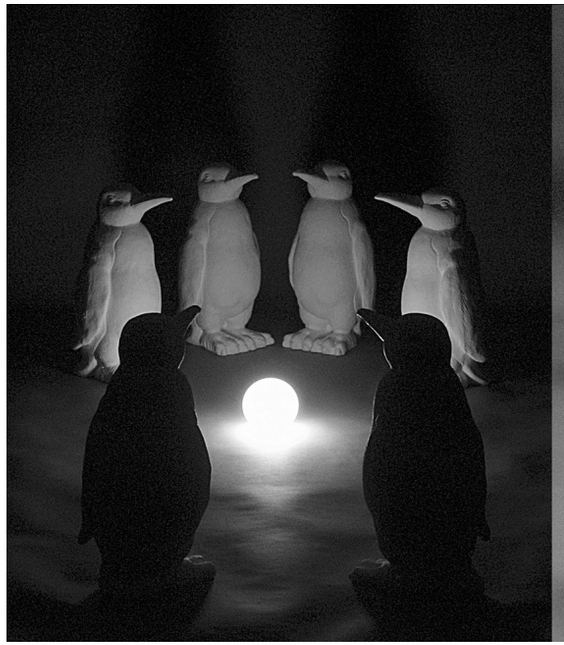


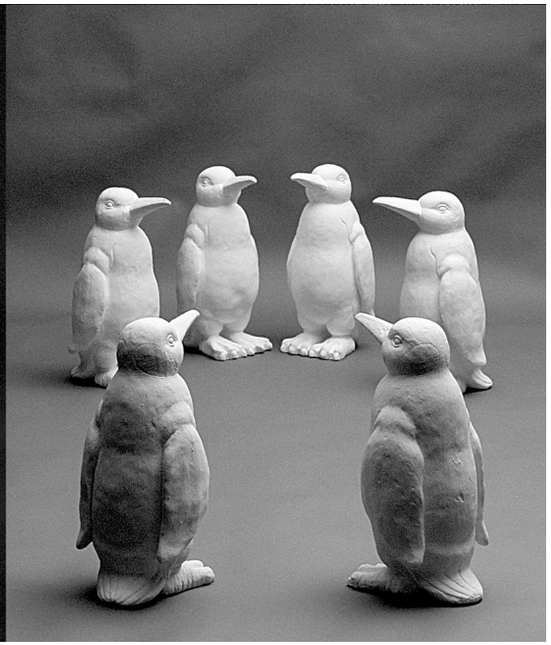
Challenges for Vision Algorithms

Vision Challenges

- View-point variations
- Illumination
- Scale, Occlusion
- Deformation,
- Intra Class variation







Challenges or opportunities?

- Images are confusing, but they also reveal the structure of the world through numerous cues
- Our job is to interpret the cues!

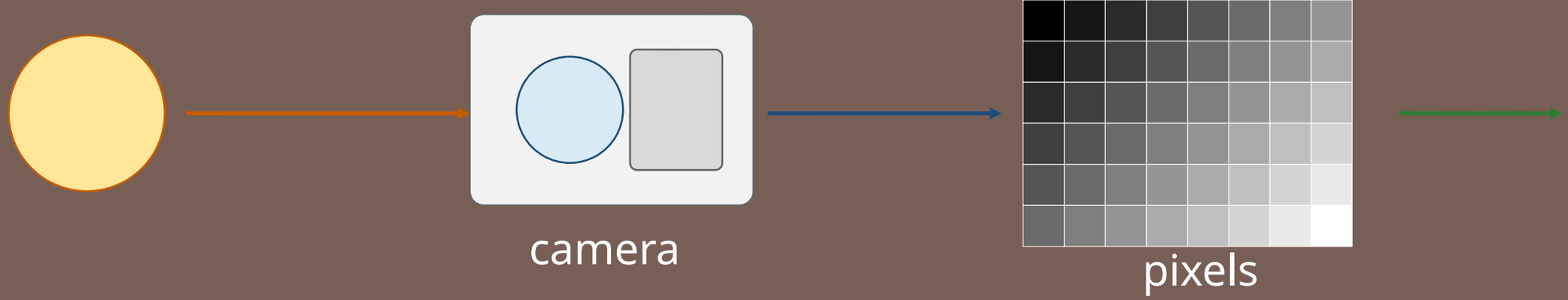


Image Formation and Acquisition

Learning outcomes

- Trace the pipeline from light in the scene to a digital array.
- Explain basic camera components, human eye components, and their analogy.
- Use pinhole and lens projection equations at an introductory level.
- Explain Bayer sampling and why demosaicing is needed.
- Predict the effects of spatial, temporal, and radiometric sampling.
- Apply pixel adjacency, connectivity, and distance measures.

Core idea

continuous world
-> optics
-> sensor samples
-> quantized numbers
-> image algorithms

What controls the image before processing starts?

Geometry

projection, viewpoint, perspective

Optics

focus, aperture, blur

Sensor

pixel pitch, response, noise

Sampling

spatial and temporal spacing

Quantization

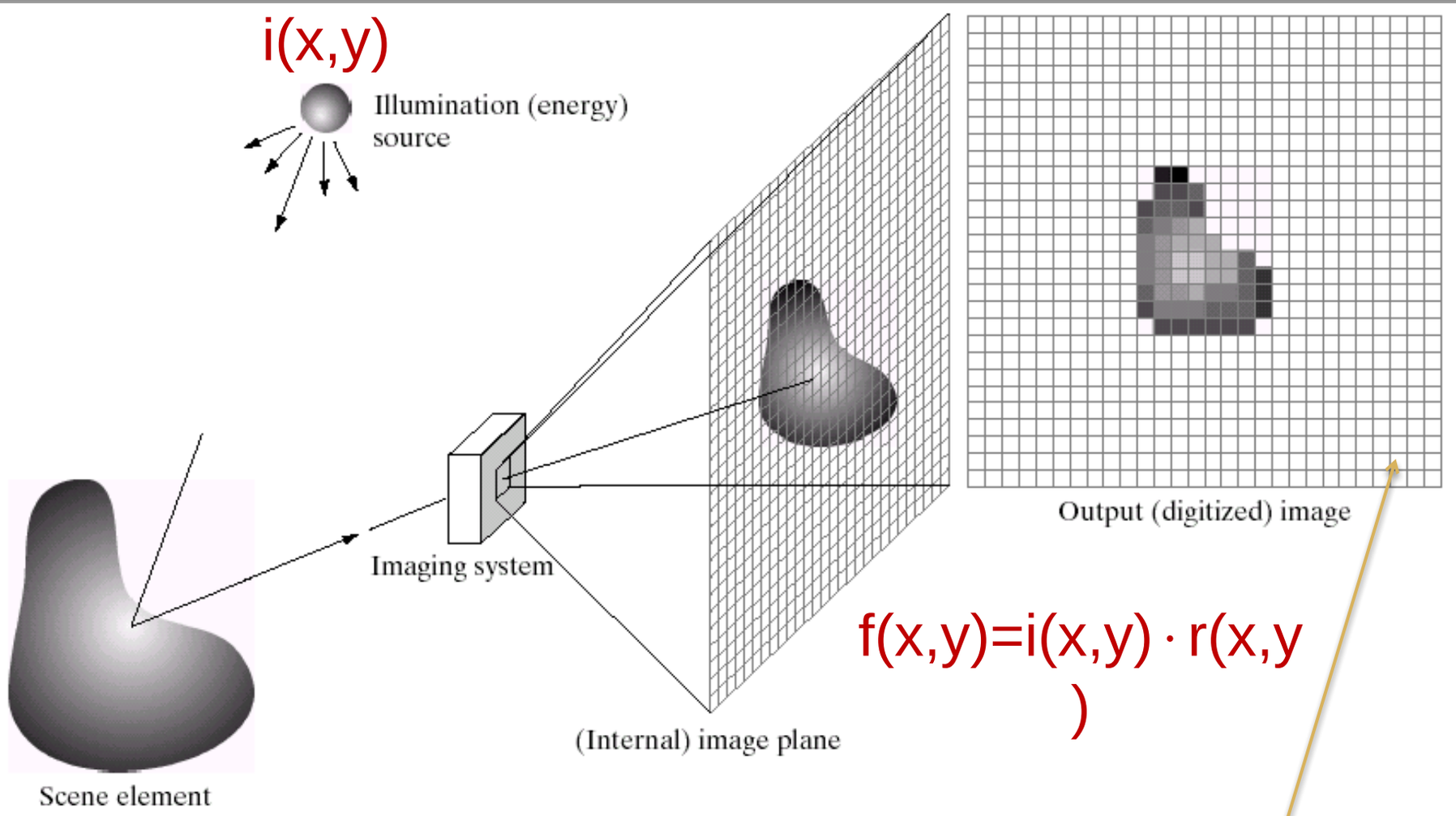
number of gray/color levels

Color filter

Bayer grid or other color sampling

Image formation

g(i,j)

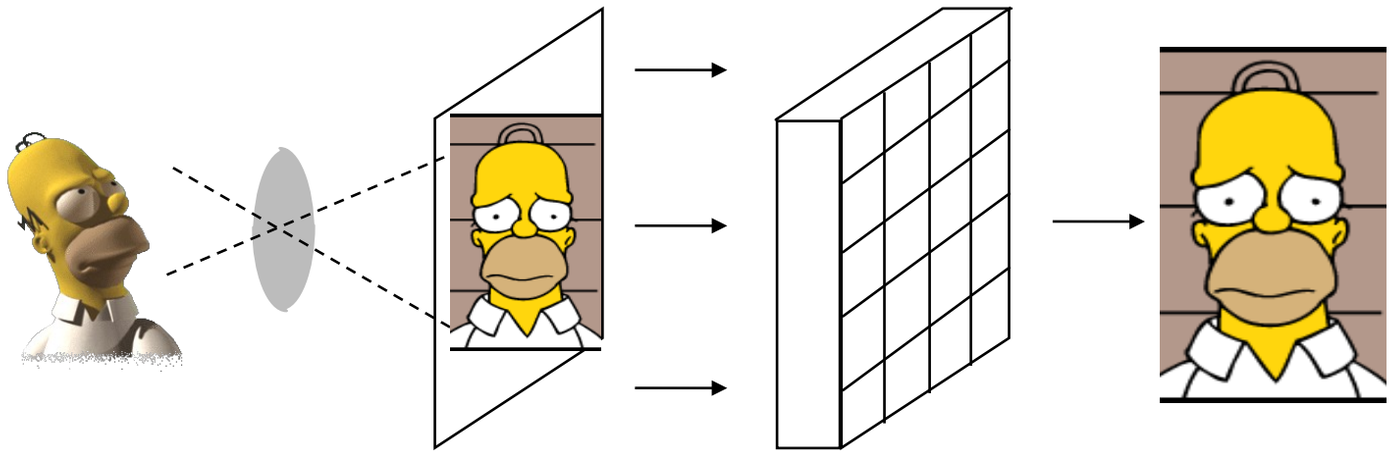


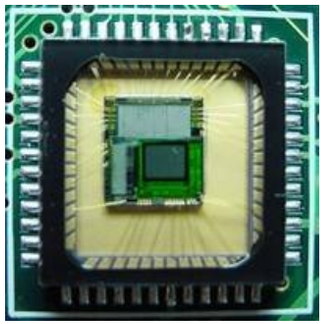
$f(x,y)=i(x,y)\cdot r(x,y)$

r(x,y)

pixel=picture element

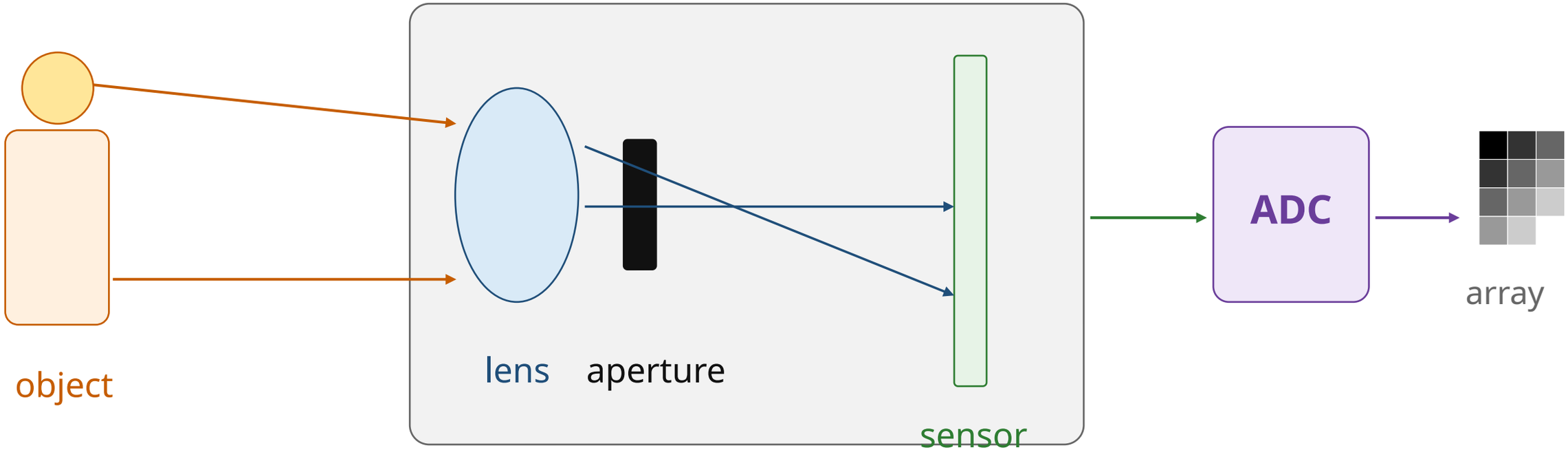
Acquisition System





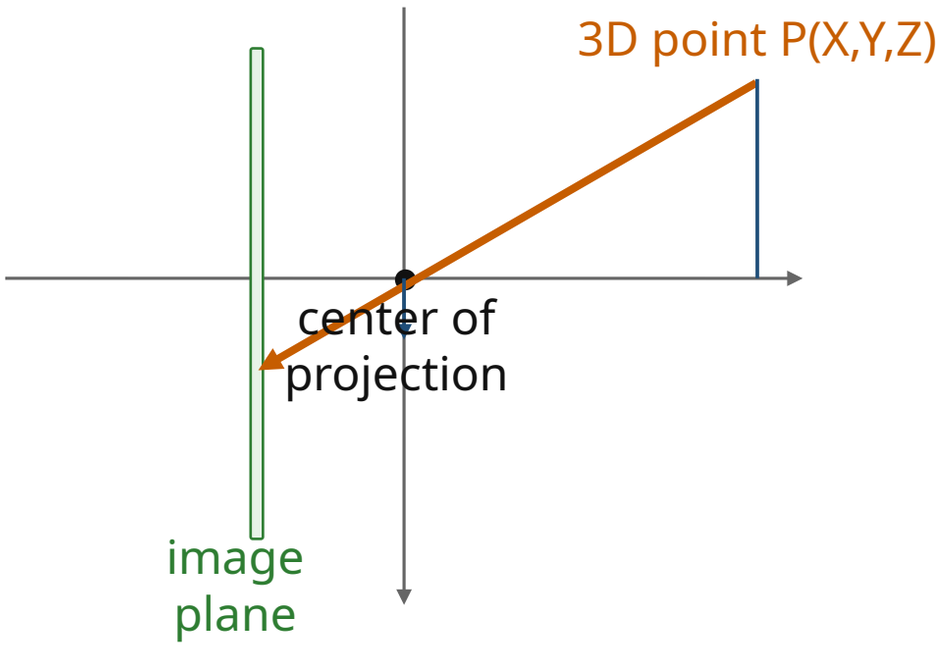
CCD/CMOS sensor

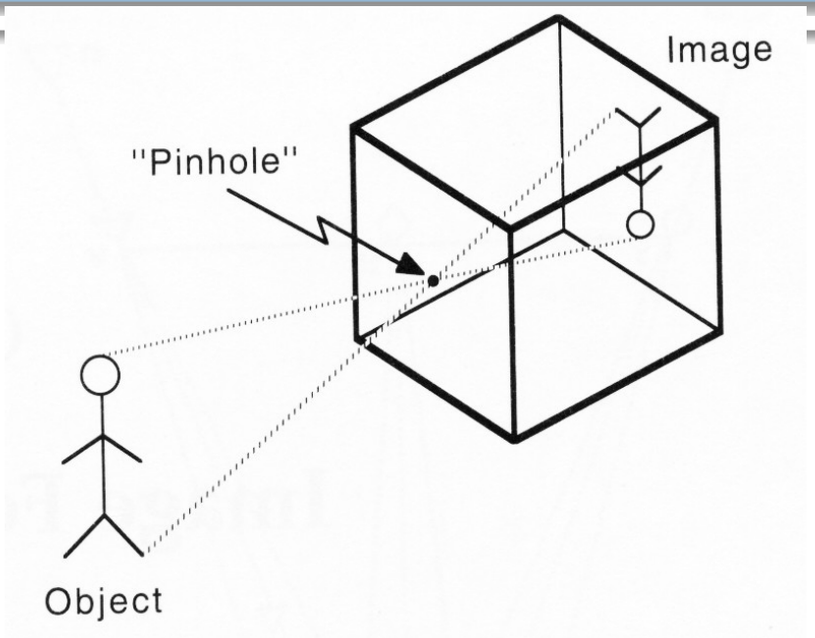
Basic camera / imaging system



- Lens forms an image on the sensor plane.
- Aperture controls light amount and depth of field.
- Sensor integrates light over area and time.
- ADC converts analog voltage into digital codes.

Pinhole camera model: simple projection





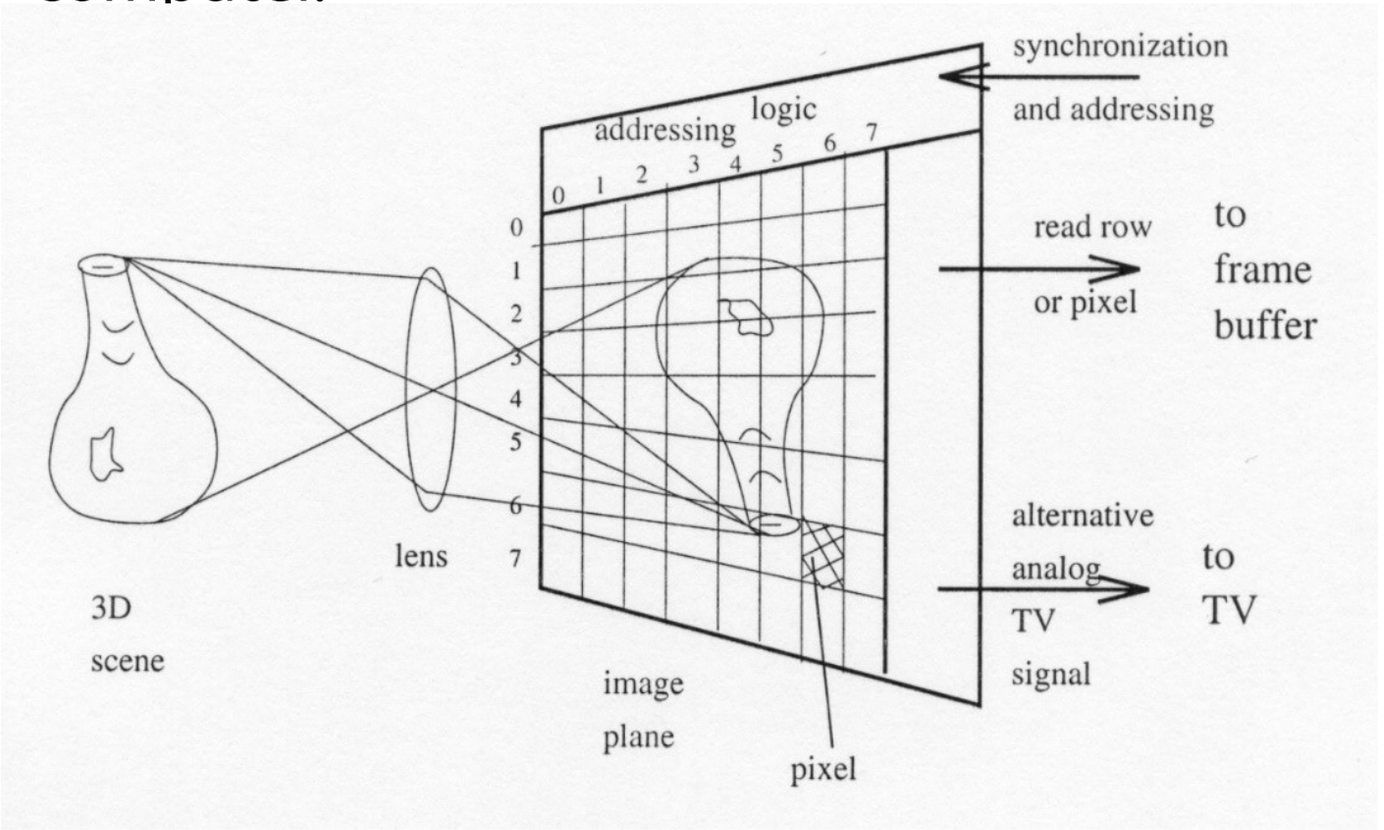
$$x = f X / Z, \qquad y = f Y / Z$$

f: focal length

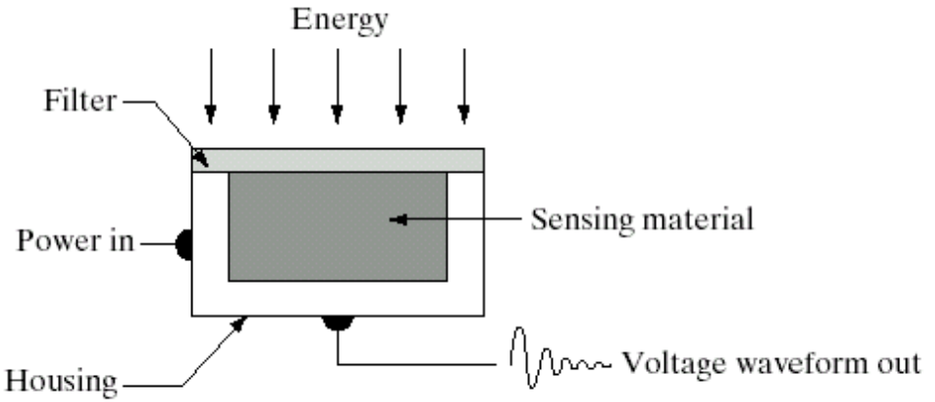
- Farther objects appear smaller because x and y are divided by depth Z.
- Parallel lines in 3D may meet at a vanishing point in the image.
- The model explains perspective geometry without lens details.

CCD (Charged-Coupled Device) Cameras

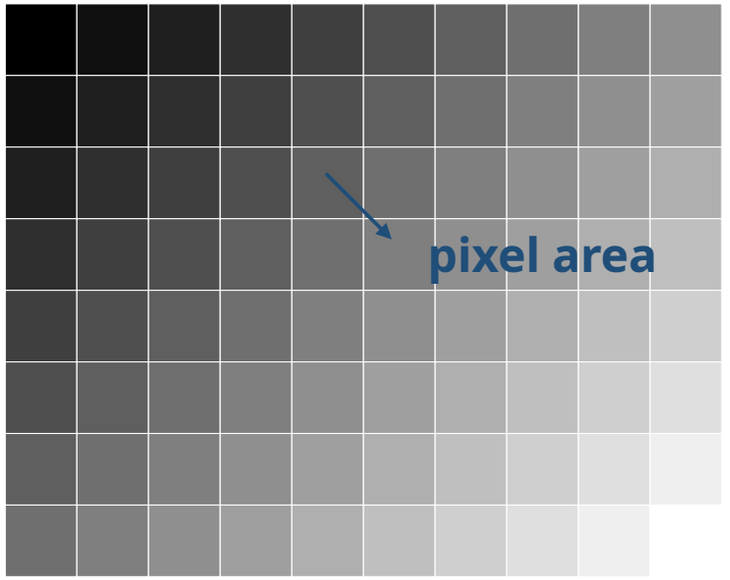
- Tiny **solid state cells** convert light energy into an electrical charge.
- The image plane acts as a digital memory that can be read row by row by a computer.



solid state cell



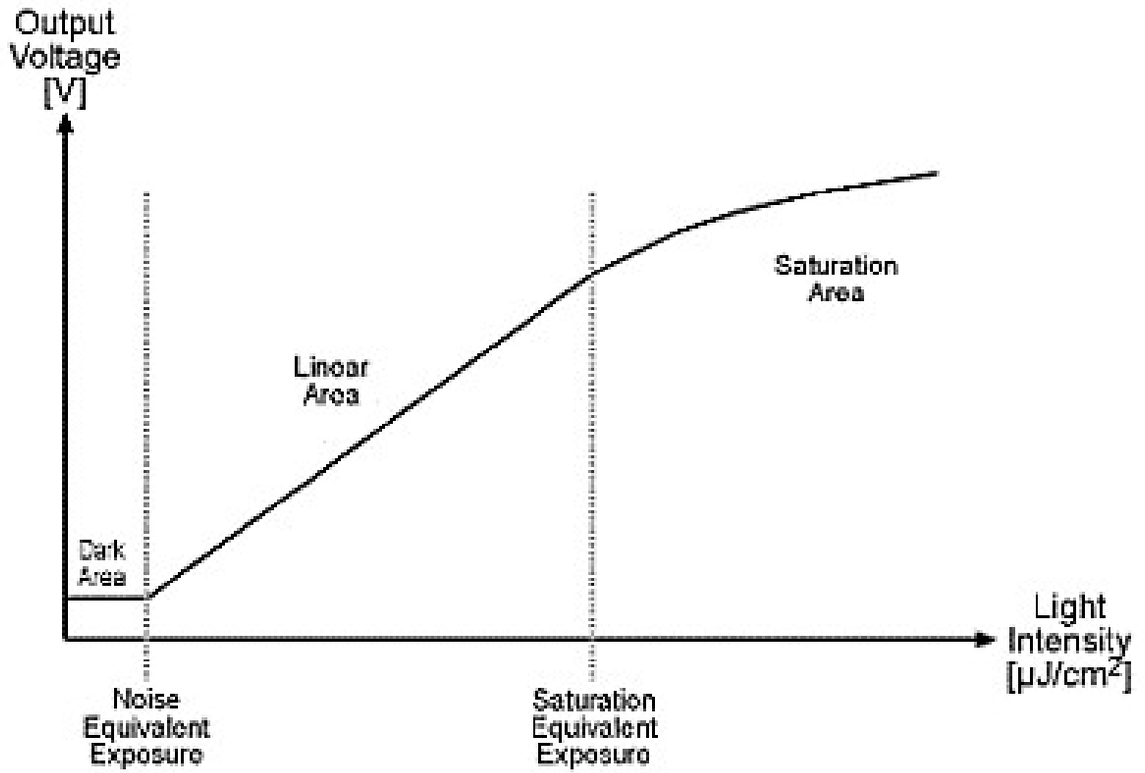
Sensor array: pixels integrate light



g[m,n] = integral over pixel area and exposure time of E(x,y,t)

- A pixel is not a mathematical point; it measures an average over a small area.
- Exposure time averages the signal over time.
- Pixel values combine useful signal with sensor noise and electronics noise.

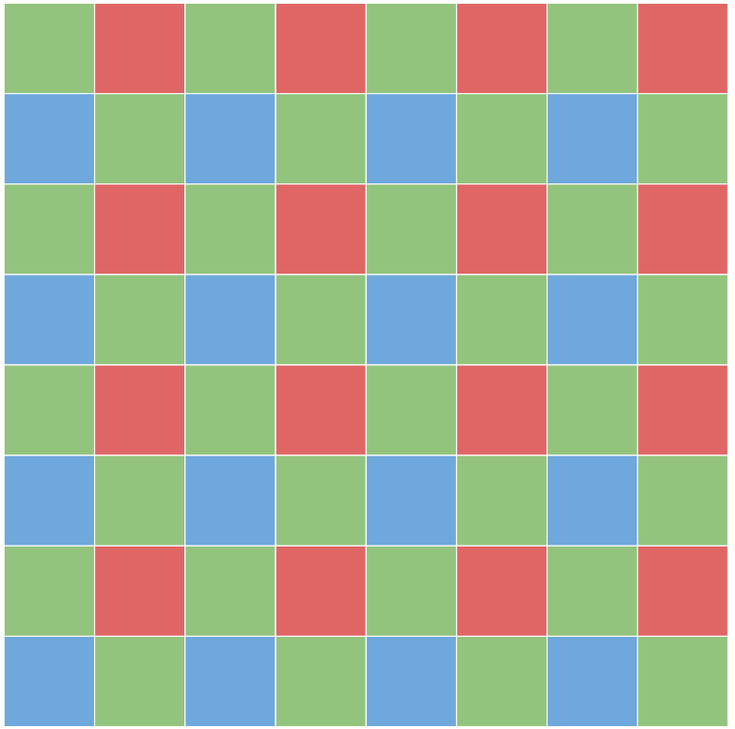
Exposure and sensor response



- Underexposure: values are near black; noise is more visible.
- Overexposure: clipping loses detail permanently.
- Dynamic range describes the span from darkest usable signal to saturation.
- Gamma or tone mapping may be applied before display.

SNR often improves when more photons are collected, until saturation.

Bayer grid: color by spatial multiplexing

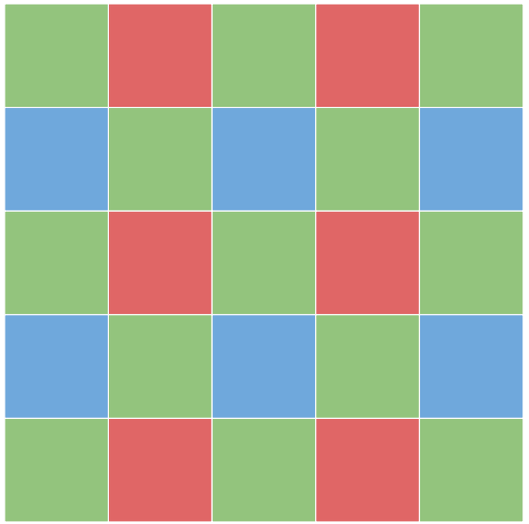


RGGB Bayer color filter array

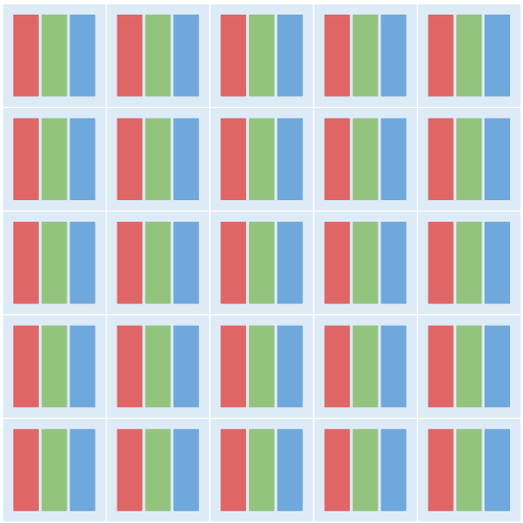
- Most camera sensors do not measure full RGB at every pixel.
- Each photosite is covered by a red, green, or blue filter.
- Green is sampled more densely because luminance detail is important.
- Demosaicing estimates missing color values.

At one pixel: measured value = one color component, not full RGB

Demosaicing: reconstruct missing colors



interpolate missing channels

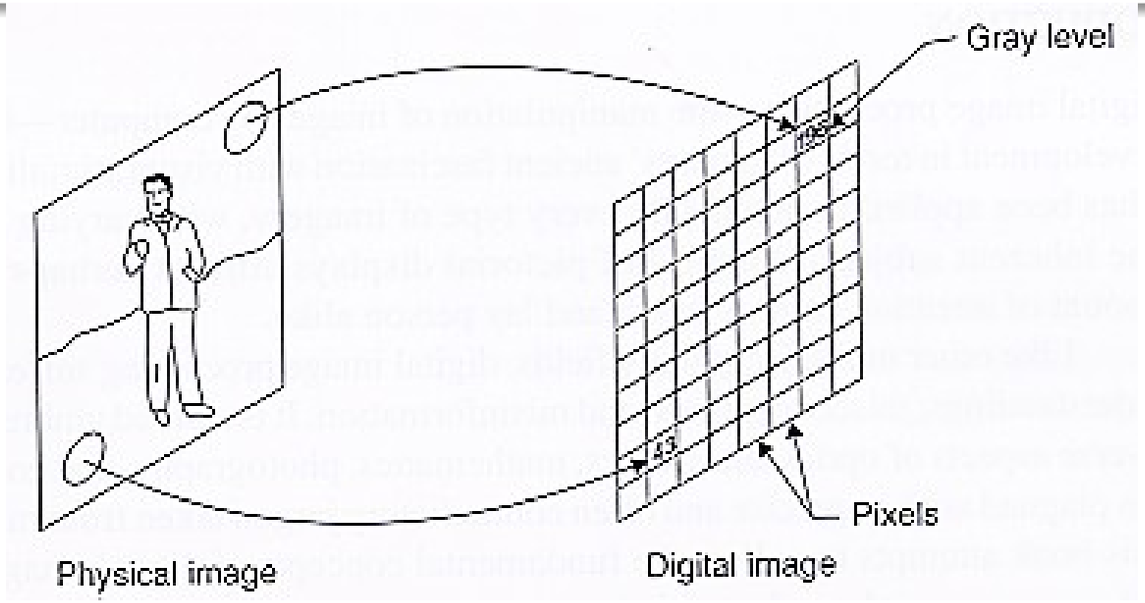


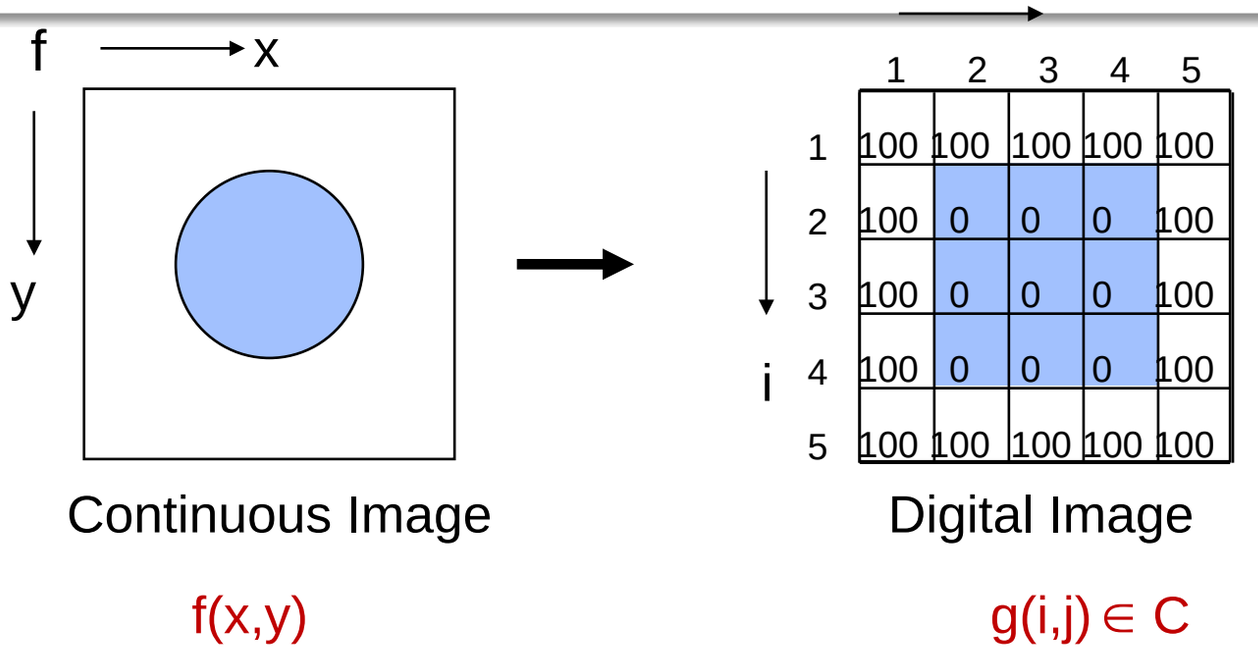
raw mosaic: one measured color per location

RGB image: three color values at each pixel

- Simple method: average nearby same-color samples.
- Better methods use edge direction to avoid color bleeding.
- Artifacts: zipper edges, false colors, loss of fine texture.

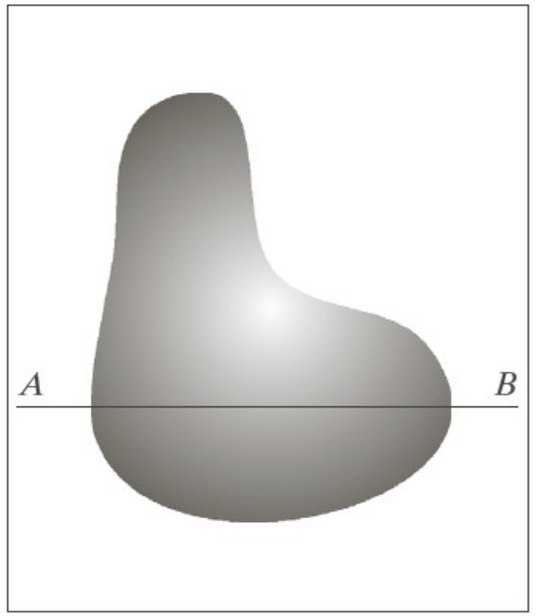
Image Digitization

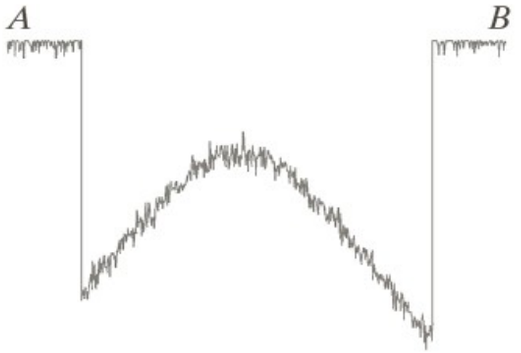


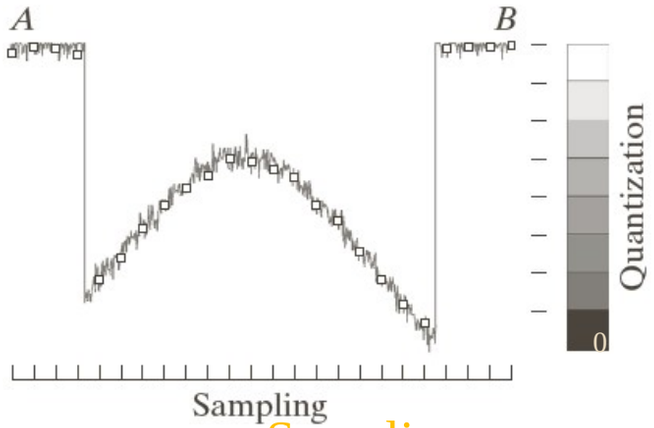


- **Sampling** is measuring the value of an image at a finite number of points (i.e., CCD array)
- **Quantization** is the representation of the measured value at the sampled point by an integer (i.e., frame grabber)

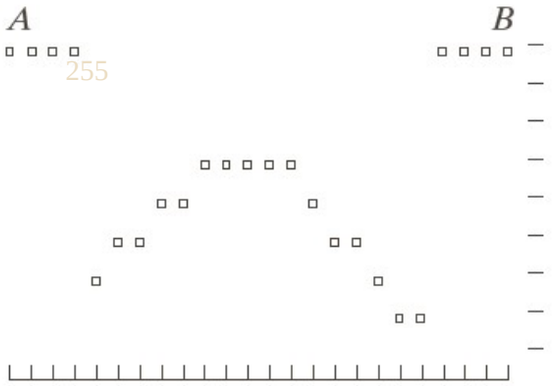
Image Digitization (cont'd)







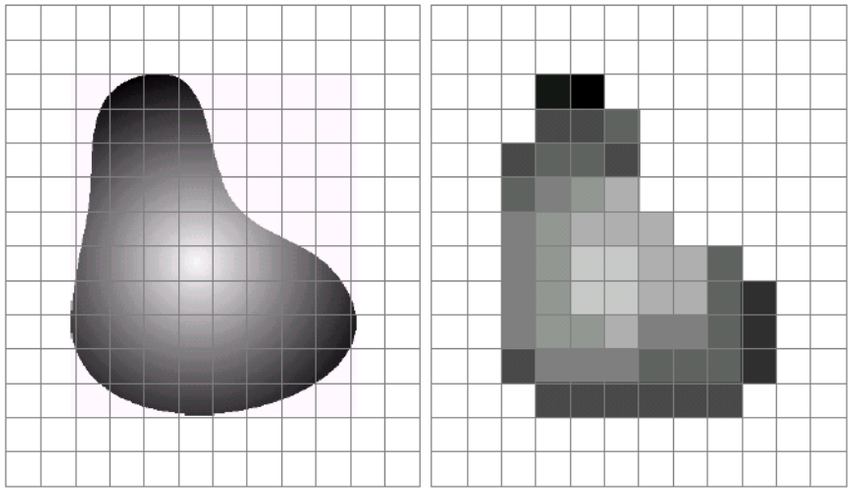
Sampling



Quantization

Physical Image

Digital Image

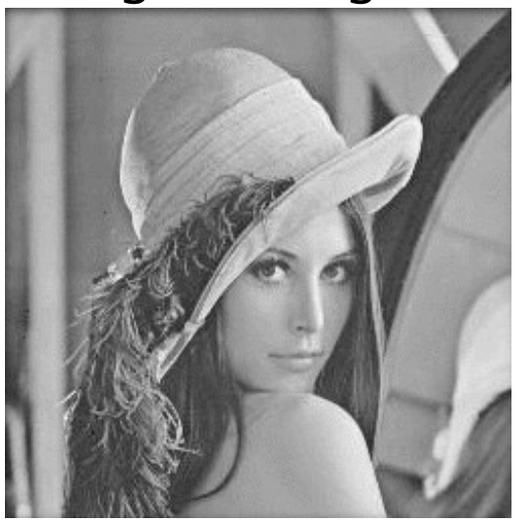


a b

FIGURE 2.17 (a) Continuos image projected onto a sensor array. (b) Result of image sampling and quantization.

Effect of Image Sampling

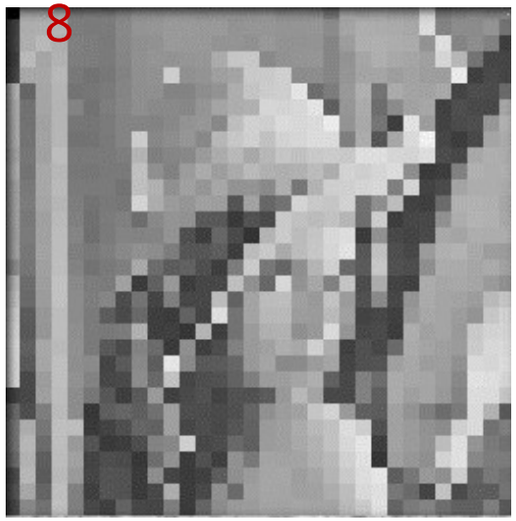
original image



sub-sampled by a factor of 2

sub-sampled by a factor of 4

sub-sampled by a factor of 8



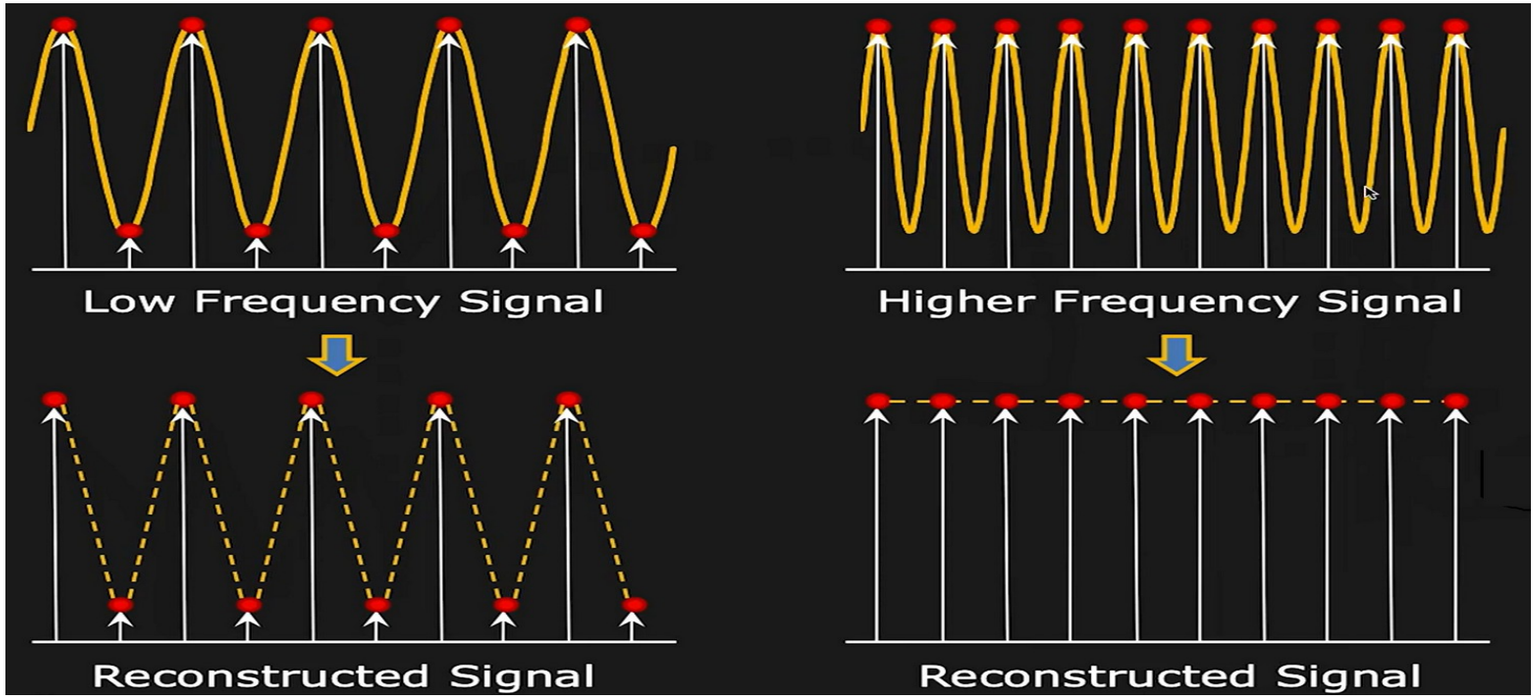
Note: images have been resized for comparison purposes

$$f[m,n] = f_c(m\,Dx, n\,Dy)$$

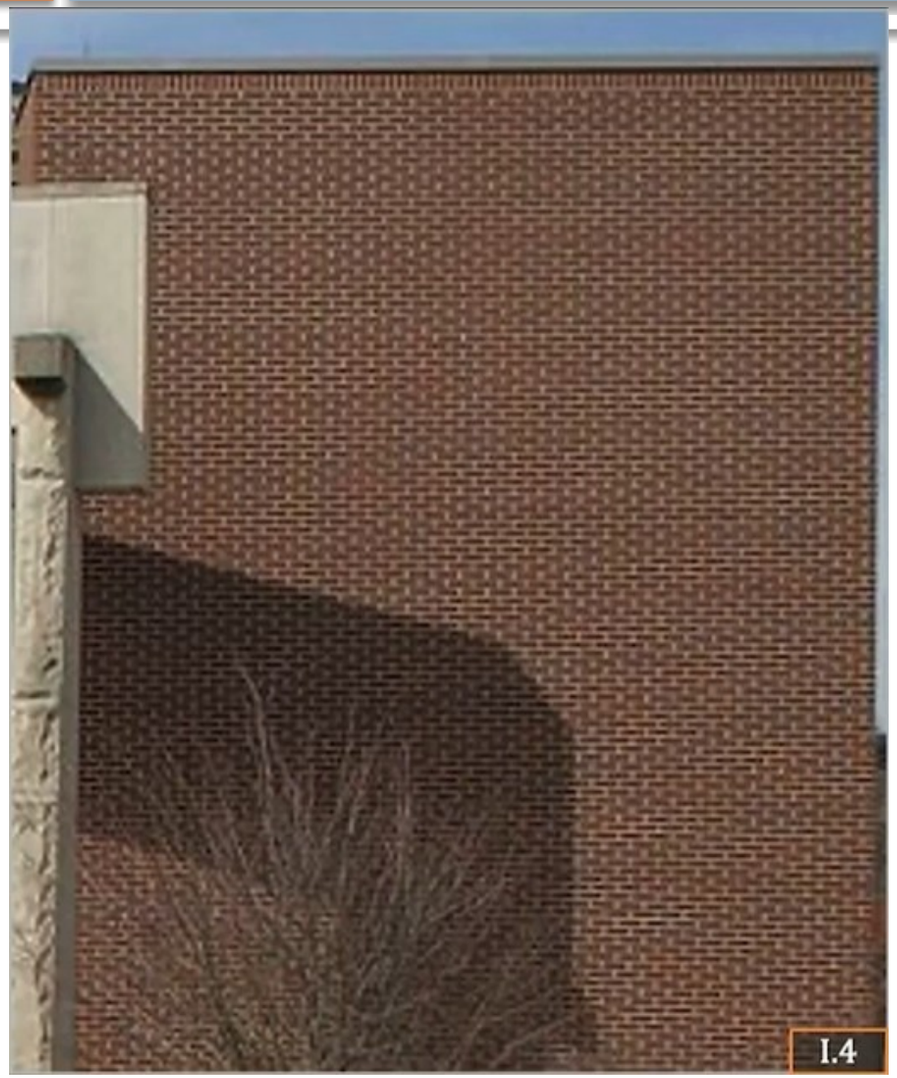
- Dx and Dy are sampling intervals.
- Smaller spacing means more pixels per unit length.
- Sampling too coarsely causes aliasing.

Nyquist Frequency

- **Nyquist Rule**: To observe details at frequency f (wavelength d) one must sample at frequency > 2f (sampling intervals < d/2)
- The Frequency 2f is the **Nyquist Frequency**.
- **Aliasing**: If the pattern wavelength is less than 2d erroneous patterns may be produced.



Aliasing - Moiré Patterns



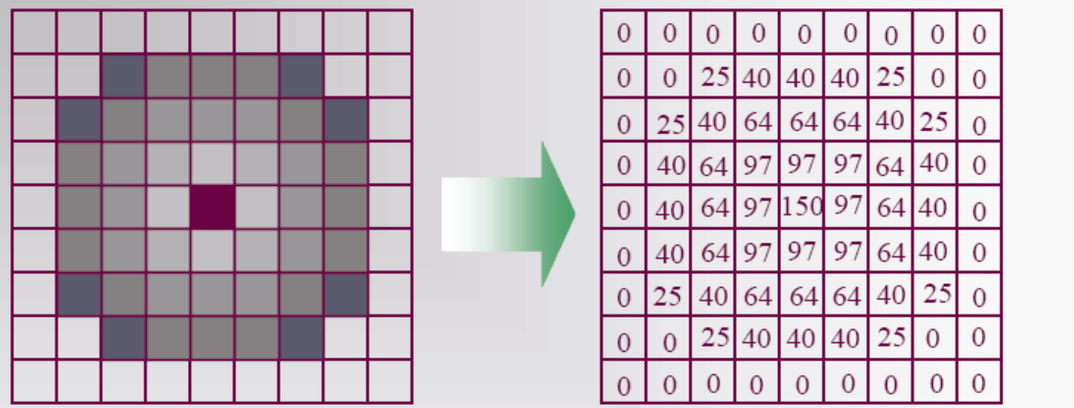
Well Sampled

Under Sampled

Quantization: continuous amplitude to discrete levels

- Choose number of gray levels (according to number of assigned bits)
- Divide continuous range of intensity values

$$L = 2^b \text{ levels}, \quad \text{code in } \{0, 1, \ldots, L\text{-}1\}$$



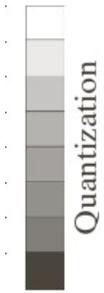
- b bits per pixel gives L possible intensity codes.
- Quantization maps a range of analog values to one code.
- More bits reduce visible banding and measurement error.
- Too few levels create false contours.

Effect of Image Quantization

256 gray levels (8 bits/pixel)

32 gray levels (5 bits/pixel)

16 gray levels (4 bits/pixel)



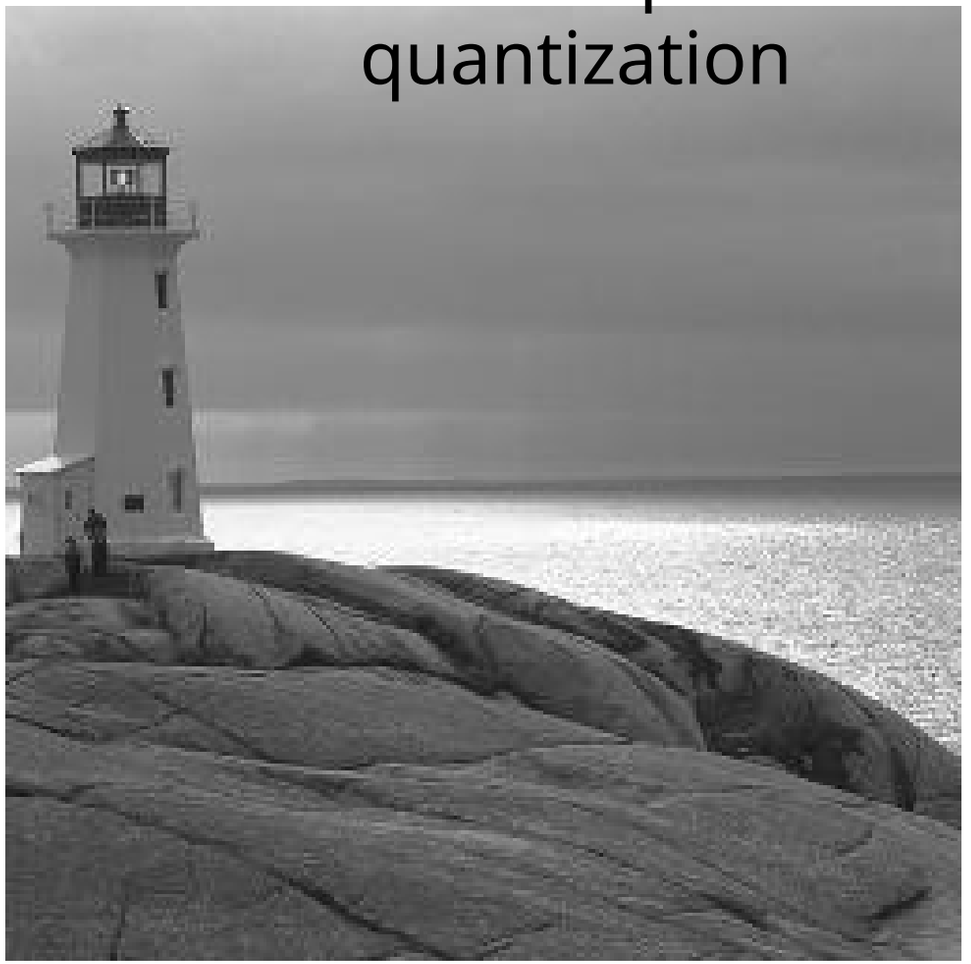
8 gray levels (3 bits/pixel)

4 gray levels (2 bits/pixel)

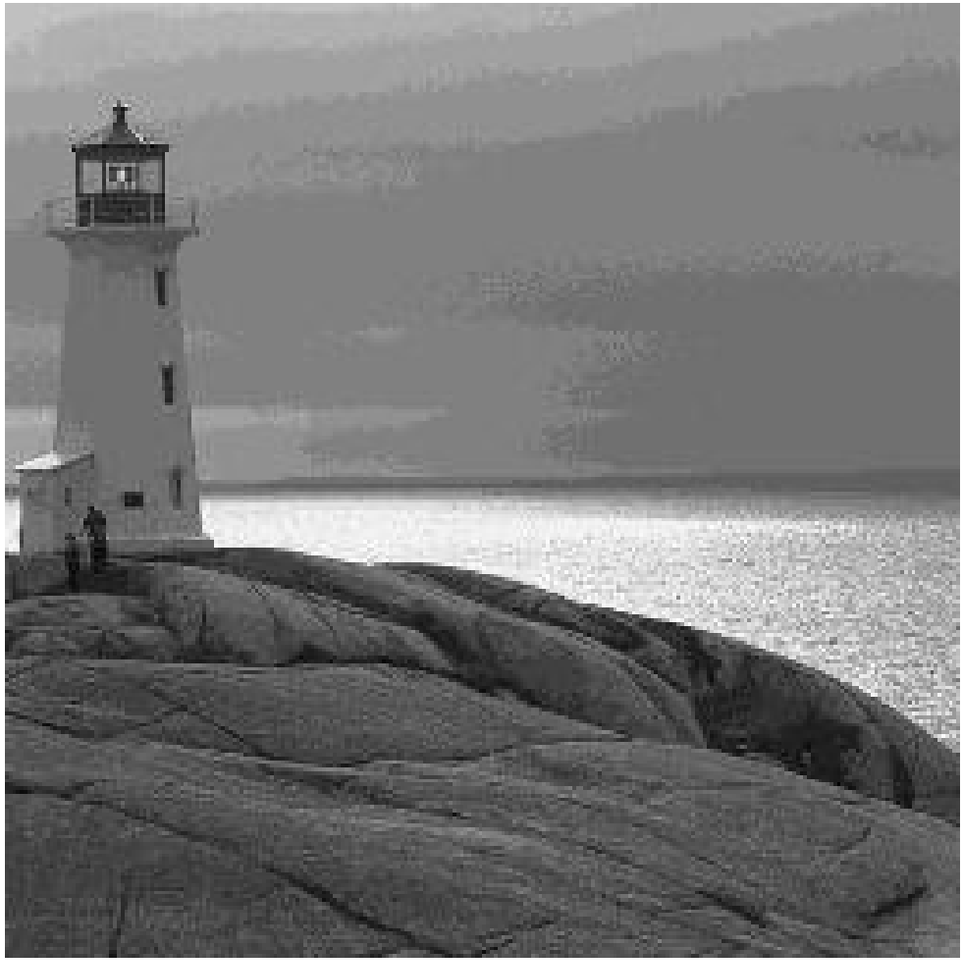
2 gray levels (1 bit/pixel)

Quantization

- Low freq. areas are more sensitive to quantization



8 bits image



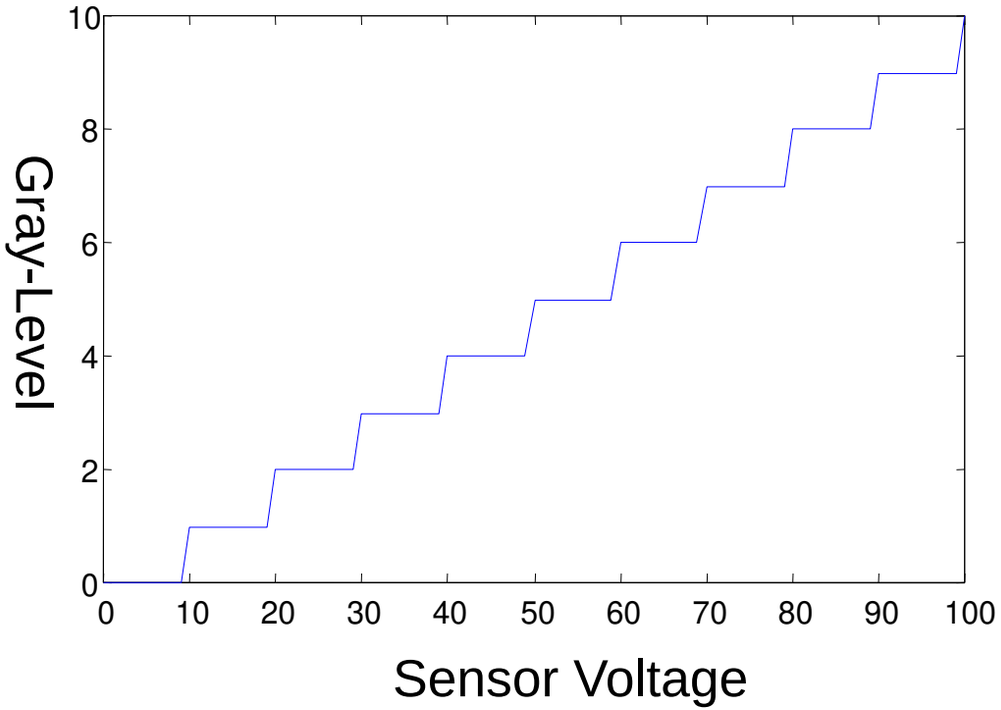
4 bits image

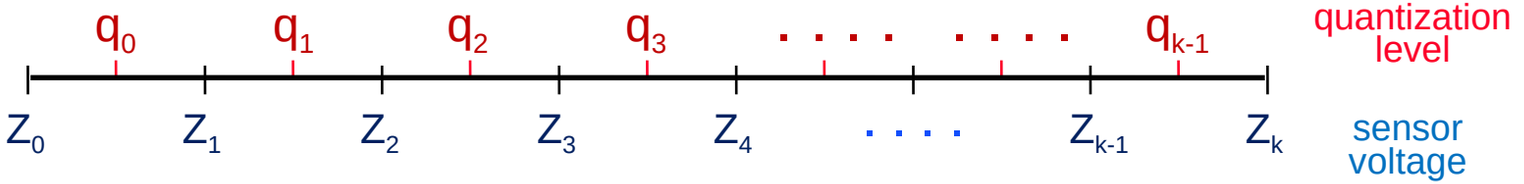
How should we quantize an image?

- Simplest approach: **uniform quantization**

$$z_{i+1} - z_i = \frac{z_k - z_0}{K}$$

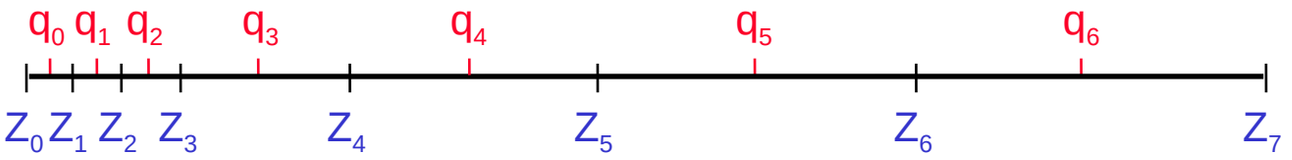
$$q_i = \frac{z_{i+1} + z_i}{2}$$





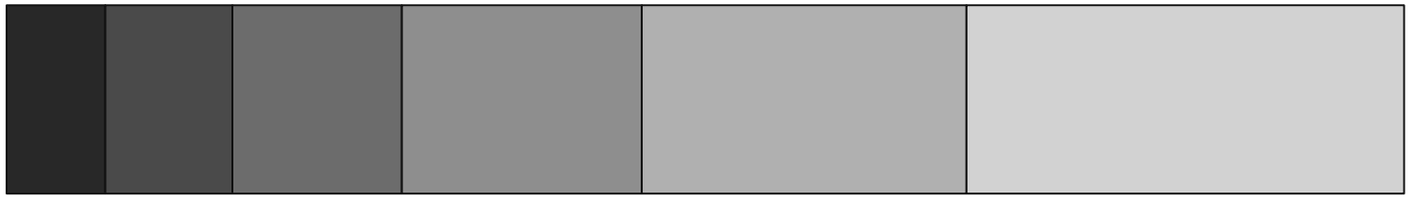
Non-uniform Quantization

- Quantize according to visual sensitivity (Weber's Law)
- Non uniform sensor voltage distribution



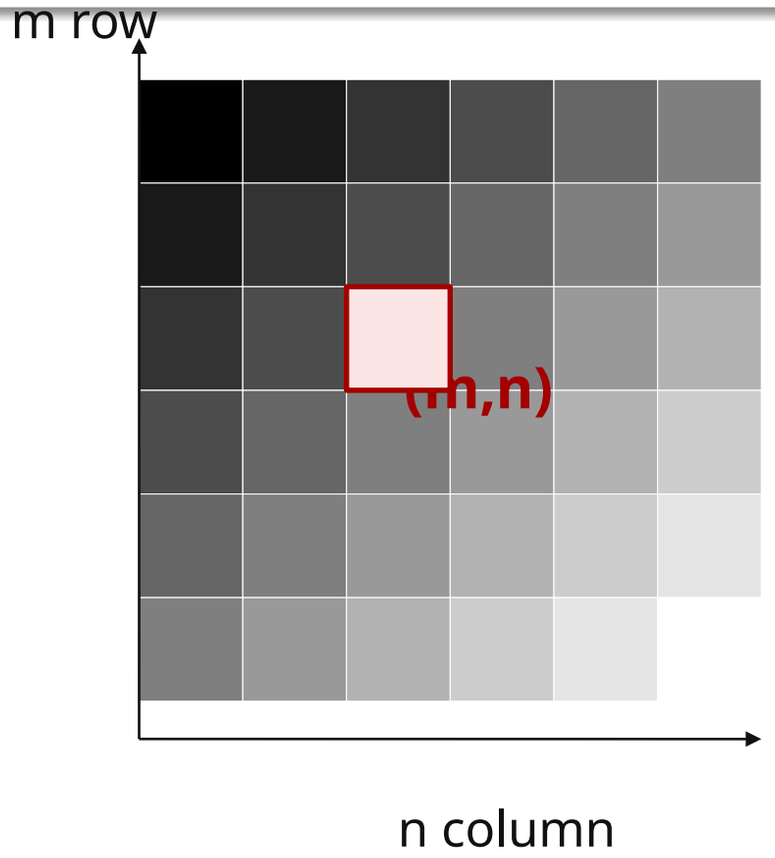
High Visual Sensitivity

Low Visual Sensitivity



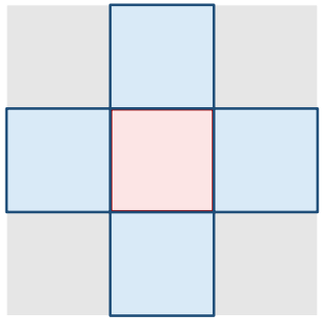
small steps in important/dark region, larger steps elsewhere

Pixel coordinate system and neighborhoods

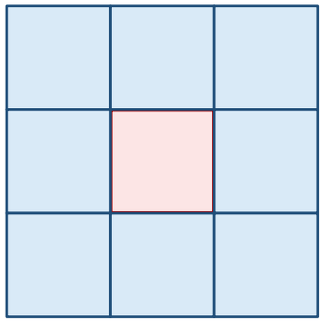


- Most image arrays use row m and column n.
- Coordinate origin is often at the upper-left corner.
- A neighborhood is a small set of nearby pixels around a center pixel.
- Neighborhood definitions are needed for filtering, segmentation, and morphology.

Pixel adjacency: 4, 8, and m-adjacency



4-neighbors

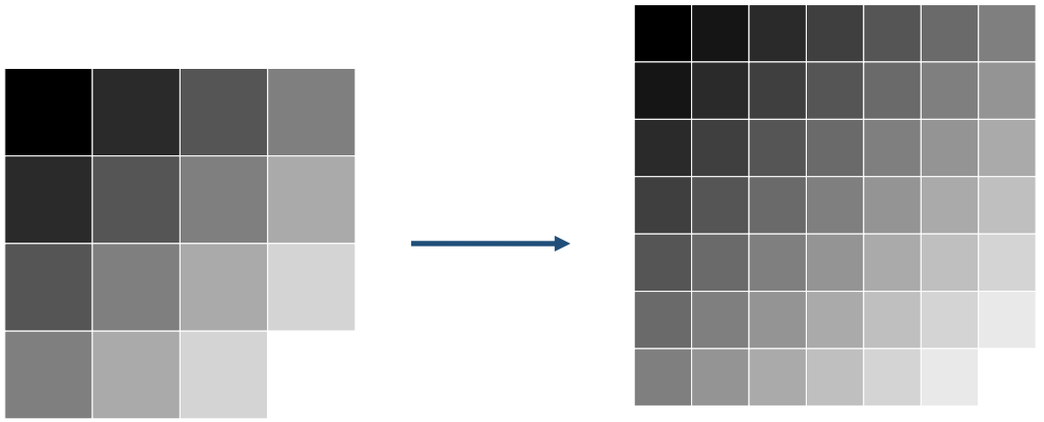


8-neighbors

- N4(p): up, down, left, right.
- ND(p): four diagonal neighbors.
- N8(p) = N4(p) union ND(p).
- m-adjacency avoids ambiguous diagonal connections when using 8-neighbor logic.

p and q are adjacent if q is in N4(p) or N8(p), depending on the rule

Resampling and interpolation after acquisition

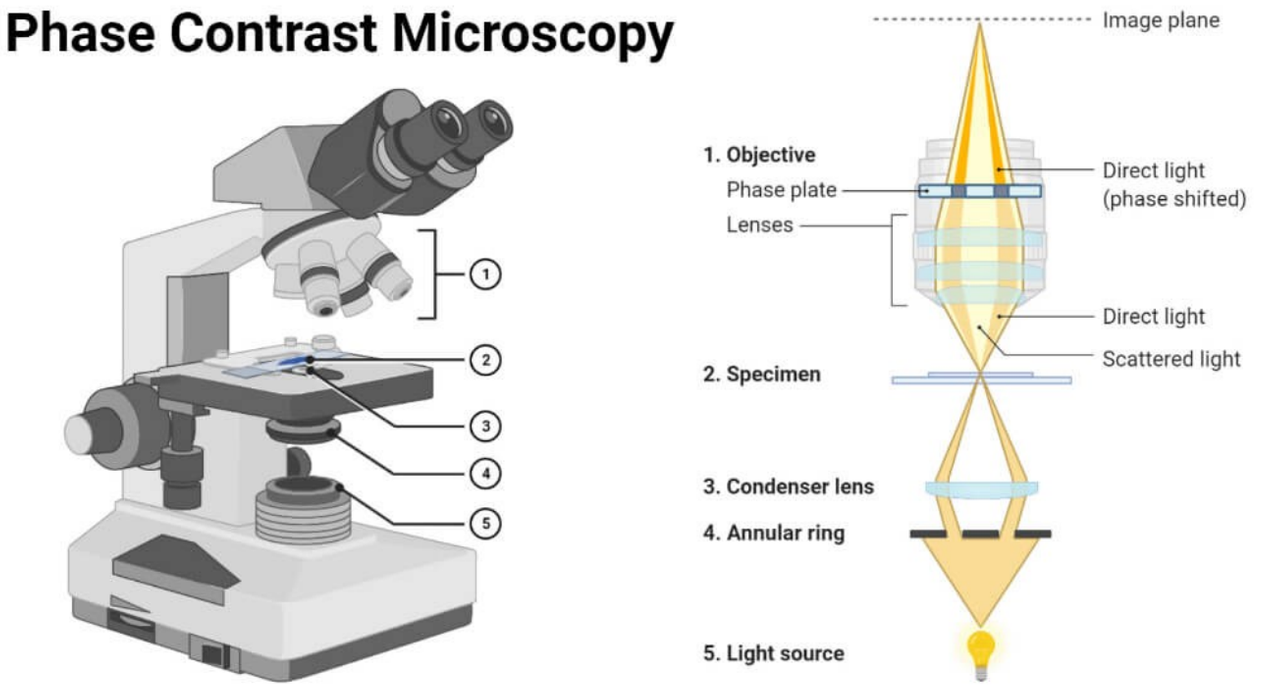


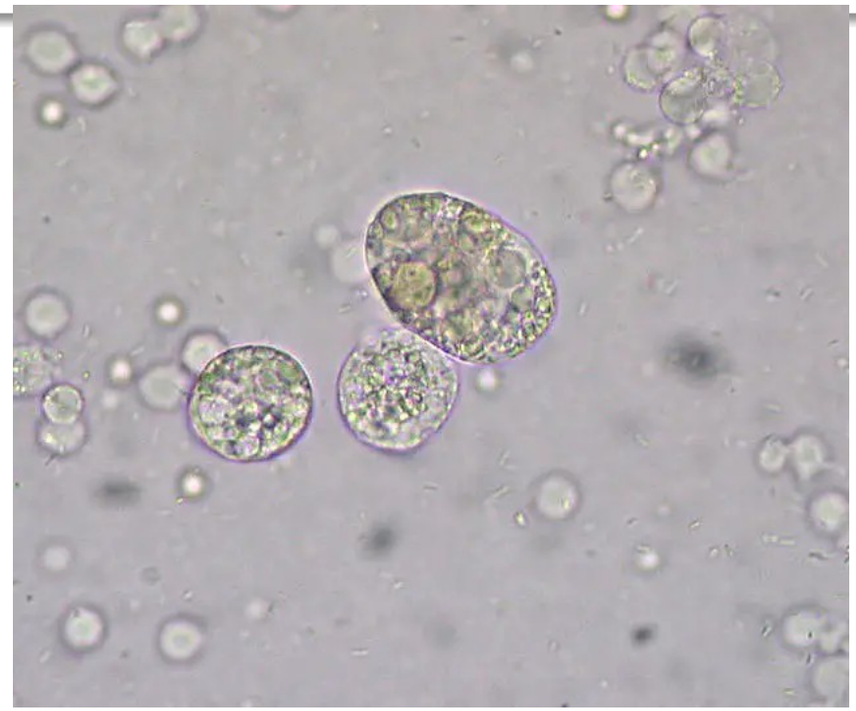
original samples *upsampled image*

- Nearest-neighbor: fast, blocky.
- Bilinear: smoother, uses local weighted average.
- Bicubic: smoother edges, more computation.
- Downsampling must use low-pass filtering to avoid aliasing.

bilinear interpolation = weighted average of 4 nearest samples

Example: microscopy / close-range imaging





- Magnification changes how scene size maps to pixel size.
- Focus depth is small, so z-position matters.
- Illumination uniformity affects measured intensity.
- Sampling must be fine enough for small structures.

Putting it together: acquisition choices affect algorithms

Low light

-> more noise

Long exposure

-> motion blur

Low sampling

-> aliasing

Low bit depth

-> banding

Bayer mosaic

-> demosaicing artifacts

Bad focus

-> blurred edges

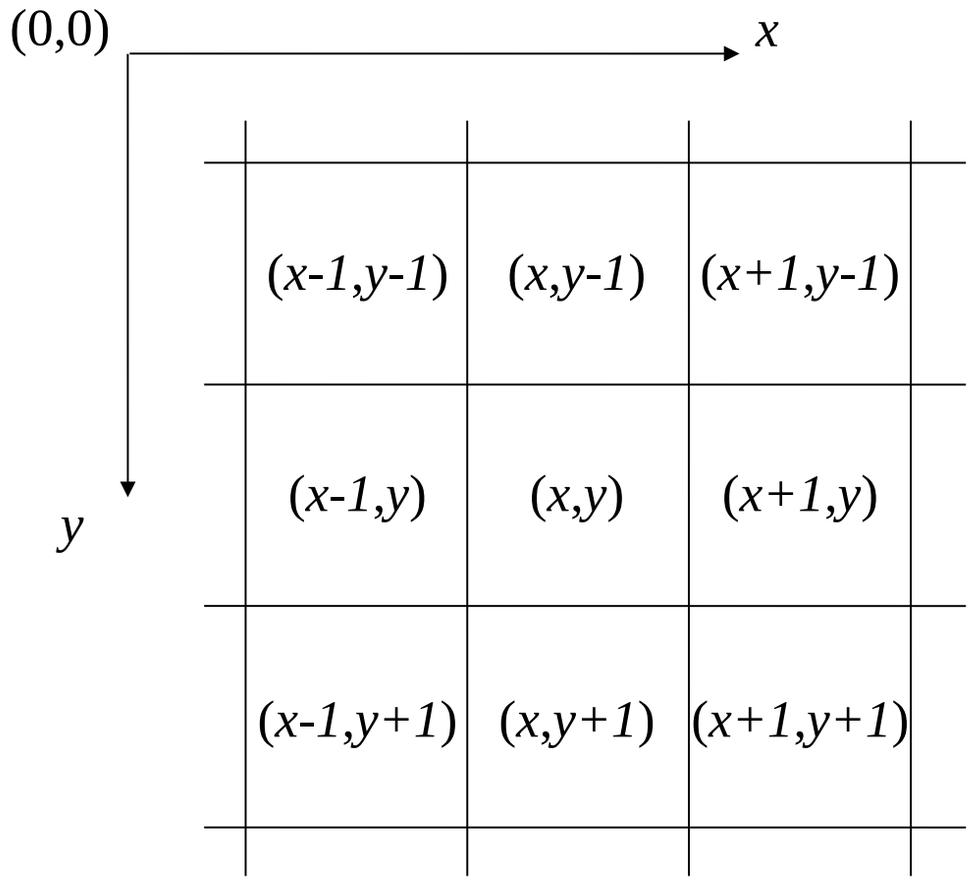
Better algorithms begin with understanding the data source.

Take-home summary

- A captured image is a measurement, not a perfect copy of the world.
- Optics define projection, focus, field of view, and blur.
- Sensors sample light over space and time; sampling too coarsely causes aliasing.
- Quantization converts analog measurements into finite numeric levels.
- Color cameras often use Bayer sampling followed by demosaicing.
- Pixel adjacency and distance definitions shape later image analysis operations.

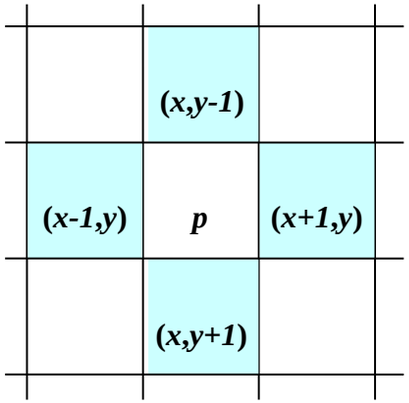
L4: Basic Relationship Between Pixels

Conventional Indexing Method



Neighbors of a Pixel

Neighborhood relation is used to tell adjacent pixels. It is useful for analyzing regions.

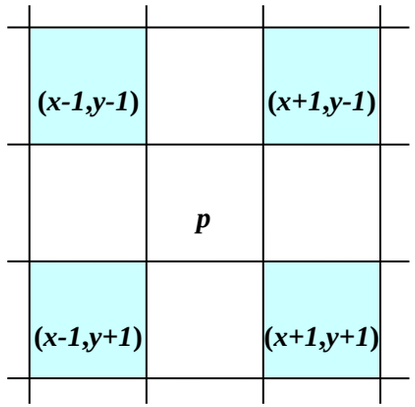


4-neighbors of *p*:

$$N_4(p) = \{(x-1,y),\ (x+1,y),\ (x,y-1),\ (x,y+1)\}$$

4-neighborhood relation considers only vertical and horizontal neighbors.

Note: $q \in N_4(p)$ implies $p \in N_4(q)$

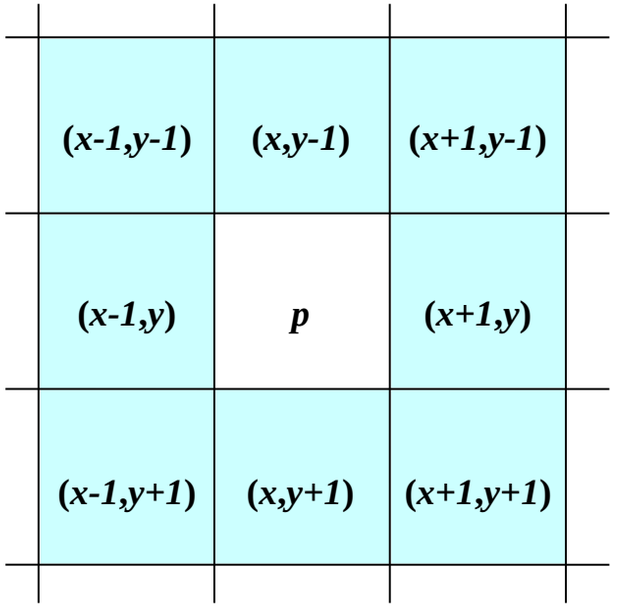


Diagonal neighbors of *p*:

$$N_D(p) = \{(x-1,y-1),\ (x+1,y-1),\ (x-1,y+1),\ (x+1,y+1)\}$$

Diagonal -neighborhood relation considers only diagonal neighbor pixels.

Neighbors of a Pixel



8-neighbors of *p*:

$$N_8(p) = \begin{Bmatrix} (x-1,y-1) \\ (x,y-1) \\ (x+1,y-1) \\ (x-1,y) \\ (x+1,y) \\ (x-1,y+1) \\ (x,y+1) \\ (x+1,y+1) \end{Bmatrix}$$

8-neighborhood relation considers all neighbor pixels.

Adjacency

- Two pixels are connected if:
  - they are neighbors,
  - their gray levels satisfy some specified criterion of similarity.

•For example, in a binary image two pixels are connected if they are 4-neighbors and have same value (0/1)

Adjacency

- Let v: a set of intensity/ gray values used to define adjacency.
- In a binary Image v={1}, if we are referring to adjacency of pixels with value 1.
- In a Gray scale image, the idea is the same, but v typically contains more elements, for example v= {180, 181, 182,….,200}
- If the possible intensity values 0 to 255, v set could be any subset of these 256 values

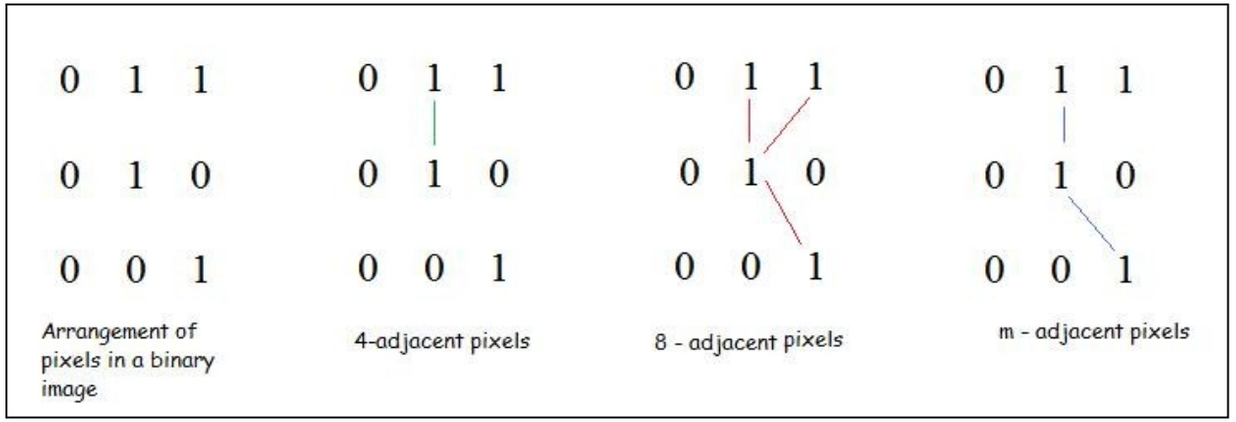
Types of Adjacency

3 types of adjacency

- 4- adjacency: 2 pixels *p* and *q* with values from *V* are 4- adjacent if *q* is in the set $N_4(p)$
- 8- adjacency: 2 pixels *p* and *q* with values from *V* are 8- adjacent if *q* is in the set $N_8(p)$
- *m*- adjacency: 2 pixels *p* and *q* with values from *V* are *m*-adjacent if

  1.q is in $N_4(p)$, *OR*

  2.q is in $N_D(p)$ *and* the set $N_4(p) \cap N_4(q)$ has no pixels whose values are from V



Types of Adjacency

4-adjacency: Two pixels p and q with the values from set 'V' are 4-adjacent if q is in the set of $N_4(p)$.

e.g. V = { 0, 1}

1	1	2
1	1	0
1	0	1

p in **RED** color

q can be any value in **GREEN** color.

Types of Adjacency

8-adjacency: Two pixels p and q with the values from set 'V' are 8-adjacent if q is in the set of $N_8(p)$.

e.g. V = { 1, 2}

0	1	1
0	2	0
0	0	1

p in **RED** color

q can be any value in **GREEN** color.

Types of Adjacency

m-adjacency: Two pixels p and q with the values from set 'V' are m-adjacent if

1. q is in $N_4(p)$, or
2. q is in $N_D(p)$ and the set $N_4(p) \cap N_4(q)$ has no pixels whose values are from 'V'

e.g. V = { 1 }

$$\begin{matrix} 0_a & 1_b & 1_c \\ 0_d & 1_e & 0_f \\ 0_g & 0_h & 1_i \end{matrix}$$

Types of Adjacency

m-adjacency: Two pixels p and q with the values from set 'V' are m-adjacent if

1. q is in $N_4(p)$

e.g. V = { 1 }

(i) b & c

0_a 1_b 1_c

0_d 1_e 0_f

0_g 0_h 1_i

Ans: b & c are m-adjacent.

Types of Adjacency

m-adjacency: Two pixels p and q with the values from set 'V' are m-adjacent if

1. q is in $N_4(p)$

e.g. V = { 1 }

(ii) b & e

0 a 1 b 1 c

0 d 1 e 0 f

0 g 0 h 1 I

Ans: b & e are m-adjacent.

Types of Adjacency

m-adjacency: Two pixels p and q with the values from set 'V' are m-adjacent if

1. q is in $N_4(p)$ OR

e.g. V = { 1 }

(iii) e & i

0 a 1 b 1 c

0 d 1 e 0 f

0 g 0 h 1 i

Pixels value of f & h are in $N_4(e)$ and $N_4(i)$. Their values are not in 'V' and they satisfy the 2nd condition

Ans: e & i are m-adjacent

Types of Adjacency

m-adjacency: Two pixels p and q with the values from set 'V' are m-adjacent if

1. q is in $N_4(p)$ OR
2. q is in $N_D(p)$ & the set $N_4(p) \cap N_4(q)$ have no pixels whose values are from 'V'.

e.g. V = { 1 }

(iv) e & c

0 a	1 b	1 c
0 d	1 e	0 f
0 g	0 h	1 I

Pixel value of b is in $N_4(c)$ and $N_4(e)$. The value is in 'V' and does not satisfy the 2nd condition

Ans: e & c are NOT m-adjacent

Problem

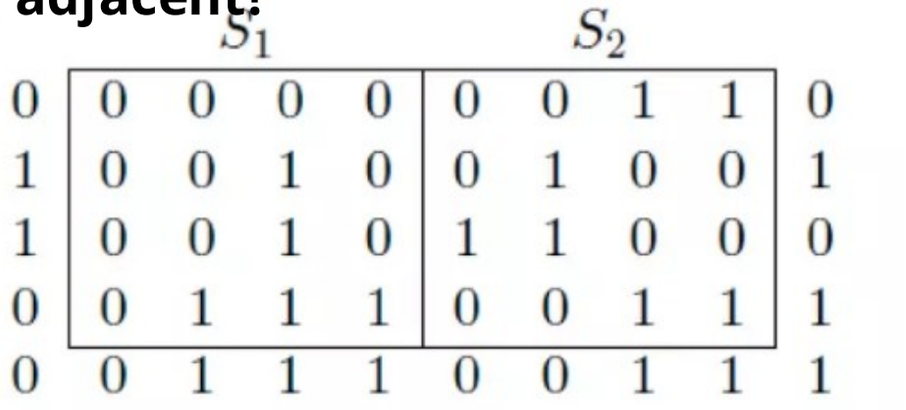
Two subsets *S1* and *S2* are adjacent, if some pixel in *S1* is adjacent to some pixel in *S2*. Adjacent means, either 4-, 8- or m-adjacency.

Consider the two image subsets, S_1 and S_2, shown in the following figure. For $V = \{1\}$, determine whether these two subsets are (a) 4-adjacent, (b) 8-adjacent, or (c) *m*-adjacent.

	S_1				S_2				
0	0	0	0	0	0	0	1	1	0
1	0	0	1	0	0	1	0	0	1
1	0	0	1	0	1	1	0	0	0
0	0	1	1	1	0	0	0	0	0
0	0	1	1	1	0	0	1	1	1

Problem

- **Consider the two image subsets S_1 and S_2 shown below. For V = {1}, determine:**
  - **How many 4-connected components are there in S_1 and S_2?**
  - **How many 8-connected components are there in S_1 and S_2?**
  - **How many m-connected components are there in S_1 and S_2?**
  - **Are S_1 and S_2 adjacent?**

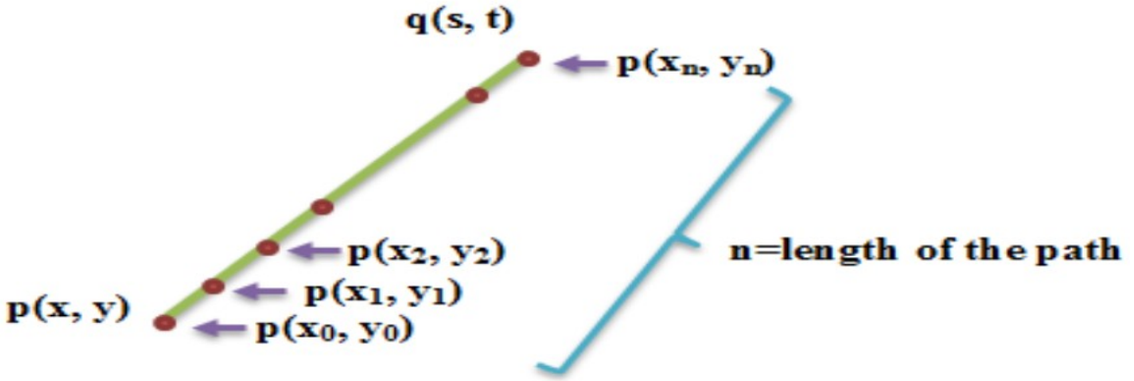


Connectivity

- 2 pixels are said to be connected if there exists a path between them.
- Let S represent a subset of pixels in an image, Two pixels p and q are said to be connected in S if there exists a path between them consisting entirely of pixels in ‘S’.

Paths

- A (digital) path (or curve) from pixel p with coordinates (x, y) to pixel q with coordinates (s, t) is a sequence of distinct pixels with coordinates (x_0, y_0) , (x_1 , y_1) ..... (x_n , y_n),
  - $(x, y) = (x_0, y_0)$ and $(s, t) = (x_n , y_n)$
  - (x_i , y_i) and (x_{i-1} , y_{i-1}) are adjacent for $1 \leq i \leq n$.
  - n is the length of the path.
  - If $(x_0, y_0) = (x_n , y_n)$, the path is closed path.
- We can define 4-, 8-, and m-paths based on the type of adjacency used.



Paths

Example 1: **Consider the image segment shown in figure (a). Compute length of the shortest-4, shortest-8 & shortest-m paths between pixels p & q where,**

V = {1, 2}

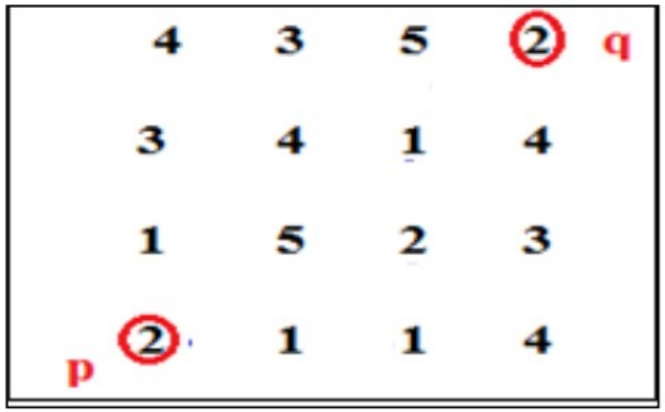


figure (a)

Paths

V = {1, 2}

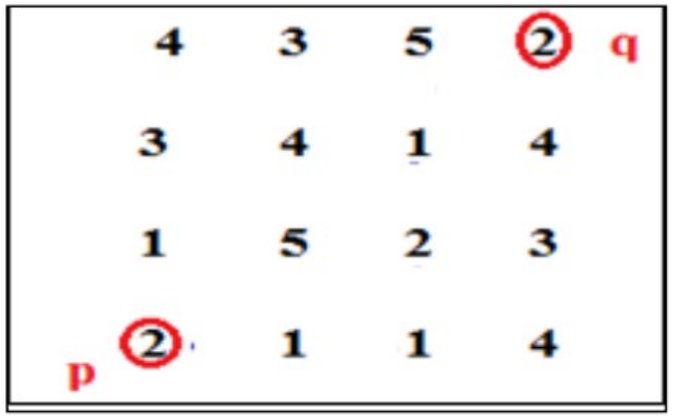
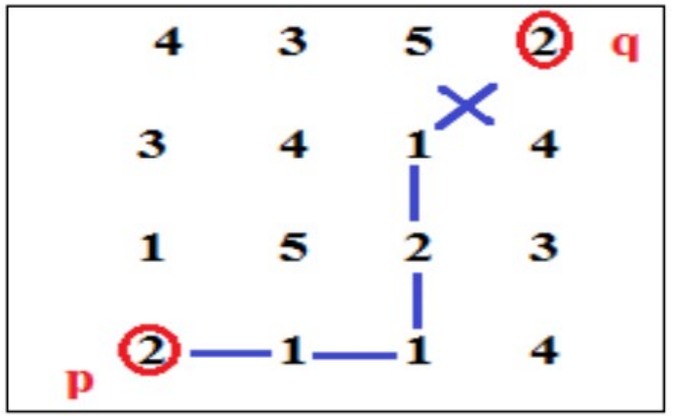
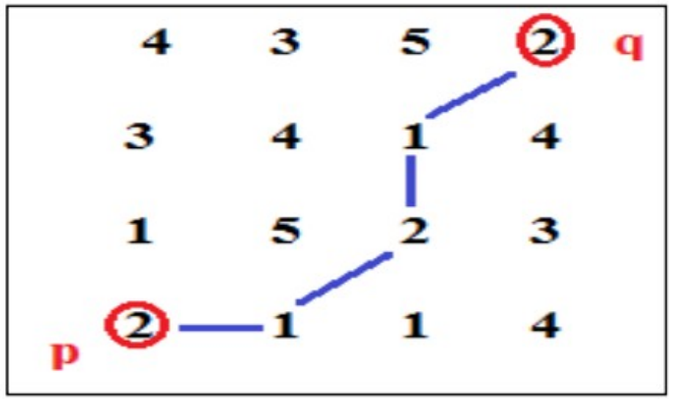


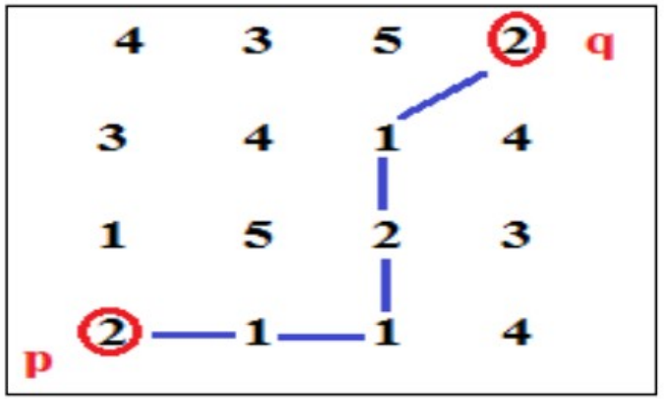
figure (a)



Shortest-4 Path does not exist



shortest-8 path = 4



shortest–m Path = 5

Paths

Example 2: Consider the image segment shown in figure. Compute length of the shortest-4, shortest-8 & shortest-m paths between pixels p & q where,

V = {1, 2}

	4	2	3	2 q
	3	3	1	3
	2	3	2	2
p	2	1	2	3

Paths

Example 3: Consider the image segment shown in figure (a). Compute length of the shortest-4, shortest-8 & shortest-m paths between pixels p & q where,

V = {2, 3}

4	5	6	4	5	4
3	5	3	4	3	4
4	4	4	2	4	3
5	3	4	3	3	2
6	3	5	3	4	6
5	4	2	3	3	6

q

p

figure (a)

Paths

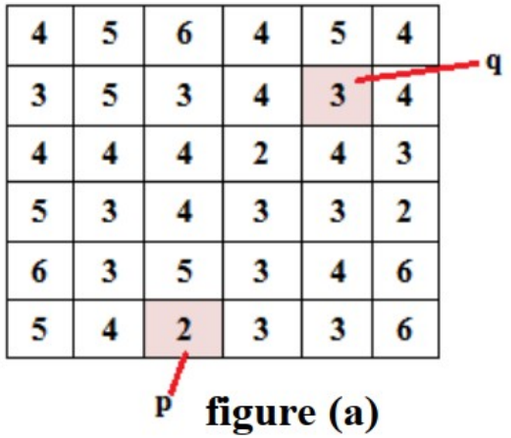
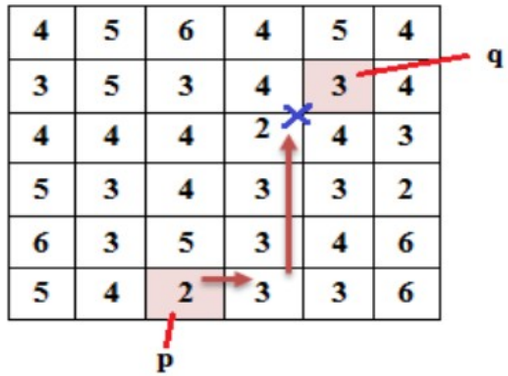
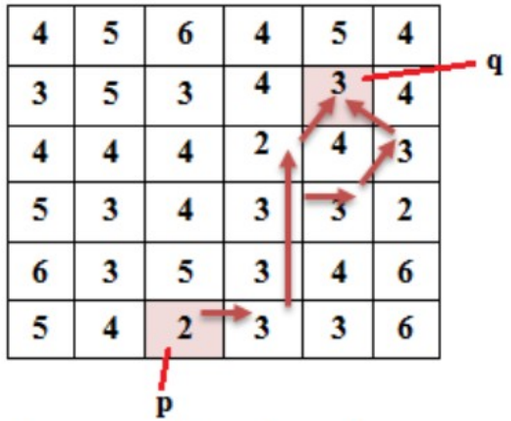


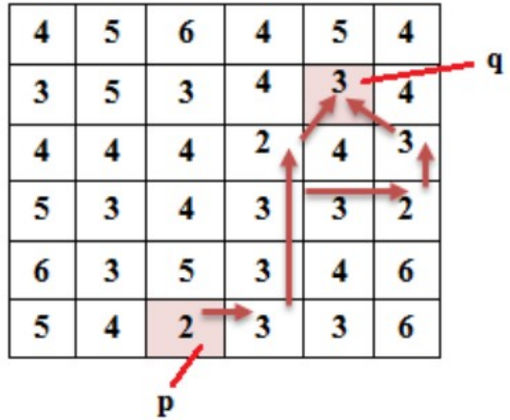
figure (a)



Shortest-4 Path does not exist



shortest-8 path = 5
= 6



shortest–m Path = 5
=7

Regions & Boundaries

Region: Let R be a subset of pixels in an image. Two regions R_i and R_j are said to be adjacent if their union form a connected set.

- Regions that are not adjacent are said to be disjoint.
- We consider 4- and 8- adjacency when referring to regions. Below regions are adjacent only if 8-adjacency is used.

1	1	1	
1	0	1	R_i
0	1	0	
0	0	1	
1	1	1	R_j
1	1	1	

Regions & Boundaries

Boundaries (border or contour): The boundary of a region R is the set of pixels in the region that have one or more neighbors that are not in R
or that are adjacent to pixels in the compliment of R.

0	0	0	0	0
0	1	1	0	0
0	1	1	0	0
0	1	1	1	0
0	1	1	1	0
0	0	0	0	0

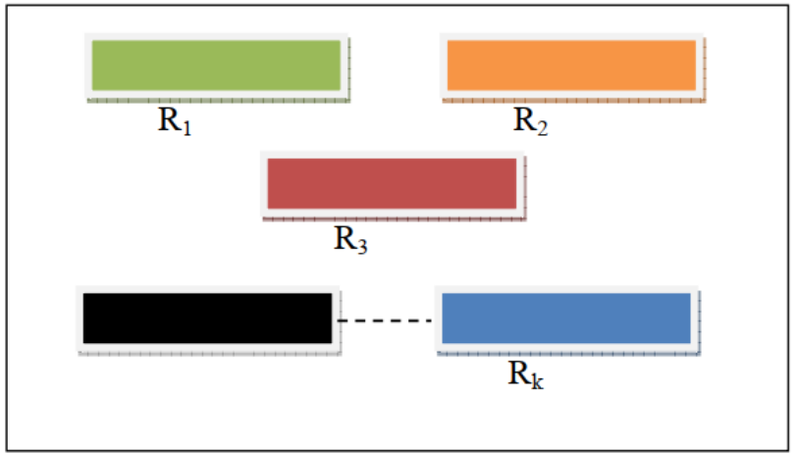
RED colored 1 is NOT a member of border if 4 or 8 connectivity is used between region and background.

Regions & Boundaries

Boundaries (border or contour): If R happens to be an entire image, then its boundary is defined as the set of pixels in the first and last rows and columns of the image.

0	0	0	0	0
0	1	1	1	0
0	1	1	1	0
0	1	1	1	0
0	1	1	1	0
0	0	0	0	0

Regions & Boundaries

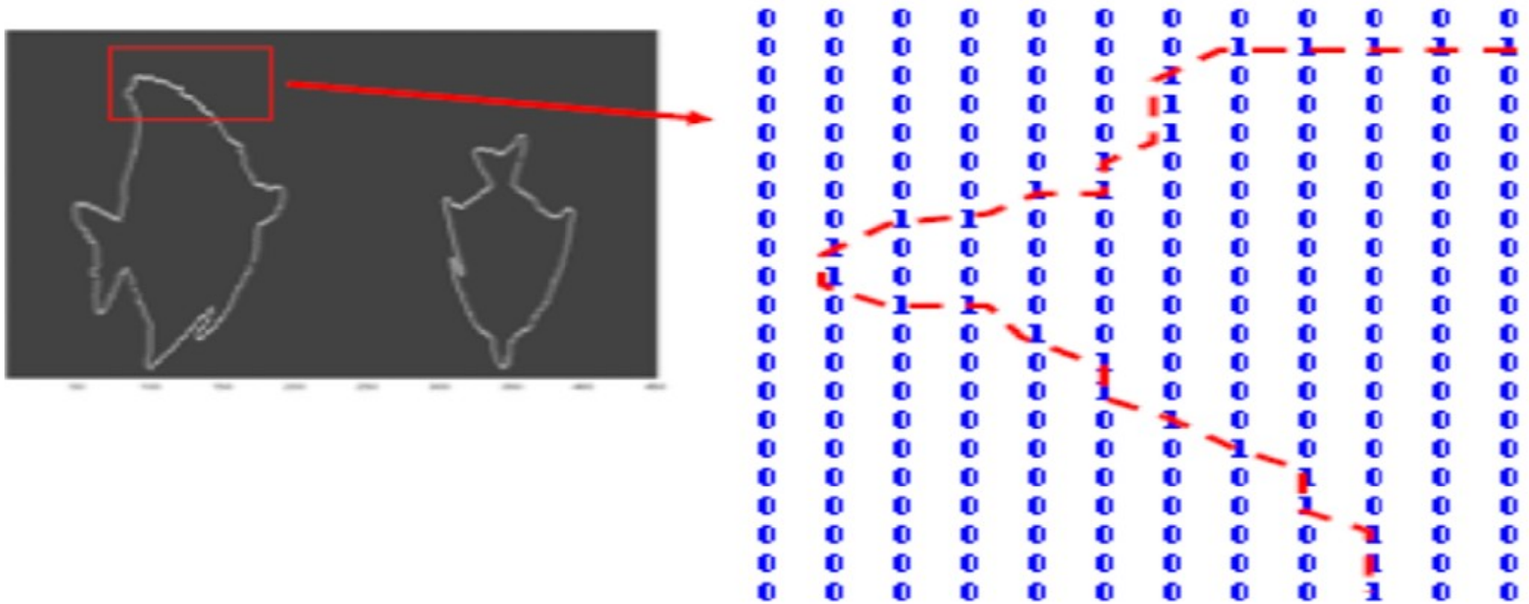


The image contains K disjoint regions, R_k, k=1, 2, …., k, none of which touches the image border.

Let: R_u **-** denote the **union** of all the K regions, $(R_u)^c$**-** denote its **complement** (**Complement** of a set S is the set of points that are not in s).

R_u **-** called **foreground** and $(R_u)^c$ **-** called **background** of the image.

Regions & Boundaries



Distance Measures

Assuming there are two image points with coordinates (x, y) and (s, t).

- A distance measure is normally conduced for evaluating how close two these two pixels are and how they are related. A number of distance measurements have been commonly used for this purpose.

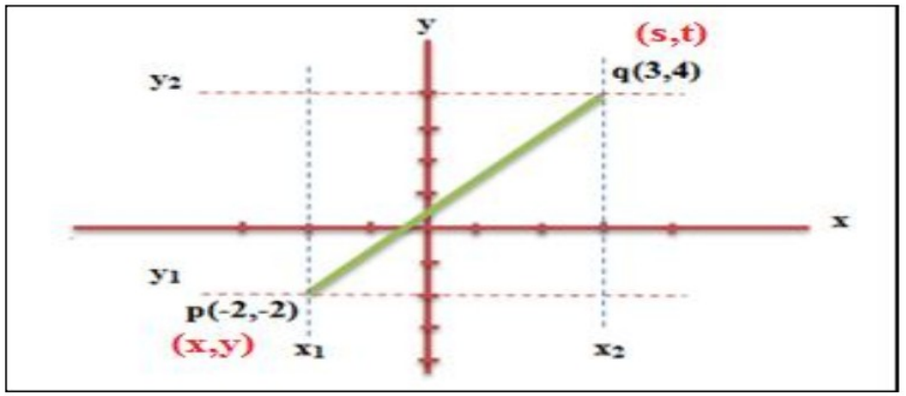
1. The Euclidean distance between two 2-D points p(x, y) and q(s, t) is defined as:

$$D_e(p,q) = \sqrt{(x-s)^2 + (y-t)^2}$$

Where:

$x = x_1$, $y = y_1$

$s = x_2$, $t = y_2$

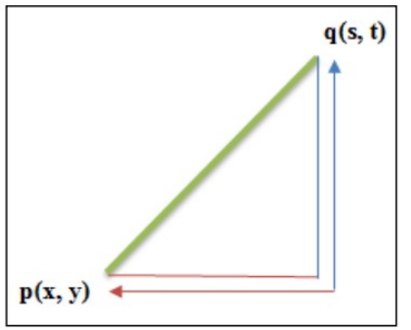


Distance Measures

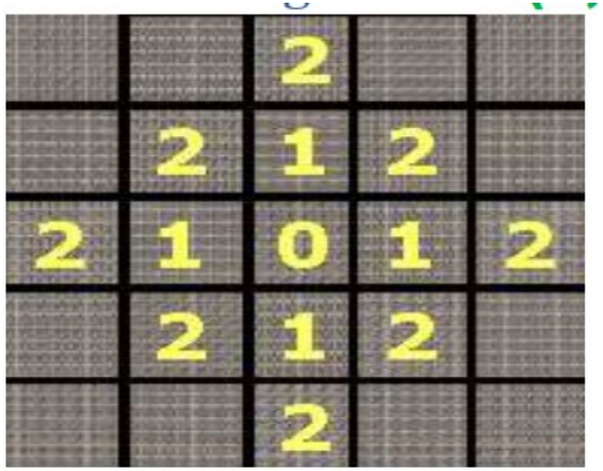
2. The D_4 distance (called the City-Block distance) between between two

2-D points p(x, y)and q(s, t) is defined as:

$$\mathbf{D_4}(\boldsymbol{p}, \boldsymbol{q}) = |\boldsymbol{x} - \boldsymbol{s}| + |\boldsymbol{y} - \boldsymbol{t}|$$



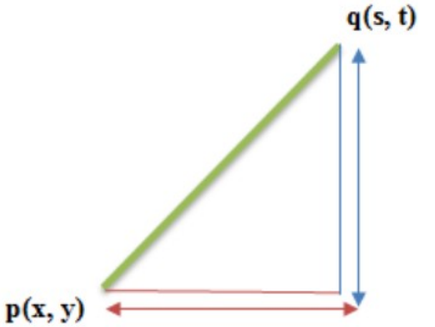
In this case, pixels having D_4 distance from (x, y) less than or equal to some value r form a diamond centered at (x, y). Pixels with D_4 distance ≤ 2 forms the following contour of constant distance.



Distance Measures

3. The D_8 distance (called the chessboard distance) between p and q is defined as:

$$\mathbf{D_8}(\boldsymbol{p}, \boldsymbol{q}) = \boldsymbol{max}\,(|\boldsymbol{x} - \boldsymbol{s}|, |\boldsymbol{y} - \boldsymbol{t}|)$$

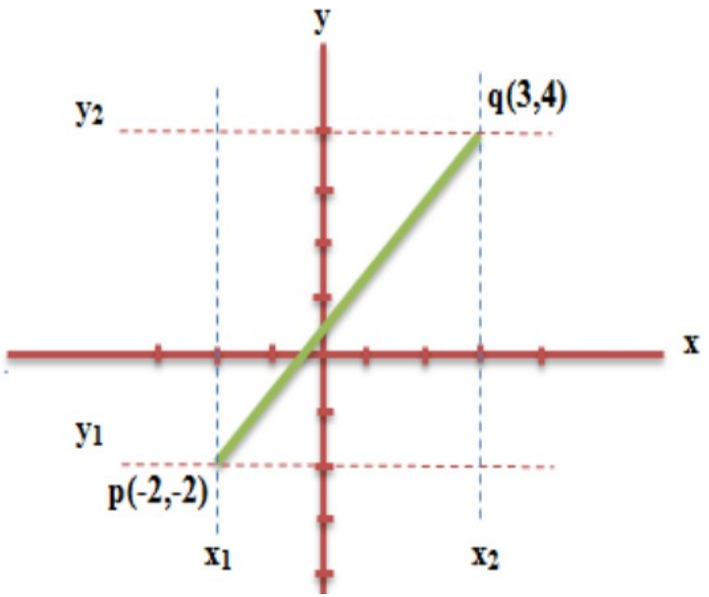


In this case, pixels having D_8 distance from (x, y) less than or equal to some value r form a square centered at (x, y).
Pixels with D_4 distance ≤ 2 forms the following contour of constant distance.

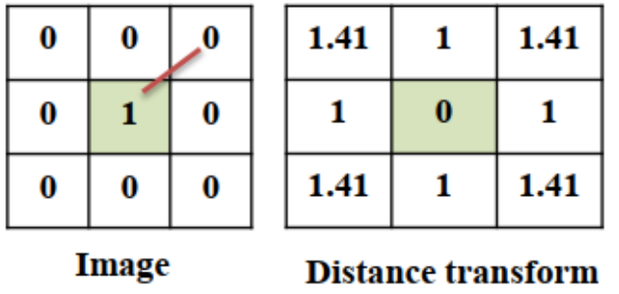
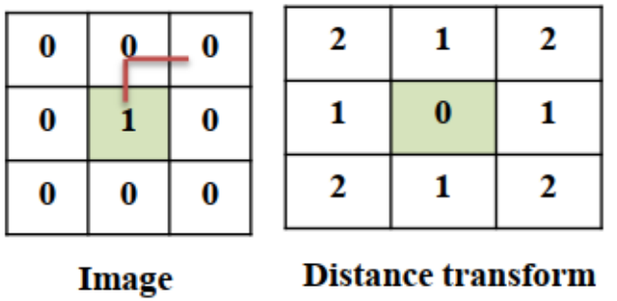
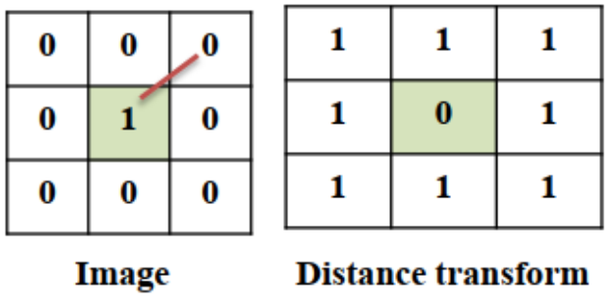
2	2	2	2	2
2	1	1	1	2
2	1	0	1	2
2	1	1	1	2
2	2	2	2	2

Distance Measures

Problem : Suppose p and q are two pixels. Calculate the distance measures D_4, D_8 and D_e

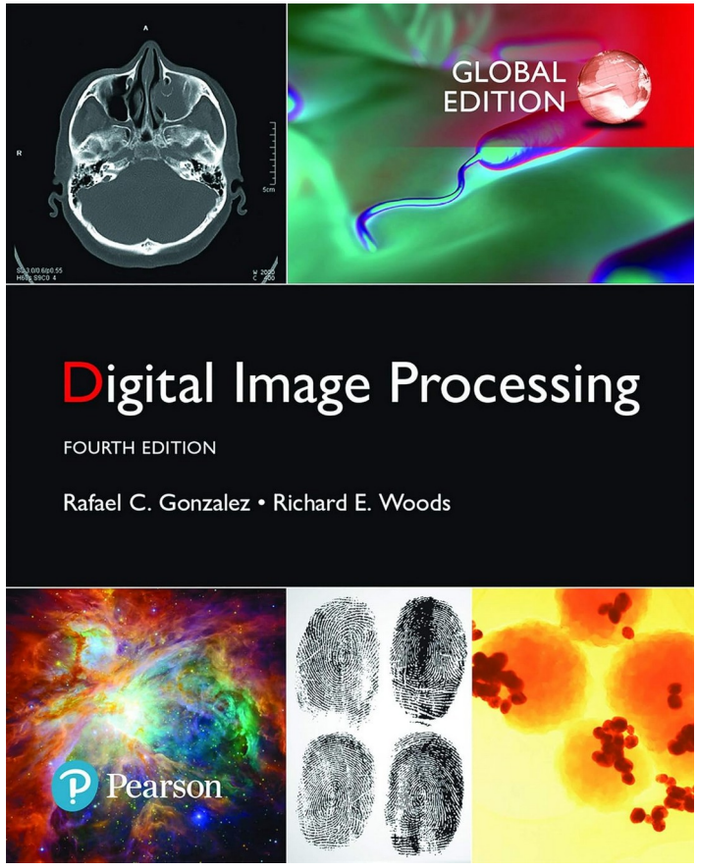


Distance Measures

Distance Metric	Description	Illustration
Euclidean P(x , y), q(s , t)	The Euclidean distance is the straight-line distance between two pixels. $D_e(p,q) = \sqrt{(x-s)^2 + (y-t)^2}$	
City Block P(x , y), q(s , t)	The city block distance metric measures the path between the pixels based on a 4-connected neighborhood. Pixels whose edges touch are 1 unit apart and pixels diagonally touching are 2 units apart. $D_4(p,q) = \lvert x-s \rvert + \lvert y-t \rvert$	
Chessboard P(x , y), q(s , t)	The chessboard distance metric measures the path between the pixels based on an 8-connected neighborhood. Pixels whose edges or corners touch are 1 unit apart. $D_8(p,q) = max(\lvert x-s \rvert, \lvert y-t \rvert)$	

References

- Digital Image Processing, Rafael C. Gonzalez, Richard E. Woods (Chapter 2)



Point Operations On Images

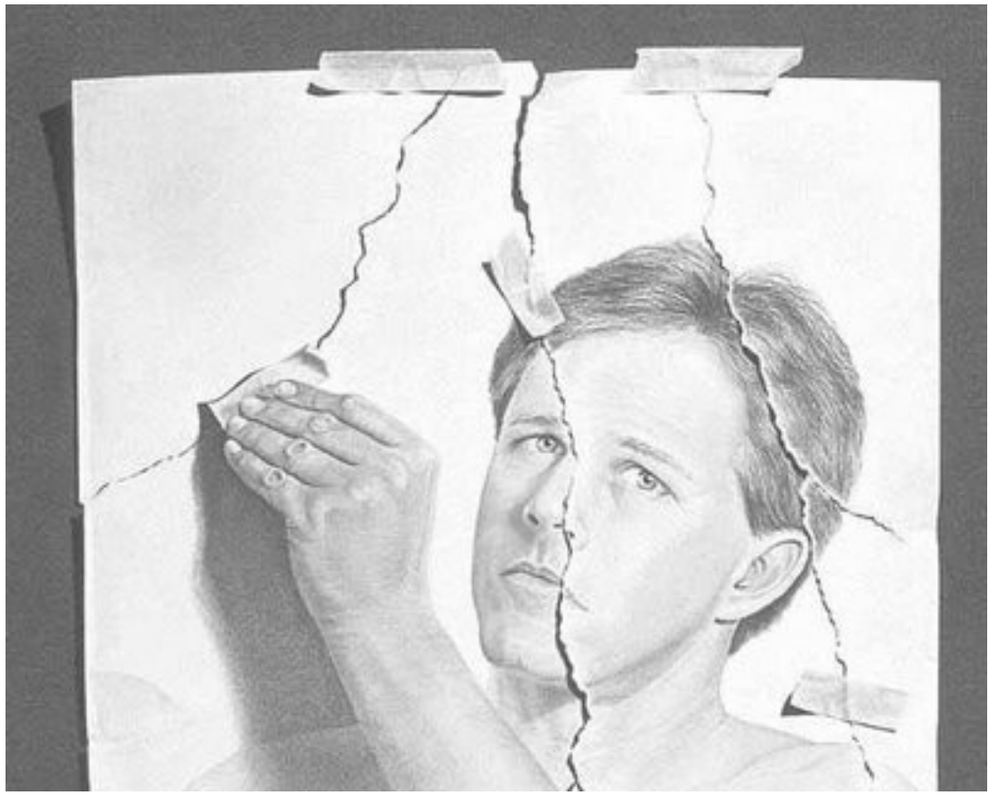
Prof. Dr. Sk. Md. Masudul Ahsan
Dept. of CSE, KUET

Learning outcomes

- **Upon completion of this lecture, students will be able to:**
  - Explain basic intensity transformation techniques for image enhancement.
  - Apply point operations and contrast stretching to modify image intensity.
  - Analyze and implement histogram equalization and histogram matching.
  - Compare global and local histogram processing methods for enhancing image quality.

Basic Image Operations

- Three types of image operations
  1. Point operations
  2. Geometric operations
  3. Spatial operations

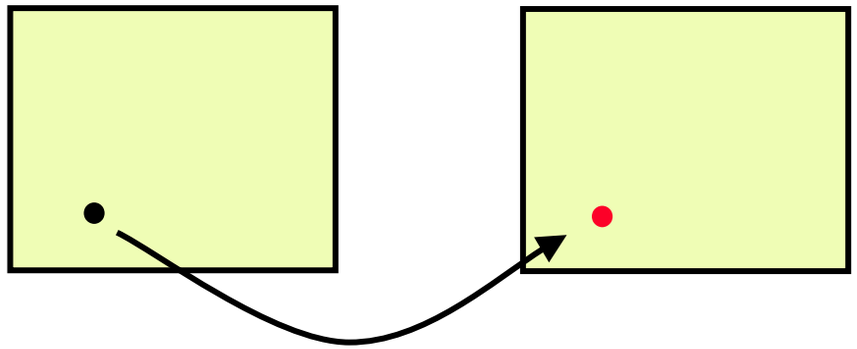


Point Operations

- Operation depends on Pixel's value.
- Context free (memory-less).
- Operation can be performed on the Histogra
- Example:

$$g(x,y)=\alpha \cdot f(x,y)+\beta$$

$$g(x,y)=M(f(x,y))$$

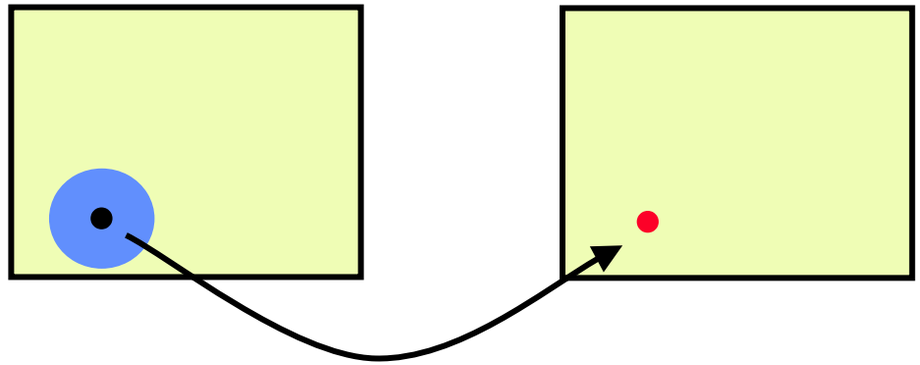


Spatial Operations

- Operation depends on Pixel's value and coordinates.
- Context dependent.
- Spatial vs.. Frequency domain.
- Example:

$$g(x,y) = \sum_{i,j \in N(x,y)} f(i,j)/n$$

$$g(x,y) = M(\{f(i,j) | (i,j) \in N(x,y)\})$$

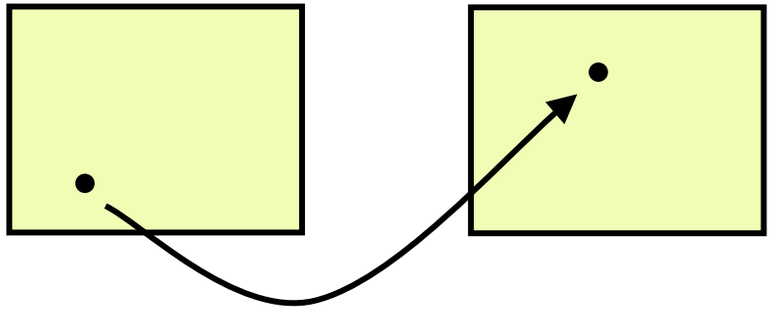


Geometric Operations

- Operation depends on pixel's coordinates.
- Context free.
- Independent of pixels value.
- Example:

$$g(x, y) = f(x + a, y + b)$$

$$g(x, y) = f(G(x, y))$$



Point Processing

- Also called Intensity transformations
- simple to implement
  - algorithms, functions, lookup tables
- used to enhance images that are devoted to visual processing
  - no general rule for stating the optimality
  - application/user-dependent
- often designed based on the properties of the image histogram

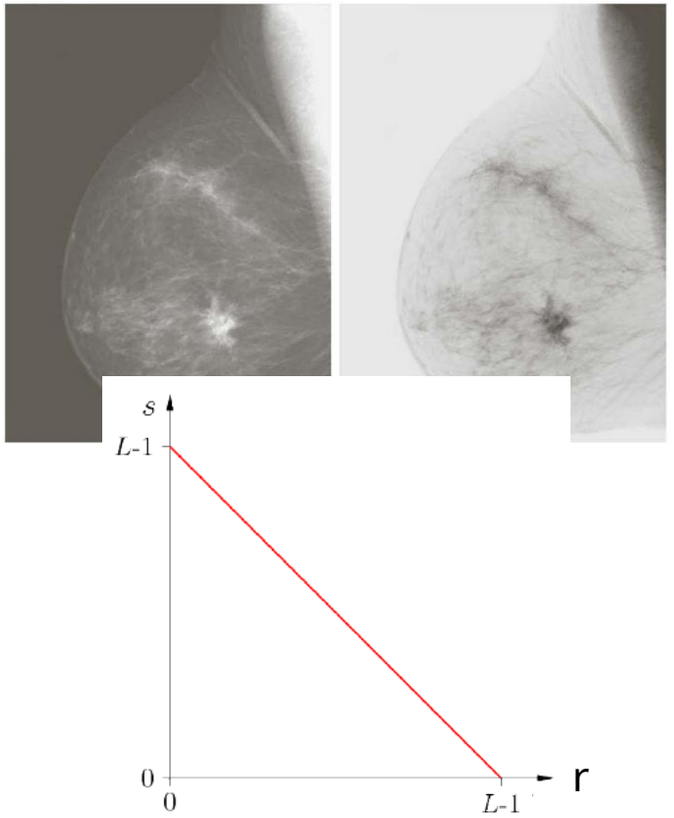


image negative

$S = T(r) = L - 1 - r$

Array/Elementwise vs Matrix Operations

- **Array Operations:** pixel by pixel

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \odot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} \\ a_{21}b_{21} & a_{22}b_{22} \end{bmatrix}$$

e.g. Array product

- **Matrix Operations:** using matrix theory

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix}$$

e.g. Matrix product

Arithmetic (image) Operations

Performed on corresponding pixels (array operations):

$$s(x,y) = f(x,y) + g(x,y)$$
$$d(x,y) = f(x,y) - g(x,y)$$
$$p(x,y) = f(x,y) \times g(x,y)$$
$$v(x,y) = f(x,y) \div g(x,y)$$

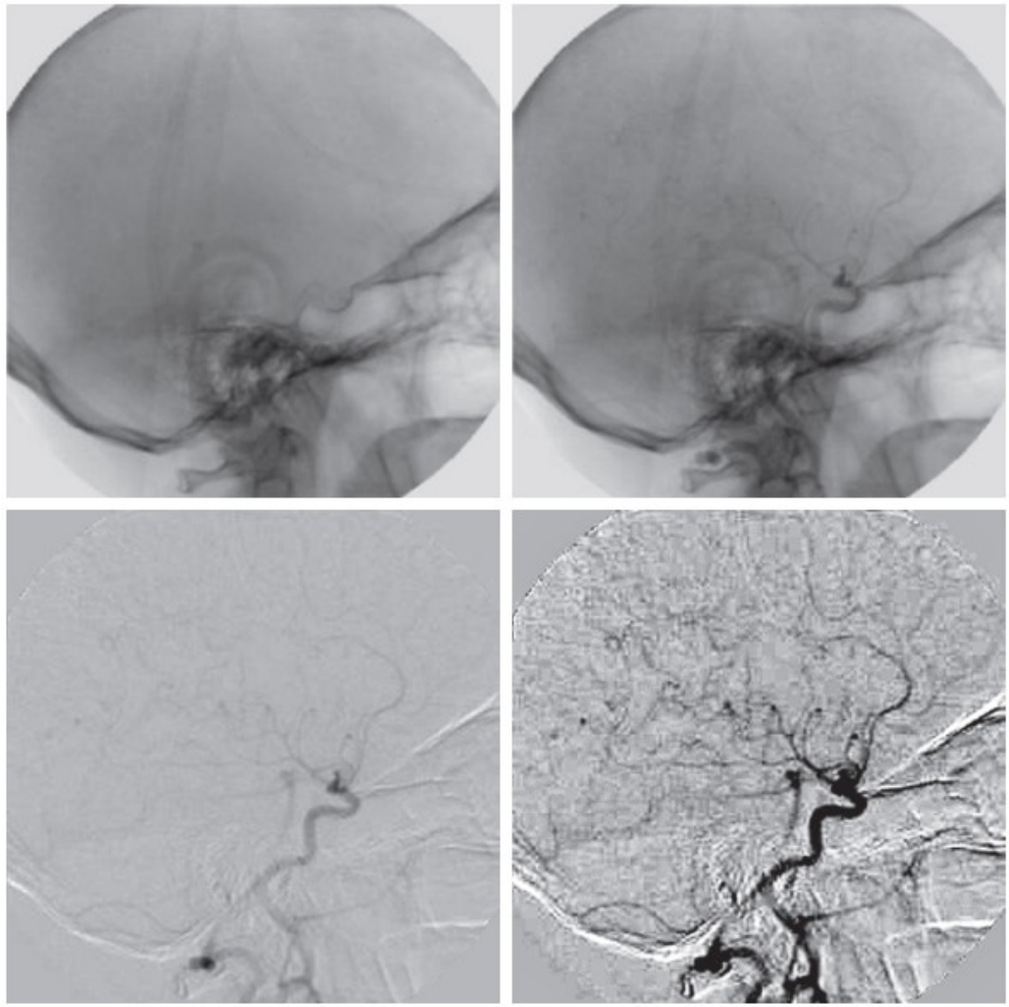
Image addition

Arithmetic (image) Operations

Image subtraction

a b
c d

FIGURE 2.32 Digital subtraction angiography. (a) Mask image. (b) A live image. (c) Difference between (a) and (b). (d) Enhanced difference image. (Figures (a) and (b) courtesy of the Image Sciences Institute, University Medical Center, Utrecht, The Netherlands.)



Arithmetic (image) Operations

Image multiplication: masks to select the Region of Interest (ROI)

Image division example: change detection (division gives the fractional change or ratio between corresponding pixel values)

Arithmetic (image) Operations

Blending:

$0.5 \times$ $+ (1-0.5) \times$ $=$

Logical Operation

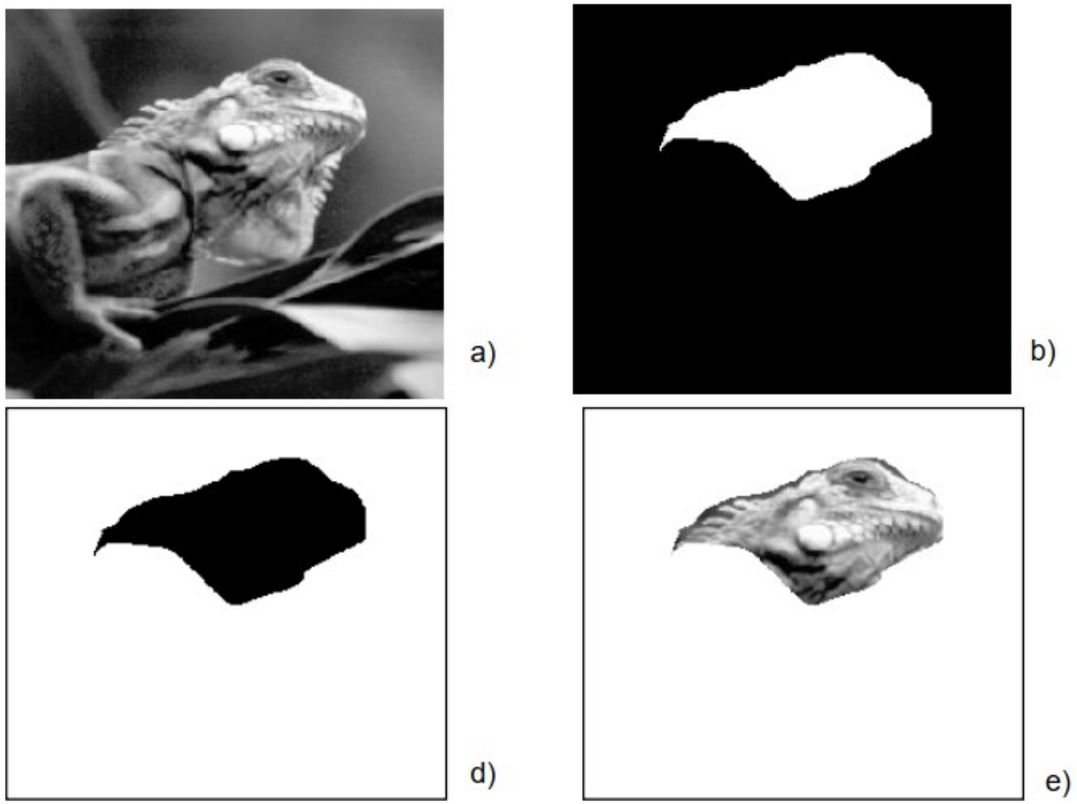
Logical Expressions

Operation	Definition
NOT	$C = \bar{A}$
OR	$C = A + B$
NOR	$C = \overline{A + B}$
AND	$C = AB$
NAND	$C = \overline{AB}$
XOR	$C = A \oplus B = A\bar{B} + \bar{A}B$
XNOR	$C = \overline{A \oplus B}$

Logical Operation

AND-OR Operation: Image masking

Figure 3.2-10 Image Masking. a) Original image, b) image mask for AND operation, c) Resulting image from (a) AND (b), d) image mask for OR operation, created by performing a NOT on mask (b), e) Resulting image from (a) OR (d).



Intensity Transformation

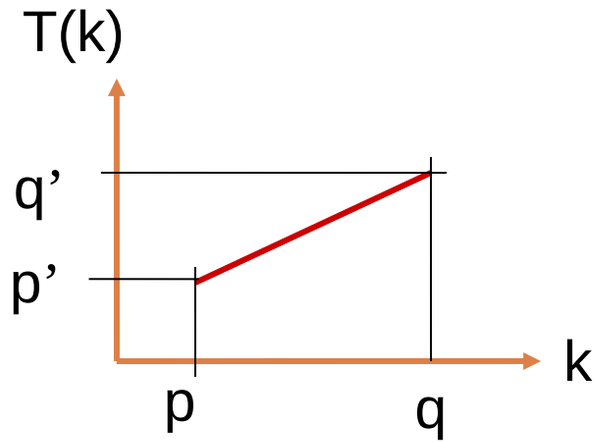
- A point operation can be defined as a mapping function:

$$k_{new} = T(k_{old})$$

where *k* stands for gray values.

- T(k) takes any value k in the source image into k_{new} in the destination image.
- Simplest case - Linear Mapping:

$$T(k) = \alpha k + \beta$$



$$\alpha = \frac{q' - p'}{q - p} \quad ; \quad \beta = p' - \alpha p$$

The Negative Mapping

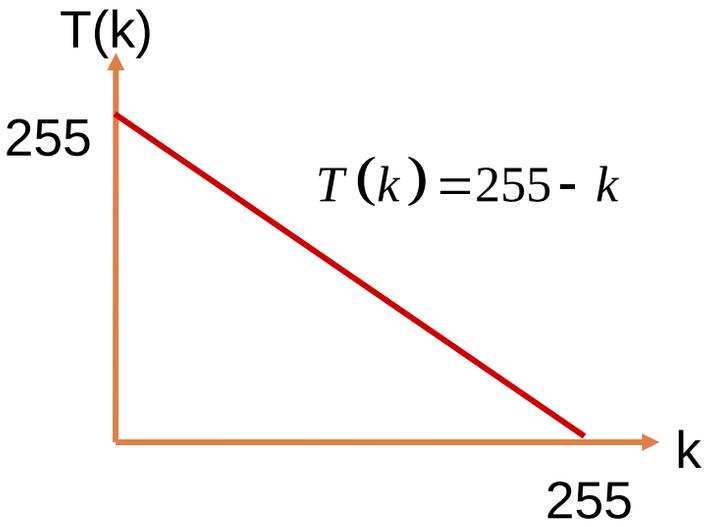
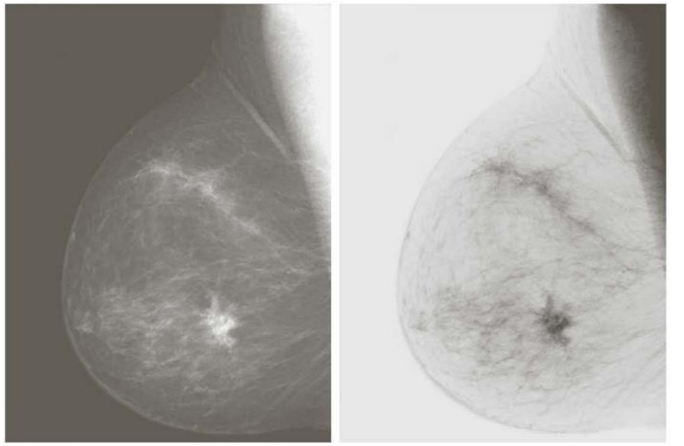
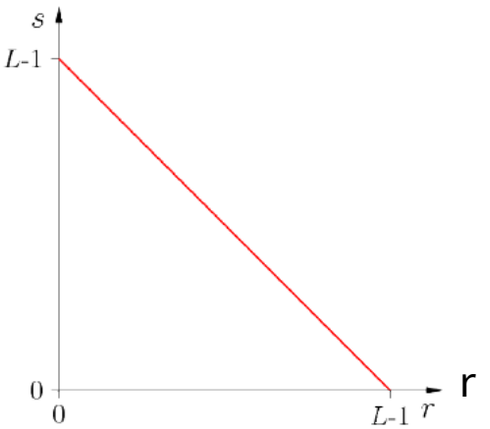


image negative

$S = T(k) = L - 1 - k$





Histogram

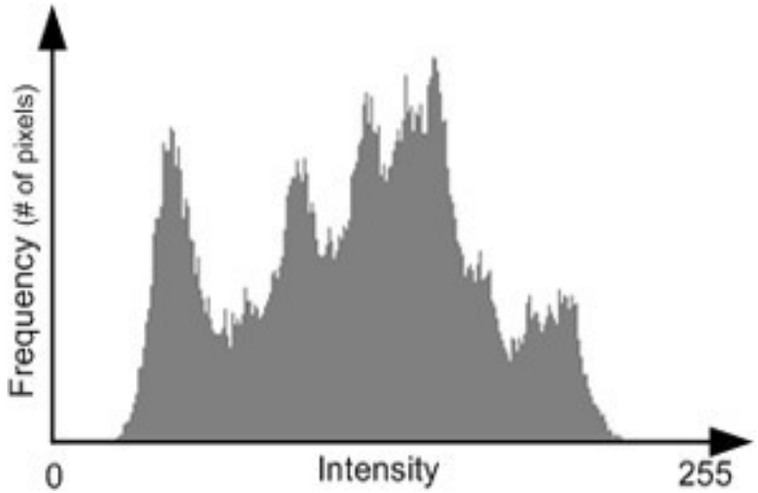
- a histogram represents the distribution of (graylevel) intensities in the image
  - the most important basic tool for image processing
- the histogram of an L-valued image is a discrete function

$$h(k) = n_k\,, \qquad k \in [0, \dots, L-1]$$

  - where n_k is the number of pixels with intensity k
  - often it is preferable to consider the histogram normalized with respect to the number of pixels

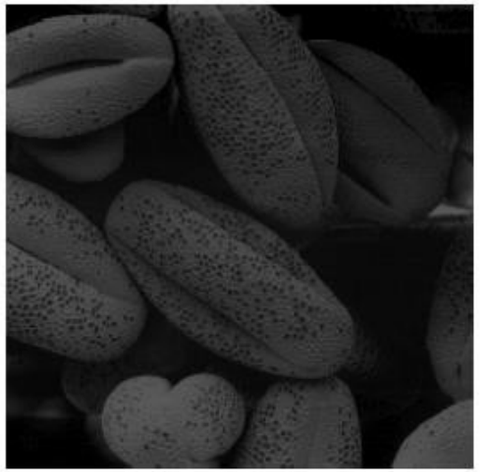
$$p(k) = \frac{n_k}{mn}$$

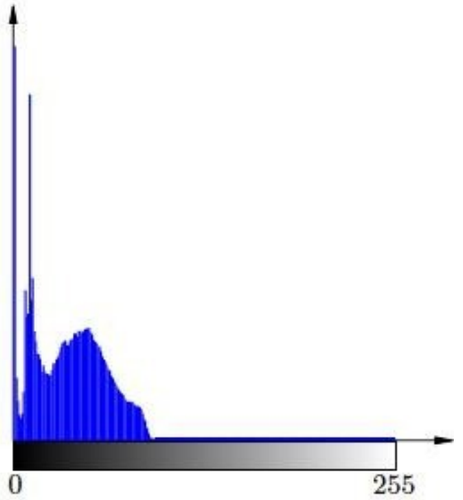
    - $p(k)$ estimates the probability density of k:
    - $m \times n$ image size

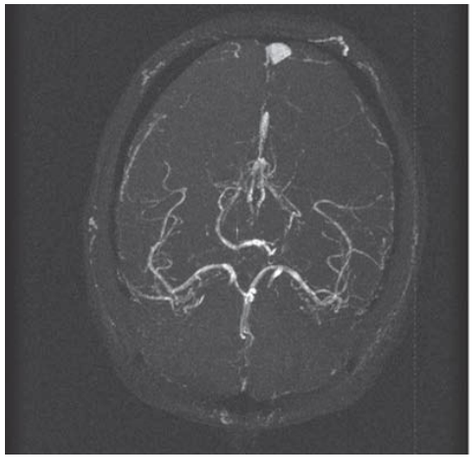


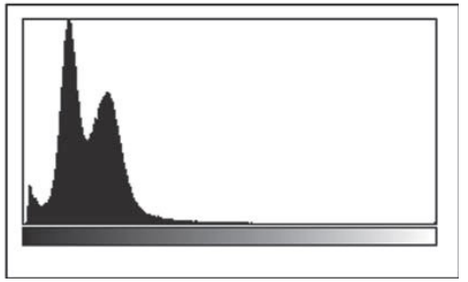
Low contrast image

- histogram components are localized in a narrow region of the ir

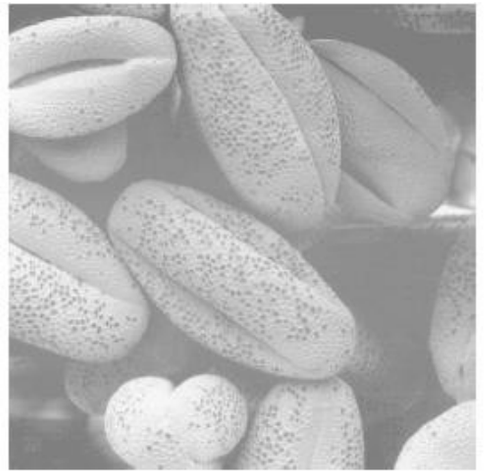


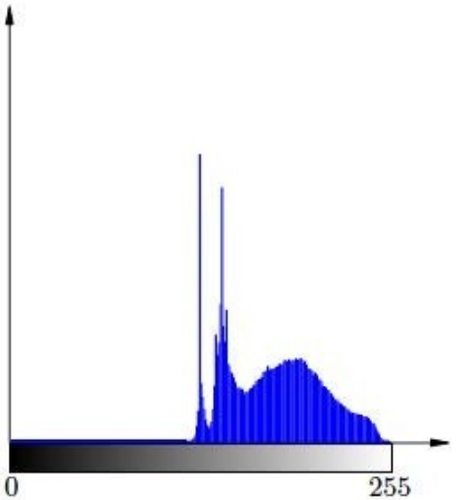


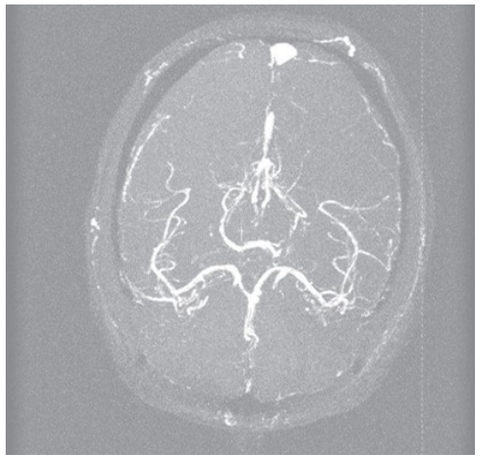


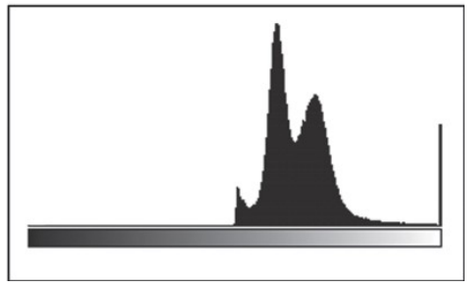


Dark image





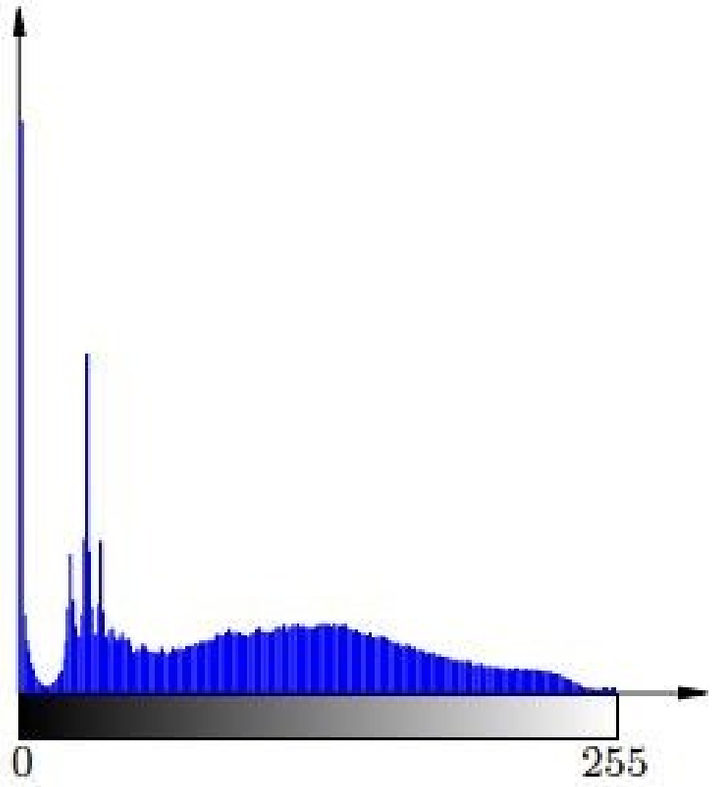




Bright

High contrast image

- the histogram components are distributed over all the intensity range
  - this is the effect pursued by the histogram based intensity trans

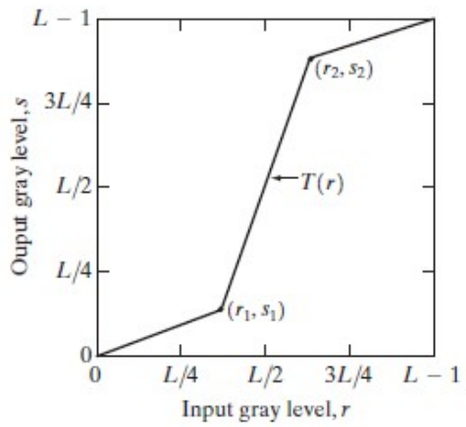


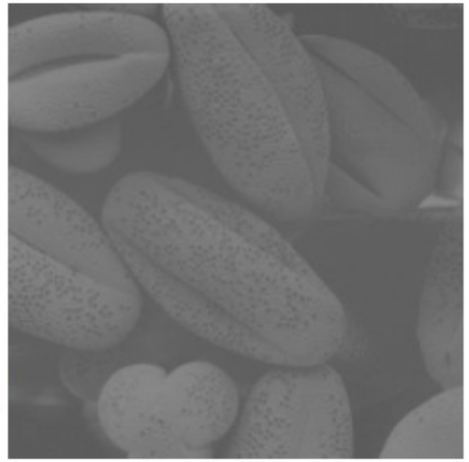
Contrast Enhancement

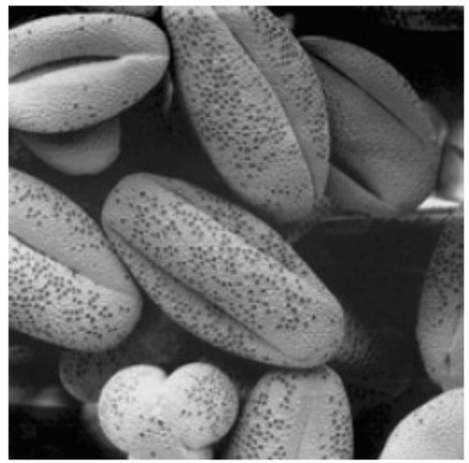
Contrast stretching

- ***Contrast stretching*** is a process that expands the range of intensity levels in an image
  - so that it spans the full intensity range of the recording medium or display device

Contrast-stretching transformations increase the contrast between the darks and the lights







a b
c d

FIGURE 3.10 Contrast stretching. (a) Form of transformation function. (b) A low-contrast image. (c) Result of contrast stretching. (d) Result of thresholding. (Original image courtesy of Dr. Roger Heady

Contrast enhancement

- use a pointwise transformation T(k) to generate a new image g from a given image f such that

$$g(i, j) = T(f(i, j))$$

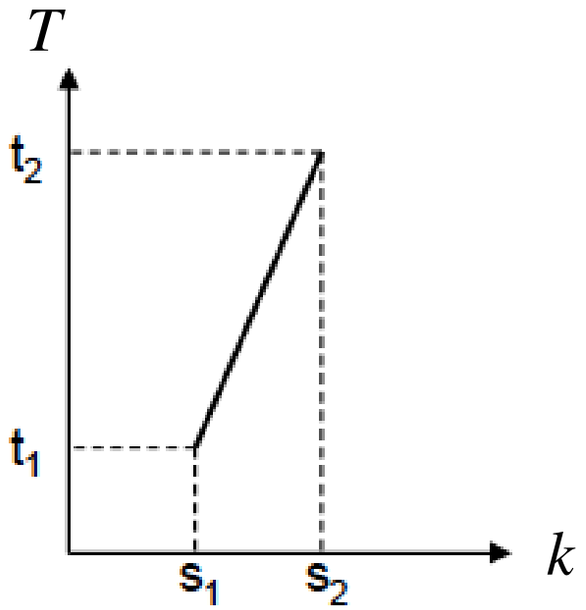
- properties that T(k) must satisfy
  - monotonically non-decreasing, so that relative brightness of pixels do not change
  - bit depth preserving, i.e. g has the same bit depth of f
  - integer (rounding/truncation may be needed)

- different possibilities
  - linear functions
  - piece-wise linear functions
  - non-linear functions
  - histogram equalization

Linear stretching

- $T(k) = ak + b$
- enhances the dynamic range by linearly stretching the original gray levels
- if the original graylevels are in $[s_1, s_2]$ and the target graylevels are $[t_1, t_2]$, then

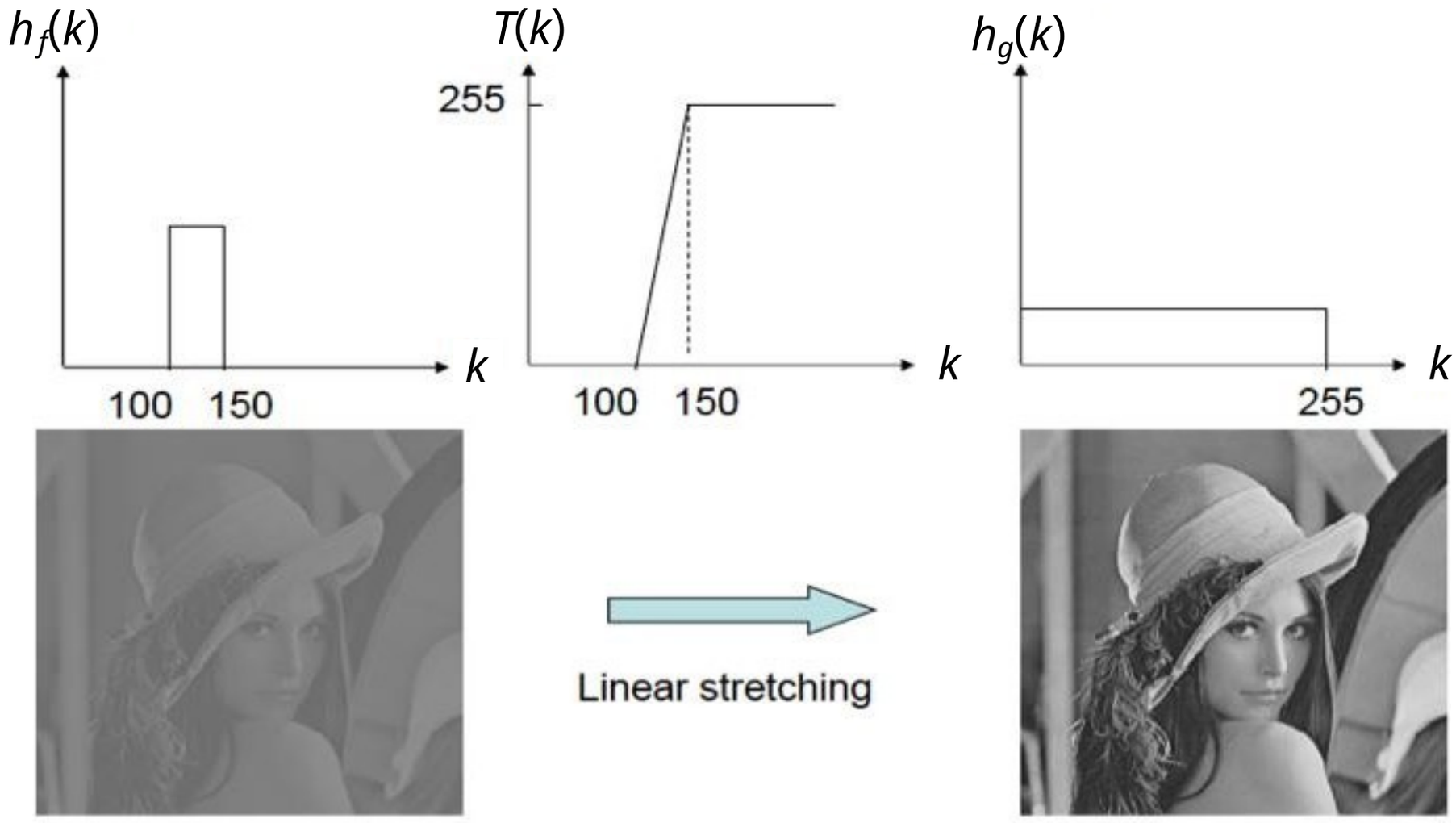
$$T(k) = (k - s_1) \cdot \frac{(t_2 - t_1)}{(s_2 - s_1)} + t_1$$



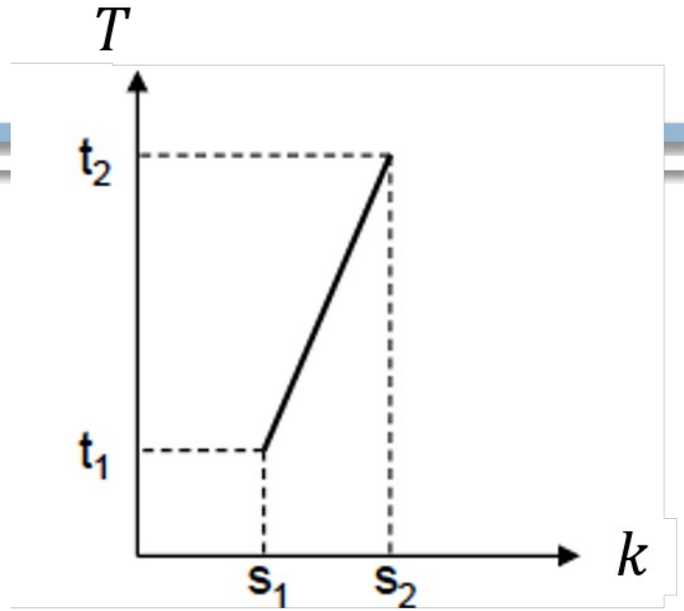
- example
  - the original graylevels are in [ ] 100, 150
  - the target graylevels are in [0, 255]
  - $T(k) = \frac{k-100}{50} \cdot 255$

in this case we refer to the linear stretching transformation as *image normalization*

Linear stretching



Linear stretching



- When s1 = t1 and s2 = t2, transformation becomes a Linear function.
- When s1 = s2, t1 = 0 and t2 = L-1, transformation becomes a thresholding function.
- When (s1, t1) = (s_{min}, 0) and (s2, t2) = (s_{max}, L-1), this is known as Min-Max Stretching.
  - Lower value of the input image is mapped to 0 and the upper value is mapped to 255.
  - All other intermediate values are reassigned new intensity values according to the following formulae

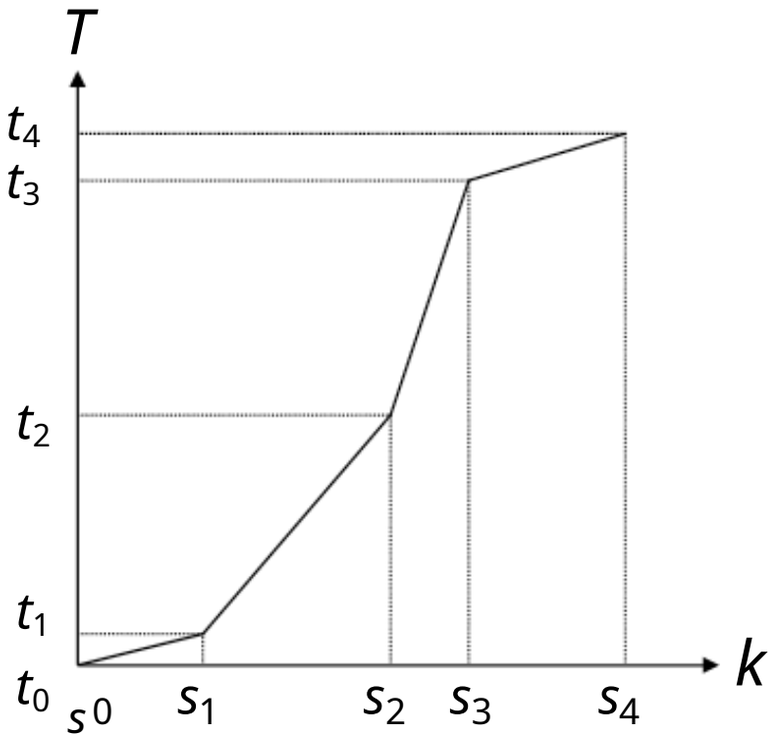
$$Xnew = \frac{Xinput - Xmin}{Xmax - Xmin} \times 255$$

Piecewise linear stretching

- N segments for input and output

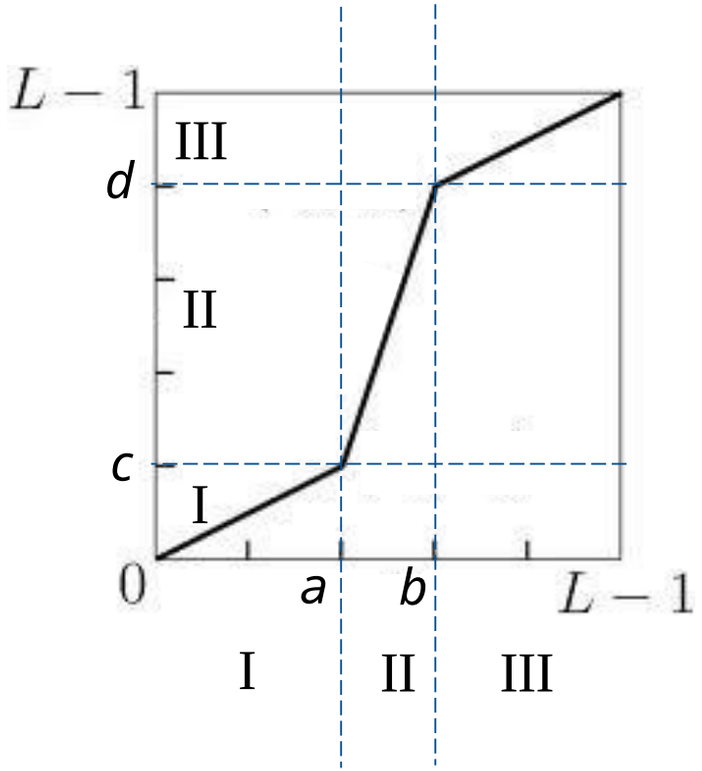
$$T(k) \quad \left(k - s_j\right) \frac{\left(t_{j+1} - t_j\right)}{\left(s_{j+1} - s_j\right)} + t_j$$

- for $s_j < k \leq s_{j+1}$
- $j = 0,1, \ldots, N-1$



Piecewise linear stretching with $N = 3$

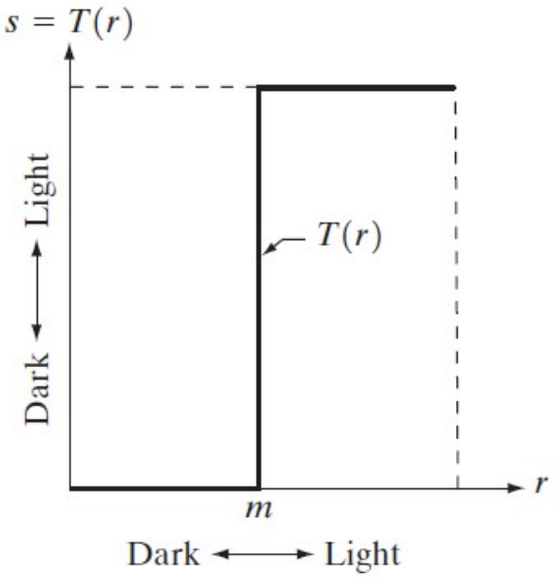
- segment I
  - slope < 45°
  - input intensities are compressed in the output
- segment II
  - slope > 45°
  - input intensities are stretched in the output
- segment III
  - slope < 45°
  - input intensities are compressed in the output

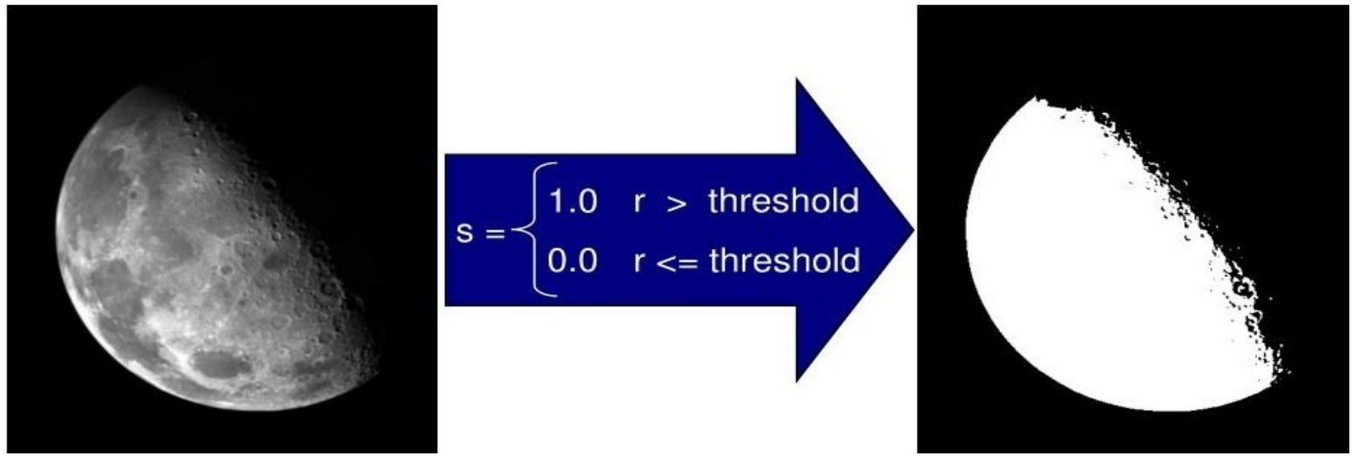


- points a and b can be determined using percentiles (e.g. 5% and 95%) of the input image histogram $h(k)$
  - ith percentile $\stackrel{\text{def}}{=}$ smallest k': $\sum_{k \le k'} h(k) \ge i.\frac{mn}{100}$

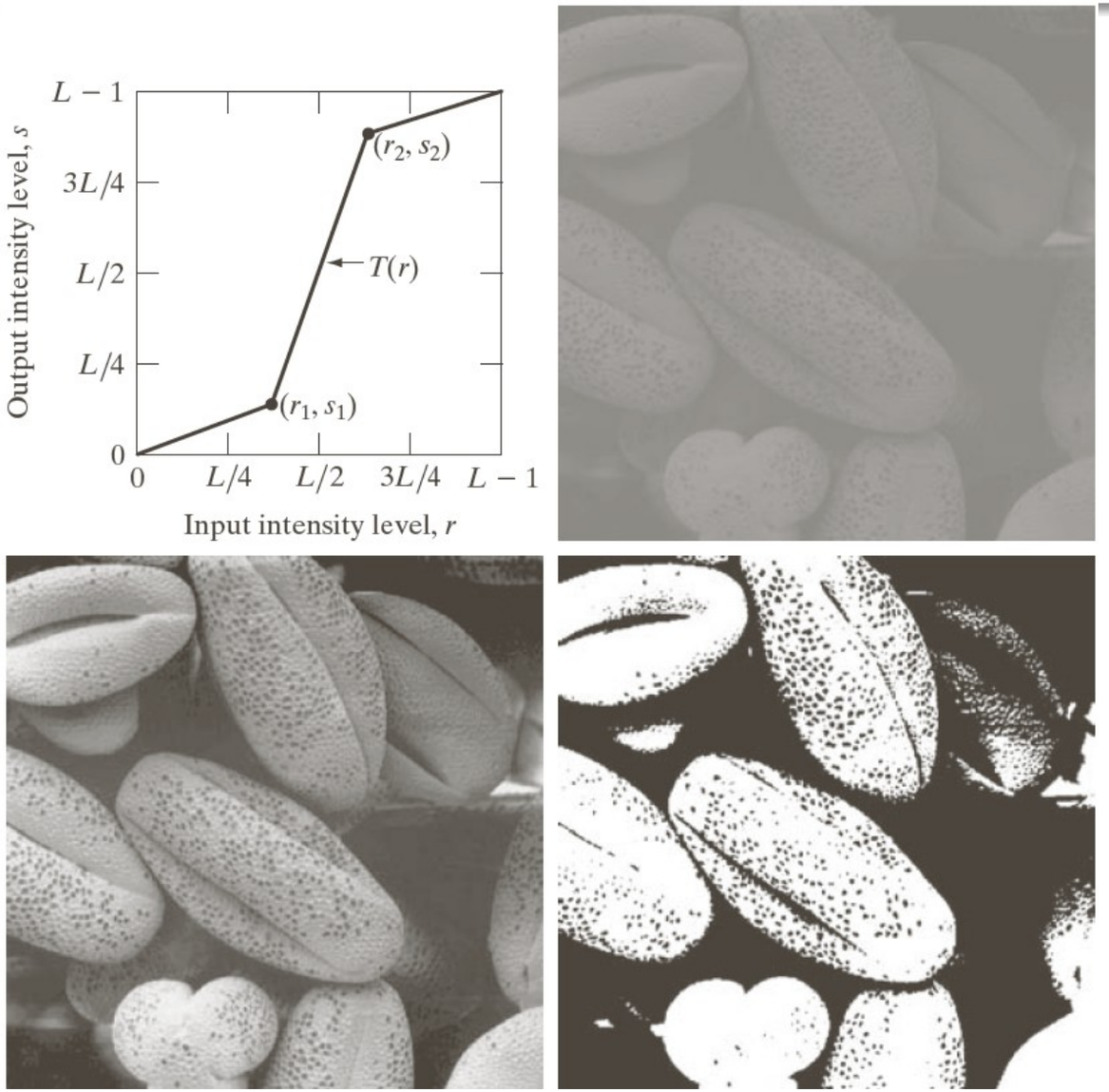
Thresholding function

- Thresholding transformations are particularly useful for segmentation in which we want to isolate an object of interest from a background





Piecewise linear stretching



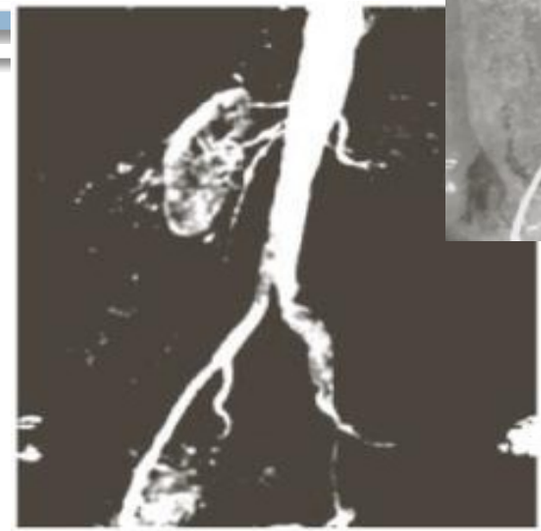
a b
c d

FIGURE 3.10
Contrast stretching. (a) Form of transformation function. (b) A low-contrast image. (c) Result of contrast stretching. (d) Result of thresholding. (Original image courtesy of Dr. Roger Heady, Research School of Biological Sciences, Australian National University, Canberra, Australia.)

Piecewise linear stretching (special case)

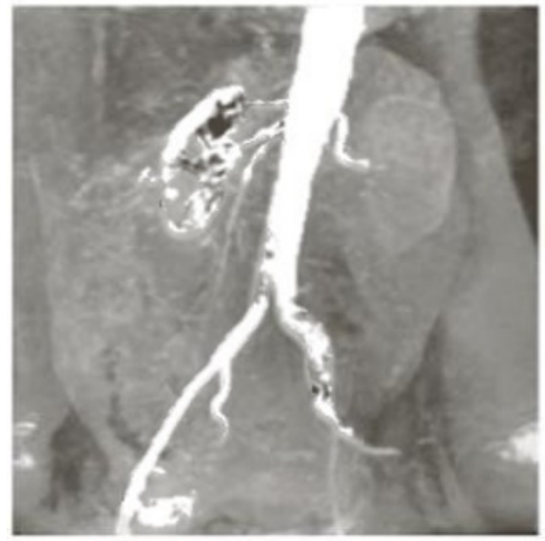
intensity level slicing

- highlights a specific intensity range
  - Enhances feature such as masses of water in satellite imagery and flaws in X-ray images.



(a)

- both transformations in (a) and (b) set the intensities in $[A, B]$ to a high value
- the transformation in (a) sets all the intensities that are not in $[A, B]$ to a low value
- the transformation in (b) preserves the intensities that are not in $[A, B]$



(b)

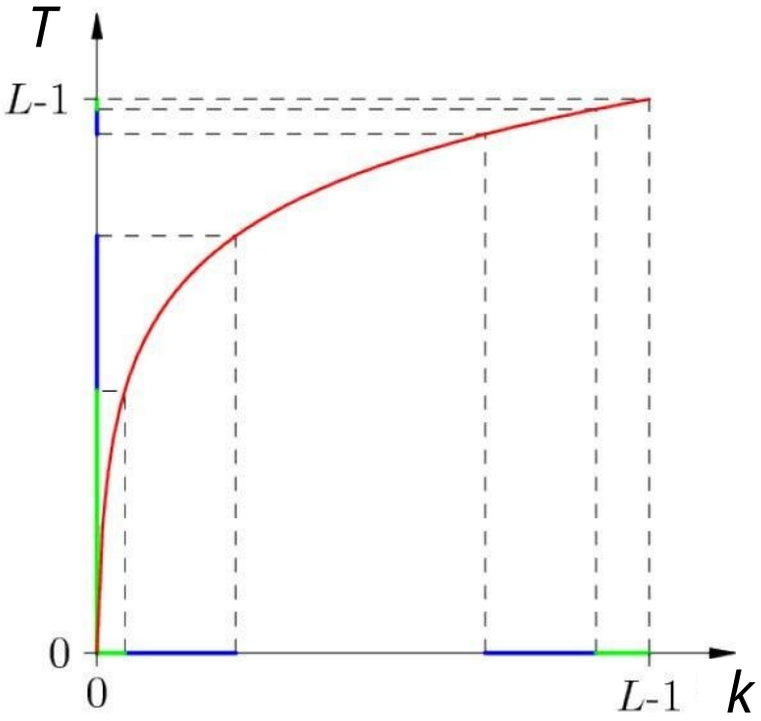
Logarithmic transformations

- nonlinear
- $T(k) = c \cdot \log(1 + k)$

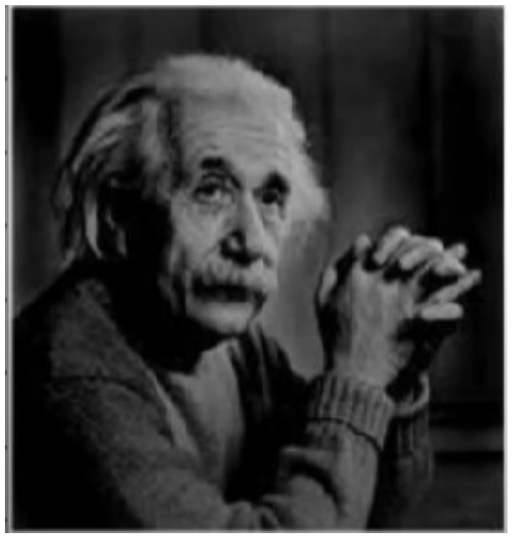
$c = \frac{L-1}{\log L}$ to preserve the original intensity range $[0, L-1]$

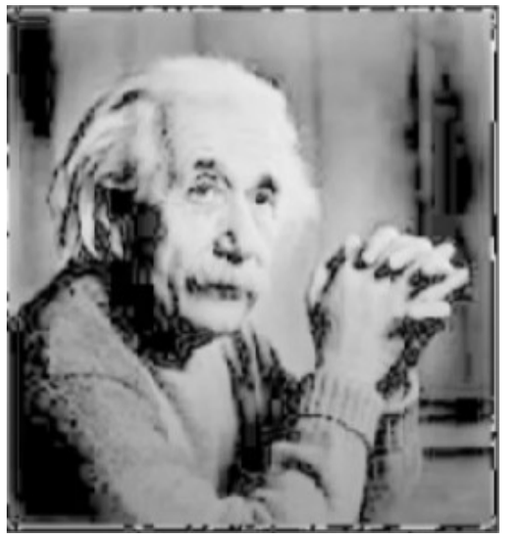
$c = \frac{255}{8}$ for grayscale image

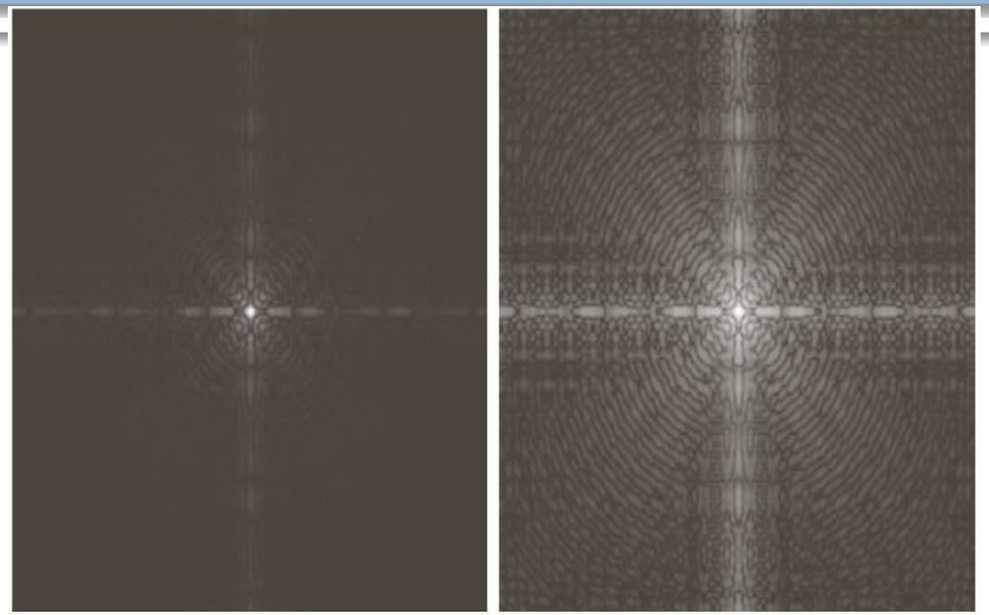
- useful to enhance dark images
  - low intensity ranges are expanded
  - high intensity ranges are compressed



Logarithmic transformations (example)





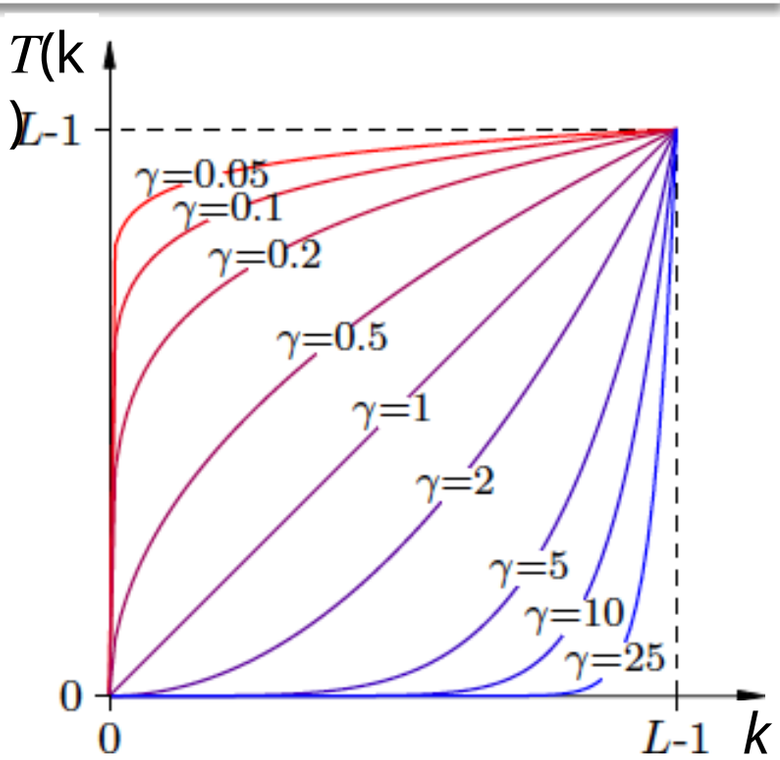


origin
al

log-
transformed

Gamma transformations

- also called power-law transformations
- $T(k) = c \cdot (k^{\gamma} - 1)^{1-\gamma}$ in order to preserve the original intensity range $[0, L-1]$
- used to correct the visualization device's output
  - thus the naming "*gamma correction*"
- a too large or too small value for γ can compromise the results
  - $\gamma = 1 \rightarrow$ identity
  - $\gamma < 1 \rightarrow$ brightening
  - $\gamma > 1 \rightarrow$ darkening
    - monotonic, reversible
    - *compress* or *stretch* certain range of gray-levels



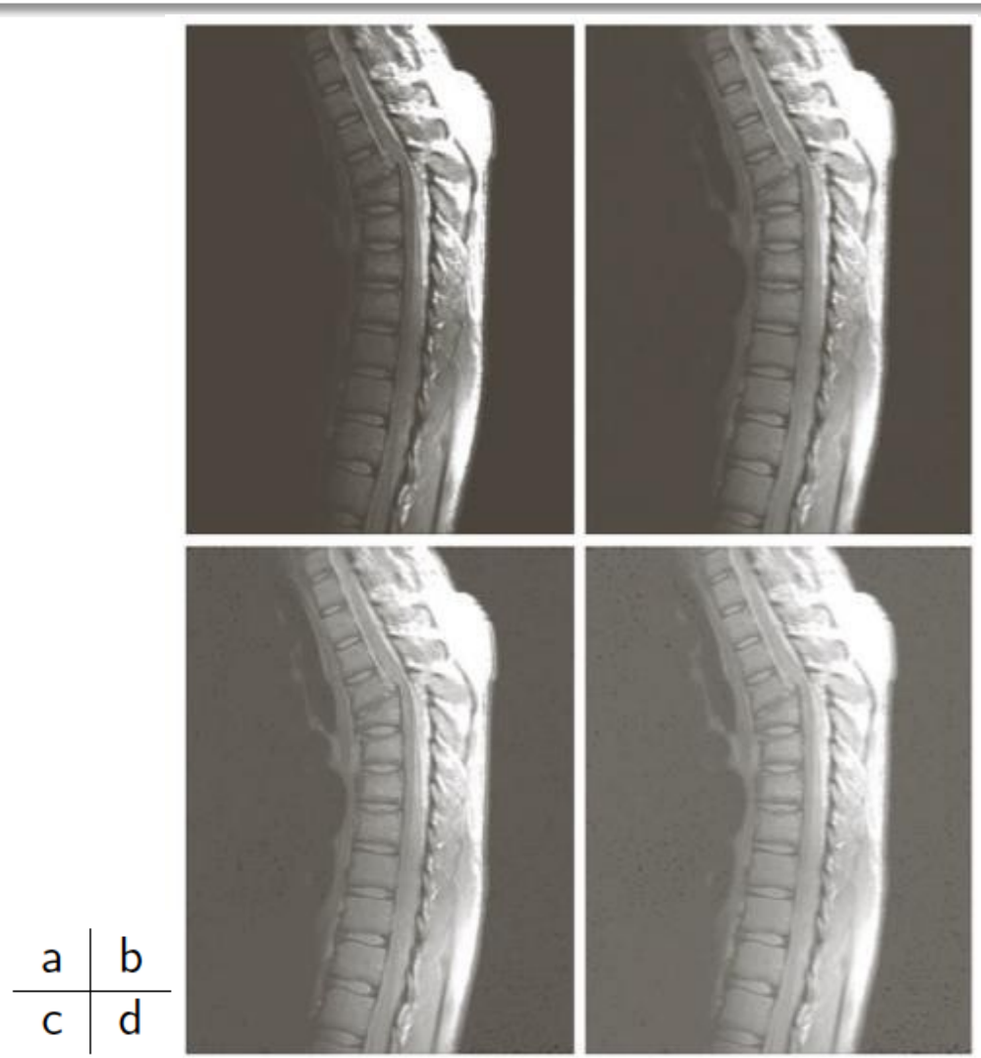
Gamma transformations (brightening)

a) original image

b) gamma-transformed with γ = 0.6

c) gamma-transformed with γ = 0.4

d) gamma-transformed with γ = 0.3



a b
c d

Gamma transformations (darkening)

a) original image

b) gamma-transformed with $\gamma = 3$

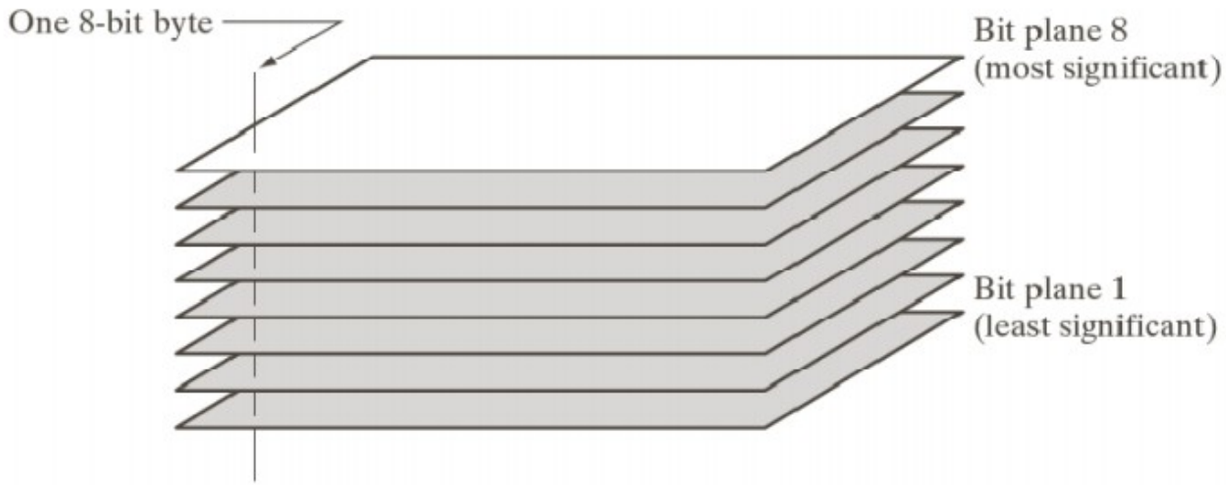
c) gamma-transformed with $\gamma = 4$

d) gamma-transformed with $\gamma = 5$

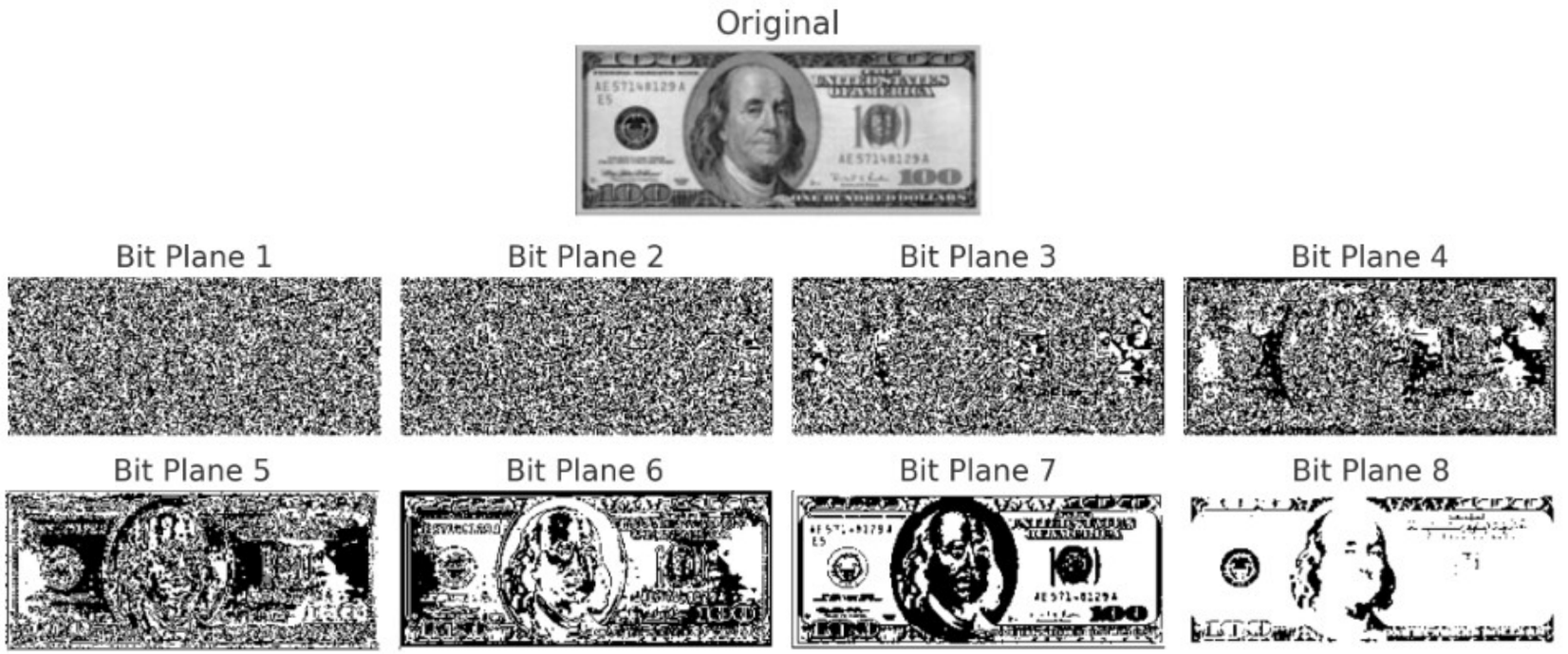
a	b
c	d

Bit Plane Slicing

- Highlight the contribution made to total image appearance by specific bits.
- An 8-bit image may be considered as being composed of eight 1-bit planes with plane 1 containing the lowest-order bit of all pixels in the image and plane 8 all the highest order bits.



Bit Plane Slicing



- Observe that the four higher-order bit planes, especially the last row, contain significant amount of the visually significant data.
- The higher-order planes contribute to more suitable intensity details in the image.

Reconstruction

- Four highest order bit planes would allow us to reconstruct the original image in acceptable details.
- Four planes instead of the original image require 50% less storage.

a b c

FIGURE 3.15 Images reconstructed using (a) bit planes 8 and 7; (b) bit planes 8, 7, and 6; and (c) bit planes 8, 7, 6, and 5. Compare (c) with Fig. 3.14(a).

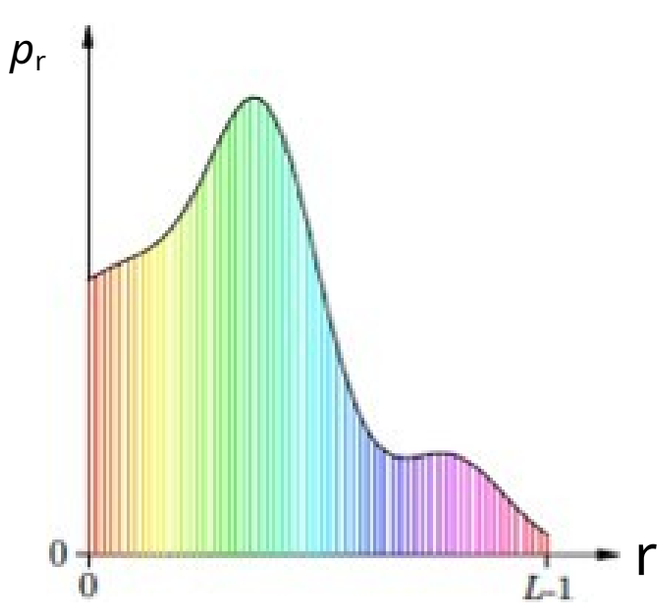
Histogram Processing

Histogram Equalization

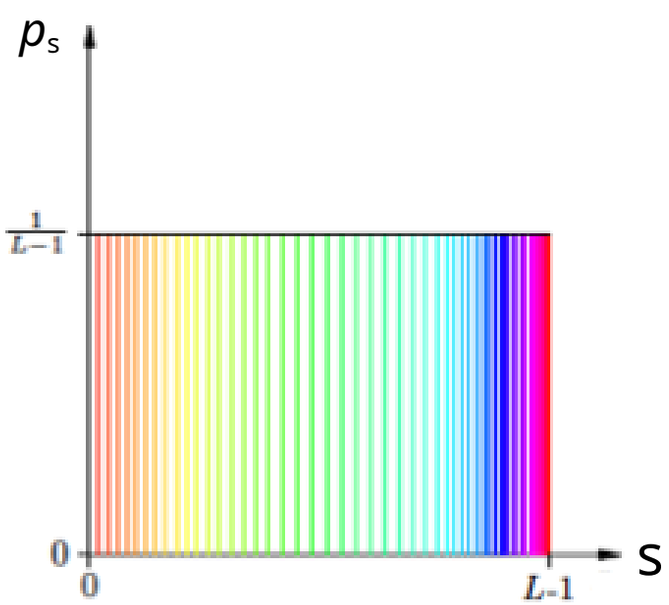
- Histogram equalization can be used to improve the visual appearance of an image.
- ***Histogram equalization*** automatically determines a transformation function that produces an output image that has a
  - uniform histogram (continuous case)
  - near uniform histogram (discrete case)

Histogram Equalization

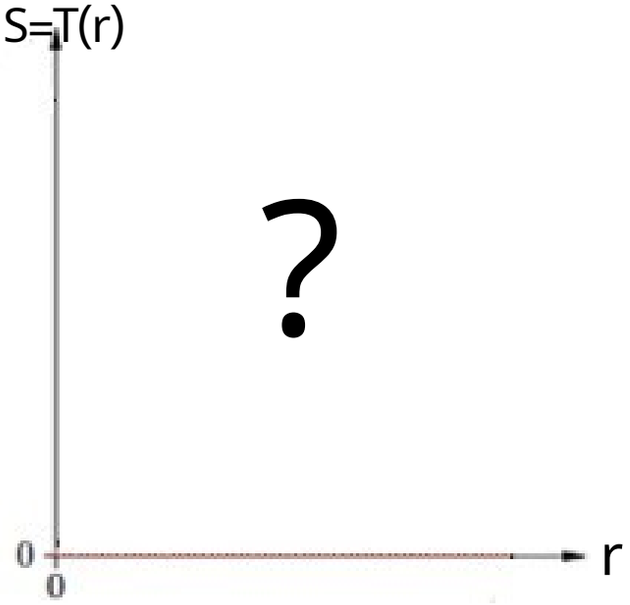
- *goal*: given the p.d.f. p_r of an input image f, we want to apply an intensity transformation T such that the output image g has uniform p.d.f. $p_s = \frac{1}{L-1}$
- *solution*: solve for $T(r)$ in the continuous domain ($r \rightarrow X$) and then obtain the discrete approximation $T(r)$



input



desired output



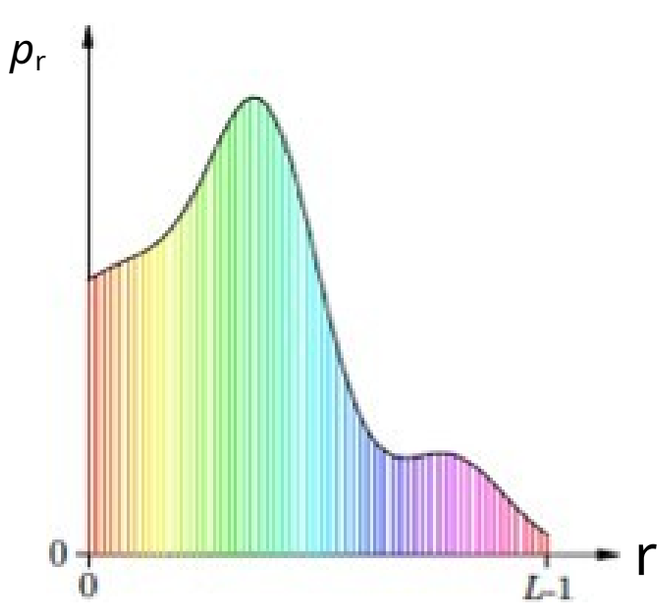
solution

HE: Solution

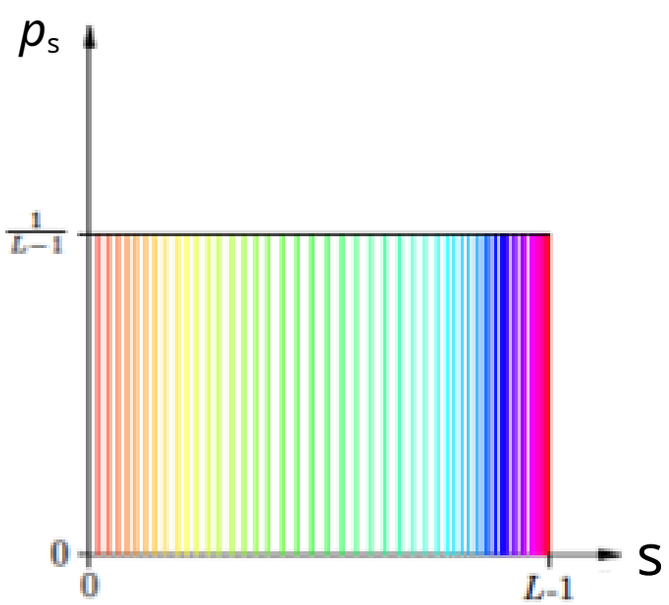
- *goal*: given the p.d.f. p_r of an input image f, we want to apply an intensity transformation T such that the output image g has uniform p.d.f. $p_s = \frac{1}{L-1}$

CDF

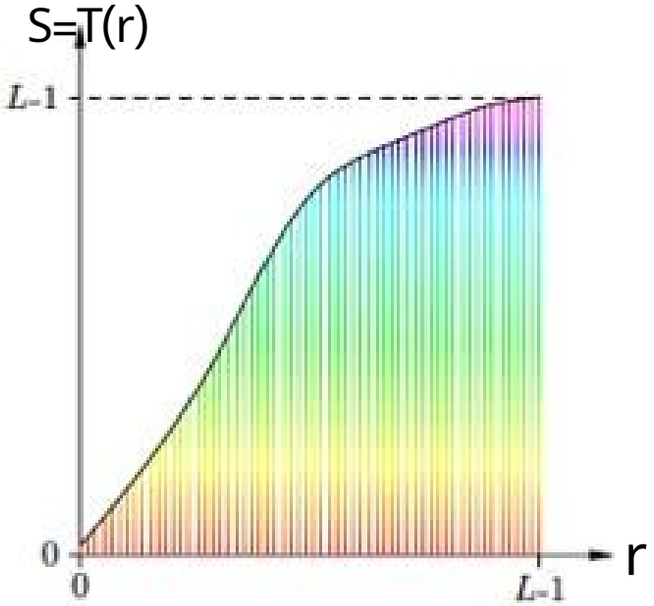
- *solution*: use $T = (L - 1)F_r$ (the $L - 1$ term preserves the intensity range)



input



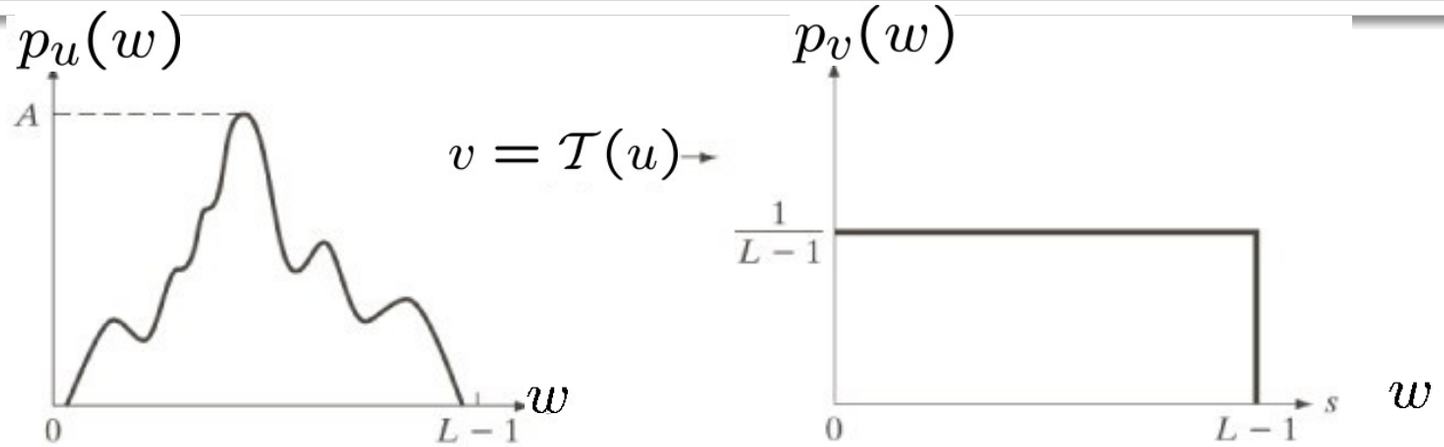
desired output



solution

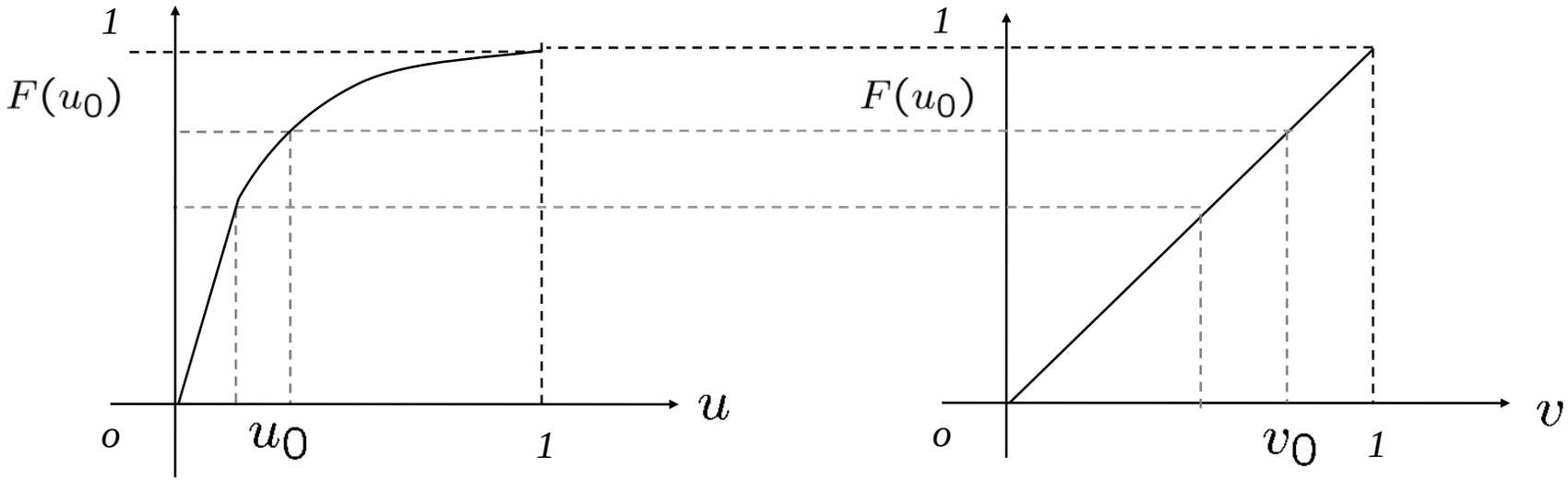
Histogram Equalization

pdf



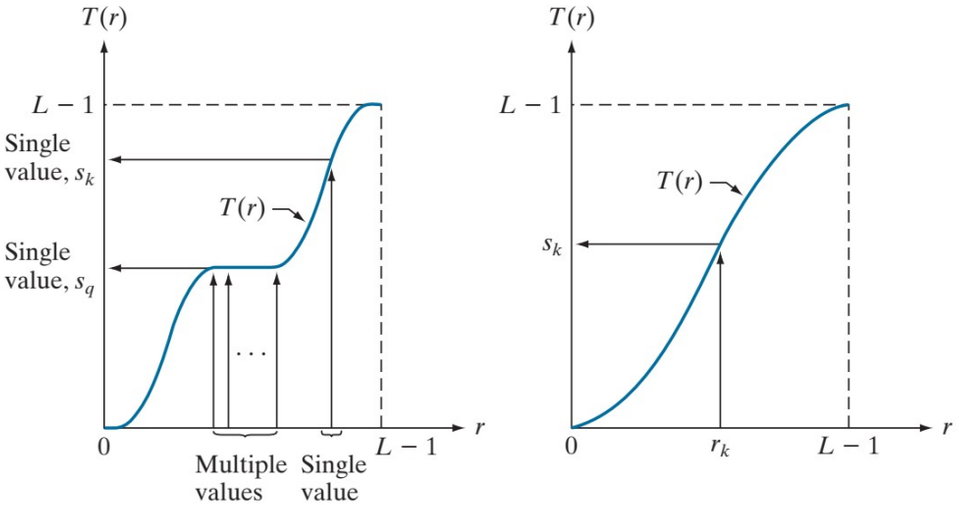
$$F_u(U) = P(U < u) = \int_0^u p_u(w)dw \qquad F_v(V) = P(V < v) = \int_0^v p_v(w)dw$$

cdf



Histogram Equalization

- Suppose an image has continuous gray levels $r \in [0, L-1]$ where
  - r= input (original) gray level
  - L= total number of intensity levels (typically 256)
- For mathematical convenience, normalize the intensity: $r \in [0, 1]$
- The goal is to find a transformation $s = T(r)$ such that the output gray levels $s$are **uniformly distributed** over $[0, 1]$.
- A uniform histogram means $p_s(s) = 1, 0 \leq s \leq 1$ where
  - $p_r(r)$= PDF of original image
  - $p_s(s)$= PDF after transformation



- Assume $s = T(r)$, where $T(r)$is
  - Continuous, monotonic increasing
  - differentiable
- Monotonicity guarantees that the order of pixel intensities is preserved.

Histogram Equalization

- **Probability Conservation**
- The probability contained in corresponding intervals must remain unchanged.
- For a small interval,

$$P(r \le R \le r + dr) = P(s \le S \le s + ds)$$

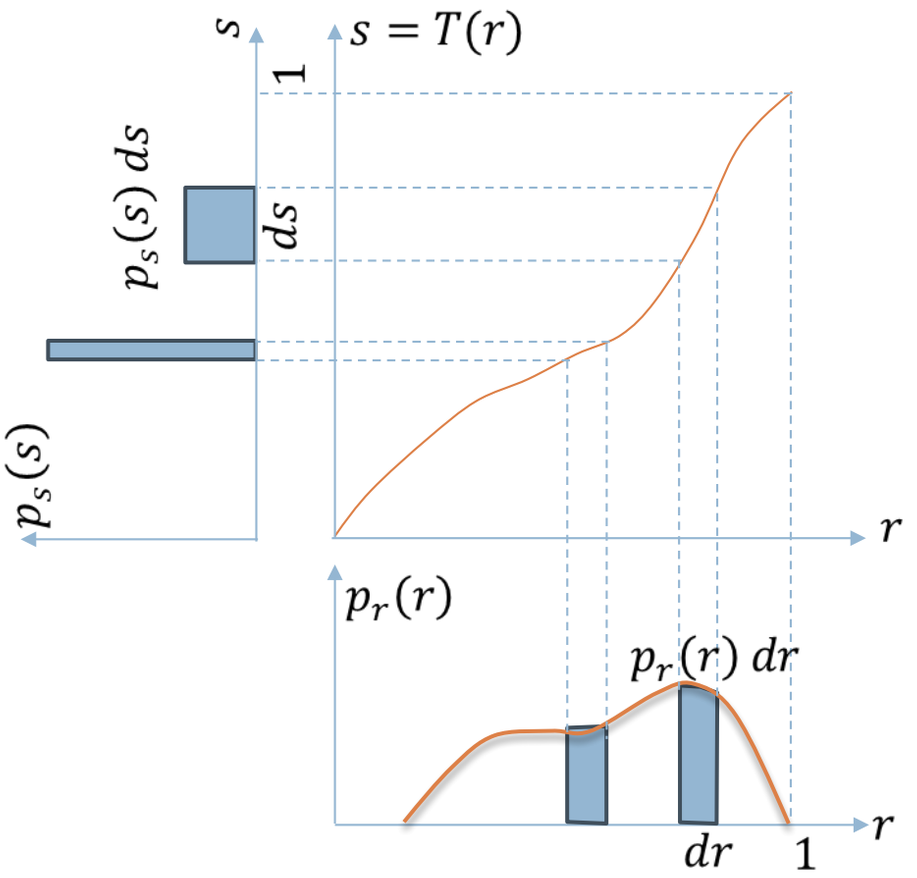
$$p_r(r)\, dr = p_s(s)\, ds$$

$$p_s(s) = p_r(r) \left|\frac{dr}{ds}\right|$$

- Since $T(r)$is increasing, $\frac{dr}{ds} > 0$
- thus

$$p_s(s) = p_r(r) \frac{dr}{ds}$$

- This is the standard **change-of-variable theorem** in probability.



Histogram Equalization

- We want $p_s(s) = 1$, Substitute into the previous equation:

$$1 = p_r(r)\frac{dr}{ds}$$

$$\frac{ds}{dr} = p_r(r) \text{ or } \quad ds = p_r(r)dr$$

- This differential equation tells us exactly how the transformation must change with intensity.
- Integrate:

$$\int_0^s d\,s = \int_0^r p_r\,(r)dr$$

$$s = \boxed{T(r) = \int_0^r p_r\,(r)\,dr}$$

- This is the histogram equalization transformation.
- the integral of a PDF from 0 to r is, by definition, the Cumulative Distribution Function (CDF) of r

Histogram Equalization - Discrete case

- Real images contain discrete gray levels $r_0, r_1, \ldots, r_{L-1}$
  - n_k = number of pixels having gray level r_k
  - N = total pixels
- The probability mass function is $p_r(r_k) = \frac{n_k}{N}$
- The discrete CDF becomes $\boxed{T(r_k) = \sum_{j=0}^{k} \frac{n_j}{N}}$
- Since gray levels must remain integers,

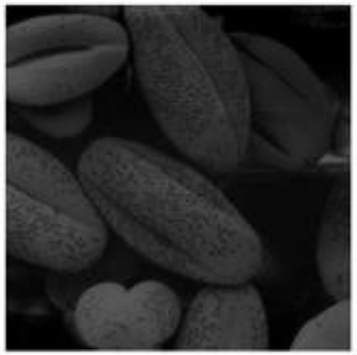
$$s_k = T(r_k) = (L-1)\sum_{j=0}^{k} p_r(r_j) = \frac{L-1}{N}\sum_{j=0}^{k} n_j \qquad 0 \le k \le L-1$$

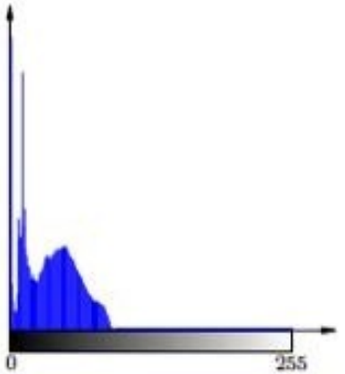
$$\boxed{s_k = (L-1) \times CDF(r_k)}$$

- After rounding, $s_k = \text{round}[(L-1)\, CDF(r_k)]$.
- The factor (L - 1) simply scales the normalized [0, 1] output back to the standard pixel range (like 255)

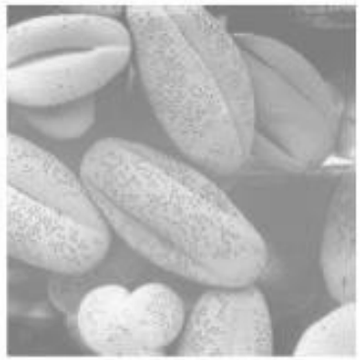
Example: HE

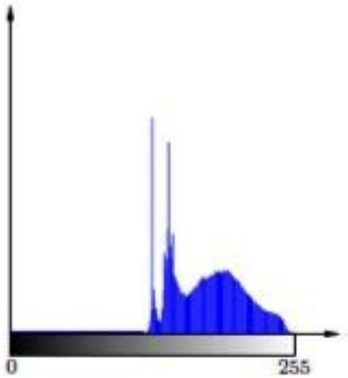
dark image

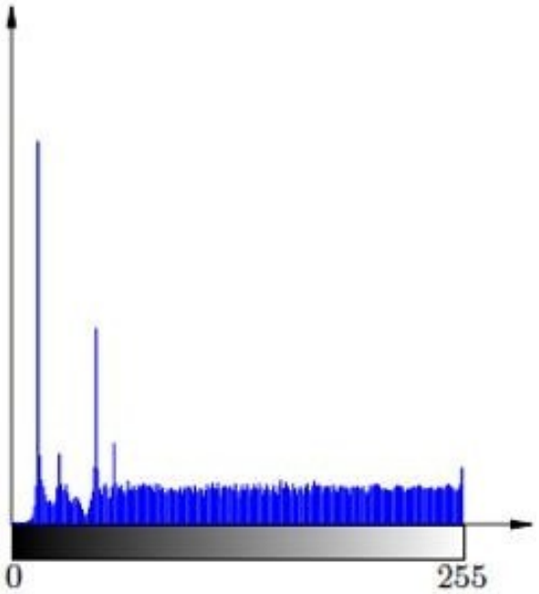


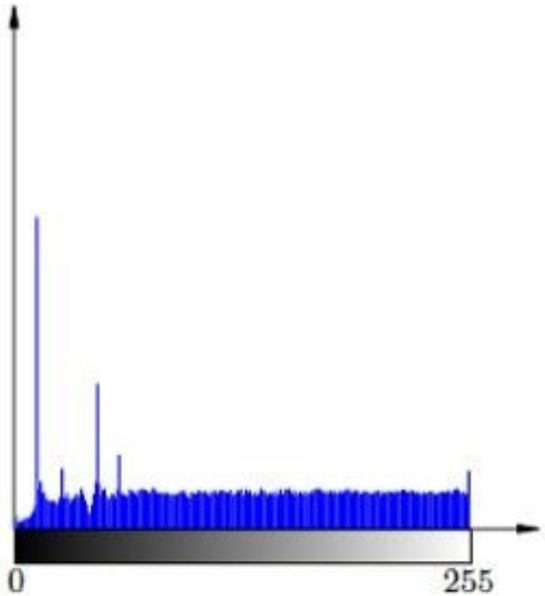


Bright image



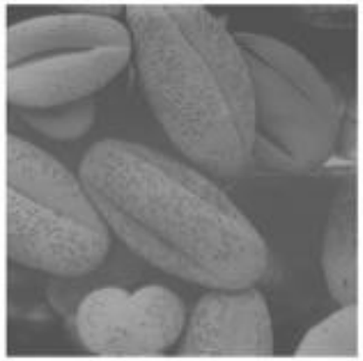


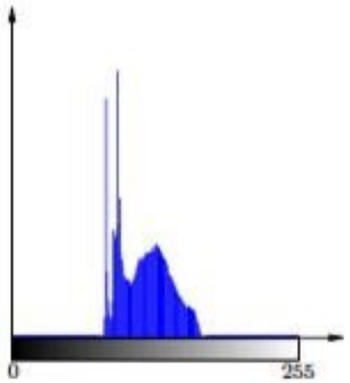




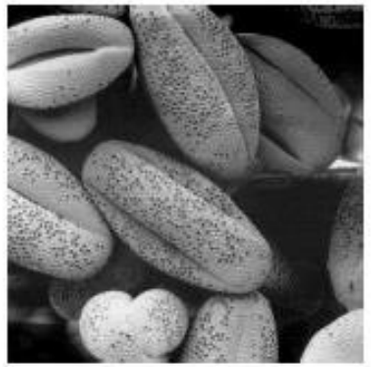
Example: HE

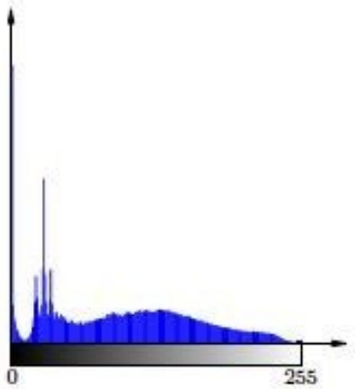
Low contrast

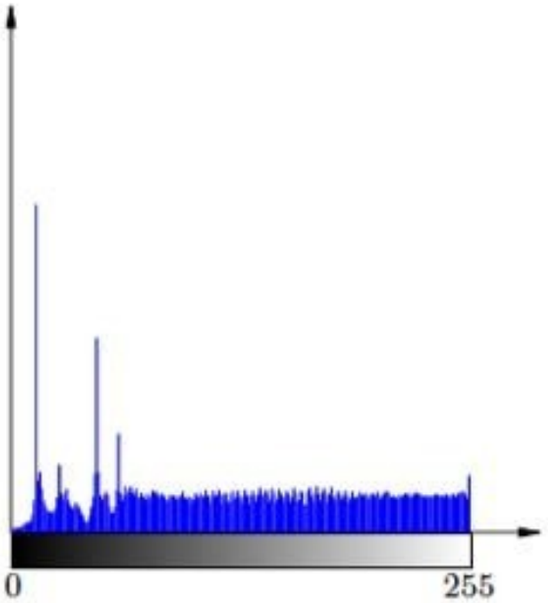


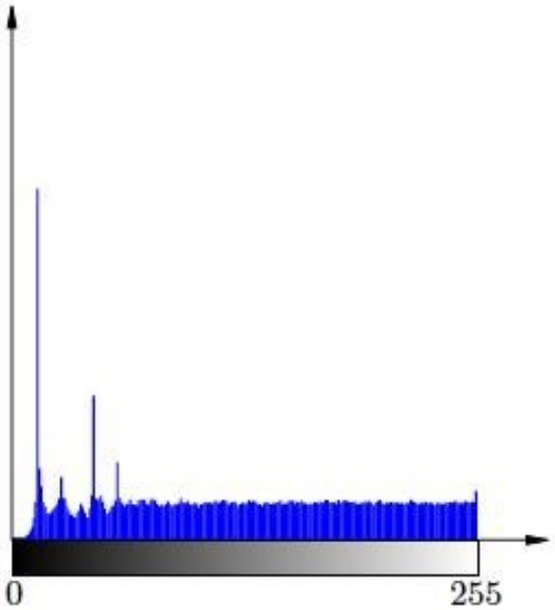


High contrast





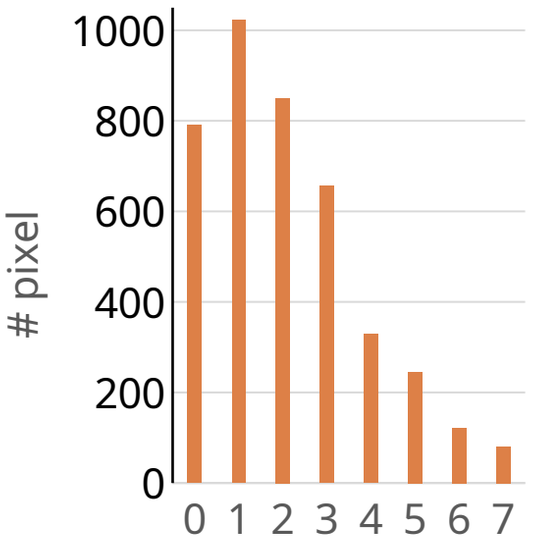


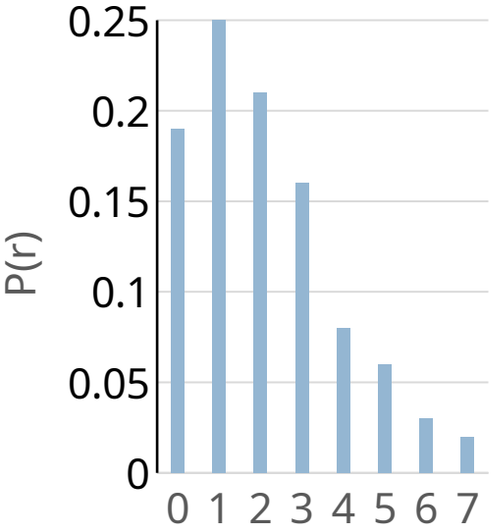


Histogram Equalization – Numerical Example

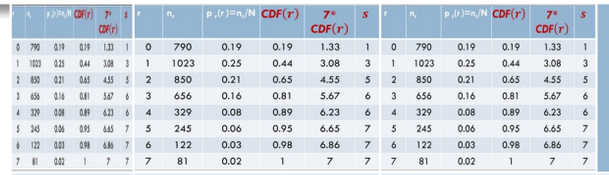
- Let f be an image with size 64x64 pixels and L=8 and let f has the intensity distribution as shown in the table

$p_r(r)=n_r/N$	n_r	r
0.19	790	0
0.25	1023	1
0.21	850	2
0.16	656	3
0.08	329	4
0.06	245	5
0.03	122	6
0.02	81	7

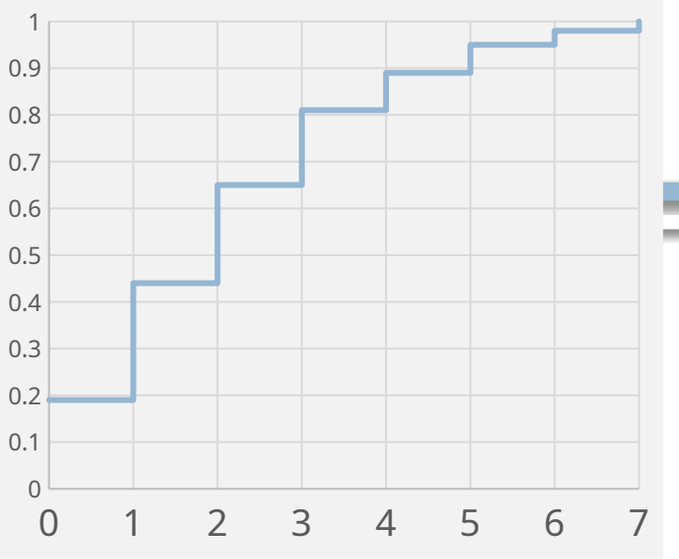


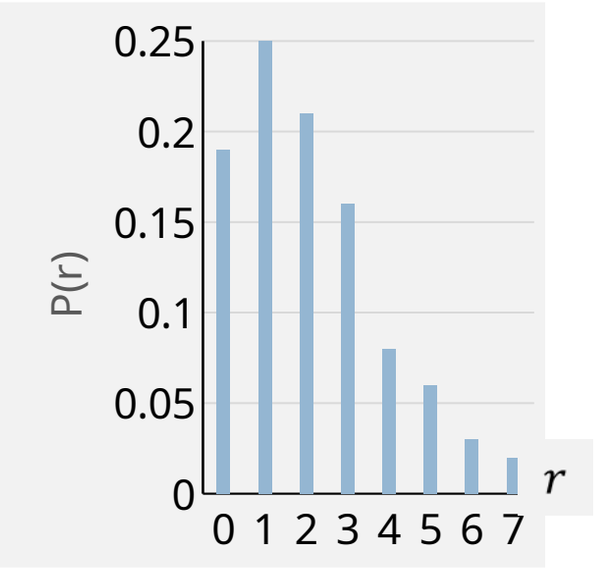


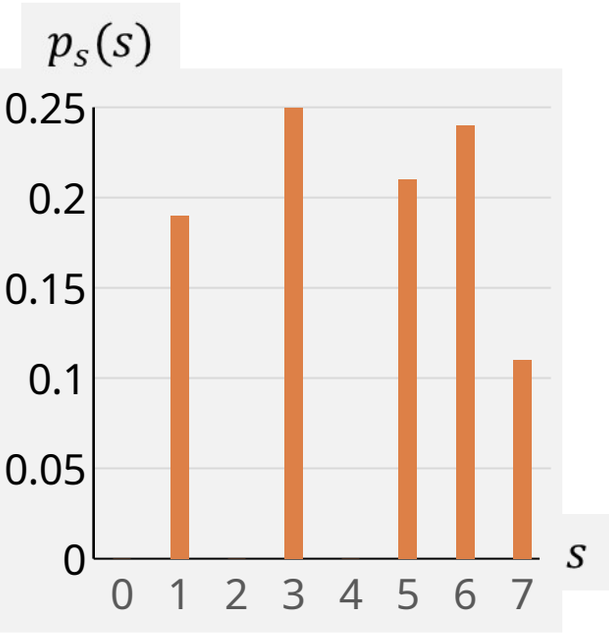
Histogram Equalization - Example



	$p_r(r)=n_r/N$	CDF(r)	7* CDF(r)	s	n_r	r
1	1.33	0.19	0.19	1	790	0
3	3.08	0.44	0.25	3	1023	1
5	4.55	0.65	0.21	5	850	2
6	5.67	0.81	0.16	6	656	3
6	6.23	0.89	0.08	6	329	4
7	6.65	0.95	0.06	7	245	5
7	6.86	0.98	0.03	7	122	6
7	7	1	0.02	7	81	7

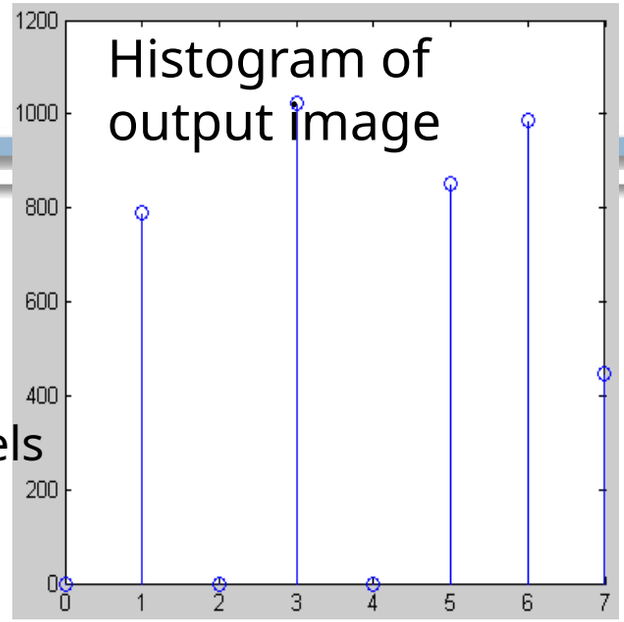






Discrete HE - Issues

- discrete histogram equalization almost **never** results in a perfectly flat, uniform histogram. Instead, you usually get a "comb-like" histogram with tall spikes and empty gaps.
- No Pixel Splitting Rule
  - Suppose image has 100 pixels and 8 gray levels
  - A perfectly uniform histogram would require $\frac{100}{8} = 12.5$ pixels per gray level.
  - But pixels must be integers.
  - Possible distributions are $13,13,12,12,13,12,13,12$ or something similar.
  - Even if the mapping were perfect, exact uniformity is mathematically impossible.



Discrete HE - Issues

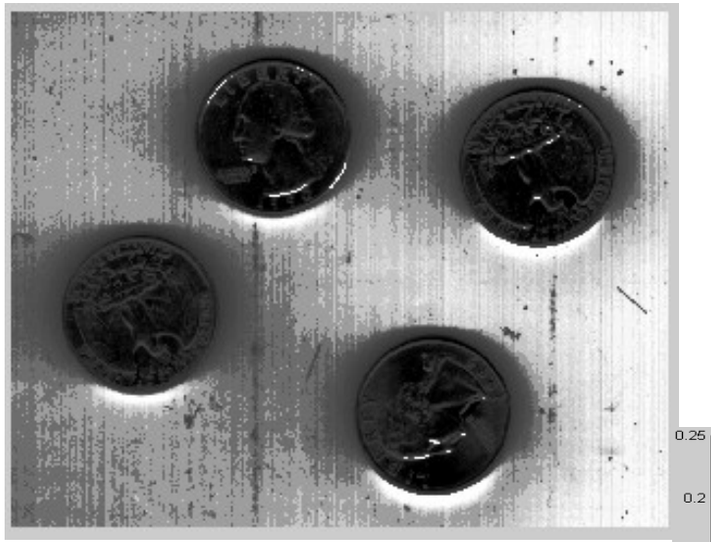
- **Quantization (Rounding Error)**
  - The mapping is actually $s_k = \text{round}\big((L-1) \times CDF(r_k)\big)$.
  - Several different CDF values may round to the same output level.
- This rounding causes two distinct phenomena:
- **Bin Merging (Spikes):** If two different original intensity levels are close together, their cumulative probabilities might be so similar that they both round to the *same* integer output. Two previously separate bins crash into one, creating a massive spike in the new histogram.
- **Bin Stretching (Gaps):** If a single bin contains a massive percentage of the image's pixels, the CDF jumps drastically at that point. When mapped, the next intensity level gets pushed far away, leaving completely empty bins (zero pixels) in between.

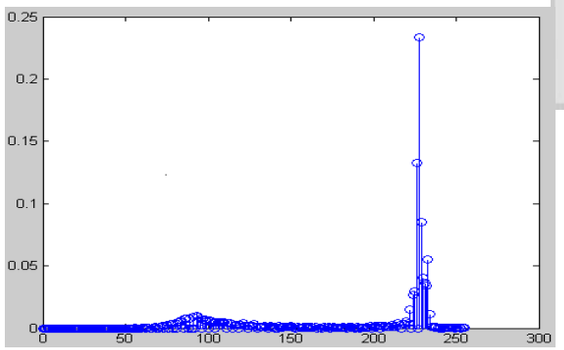
Discrete HE - Issues

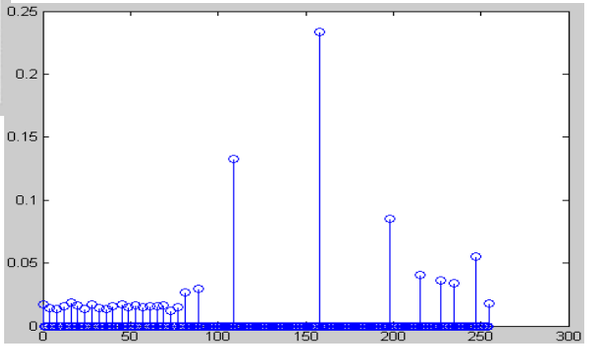
- **Empty Gray Levels Cannot Be Created**
  - Suppose an image contains only gray levels 50–80.
  - Histogram equalization **cannot invent new pixel intensities** between every adjacent level in a way that guarantees equal occupancy. It only **reassigns existing pixels.**
  - Thus there may not be enough distinct input levels to fill every output level.
- **Plateaus in the Discrete CDF**
- Continuous CDF always increases smoothly.
- Whenever a gray level has zero pixels, $n_k = 0$, the Discrete CDF does not increase.
- Consequently, multiple input gray levels produce the same CDF value and therefore the same output gray level. (many to one map)
- These flat regions (plateaus) prevent a perfectly uniform histogram.

Histogram Equalization-Comments:

- Histogram equalization may not always produce desirable results.
- It can produce false edges and regions. It can also increase image “graininess” and “patchiness.”
  - this is the case of dark/bright *objects* on homogeneous bright/dark *background*



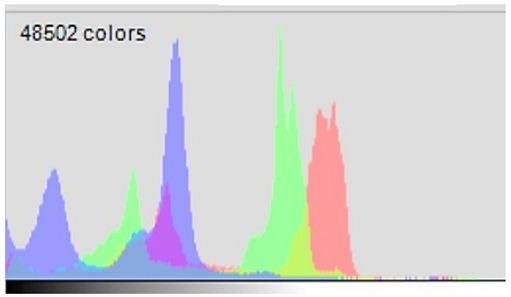




Histogram Matching

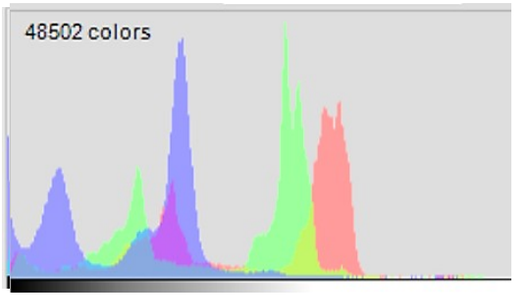
Histogram Specification (Histogram Matching)

- Histogram equalization yields an image whose pixels are (in theory) uniformly distributed among all gray levels.
- Sometimes, this may not be desirable. Instead, we may want a transformation that yields an output image with a prespecified histogram. This technique is called **histogram specificatio**n
- Goal
  - Derive a point operation that maps the input image to an output image that has the user-specified histogr



Left Image

Right Image



Images from a stereo pair of inexpensive web cams. Such cameras have different color characteristics of-the-shelf. Once can be corrected to match the other using histo.

Histogram Specification

- Given an input image with gray levels $r \in [0, L-1]$ having probability density function $p_r(r)$,
- we want to produce an output image $z = H(r)$
- whose gray-level distribution matches a **specified target distribution** $p_z(z)$, rather than a uniform one.
- Examples of target histograms include:
  - a Gaussian histogram,
  - the histogram of another image,
  - a bimodal histogram,
  - any user-defined distribution.

original target R, G, & B remapped

Remap one Image to have the RGB CDF of Another

Histogram Specification – Continuous formulation

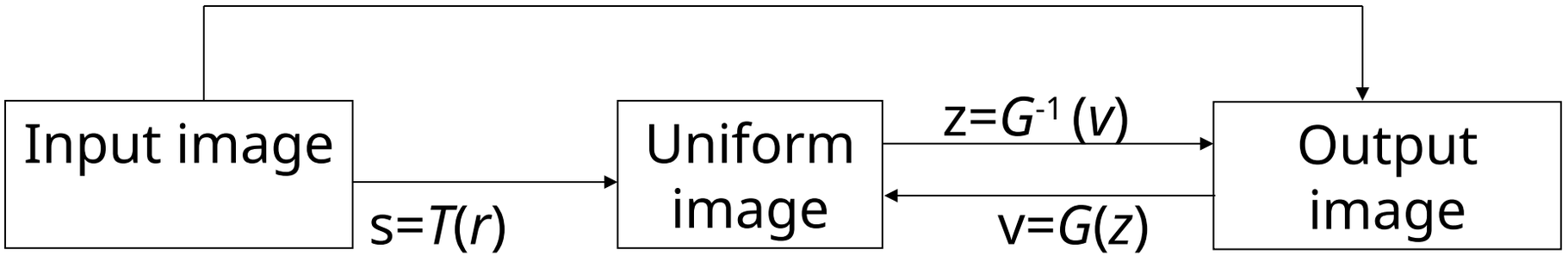
1. **Equalize the input image** by mapping each gray level to its cumulative probability:

$$s = T(r) = \int_0^r p_r(w)\, dw.$$

   - This converts the input to a uniform probability domain.

1. **Describe the desired histogram** through its CDF: $G(z) = \int_0^z p_z(t)\, dt.$
2. **Match equal cumulative probabilities:** $T(r) = G(z).$
3. **Solve for the output gray level:** $\boxed{z = G^{-1}\big(T(r)\big) = H(r).}$

$$z=H(r) = G^{-1}(v=s=T(r)) = G^{-1}(T(r))$$



Histogram Matching: Example

Assuming continuous intensity values, suppose that an image has the intensity PDF

$$p_r(r) = \begin{cases} \dfrac{2r}{(L-1)^2}, & \text{for } 0 \le r \le L-1 \\ 0, & \text{otherwise} \end{cases}$$

Find the transformation function that will produce an image whose intensity PDF is

$$p_z(z) = \begin{cases} \dfrac{3z^2}{(L-1)^3}, & \text{for } 0 \le z \le (L-1) \\ 0, & \text{otherwise} \end{cases}$$

Histogram Matching: Example

Find the histogram equalization transformation for the input image

$$s = T(r) = (L-1)\int_0^r p_r(w)dw = (L-1)\int_0^r \frac{2w}{(L-1)^2}dw = \frac{r^2}{L-1}$$

Find the histogram equalization transformation for the specified histogram

$$G(z) = (L-1)\int_0^z p_z(t)dt = (L-1)\int_0^z \frac{3t^2}{(L-1)^3}dt = \frac{z^3}{(L-1)^2} = s$$

The transformation function, H

$$z = \left[(L-1)^2 s\right]^{1/3} = \left[(L-1)^2 \frac{r^2}{L-1}\right]^{1/3} = \left[(L-1)r^2\right]^{1/3}$$

Histogram Specification – Discrete case

- Obtain $p_r(r)$ from the input image and then obtain the values of s, round the value to the integer range [0, L-1].

$$s = T(r) = (L-1)\sum_{j=0}^{r} p_r(j) = \frac{(L-1)}{MN}\sum_{j=0}^{r} n_j$$

- Use the specified PDF and obtain the transformation function G(z), round the value to the integer range [0, L-1].

$$G(z) = (L-1)\sum_{i=0}^{z} p_z(i) = s$$

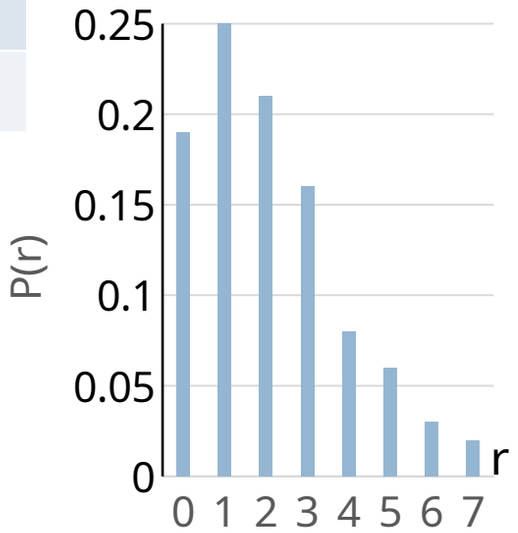
- Mapping from s to z $\quad z = G^{-1}(s)$

Because G^{-1} rarely exists exactly in the discrete case, we choose the output level with the **closest CDF value.**

Histogram Specification – Discrete case

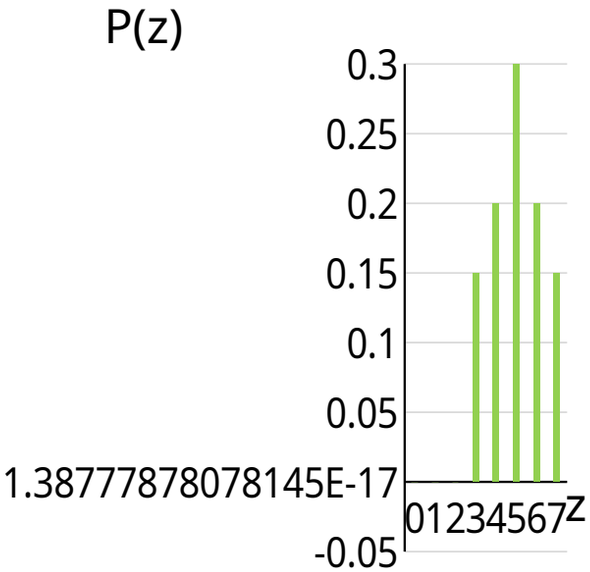
$p_r(r)=n_r/N$	n_r	r
0.19	790	0
0.25	1023	1
0.21	850	2
0.16	656	3
0.08	329	4
0.06	245	5
0.03	122	6
0.02	81	7

original



target

$p_z(z)$	z
0	0
0	1
0	2
0.15	3
0.2	4
0.3	5
0.2	6
0.15	7



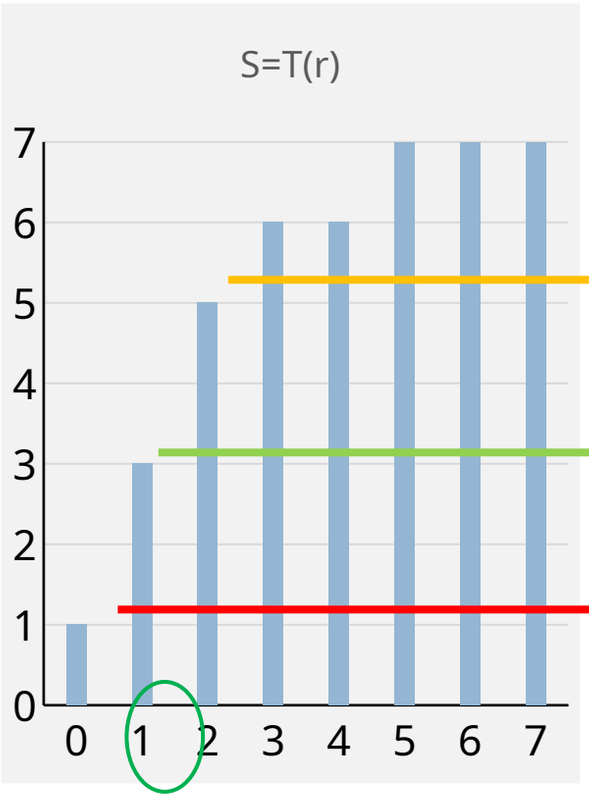
Histogram Specification

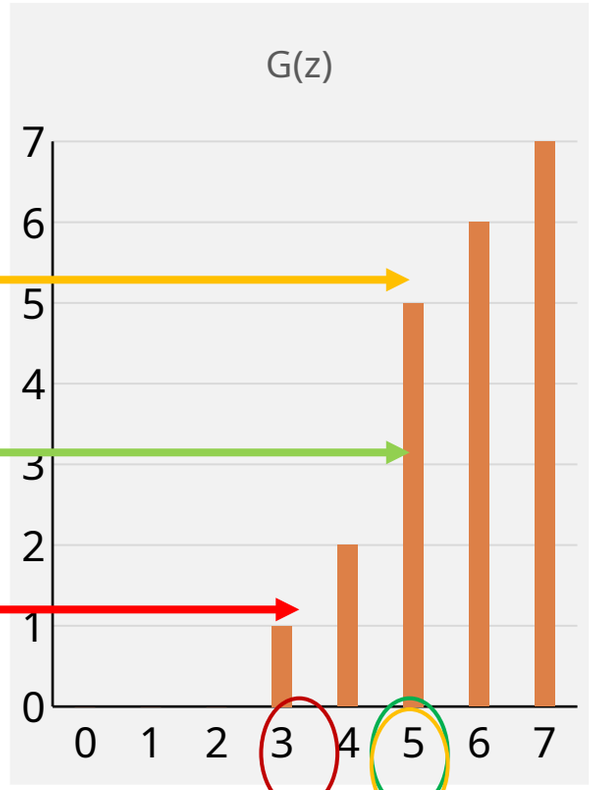
r	$p_r(r)$	$s = T(r)$
0	0.19	1
1	0.25	3
2	0.21	5
3	0.16	6
4	0.08	6
5	0.06	7
6	0.03	7
7	0.02	7

$p_r(r)$	r
0.19	0
0.25	1
0.21	2
0.16	3
0.08	4
0.06	5
0.03	6
0.02	7

G(z)	CDF(z)	$p_z(z)$	z
0	0	0	0
0	0	0	1
0	0	0	2
1	0.15	0.15	3
2	0.35	0.2	4
5	0.65	0.3	5
6	0.85	0.2	6
7	1	0.15	7

z	s
3	1
5 / 4	3
5	5
6	6
7	7





If $\mathbf{f}(x,y) = r$ then let $g(x,y) = z$ where z is such that

$$G(z-1) < [\, s=T(r)] \text{ AND } [\, s=T(r)] \leq G(z)$$

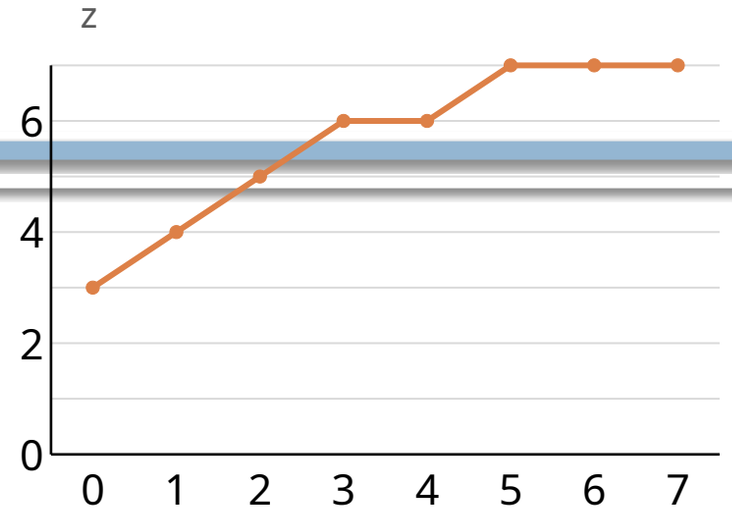
Histogram Specification - LUT

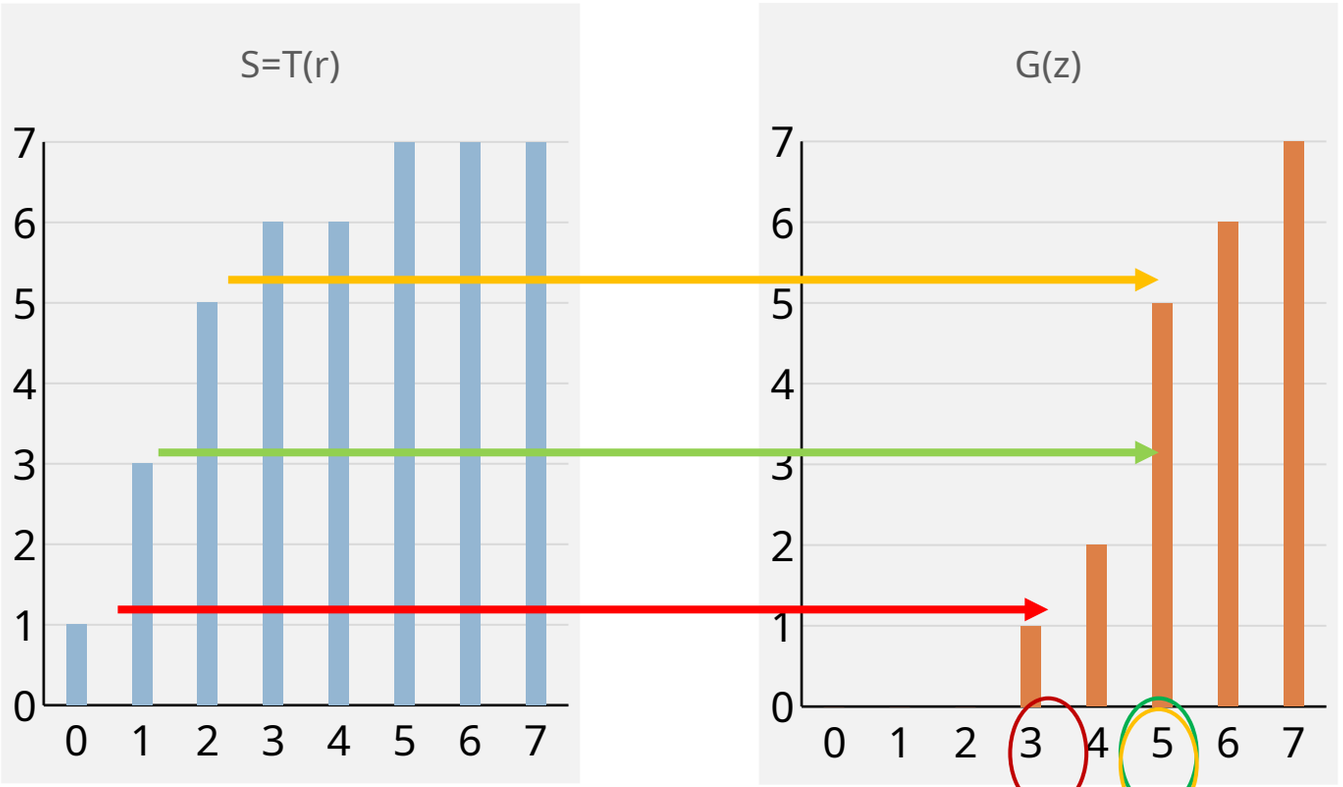
r	z	s
0	3	1
1	5 / 4	3
2	5	5
3	6	6
4	7	7
5		
6		
7		

LUT

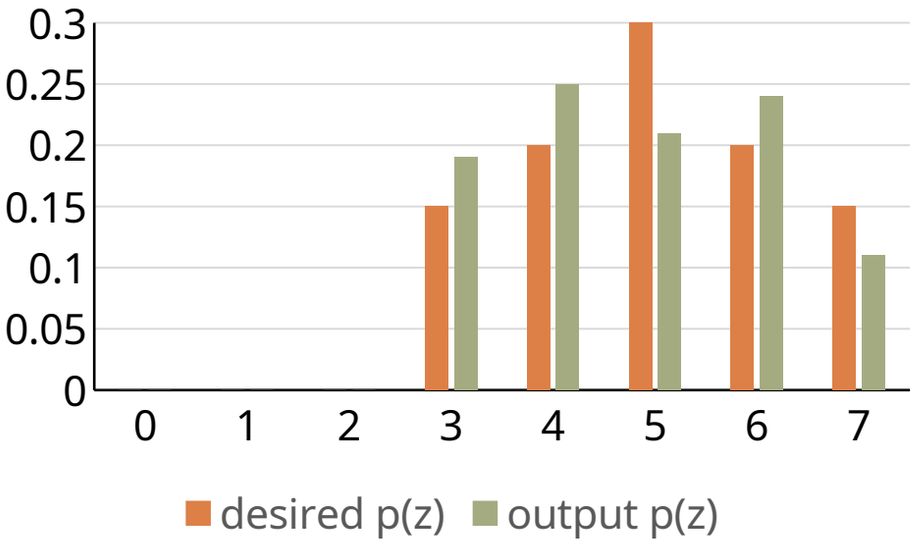
z = H (r)

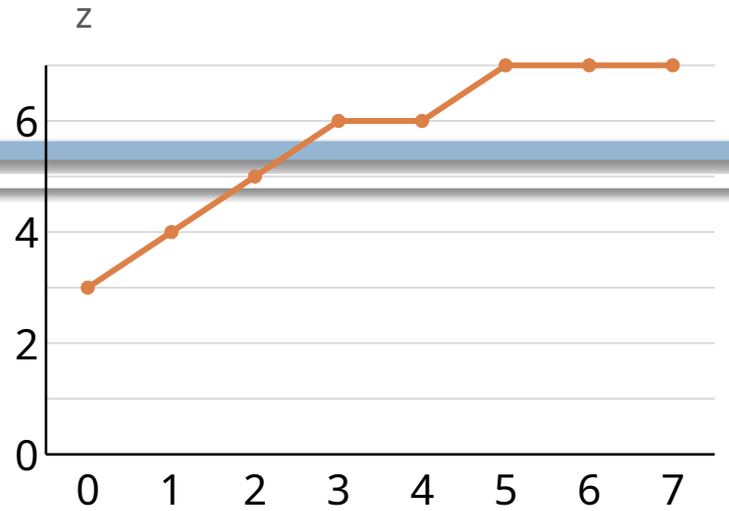
z	r
3	0
4	1
5	2
6	3
6	4
7	5
7	6
7	7



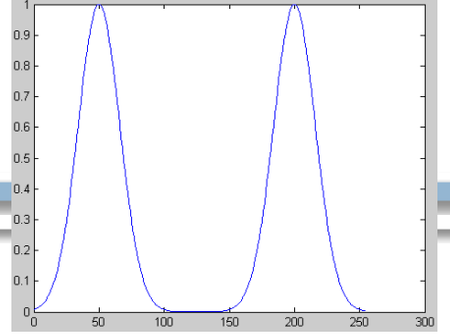


Histogram Specification

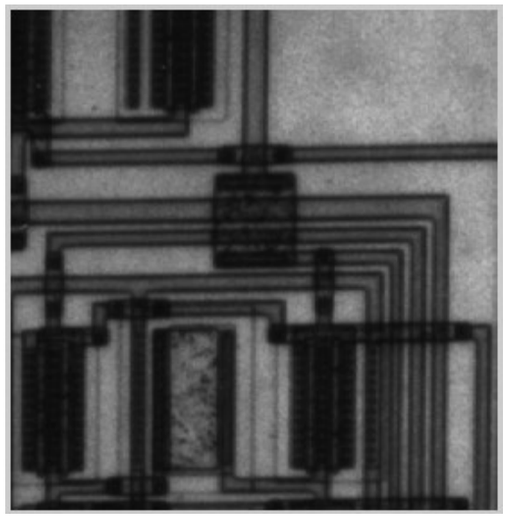


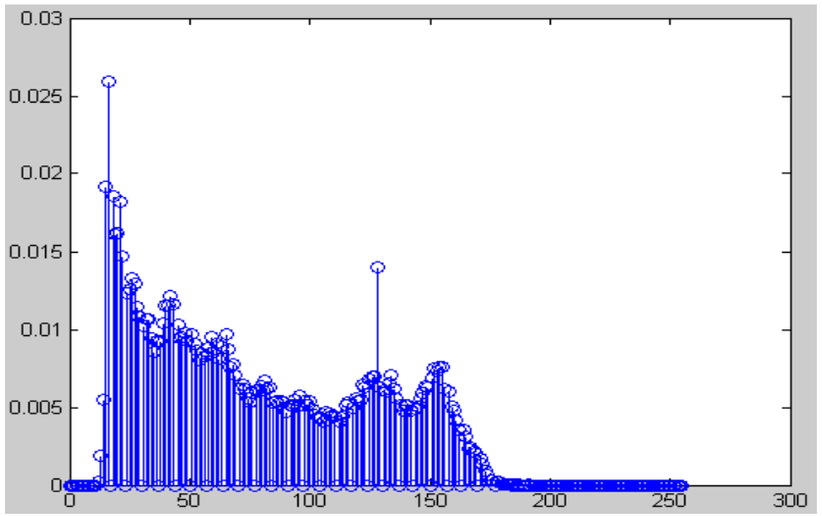


Histogram Specification - Example

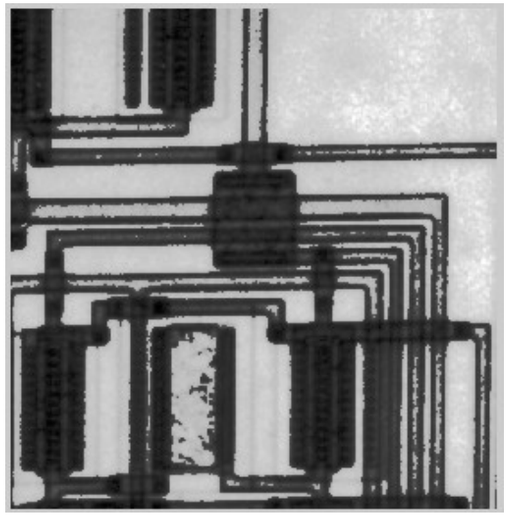


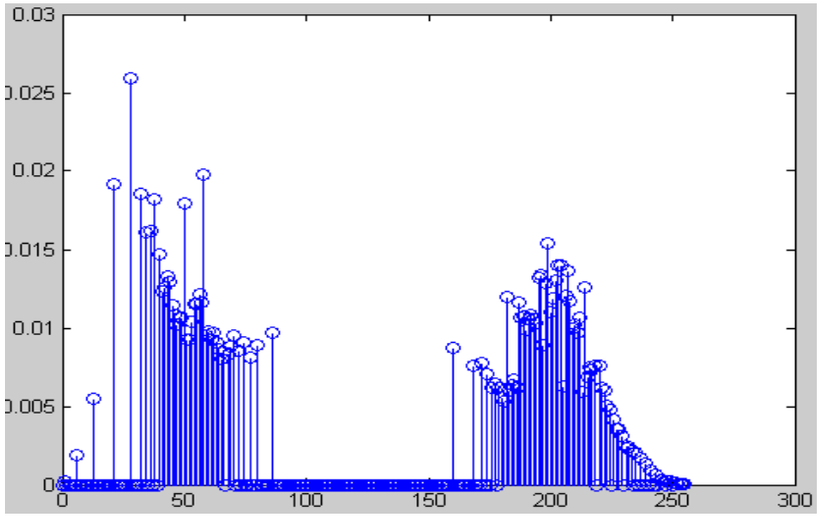
Desired histogram





Original image and its histogram





Histogram specified image and its histogram

Histogram Matching for Stereo Color Cor

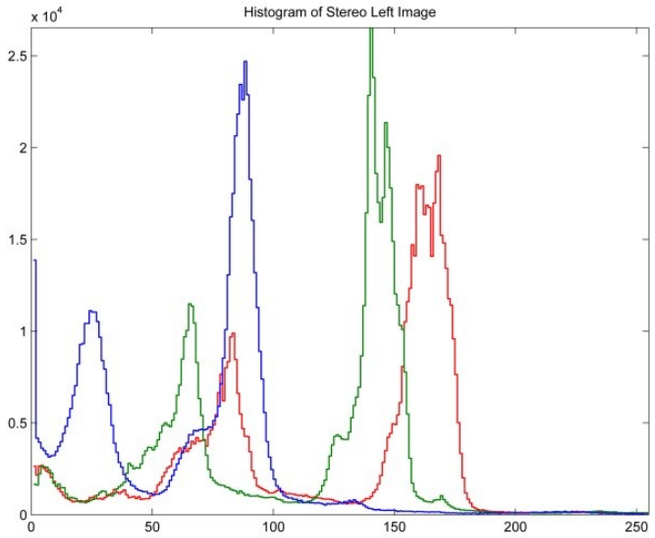
Robot's view of boxes to move.
http://www.universalrobotics.com/

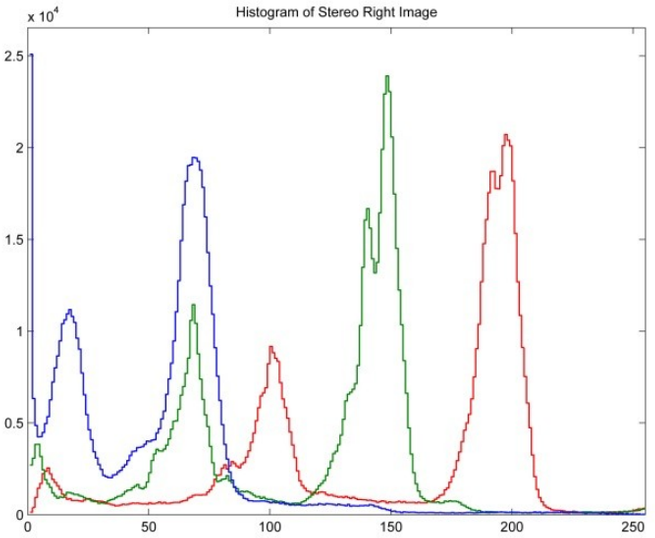
Left Image

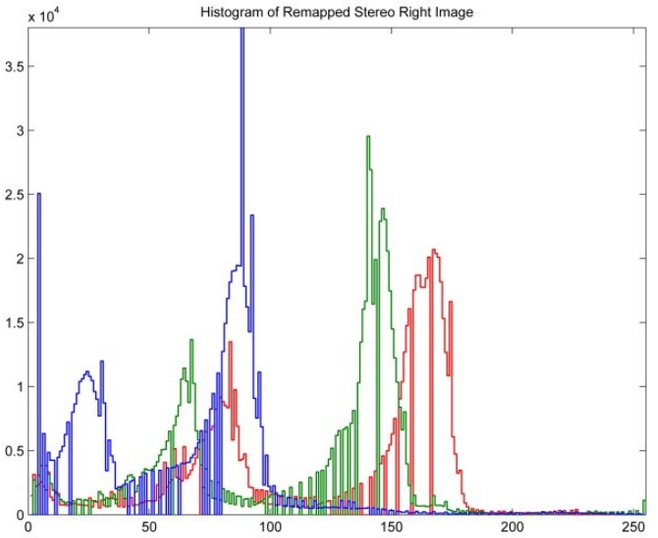
Right Image

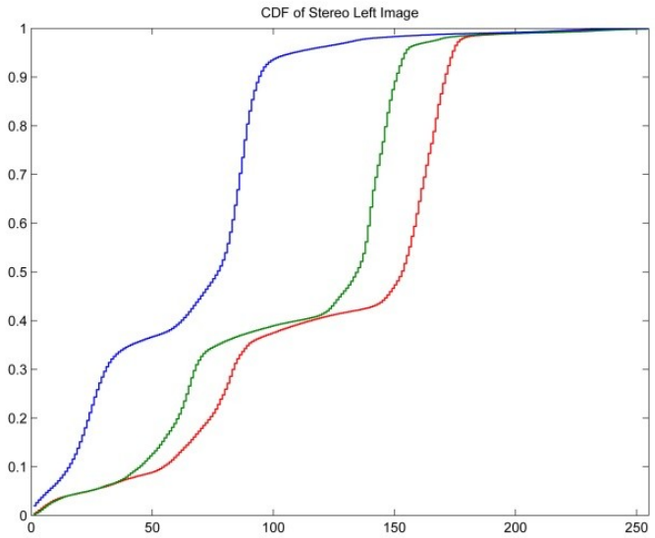
Images from a stereo pair of inexpensive web cams. Such cameras have different color characteristics off-the-shelf. One can be corrected to match the other using histo. matching.

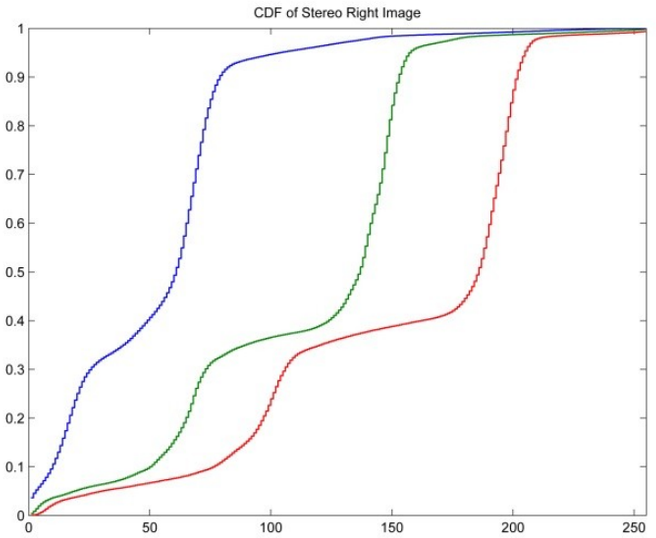
Histogram Matching for Stereo Color Correction

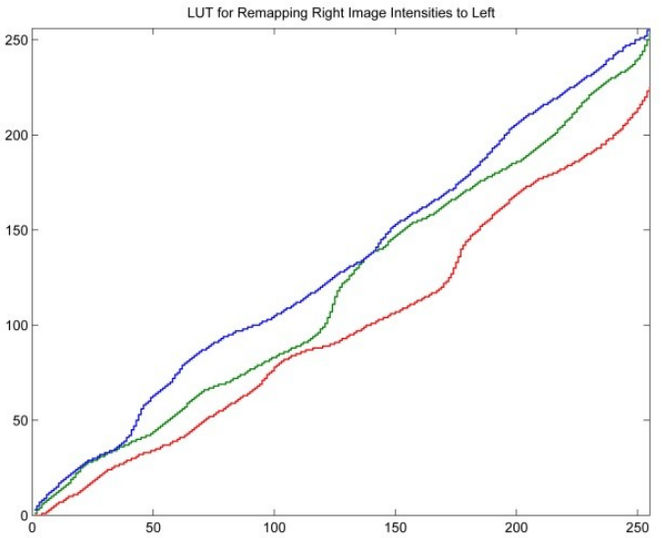












Histogram Ma eo Color Correction

Robot's view of boxes to move.
http://www.universalrobotics.com/

Left Image

Right Image

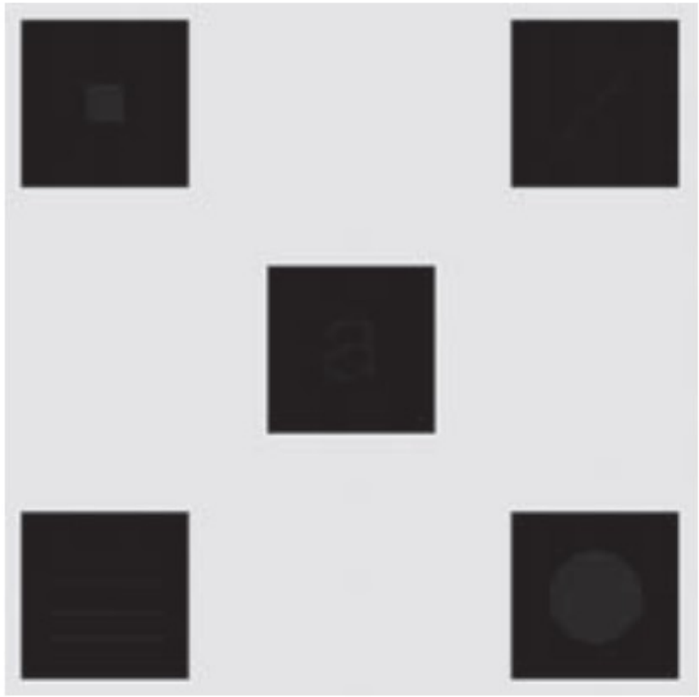
Right image histogram matched to left image.

1999-2018 by Richard Alan Peters II

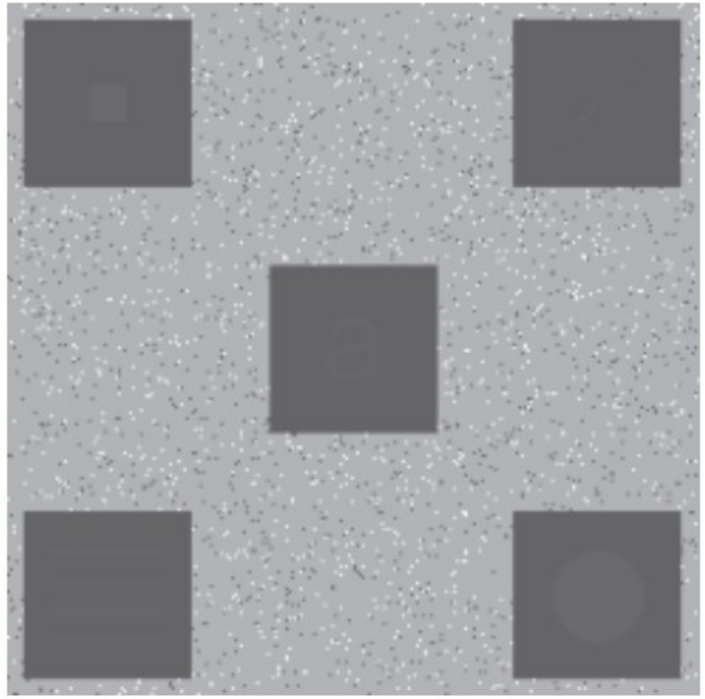
Local Histogram Processing

Local histogram Processing

- problem: global spatial processing not always desirable
- solution: apply point operations to a pixel neighborhood with a sliding window
  - Define a neighborhood and move its center from pixel to pixel. At each location, the histogram of the points in the neighborhood is computed and



Origin



Global

Local HE, 3 × 3 neighborhood

Using Histogram Statistics for Image Enhancement

Denote:
r – intensity value in the range [0, L-1],
p(r) - histogram component corresponding to value r .

m is the mean (average intensity) value of *r*

$$m = \sum_{i=0}^{L-1} r_i p(r_i)$$

The second central moment (also called variance) is particularly important

The intensity variance: $\sigma^2 = \mu_2(r) = \sum_{i=0}^{L-1} (r_i - m)^2 p(r_i)$

- Consider the following 2-bit image of size 5 × 5:

0	0	1	1	2
1	2	3	0	1
3	3	2	2	0
2	3	1	0	0
1	1	3	2	2

$$p(r_0) = \frac{6}{25} = 0.24;\ p(r_1) = \frac{7}{25} = 0.28;$$

$$p(r_2) = \frac{7}{25} = 0.28;\ p(r_3) = \frac{5}{25} = 0.20$$

$$m = \sum_{i=0}^{3} r_i\, p(r_i)$$

$$= (0)(0.24) + (1)(0.28) + (2)(0.28) + (3)(0.20)$$

$$= 1.44$$

$$m = \frac{1}{25} \sum_{x=0}^{4} \sum_{y=0}^{4} f(x, y)$$

$$= 1.44$$

Using Histogram Statistics for Image Enhancement

Let (x, y) be the coordinates of a pixel in an image, and let S_{xy} denote a neighborhood (subimage) of specified size, centered at (x, y).

The ***mean value*** of the pixels in this neighborhood is given by

$$m_{S_{xy}} = \sum_{i=0}^{L-1} r_i p_{S_{xy}}(r_i)$$

where $P_{S_{xy}}$ is the histogram of the pixels in region S_{xy}.

The ***variance*** of the pixels *in the* neighborhood is given by

$$\sigma^2_{S_{xy}} = \sum_{i=0}^{L-1} (r_i - m_{S_{xy}})^2 p_{S_{xy}}(r_i)$$

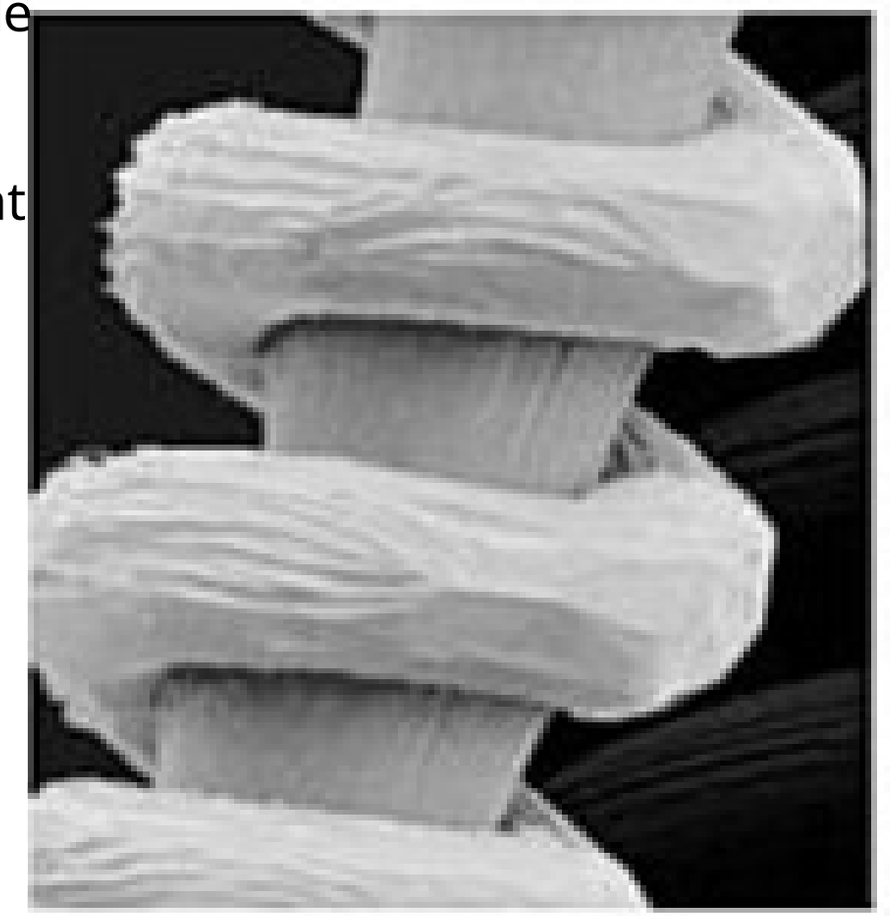
Using Histogram Statistics for Image Enhancement

- In this particular case, the problem is to enhance dark areas while leaving the light area as unchanged as possible because it does not require enhancement.

- Formula that can tell the difference between dark and light and, at the same time, is capable of enhancing only the dark areas.

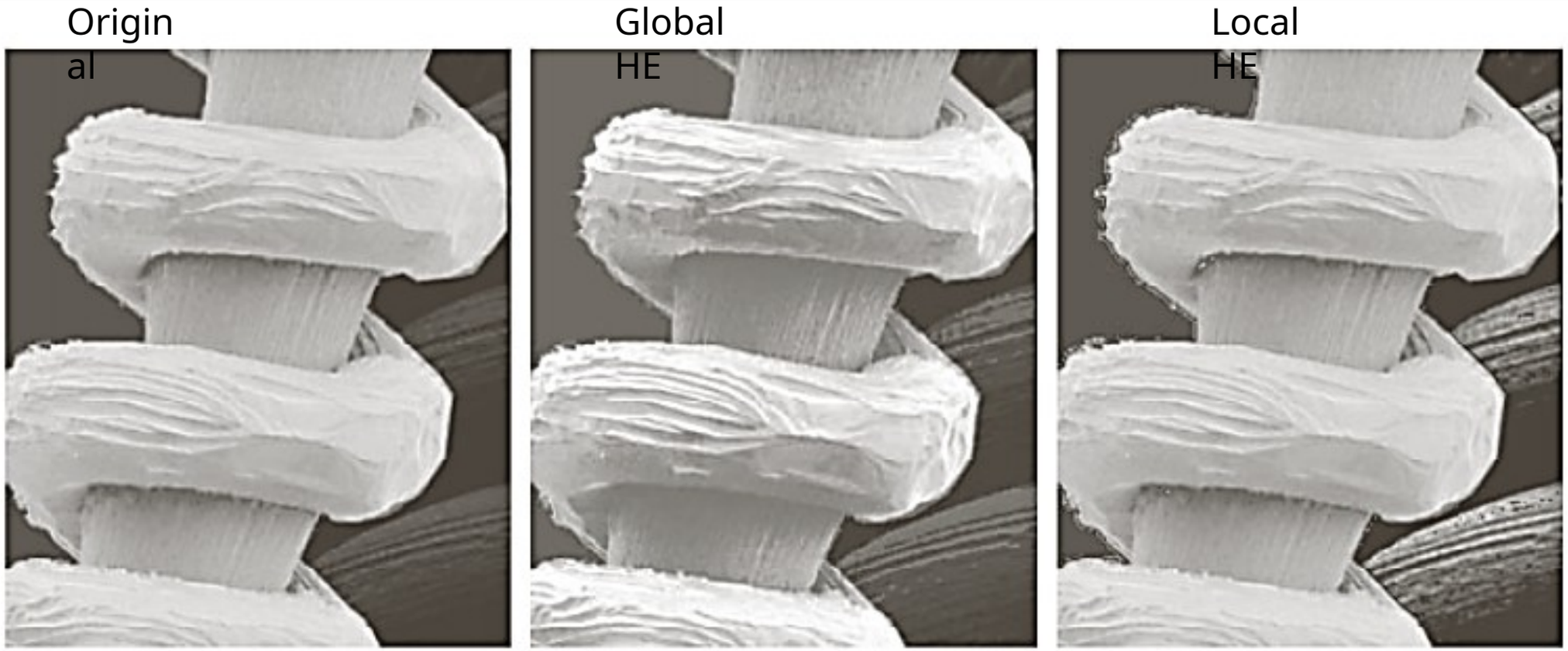
$$g(x, y) = \begin{cases} E \cdot f(x, y) & \text{if } m_{S_{xy}} \le k_0 m_G \text{ AND } k_1\sigma_G \le \sigma_{S_{xy}} \le k_2\sigma_G \\ f(x, y) & \text{otherwise} \end{cases}$$

where, as indicated previously, E, k_0, k_1, and k_2 are specified parameters; M_G is the global mean of the input image; and D_G is its global standard deviation.



SEM image of Tungsten filament

Using Histogram Statistics for Image Enhancement



a b c

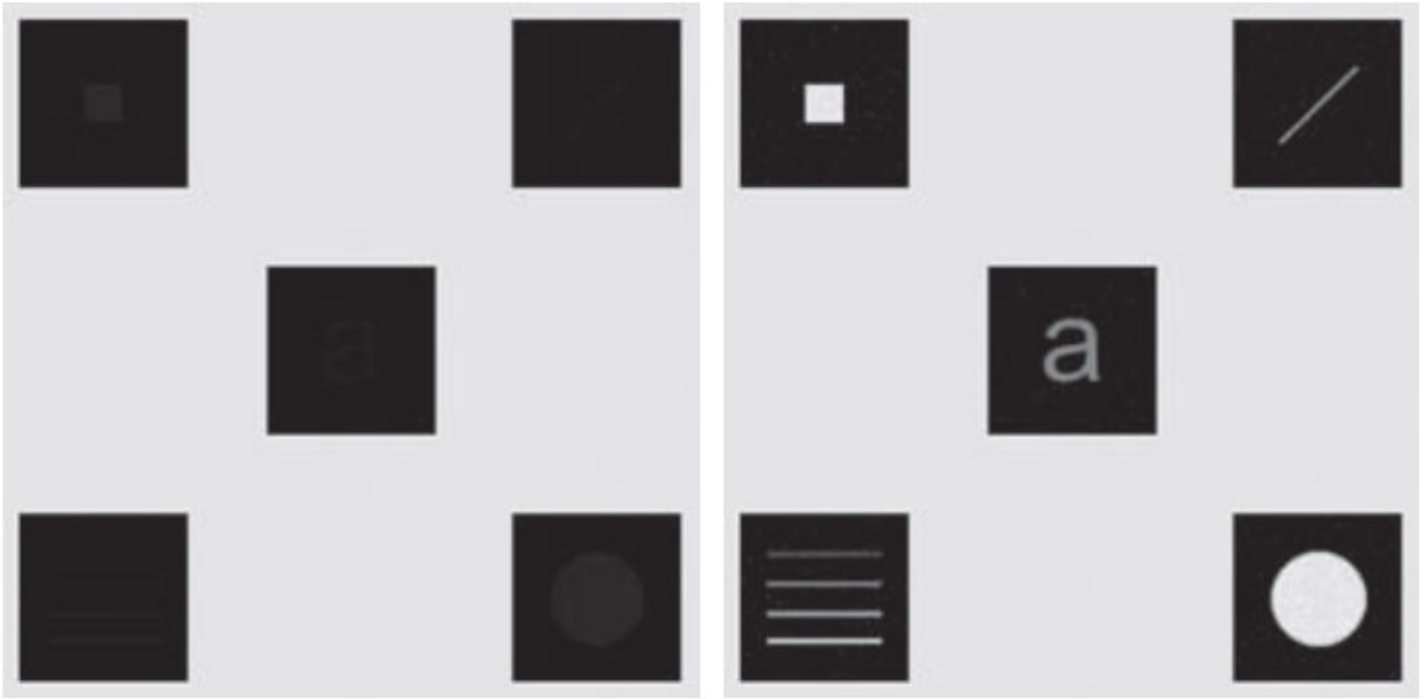
FIGURE 3.27 (a) SEM image of a tungsten filament magnified approximately 130×. (b) Result of global histogram equalization. (c) Image enhanced using local histogram statistics. (Original image courtesy of Mr. Michael Shaffer, Department of Geological

Using Histogram Statistics for Image Enhancement

$$g(x,y) = \begin{cases} C\,f(x,y) & \text{if } k_0 m_G \le m_{S_{xy}} \le k_1 m_G \text{ AND } k_2 \sigma_G \le \sigma_{S_{xy}} \le k_3 \sigma_G \\ f(x,y) & \text{otherwise} \end{cases}$$

$k0 = 0$ and $k1 = 0.1$
$k2 = 0$ and $k3 = 0.1$

$mG = 161$, $\sigma G = 103$
C = 22.8



Summary

- Key ideas to remember before implementation and problem solving
- **Point operations**
  - Modify each pixel independently using s = T(r); they are memory-less and can be implemented with lookup tables.
- **Intensity transforms**
  - Negative, thresholding, contrast stretching, log, gamma, and bit-plane slicing enhance selected brightness ranges.
- **Histogram methods**
  - Histogram equalization spreads intensities automatically, while histogram matching forces a desired or reference distribution.
- **Local enhancement**
  - AHE/CLAHE and local histogram statistics improve details in specific regions while controlling noise and over-e

Takeaway: choose T(r) according to the enhancement goal - global contrast, local detail, brightness correction, or distribution matching.

Spatial Filtering

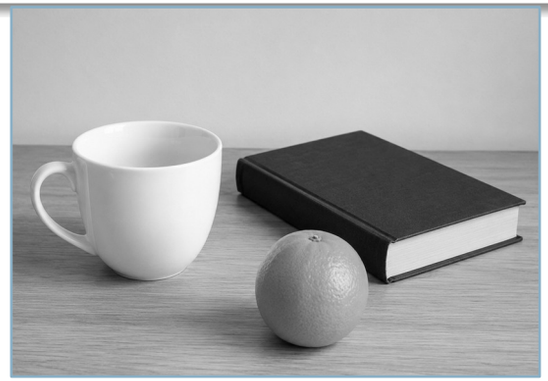
Dr. Sk. Md. Masudul Ahsan
Professor
Dept. of CSE, KUET

Convolution and Cross Correlation

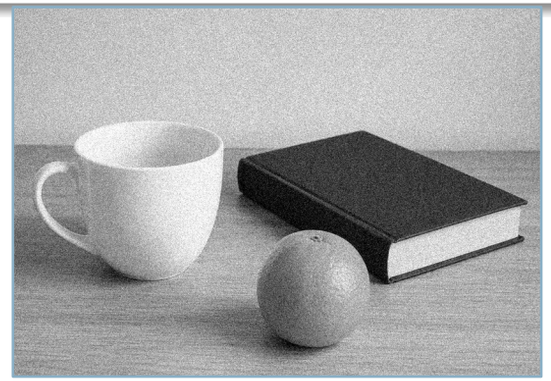
Learning Outcomes

- Explain spatial filtering as a neighborhood operation.
- Differentiate convolution, correlation, and cross-correlation.
- Compute 1D and 2D discrete filter responses.
- Choose filters for smoothing, sharpening, edges, and matching.

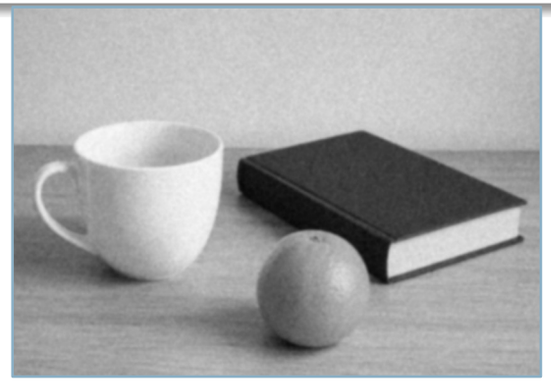
Why Spatial Filtering?



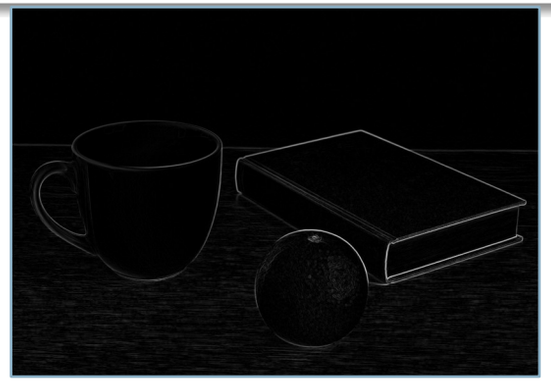
Original



Noisy image



Smoothed



Edges

- Point operations use only the current pixel value.
- Spatial filtering uses the current pixel and its local neighborhood.
- This enables noise reduction, edge detection, texture analysis, and pattern matching.

- Pixel context matters.
- Filter = mask + local operation.
- Uses: denoise, sharpen, detect edges, match templates.

Spatial Filtering Terminology

Term	Meaning
Image, f(x,y)	2D array of pixel intensities
Mask / Kernel / Filter, w	Small neighborhood of weights
Output, g(x,y)	Filtered image after moving the mask
Anchor / Center	Mask position aligned with the current pixel

output = weighted sum of neighboring pixels

Linear Shift-Invariant Filtering

Linearity

$$T(a\,f + b\,g) = aT(f) + bT(g)$$

Shift invariance
Same rule at every position

shift input ⇒ shifted output

- Linear: weighted sums preserve additivity and scal
- Shift-invariant: the same rule is used at every position.
- Most classical spatial filters are designed as LSI systems.

Image filtering = local LSI operation

- Linear spatial filters use a weighted sum.
- This model leads naturally to convolution and correlation.

1D Continuous Convolution

- Convolution is defined as follows:

$$f(x) * g(x) = (f * g)(x) = \int_{-\infty}^{\infty} f(a)\, g(x - a)\, da$$

1. Flip g(a) → g(-a)
2. Shift it by x: g(x-a) $-\infty < x < \infty$
3. Compute area overlap f(a)g(x-a)

For images: f = image, g = mask/kernel.

- Convolution is commutative:

$$f(x) * g(x) = g(x) * f(x)$$

Think of f(x) as the image and g(x) as the mask although you can reverse their roles!

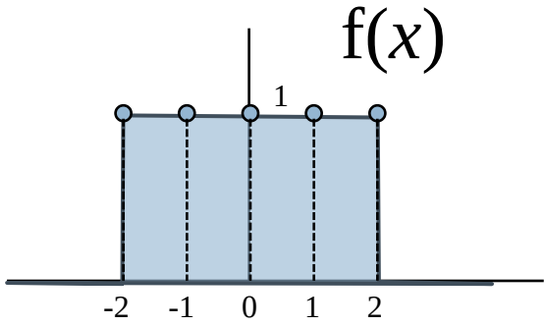
1D Discrete Convolution

- Replace integral with summation
- Integration variable becomes an index.
- Displacements take place in discrete increments
- For finite sequences of lengths A and B, full output length = A + B − 1

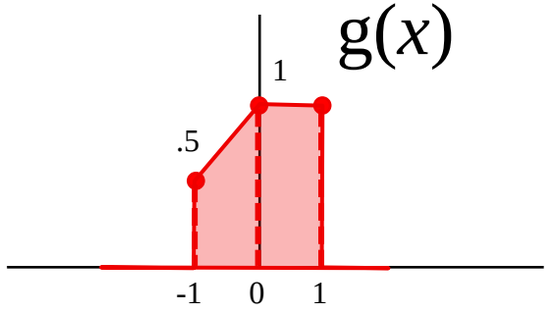
$$f(x) * g(x) = \sum_{m} f(m)g(x - m) \quad ; \quad -\infty < x < \infty$$

Example sizes: 5 samples * 3 samples → 7 output samples

1D Discrete Example



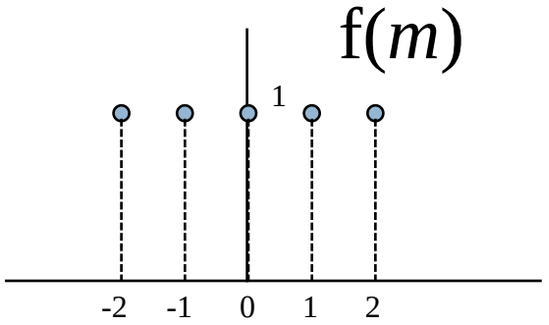
5 samples

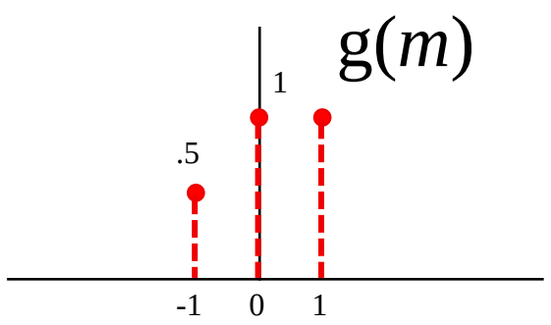


3 samples

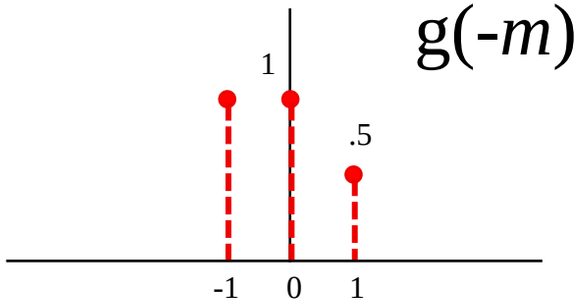
$$f(x) * g(x) = \sum_{m=-\infty}^{\infty} f(m)g(x-m), -\infty < x < \infty$$

Example

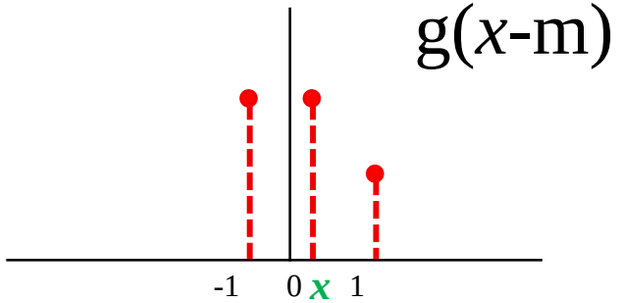




$$f(x) * g(x) = \sum_{m=-\infty}^{\infty} f(m)g(x-m), -\infty < x < \infty$$



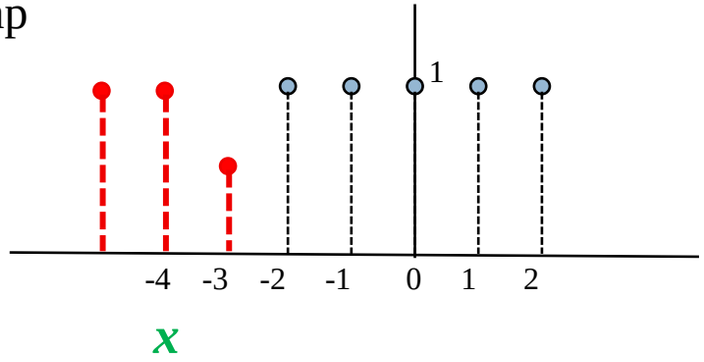
Compute the convolution assuming discrete val $-\infty < x < \infty$



Example (cont'd)

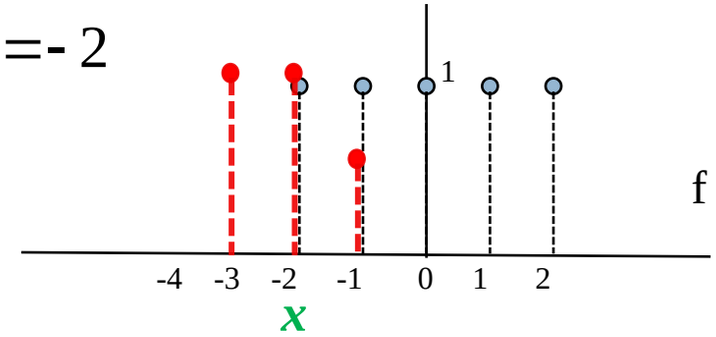
$x = -4 \; or \; x < -4$

no overlap



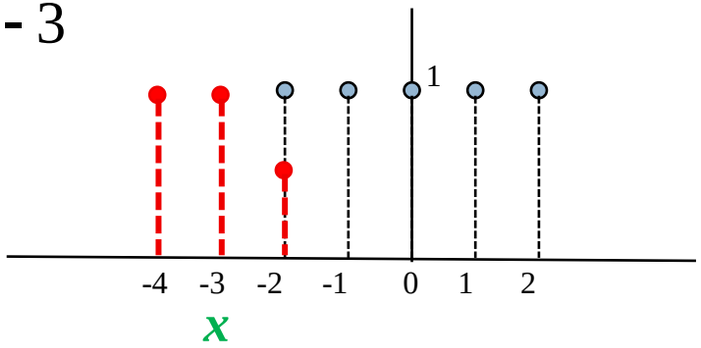
f *g = 0

$x = -2$



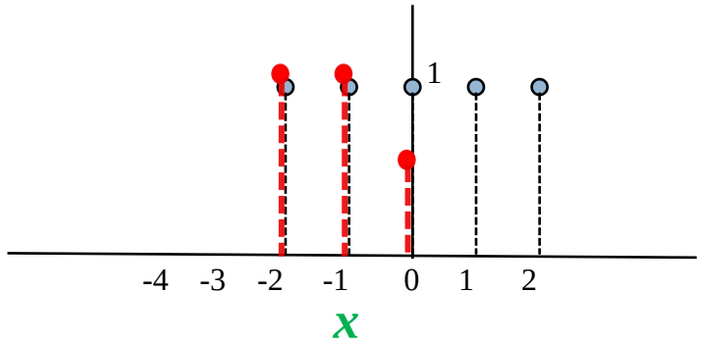
f *g = 1.5

$x = -3$



f *g = 0.5

$x = -1$

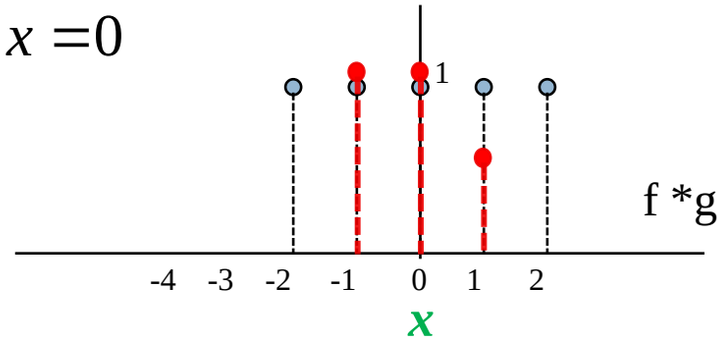


f *g = 2.5

Note that I show some red samples taller and slightly off for clarity!

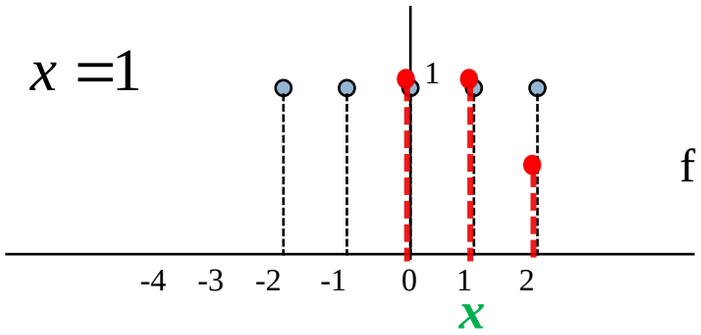
Example (cont'd)

$x = 0$



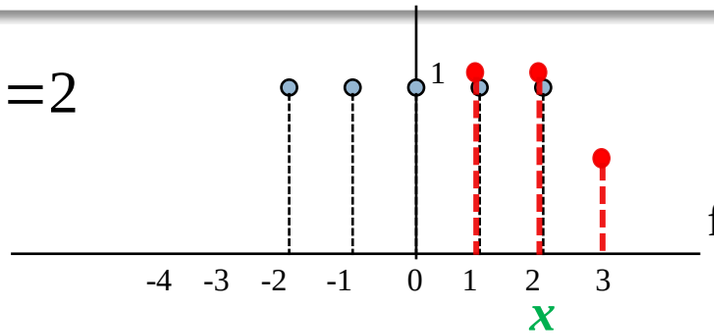
f *g = 2.5

$x = 1$



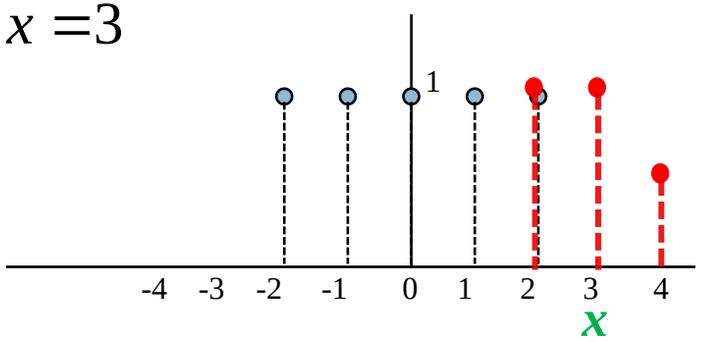
f *g = 2.5

$x = 2$



f *g = 2

$x = 3$



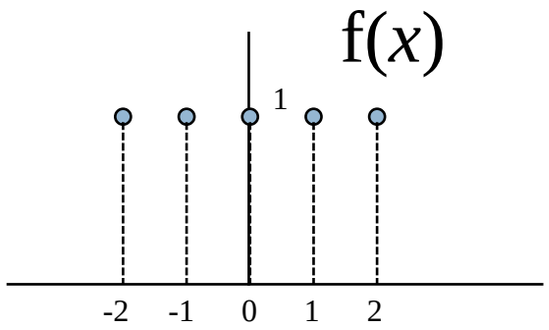
f *g = 1

$x \geq 4$

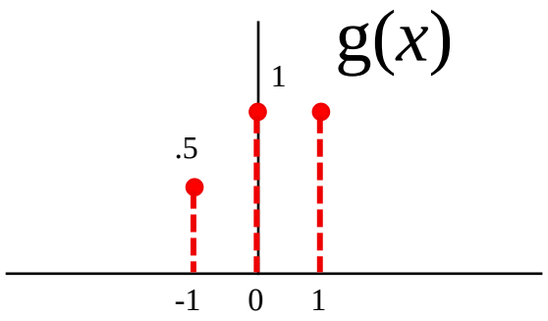
f *g = 0

no overlap

Example

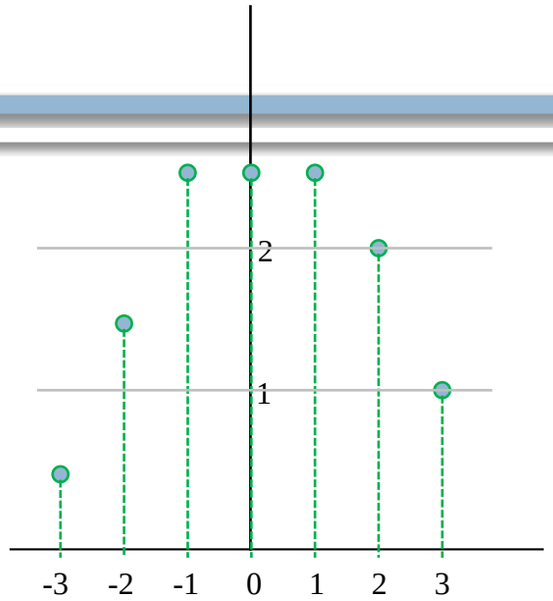


5 samples



3 samples

=



7 samples

length of f*g = length of f + length of g - 1

Example

Convolute two sequences x[n] = {1,2,3} & h[n] = {-1,2,2}

×	1	2	3
-1	-1	-2	-3
2	2	4	6
2	2	4	6

Convoluted output y[n] = [-1, -2+2, -3+4+2, 6+4, 6]

= [-1, 0, 3, 10, 6]

1D Discrete Cross-Correlation

$$(f \star g)[x] = \Sigma_m f[m]\, g[x + m]$$

- The kernel/template is shifted without flipping.
- Cross-correlation measures similarity between two different signals.
  - The output is large when the local signal looks like the template.
  - This is the basis of template matching in images.
- Autocorrelation is correlation of a signal with itself.

corr = dot product at each shift

conv = corr with flipped kernel

Convolution vs Correlation

Aspect	Convolution	Correlation / Cross-correlation
Kernel orientation	Flipped before sliding	Not flipped
Main use	Filtering and system response	Similarity or template matching
Associative?	Yes	Not generally
Same result when	Kernel is symmetric	Kernel is symmetric

Moving from 1D Signals to 2D Images

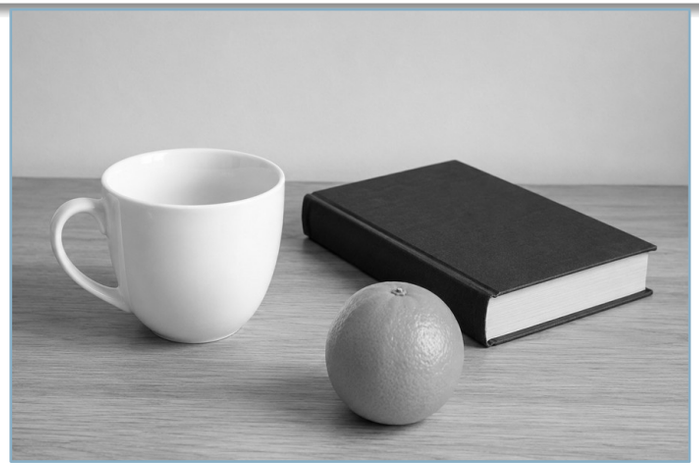


Image = 2D function f[x,y]

80	82	85
78	81	84
75	79	82

3×3 neighborhood

- A spatial filter is a small 2D kernel w[s,t].
- The kernel center is placed over each pixel.
- The response becomes the output pixel value.

2D Discrete convolution

- similar to 1D - we now have to iterate over 2 axis instead of 1.

$$f(x,y) * g(x,y) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} f(m,n)g(x-m, y-n); \quad -\infty < x, y < \infty$$

a	b	c
d	e	f
g	h	i

→

i	h	g
f	e	d
c	b	a

Original kernel **Flipped kernel**

• Flip the kernel in both directions: left-right and top-bottom.

• Then compute the same weighted sum as

2D Discrete convolution

- Suppose f(x,y) is A x B and g(x,y) is C x D
- The size of f(x,y) * g(x,y) would be N x M where

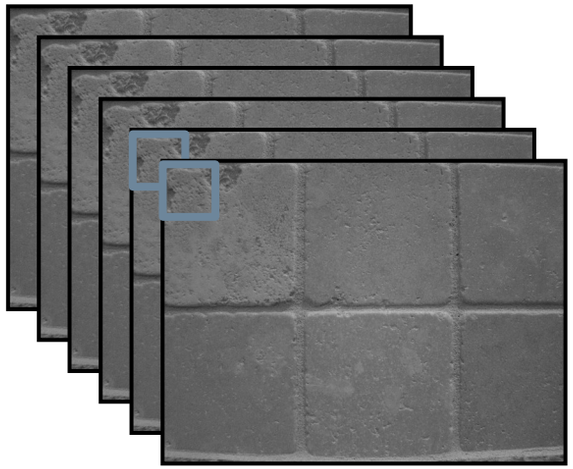
  N=A+C-1 and M=B+D-1

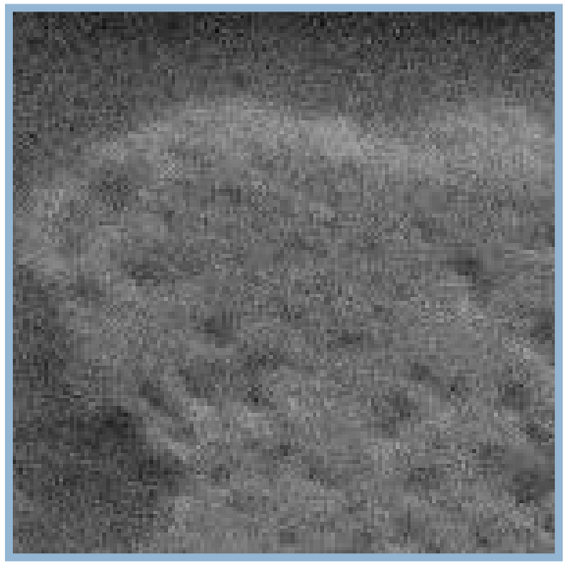
- Form extended images (i.e., pad with zeroes):

$$f_e(x,y) = \begin{cases} f(x,y) & 0 \le x \le A-1 \text{ and } 0 \le y \le B-1 \\ 0 & A \le x \le M-1 \text{ and } B \le y \le N-1 \end{cases}$$

$$g_e(x) = \begin{cases} g(x,y) & 0 \le x \le C-1 \text{ and } 0 \le y \le D-1 \\ 0 & C \le x \le M-1 \text{ and } D \le y \le N-1 \end{cases}$$

Motivation: noise reduction





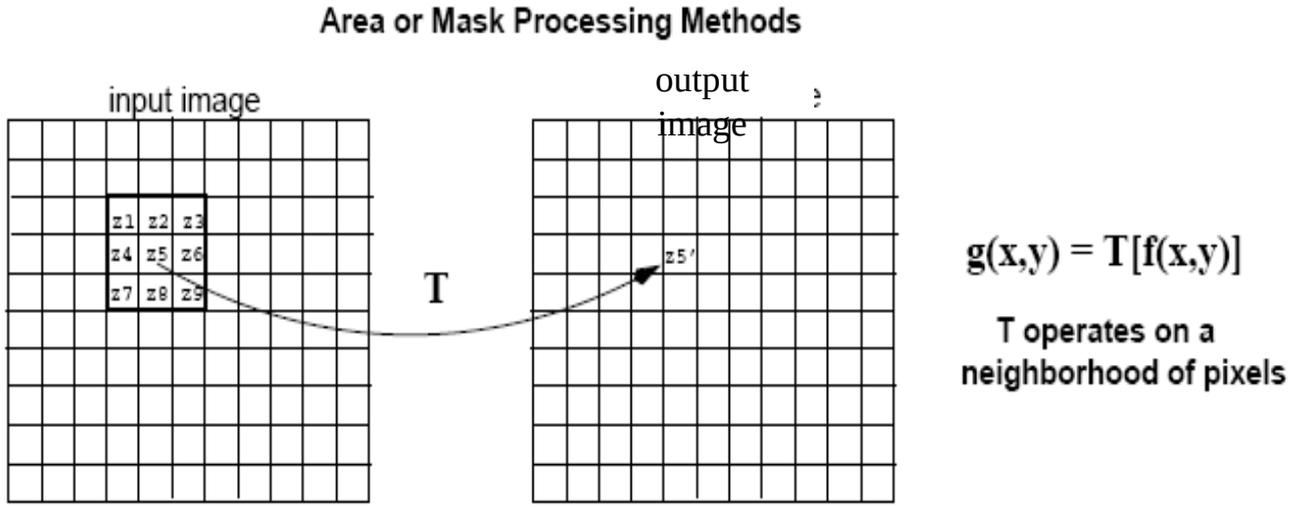
- How could we reduce the noise, i.e., give an estimate of the true intensities?
- **What if there's only one image?**

Mechanics of spatial filtering

- The process consists simply of moving the center of the filter mask from point to point in an image.
- At each point (x,y) the response of the filter at that point is calculated using a predefined relationship
  - For convolution
    - Multiply each kernel (flip) coefficient by its aligned pixel.
    - Sum all products to produce g[x,y].
    - Move the kernel to the next pixel and repe

w1	w2	w3
w4	w5	w6
w7	w8	w9

mask weights:



Mechanics – 2D Convolution

- As filter
  - Mechanics

$H[u, v]$

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$F[x, y]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$G[x, y]$

	0								

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

After flip

Source: S. Seitz

Convolution In 2D

$F[x, y]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$G[x, y]$

	0	10							

Source: S. Seitz

Convolution In 2D

$F[x, y]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$G[x, y]$

	0	10	20						

Source: S. Seitz

Convolution In 2D

$F[x, y]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

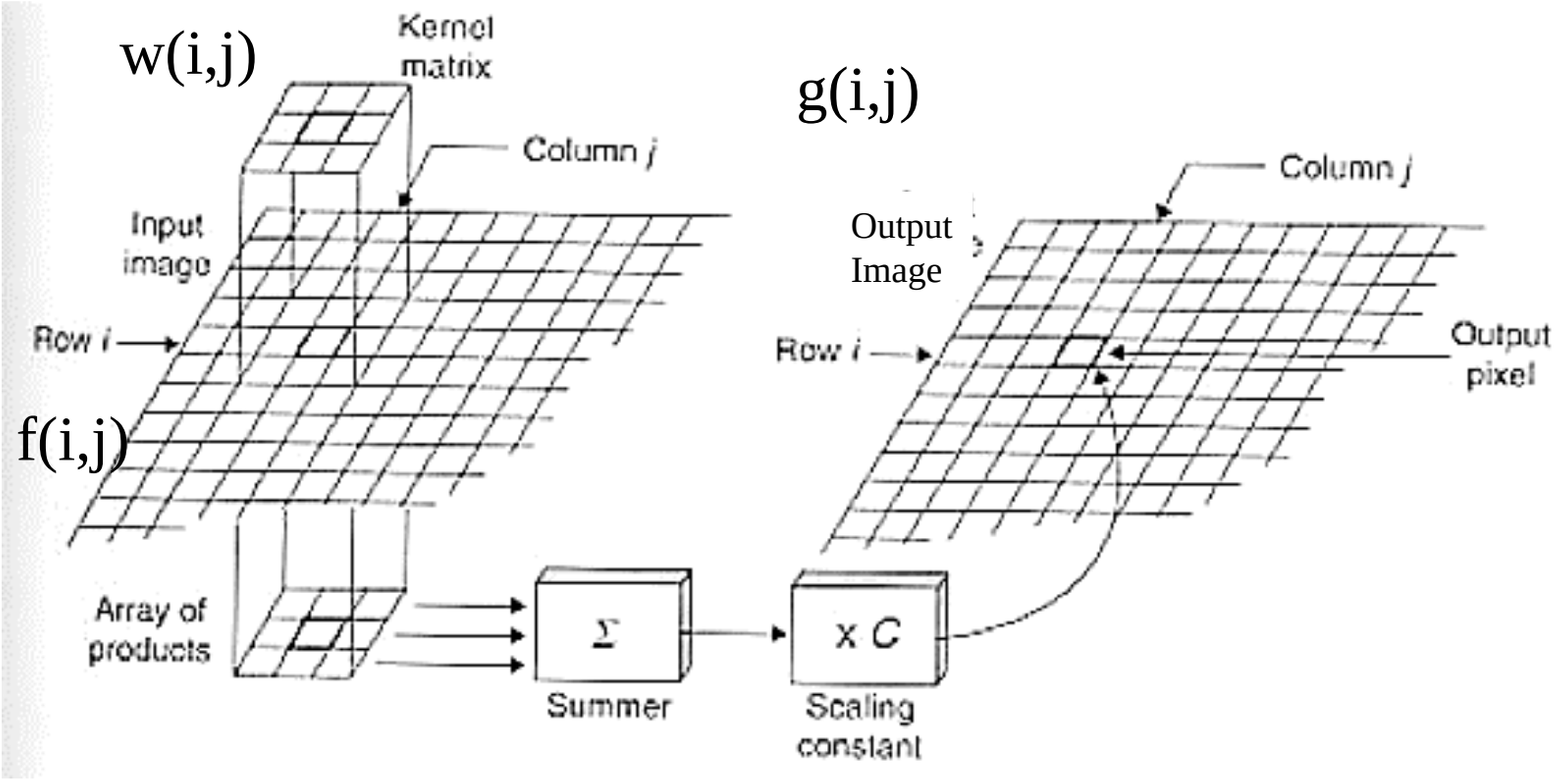
$G[x, y]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

Source: S. Seitz

2D Correlation - Mechanics

No Flip



$$w(x,y)\star f(x,y) = \sum_{s=-a}^{a}\sum_{t=-b}^{b} w(s,t)f(x+s,y+t)$$

Properties of Convolution

Property	Expression	Meaning
Commutative	f * g = g * f	signal and kernel can swap roles
Associative	(f * g) * h = f * (g * h)	filters can be combined
Distributive	f * (g + h) = f*g + f*h	filter sum equals sum of filtered results
Identity	f * δ = f	unit impulse leaves signal unchanged
Shift-invariant	shift(f) * g = shift(f*g)	same behavior at every position
Differentiation		

Property	Expression	Meaning
Commutative	f * g = g * f	signal and kernel can swap roles
Associative	(f * g) * h = f * (g * h)	filters can be combined
Distributive	f * (g + h) = f*g + f*h	filter sum equals sum of filtered results
Identity	f * δ = f	unit impulse leaves signal unchanged
Shift-invariant	shift(f) * g = shift(f*g)	same behavior at every position

These properties make convolution convenient for image-filter design.

Proof these properties

Theorem 4.1 (Commutativity of convolution)**.** *Convolution is commutative. That is, for any two functions x and h,*

$$x * h = h * x. \tag{4.16}$$

In other words, the result of a convolution is not affected by the order of its operands.

Proof. We now provide a proof of the commutative property stated above. To begin, we expand the left-hand side of (4.16) as follows:

from definition of convolution

$$x * h(t) = \int_{-\infty}^{\infty} x(\tau) h(t-\tau) d\tau.$$

$h * x(t) = \int_{-\infty}^{\infty} h(\tau)\, x(t-\tau)\, d\tau$

Next, we perform a change of variable. Let $v = t - \tau$ which implies that $\tau = t - v$ and $d\tau = -dv$. Using this change of variable, we can rewrite the previous equation as

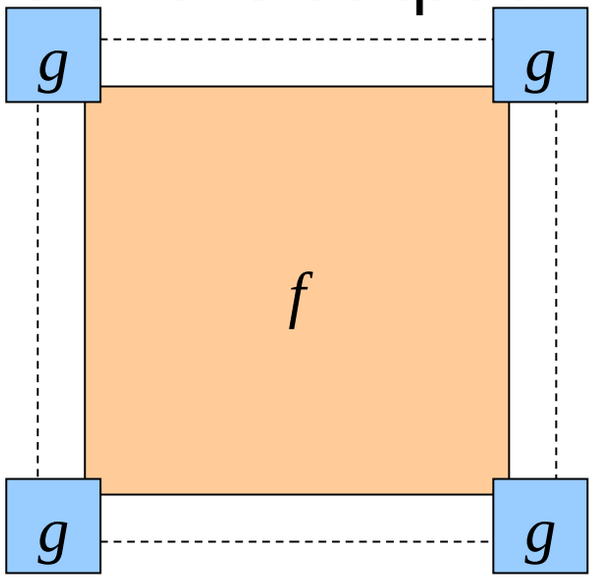
$$\begin{aligned}
x * h(t) &= \int_{t+\infty}^{t-\infty} x(t-v) h(v)(-dv) && \text{from change of variable} \\
&= \int_{\infty}^{-\infty} x(t-v) h(v)(-dv) && \text{infinity dominates sums} \\
&= \int_{-\infty}^{\infty} x(t-v) h(v) dv && \int_a^b f(x)\,dx = -\int_b^a f(x)\,dx \\
&= \int_{-\infty}^{\infty} h(v) x(t-v) dv && \text{rearrange factors} \\
&= h * x(t). && \text{definition of convolution}
\end{aligned}$$

Remember that changing integration variable changes limits!

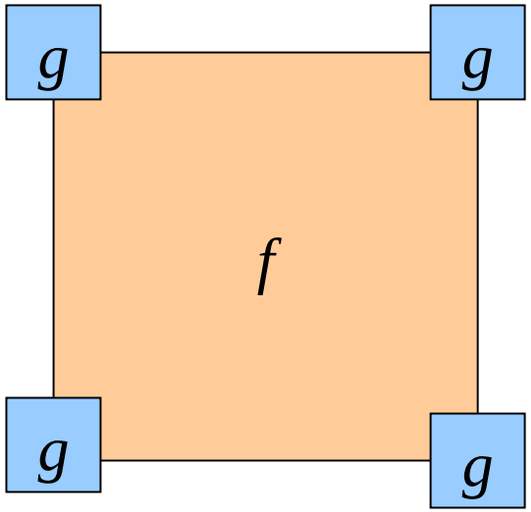
(Note that, above, we used the fact that, for any function f, $\int_a^b f(x)dx = -\int_b^a f(x)dx$.) Thus, we have proven that convolution is commutative. ■

Boundary issues

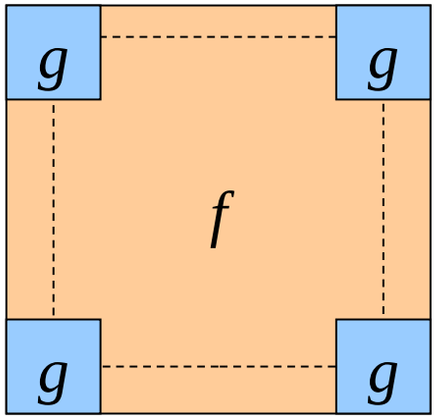
- What is the size of the output?



Full



Same



valid

- At borders, a kernel may fall partly outside the image.
- We need an extrapolation or padding rule.
- Boundary choice can visibly affect filtered images.

Source: S. Lazebnik

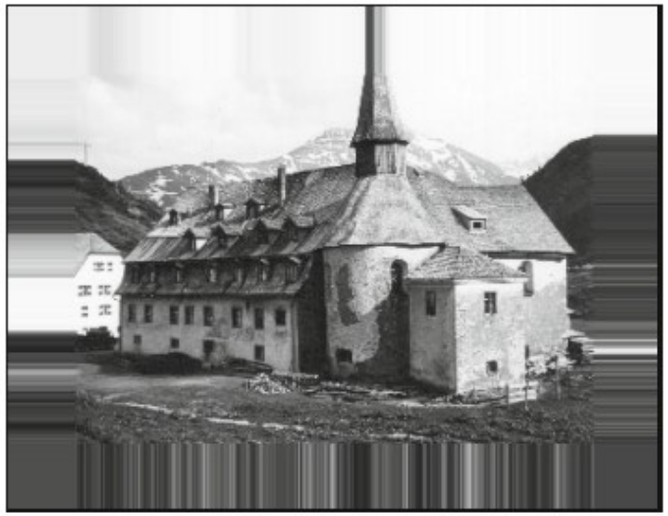
Output Size and Boundaries

Mode	Meaning	Output size
Full	All overlap positions	A+C−1 by B+D−1
Same	Same size as input image	A by B
Valid	Only complete overlap	A−C+1 by B−D+1

Method	Idea	
Zero padding	outside pixels = 0	Darker border possible
Replicate	copy nearest border pixel	
Reflect	mirror around border	Usually smoother border
Circular	wrap around to opposite	

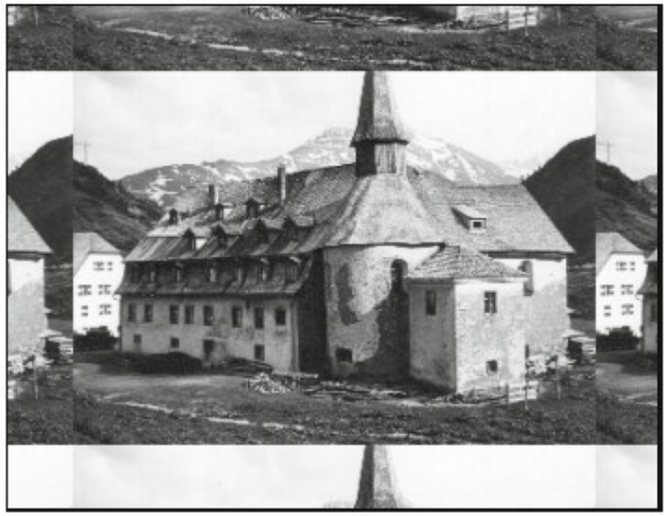
Boundary issues

(b)



(c)

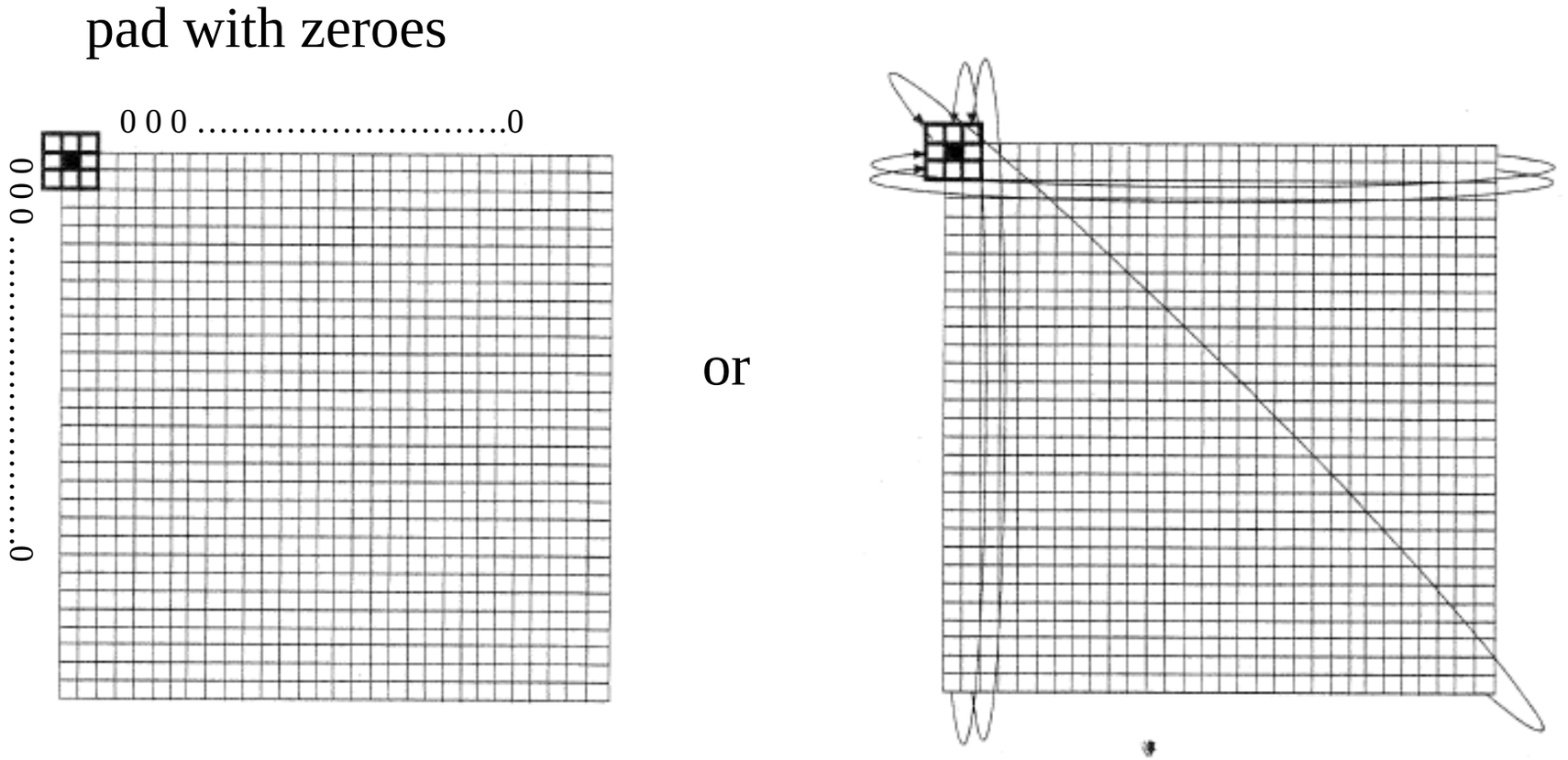
(d)



(e)

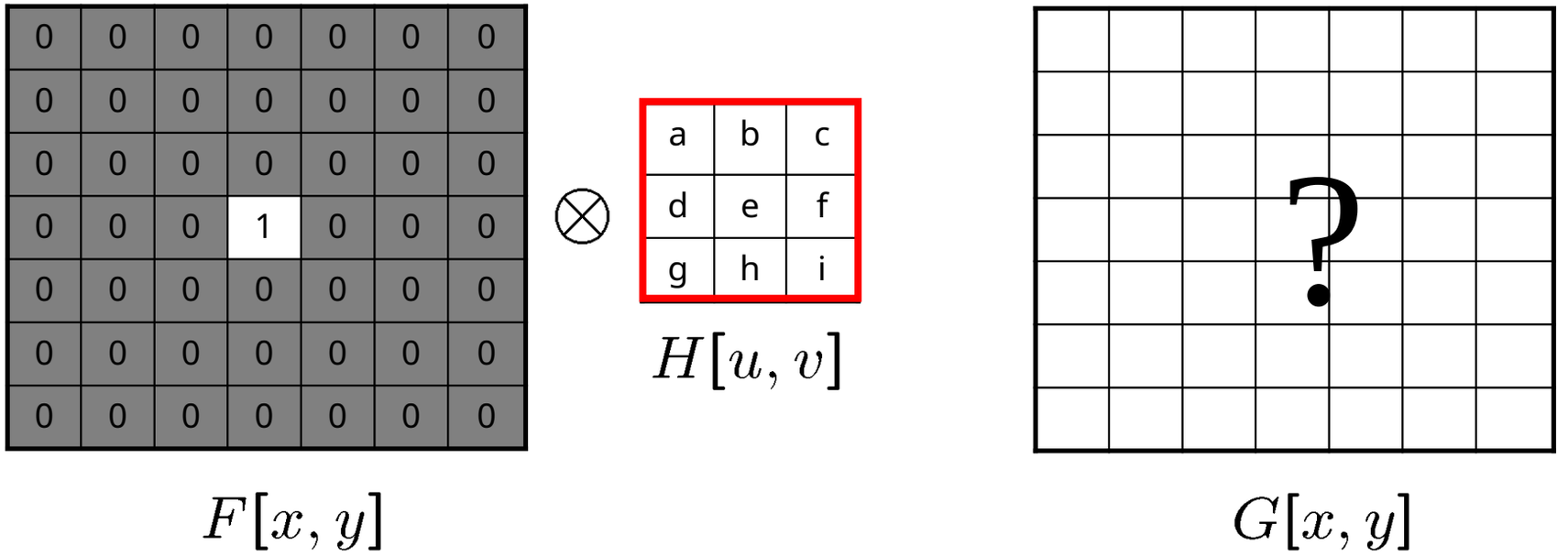
Mechanics of spatial filtering

- Handling Pixels Close to Boundaries

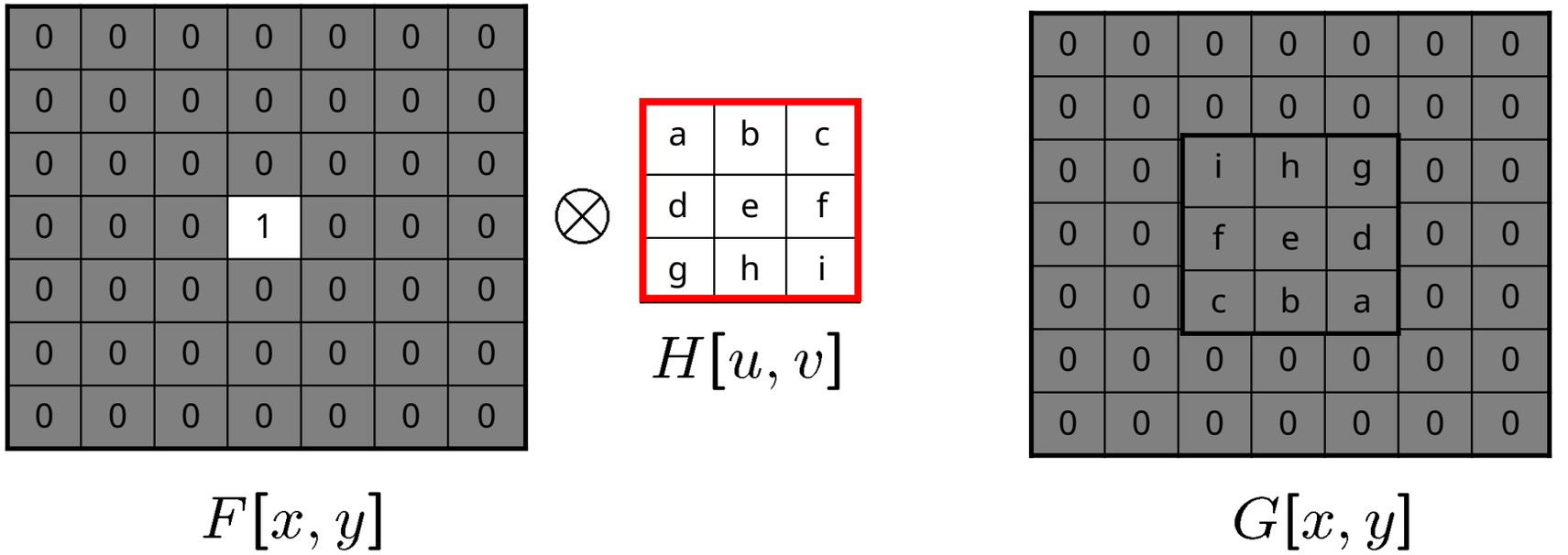


Filtering an impulse signal

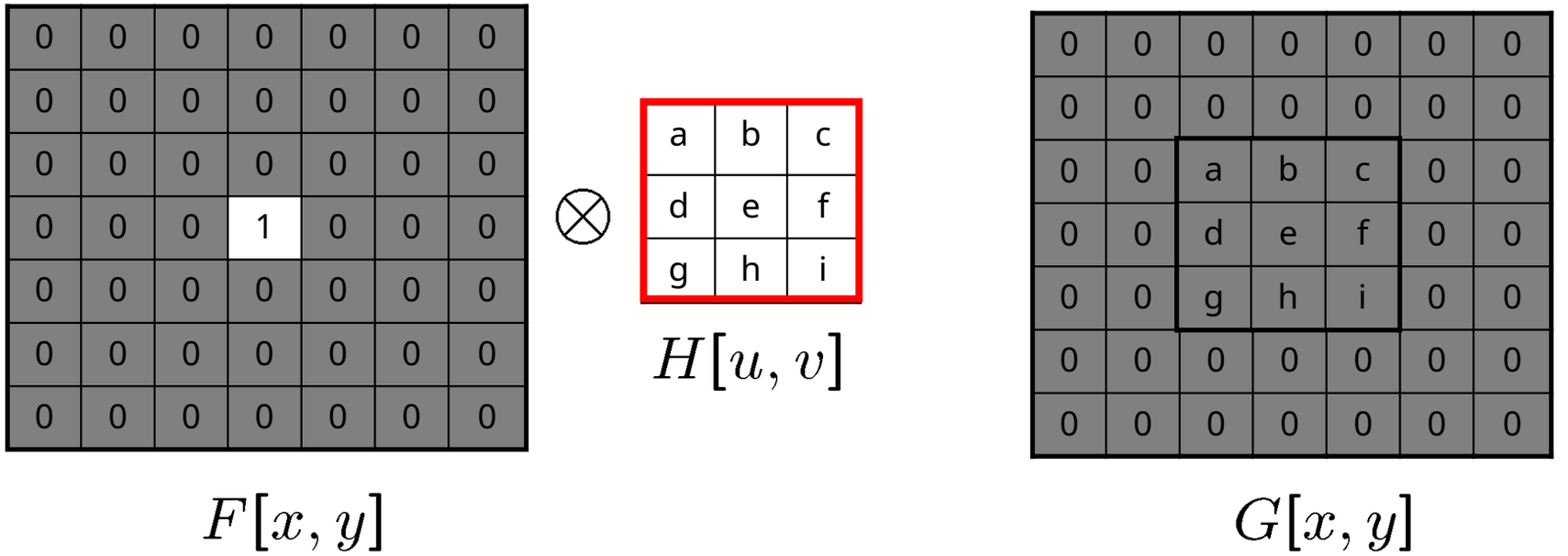
What is the result of filtering the impulse signal (image) *F* with the arbitrary kernel *H*?



Filtering an impulse signal - Correlation



Filtering an impulse signal - Convolution



Separability of 2D convolution

- If a kernel H can be expressed as multiple kernels H_i in the form

$$H = H_1 * H_2 * \ldots * H_n$$

- then ,

$$I * H = I * (H_1 * H_2 * \ldots * H_n)$$
$$= (\ldots((I * H_1) * H_2) * \ldots * H_n)$$

Applying smaller kernels H_1 H_2 ... H_N one by one computationally cheaper than apply 1 large kernel H

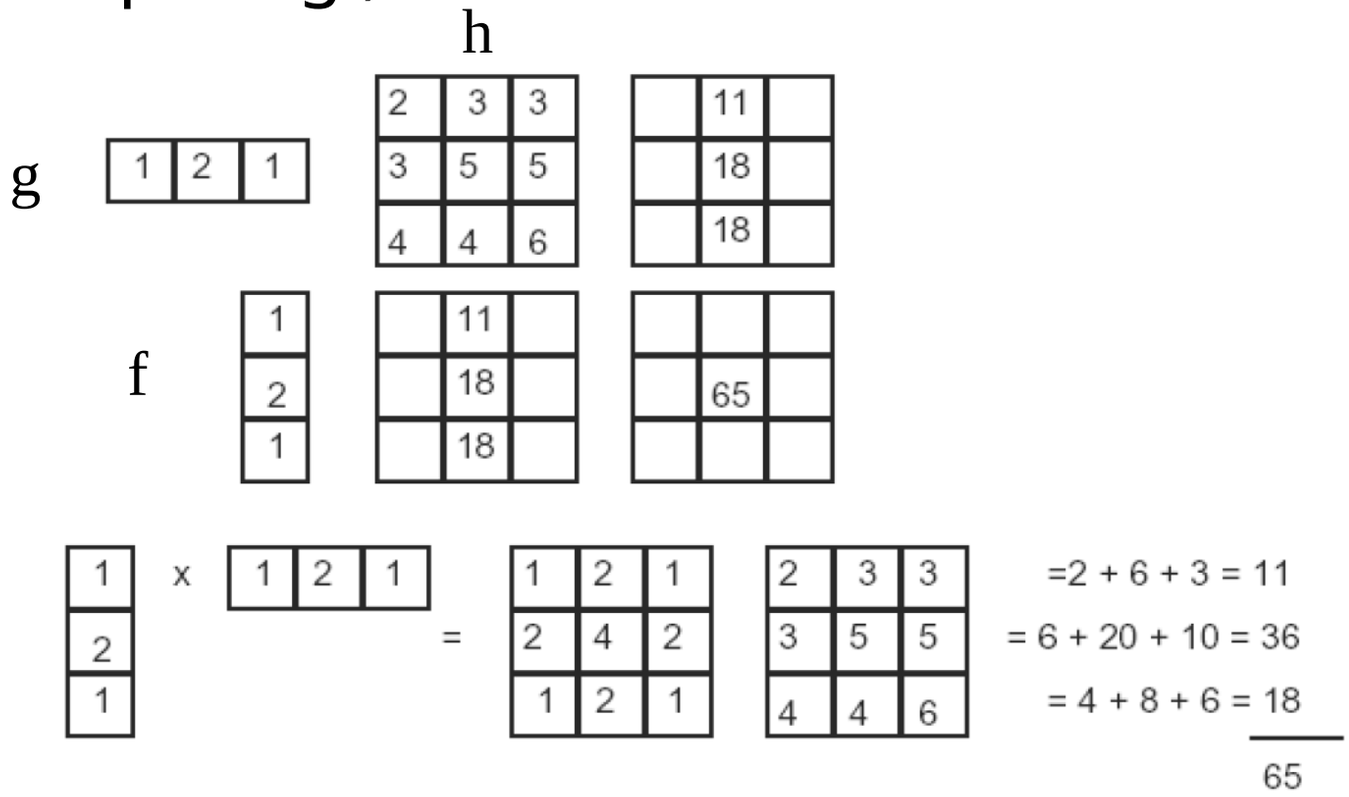
$$K_{5\times5} = \frac{1}{256}\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$

$$K_1 = K_2 = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}.$$

$$K_{5\times5} = K_1 * K_2$$

Separability of 2D convolution in x, y

- A 2D kernel is separable if it can be represented as the outer product of two 1D vectors (one representing a row and the other a column).
- If so, we can factor into two steps: e.g.,
  - Convolve all rows
  - Convolve all columns



$$f * (g * h) = (f * g) * h$$

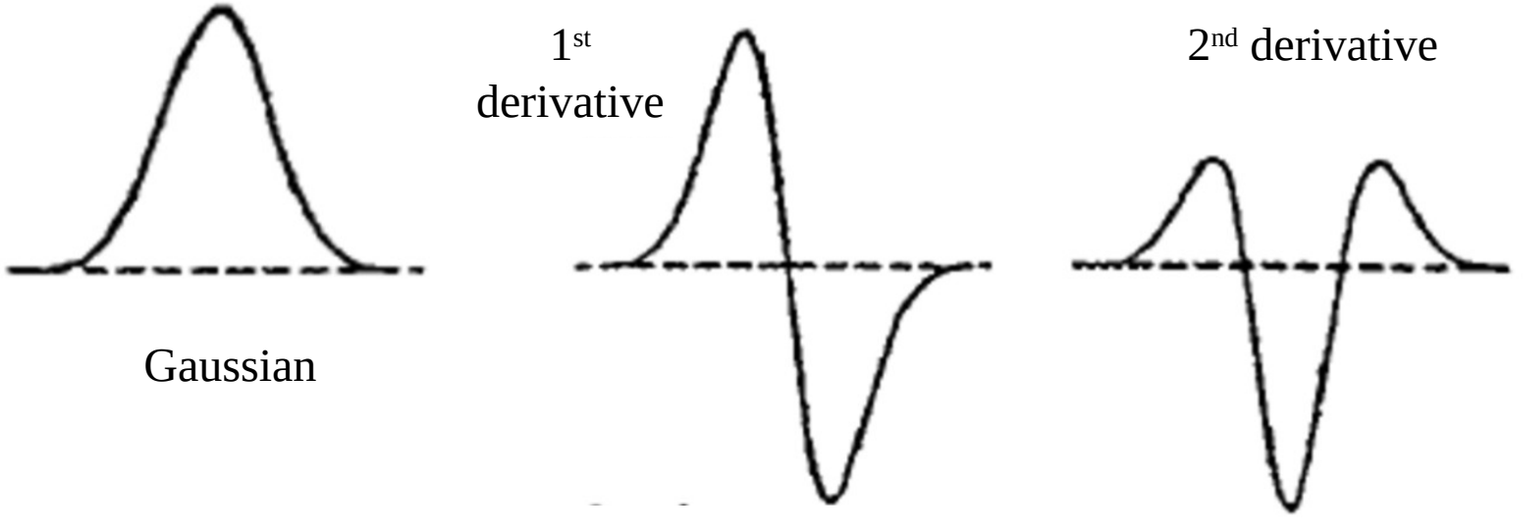
Separability - benefit

- Filtering an M-by-N image with a P-by-Q filter kernel requires roughly **MNPQ** multiplies and adds (assuming we aren't using an implementation based on the FFT).
- If the kernel is separable, you can filter in two steps.
  - The first step requires about MNP multiplies and adds.
  - The second requires about MNQ multiplies and adds
  - for a total of MN(P + Q).
- The computational advantage of separable convolution versus nonseparable convolution is therefore: **PQ/(P+Q)**
- For a 9-by-9 filter kernel, that's a theoretical speed-up of 4.5.

How do we choose the mask weights?

- Typically, by sampling certain functions:

w1	w2	w3
w4	w5	w6
w7	w8	w9



$$G(x;\sigma) = \frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{x^2}{2\sigma^2}\right).$$

$$g = [0.004432 \quad 0.053991 \quad 0.241971 \quad 0.398942 \quad 0.241971 \quad 0.053991 \quad 0.004432]$$

$$g_I = \frac{1}{1000}[4 \quad 54 \quad 242 \quad 400 \quad 242 \quad 54 \quad 4]$$

$$\frac{dG(x)}{dx} = -\frac{x}{\sigma^2}G(x).$$

$$g' = [0.013295 \quad 0.107981 \quad 0.241970 \quad 0 \quad -0.241970 \quad -0.107981 \quad -0.013295]$$

$$g'_I = [13 \quad 108 \quad 242 \quad 0 \quad -242 \quad -108 \quad -13]$$

$$\frac{d^2G(x)}{dx^2} = \frac{x^2-\sigma^2}{\sigma^4}G(x).$$

$$g'' = [0.035455 \quad 0.161973 \quad 0 \quad -0.398942 \quad 0 \quad 0.161973 \quad 0.035455].$$

How do we choose the mask weights?

$$K_G \approx \begin{bmatrix} 0.000020 & 0.000239 & 0.001073 & 0.001769 & 0.001073 & 0.000239 & 0.000020 \\ 0.000239 & 0.002917 & 0.013071 & 0.021551 & 0.013071 & 0.002917 & 0.000239 \\ 0.001073 & 0.013071 & 0.058582 & 0.096585 & 0.058582 & 0.013071 & 0.001073 \\ 0.001769 & 0.021551 & 0.096585 & 0.159241 & 0.096585 & 0.021551 & 0.001769 \\ 0.001073 & 0.013071 & 0.058582 & 0.096585 & 0.058582 & 0.013071 & 0.001073 \\ 0.000239 & 0.002917 & 0.013071 & 0.021551 & 0.013071 & 0.002917 & 0.000239 \\ 0.000020 & 0.000239 & 0.001073 & 0.001769 & 0.001073 & 0.000239 & 0.000020 \end{bmatrix}.$$

Integer Gaussian kernel

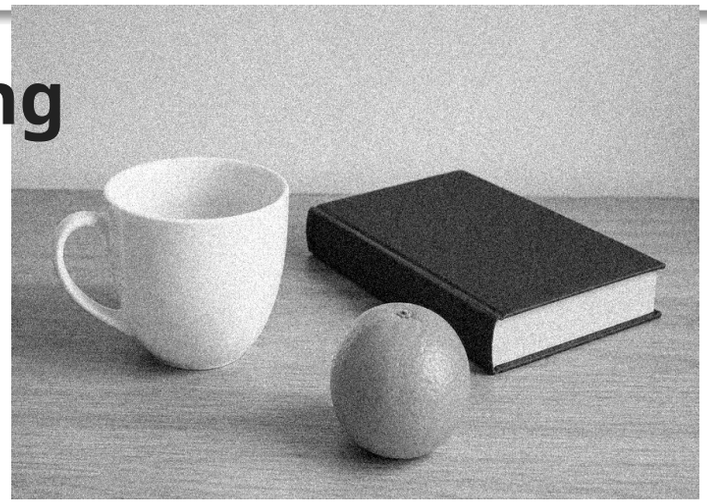
$$K_{G,I} = \frac{1}{1000}\begin{bmatrix} 0 & 0 & 1 & 2 & 1 & 0 & 0 \\ 0 & 3 & 13 & 21 & 13 & 3 & 0 \\ 1 & 13 & 59 & 97 & 59 & 13 & 1 \\ 2 & 22 & 97 & 158 & 97 & 22 & 2 \\ 1 & 13 & 59 & 97 & 59 & 13 & 1 \\ 0 & 3 & 13 & 21 & 13 & 3 & 0 \\ 0 & 0 & 1 & 2 & 1 & 0 & 0 \end{bmatrix}$$

2D Gaussian, $\sigma = 1$

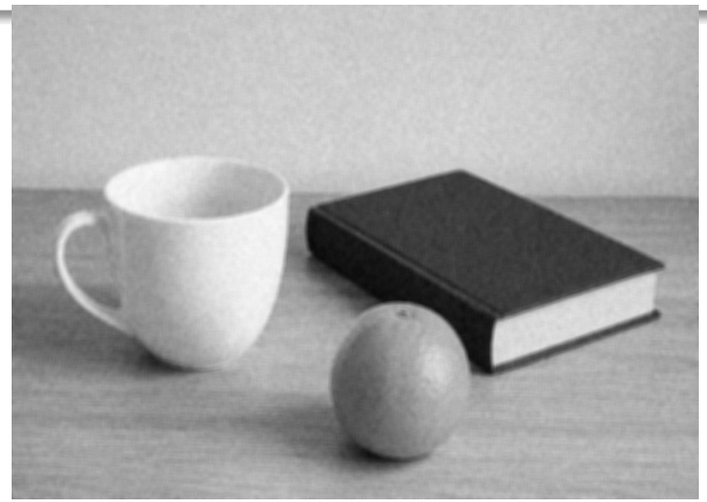
$$G(x, y) = \frac{1}{2\pi} e^{-(x^2+y^2)/2}$$

Convolution uses

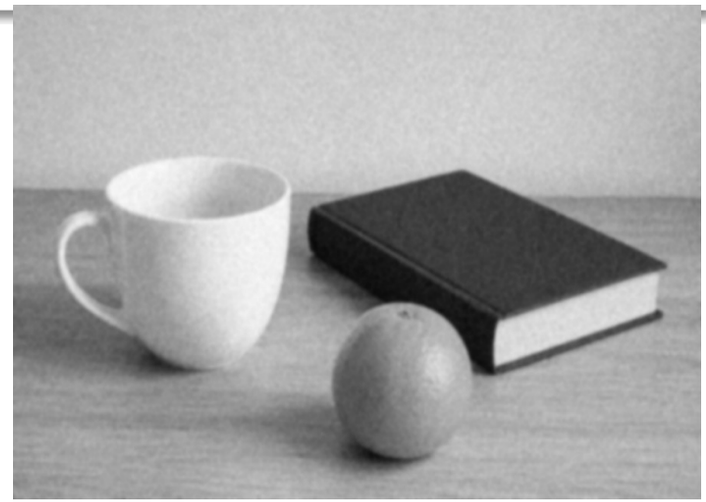
Smoothing



Noisy input



Box blur



Gaussian blur

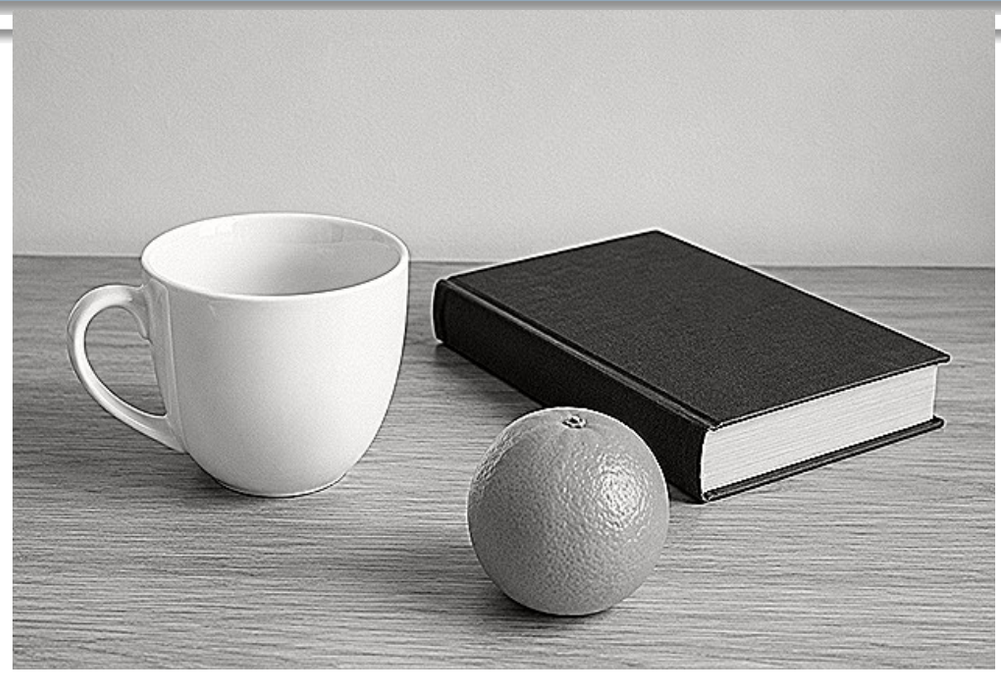
Box filter: every neighbor has equal weight

- Averaging reduces random noise because independent errors cancel out.
- Larger kernels smooth more strongly but blur edges.

Convolution uses

Sharpenin

Original



Sharpened

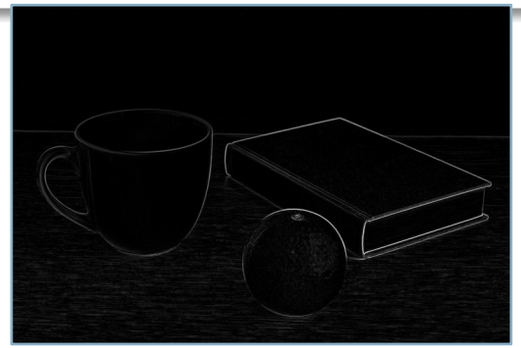
- Sharpening emphasizes local intensity changes.
- High center weight preserves the pixel; negative neighbors subtract blur.

0	**−1**	**0**
−1	5	−1
0	−1	0

Convolution uses

Edge Detection

Original



Sobel edges

−1	0	1
−2	0	2
−1	0	1

Sobel X kernel

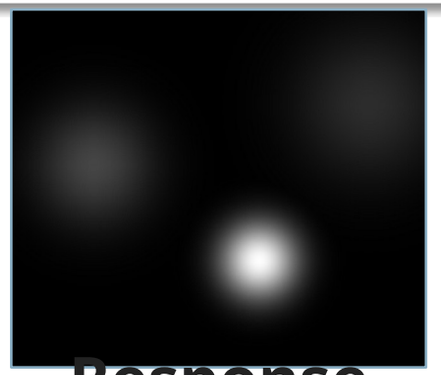
- Edges are locations with strong intensity change.
- Derivative filters respond to horizontal or vertical changes.
- Sobel combines smoothing and differentiation.

Correlation uses

Template Matching

Search image

Template



Response map

- Slide the template over the image without flipping it.
- Compute similarity at each location.
- The best match is the maximum response.

Summary

- Spatial filtering uses local neighborhoods.
- Convolution flips the kernel; correlation does not.
- 1D integrals become sums;
- 2D filtering uses nested sums.
- Convolution has useful algebraic properties.
- Core uses: smoothing, sharpening, edges, matching.

Image Enhancement

Prof. Dr. Sk. Md. Masudul Ahsan
Dept. of CSE, KUET

Learning outcomes

What students should be able to do

- Explain why local neighborhood operations enhance images in the spatial domain.
- Select suitable smoothing filters for random, Gaussian and salt-and-pepper noise.
- Apply sharpening concepts using Laplacian and unsharp masking filters.
- Describe the role of first derivative filters in edge detection.
- Compare enhancement results and identify common artifacts.

3x3 neighborhood

z1	z2	z3
z4	z5	z6
z7	z8	z9

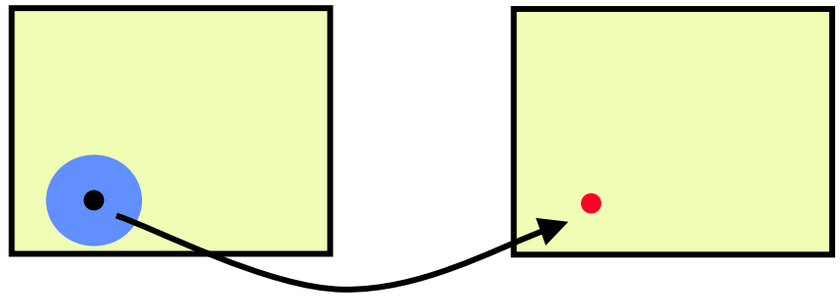
Most spatial filters are built around this local pixel neighborhood.

Where spatial enhancement fits

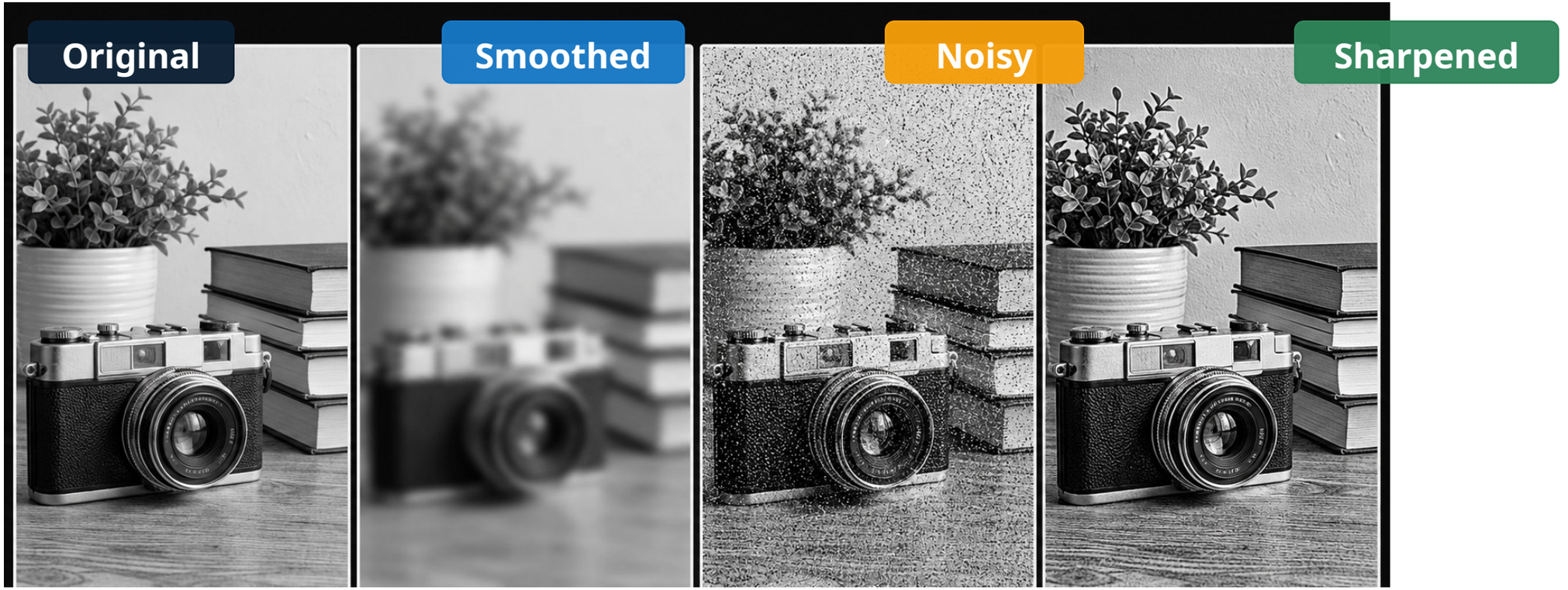
Image enhancement overview

- Spatial domain: manipulate pixel values directly.
- The output pixel depends on nearby pixels in a small window.
- Enhancement improves visibility, not necessarily physical accuracy.

$$g(x,y) = M(\{ f(i,j) | (i,j) \in N(x,y) \})$$



What enhancement tries to change



A filter is useful only when its effect matches the image problem.

Two families of spatial filters

Smoothing
Low-pass

- Reduces noise and small fluctuations
- Blurs edges and fine detail
- Examples: mean, Gaussian, median

Sharpening
High-pass

- Emphasizes rapid intensity change
- Highlights edges and fine structures
- Examples: Laplacian, USM, Sobel

Trade-off: noise suppression vs edge preservation

SMOOTHING FILTERS

Reduce noise and local fluctuations before interpretation

Smoothing Spatial Filters

- Smoothing filters are used for blurring and for noise reduction.
  - Blurring is used in preprocessing steps, such as removal of small details from an image prior to object extraction, and bridging of small gaps in lines or curves
    - Noise reduction can be accomplished by blurring
    - Reduce “sharp” transitions in intensities
    - smoothing of false contours
- There are 2 way of smoothing spatial filters
  - Linear Filters/ convolution filters
  - Order-Statistics Filters

Smoothing Linear Filters

- Simply the average of the pixels contained in the neighborhood of the mask.
- Sometimes called “averaging filters”

$$\frac{1}{9}\times\begin{bmatrix}1&1&1\\1&1&1\\1&1&1\end{bmatrix}$$

Standard average
Mean filter

$$\frac{1}{16}\times\begin{bmatrix}1&2&1\\2&4&2\\1&2&1\end{bmatrix}$$

Weighted average

$$\frac{1}{25}\times\begin{bmatrix}1&1&1&1&1\\1&1&1&1&1\\1&1&1&1&1\\1&1&1&1&1\\1&1&1&1&1\end{bmatrix}$$

$$g(x,y)=\frac{\sum_{s=-a}^{a}\sum_{t=-b}^{b}h(s,t)\,f(x-s,y-t)}{\sum_{s=-a}^{a}\sum_{t=-b}^{b}h(s,t)}$$

Replace center pixel by local average

Smoothing Linear Filters - example

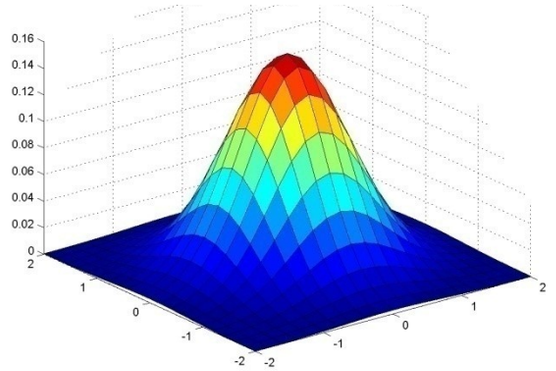
- Mask size determines the degree of smoothing (loss of detail).
  - Good for mild additive random noise.
  - Every neighbor contributes equally.
  - Problem: strong edge blurring.

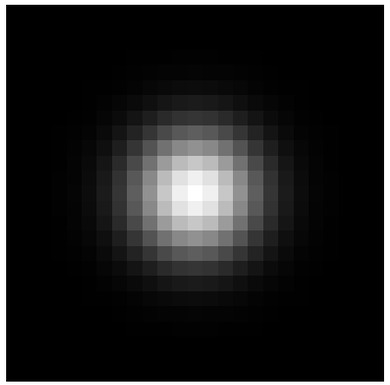
Mean filter is simple, but it cannot distinguish noise from real edges.

Smoothing filters: Gaussian filter

- Weight contributions of neighboring pixels by nearness

$$G_\sigma = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$





0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

- Gaussian weights are smooth and symmetric.
- Parameter sigma controls the amount of blur.
- Larger kernels reduce noise more but remove detail more.

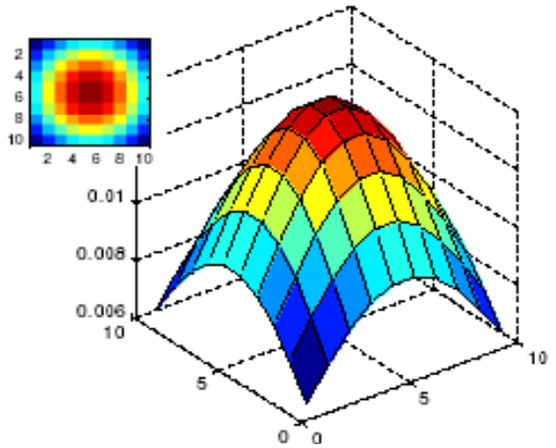
5 x 5, σ = 1

1	4	7	4	1
4	20	33	20	4
7	33	55	33	7
4	20	33	20	4
1	4	7	4	1

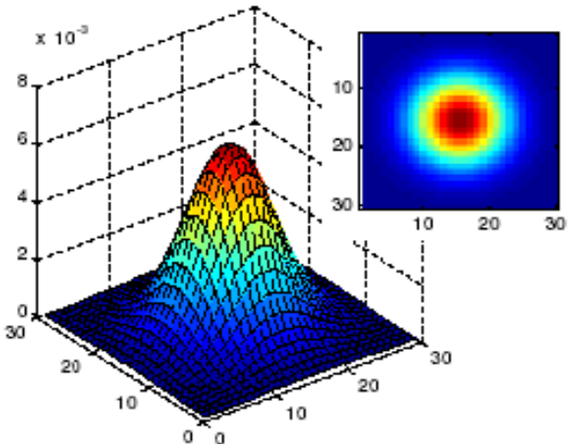
Slide credit: Christopher Rasmussen

Gaussian filters

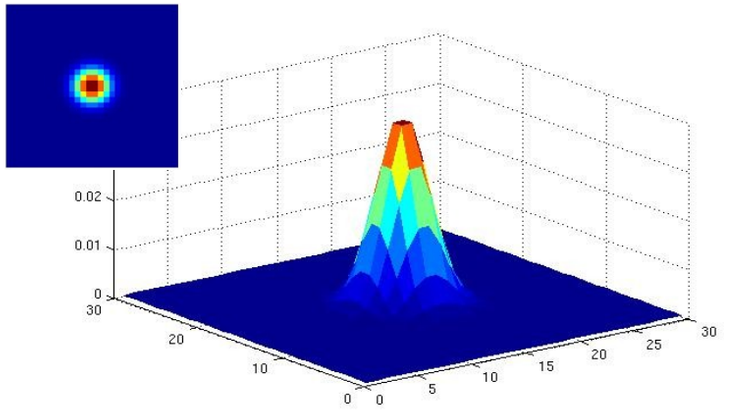
- What parameters matter here?
- **Std Deviation** of Gaussian: determines extent of smoothing



σ = 5 with 10 x 10 kernel



σ = 5 with 30 x 30 kernel



σ = 2 with 30 x 30 kernel

Gaussian filters

- Remove “high-frequency” components from the image (low-pass filter)
  - Images become more smooth
- Convolution with self is another Gaussian
  - **Prove it.**
  - So can smooth with small-width kernel, repeat, and get same result as larger-width kernel would have
  - Convolving two times with Gaussian kernel of width σ is same as convolving once with kernel of width $\sigma\sqrt{2}$
- *Separable* kernel
  - Factors into product of two 1D Gaussians

Source: K.

Convolution of two gaussians is gaussian

$$g_1(x) = \frac{1}{\sqrt{2\pi}\sigma_1}\exp\Big(-\frac{(x-\mu_1)^2}{2\sigma_1^2}\Big), \qquad g_2(x) = \frac{1}{\sqrt{2\pi}\sigma_2}\exp\Big(-\frac{(x-\mu_2)^2}{2\sigma_2^2}\Big)$$

$$(g_1 * g_2)(x) = \int_{-\infty}^{\infty} g_1(x-y)\, g_2(y)\, dy$$

$$(g_1 * g_2)(x) = \frac{1}{2\pi\sigma_1\sigma_2}\int_{-\infty}^{\infty}\exp\Big(-\frac{(x-y-\mu_1)^2}{2\sigma_1^2} - \frac{(y-\mu_2)^2}{2\sigma_2^2}\Big)\, dy$$

$$-\frac{(x-y-\mu_1)^2}{2\sigma_1^2} - \frac{(y-\mu_2)^2}{2\sigma_2^2} = -A\left(y - \frac{B}{A}\right)^2 + \frac{B^2}{A} - \frac{(x-\mu_1)^2}{2\sigma_1^2} - \frac{\mu_2^2}{2\sigma_2^2}$$

Where,

$$A = \frac{1}{2\sigma_1^2} + \frac{1}{2\sigma_2^2}, \qquad B = \frac{x-\mu_1}{2\sigma_1^2} + \frac{\mu_2}{2\sigma_2^2}$$

$$(g_1 * g_2)(x) = \frac{1}{2\pi\sigma_1\sigma_2}\exp\Big(\frac{B^2}{A} - \frac{(x-\mu_1)^2}{2\sigma_1^2} - \frac{\mu_2^2}{2\sigma_2^2}\Big)\int_{-\infty}^{\infty}\exp\Big(-A\Big(y - \frac{B}{A}\Big)^2\Big)\, dy.$$

$$\int_{-\infty}^{\infty} e^{-a(x+b)^2}\, dx = \sqrt{\frac{\pi}{a}}.$$

The integral equals $\sqrt{\pi/A}$. Simplifying the constants and the exponent (straightforward but a bit of algebra) yields the closed form

$$(g_1 * g_2)(x) = \frac{1}{\sqrt{2\pi(\sigma_1^2+\sigma_2^2)}}\exp\Big(-\frac{(x-(\mu_1+\mu_2))^2}{2(\sigma_1^2+\sigma_2^2)}\Big).$$

So the convolution is again a Gaussian with mean $\mu = \mu_1 + \mu_2$ and variance $\sigma^2 = \sigma_1^2 + \sigma_2^2$.

Separability of the Gaussian filter

$$G_\sigma(x,y) = \frac{1}{2\pi\sigma^2}\exp^{-\frac{x^2+y^2}{2\sigma^2}}$$

$$= \left(\frac{1}{\sqrt{2\pi}\sigma}\exp^{-\frac{x^2}{2\sigma^2}}\right)\left(\frac{1}{\sqrt{2\pi}\sigma}\exp^{-\frac{y^2}{2\sigma^2}}\right)$$

The 2D Gaussian can be expressed as the product of two functions, one a function of x and the other a function of y

In this case, the two functions are the (identical) 1D Gaussian

Source: D.

Smoothing filters: Box filter

- One extreme value strongly affects the mean.
- This is why averaging is weak against impulse noise.

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$F[x, y]$

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$H[u, v]$

-	-	-	-	-	-	-	-	-	-
-	0	10	20	30	30	30	20	10	-
-	0	20	40	60	60	60	40	20	-
-	0	30	60	90	90	90	60	30	-
-	0	30	50	80	80	90	60	30	-
-	0	30	50	80	80	90	60	30	-
-	0	20	30	50	50	60	40	20	-
-	10	20	30	30	30	30	20	10	-
-	10	10	10	0	0	0	0	0	-
-	-	-	-	-	-	-	-	-	-

Source: S. Seitz

Smoothing filters: Gaussian filter

- This kernel is an approximation of a Gaussian function:

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$F[x, y]$

$$\frac{1}{16}$$

1	2	1
2	4	2
1	2	1

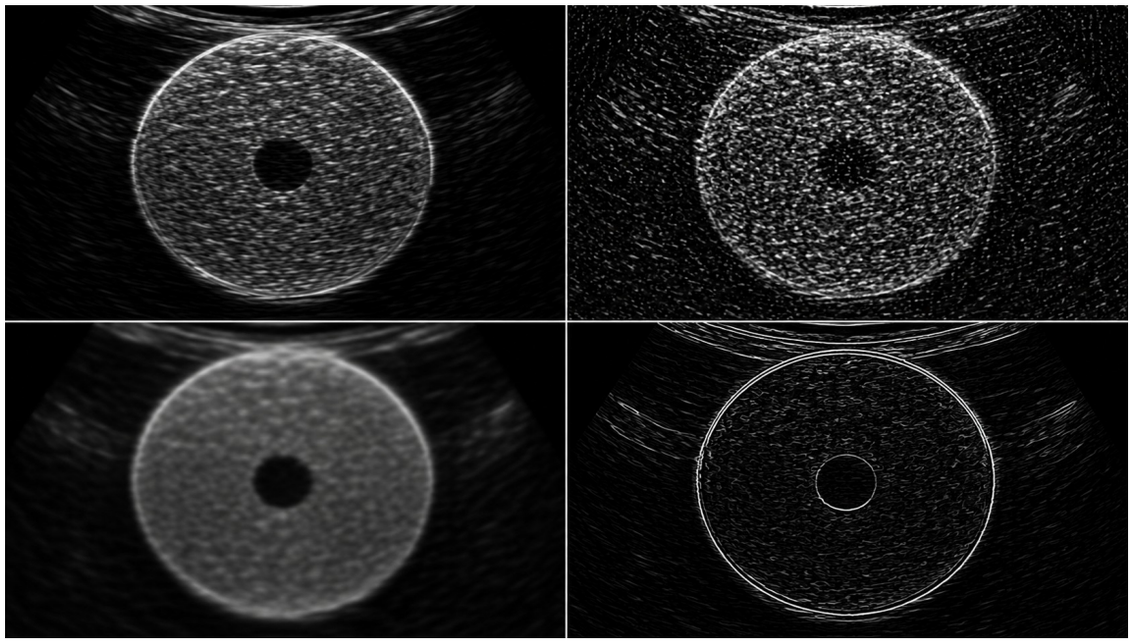
$H[u, v]$

-	-	-	-	-	-	-	-	-	-
-	0	6	17	23	23	23	17	6	-
-	0	17	51	68	68	68	51	17	-
-	0	23	68	90	90	90	68	23	-
-	0	23	62	79	84	90	68	23	-
-	0	23	56	68	79	90	68	23	-
-	0	17	45	56	62	68	51	17	-
-	6	17	23	23	23	23	17	6	-
-	11	23	11	0	0	0	0	0	-
-	-	-	-	-	-	-	-	-	-

Source: S. Seitz

When smoothing helps and when it hurts

Interpretation with visual examples



- Helps: random speckle, acquisition noise, pre-processing before segmentation.
- Hurts: thin lines, boundaries, small lesions or small objects.
- Always inspect edges after smoothing.

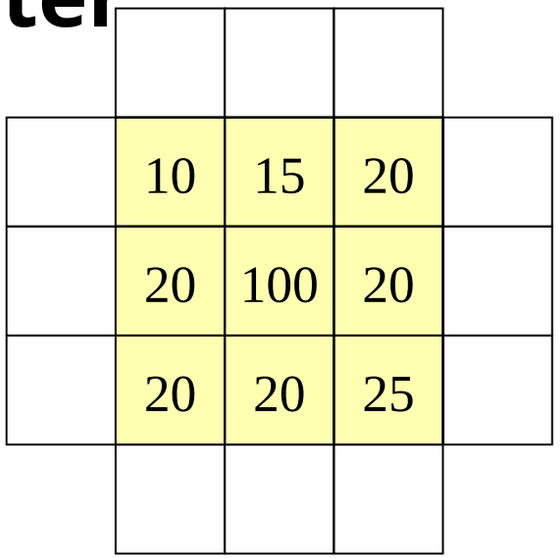
Smoothing is not simply "making the image nicer"; it changes measurements.

Nonlinear spatial filtering

- also operate on neighborhoods, and the mechanics of sliding a mask past an image are the same.
- **Order-Statistics Filters**
  - response is based on ordering (ranking) of the pixels contained in the filter area
- Best-known “median filter”
  - Good noise-reduction capabilities with less smoothing (e.g. impulse noise, or salt-and-pepper noise)
  - Edges are often preserved better than with mean filtering

Non-linear smoothing

Median filter



10, 15, 20, 20, 20, 20, 20, 25, 100

5th

$$g(x,y) = \underset{(s,t)\in W_{(x,y)}}{median}\{f(s,t)\}$$

Crop region of neighborhood
Sort the values of the pixel in our region
In the mxn mask the median is (mxn div 2)+1

It is widely used as it is very effective at removing noise while **preserving edges**.
It is particularly effective at removing 'salt and pepper' type noise.

Smoothing filters: Median filter

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$F[x, y]$

-	-	-	-	-	-	-	-	-	-
-	0	0	0	0	0	0	0	0	-
-	0	0	0	90	90	90	0	0	-
-	0	0	90	90	90	90	90	0	-
-	0	0	90	90	90	90	90	0	-
-	0	0	90	90	90	90	90	0	-
-	0	0	0	90	90	90	0	0	-
-	0	0	0	0	0	0	0	0	-
-	0	0	0	0	0	0	0	0	-
-	-	-	-	-	-	-	-	-	-

Source: S. Seitz

Box filter

-	-	-	-	-	-	-	-	-	-
-	0	10	20	30	30	30	20	10	-
-	0	20	40	60	60	60	40	20	-
-	0	30	60	90	90	90	60	30	-
-	0	30	50	80	80	90	60	30	-
-	0	30	50	80	80	90	60	30	-
-	0	20	30	50	50	60	40	20	-
-	10	20	30	30	30	30	20	10	-
-	10	10	10	0	0	0	0	0	-
-	-	-	-	-	-	-	-	-	-

Source: S. Seitz

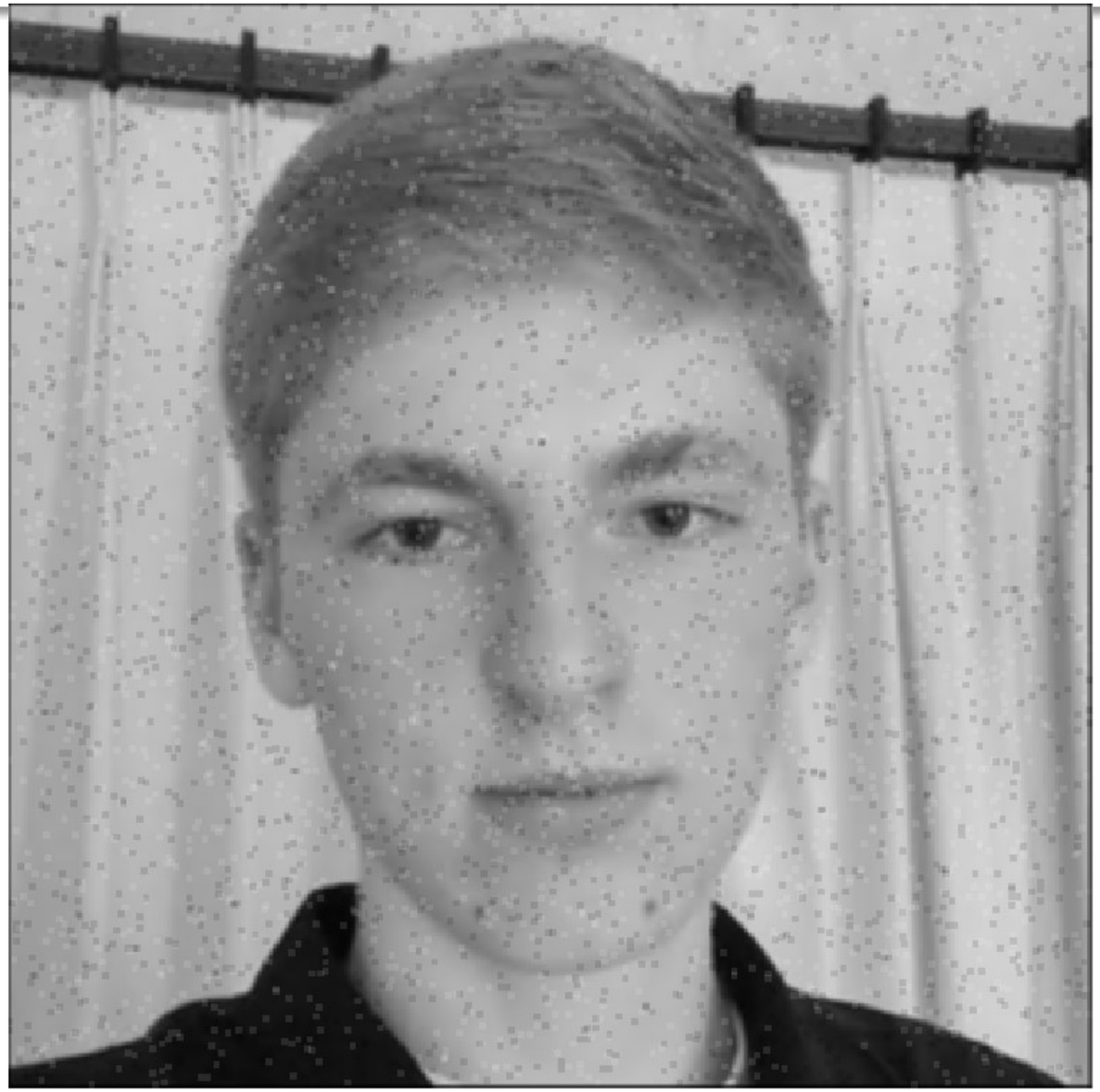
Gaussian filter

-	-	-	-	-	-	-	-	-	-
-	0	6	17	23	23	23	17	6	-
-	0	17	51	68	68	68	51	17	-
-	0	23	68	90	90	90	68	23	-
-	0	23	62	79	84	90	68	23	-
-	0	23	56	68	79	90	68	23	-
-	0	17	45	56	62	68	51	17	-
-	6	17	23	23	23	23	17	6	-
-	11	23	11	0	0	0	0	0	-
-	-	-	-	-	-	-	-	-	-

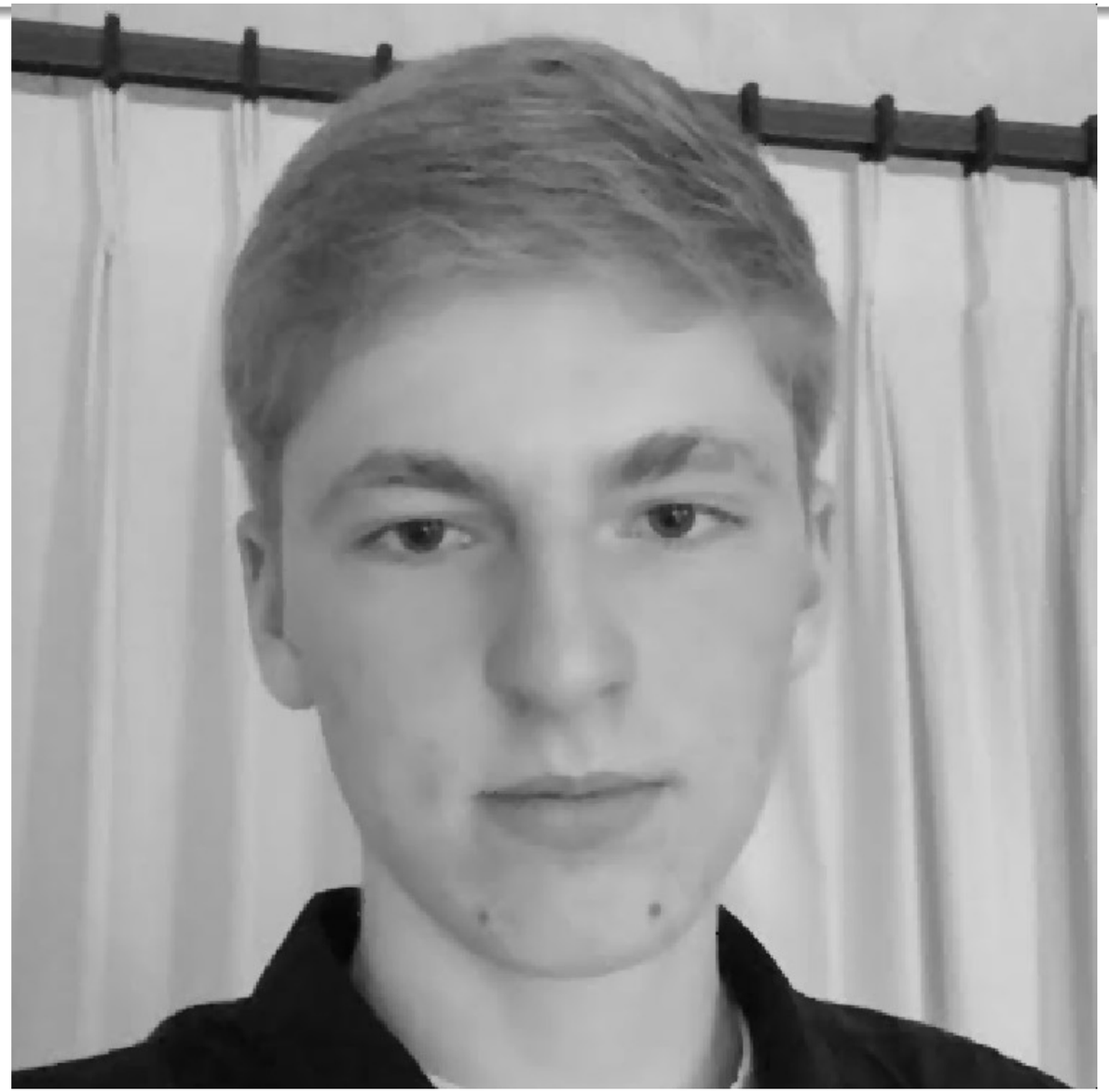
Source: S. Seitz

Noise – Salt and Pepper Jack

Mean Jack – 3 x 3 filter



Median Jack – 3 x 3



- Median removes isolated extremes without averaging them into neighbors.
- Use 3x3 first; increase size only if needed.

Median vs mean: quick decision rule

Which filter should I choose?

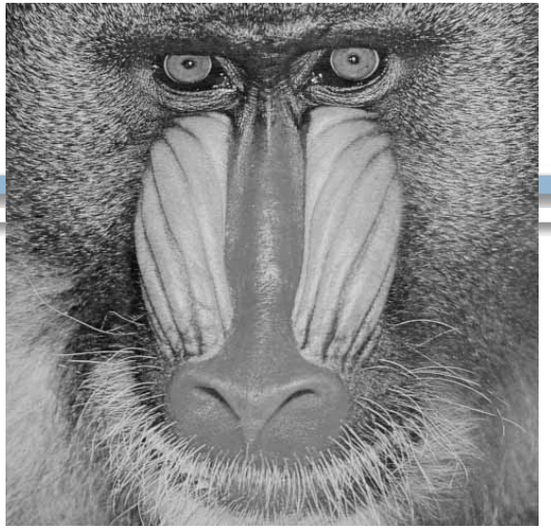
Image issue	Better choice	Why
Gaussian-like noise	Mean / Gaussian	Averages small variations
Salt-and-pepper noise	Median	Rejects extremes
Important thin edges	Median / Gaussian	Less edge spreading
Need smooth background	Gaussian	Controlled blur

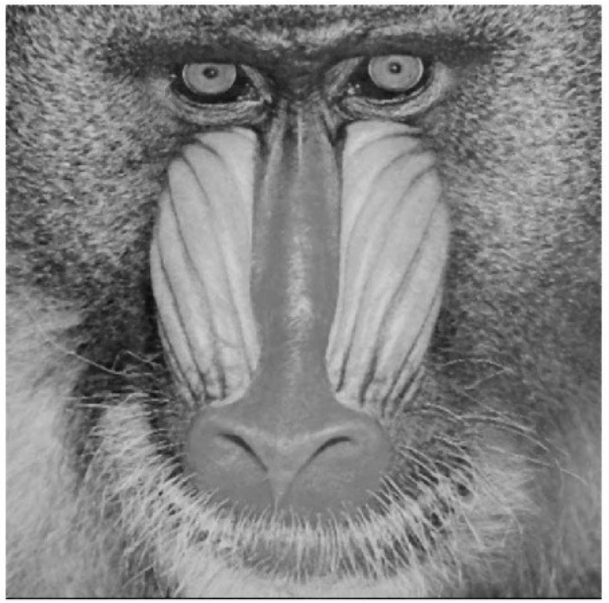
Do not use a large filter just because the noise is visible.

Order statistics filters

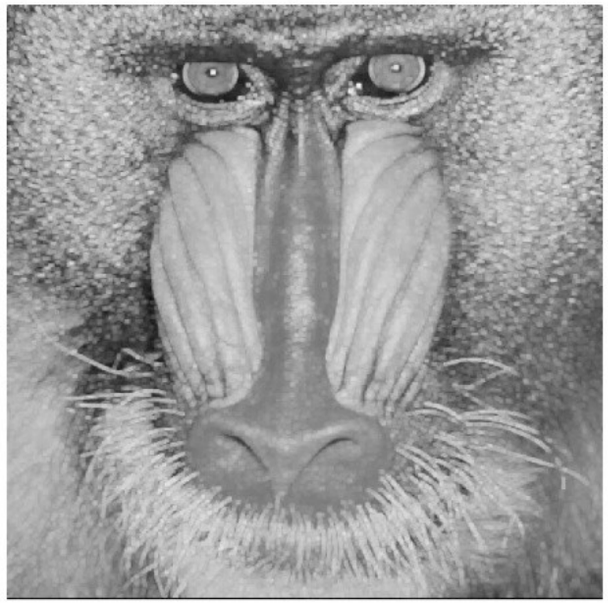
- Other order statistics:
  - Min filter, max filter, x-percentile …

original

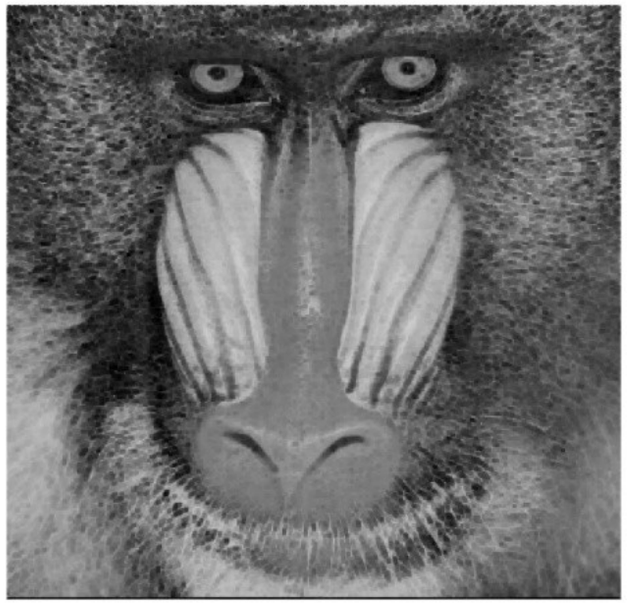




$$g(x,y) = \underset{(s,t)\in W_{(x,y)}}{median}\{f(s,t)\}$$



$$g(x,y) = \min_{(s,t)\in W_{(x,y)}}\{f(s,t)\}$$



$$g(x,y) = \max_{(s,t)\in W_{(x,y)}}\{f(s,t)\}$$

SHARPENING FILTERS

Highlight edges, boundaries and small image details

Sharpening Spatial Filters

- highlight fine detail in an image or to enhance detail that has been blurred
  - either in error or as an natural effect of a particular method of image acquisition.
- Normally, sharpening is done by highlighting the transitions in intensity
- Methods
  - Unsharp masking
  - High Boost filtering
  - Laplacian (2nd order derivative)
  - Gradient (1st order derivative)

- increases contrast around edges and fine structures.
- often increases noise as well, because noise is high-frequency.
- Good sharpening is controlled, not excessive.

High-pass content = image detail + edges + noise

Unsharp masking and High boost Filtering

1. Blur the original image f(x,y)
2. Subtract the blurred image $\bar{f}(x,y)$ from the original (result called the mask):

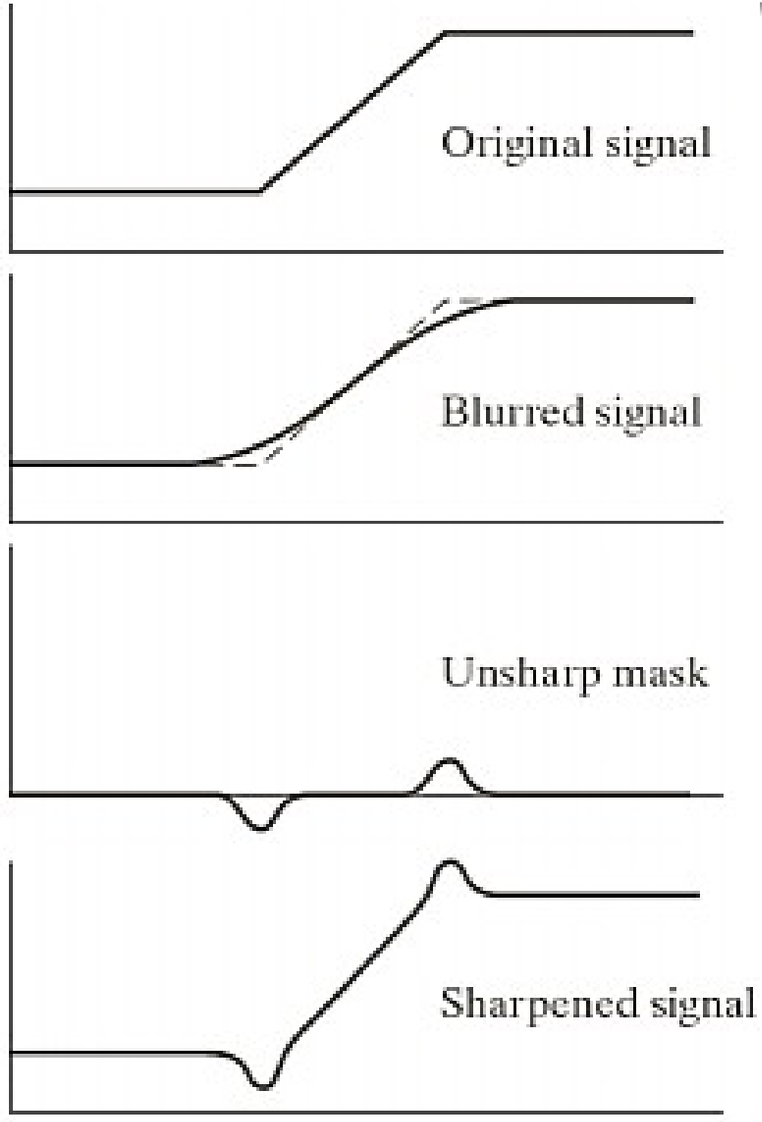
$$g_{mask}(x,y) = f(x,y) - \bar{f}(x,y)$$

3. Add the mask to the original:

$$g(x,y) = f(x,y) + k \times g_{mask}(x,y)$$

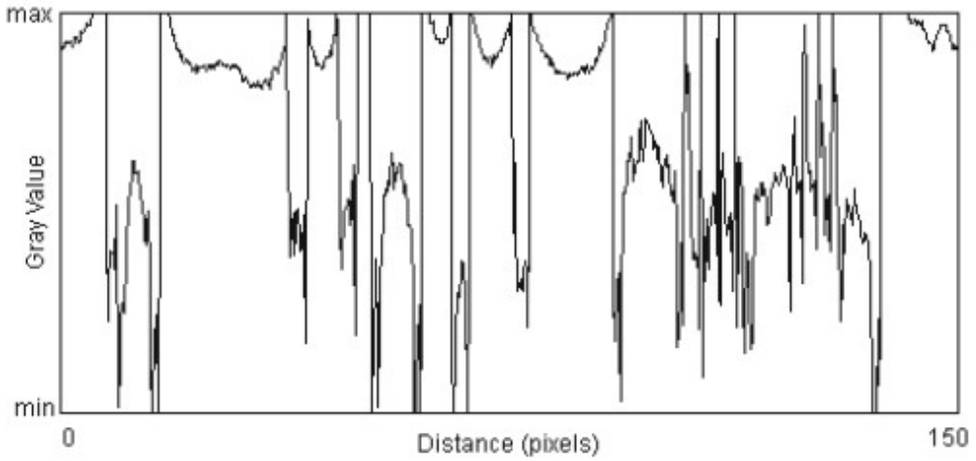
$$\check{I} \leftarrow I + a \cdot (I - \tilde{I}) = (1+a) \cdot I - a \cdot \tilde{I}.$$

- k = 1 => sharp signal/image
- k > 1 => highboost filtering



USM Example

Gaussian blurring kernel



(c) $\sigma = 10.0$

Finite differences

definition of a derivative using **forward difference**

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

$$f(x) = f(x+1) - f(x)$$

$$h = 1$$

Alternative: use central difference

$$f'(x) = \lim_{h \to 0} \frac{f(x+0.5h) - f(x-0.5h)}{h}$$

For discrete signals: Remove limit and set h = 2

$$f'(x) = \frac{f(x+1) - f(x-1)}{2}$$

What convolution kernel does this correspond to?

-1	0	1	?

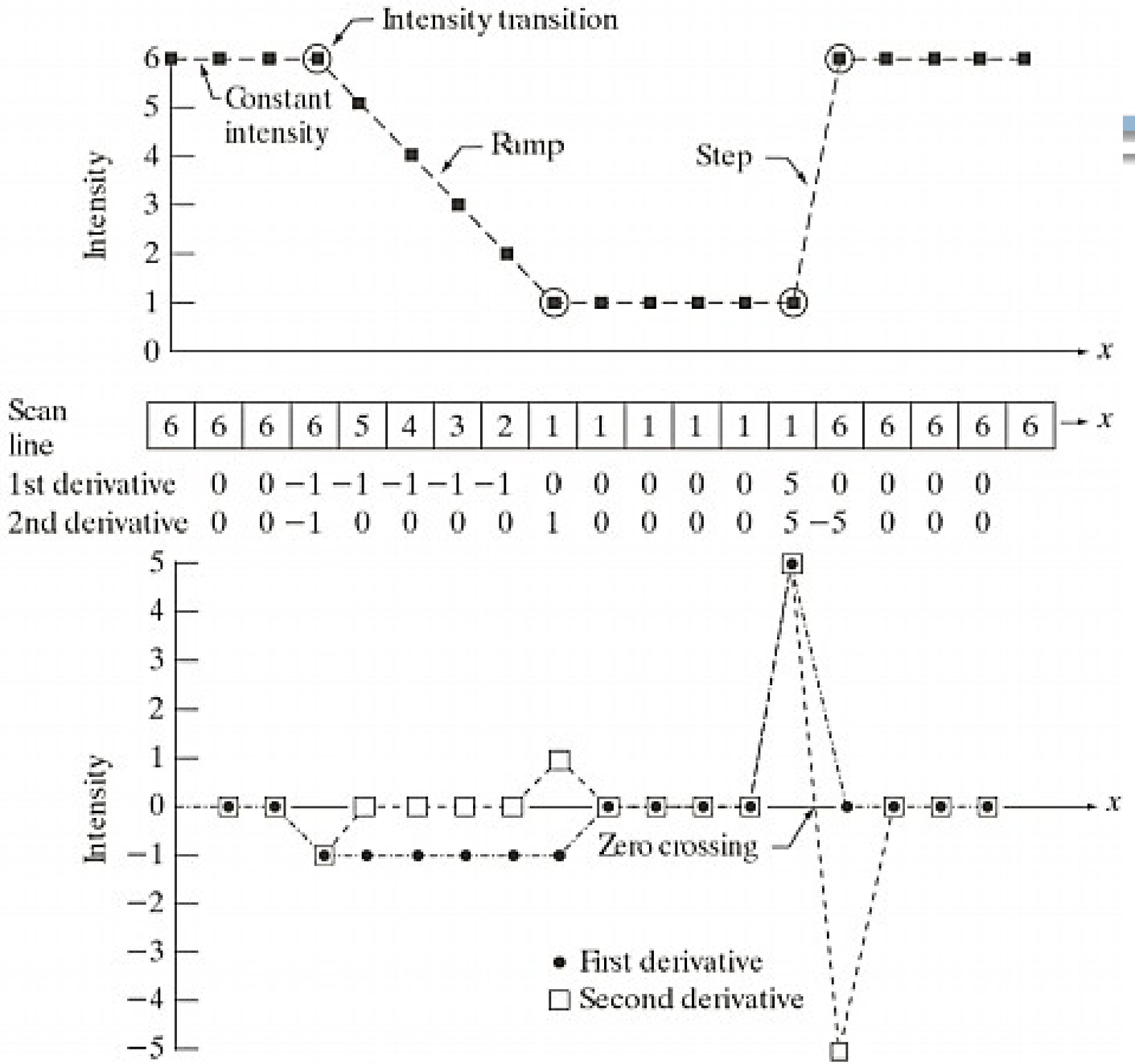
1	0	-1	?

$$\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x).$$

1	-2	1

Foundation

1st der. – forward
2nd der. – central



The Laplacian (2nd order derivative)

Shown by Rosenfeld and Kak[1982] that the simplest isotropic derivative operator is the **Laplacian** is defined as

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Discrete form of derivative

$$\frac{\partial^2 f}{\partial x^2} = f(x+1, y) + f(x-1, y) - 2f(x, y)$$

$$\frac{\partial^2 f}{\partial y^2} = f(x, y+1) + f(x, y-1) - 2f(x, y)$$

$f(x-1,y)$ $f(x,y)$ $f(x+1,y)$

$f(x,y-1)$

$f(x,y)$

$f(x,y+1)$

$$\nabla^2 f = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$

0	0	0
1	-2	1
0	0	0

+

0	1	0
0	-2	0
0	1	0

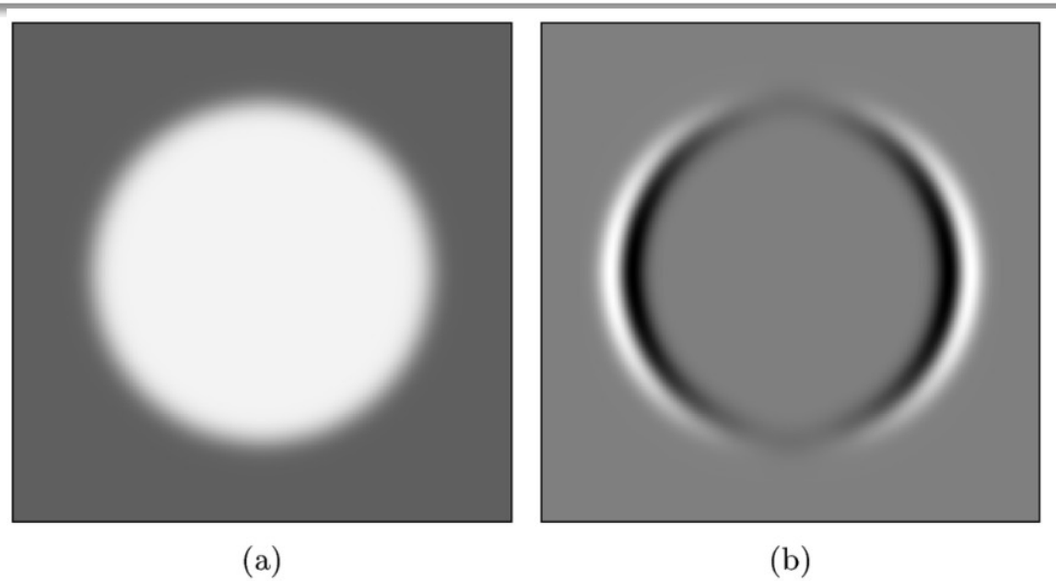
=

0	1	0
1	-4	1
0	1	0

- Sum of mask coefficients is zero.
- Flat areas become near zero.
- Edges and noise produce strong responses.

The Laplacian - mask

Synthetic test Image



Second partial derivative in horizontal direction

Second partial derivative in Vertical direction

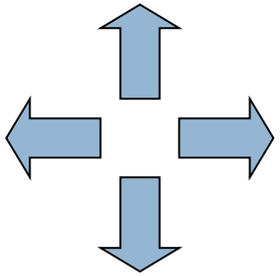
Laplace filter

Laplacian – mask variants

Isotropic mask with 90° rotation increment

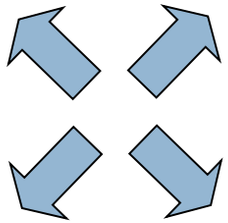
0	1	0
1	-4	1
0	1	0

0	-1	0
-1	4	-1
0	-1	0



1	0	1
0	-4	0
1	0	1

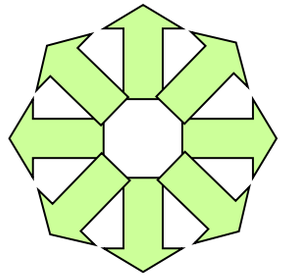
-1	0	-1
0	4	0
-1	0	-1



Isotropic mask with 45° rotation increment

1	1	1
1	-8	1
1	1	1

-1	-1	-1
-1	8	-1
-1	-1	-1



$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & -12 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Sharpening using Laplacian

$$g(x,y) = \begin{cases} f(x,y) - w.\nabla^2 f(x,y) & \text{If the center coefficient is negative} \\ f(x,y) + w.\nabla^2 f(x,y) & \text{If the center coefficient is positive} \end{cases}$$

Where

- f(x,y) is the original image
- $\nabla^2 f$ is Laplacian filtered image
- g(x,y) is the sharpen image

w is sharpening strength

-1	-1	-1
-1	9	-1
-1	-1	-1

One step convolution mask

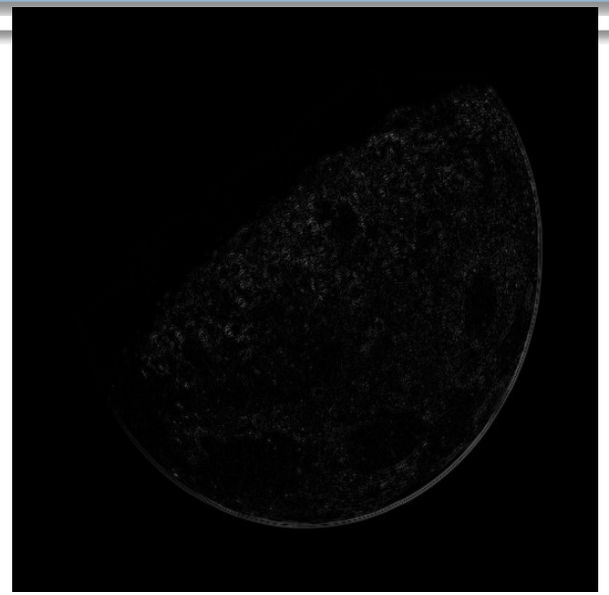
$$g(x,y) = f(x,y) - f(x+1,y) - f(x-1,y) - f(x,y+1) - f(x,y-1) + 4f(x,y)$$

$$= 5f(x,y) - f(x+1,y) - f(x-1,y) - f(x,y+1) - f(x,y-1)$$

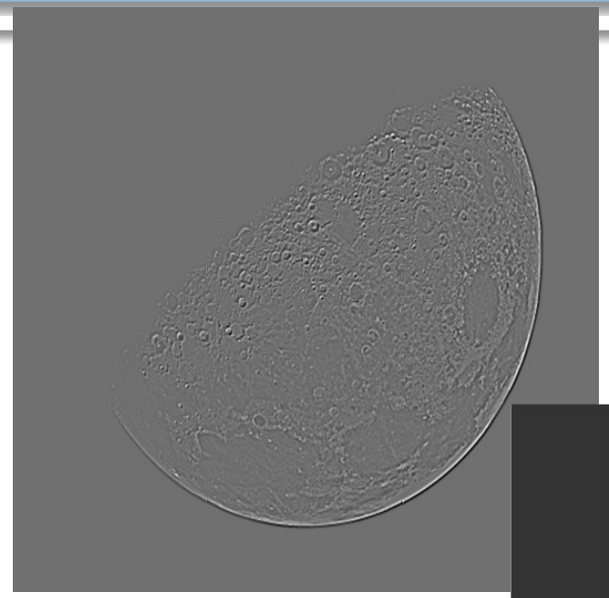
0	-1	0
-1	5	-1
0	-1	0

Laplacian - Example

Original



without scaling



Laplacian

with scaling

Sharpened

0	-1	0
-1	4	-1
0	-1	0

Laplace vs. USM filter

$$H^{\mathrm{L}} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix} - 5 \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} = 5 \cdot (\tilde{H}^{\mathrm{L}} - \delta)$$

Laplace sharpening

$$\begin{aligned} \check{I}_L &\leftarrow I - w \cdot (H^{\mathrm{L}} * I) \qquad = I - w \cdot \left(5(\tilde{H}^{\mathrm{L}} - \delta) * I\right) \\ &= I - 5w \cdot (\tilde{H}^{\mathrm{L}} * I - I) = I + 5w \cdot (I - \tilde{H}^{\mathrm{L}} * I) \\ &= I + 5w \cdot M^{\mathrm{L}}, \end{aligned}$$

Which is in the form of a USM filter

$$\check{I} \leftarrow I + a \cdot M$$

Laplacian sharpening is thus a special case of a USM filter

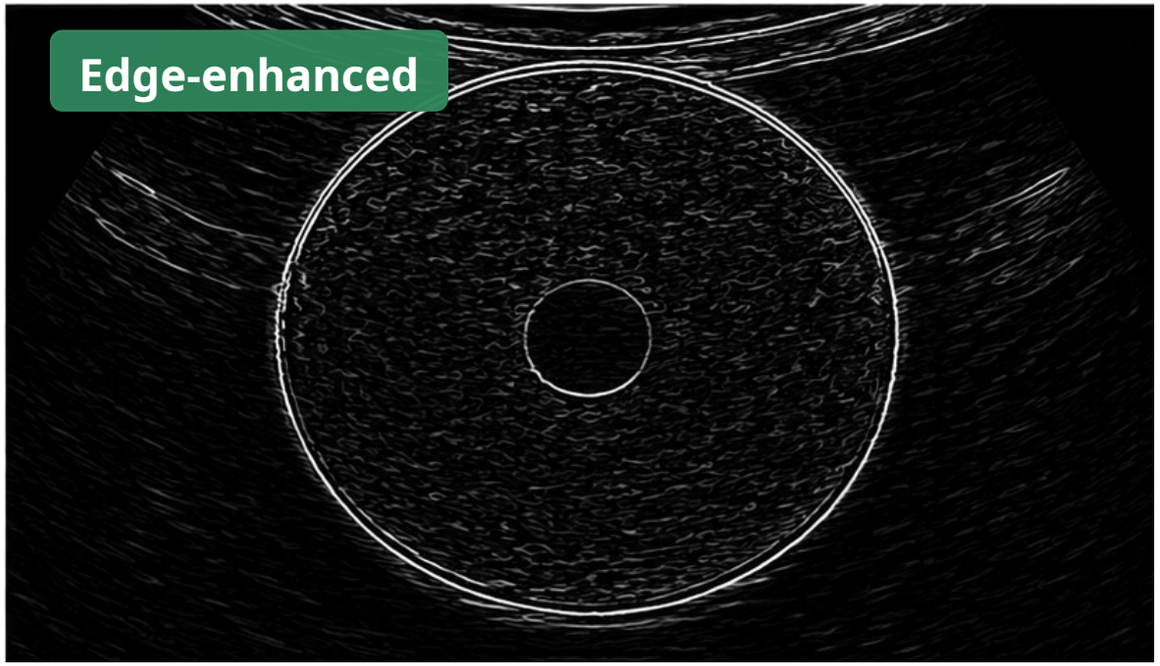
$$M = M^{L} = (I - \tilde{H}^{L} * I)$$

sharpening factor a = 5w

$$\tilde{H}^{\mathrm{L}} = \frac{1}{5} \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Laplacian filtering: strengths and limits

Second derivative behavior



- Useful for making fine structures more visible.
- Very sensitive to noise; smoothing before Laplacian may help.
- May produce double-edge effects around step transitions.

Laplacian is powerful, but it needs careful scaling and visual inspection.

The Gradient (1st order derivative)

The gradient of function f(x,y) is

$$\nabla f = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

The magnitude of this vector

$$mag(\nabla f) = \sqrt{G_x^2 + G_y^2}$$

Gradient is a linear but not rotation invariant (isotropic)
Magnitude is not linear, but is isotropic
Approximation

$$mag(\nabla f) \approx |G_x| + |G_y|$$

Isotropic property lost
(in general)

The Sobel filter

Horizontal Sober filter:

1	0	-1
2	0	-2
1	0	-1

=

1
2
1

*

1	0	-1

Vertical Sobel filter:

1	2	1
0	0	0
-1	-2	-1

=

1
0
-1

*

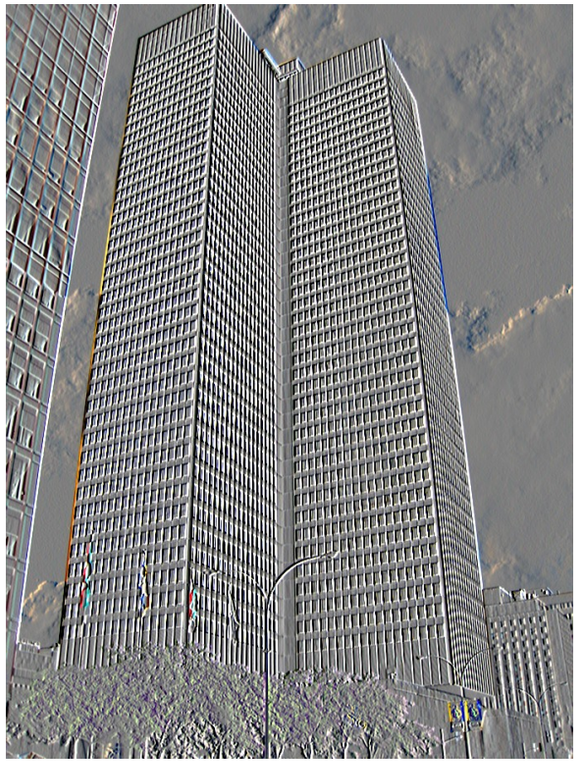
1	2	1

local gradient components are obtained from
the filter results by appropriate scaling

$$\nabla I(u,v) \approx \frac{1}{8} \cdot \begin{pmatrix} (I * H_x^{\mathrm{S}})(u,v) \\ (I * H_y^{\mathrm{S}})(u,v) \end{pmatrix}$$

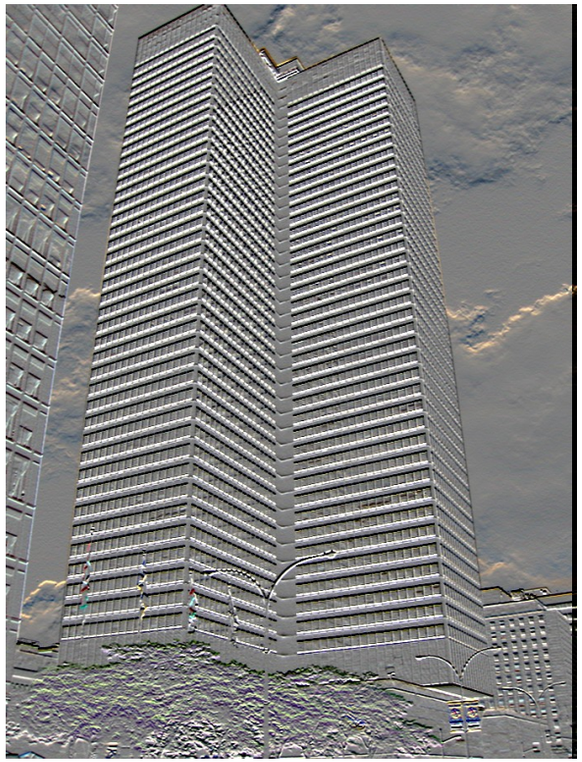
Sobel filter example

original



which Sobel filter?

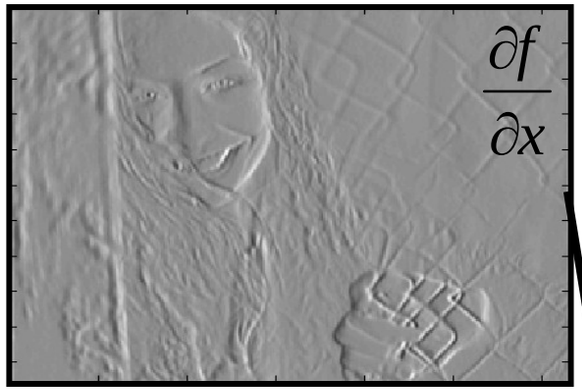
horizontal

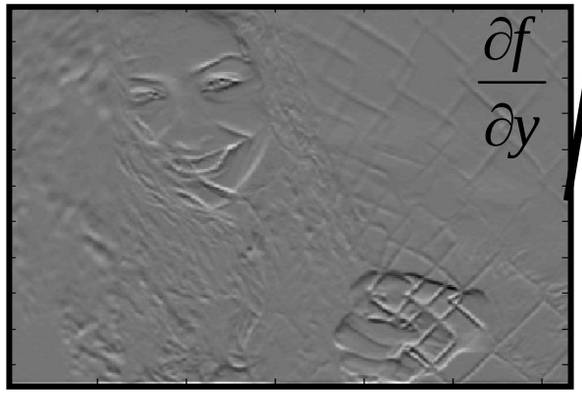


which Sobel filter?

Vertical

Example: Gradient Magnitude Image



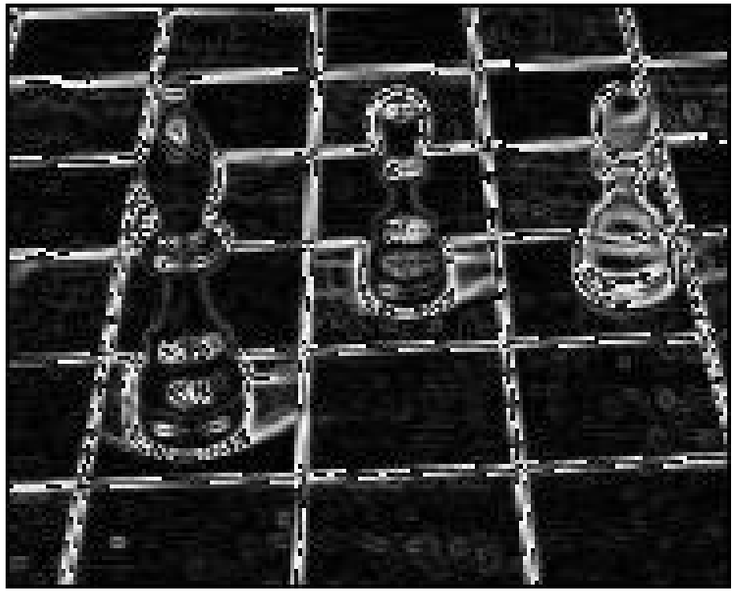


Gradient Magnitude

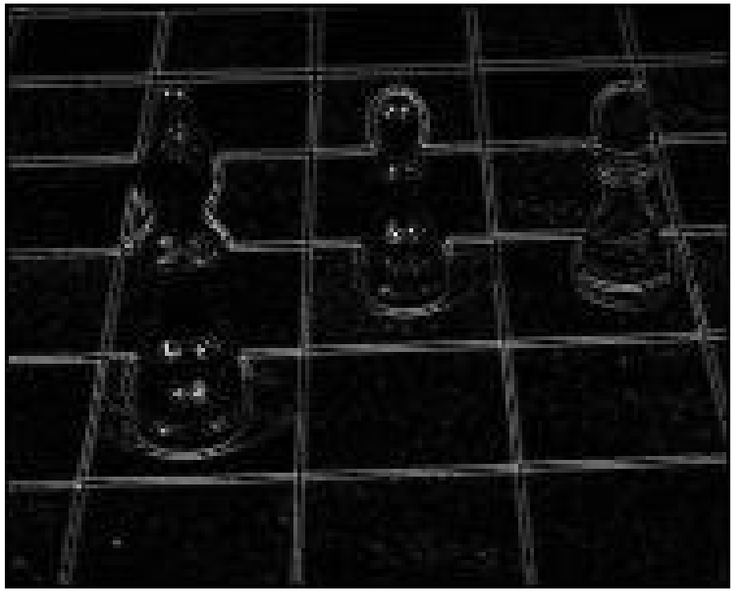
$$: \sqrt{\frac{\partial f}{\partial x}^2 + \frac{\partial f}{\partial y}^2}$$

(isotropic)

Example: Laplacian vs Gradient



Sobel



Laplacian

Other operators

- Prewitt

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -1 & 0 & +1 \\ -1 & 0 & +1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ +1 & +1 & +1 \end{bmatrix}$$

Roberts

-1	0
0	1

G_x

0	-1
1	0

G_y

- Scharr

$$G_x = \begin{bmatrix} -3 & 0 & +3 \\ -10 & 0 & +10 \\ -3 & 0 & +3 \end{bmatrix} \quad G_y = \begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 0 \\ +3 & +10 & +3 \end{bmatrix}$$

- Roberts: small, very sensitive to noise.
- Prewitt: simple 3x3 gradient estimate.
- Sobel: includes smoothing with stronger center row/column.

Other operators

- Kirsch

$$H_0^{\mathrm{K}} = \begin{bmatrix} -5 & 3 & 3 \\ -5 & 0 & 3 \\ -5 & 3 & 3 \end{bmatrix}, \quad H_4^{\mathrm{K}} = \begin{bmatrix} 3 & 3 & -5 \\ 3 & 0 & -5 \\ 3 & 3 & -5 \end{bmatrix}$$

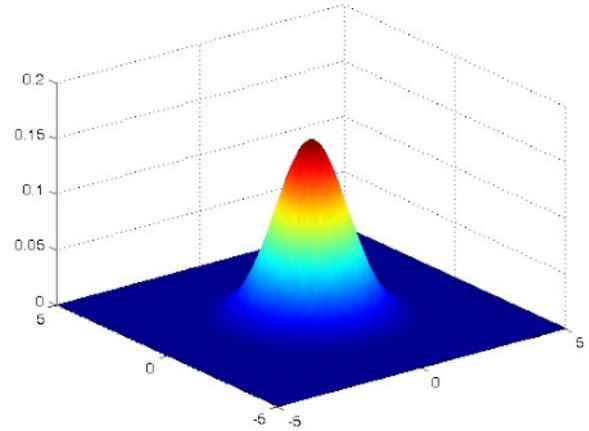
$$H_1^{\mathrm{K}} = \begin{bmatrix} -5 & -5 & 3 \\ -5 & 0 & 3 \\ 3 & 3 & 3 \end{bmatrix}, \quad H_5^{\mathrm{K}} = \begin{bmatrix} 3 & 3 & 3 \\ 3 & 0 & -5 \\ 3 & -5 & -5 \end{bmatrix}$$

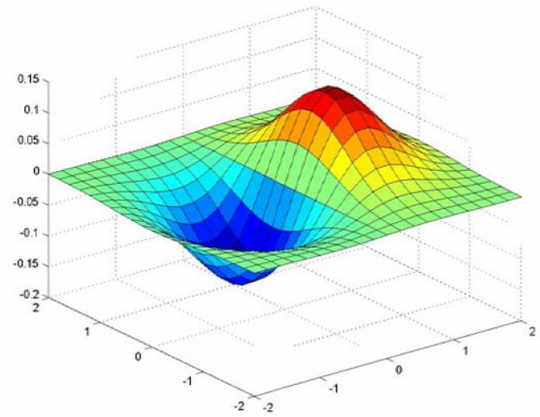
$$H_2^{\mathrm{K}} = \begin{bmatrix} -5 & -5 & -5 \\ 3 & 0 & 3 \\ 3 & 3 & 3 \end{bmatrix}, \quad H_6^{\mathrm{K}} = \begin{bmatrix} 3 & 3 & 3 \\ 3 & 0 & 3 \\ -5 & -5 & -5 \end{bmatrix}$$

$$H_3^{\mathrm{K}} = \begin{bmatrix} 3 & -5 & -5 \\ 3 & 0 & -5 \\ 3 & 3 & 3 \end{bmatrix}, \quad H_7^{\mathrm{K}} = \begin{bmatrix} 3 & 3 & 3 \\ -5 & 0 & 3 \\ -5 & -5 & 3 \end{bmatrix}$$

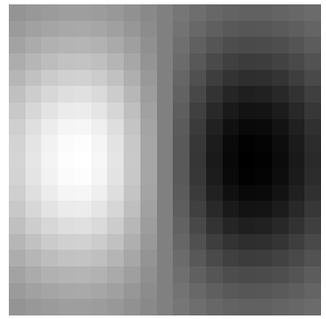
If larger derivative kernel is needed

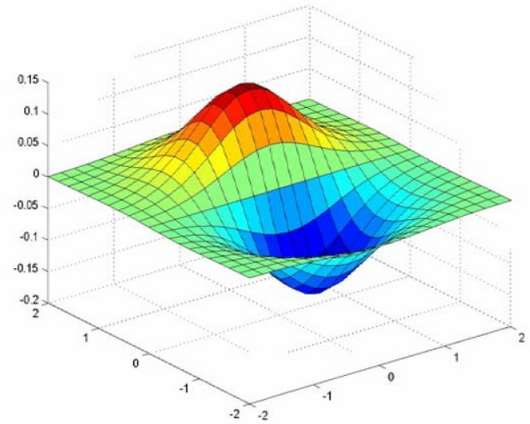
- Use Gaussian derivative kernels



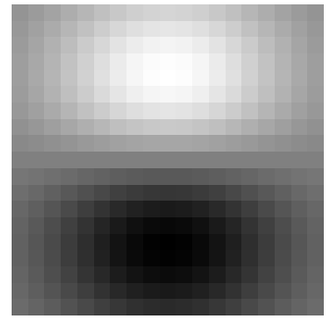


x-direction



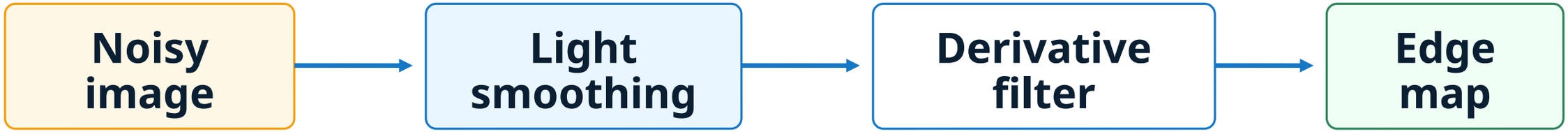


y-direction



Derivative filters need noise control

Why smoothing often comes first



- Derivative masks react strongly to random intensity variation.
- A small Gaussian smoothing step improves edge stability.
- Too much smoothing shifts or removes small edges.

Applicatic

a) original

b) Laplacian

c) a + b

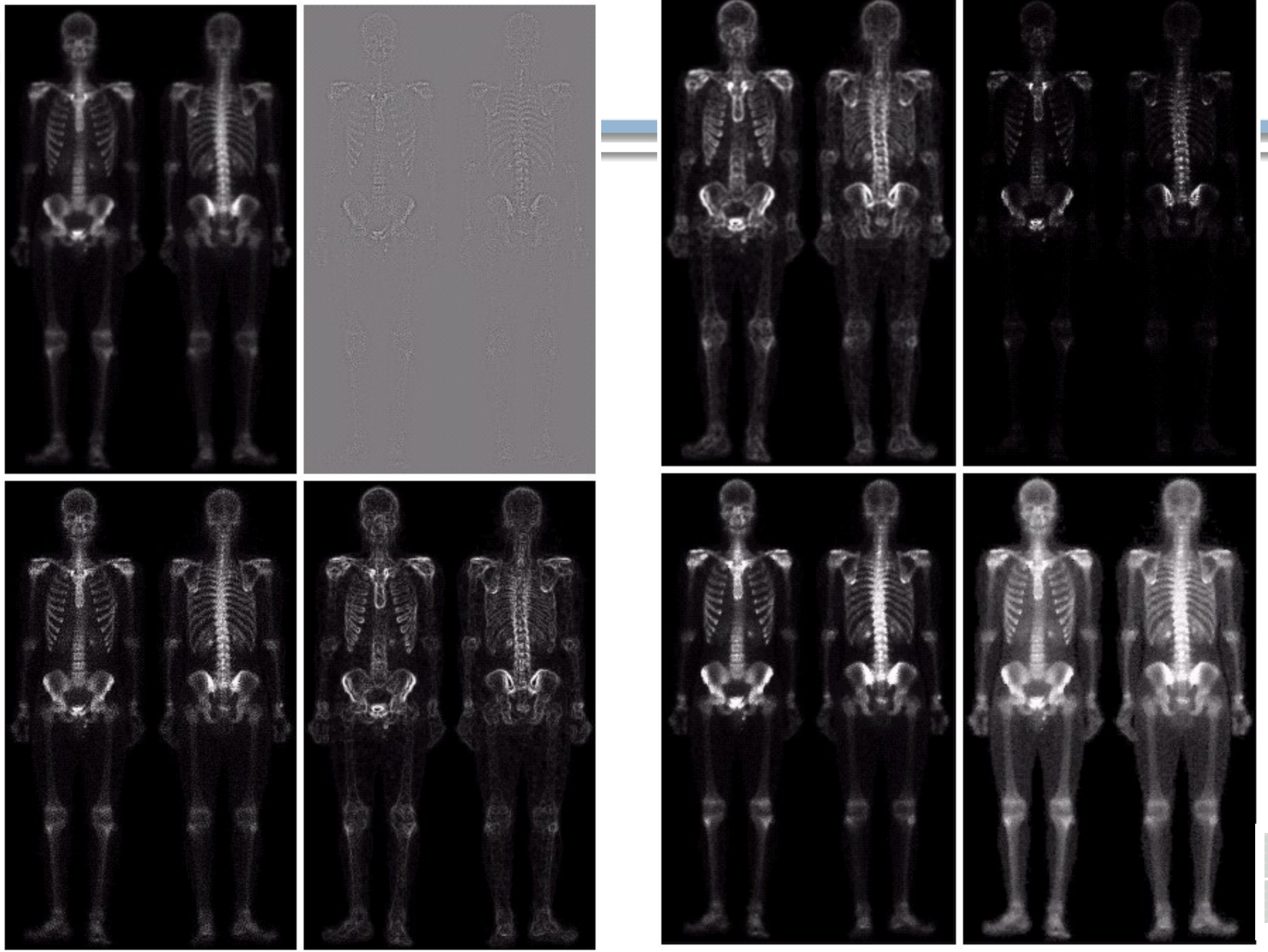
d) Sobel

e) Avg of d

f) c*e

g) a+f

h) γ transform



a b
c d

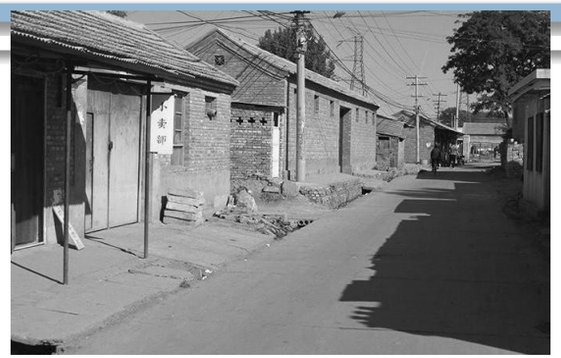
e f
g h

Think-Pair-Share

* = Convolution operator

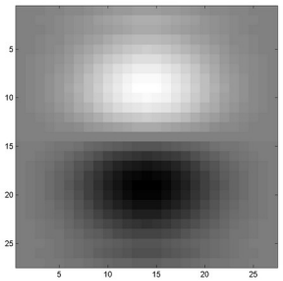
a) _ = D * B
b) A = _ * _
c) F = D * _
d) _ = D * D

D

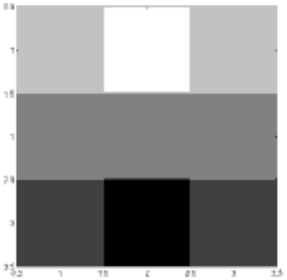


H

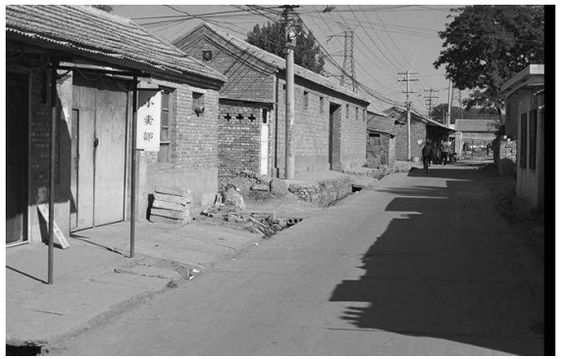
A



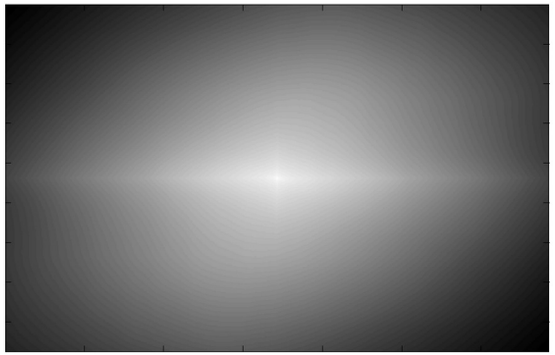
B



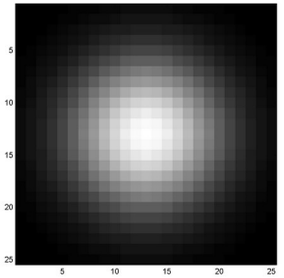
F



I

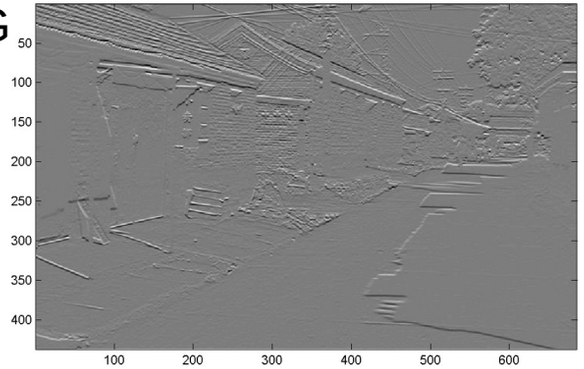


C



E

G



Hoiem

Summary

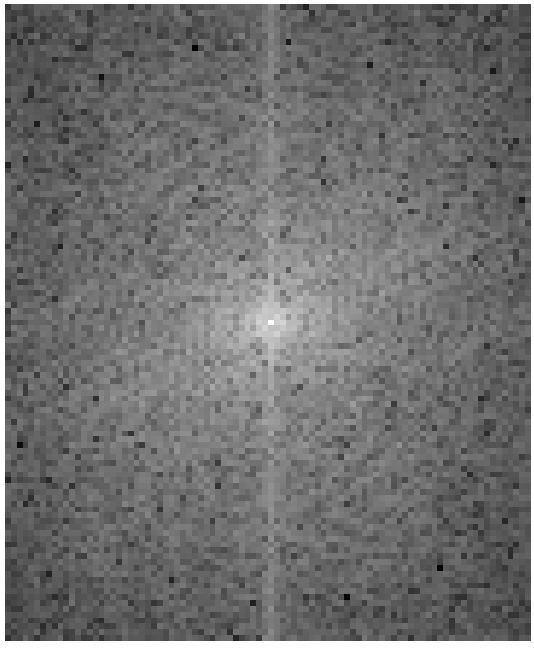
- Smoothing filters reduce noise but may blur important boundaries.
- Median filtering is best for impulse noise and preserves many edges.
- Laplacian and USM sharpen by emphasizing high-frequency detail.
- First derivative filters estimate gradients for basic edge detection.
- Parameter choice must be justified by the image problem and artifacts.

Enhancement is a controlled trade-off, not a fixed recipe.

Fourier Transform 1D

- Efficient Data Representation
- Discrete Fourier Transform - 1D
- Continuous Fourier Transform - 1D
- Examples

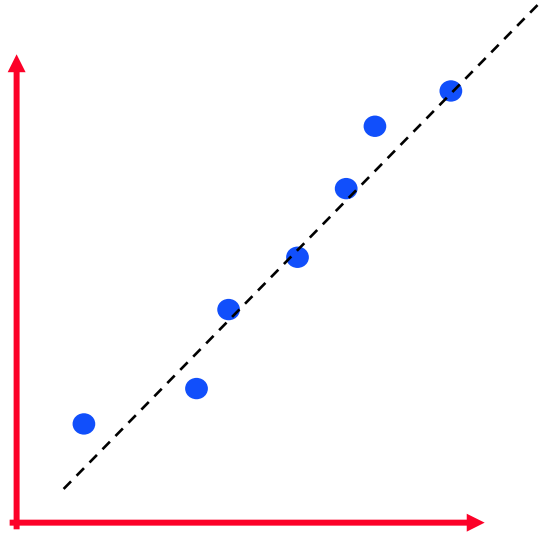
The Fourier Transform

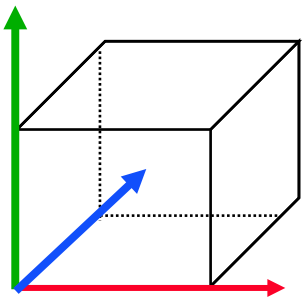


Jean Baptiste Joseph Fourier

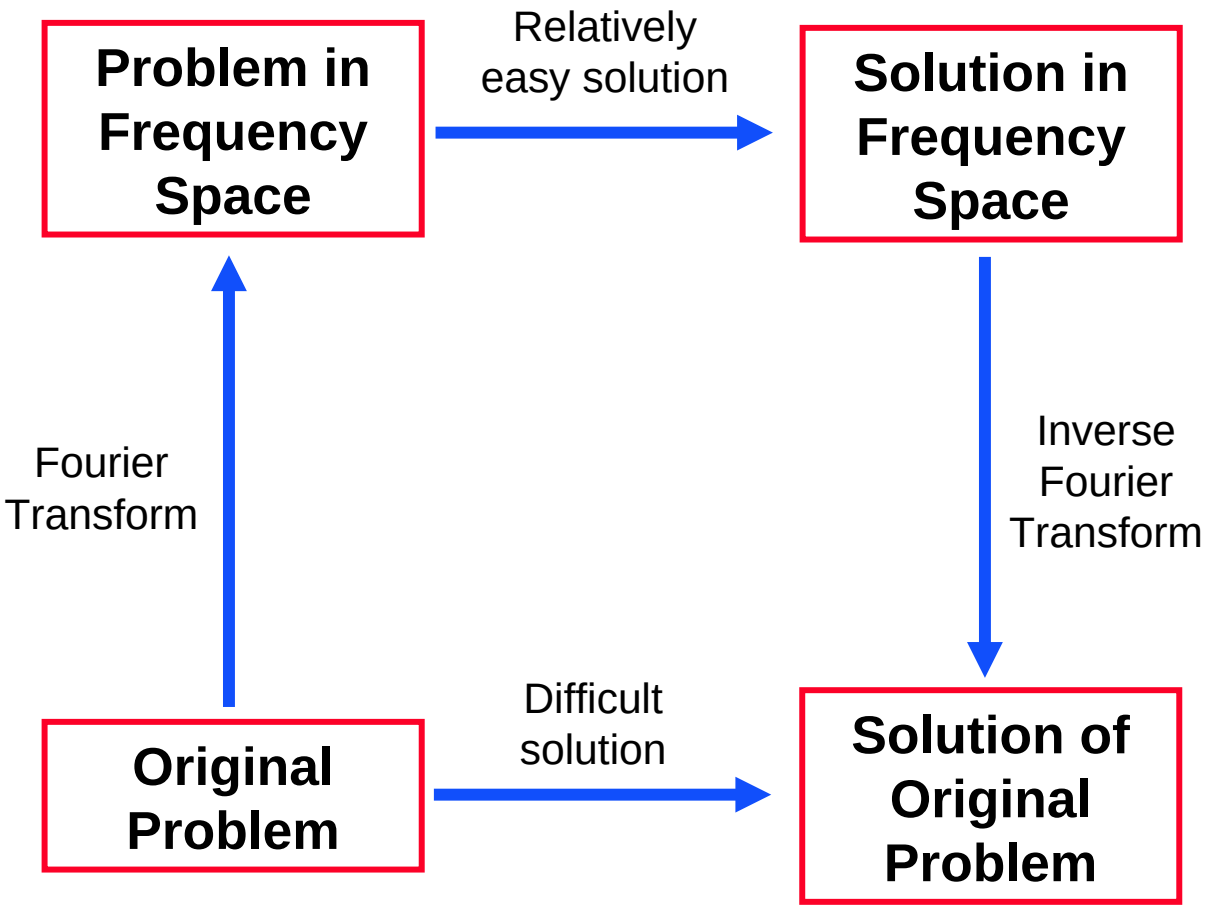
Efficient Data Representation

- Data can be represented in many ways.
- There is a great advantage using an appropriate representation.
- Examples:
  - Equalizers in Stereo systems.
  - Noisy points along a line
  - Color space red/green/blue v.s. Hue/Brightness





Why do we need representation in the frequency domain?

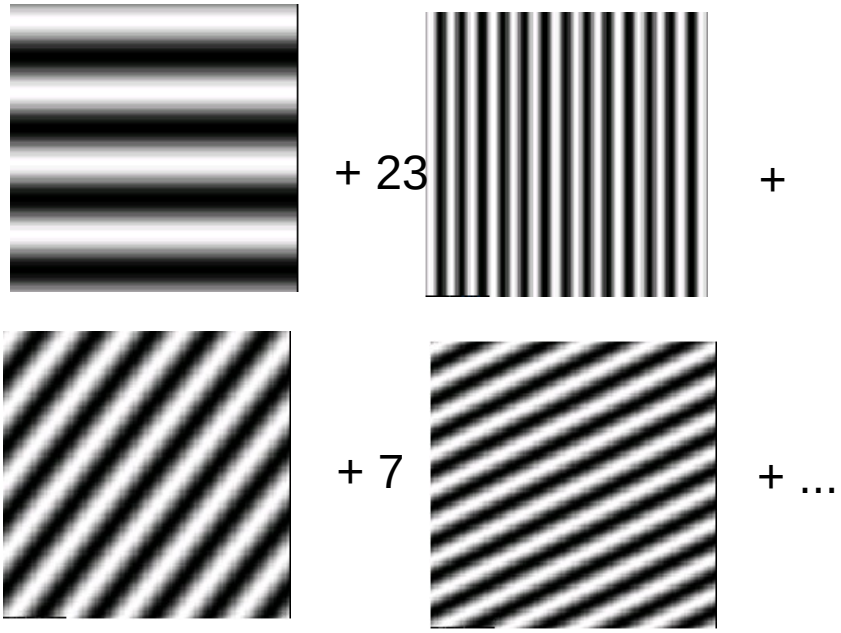


How can we enhance such an image?

Solution: Image Representation

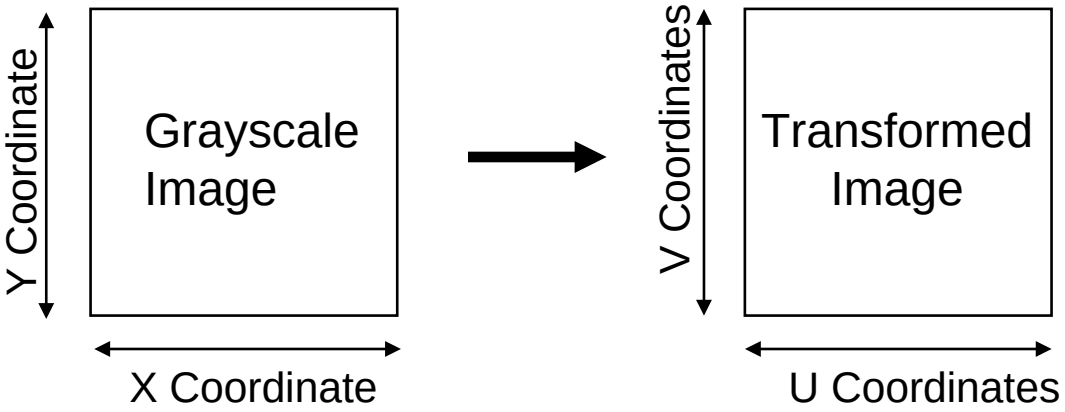
$$\begin{bmatrix} 2 & 1 & 3 \\ 5 & 8 & 7 \\ 0 & 3 & 5 \end{bmatrix} = 2 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} + 1 \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \\ + 3 \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} + 5 \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \dots$$

$$= 3 \quad + 23 \quad + \\ + 10 \quad + 7 \quad + \dots$$



Transforms

1. Basis Functions.
2. Method for finding the image given the transform coefficients.
3. Method for finding the transform coefficients given the image.

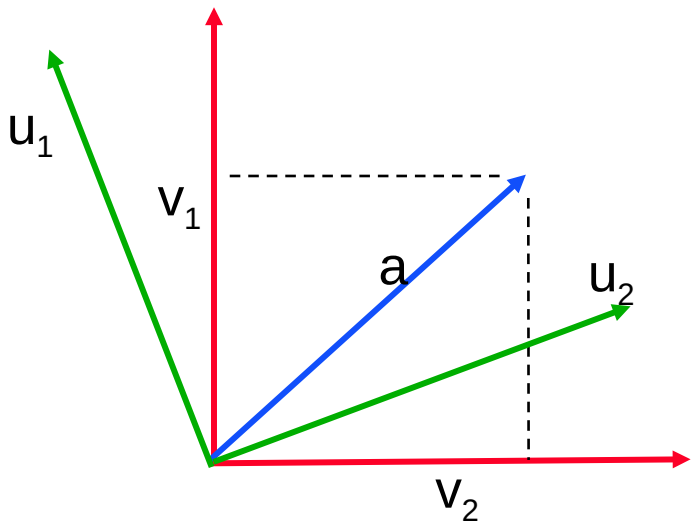


Representation in different bases

- It is possible to go back and forth between representations:

$$a_u(i) = \langle a_v, u_i \rangle$$

$$a_v = \sum_i a_u(i) u_i$$



The Inner Product

- Discrete vectors (real numbers):

$$\langle a, b \rangle = a^T b = \sum_i a(i) b(i)$$

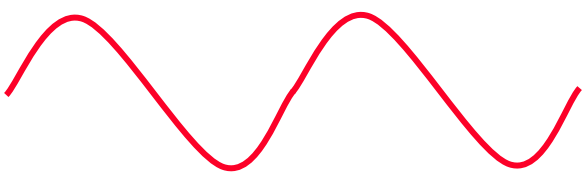
- Discrete vectors (complex numbers):

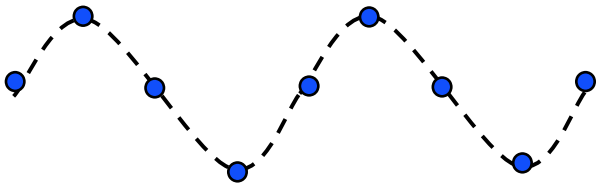
$$\langle a, b \rangle = a^H b = \sum_i a^*(i) b(i)$$

The vector $\mathbf{a}^H$ denotes the conjugate transpose of $\mathbf{a}$.

- Continuous functions:

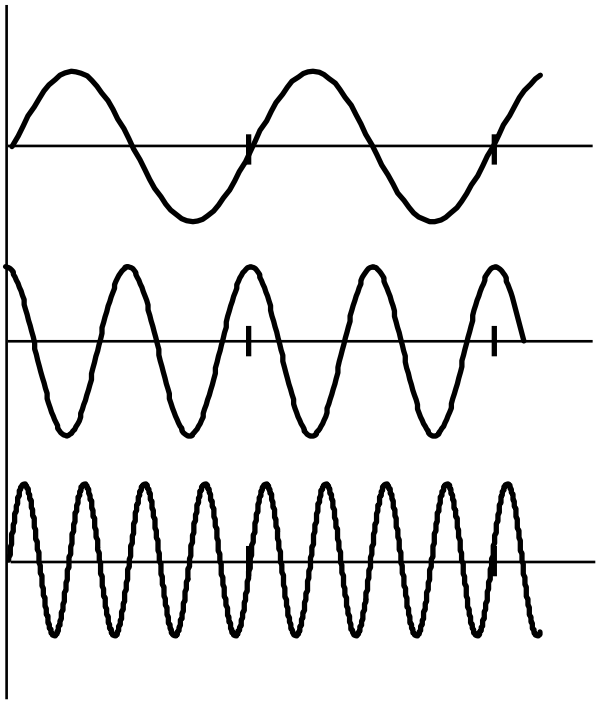
$$\langle f(x), g(x) \rangle = \int_x f^*(x) g(x) dx$$





The Fourier basis functions

Basis Functions are sines and cosines

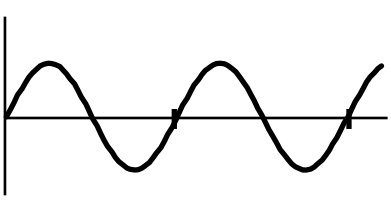


sin(x)

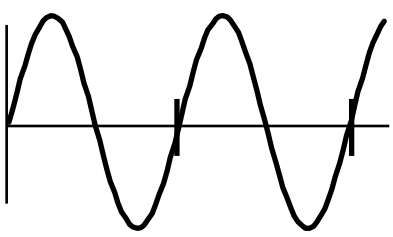
cos(2x)

sin(4x)

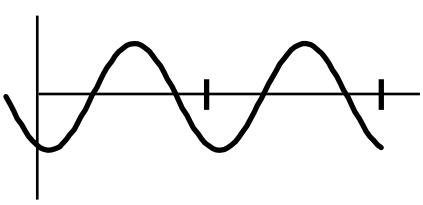
The transform coefficients determine the amplitude and phase:



a sin(2x)

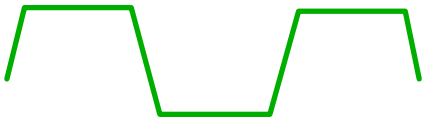


2a sin(2x)



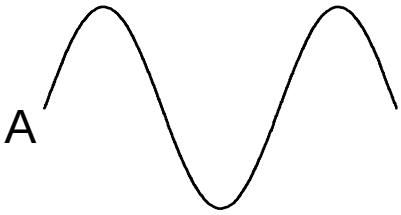
-a sin(2x+ϕ)

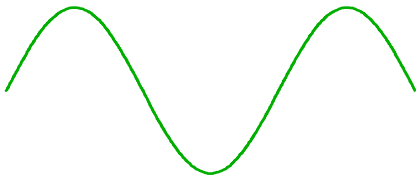
Every function equals a sum of sines and cosines



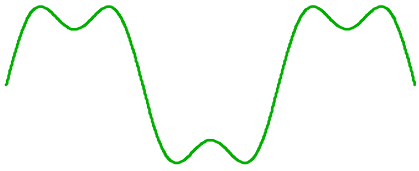
=

3 sin(x)



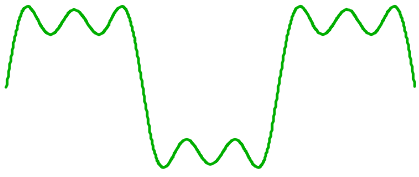


+ 1 sin(3x)



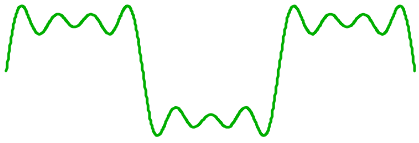
A+B

+ 0.8 sin(5x)



A+B+C

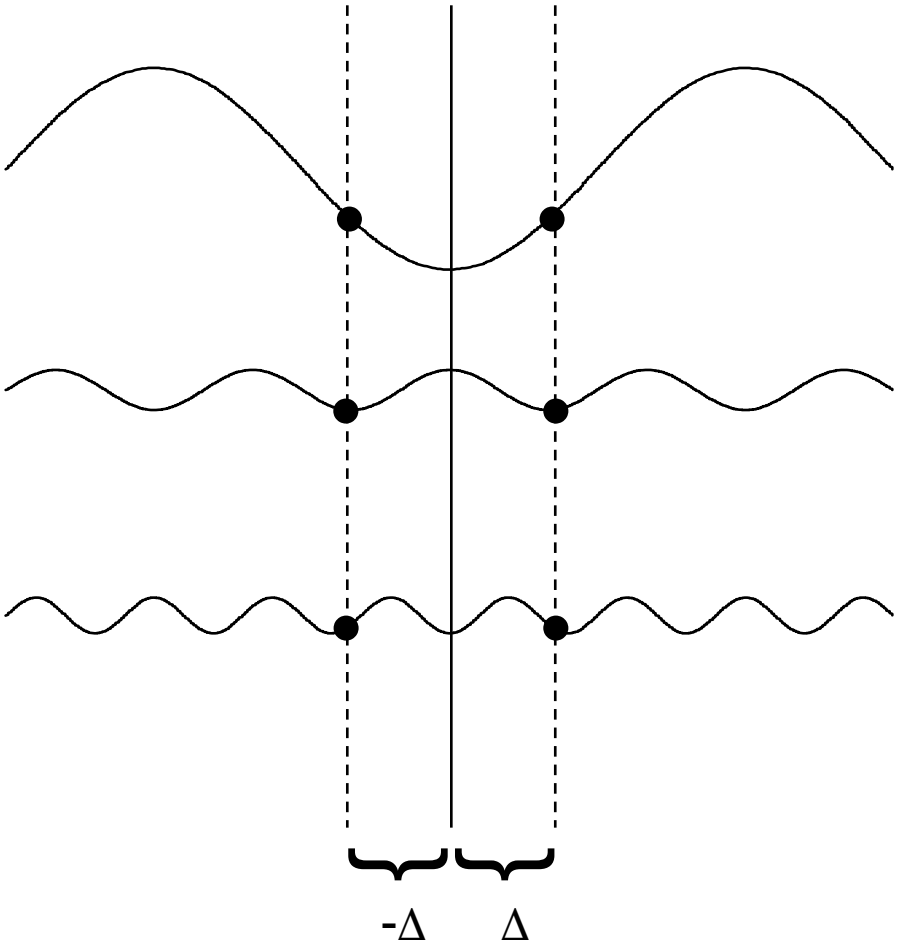
+ 0.4 sin(7x)



A+B+C+D

Sum of cosines only → symmetric functions

Sum of sines only → antisymmetric functions



Fourier Coefficients

$$f(x) = C_0 + C_1\cos(x) + S_1\sin(x) + \ldots + C_k\cos(kx) + S_k\sin(kx) + \ldots$$

Terms are considered in pairs:

$$C_k\cos(kx) + S_k\sin(kx) = R_k \sin(kx + \theta_k)$$

where $R_k = \sqrt{C_k^2 + S_k^2}$ **and** $\theta_k = tan^{-1}\left(\frac{S_k}{C_k}\right)$

Using Complex Numbers:

$$\cos(kx)\ ,\ \sin(kx) \longrightarrow e^{ikx}$$

$$C_k\cos(kx) + S_k\sin(kx) \longrightarrow \underbrace{R\, e^{i\theta}}\ e^{ikx}$$

Amplitude+phase

The 1D Continues Fourier Transform

- The Continuous Fourier Transform finds the F(ω) given the (cont.) signal f(x):

$$F(\omega) = \int_x f(x)\, e^{-i2\pi\omega x}\, dx$$

$B_\omega(x)=e^{i2\pi\omega x}$ is a complex wave function for each (continues) given ω .

- The inverse Continuous Fourier Transform composes a signal f(x) given F(ω):

$$f(x) = \int_k F(\omega)\, e^{i2\pi\omega x}\, d\omega$$

- F(ω) and f(x) are continues.

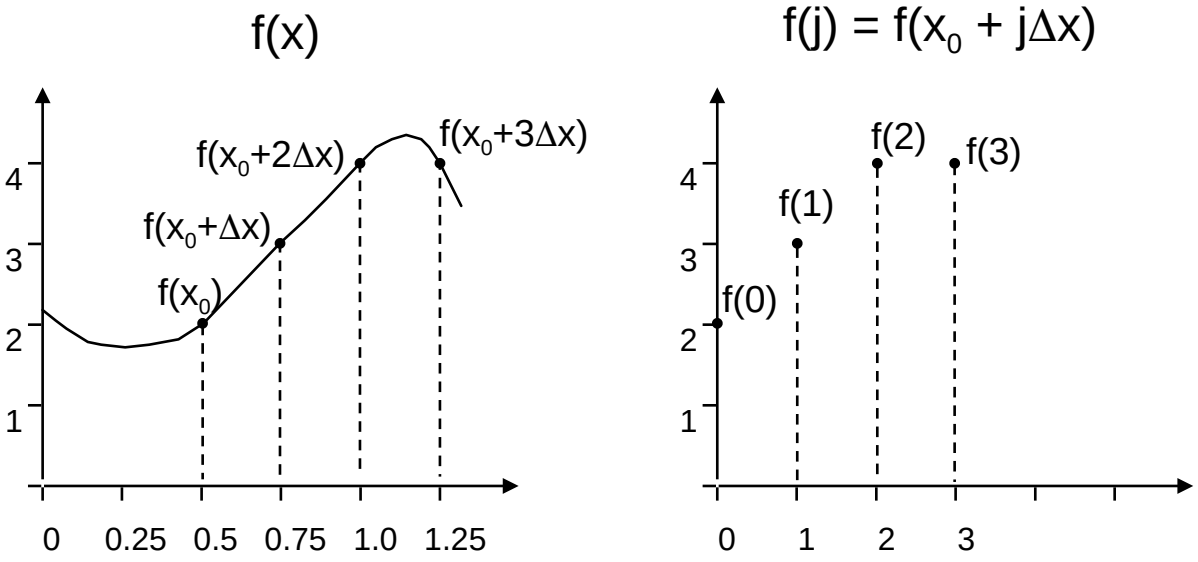
Continuous vs sampled Signals

Sampling: Move from $f(x)$ $(x \in R)$ to $f(x_j)$ $(j \in Z)$ by sampling at equal intervals.

$$f(x_0),\ f(x_0+\Delta x),\ f(x_0+2\Delta x),\ \ldots.\ ,\ f(x_0+[n-1]\Delta x),$$

Given N samples at equal intervals, we redefine f as:

$$f(j) = f(x_0+j\Delta x) \qquad j = 0, 1, 2, \ldots , N-1$$



- The discrete basis functions are

$$b_k(x) = \frac{1}{\sqrt{N}} e^{\frac{2\pi i k x}{N}} \qquad k = 0..N-1 \qquad x = 0..N-1$$

- For frequency k the Fourier coefficient is:

$$C_k \cos\left(\frac{2\pi kx}{N}\right) + S_k \sin\left(\frac{2\pi kx}{N}\right) \rightarrow \left(R_k e^{i\theta_k}\right) e^{i2\pi kx/N}$$

$$F(k) = R_k e^{i\theta_k}$$

The Discrete Fourier Transform (DFT)

$$F(k) = \langle f(x), b_k(x) \rangle = \sum_{x=0}^{N-1} f(x) e^{\frac{-2\pi i k x}{N}}$$

k = 0, 1, 2, ..., N-1

Matlab: F=fft(f);

The Inverse Discrete Fourier Transform (IDFT) is defined as:

$$f(x) = \frac{1}{N} \sum_{k=0}^{N-1} F(k) e^{\frac{2\pi i k x}{N}}$$

x = 0, 1, 2, ..., N-1

Matlab: F=ifft(f);

Remark: Normalization constant might be different!

Discrete Fourier Transform - Example

$$f(x) = [2 \;\; 3 \;\; 4 \;\; 4]$$

$$F(0) = \sum_{x=0}^{3} f(x)\, e^{\frac{-2\pi i 0 x}{4}} = \sum_{x=0}^{3} f(x)\, 1$$

$$= (f(0) + f(1) + f(2) + f(3)) = (2+3+4+4) = 13$$

$$F(1) = \sum_{x=0}^{3} f(x)\, e^{\frac{-2\pi i x}{4}} = [2e^{0}+3e^{-i\pi/2}+4e^{-\pi i}+4e^{-i3\pi/2}] = [-2+i]$$

$$F(2) = \sum_{x=0}^{3} f(x)\, e^{\frac{-4\pi i x}{4}} = [2e^{0}+3e^{-i\pi}+4e^{-2\pi i}+4e^{-3\pi i}] = [-1-0i] = -1$$

$$F(3) = \sum_{x=0}^{3} f(x)\, e^{\frac{-6\pi i x}{4}} = [2e^{0}+3e^{-i3\pi/2}+4e^{-3\pi i}+4e^{-i9\pi/2}] = [-2-i]$$

DFT of $[2 \;\; 3 \;\; 4 \;\; 4]$ is $[\,13 \;\; (-2+i) \;\; -1 \;\; (-2-i)\,]$

The Fourier Transform - Summary

- F(k) is the Fourier transform of f(x):

$$\tilde{F}\{f(x)\} = F(k)$$

- f(x) is the inverse Fourier transform of F(k):

$$\tilde{F}^{-1}\{F(k)\} = f(x)$$

- f(x) and F(k) are a Fourier pair.

- f(x) is a representation of the signal in the Spatial Domain and F(k) is a representation in the Frequency Domain.

- The Fourier transform F(k) is a function over the complex numbers:

$$F(k) = R_k e^{i\theta_k}$$

  - R_k tells us how much of frequency k is needed.

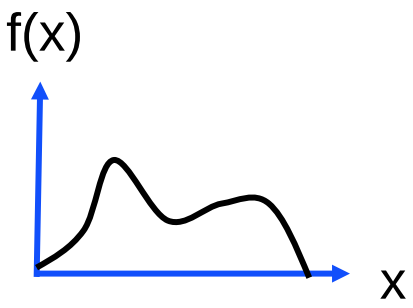
    θ_k tells us the shift of the Sine wave with frequency k.

- Alternatively:

$$F(k) = a_k + ib_k$$

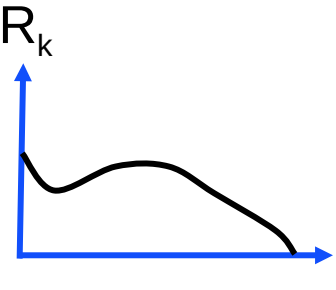
  - a_k tells us how much of cos with frequency k is needed.
  - b_k tells us how much of sin with frequency k is needed.

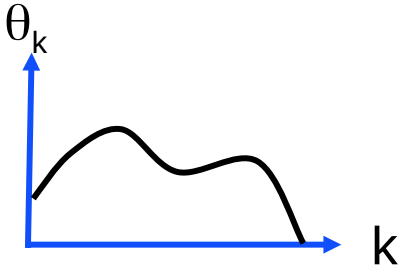
The Frequency Domain



The signal f(x)

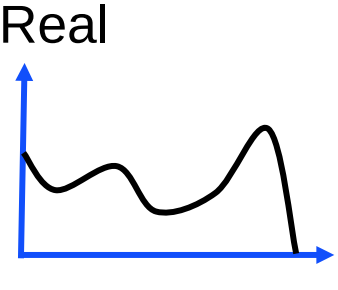
$$F(k) = R_k e^{i\theta_k}$$

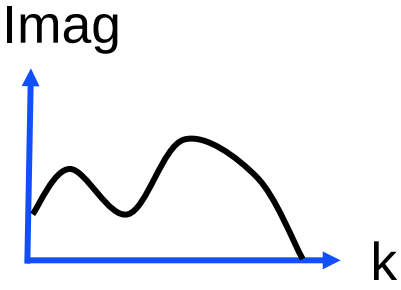




Amplitude (spectrum) and Phase

$$F(k) = a_k + ib_k$$



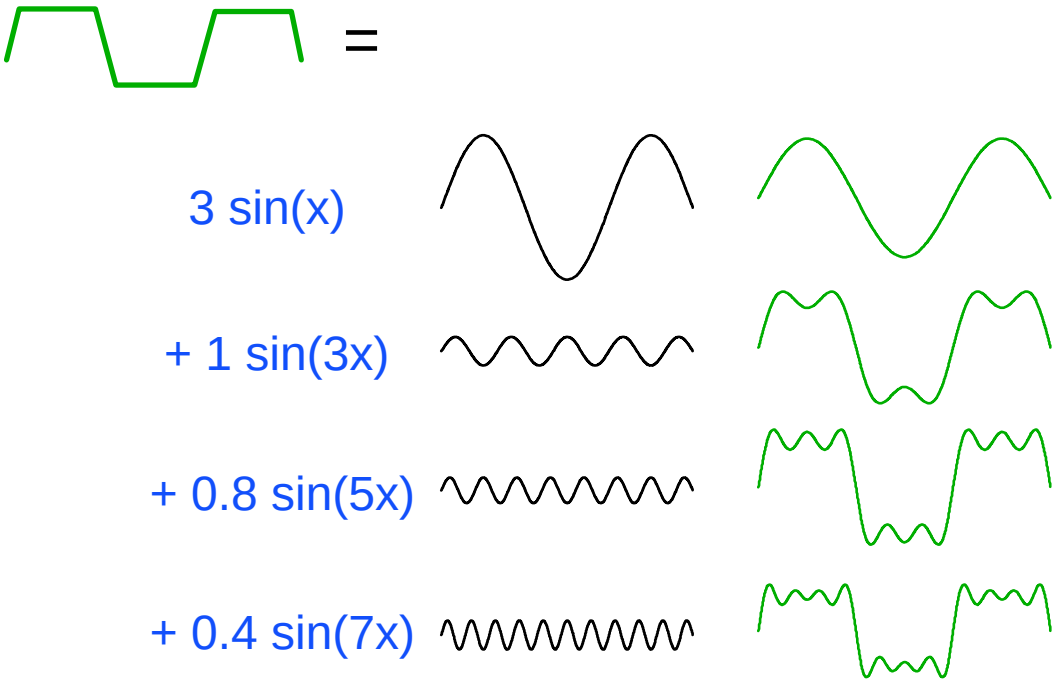


Real and Imaginary

- R_k - is the amplitude of F(k).

∀ θ_k - is the phase of F(k).

- $|R_k|^2 = F^*(k)\,F(k)$ - is the power spectrum of F(k) .
- If a signal f(x) has a lot of fine details $|R_k|^2$ will be high for high k.
- If the signal f(x) is "smooth" $|R_k|^2$ will be low for high k.



DEMO

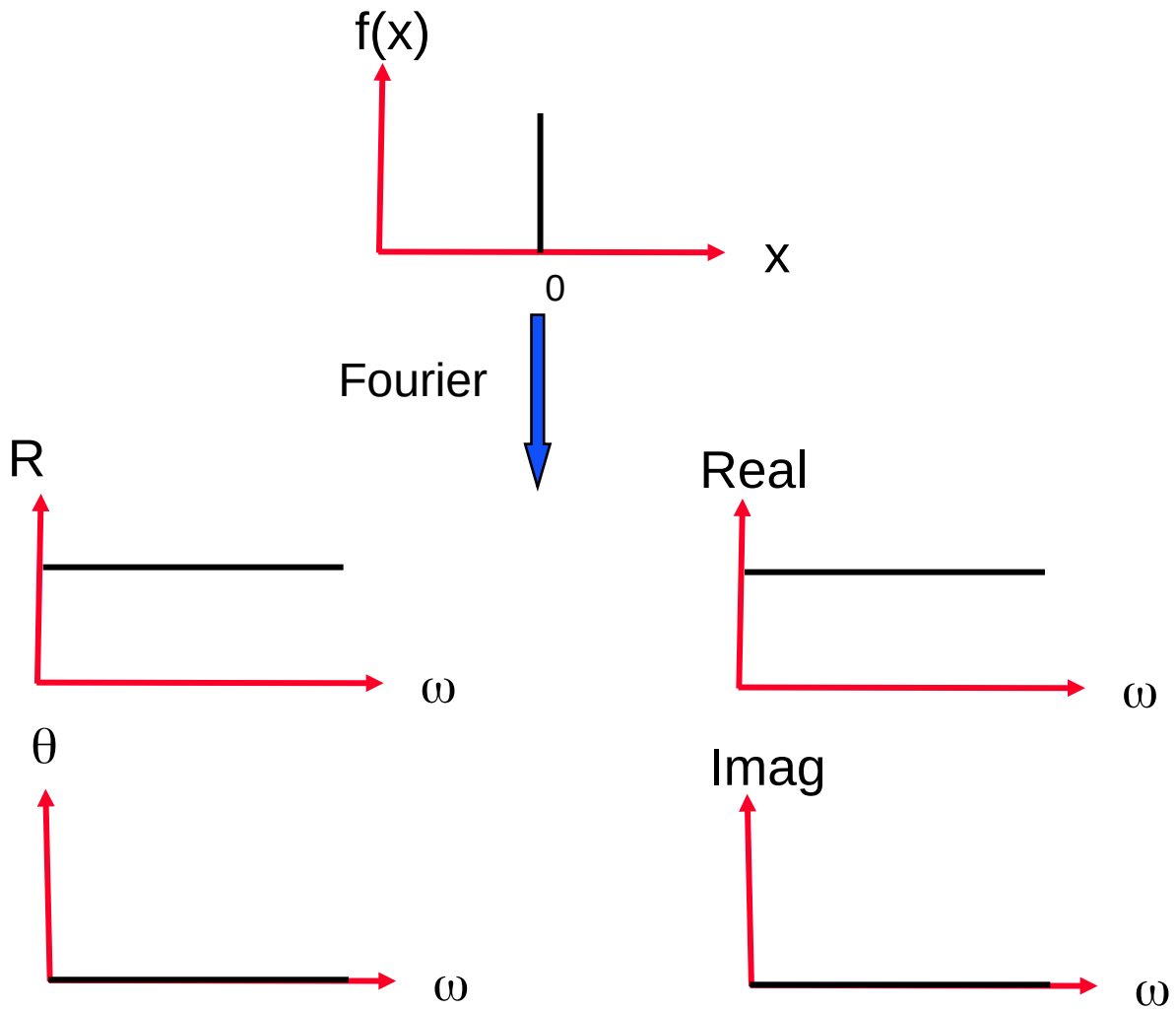
Examples:

The Delta Function:

- Let $f(x) = \delta(x)$

$$\left(\lim_{x \to 0} \delta(x) = \infty \; ; \quad \int \delta(x)\,dx = 1 \right)$$
$$\left(\int g(x)\,\delta(x - x_0)\,dx = g(x_0) \right)$$

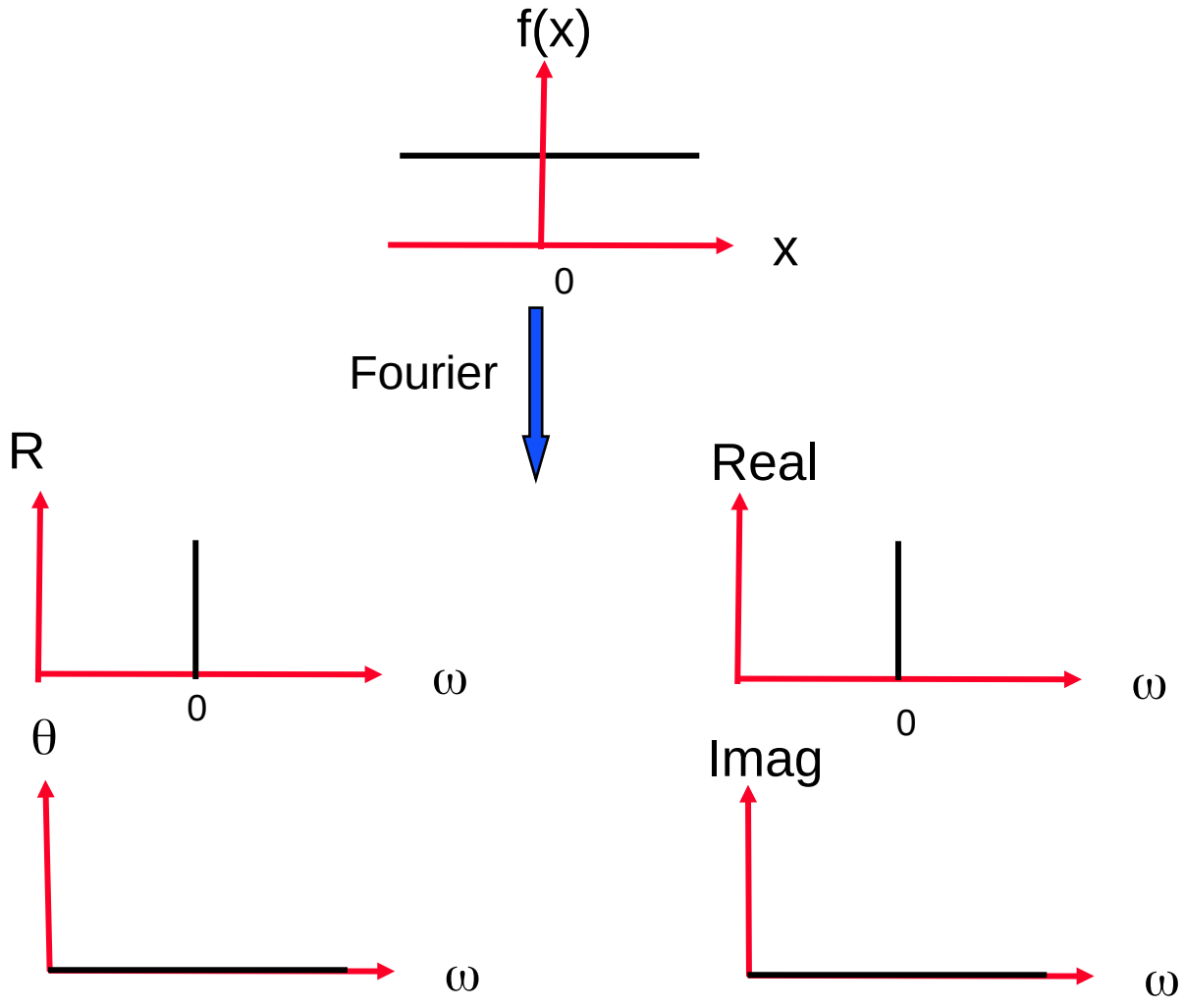
$$F(\omega) = \int_{-\infty}^{\infty} \delta(x) \cdot e^{-i2\pi\omega x}\,dx = 1$$



The Constant Function:

- Let $f(x) = 1$

$$F(\omega) = \int_{-\infty}^{\infty} e^{-i2\pi\omega x}\, dx = \delta(\omega)$$

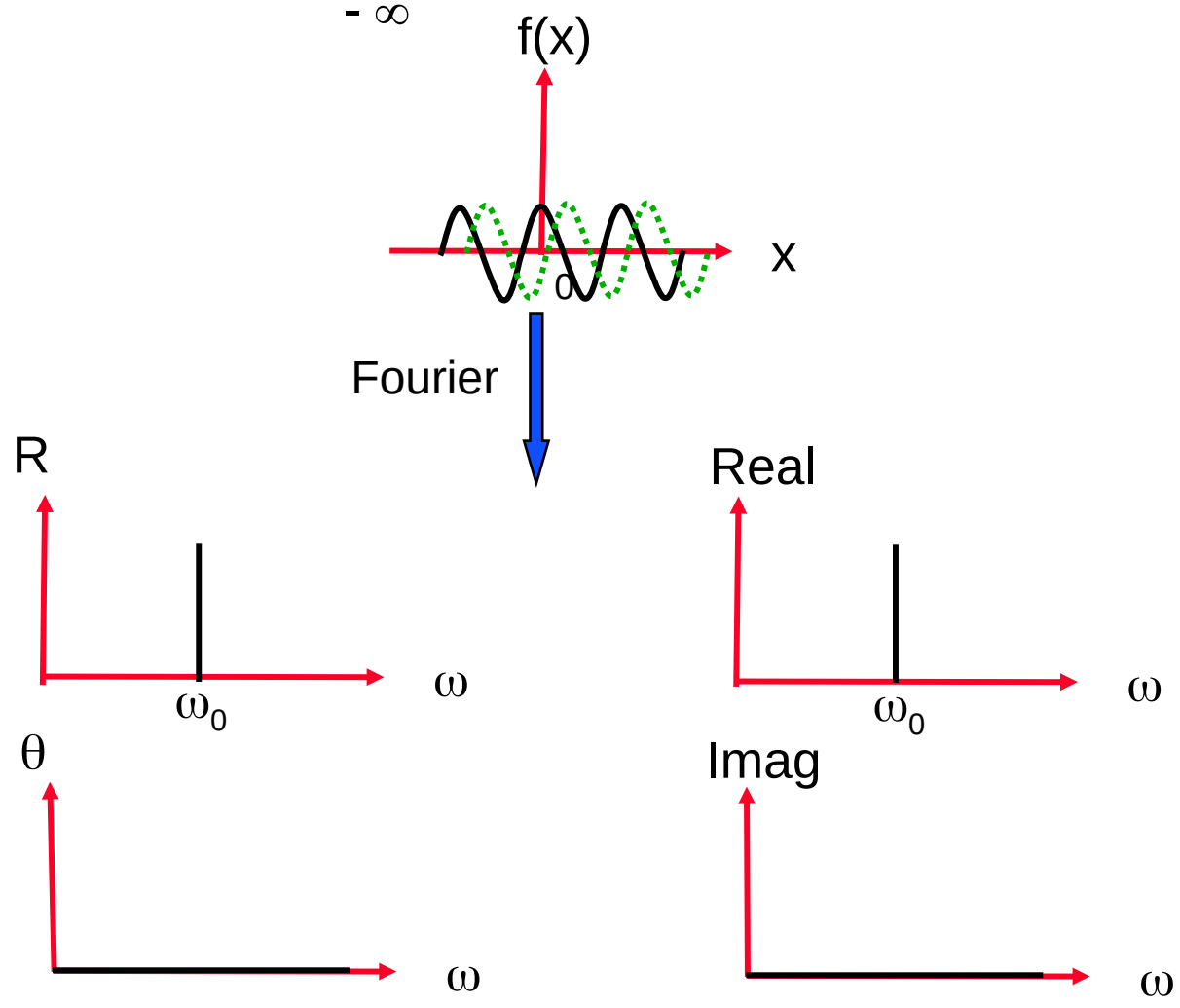


A Basis Function:

- Let $f(x) = e^{i2\pi\omega_0 x}$

$$F(\omega) = \int_{-\infty}^{\infty} e^{i2\pi\omega_0 x} e^{-i2\pi\omega x}\, dx$$

$$= \int_{-\infty}^{\infty} e^{-i2\pi(\omega-\omega_0)x}\, dx = \delta(\omega-\omega_0)$$

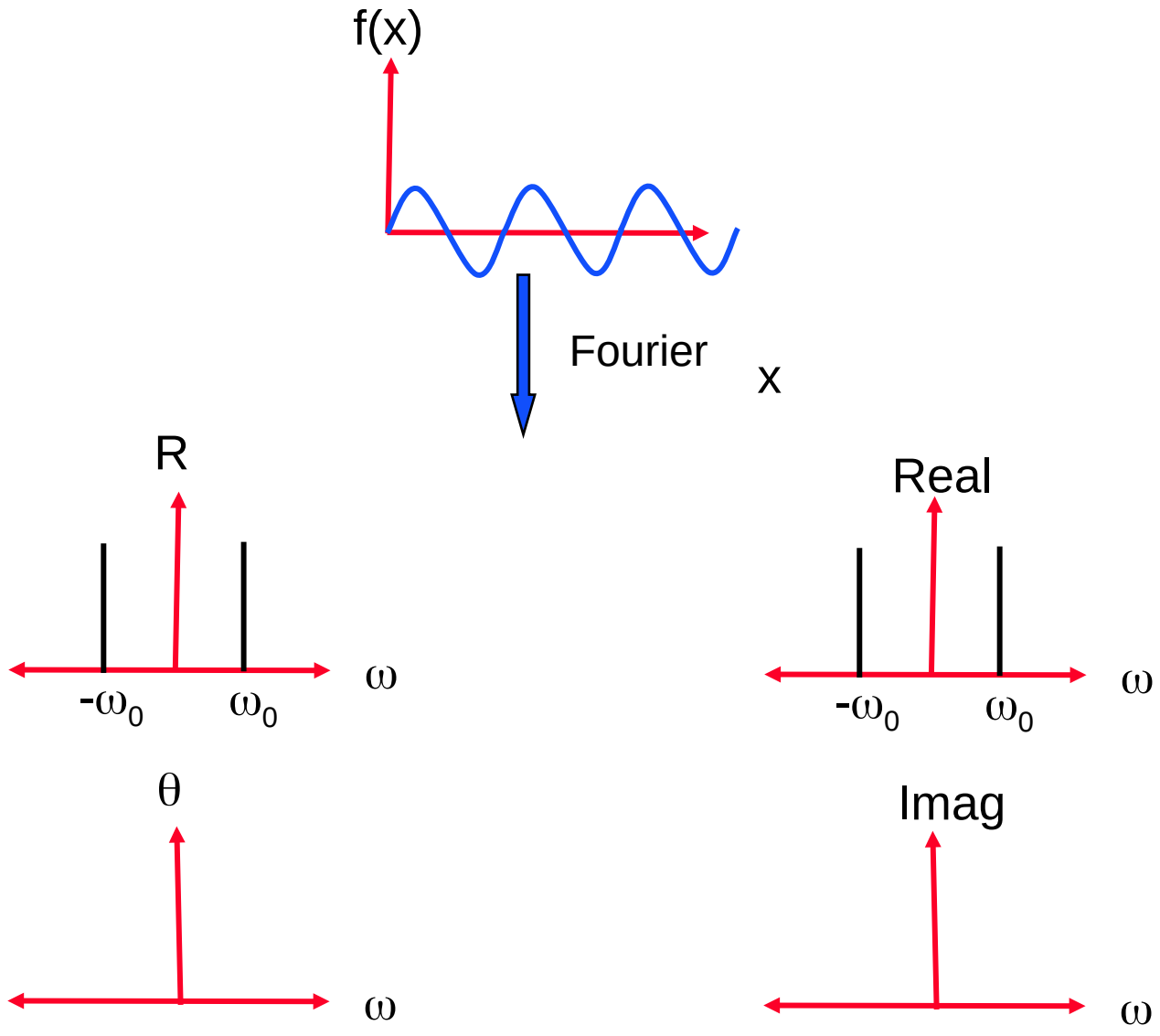


The Cosine wave:

- Let $f(x) = \cos(2\pi\omega_0 x)$

$$F(\omega) = \int_{-\infty}^{\infty} \frac{1}{2}\left(e^{i2\pi\omega_0 x} + e^{-i2\pi\omega_0 x}\right) e^{-i2\pi\omega x}\, dx =$$

$$= \frac{1}{2}\left[\delta(\omega - \omega_0) + \delta(\omega + \omega_0)\right]$$

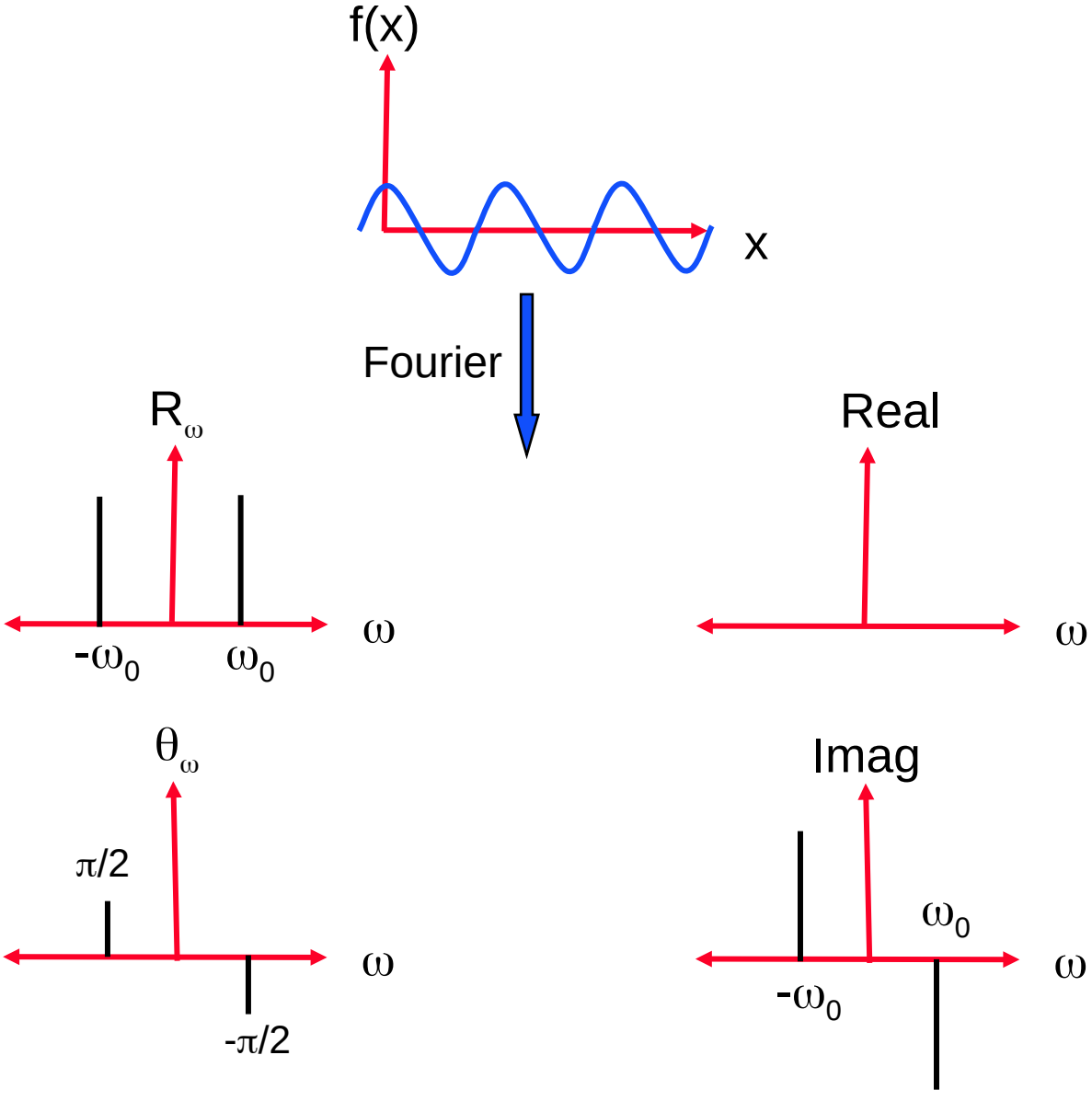


The Sine wave:

- Let $f(x) = \sin(2\pi\omega_0 x)$

$$F(\omega) = \int_{-\infty}^{\infty} \frac{i}{2}\left(e^{-i2\pi\omega_0 x} - e^{i2\pi\omega_0 x}\right) e^{-i2\pi\omega x}\, dx =$$

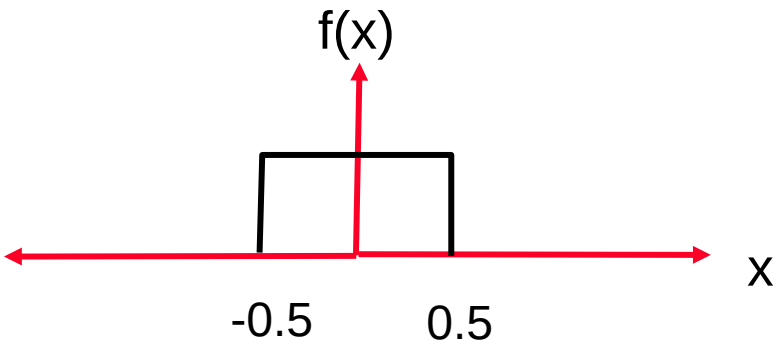
$$= \frac{i}{2}\left[\delta(\omega+\omega_0) - \delta(\omega-\omega_0)\right]$$



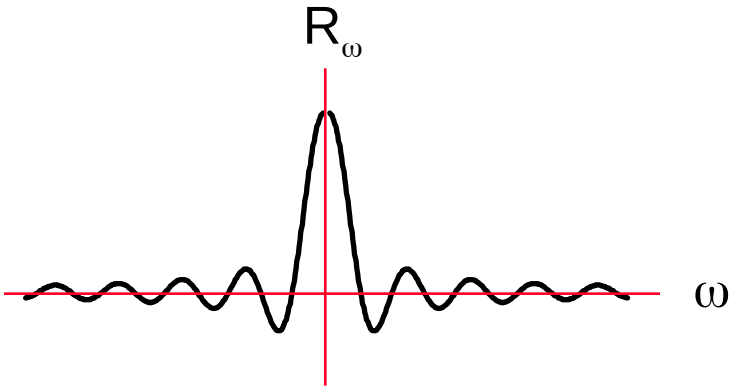
The Window Function (rect):

- Let $\text{rect}_{1/2}(x) = \begin{cases} 1 & \text{if } |x| < \frac{1}{2} \\ 0 & \text{otherwise} \end{cases}$

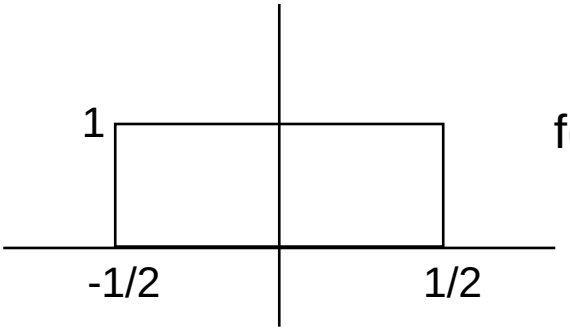
$$F(\omega) = \int_{-0.5}^{0.5} e^{-i2\pi\omega x}\, dx = \frac{\sin(\pi\omega)}{\pi\omega} = \text{sinc}(\pi\omega)$$



Fourier



Proof:



$$f(x) = \text{rect}_{1/2}(x) = \begin{cases} 1 & |x| \le 1/2 \\ 0 & \text{otherwise} \end{cases}$$

$$F(\omega) = \int_{-\infty}^{\infty} f(x) e^{-2\pi i \omega x} dx = \int_{-1/2}^{1/2} e^{-2\pi i \omega x} dx$$

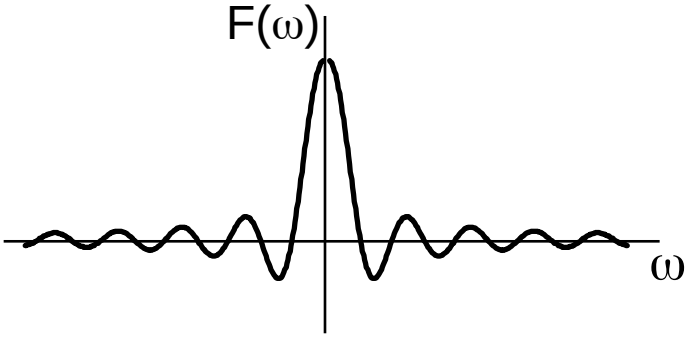
$$= \frac{1}{-2\pi i \omega} \left[e^{-2\pi i \omega x} \right]_{-1/2}^{1/2}$$

$$= \frac{1}{-2\pi i \omega} \left[e^{-\pi i \omega} - e^{\pi i \omega} \right]$$

$$= \frac{1}{-2\pi i \omega} \left[\cancel{\cos(\pi\omega)} - i\sin(\pi\omega) - \cancel{\cos(\pi\omega)} - i\sin(\pi\omega) \right]$$

$$= \frac{\sin(\pi\omega)}{\pi\omega} = \text{SINC}(\omega)$$

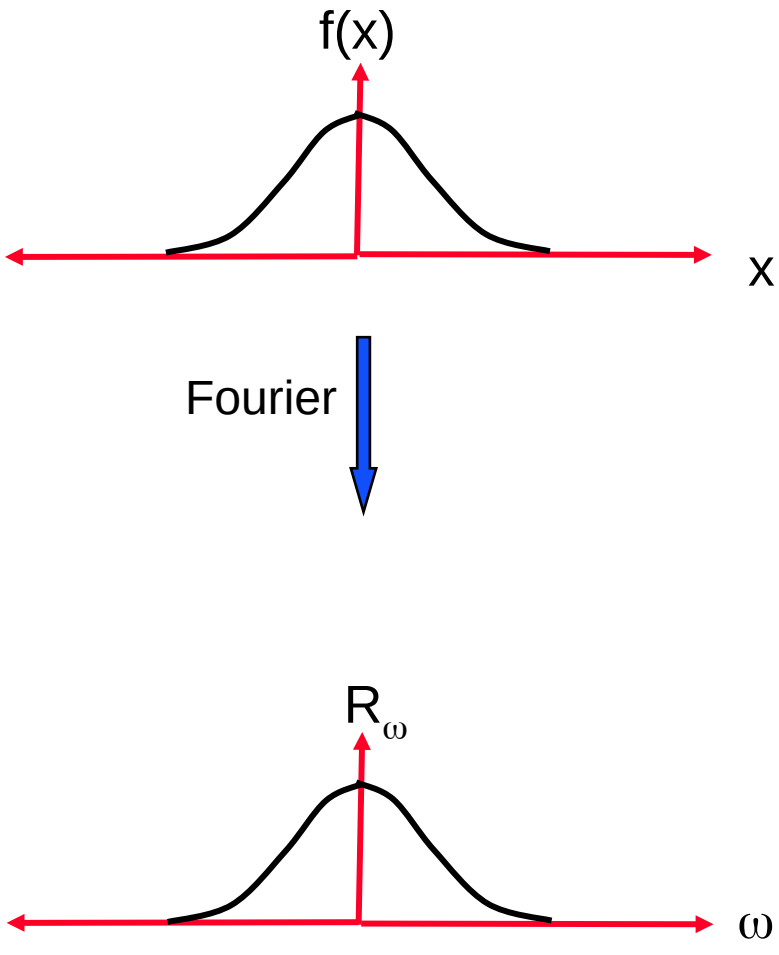
sinc(ω)



The Gaussian:

- Let $f(x) = e^{-\pi x^2}$

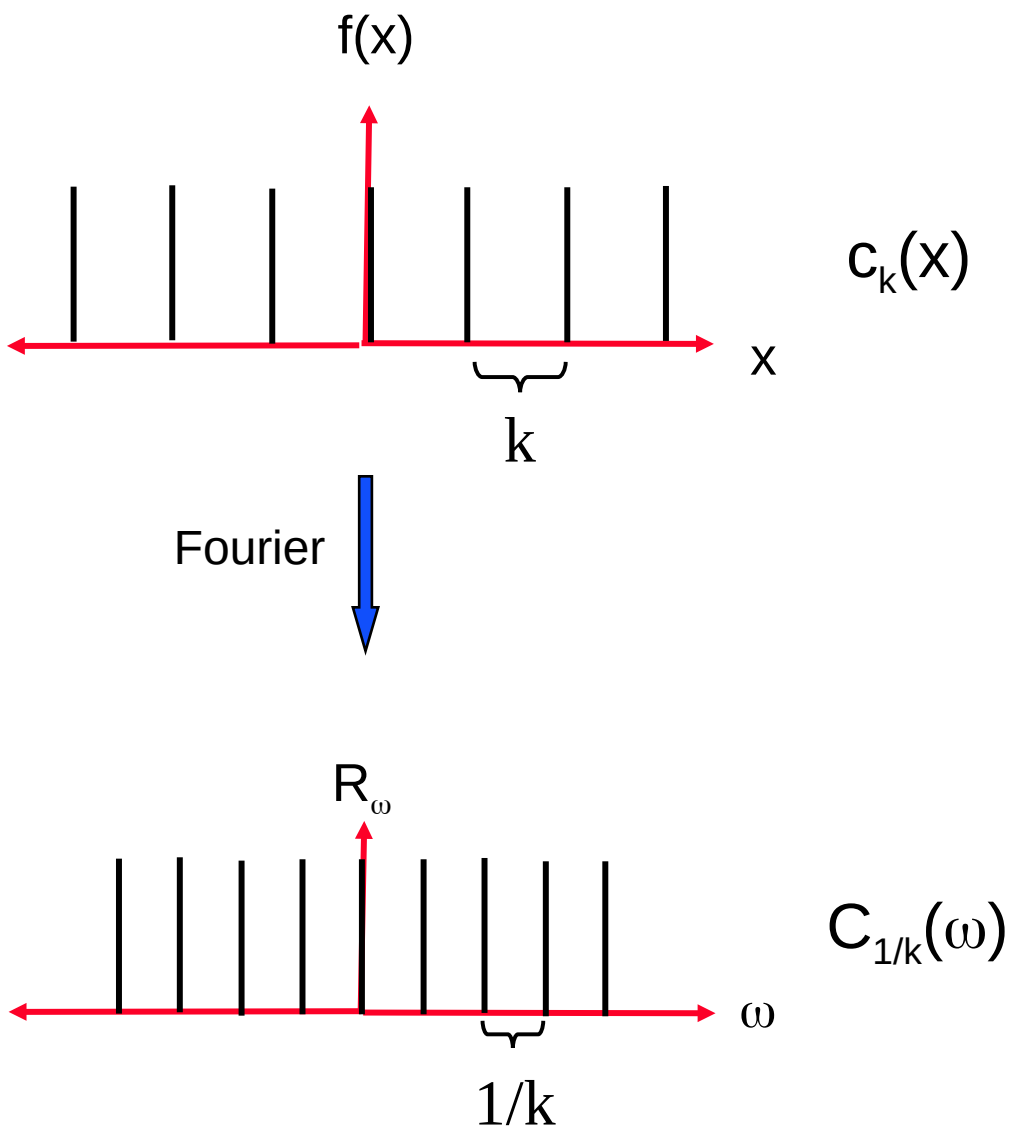
$$F(\omega) = e^{-\pi \omega^2}$$



The Comb Function:

- Let $c_k(x) = \delta(x \mod k)$

$$\tilde{F}\{c_k\} = \delta\left(\omega \mod \frac{1}{k}\right) = C_{1/k}(\omega)$$



Properties of The Fourier Transform

- Linearity:

$$\tilde{F}[\alpha f] = \alpha \tilde{F}[f]$$

- Distributive (additivity):

$$\tilde{F}[f_1 + f_2] = \tilde{F}[f_1] + \tilde{F}[f_2]$$

- DC (average):

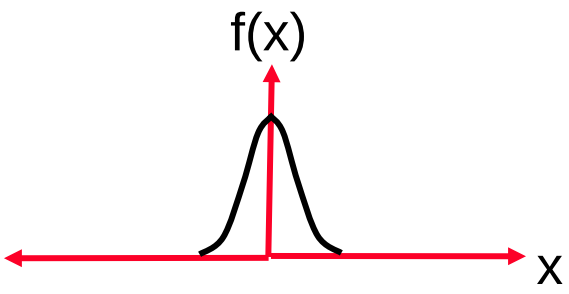
$$F(0) = \int f(x) e^{0}\, dx$$

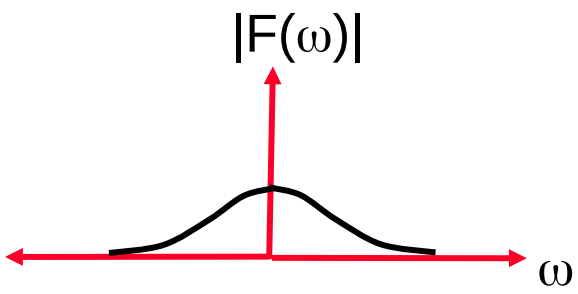
- Symmetric:

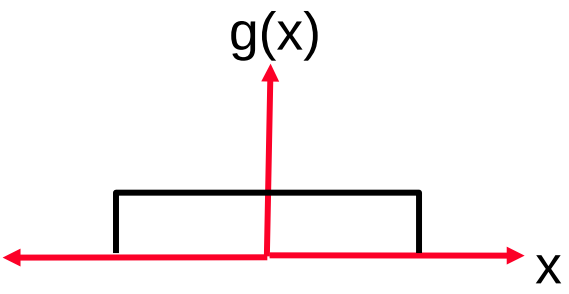
  If f(x) is real then,

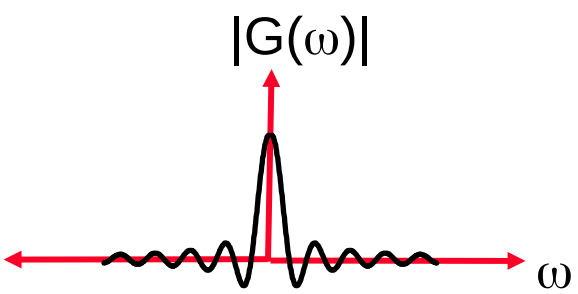
$$F(\omega) = F^{*}(-\omega) \quad thus \quad |F(\omega)| = |F(-\omega)|$$

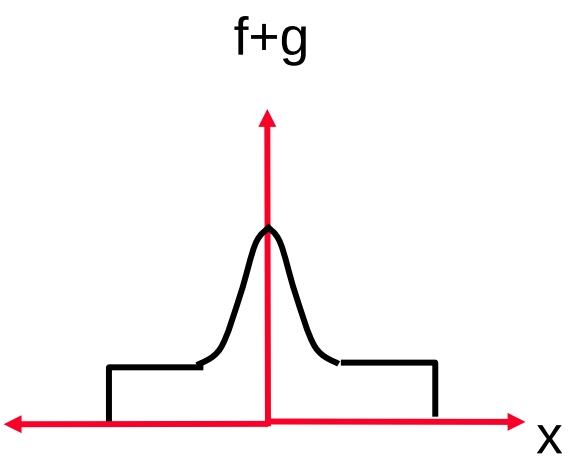
Distributive: $\tilde{F}\{f+g\} = \tilde{F}\{f\} + \tilde{F}\{g\}$

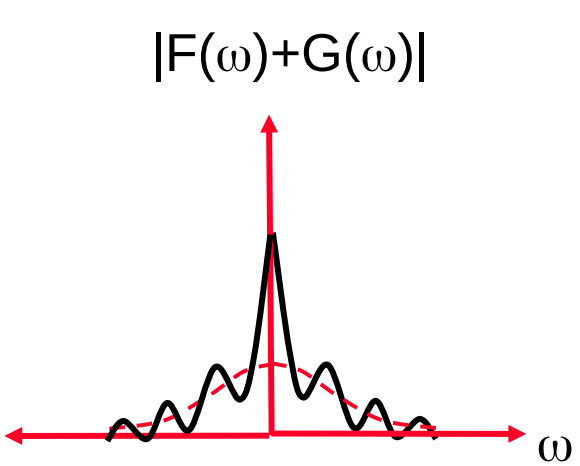












Transformations

- Translation:

$$\tilde{F}\left[f(x - x_0)\right] = F(\omega)e^{-2\pi i\omega x_0}$$

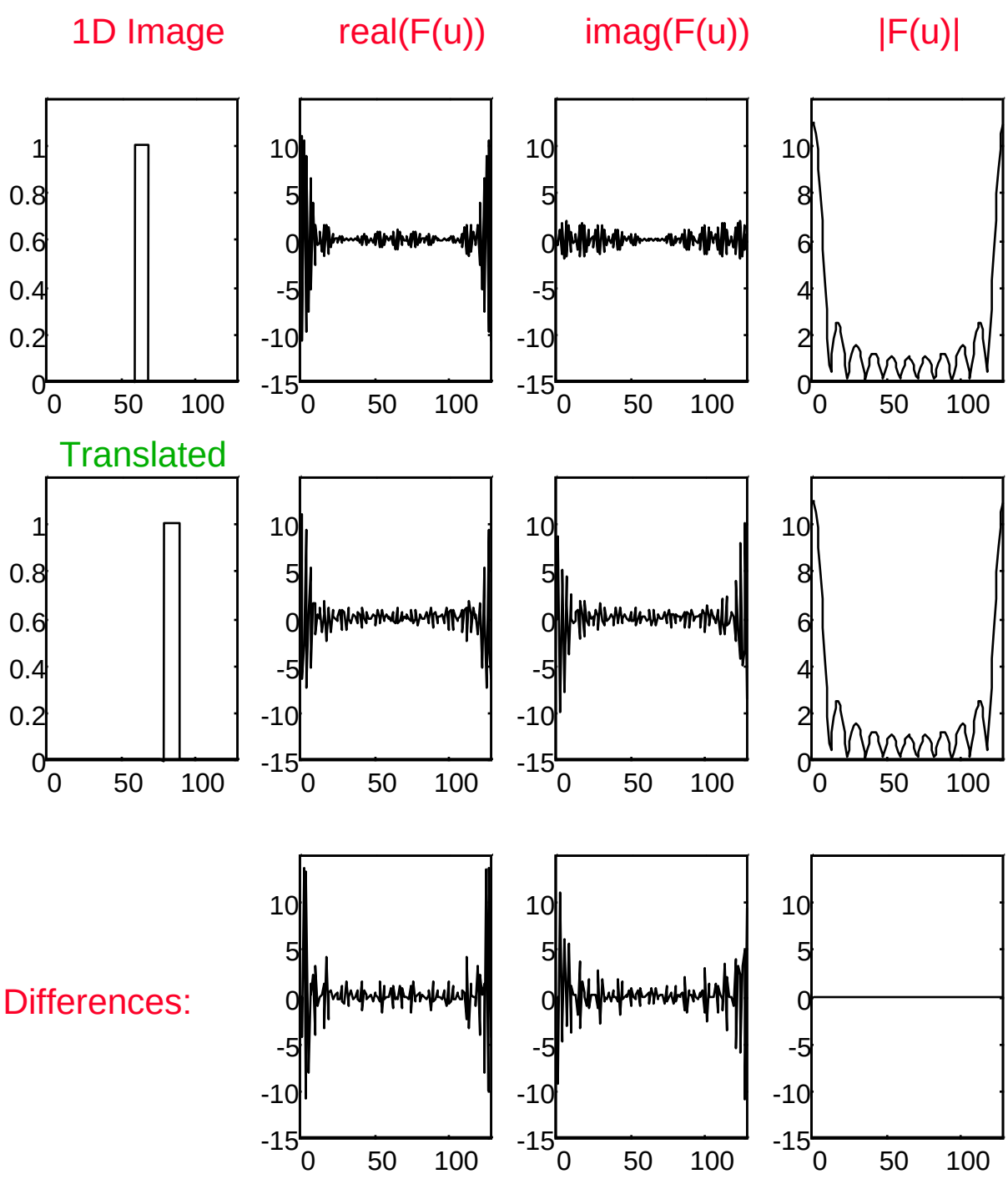
The Fourier Spectrum remains unchanged under translation:

$$|F(\omega)| = \left|F(\omega)e^{-2\pi i\omega x_0}\right|$$

- Scaling:

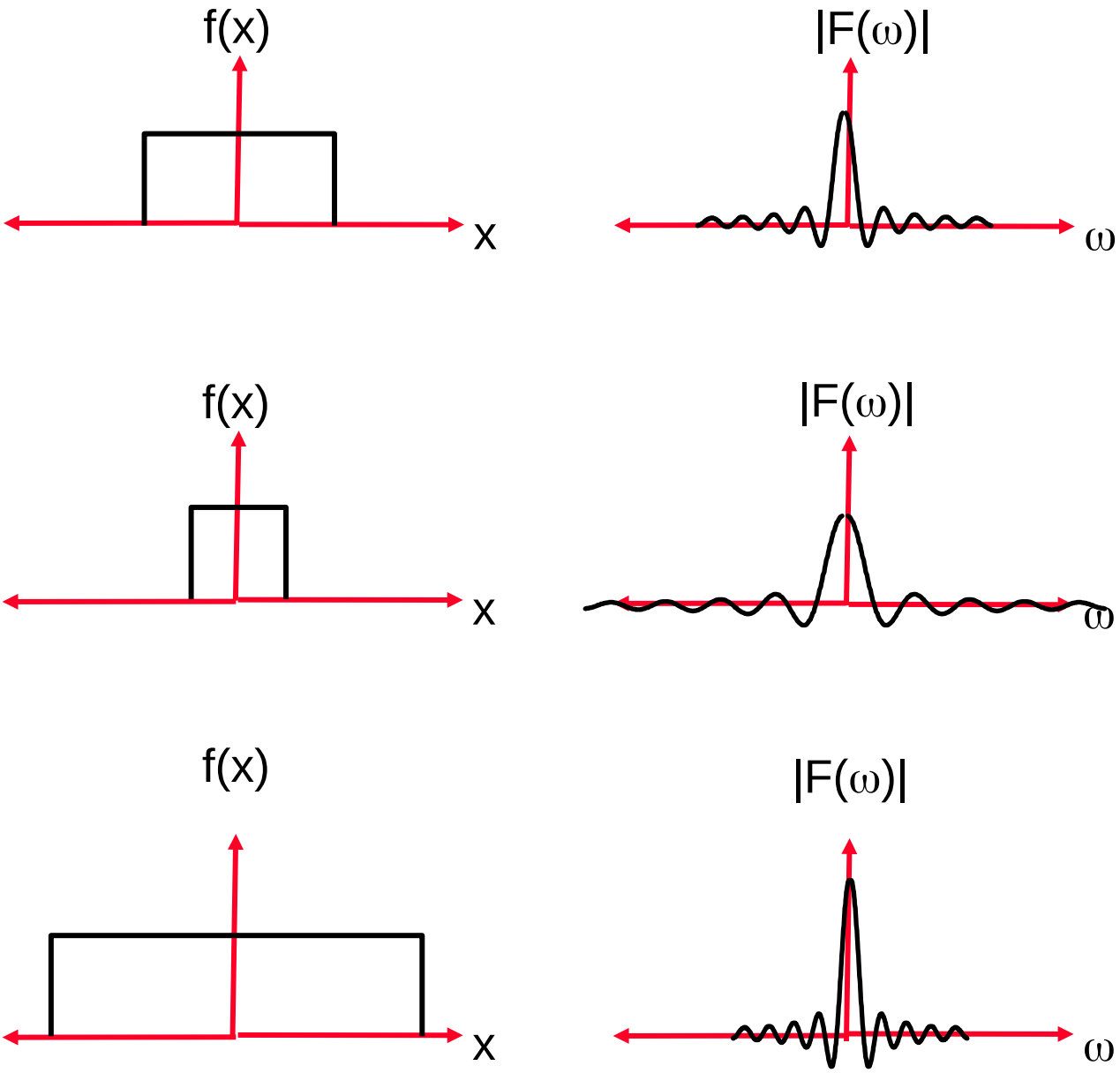
$$\tilde{F}\left[f(a\,x)\right] = \frac{1}{|a|}F\left(\frac{\omega}{a}\right)$$

Example - Translation

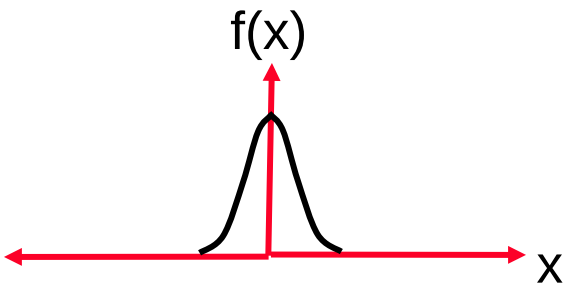


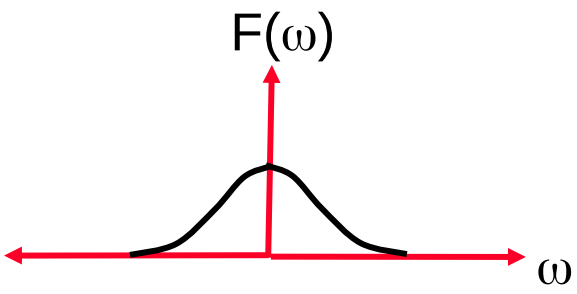
Change of Scale- 1D:

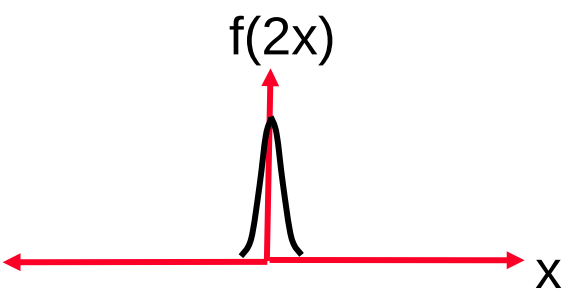
$$\text{if } \tilde{F}\{f(x)\} = F(\omega) \text{ then } \tilde{F}\{f(ax)\} = \frac{1}{|a|} F\left(\frac{\omega}{a}\right)$$

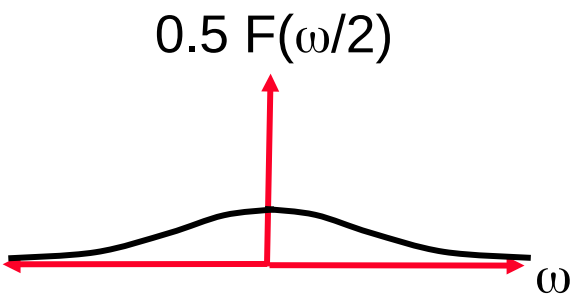


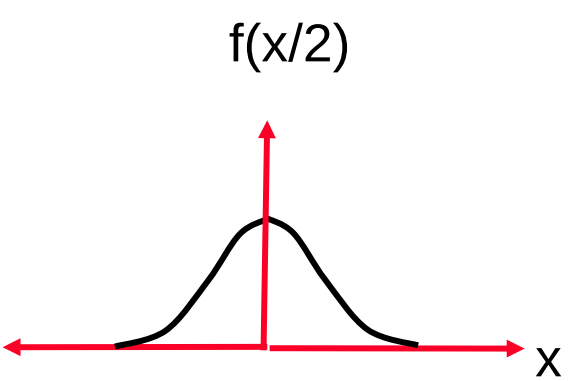
Change of Scale

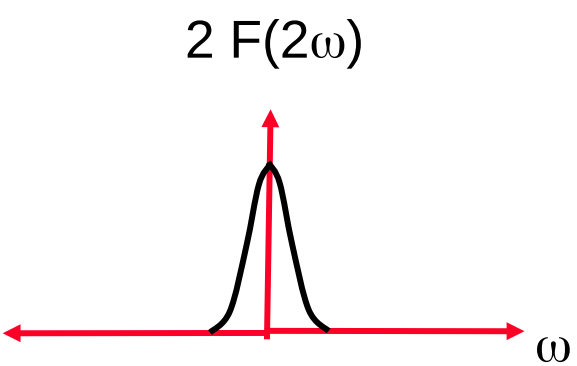






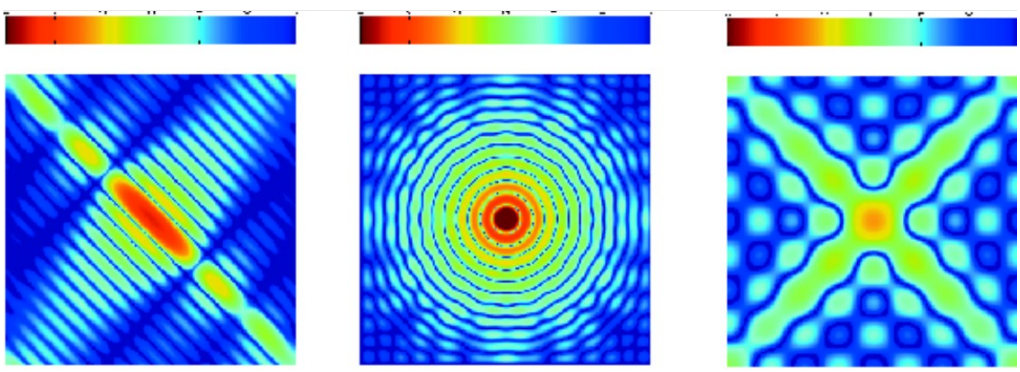


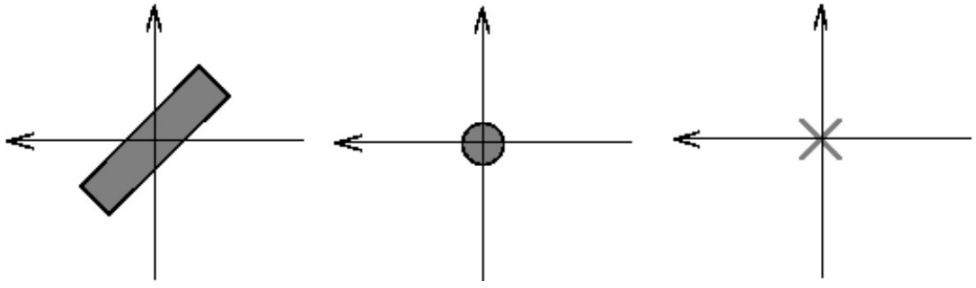




Fourier Transform 2D

- Discrete Fourier Transform - 2D
- Continues Fourier Transform - 2D
- Fourier Properties
- Convolution Theorem





The 2D Discrete Fourier Transform

- For an image $f(x,y)$ $x=0..N-1$, $y=0..M-1$, there are two-indices basis functions $B_{u.v}(x,y)$:

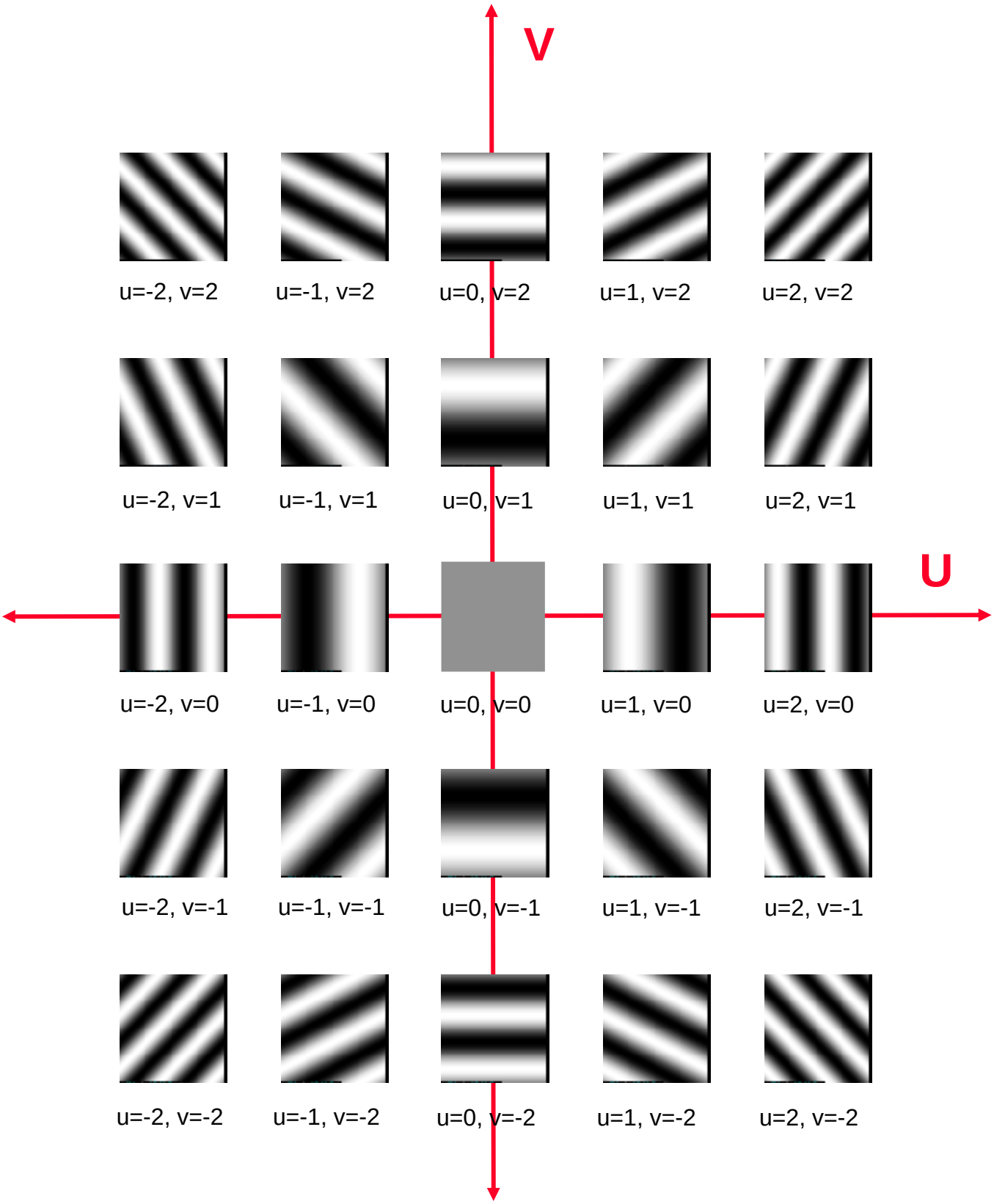
$$B_{u,v}(x,y) = \frac{1}{\sqrt{MN}} e^{2\pi i \left(\frac{ux}{N} + \frac{vy}{M} \right)}$$

$u=0..N-1$, $M=0..M-1$

- The inner product of 2 functions (in 2D) is defined similarly to the 1D case :

$$F(u,v) = \langle f(x,y), B_{u,v}(x,y) \rangle =$$

$$= \sum_{x=0}^{N-1} \sum_{y=0}^{M-1} f(x,y) B^{*}_{u,v}(x,y)$$



Note, b_{uv} and $b_{-u,-v}$ have similar frequencies but inverted shifts

The 2D Discrete Fourier Transform

- The 2D Discrete Fourier Transform (DFT) is defined as:

$$F(u,v) = \sum_{x=0}^{N-1} \sum_{y=0}^{M-1} f(x,y) e^{-2\pi i (ux/N + vy/M)}$$

$u = 0, 1, 2, ..., N-1$

$v = 0, 1, 2, ..., M-1$

Matlab: F=fft2(f);

- The Inverse Discrete Fourier Transform (IDFT) is defined as:

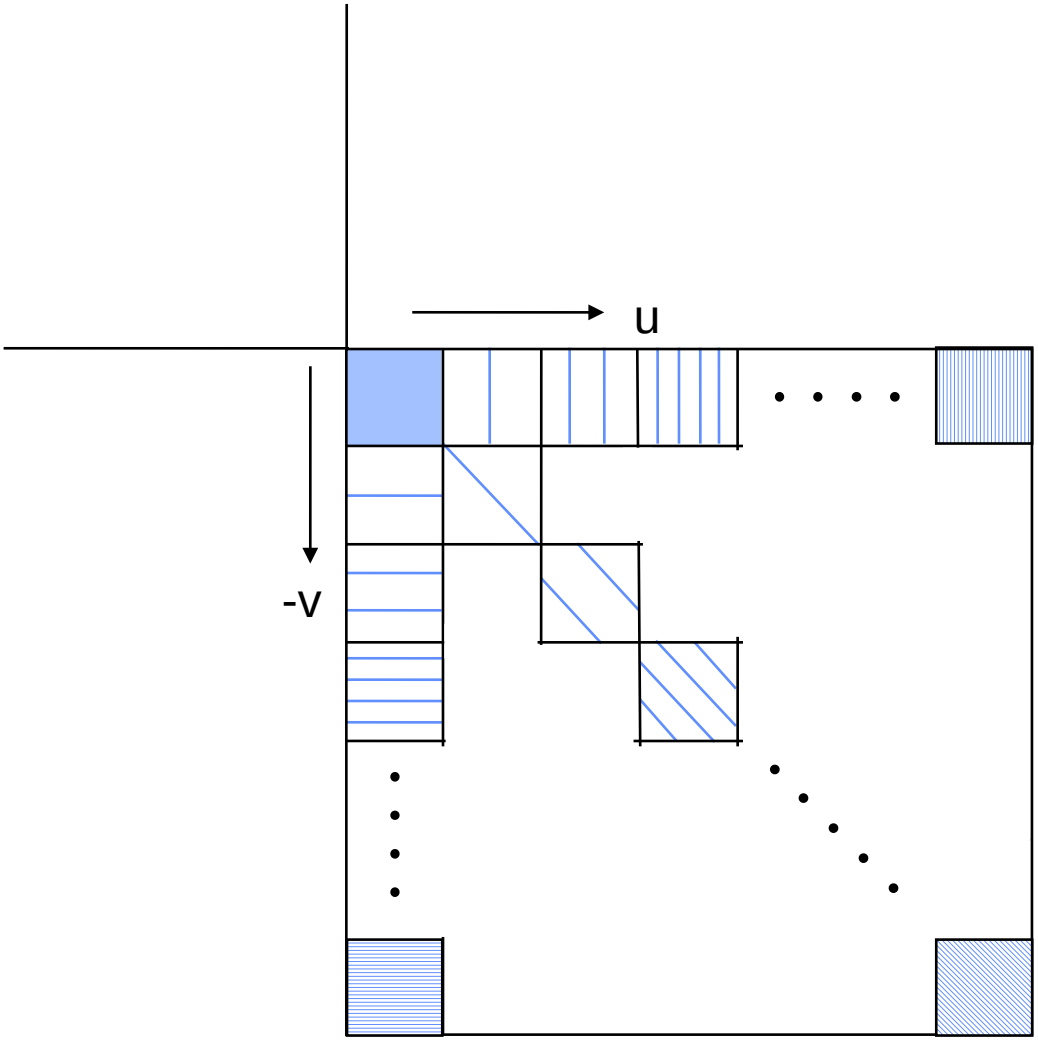
$$f(x,y) = \frac{1}{MN} \sum_{u=0}^{N-1} \sum_{v=0}^{M-1} F(u,v) e^{2\pi i (ux/N + vy/M)}$$

$y = 0, 1, 2, ..., N-1$

$x = 0, 1, 2, ..., M-1$

Matlab: F=ifft2(f);

Placing the coefficients



Visualizing the Fourier Transform Image using Matlab Routines

- F(u,v) is a Fourier transform of f(x,y) and it has complex entries.

```
F = fft2(f);
```

- In order to display the Fourier Spectrum |F(u,v)|
  - Reduce dynamic range of |F(u,v)| by displaying the log:

```
D = log(1+abs(F));
```

  - Cyclically rotate the image so that F(0,0) is in the center:

```
D = fftshift(D);
```

Example:

|F(u)| = 100 4 2 1 0 0 1 2 4

Display in Range([0..10]):

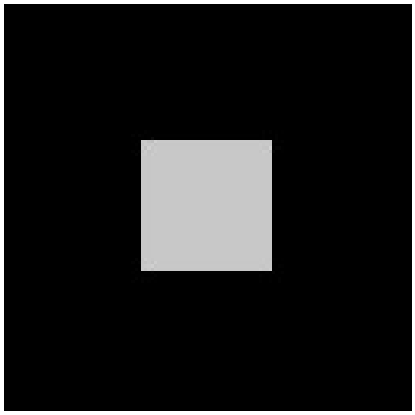
log(1+|F(u)|) = 4.62 1.61 1.01 0.69 0 0 0.69 1.01 1.61

log(1+|F(u)|)/0.0462 = 10 4 2 1 0 0 1 2 4

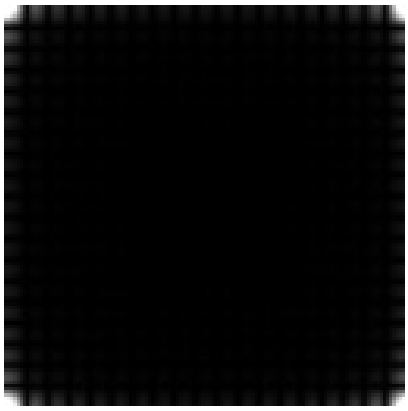
fftshift(log(1+|F(u)|) = 0 1 2 4 10 4 2 1 0

Visualizing the Fourier Image - Example

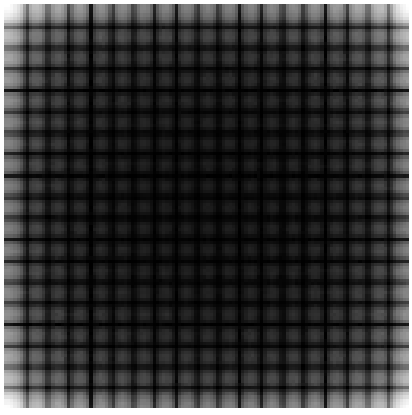
Original



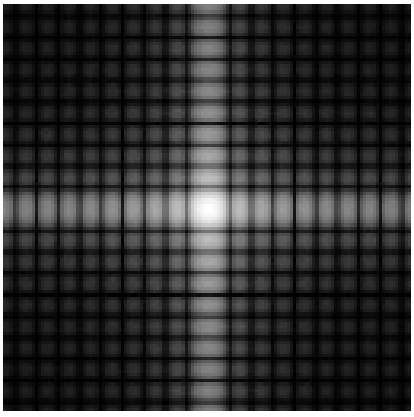
|F(u,v)|



log(1 + |F(u,v)|)

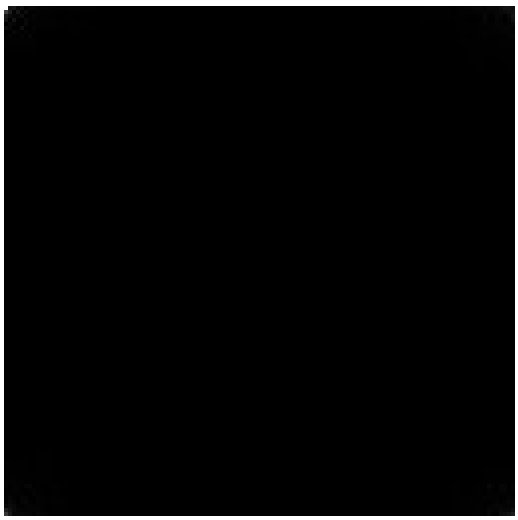


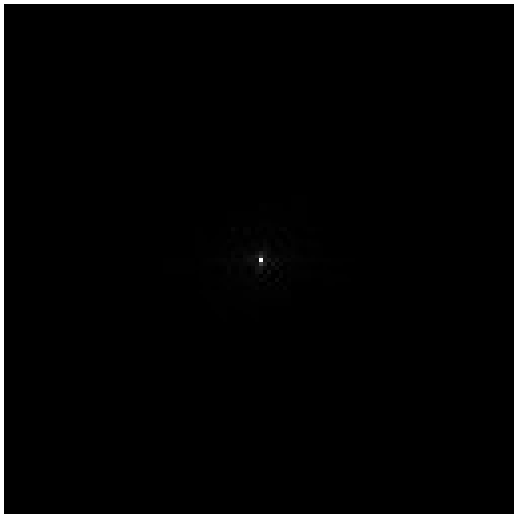
fftshift(log(1 + |F(u,v)|))



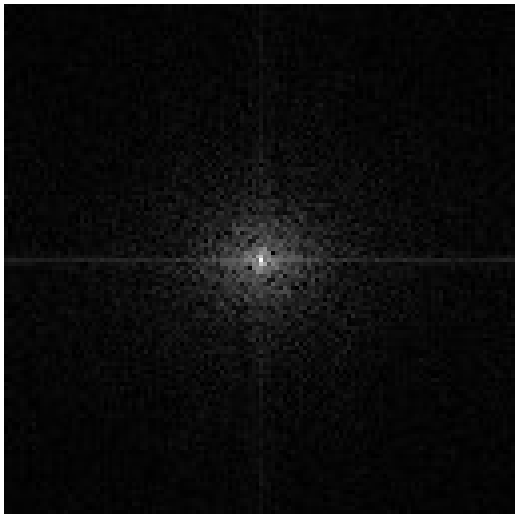
Original

Fourier Image = |F(u,v)|





Shifted Fourier Image



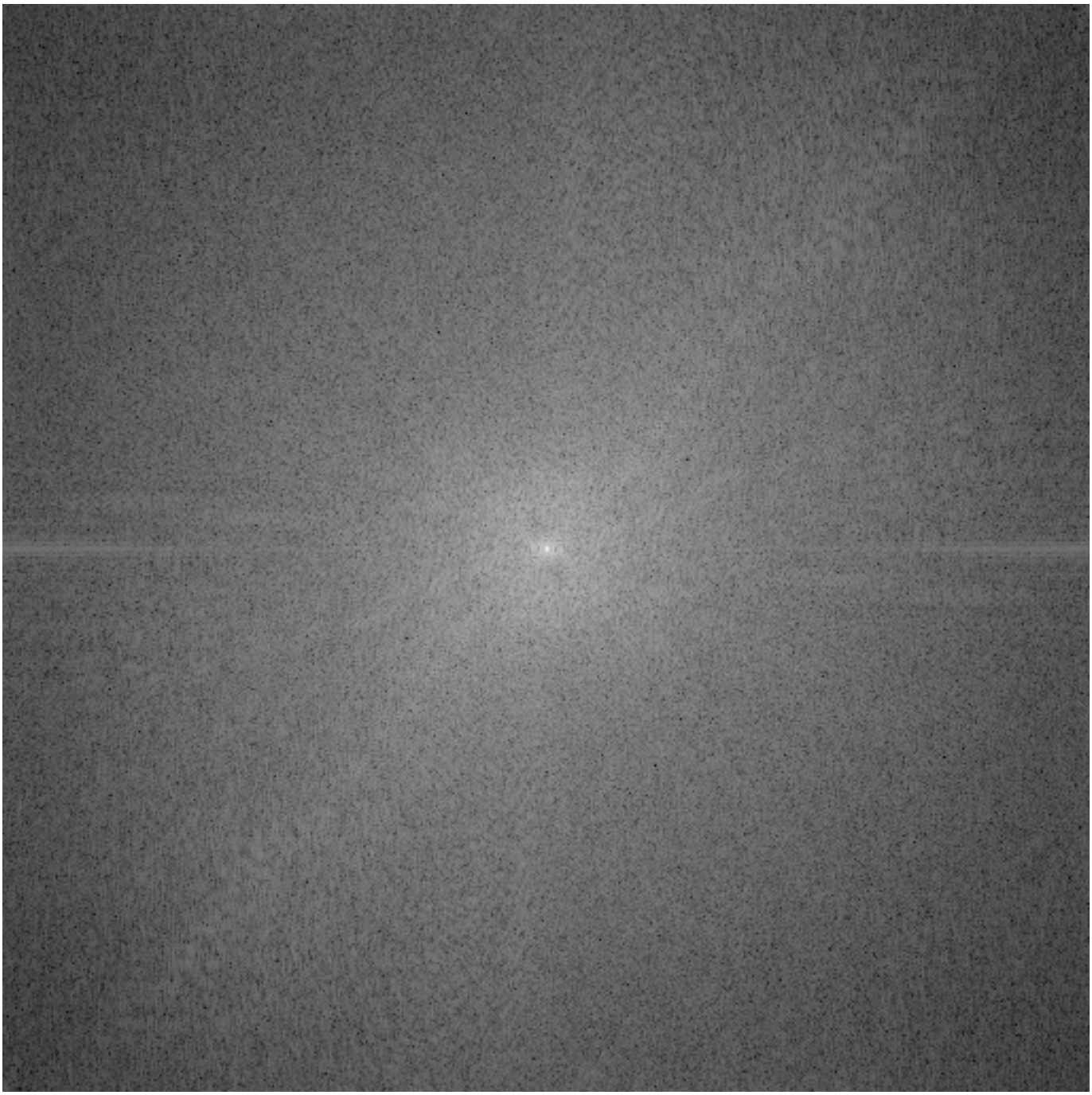
Shifted Log Fourier Image = log(1+ |F(u,v)|)

Phase and Magnitude

- Curious fact
  - all natural images have about the same magnitude transform
  - hence, phase seems to matter, but magnitude largely doesn't
- Demonstration
  - Take two pictures, swap the phase transforms, compute the inverse - what does the result look like?

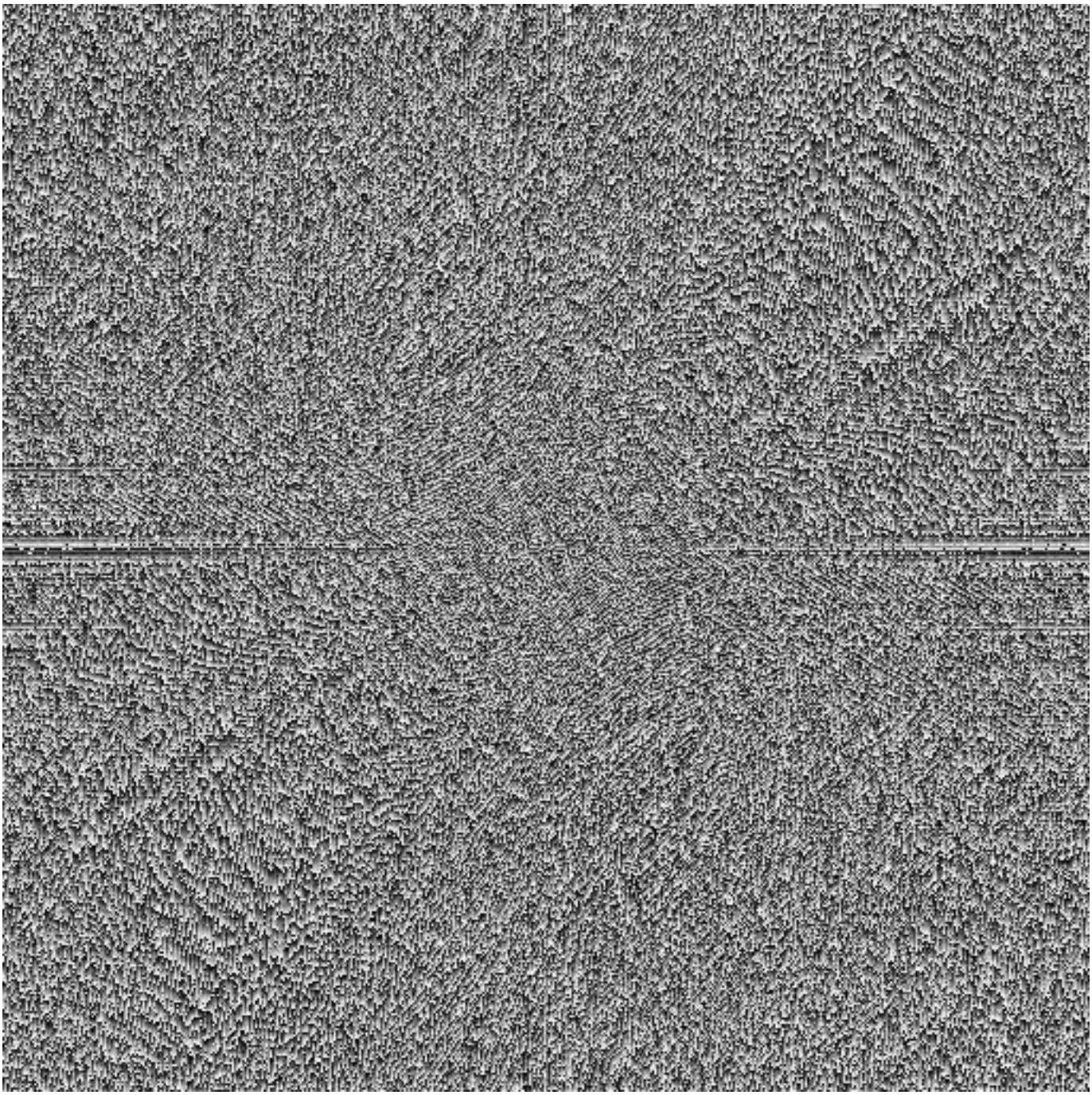
Slide: Freeman & Durand

Slide: Freeman & Durand



This is the magnitude transform of the cheetah pic

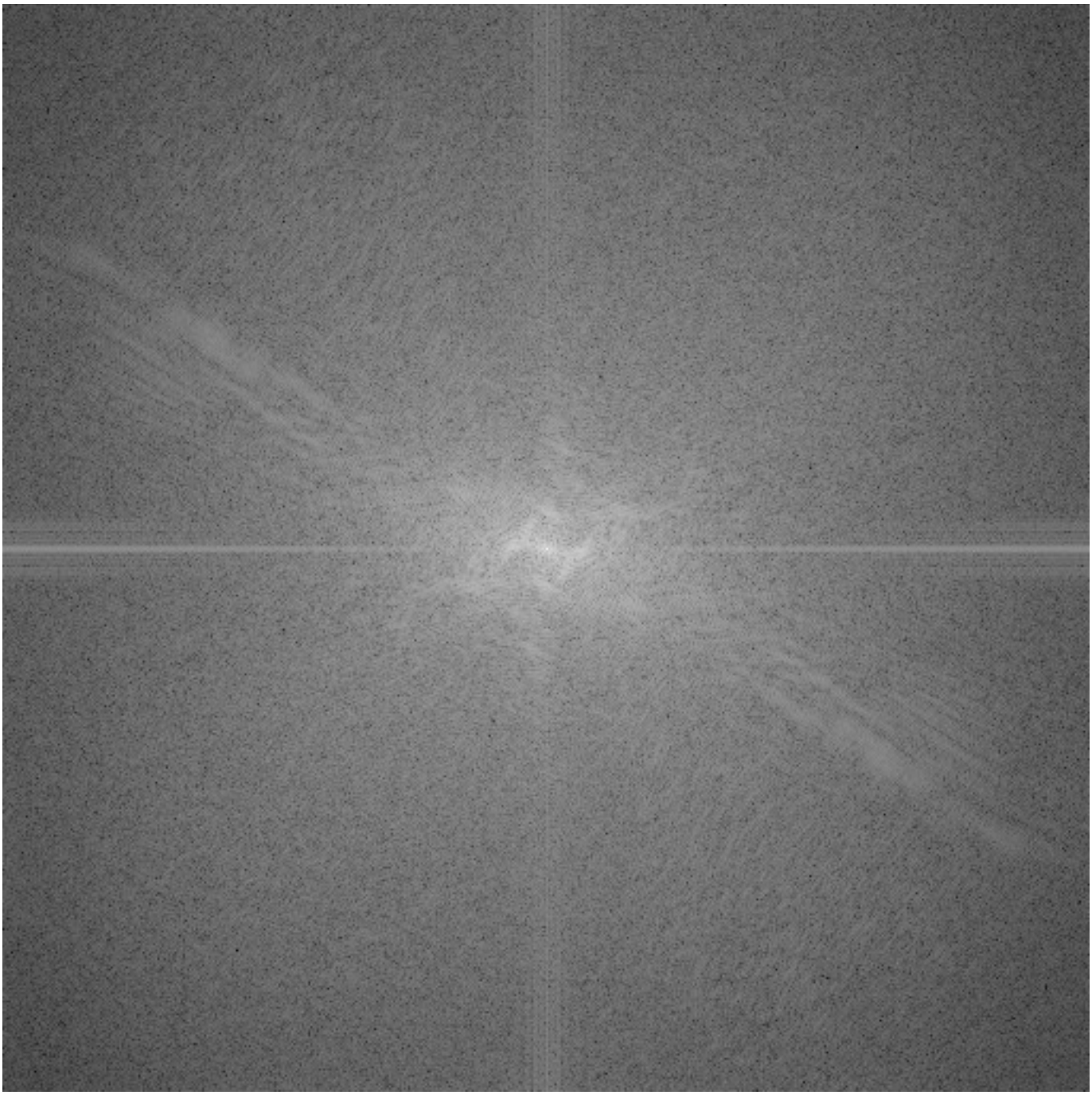
Slide: Freeman & Durand



This is the phase transform of the cheetah pic

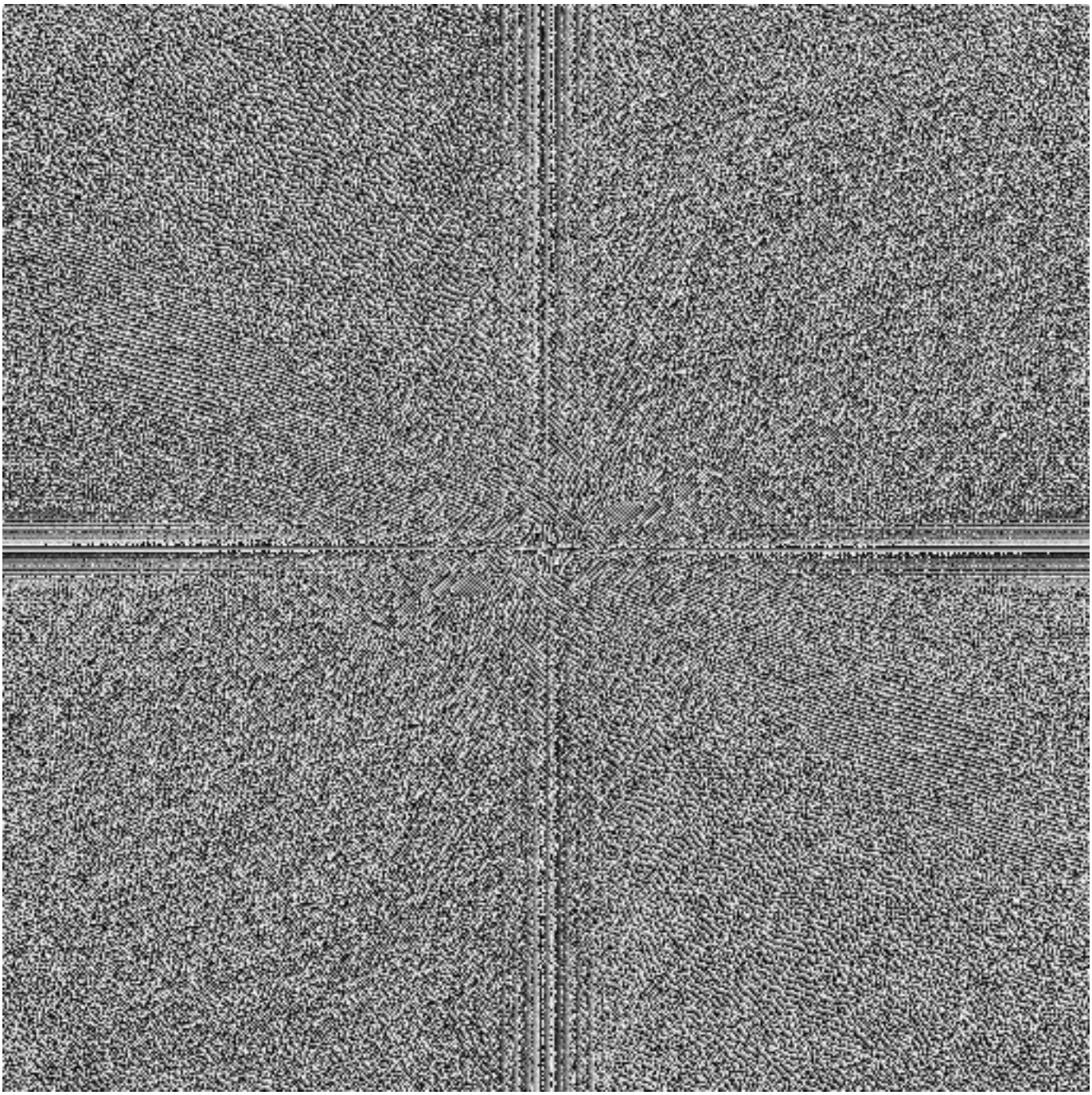
Slide: Freeman & Durand

Slide: Freeman & Durand



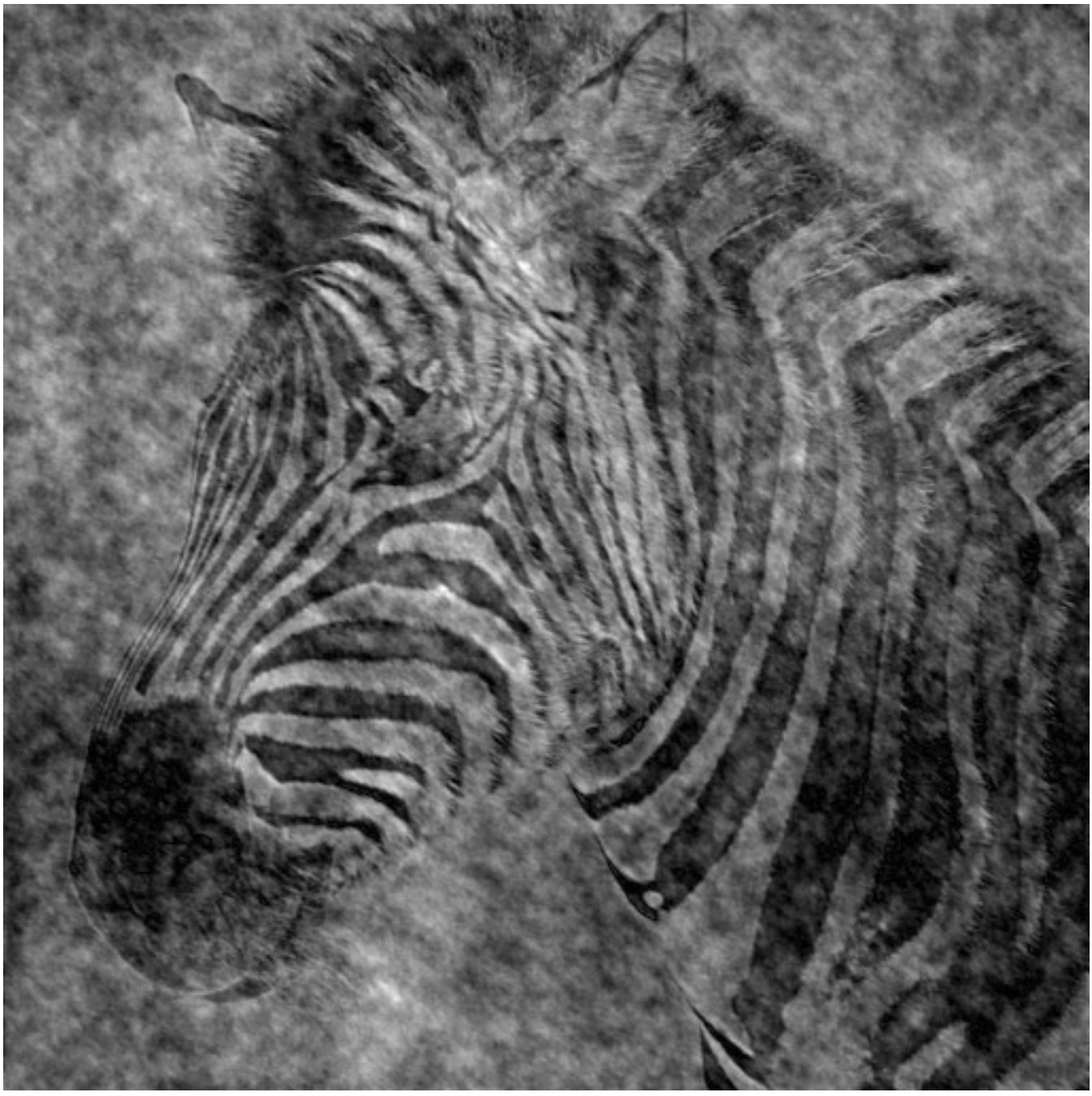
This is the magnitude transform of the zebra pic

Slide: Freeman & Durand



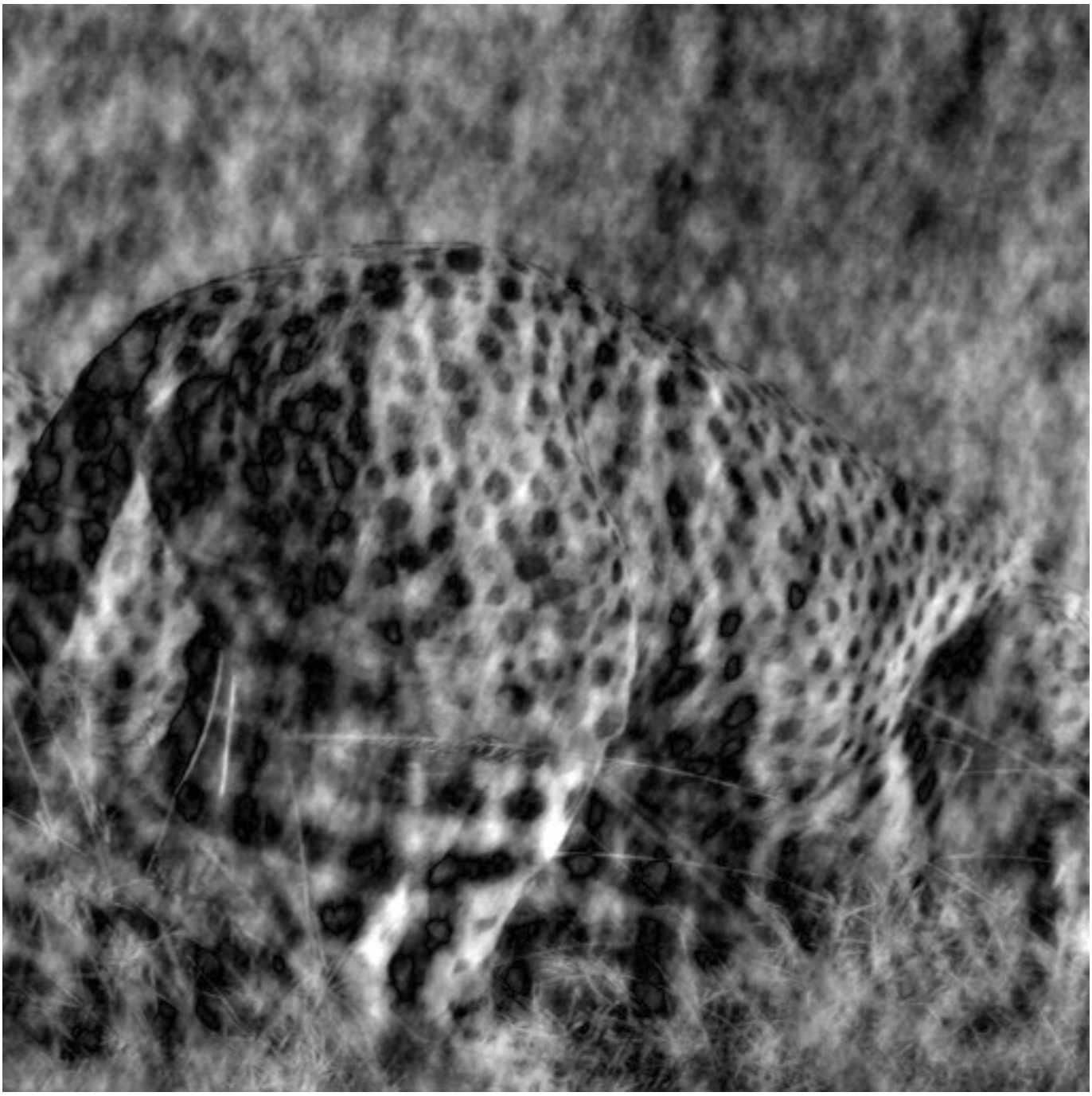
This is the phase transform of the zebra pic

Slide: Freeman & Durand



Reconstruction with zebra phase, cheetah magnitude

Slide: Freeman & Durand

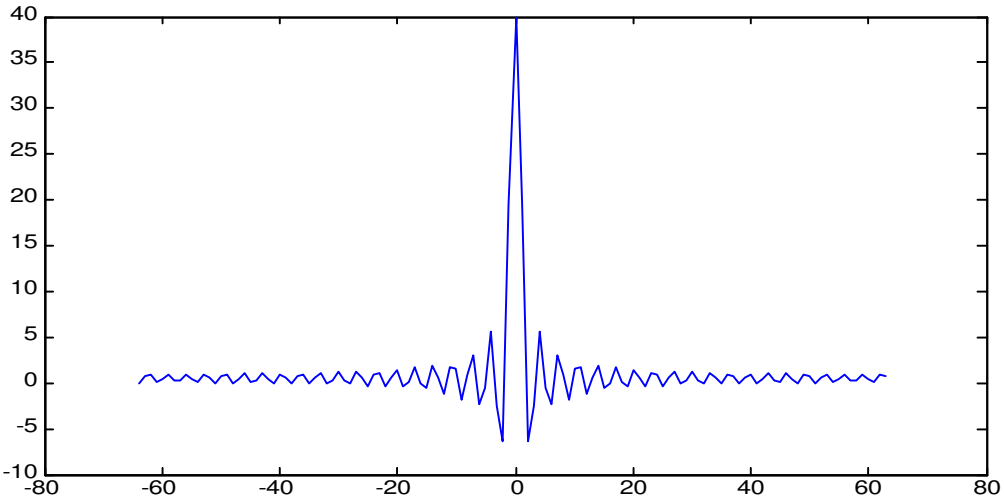


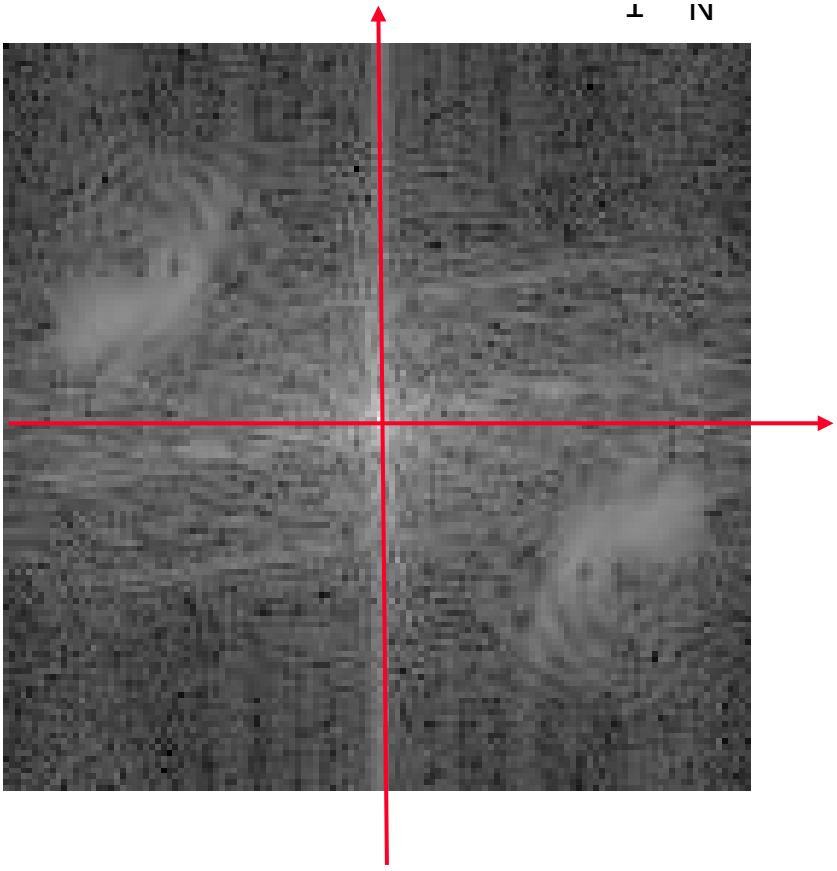
Reconstruction with cheetah phase, zebra magnitude

Slide: Freeman & Durand

Properties of the 2D Fourier Transform

Symmetry of the Fourier Transform for real signals





For real signals/images: $F(u,v) = F^{*}(-u,-v)$

Properties: Cont.

- Separability

$$F(u,v) = \sum_x \sum_y f(x,y) e^{-2\pi i \left(\frac{ux}{N} + \frac{vy}{M} \right)} =$$

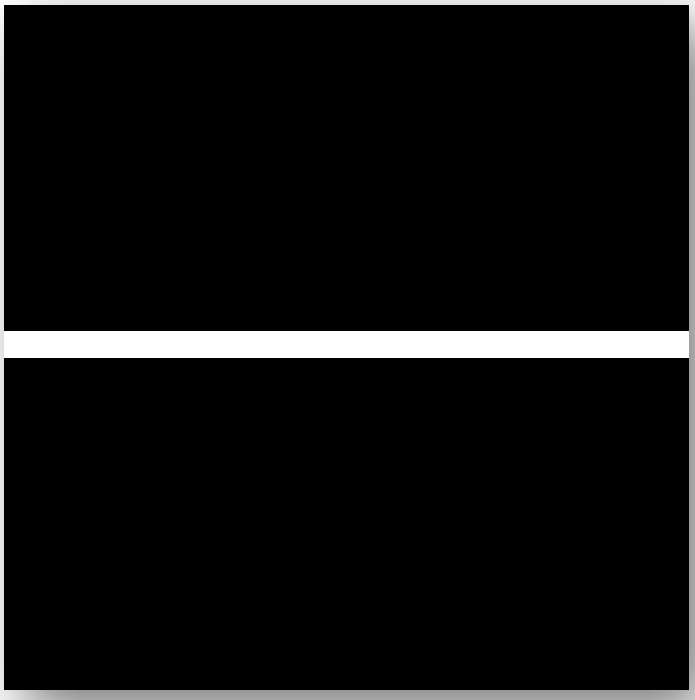
$$= \sum_x \left(\sum_y f(x,y) e^{-2\pi i \frac{vy}{N}} \right) e^{-2\pi i \frac{ux}{N}} = \sum_x F(x,v) e^{-2\pi i \frac{ux}{N}}$$

- Thus, performing a 2D Fourier Transform is equivalent to performing 2 1D transforms:
  1. Perform 1D transform on EACH column of image f(x,y), obtaining F(x,v).
  2. Perform 1D transform on EACH row of F(x,v), obtaining F(u,v).

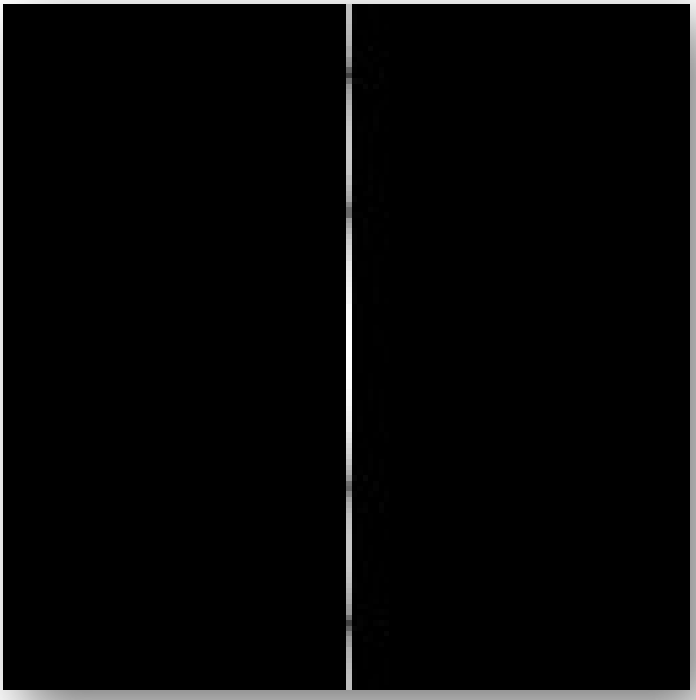
- Higher Dimensions: Fourier in any dimension can be performed by applying 1D transform on each dimension.

Example

2D Image



Fourier Spectrum



Properties: 2D Transformations

- **Translation**:

$$\tilde{F}\left[f(x-x_0, y-y_0)\right] = F(u,v)e^{-2\pi i\left(\frac{ux_0}{N}+\frac{vy_0}{M}\right)}$$

The Fourier Spectrum remains unchanged under translation:

$$|F(u,v)| = \left|F(u,v)e^{-2\pi i\left(\frac{ux_0}{N}+\frac{vy_0}{M}\right)}\right|$$

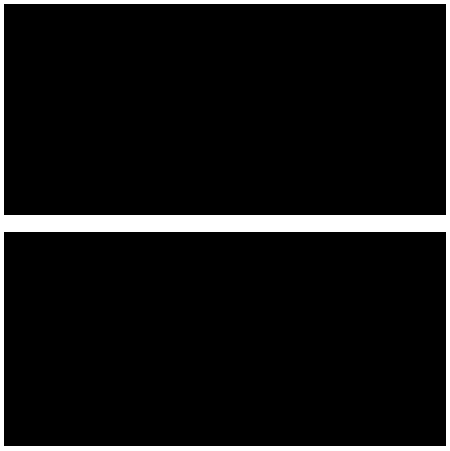
- **Rotation**: Rotation of f(x,y) by θ $\rightarrow$ rotation of F(u,v) by θ

- **Scaling**:

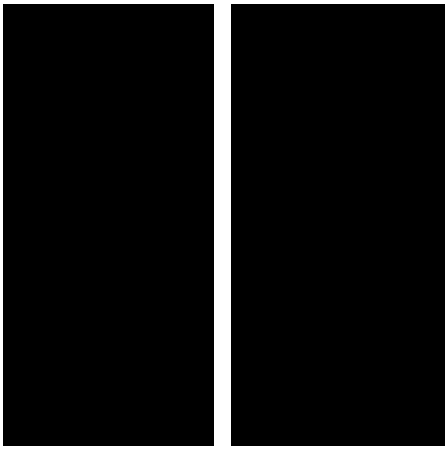
$$\tilde{F}\left[f(a\,x, b\,y)\right] = \frac{1}{|ab|}F\left(\frac{u}{a}, \frac{v}{b}\right)$$

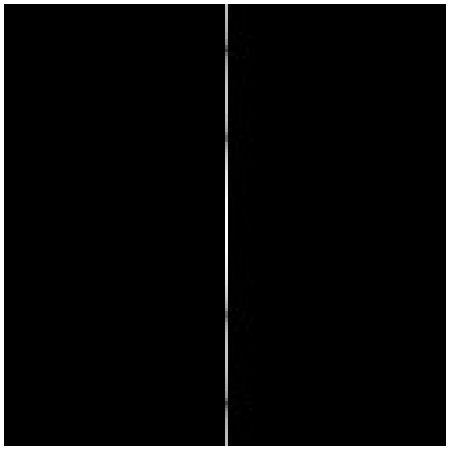
Example - Rotation

2D Image

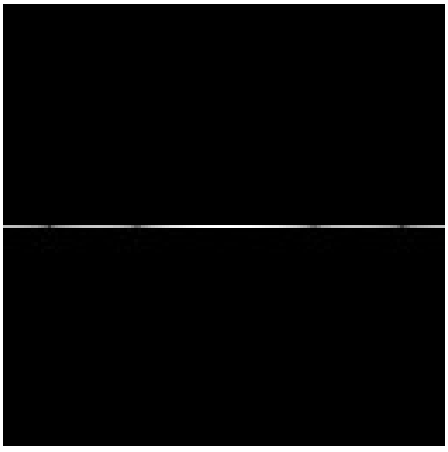


2D Image - Rotated





Fourier Spectrum



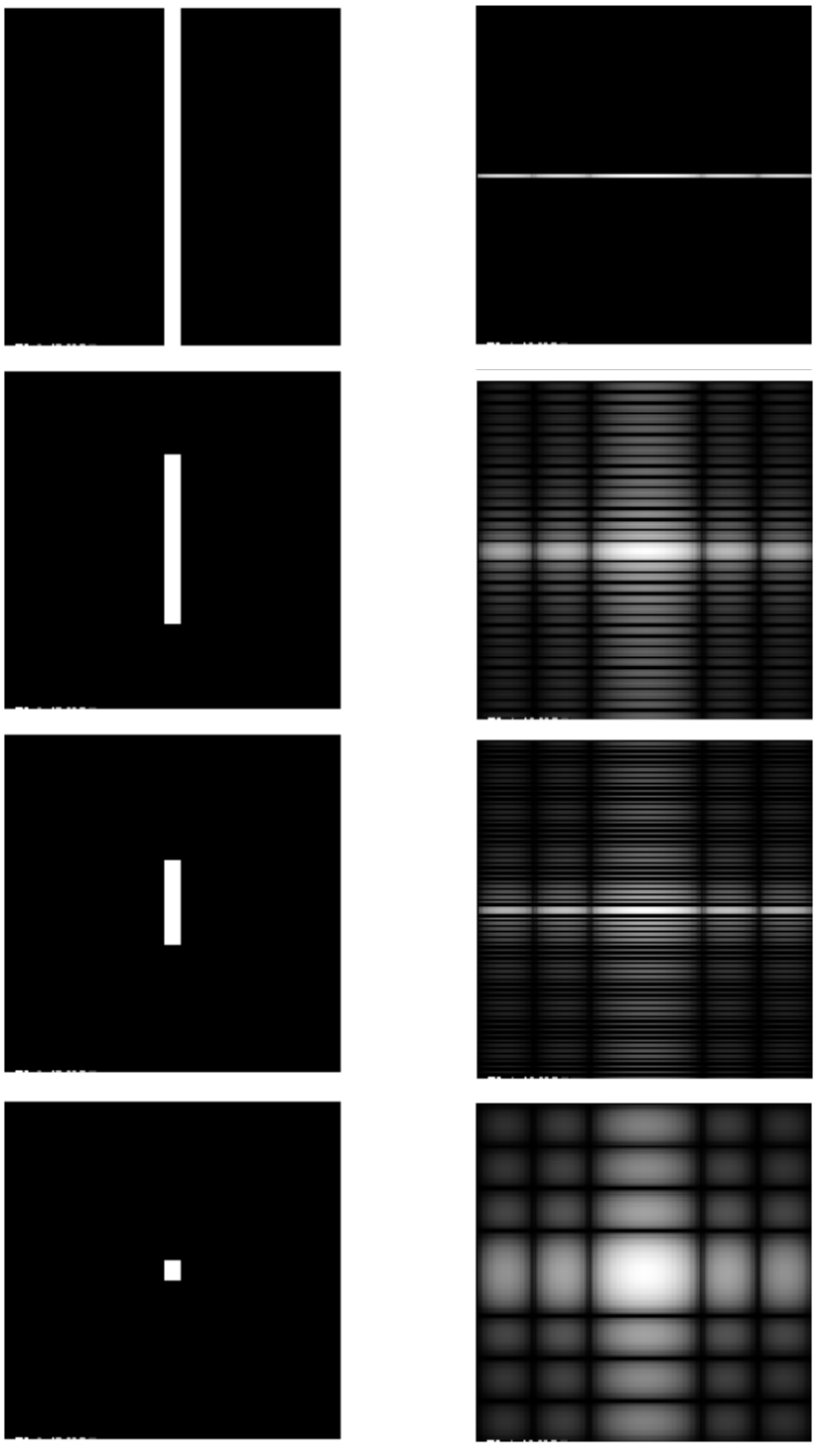
Fourier Spectrum

Fourier Transform Examples

Image Domain Frequency Domain

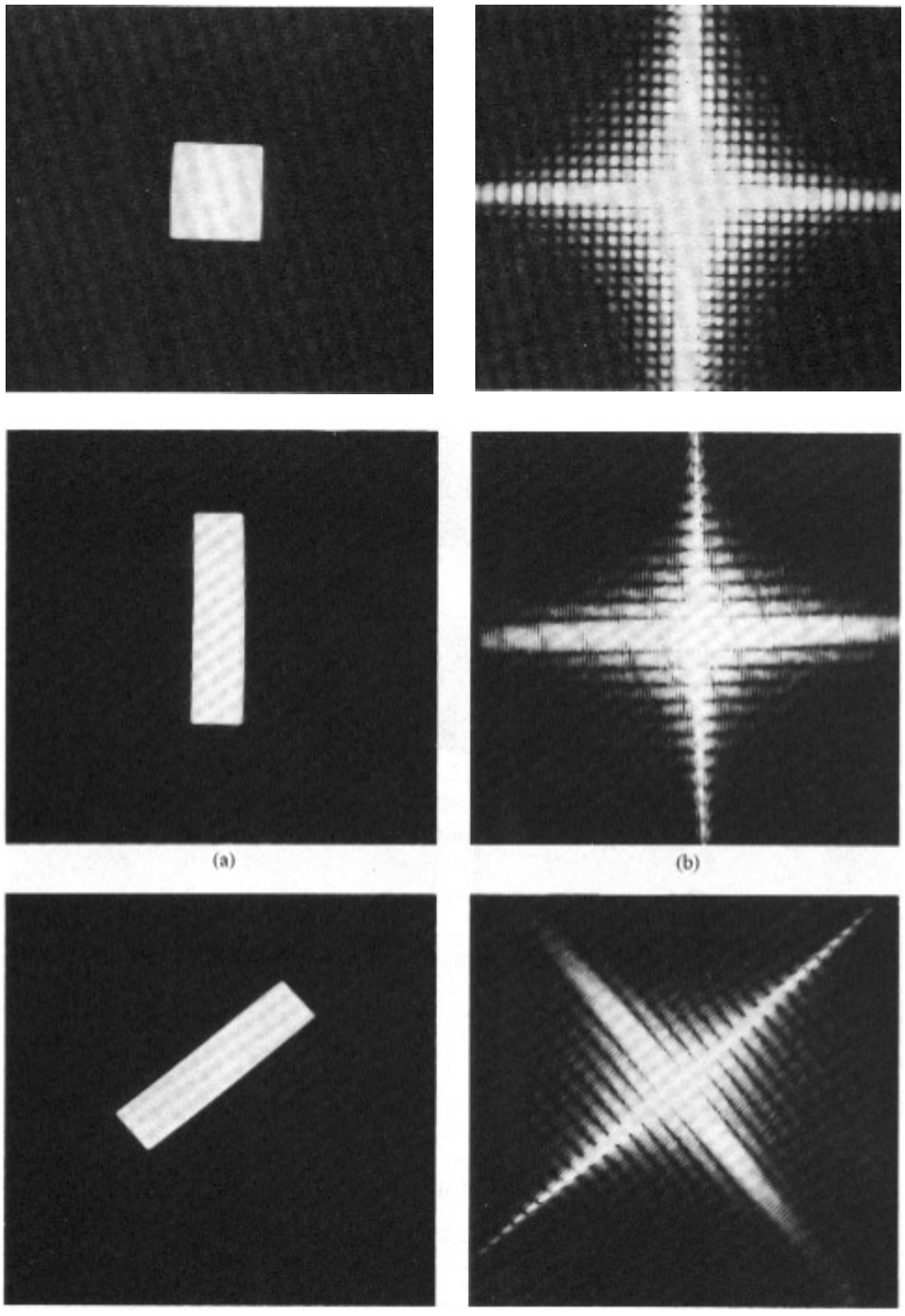
Fourier Transform Examples

Image Domain Frequency Domain



Fourier Transform Examples

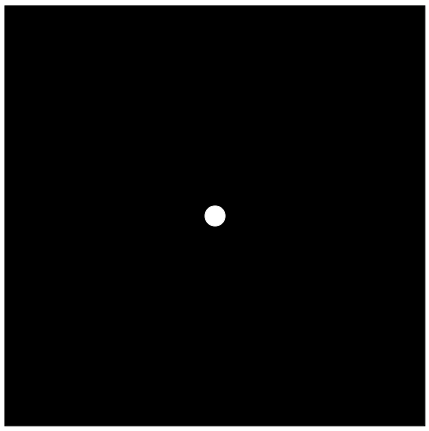
Image Domain Frequency Domain

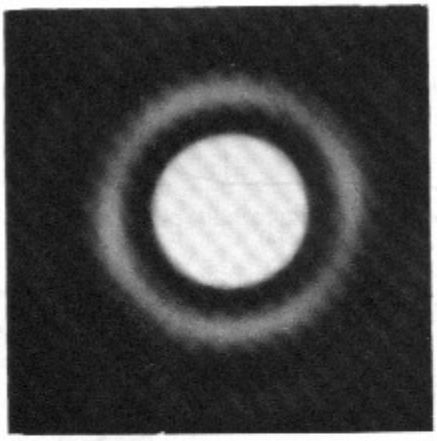


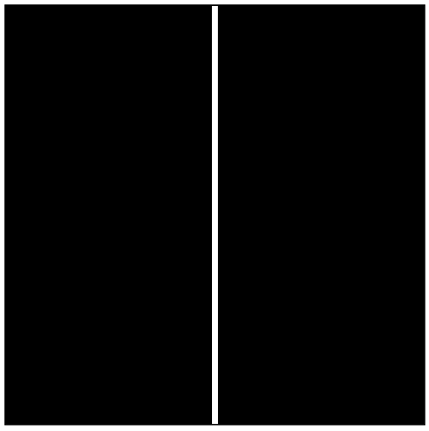
Fourier Transform Examples

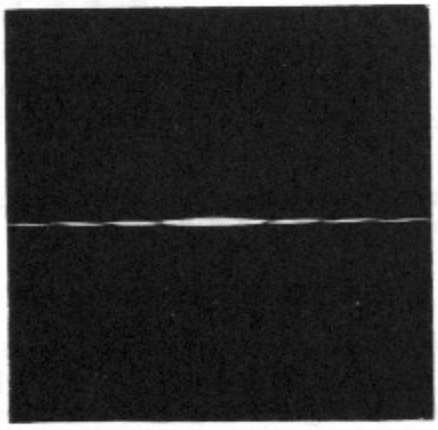
Image Domain

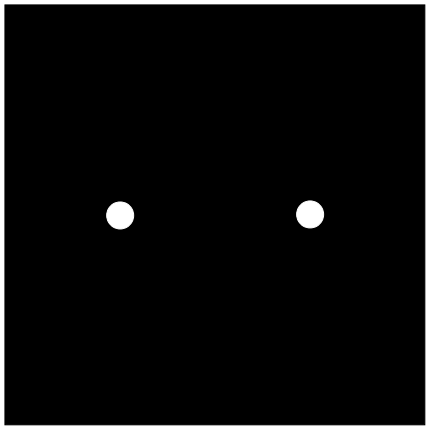
Frequency Domain

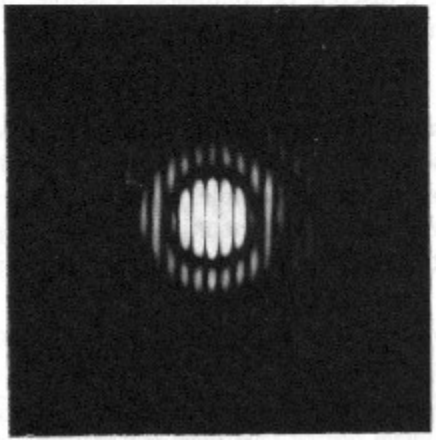








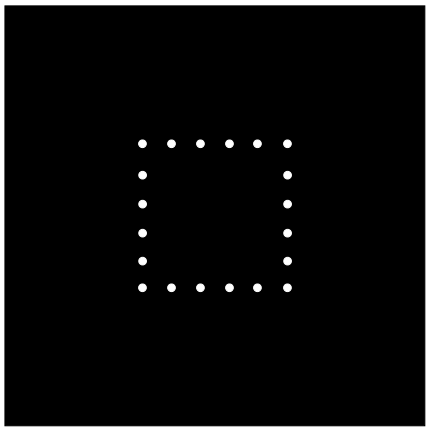


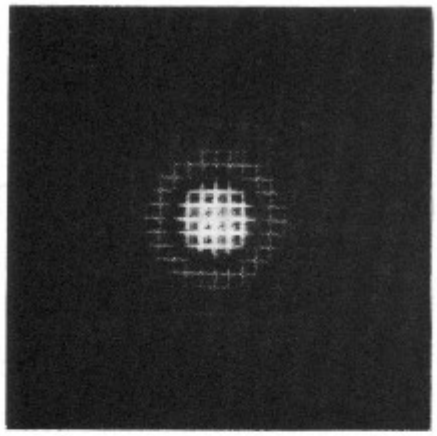


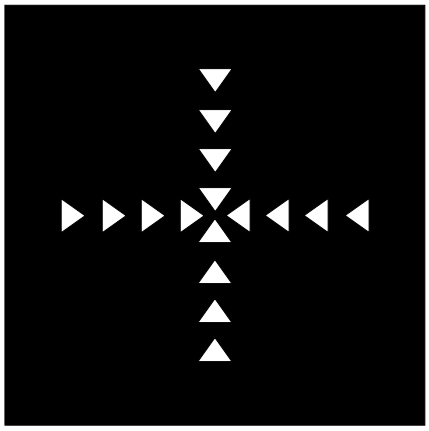
Fourier Transform Examples

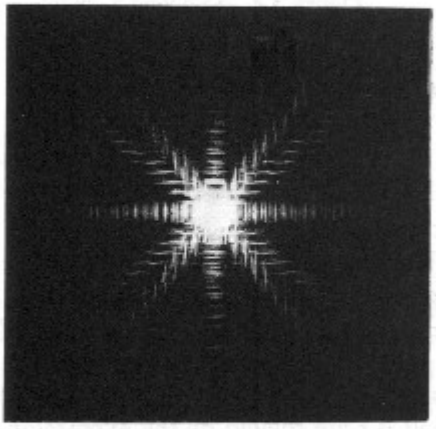
Image Domain

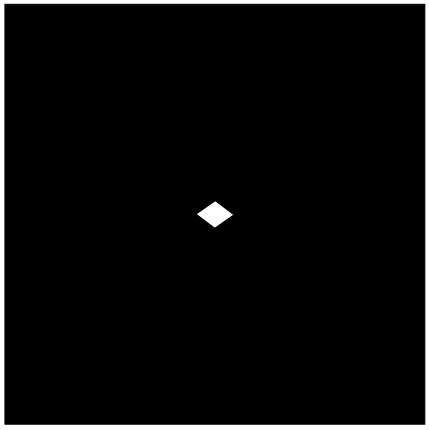
Frequency Domain

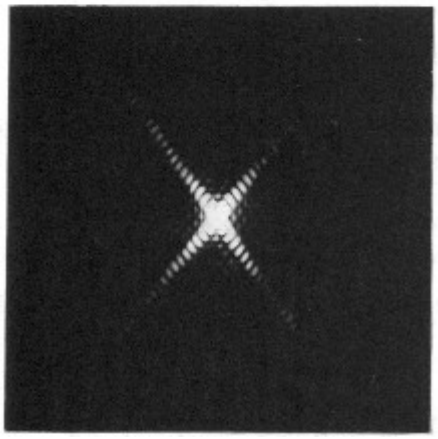




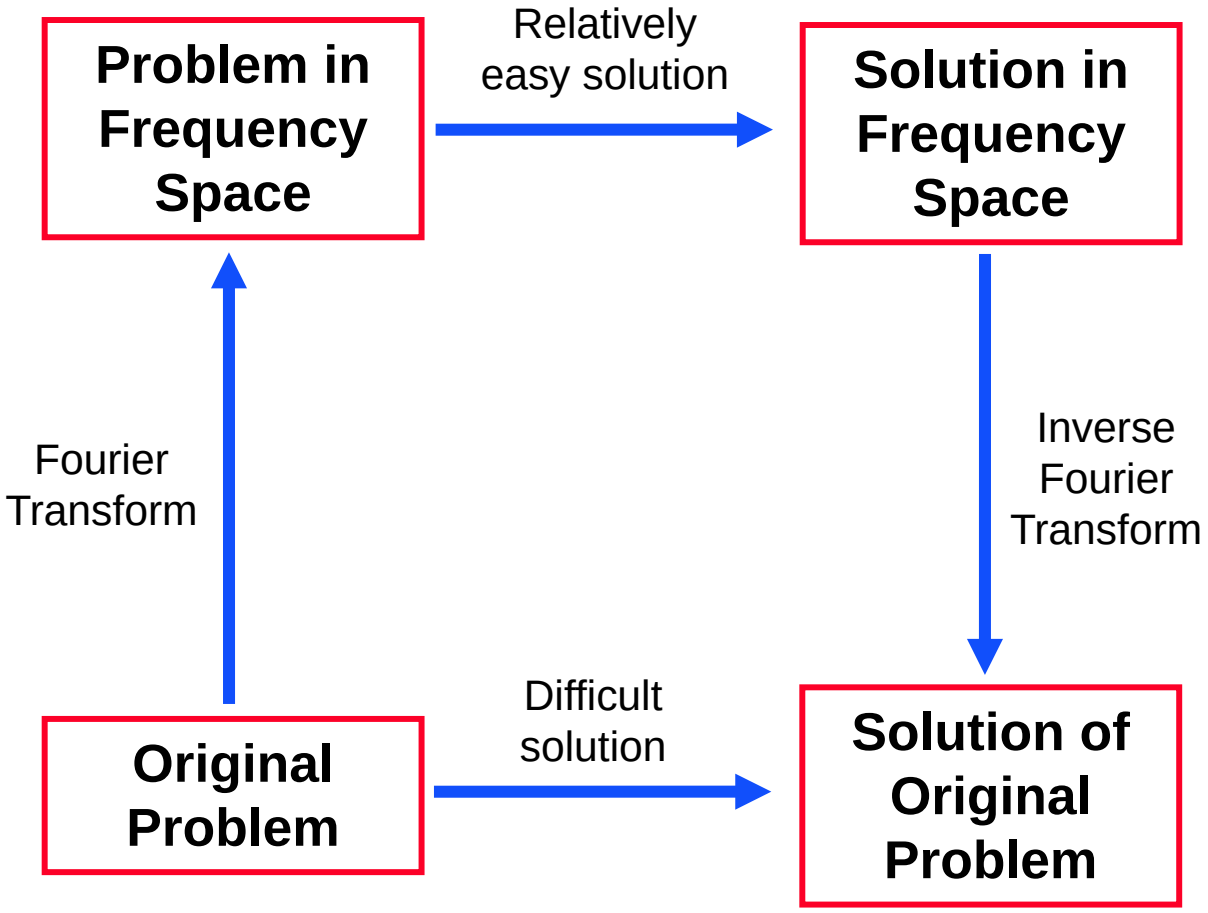








Why do we need representation in the frequency domain?



The Convolution Theorem

$g = f * h$	$g = f\ h$
implies	implies
$G = F\ H$	$G = F * H$

Convolution in one domain is multiplication in the other and vice versa

The Convolution Theorem

$$\tilde{F}\{f(x) \star g(x)\} = \tilde{F}\{f(x)\}\,\tilde{F}\{g(x)\}$$

and likewise

$$\tilde{F}\{f(x)\,g(x)\} = \tilde{F}\{f(x)\} \star \tilde{F}\{g(x)\}$$

The Convolution Theorem

- A convolution can be represented as a matrix multiplication:

$$y=Ax$$

where A is a circulant matrix.

- Let F be a matrix composed of the Fourier bases:

$$F^T y=F^T AFF^T x$$

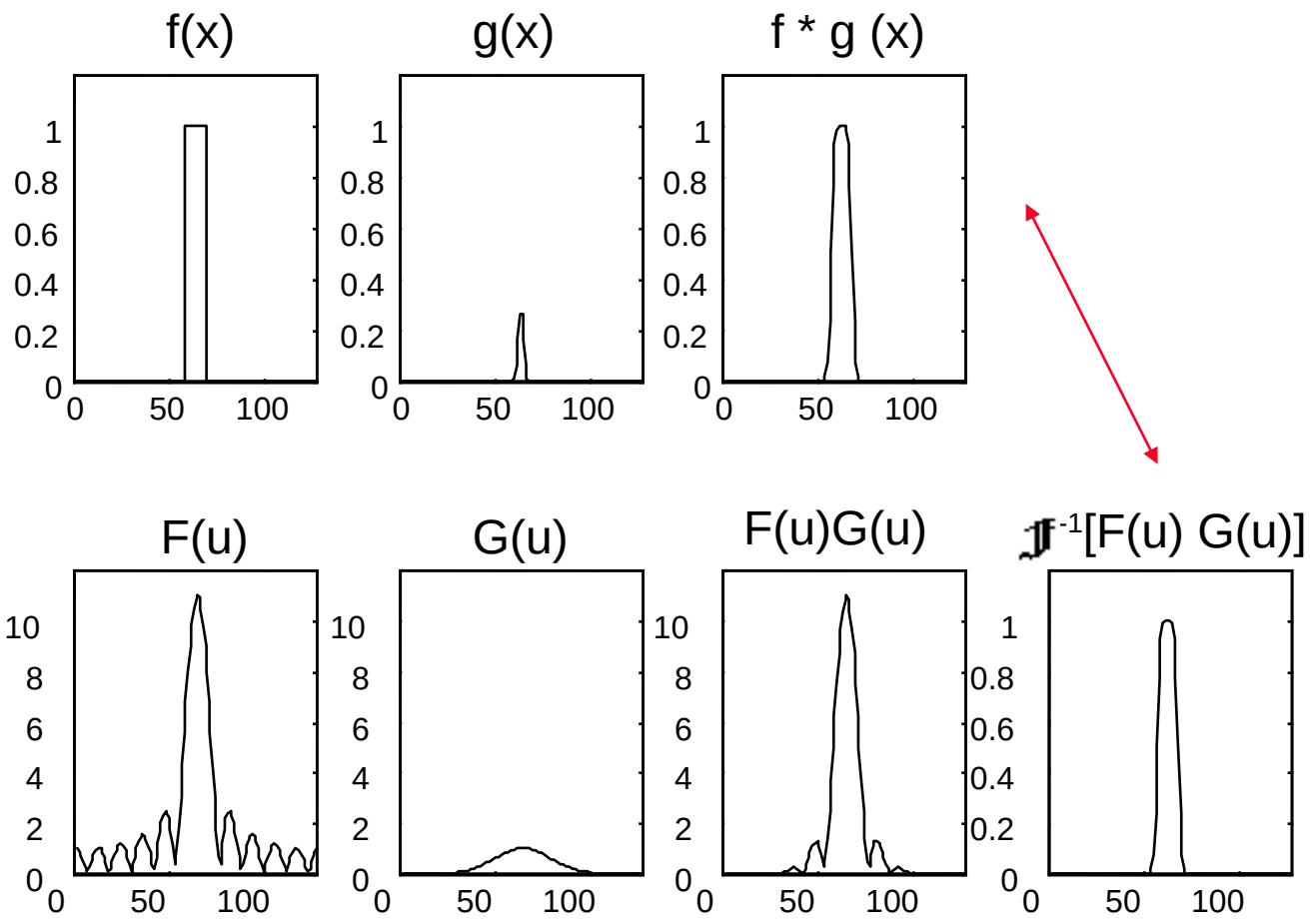
- Fourier bases are the eigenvectors of circulant matrices, thus:

$$Y=DX$$

where X and Y are the Fourier pairs of x and y, resp. and D is a diagonal matrix.

- The Convolution theorem is nothing than diagonalizing the system.

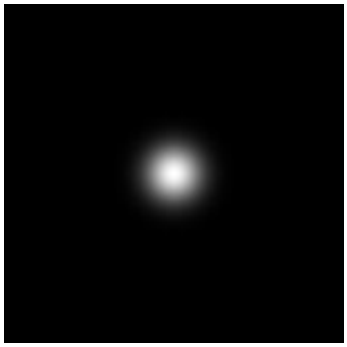
Convolution Theorem - Example



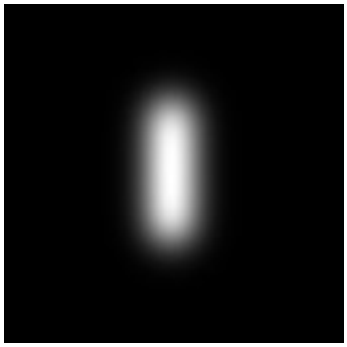
Convolution Theorem - 2D Example

f(x,y)

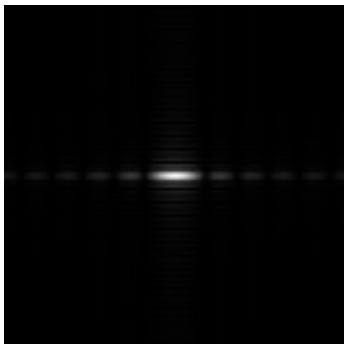
g(x,y)



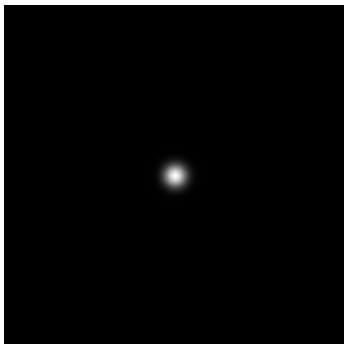
f * g (x,y)



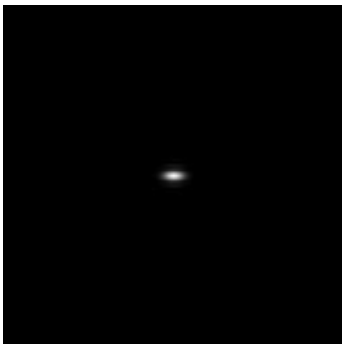
F(u,v)



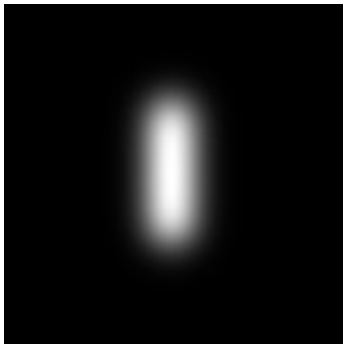
G(u,v)



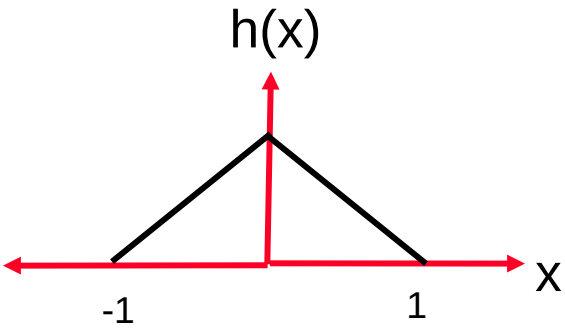
F(u,v) G(u,v)



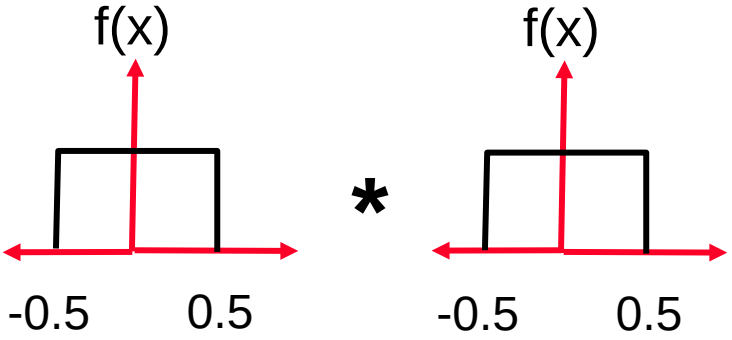
$\mathcal{F}^{-1}[F(u,v) \quad ' G(u,v)]$



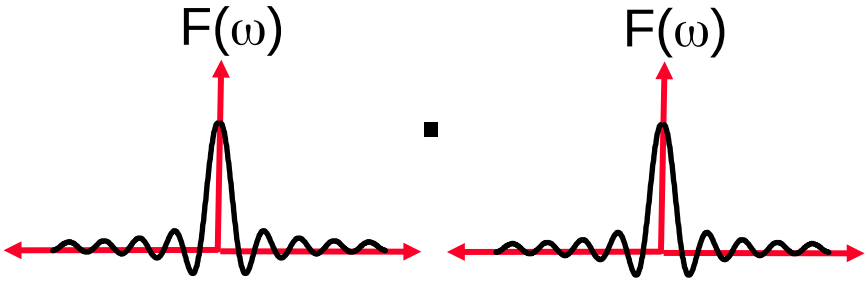
Example: What is the Fourier Transform of:



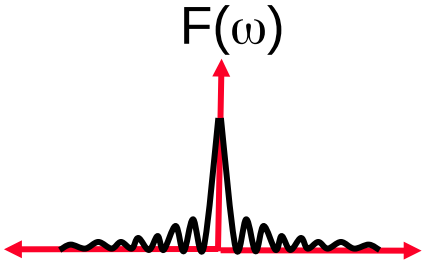
h(x) **=**



H(ω) **=**



=



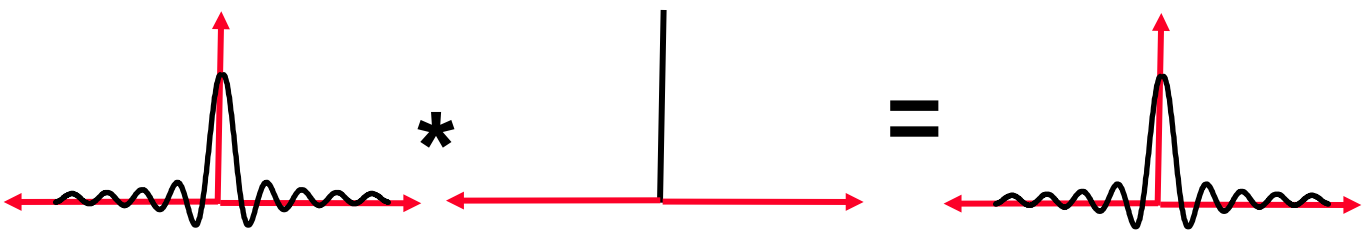
Example: What is the Fourier Transform of the Dirac Function?

$$\delta(x) = \begin{cases} 1 & if \quad x = 0 \\ 0 & otherwise \end{cases}$$

Proof : Consider any function **f(x)**

$$f(x) * \delta(x) = f(x)$$

$$F(u) \cdot \mathrm{F}[\delta(x)] = F(u)$$



$$\tilde{F}[\delta(x)] = 1$$

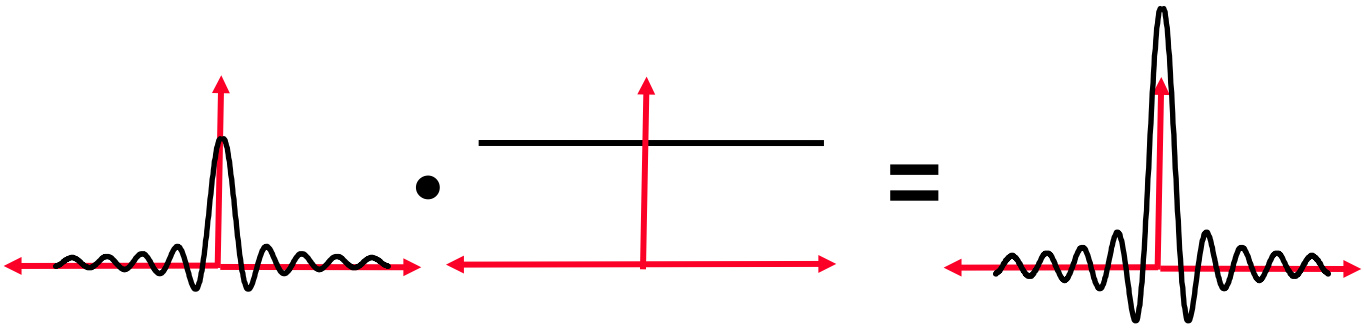
Example: What is the Fourier Transform of a constant function?

$$g(x) = c$$

Proof : Consider any function **g(x)**

$$f(x)g(x) = cf(x)$$

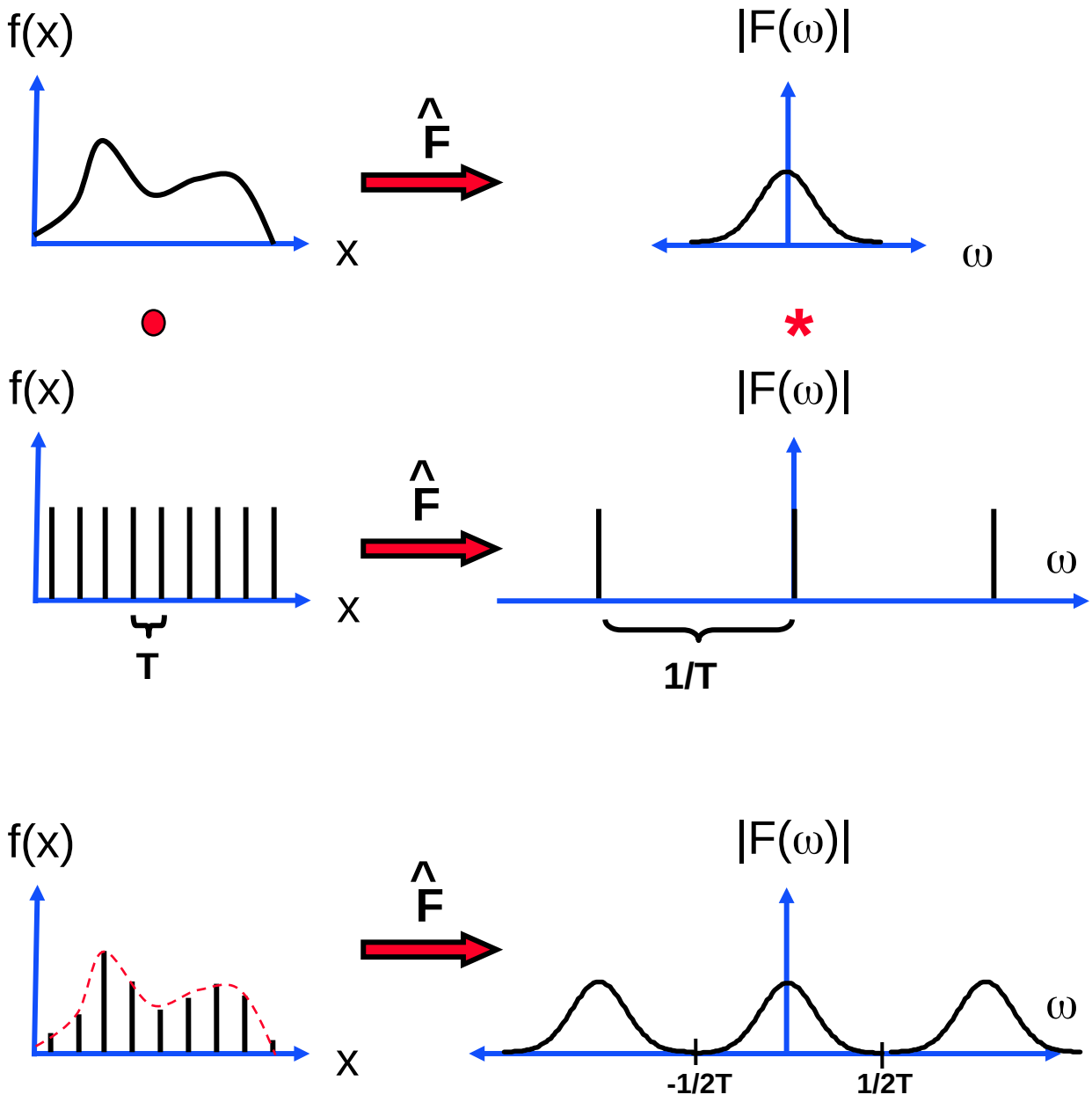
$$F(u) \star G(u) = cF(u)$$



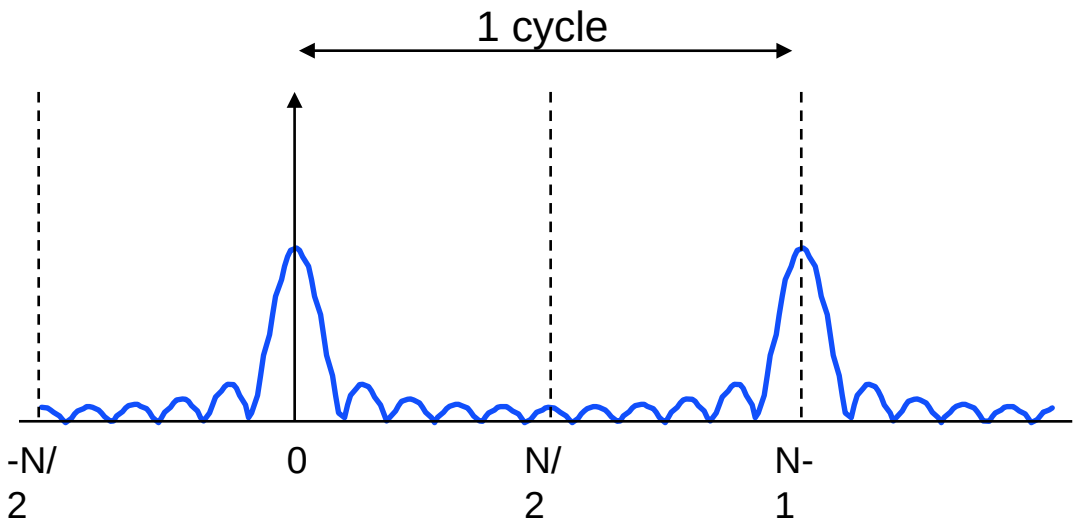
$$\tilde{F}[c] = c\delta(u)$$

Sampling The Image

- Sampling a function f(x) with impulse train of cycle T produces replicas in the frequency domain with cycle 1/T:

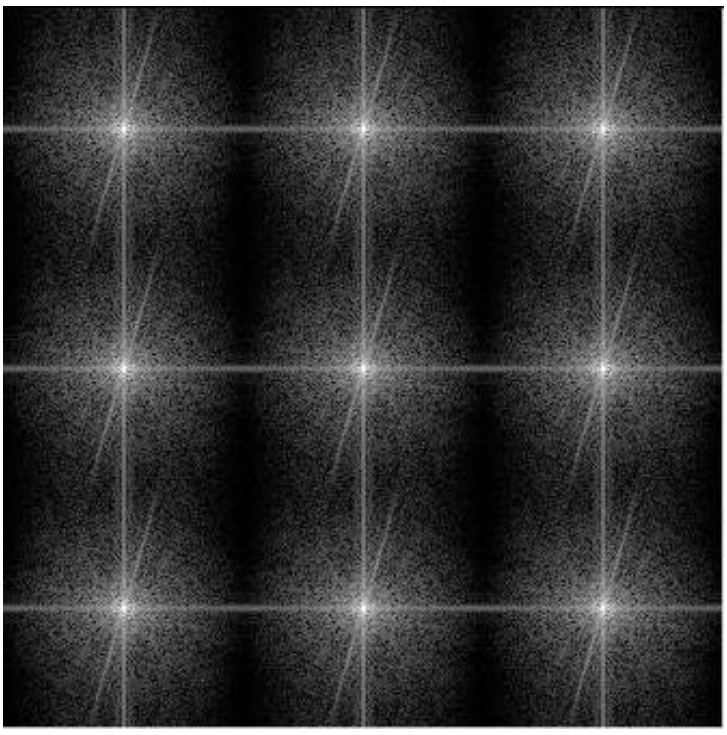


Cyclic and symmetry of discrete signals



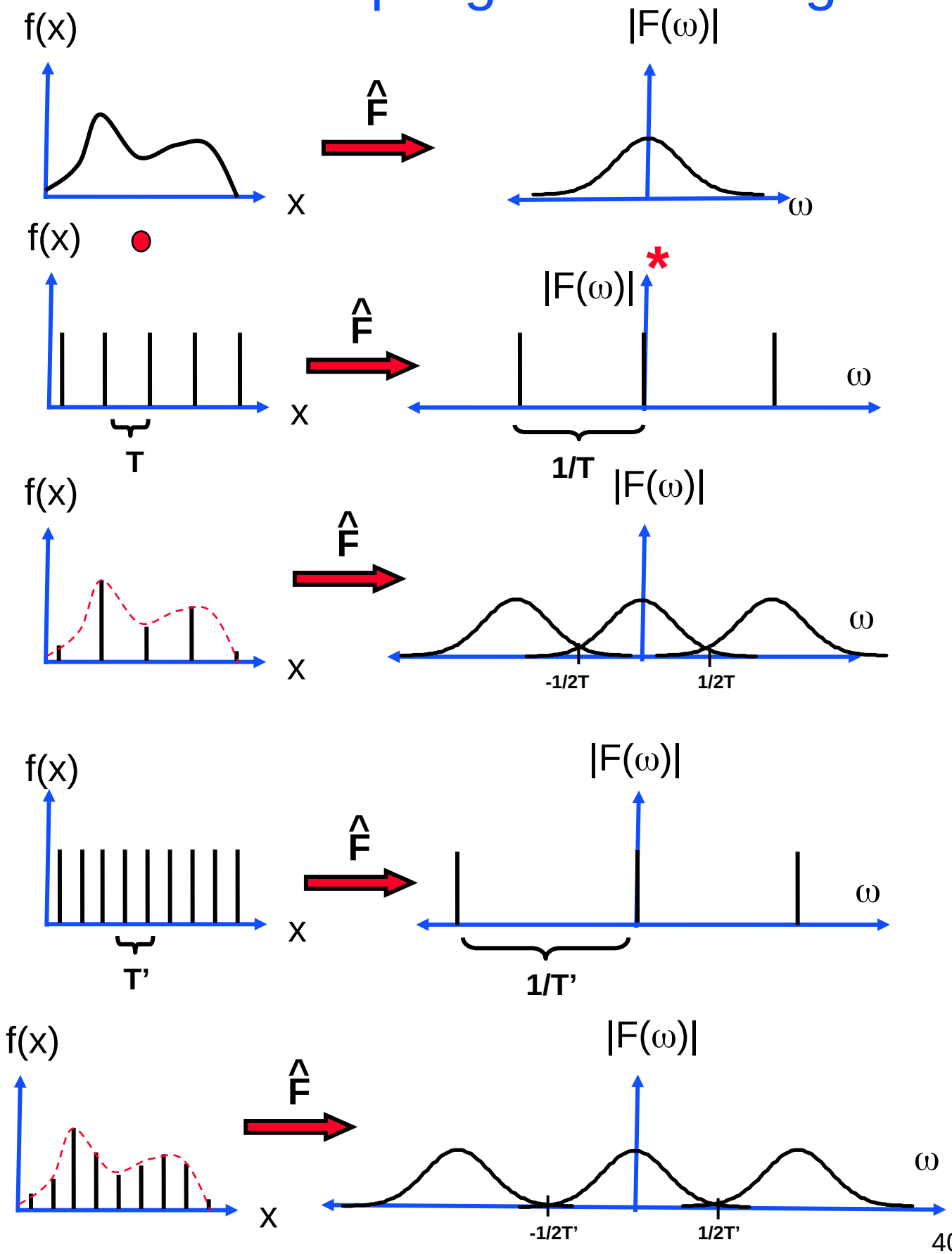
Due to replicas: $F(k)=F(N+k)$

Due to symmetry: $F(k)=F^*(-k)=F^*(N-k)$



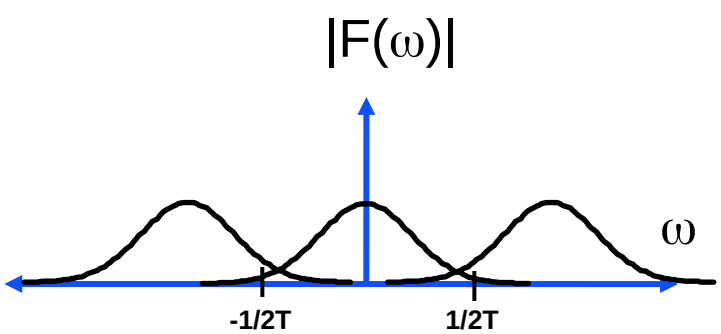
In 2D: $F(u,v)=F(u+N,v)=F(u,v+M)=F(u+N,v+M)$

Undersampling and Aliasing

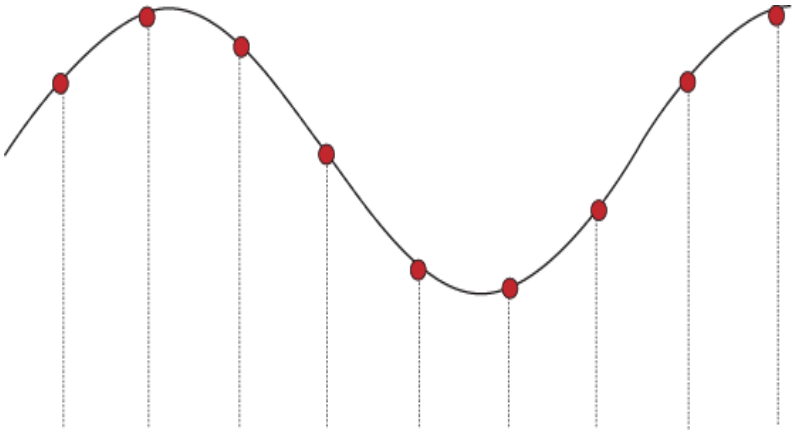


Critical Sampling

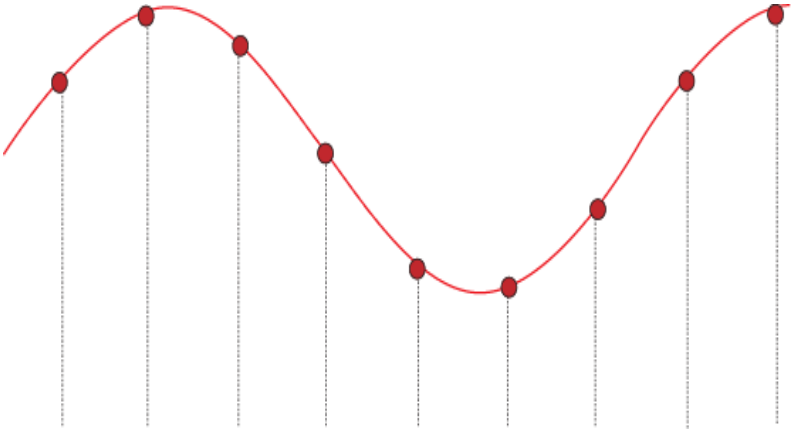
- If the maximal frequency of f(x) is ω_{max} , it is clear from the above replicas that ω_{max} should be smaller that 1/2T.
- Alternatively: $\frac{1}{T} > 2\omega_{max}$
- **Nyquist Theorem**: If the maximal frequency of f(x) is ω_{max} the sampling rate should be larger than $2\omega_{max}$ in order to fully reconstruct f(x) from its samples.
  $2\omega_{max}$ is called the **Nyquist frequency**.
- If the sampling rate is smaller than $2\omega_{max}$ overlapping replicas produce **aliasing**.



Critical Sampling



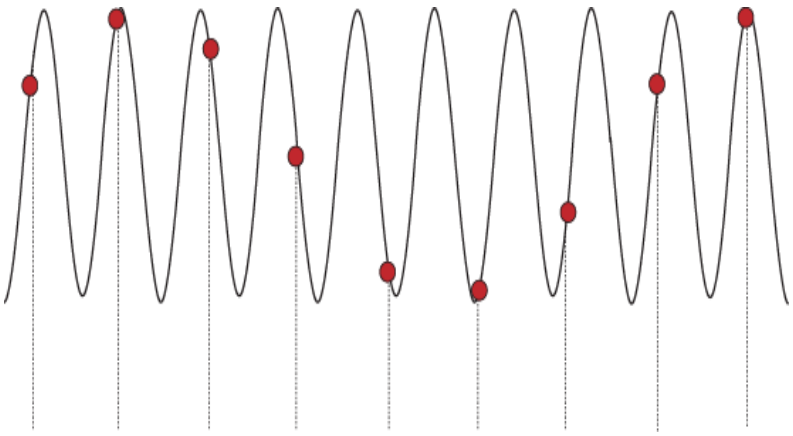
Input



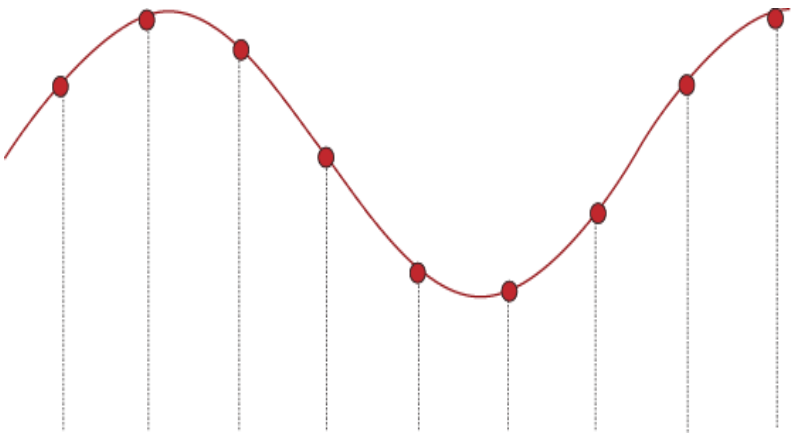
Reconstructed

Demo: B. Freeman

Aliasing



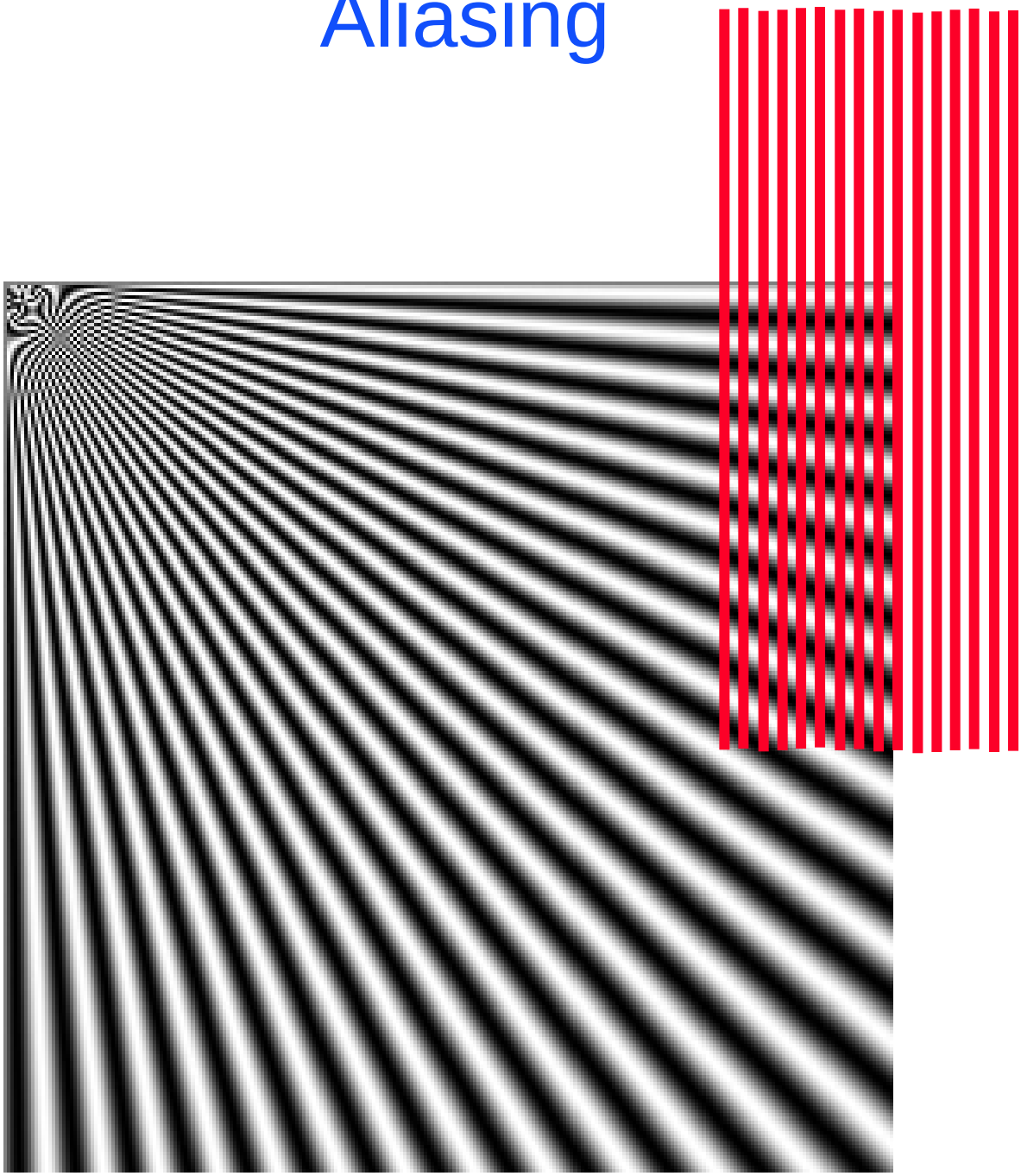
Input



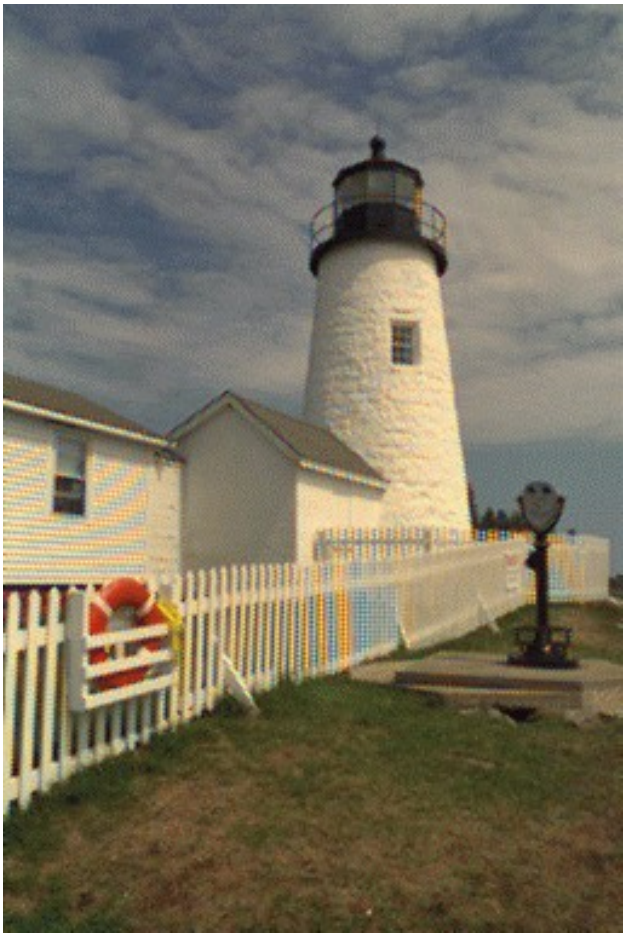
Reconstructed

Demo: B. Freeman

Aliasing



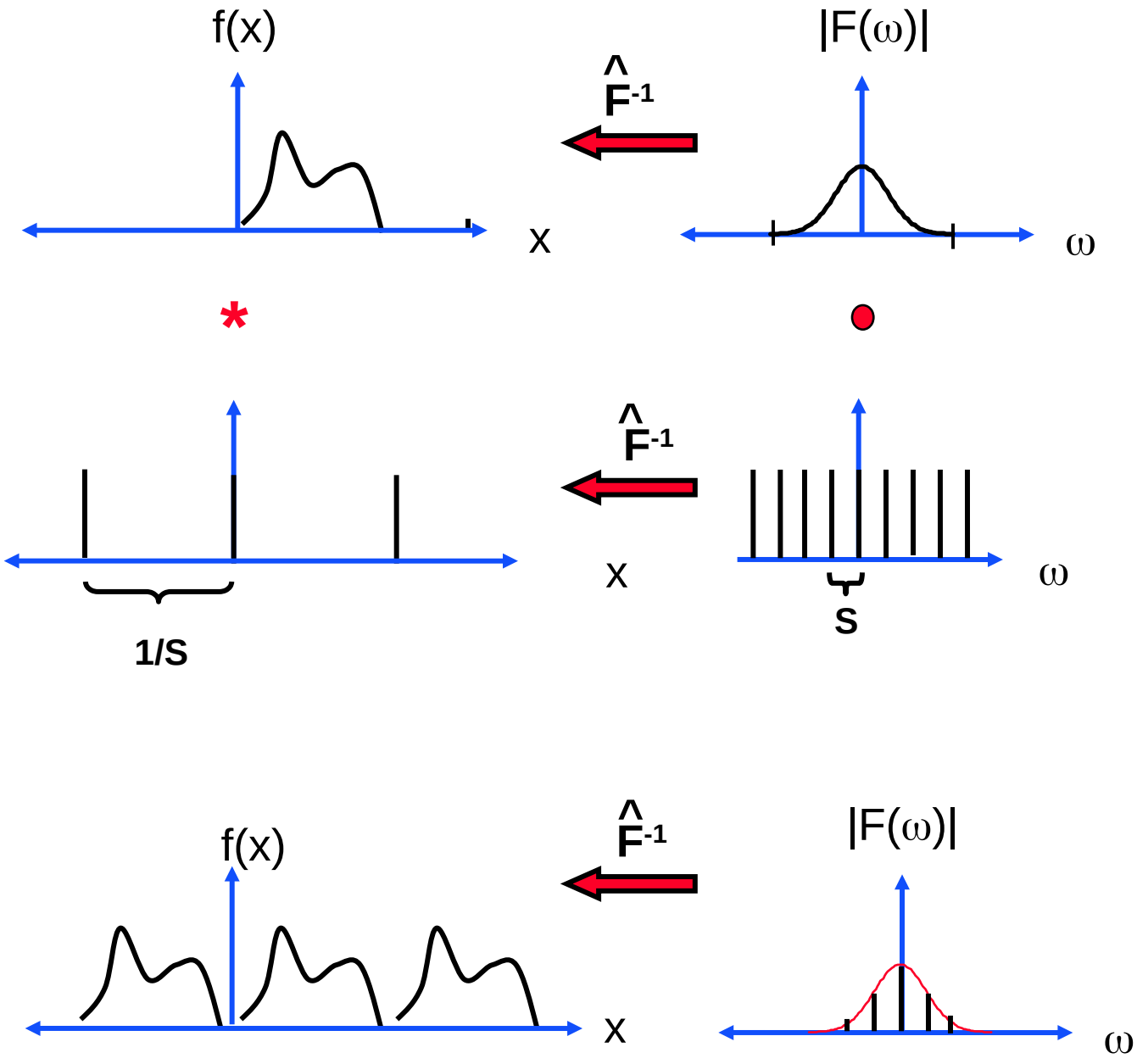
Aliasing in Color Images



- Image demosaicing may produce aliasing in color planes.
- How can we reduce aliasing in digital cameras?

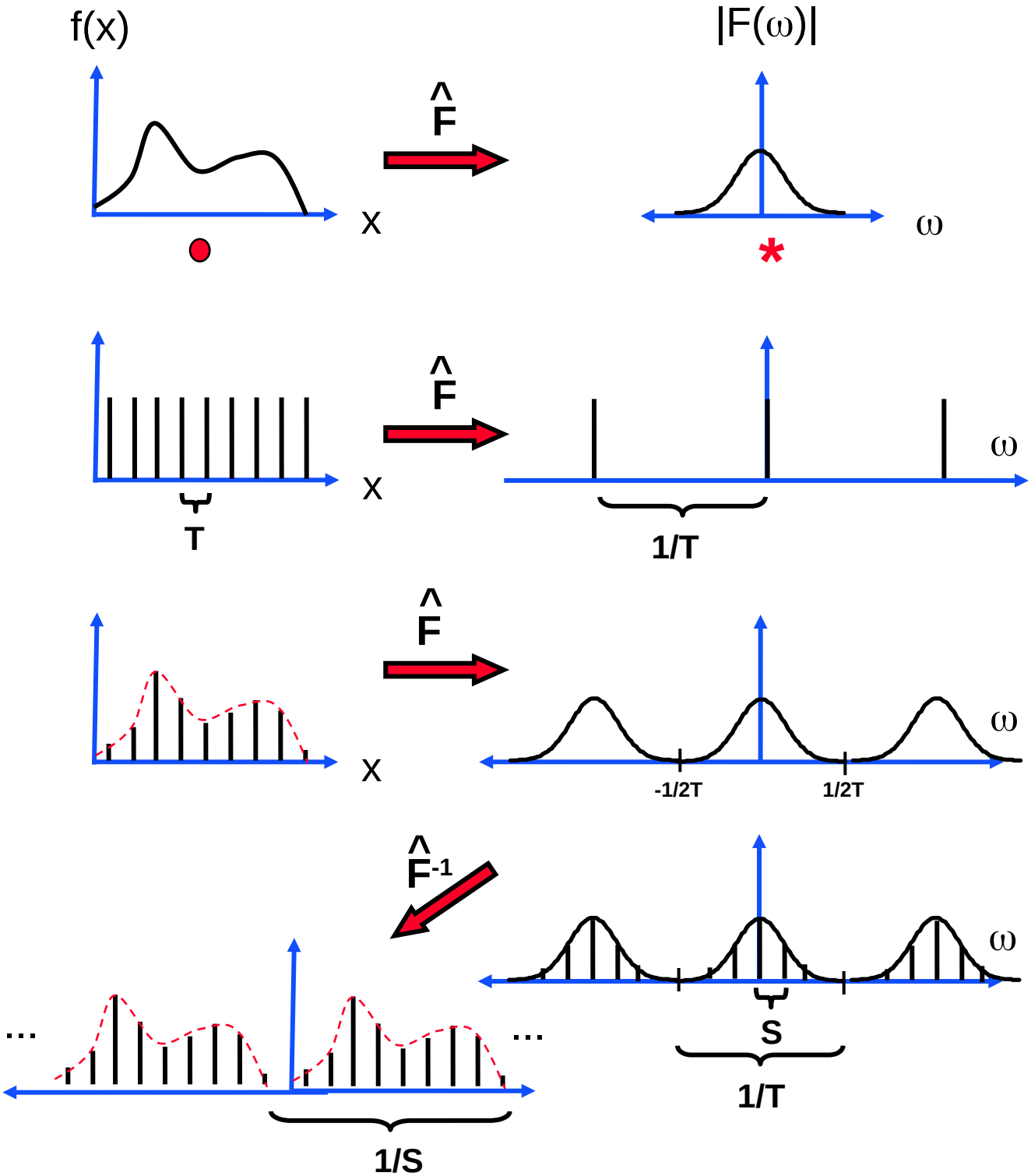
Sampling the Transform

- Sampling a function F(ω) with impulse train of cycle S produces replicas in the image domain with cycle 1/S:



Sampling Image & Transform

- Sampling both f(x) with impulse train of cycle T and F(ω) with impulse train of cycle S:



Number of Samples

- **Question**: Assuming f(x) was samples with N samples. What is the minimal number of samples in F(ω) in order to fully reconstruct f(x) ?

- **Answer**:
  - If we sample f(x) with N samples of cycle T, the support of f(x) is NT.
  - The support of F(ω) is 1/T in the frequency domain.
  - If we sample F(ω) with M samples, the sample cycle is 1/MT.
  - The replicas in the spatial domain are each MT.
  - In order to avoid replicas overlap, MT should be equal or bigger than NT (the function support).

$$M \geq N$$

Optimal Interpolation

- If sampling rate is above Nyquist - it is possible to fully reconstruct f(x) from its samples:

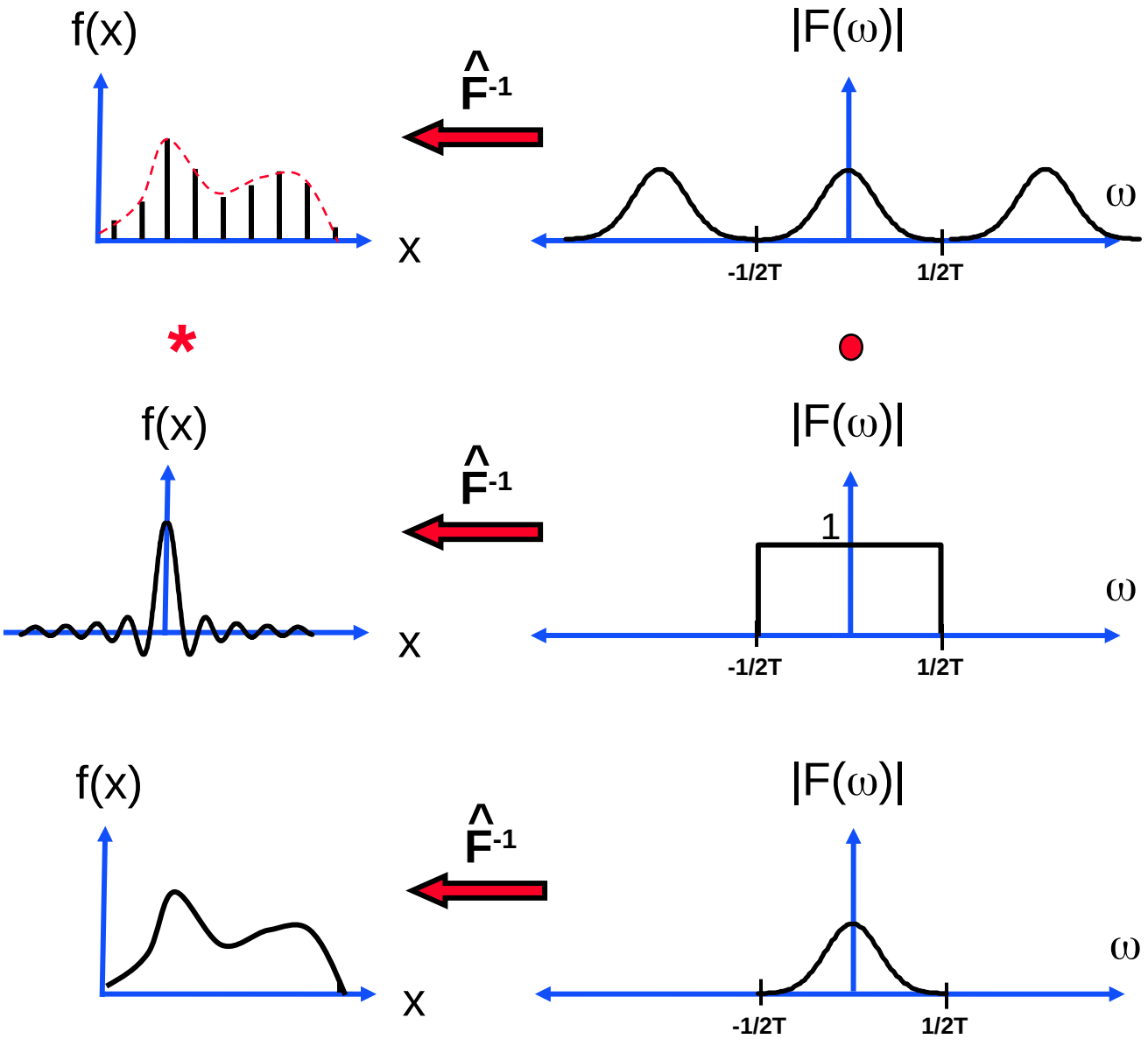
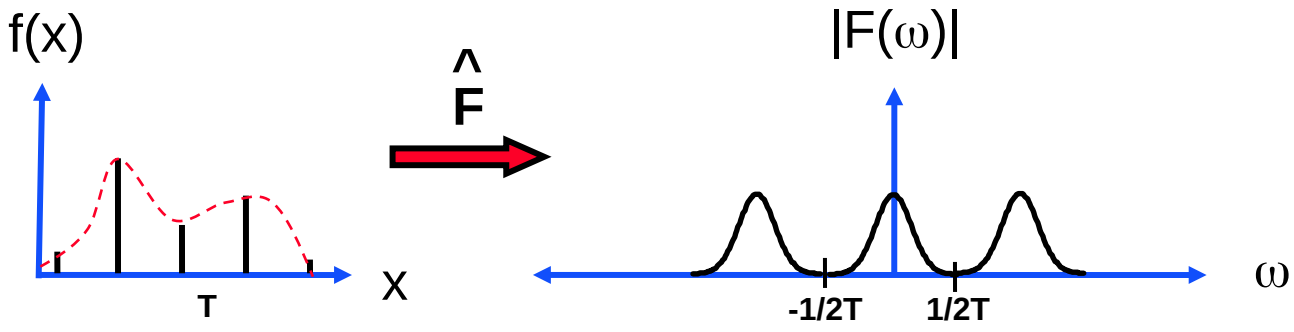


Image Scaling

- How is it possible to scale a digitized image?



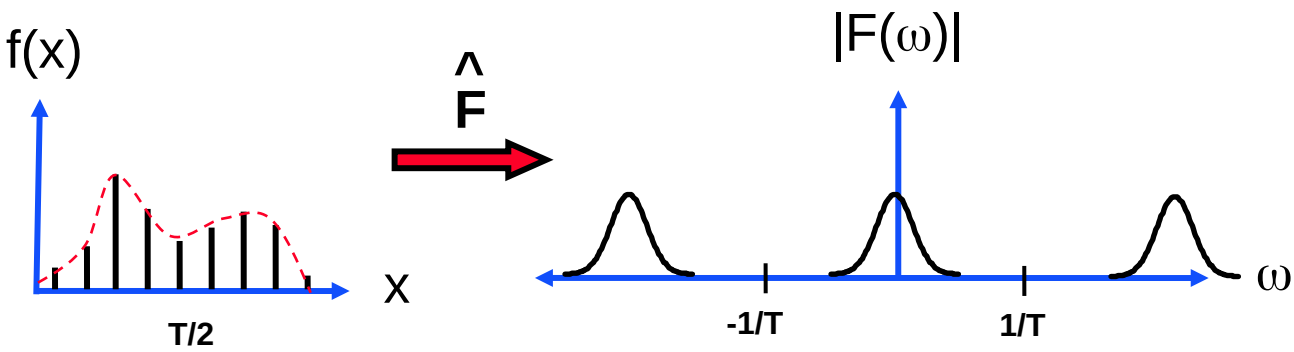
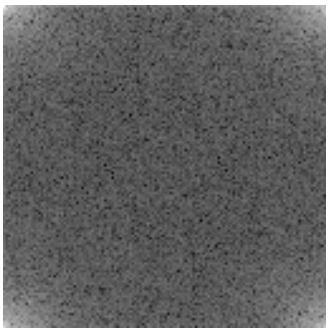
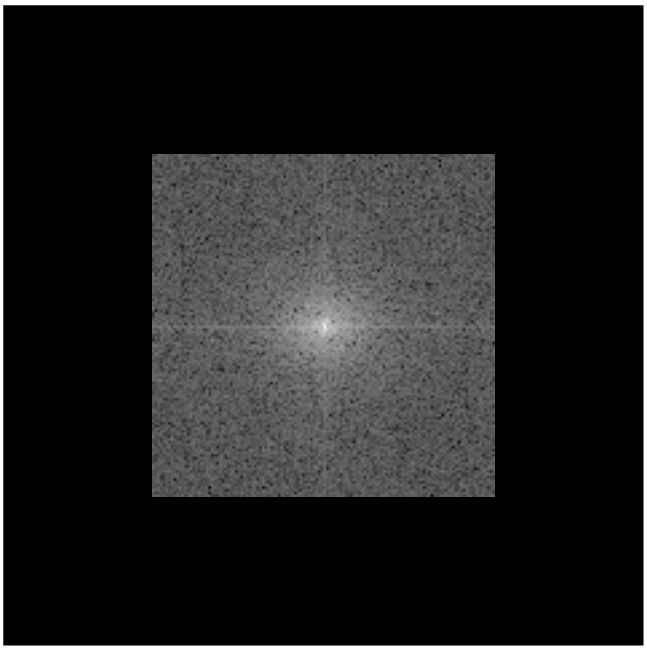


Image Scaling: Example





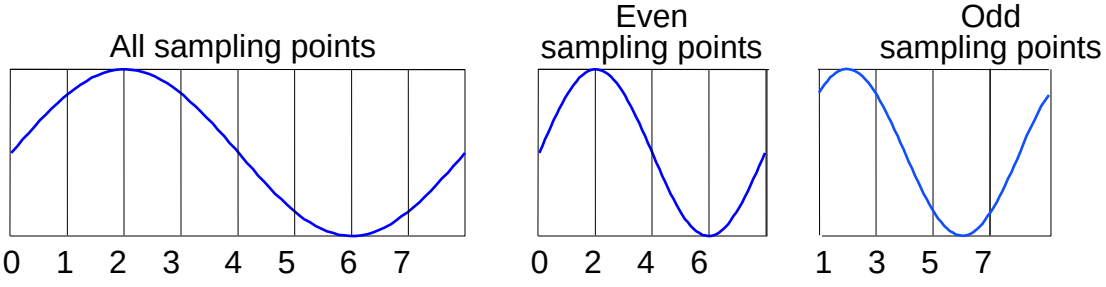
Fast Fourier Transform - FFT

$$F(u) = \frac{1}{N}\sum_{x=0}^{N-1} f(x) e^{\frac{-2\pi i u x}{N}} \qquad u = 0, 1, 2, \ldots, N-1$$

$O(n^2)$ operations

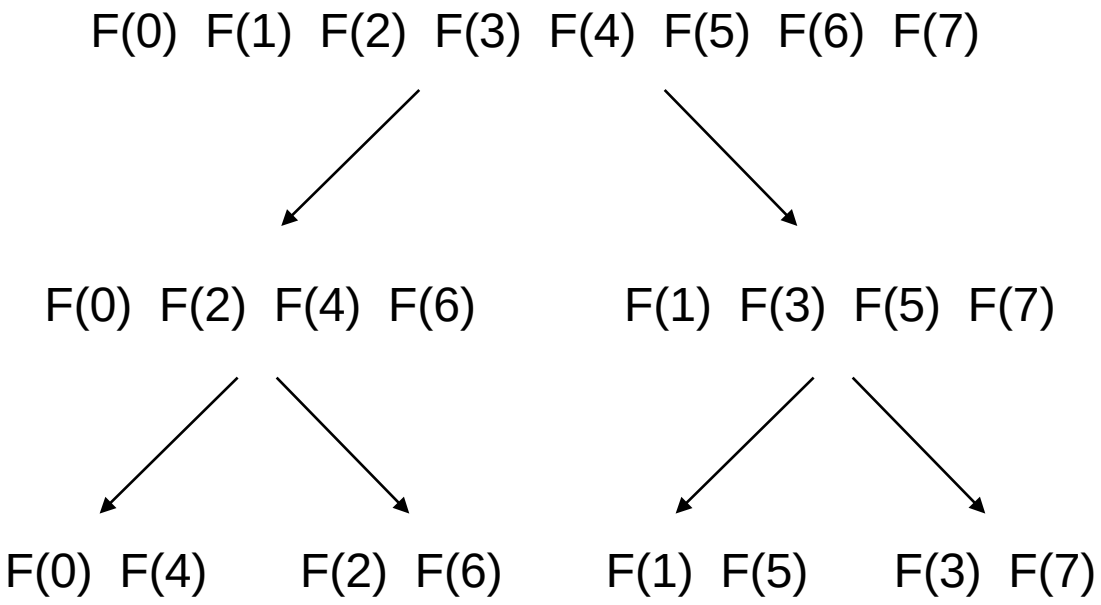
$$F(u) = \underbrace{\frac{1}{N}\sum_{x=0}^{N/2-1} f(2x) e^{\frac{-2\pi i u 2x}{N}}}_{\text{even } x} + \underbrace{\frac{1}{N}\sum_{x=0}^{N/2-1} f(2x+1) e^{\frac{-2\pi i u (2x+1)}{N}}}_{\text{odd } x}$$

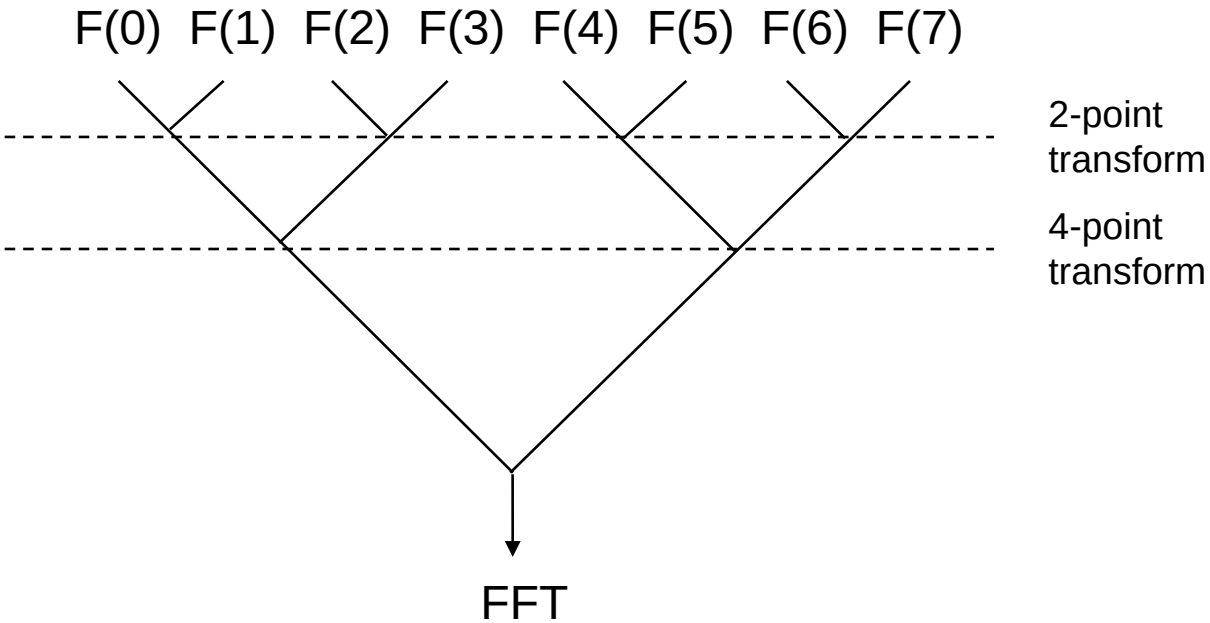
$$= \frac{1}{2}\left[\underbrace{\frac{1}{N/2}\sum_{x=0}^{N/2-1} f(2x) e^{\frac{-2\pi i u x}{N/2}}}_{\text{Fourier Transform of of N/2 even points}} + e^{\frac{-2\pi i u}{N}} \underbrace{\frac{1}{N/2}\sum_{x=0}^{N/2-1} f(2x+1) e^{\frac{-2\pi i u x}{N/2}}}_{\text{Fourier Transform of of N/2 odd points}} \right]$$



The Fourier transform of N inputs, can be performed as 2 Fourier Transforms of N/2 inputs each + one complex multiplication and addition for each value.
Thus, if F(N) is the computation complexity of FFT:

$$F(N)=F(N/2)+F(N/2)+O(N) \Rightarrow F(N)=N \log N$$





FFT : O(N log(N)) operations

FFT of NxN Image: $O(N^2 \log(N))$ operations

Reference: James W. Cooley and John W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.* **19**, 297–301 (1965).

DFT and Its Properties

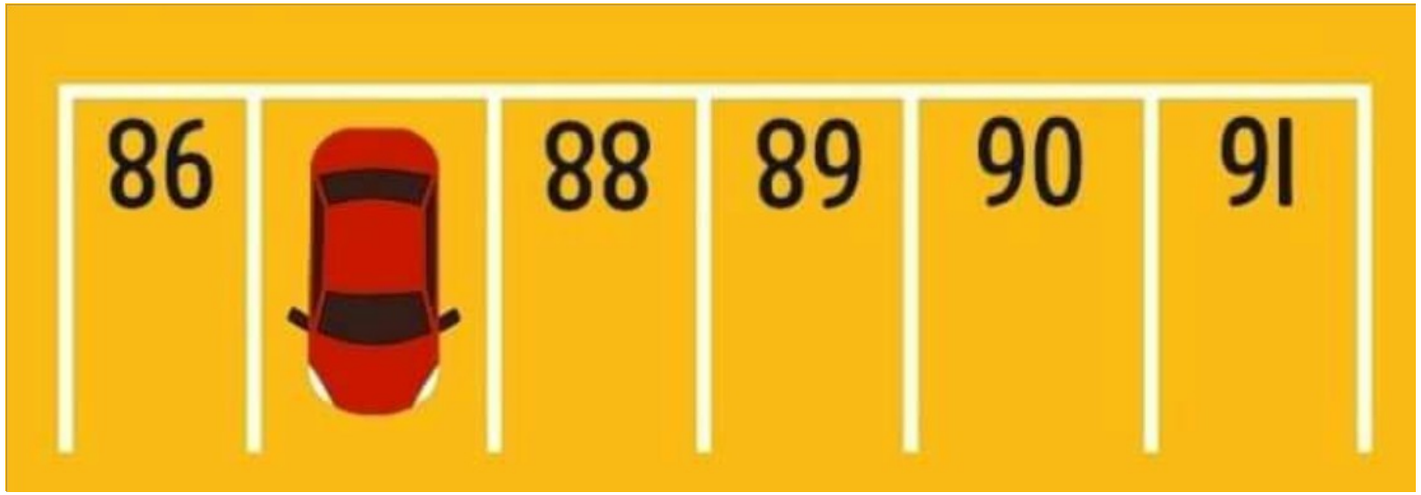
Prof. Dr. Sk. Md. Masudul Ahsan
Dept. of CSE, KUET

Learning outcomes

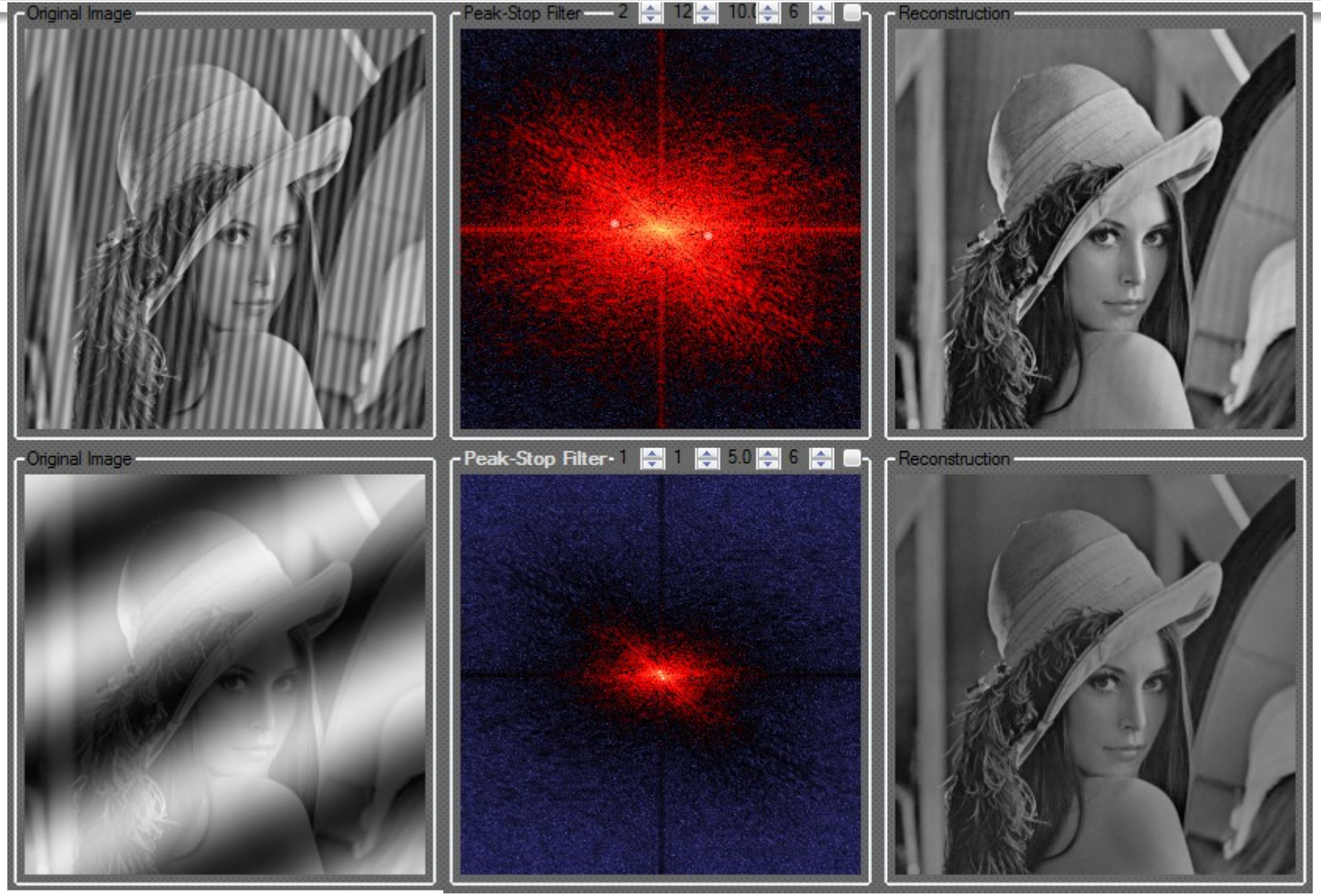
- Explain why frequency content is useful for image processing.
- Move from 1D Fourier series to 1D DFT and matrix DFT.
- Define and interpret the 2D DFT of an image.
- Prove important 2D DFT properties used in filtering.

Motivation

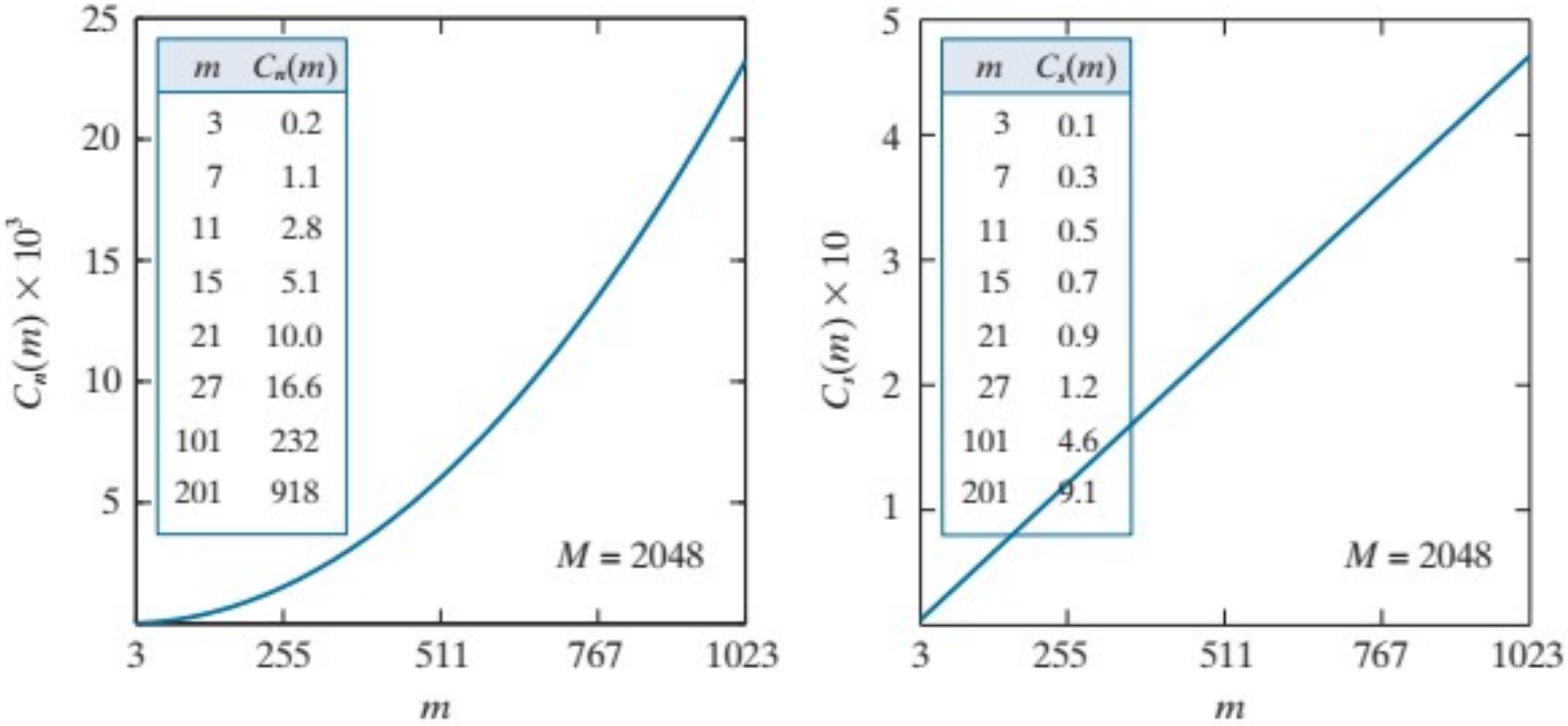
- Sometimes a problem is difficult in one perspective but easy in other



Motivation


Original Image
Peak-Stop Filter 2 12 10.0 6
Reconstruction
Original Image
Peak-Stop Filter 1 1 5.0 6
Reconstruction

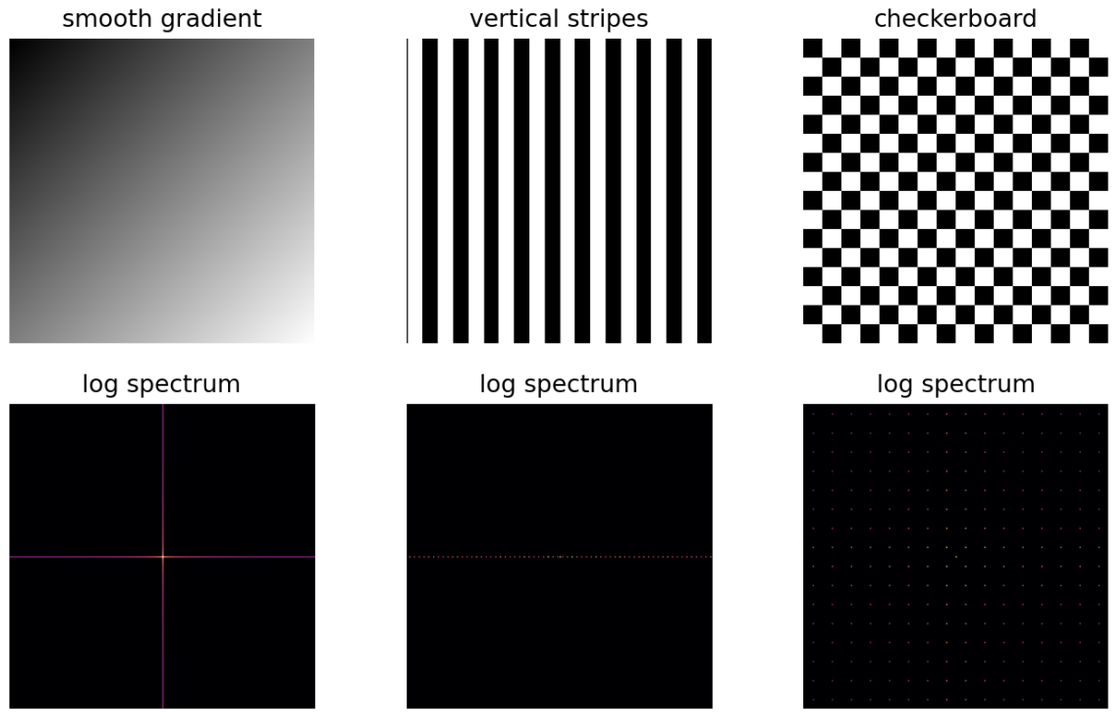
Motivation



(a) Computational advantage of the FFT over nonseparable spatial kernels. (b) Advantage over separable kernels. The numbers for C m ( ) in the inset tables are not to be multiplied by the factors of 10 shown for the curves.

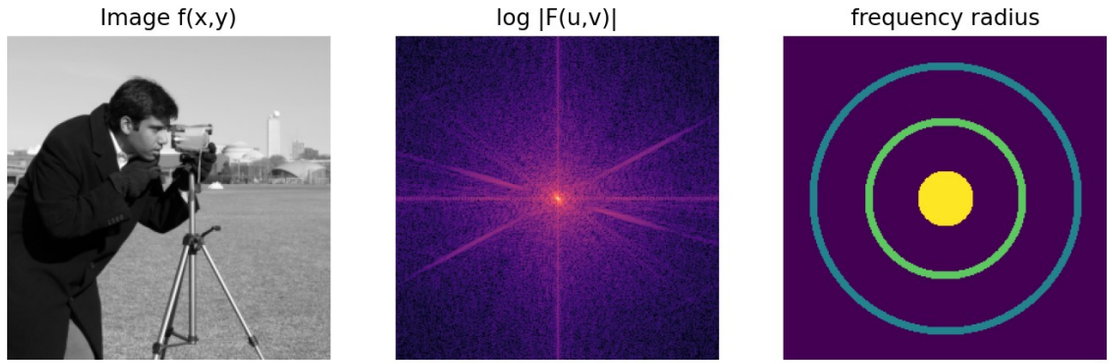
Spatial vs frequency view

- Spatial domain: intensity at each pixel location.
- Frequency domain: amount of each 2D wave pattern.
- Low frequency means slow spatial change.
- High frequency means rapid spatial change.



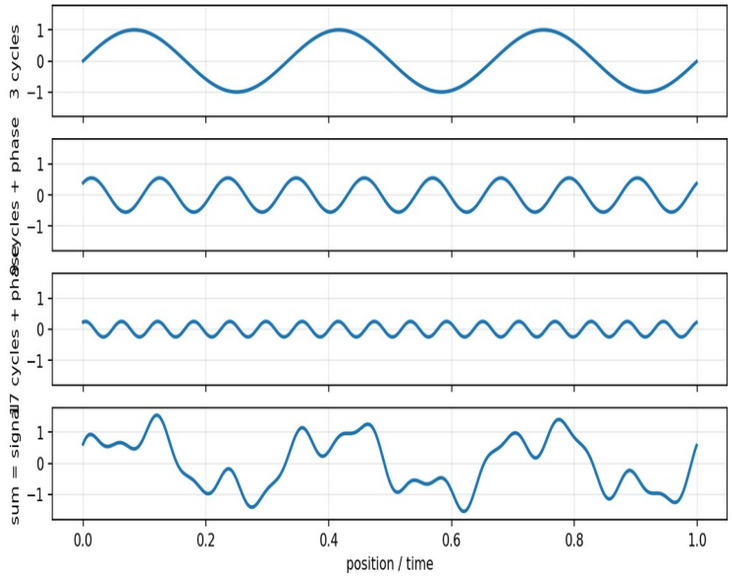
Why Fourier for images?

- Slow variations: illumination, background, smooth regions.
- Fast variations: edges, texture, noise, fine details.
- Fourier analysis separates slow and fast spatial changes.
- This helps us blur, sharpen, denoise, and inspect periodic artifacts.



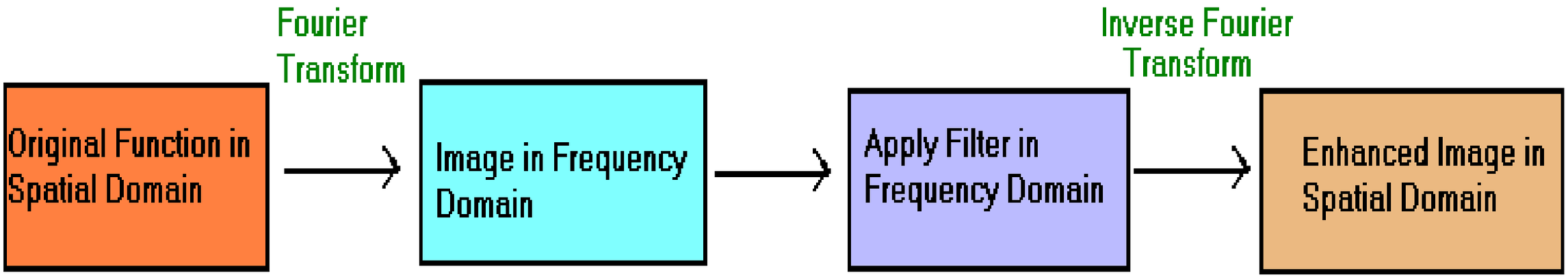
Background

- Any function that periodically repeats itself can be expressed as the sum of sines and/or cosines of different frequencies, each multiplied by a different coefficient (Fourier series).
- Fourier transform
  - even functions that are not periodic (but whose area
    under the curve is finite) can be expressed as the
    integral of sines and/or cosines multiplied by a weighting function



Our Interest in Fourier Transform

- We will be dealing only with functions (images) of finite duration so we will be interested only in Fourier Transform



Fourier Series – the basic idea

$$f(t) = \sum_{n=0}^{\infty} a_n \cos\left(\frac{2\pi nt}{T}\right) + \sum_{n=1}^{\infty} b_n \sin\left(\frac{2\pi nt}{T}\right)$$

$$= \sum_{n=0}^{\infty} a_n \cos(\omega_n t) + \sum_{n=1}^{\infty} b_n \sin(\omega_n t)$$

Function in Time domain

Basis function

Angular freq.

$$f(x) = \sum_{n=0}^{\infty} a_n \cos\left(\frac{2\pi nx}{\lambda}\right) + \sum_{n=1}^{\infty} b_n \sin\left(\frac{2\pi nx}{\lambda}\right)$$

Spatial variable

Spatial period

$$= \sum_{n=0}^{\infty} a_n \cos(k_n x) + \sum_{n=1}^{\infty} b_n \sin(k_n x)$$

Spatial domain

Fourier Spectrum

Fourier Spectrum

$$a_n = \frac{2}{\lambda}\int_{-\lambda/2}^{\lambda/2} f(x)\cos(k_n x)\,\mathrm{d}x$$

$$b_n = \frac{2}{\lambda}\int_{-\lambda/2}^{\lambda/2} f(x)\sin(k_n x)\,\mathrm{d}x$$

Complex form

$$f(x) = \sum_{n=-\infty}^{\infty} c_n \exp\left(\frac{\mathrm{i}2\pi n x}{\lambda}\right)$$

$$= \sum_{n=-\infty}^{\infty} c_n \exp(\mathrm{i}k_n x)$$

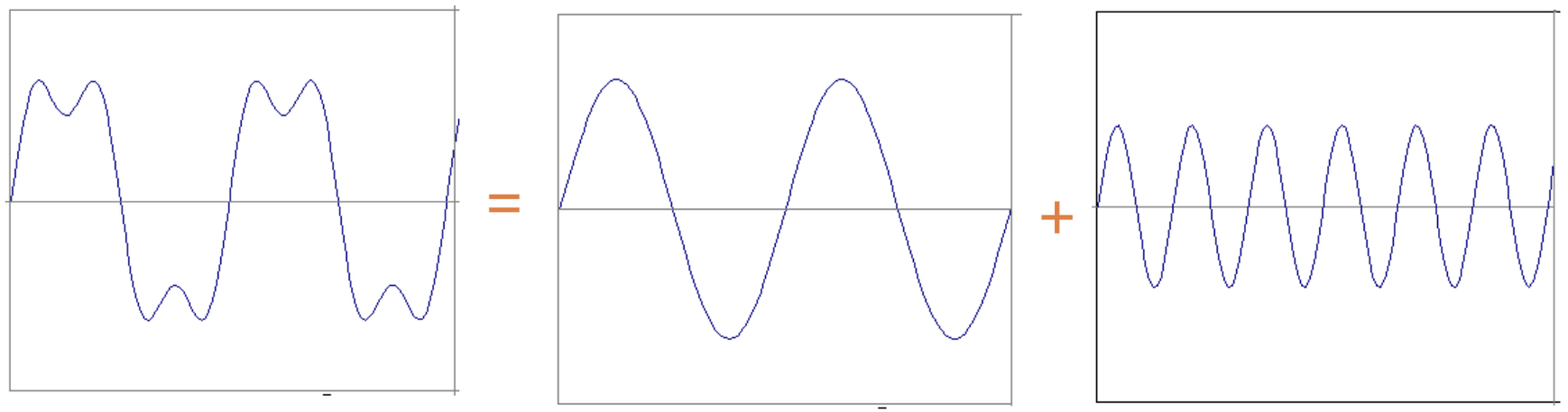
$$e^{j\theta} = \cos\theta + j\sin\theta$$

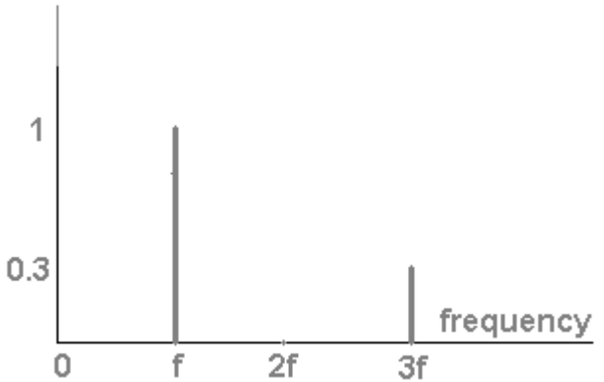
$$c_n = \frac{1}{\lambda}\int_{-\lambda/2}^{\lambda/2} f(x)\exp(\mathrm{i}k_n x)\,\mathrm{d}x$$

F(u)

Frequency Spectra

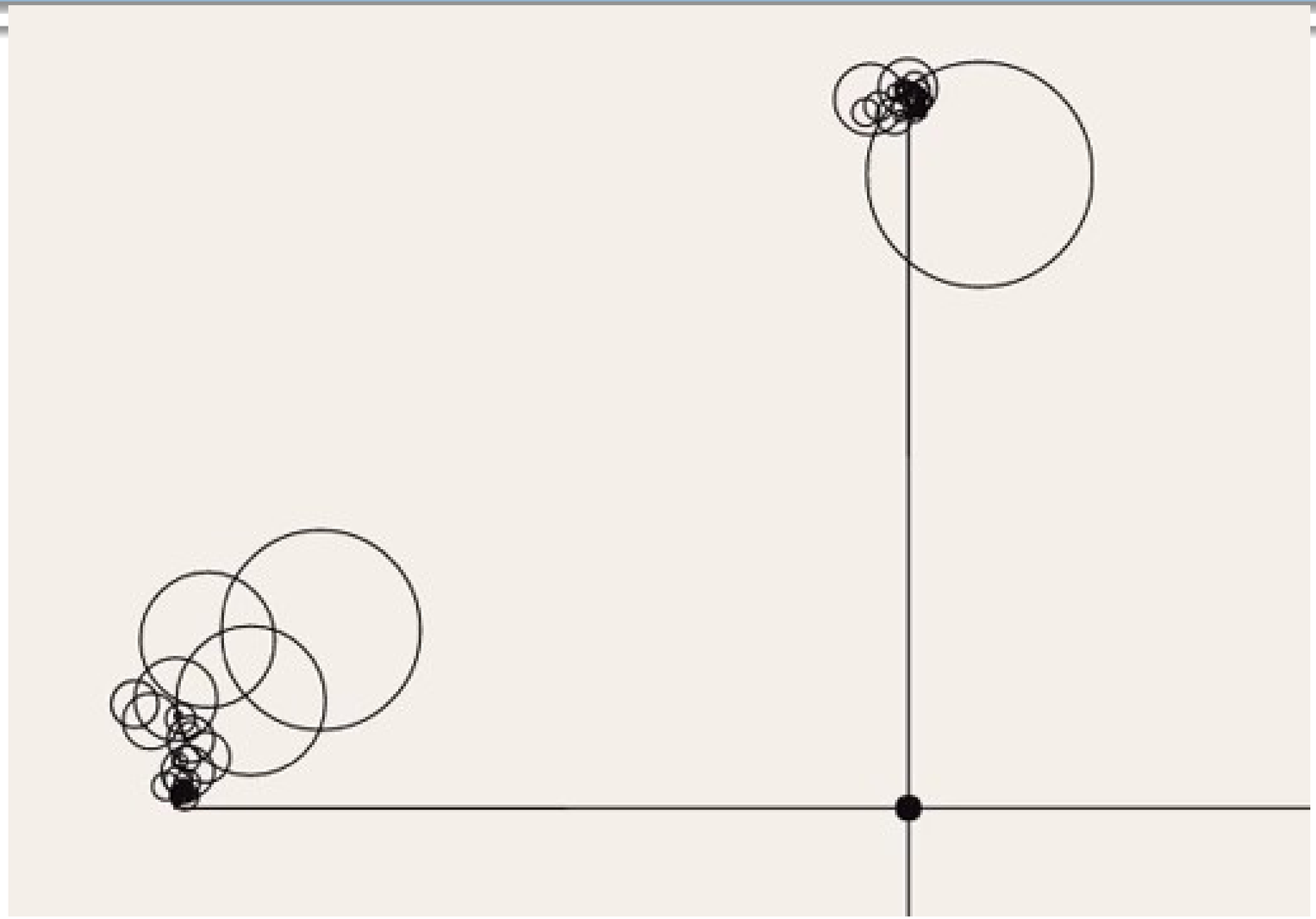
- example : $g(t) = \sin(2\pi f\, t) + (1/3)\sin(2\pi(3f)\, t)$





Slides: Efros

One series in each of x and y



Generative Art, じゃがりきん, V

Further study

- Chapter 5 of
  - Fundamentals of Digital Image Processing, by
    - Chris Solomon, Toby Breckon

- Proof of DFT properties
  - Digital Image Processing
    - S Jayaraman

1D Fourier Transform

- Fourier transform *F(u)* of a continuous function *f(x)* is:

$$e^{j\theta} = \cos\theta + j\sin\theta$$

Forward Fourier transform:

$$F(u) = \int_{-\infty}^{\infty} f(x) e^{-j2\pi ux} dx$$

Inverse transform:

$$f(x) = \int_{-\infty}^{\infty} F(u) e^{j2\pi ux} du$$

$$j = \sqrt{-1}$$

- (u) is the frequency variable.
- $F(u)$ is composed of an infinite sum of sine and cosine terms
- Each value of u determines the frequency of its corresponding sine-cosine pair.

1D DFT

- *A digital signal f(x), x*=0,1,…,M-1 *has samples f[0], f[1], …, f[M-1].*
- *The DFT uses M frequency samples u = 0, 1, …, M-1.*
- *F(u), u*=0,1,…,M-1 DFT of f(x)

Forward discrete Fourier transform:

$$F(u) = \frac{1}{M} \sum_{x=0}^{M-1} f(x) e^{-j2\pi \frac{u}{M} x}$$

Inverse transform:

$$f(x) = \sum_{u=0}^{M-1} F(u) e^{j2\pi \frac{u}{M} x}$$

- The location of 1/M multiplier doesn't matter
  - in front of the Fourier transform
  - in front of the inverse Fourier transform

1D DFT

- Since $e^{j\theta} = \cos\theta + j\sin\theta$ and the fact $\cos(-\theta) = \cos\theta$

  then discrete Fourier transform can be written as

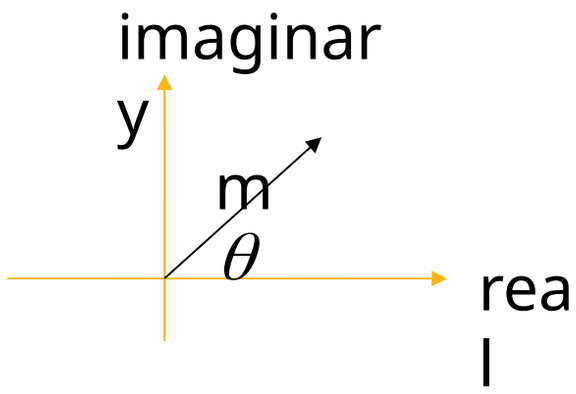
$$F(u) = \frac{1}{M}\sum_{x=0}^{M-1} f(x)[\cos 2\pi ux/M - j\sin 2\pi ux/M]$$

$$\text{for } u = 0,1,2,\ldots,M-1$$

- F[u] is a weighted sum over all samples.
- The weights are complex sinusoids of frequency u.
- Magnitude indicates strength; phase indicates alignment.
- The inverse DFT reconstructs the samples exactly.

1D DFT

- The Fourier transform of a real function is generally complex
- $F(u)$ can be expressed in polar coordinates:



$$F(u) = |F(u)| e^{j\phi(u)}$$

$$\text{where } |F(u)| = \left[R^2(u) + I^2(u)\right]^{\frac{1}{2}}$$

Magnitude or spectrum

$$\phi(u) = \tan^{-1}\left[\frac{I(u)}{R(u)}\right]$$

Phase angle or phase spectrum

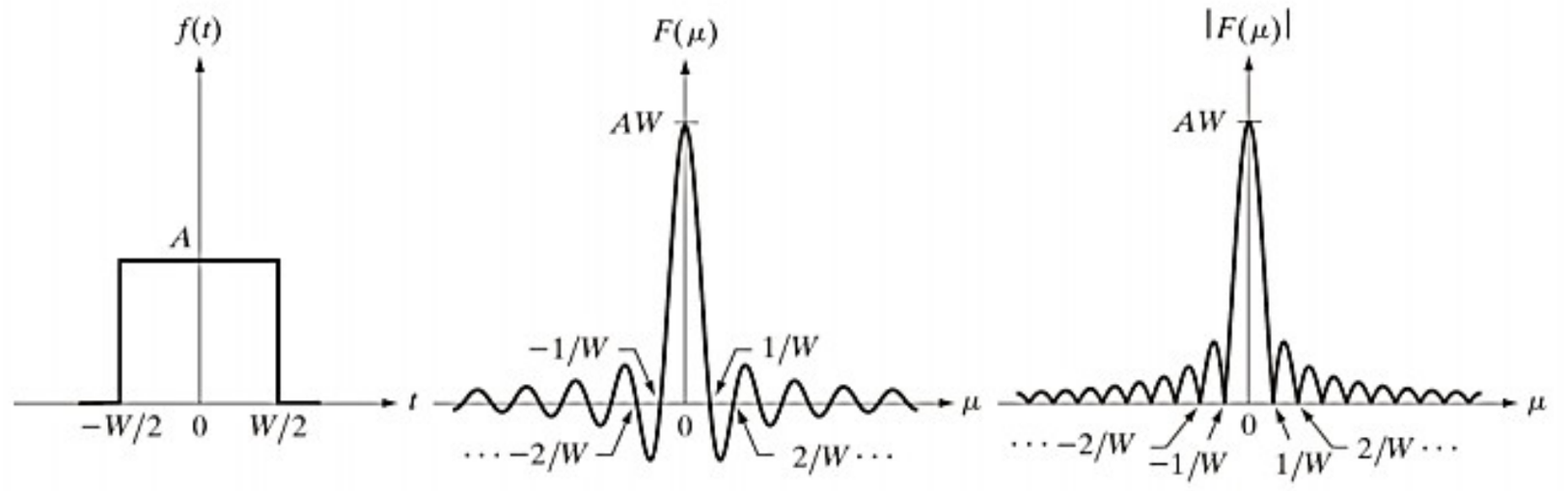
  - $R(u)$: the real part of $F(u)$
  - $I(u)$: the imaginary part of $F(u)$
- Power spectrum: $P(u) = |F(u)|^2 = R^2(u) + I^2(u)$

1D – Fourier Transform-example

$$F(\mu) = \int_{-\infty}^{\infty} f(t) e^{-j2\pi\mu t} dt = \int_{-W/2}^{W/2} A e^{-j2\pi\mu t} dt$$

$$= A\left[-\frac{1}{j2\pi\mu} e^{-j2\pi\mu t} \right]_{-w/2}^{w/2} = \frac{A}{j2\pi\mu}\left[e^{j\pi\mu W} - e^{-j\pi\mu W} \right]$$

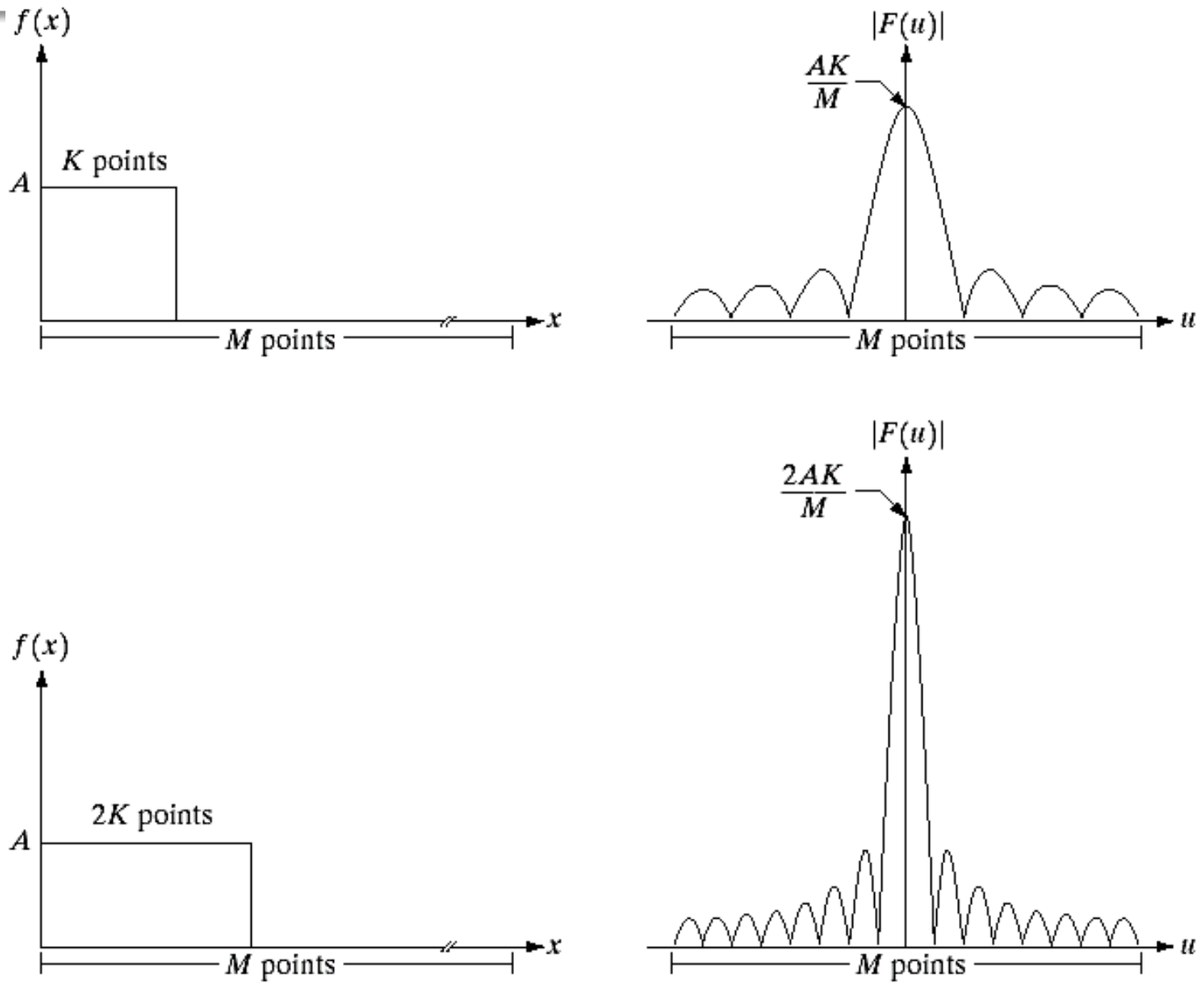
$$= \frac{A}{j2\pi\mu} 2j Sin(\pi\mu W) = AW \frac{\sin(\pi\mu W)}{(\pi\mu W)} = AW\, sinc(\pi\mu W)$$



a b c

FIGURE 4.4 (a) A simple function; (b) its Fourier transform; and (c) the spectrum. All functions extend to infinity in both directions.

1D DFT - example



a b
c d

FIGURE 4.2 (a) A discrete function of M points, and (b) its Fourier spectrum. (c) A discrete function with twice the number of nonzero points, and (d) its Fourier spectrum.

1D DFT in Matrix

$$F(u) = \frac{1}{M} \sum_{x=0}^{M-1} f(x) e^{-j2\pi \frac{u}{M} x}$$

$$F(u) = \frac{1}{M} \sum_{x=0}^{M-1} f(x) \omega^{ux}$$

$$F(0) = \frac{1}{M} \sum_{x=0}^{M-1} f(x) \omega^{0} = \frac{1}{M} [\omega^{0} f(0) + \omega^{0} f(1) + \ldots + \omega^{0} f(M-1)]$$

$$= \frac{1}{M} [\omega^{0} \quad \omega^{0} \quad \ldots \quad \omega^{0}] \begin{bmatrix} f(0) \\ f(1) \\ \ldots \\ f(M-1) \end{bmatrix}$$

$$\omega^{0} = \left(e^{\frac{-j2\pi}{M}} \right)^{0} = 1$$

1D DFT in Matrix

$$F(1) = \frac{1}{M}\sum_{x=0}^{M-1} f(x)\omega^{x} = \frac{1}{M}[\omega^{0} \quad \omega^{1} \quad \omega^{2} \quad \ldots \quad \omega^{M-1}] \begin{bmatrix} f(0) \\ f(1) \\ f(2) \\ \ldots \\ f(M-1) \end{bmatrix}$$

$$F(2) = \frac{1}{M}\sum_{x=0}^{M-1} f(x)\omega^{2x} = \frac{1}{M}[\omega^{0} \quad \omega^{2} \quad \omega^{4} \quad \ldots \quad \omega^{2M-2}] \begin{bmatrix} f(0) \\ f(1) \\ f(2) \\ \ldots \\ f(M-1) \end{bmatrix}$$

1D DFT in Matrix

$$F(u) = \frac{1}{M}\begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^1 & \omega^2 & \dots & \omega^{M-1} \\ 1 & \omega^2 & \omega^4 & \dots & \omega^{2M-2} \\ 1 & \omega^3 & \omega^6 & \dots & \omega^{3M-3} \\ \dots & & & & \\ 1 & \omega^{M-1} & \omega^{2M-2} & \dots & \omega^{(M-1)^2} \end{bmatrix} \begin{bmatrix} f(0) \\ f(1) \\ f(2) \\ f(3) \\ \dots \\ f(M-1) \end{bmatrix}$$

$$\mathbf{F(u) = W\, f(x)}$$

4 point DFT matrix

$$W = \frac{1}{4}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega^1 & \omega^2 & \omega^3 \\ 1 & \omega^2 & \omega^4 & \omega^6 \\ 1 & \omega^3 & \omega^6 & \omega^9 \end{bmatrix} = \frac{1}{4}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

$$\omega = e^{\frac{-j2\pi}{4}} = \cos\frac{\pi}{2} - j\sin\frac{\pi}{2} = -j$$

1D DFT in Matrix

- 4 point DFT matrix

$$W = \frac{1}{4}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega^1 & \omega^2 & \omega^3 \\ 1 & \omega^2 & \omega^4 & \omega^6 \\ 1 & \omega^3 & \omega^6 & \omega^9 \end{bmatrix} = \frac{1}{4}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

  - Since

$$\omega = e^{\frac{-j2\pi}{4}} = \cos\frac{\pi}{2} - j\sin\frac{\pi}{2} = -j$$

- Find DFT of f(x)=[1,0,1,-1]

1D DFT in Matrix

- 8 point DFT matrix

$$W = \frac{1}{\sqrt{8}} \begin{bmatrix} \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 \\ \omega^0 & \omega^1 & \omega^2 & \omega^3 & \omega^4 & \omega^5 & \omega^6 & \omega^7 \\ \omega^0 & \omega^2 & \omega^4 & \omega^6 & \omega^8 & \omega^{10} & \omega^{12} & \omega^{14} \\ \omega^0 & \omega^3 & \omega^6 & \omega^9 & \omega^{12} & \omega^{15} & \omega^{18} & \omega^{21} \\ \omega^0 & \omega^4 & \omega^8 & \omega^{12} & \omega^{16} & \omega^{20} & \omega^{24} & \omega^{28} \\ \omega^0 & \omega^5 & \omega^{10} & \omega^{15} & \omega^{20} & \omega^{25} & \omega^{30} & \omega^{35} \\ \omega^0 & \omega^6 & \omega^{12} & \omega^{18} & \omega^{24} & \omega^{30} & \omega^{36} & \omega^{42} \\ \omega^0 & \omega^7 & \omega^{14} & \omega^{21} & \omega^{28} & \omega^{35} & \omega^{42} & \omega^{49} \end{bmatrix} = \frac{1}{\sqrt{8}} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \omega & -i & -i\omega & -1 & -\omega & i & i\omega \\ 1 & -i & -1 & i & 1 & -i & -1 & i \\ 1 & -i\omega & i & \omega & -1 & i\omega & -i & -\omega \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & -\omega & -i & i\omega & -1 & \omega & i & -i\omega \\ 1 & i & -1 & -i & 1 & i & -1 & -i \\ 1 & i\omega & i & -\omega & -1 & -i\omega & -i & \omega \end{bmatrix}$$

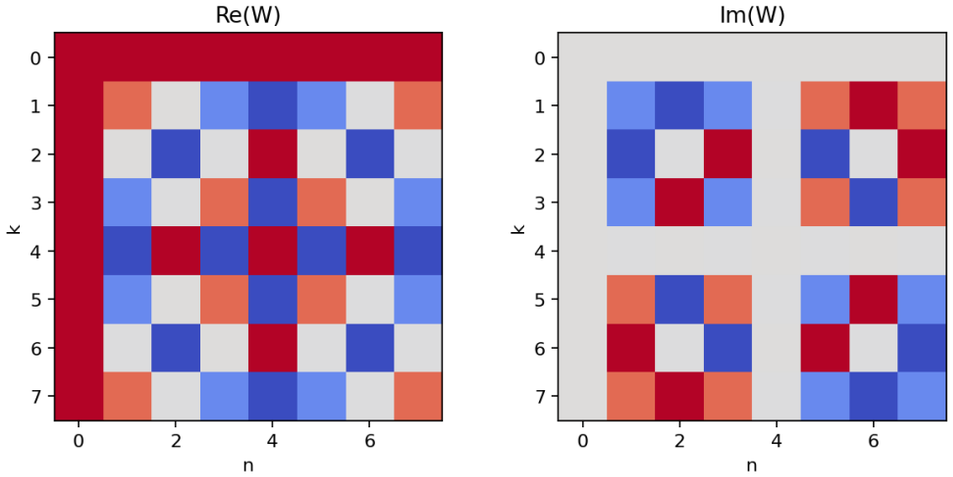
- Since

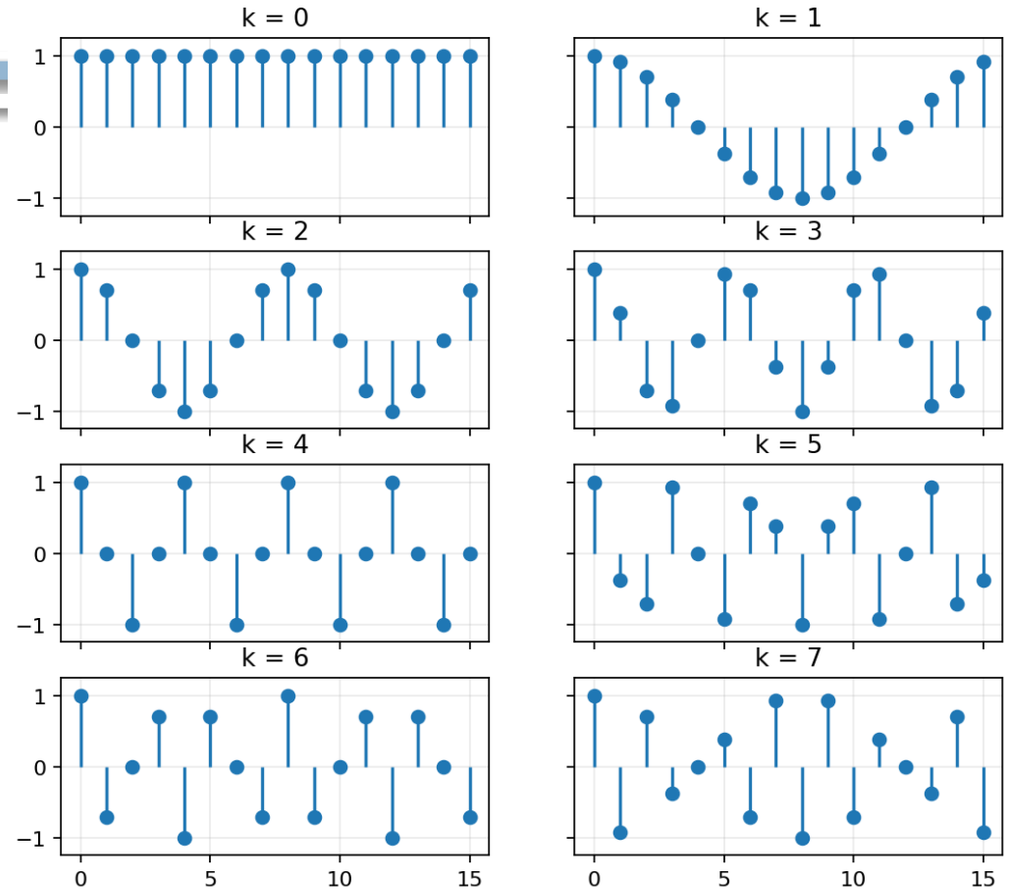
$$\omega = e^{-\frac{2\pi i}{8}} = \frac{1}{\sqrt{2}} - \frac{i}{\sqrt{2}}$$

1D DFT in Matrix

- 8 point DFT matrix

$$\frac{1}{\sqrt{8}}\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \omega & -i & -i\omega & -1 & -\omega & i & i\omega \\ 1 & -i & -1 & i & 1 & -i & -1 & i \\ 1 & -i\omega & i & \omega & -1 & i\omega & -i & -\omega \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & -\omega & -i & i\omega & -1 & \omega & i & -i\omega \\ 1 & i & -1 & -i & 1 & i & -1 & -i \\ 1 & i\omega & i & -\omega & -1 & -i\omega & -i & \omega \end{bmatrix}$$





$$\omega = e^{-\frac{2\pi i}{8}} = \frac{1}{\sqrt{2}} - \frac{i}{\sqrt{2}}$$

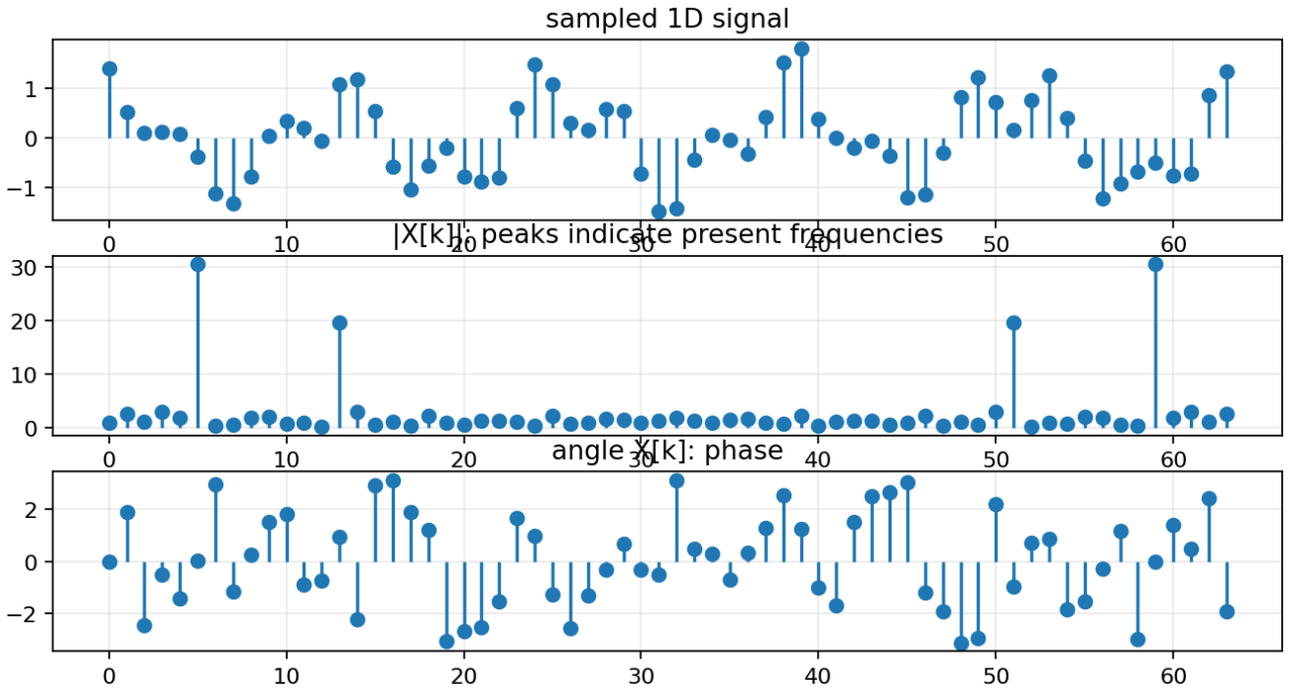
1D DFT in Matrix

- Forward DFT is a matrix-vector multiplication.
- Rows of W are orthogonal complex exponentials.
- The inverse uses W* (complex conjugate) scaled by 1/M.
- FFT algorithms compute this structure efficiently.

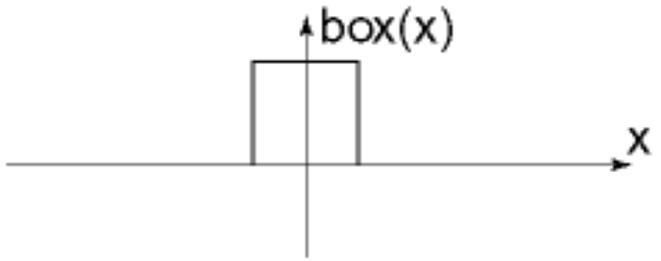
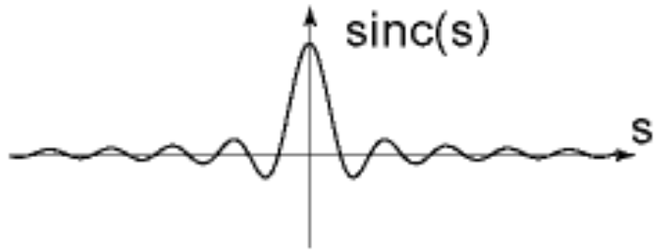
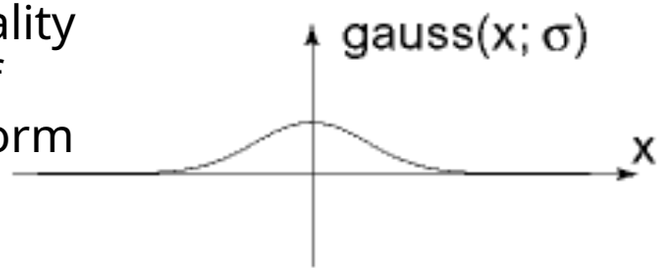
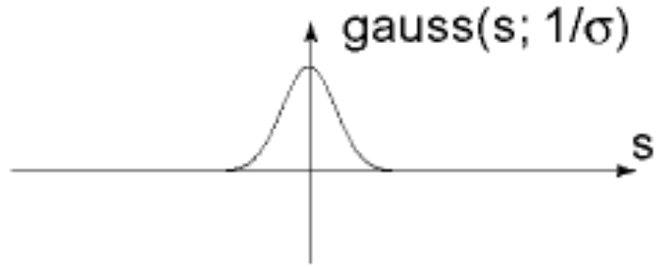
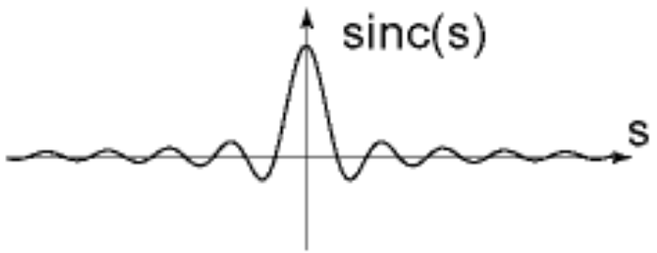
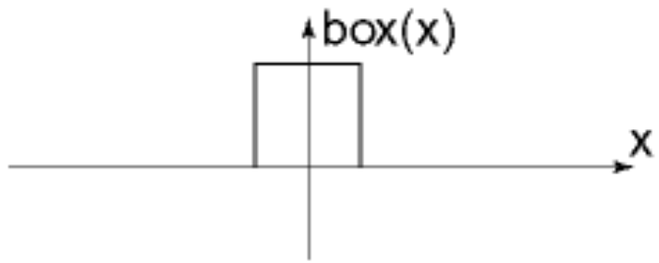
- Each DFT row is a complex sinusoidal basis pattern.
- A matching frequency creates a large coefficient.
- The second half represents negative-frequency components.
- This is why the DFT detects periodic patterns.

Example: 1D DFT spectrum

Two cosine components produce pairs of peaks.
Noise spreads small energy across many frequencies.
Magnitude reveals which frequencies are present.
Phase tells where those components align in the signal.



Fourier transform pairs

spatial domain	frequency domain
	
	
	

Note the symmetry: duality property of Fourier transform

From 1D DFT to 2D DFT

An image is a two-dimensional sampled function, so the basis becomes 2D waves.

Images as 2D arrays

- Let f[x, y] be intensity at column x and row y.
- Width M gives horizontal frequency index u.
- Height N gives vertical frequency index v.
- F[u, v] measures one 2D sinusoidal pattern.

$$f[x, y], \quad x = 0, \ldots, M-1, \quad y = 0, \ldots, N-1$$

2D DFT definition

- Forward transform

$$F(u,v) = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) e^{-j2\pi(ux/M + vy/N)}$$

$$\text{for } u = 0,1,2,\ldots,M-1, v = 0,1,2,\ldots,N-1$$

- Inverse transform:

$$f(x,y) = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u,v) e^{j2\pi(ux/M + vy/N)}$$

$$\text{for } x = 0,1,2,\ldots,M-1, y = 0,1,2,\ldots,N-1$$

- u, v : the transform or frequency variables
- x, y : the spatial or image variables

- The 2D DFT sums over both spatial dimensions.
- The exponential contains horizontal and vertical frequencies.
- The output F[u, v] is complex-valued.
- F[0,0] equals the sum of all pixels.

2D DFT

Similarly, we define the Fourier spectrum, phase angle, and power spectrum as follows:

$$F(u,v) = \left|F(u,v)\right| e^{j\phi(u,v)}$$

$$\left|F(u,v)\right| = \left[R^2(u,v) + I^2(u,v)\right]^{\frac{1}{2}} \quad \text{(spectrum)}$$

$$\phi(u,v) = \tan^{-1}\left[\frac{I(u,v)}{R(u,v)}\right] \quad \text{(phase angle)}$$

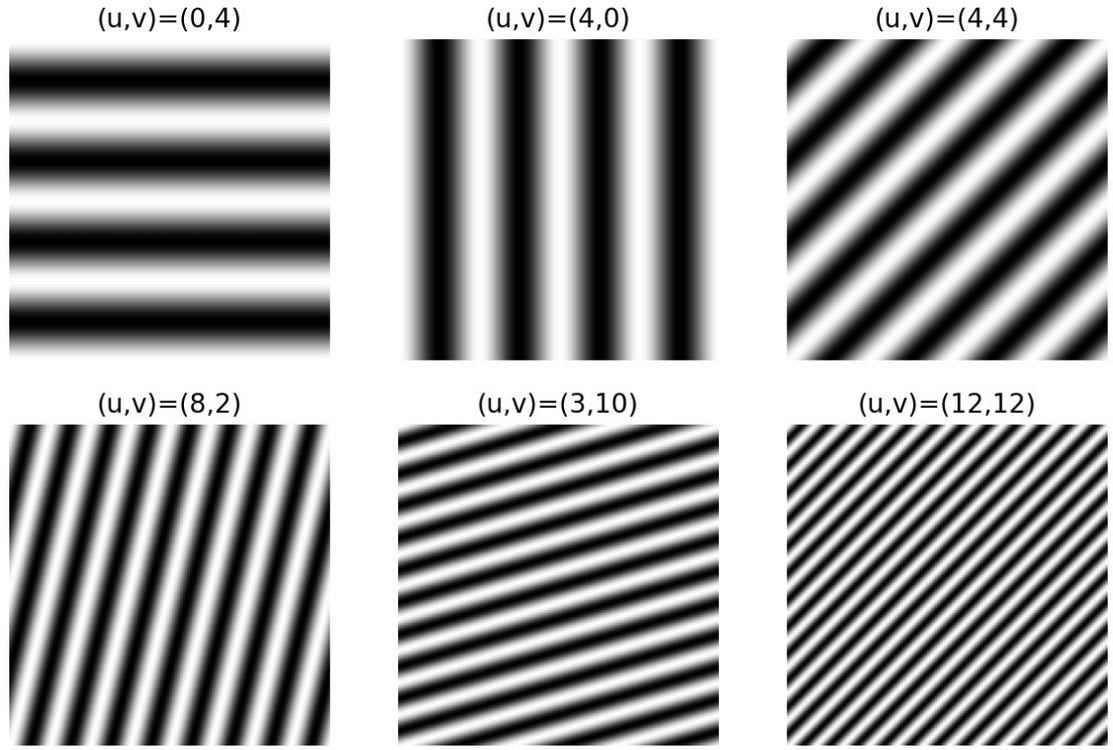
$$P(u,v) = \left|F(u,v)\right|^2 = R^2(u,v) + I^2(u,v) \quad \text{(power spectrum)}$$

$R(u,v)$: the real part of $F(u,v)$

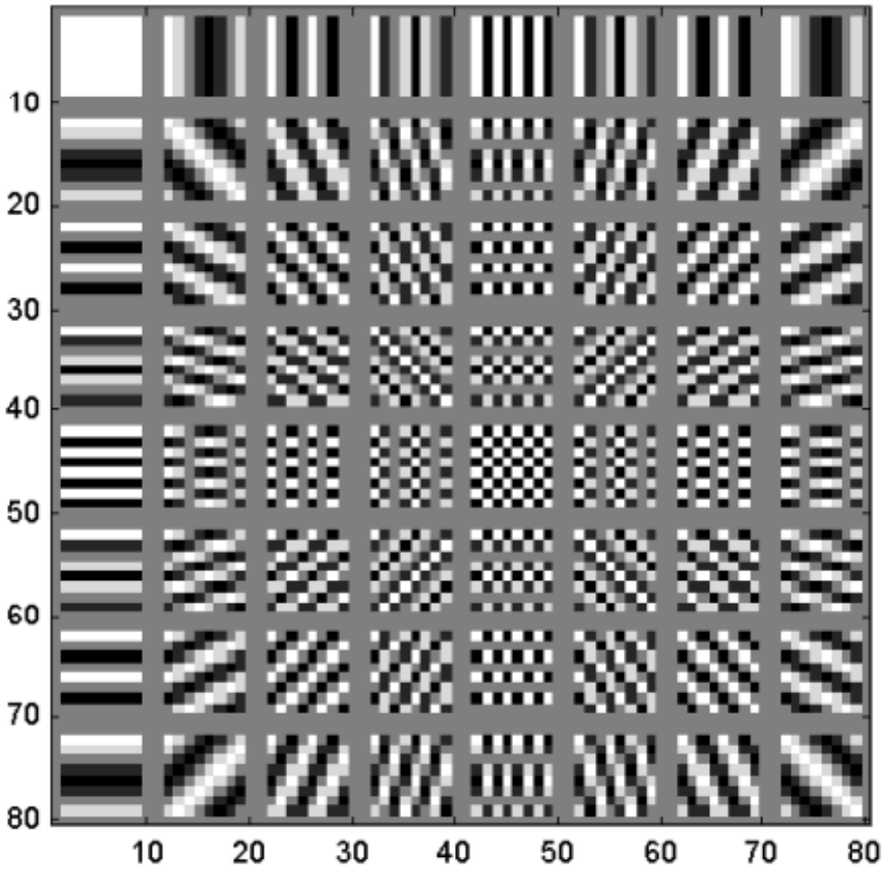
$I(u,v)$: the imaginary part of $F(u,v)$

2D basis images

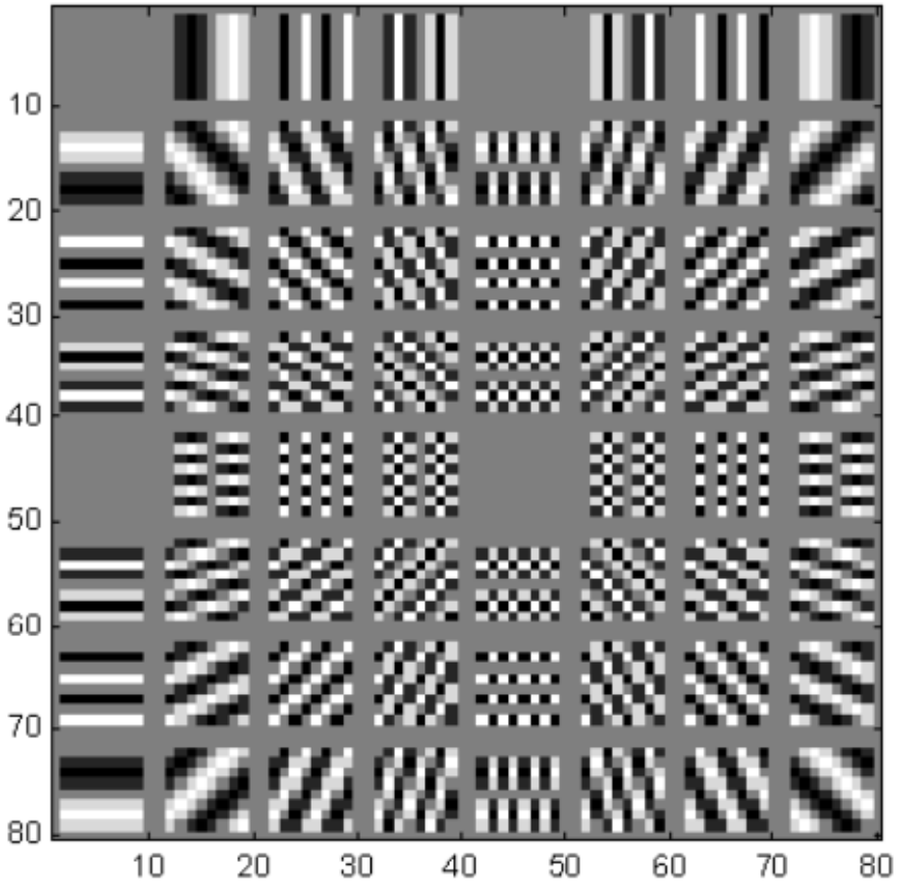
- The basis varies sinusoidally across x and y.
- u controls horizontal oscillation.
- v controls vertical oscillation.
- Large u or v creates rapid image variation.



Fourier Bases



Basis Images for 8x8 DFT (Real Part)



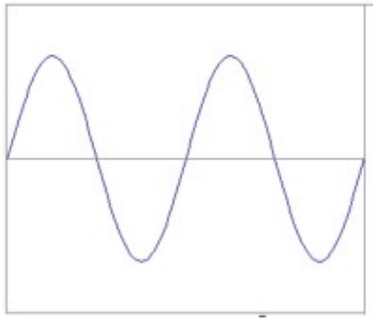
Basis Images for 8x8 DFT (Imaginary Part)

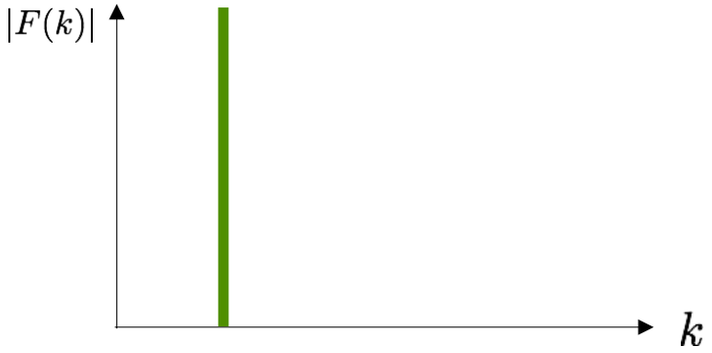
Examples

Spatial domain visualization

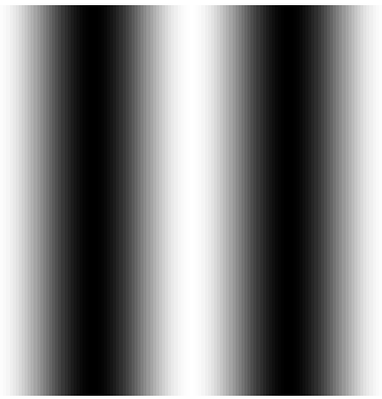
Frequency domain visualization

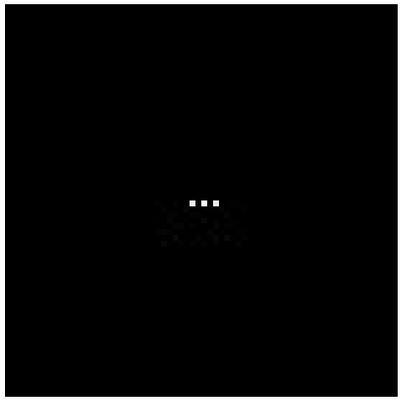
1D





2D

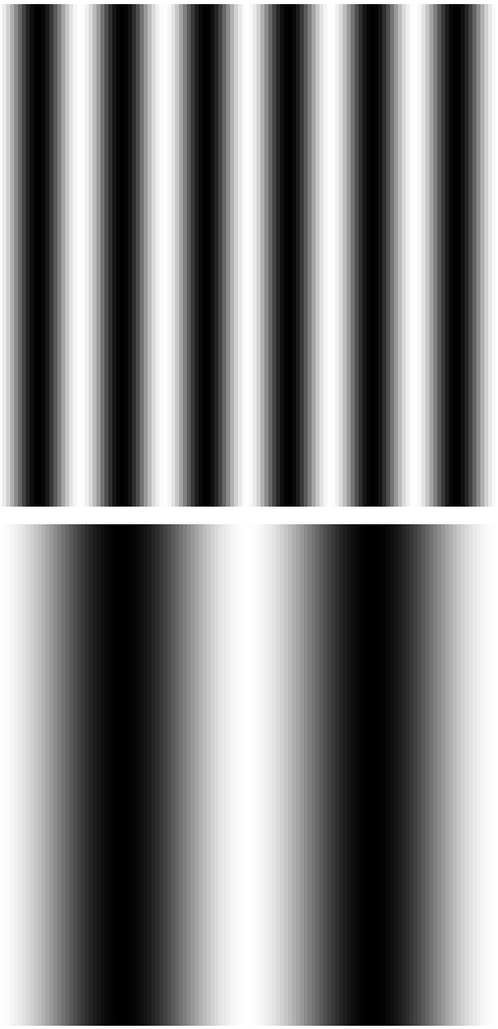




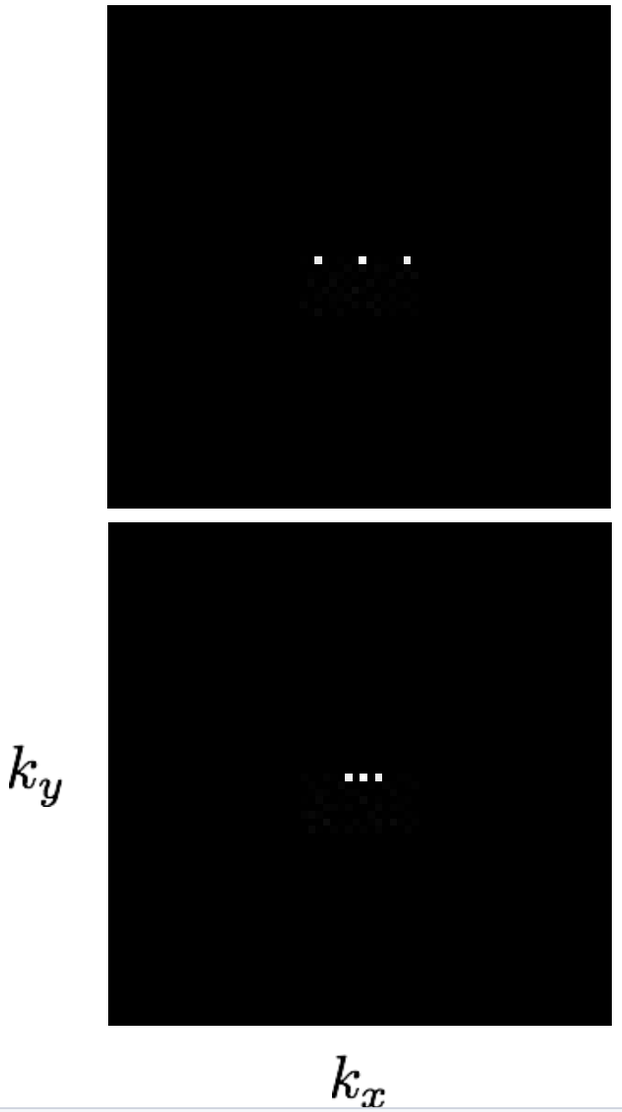
What do the three dots correspond to?

Examples

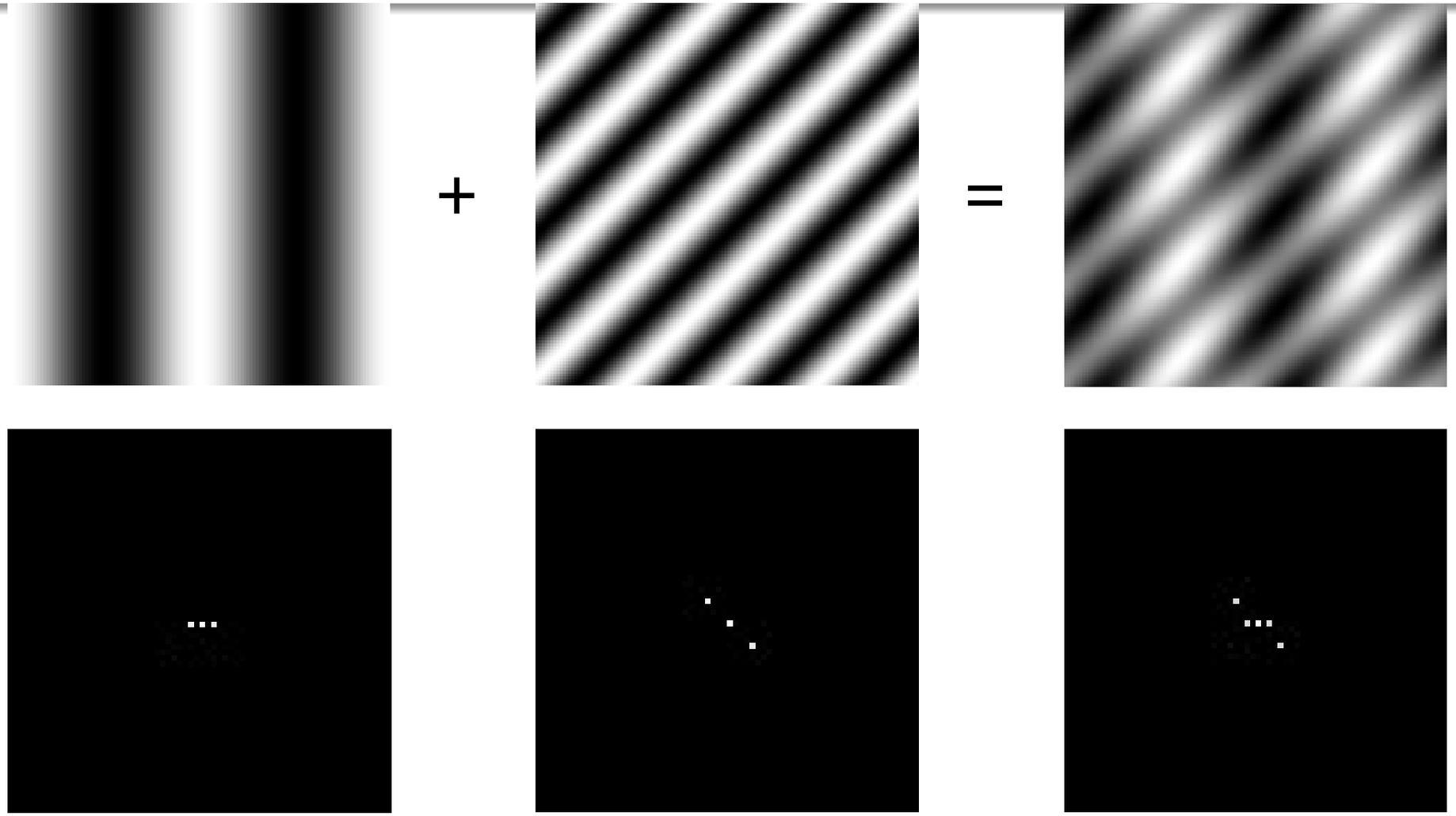
Spatial domain visualization



Frequency domain visualization

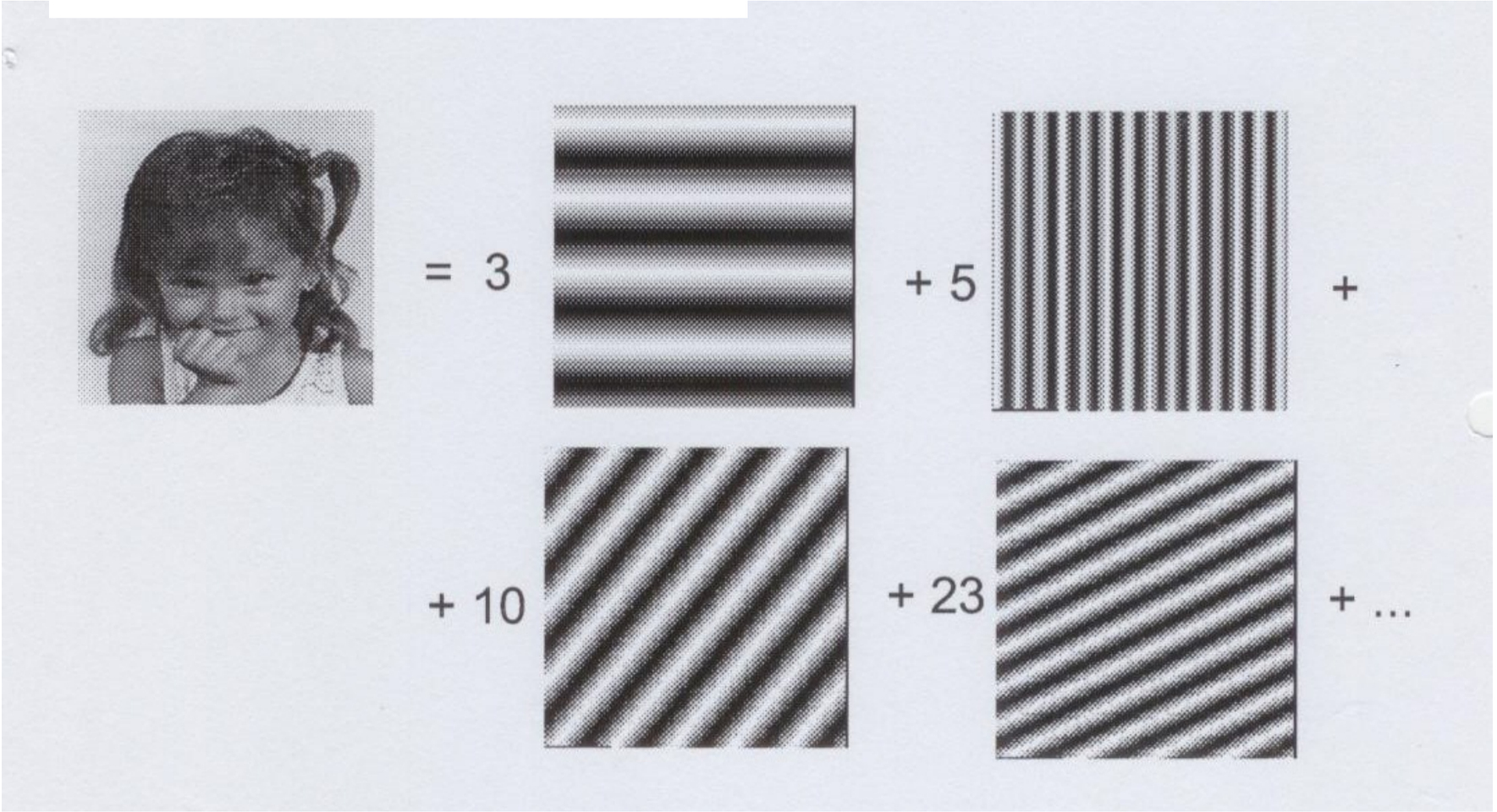


Examples



2D DFT Interpretation

$$f(x,y) = \frac{1}{N}\sum_{u=0}^{N-1}\sum_{v=0}^{N-1} F(u,v)e^{j2\pi(\frac{ux+vy}{N})},$$

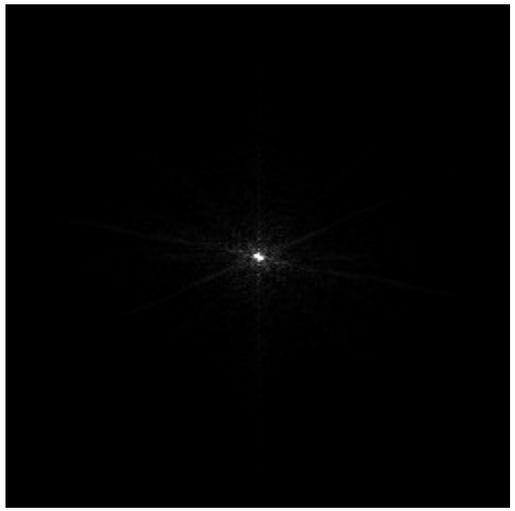


How to visualize DFT?

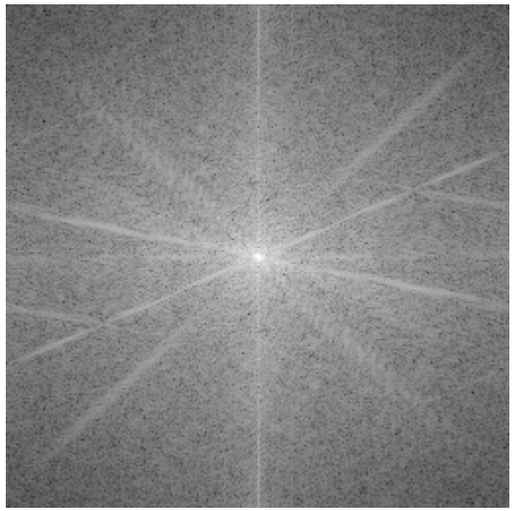
- Typically, visualize the magnitude (spectrum): |F(u,v)|
- Need to apply some “stretching” since the dynamic range of |F(u,v)| is very large:

$$D(u,v) = \log(1 + |F(u,v)|)$$

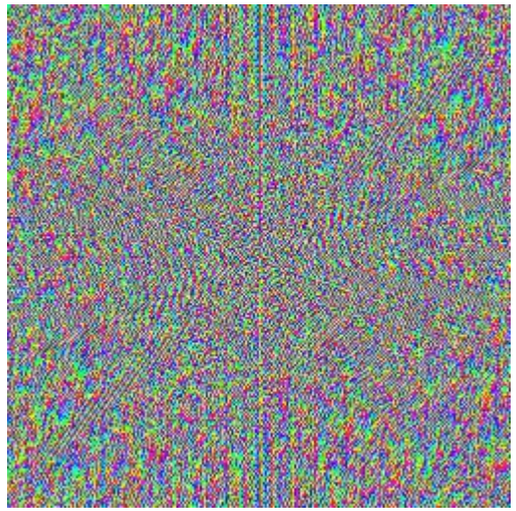
original



magnitude



Log magnitude

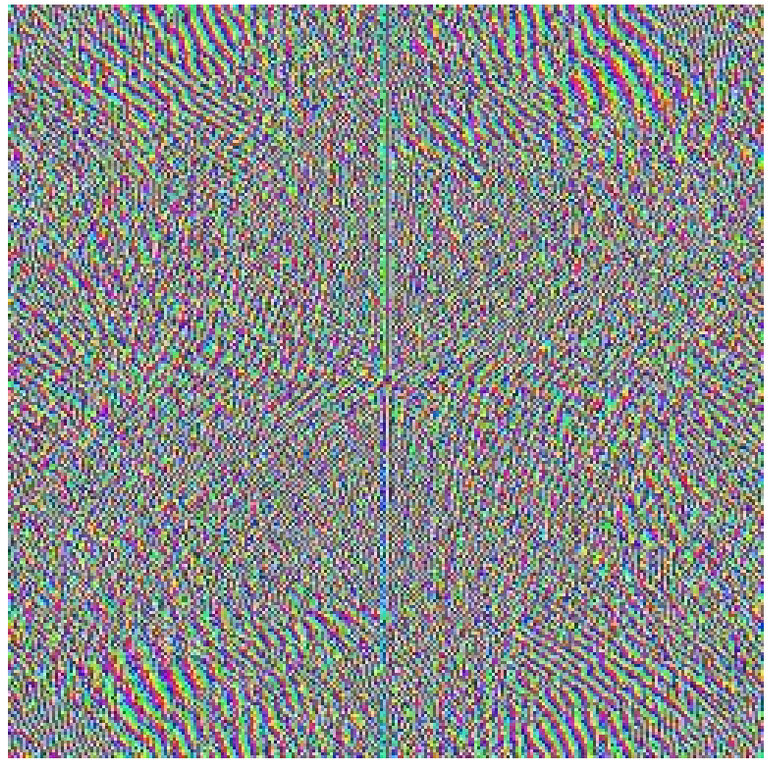


phase

Magnitude and Phase of DFT

- What is more important?

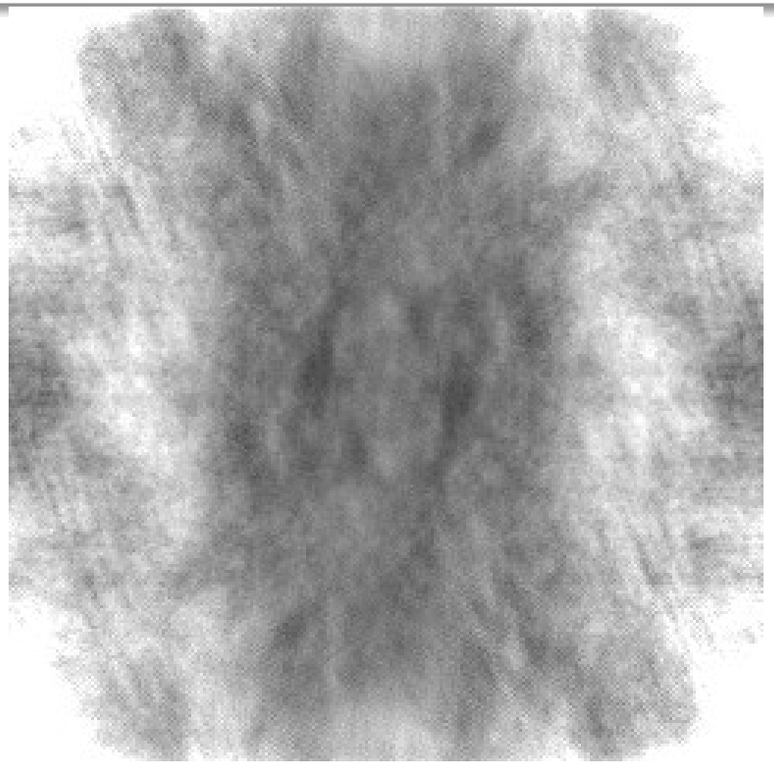
magnitude



phase

- Hint: use the inverse 2D DFT to reconstruct the input image using only magnitude or phase information.

Magnitude and Phase of DFT (cont'd)



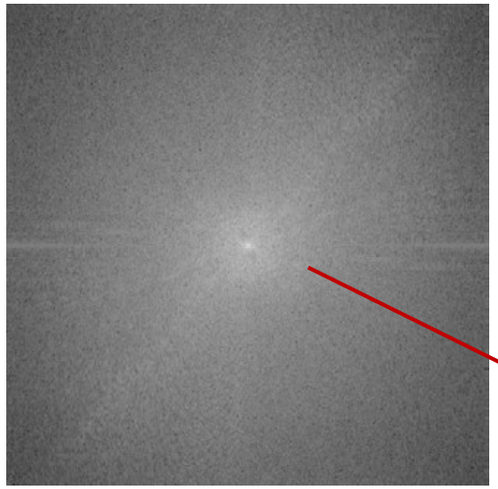
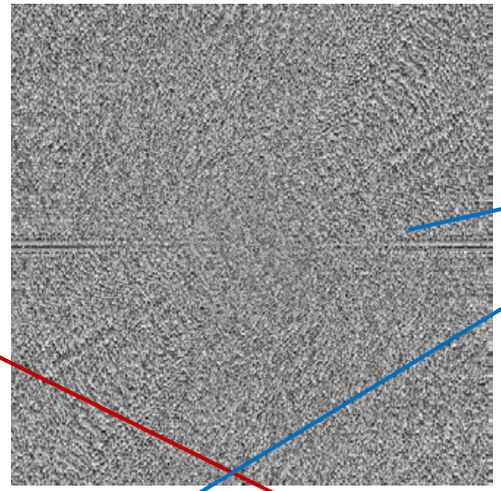
Reconstructed image using magnitude only

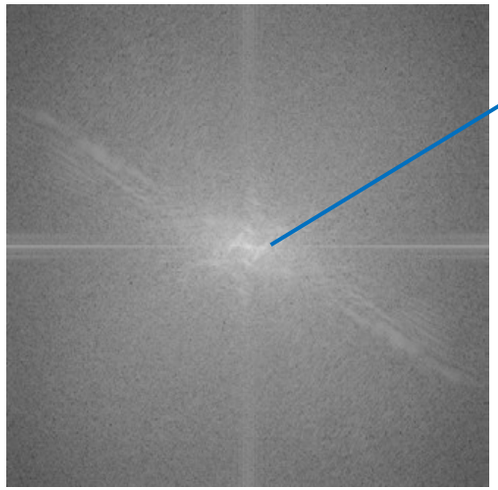
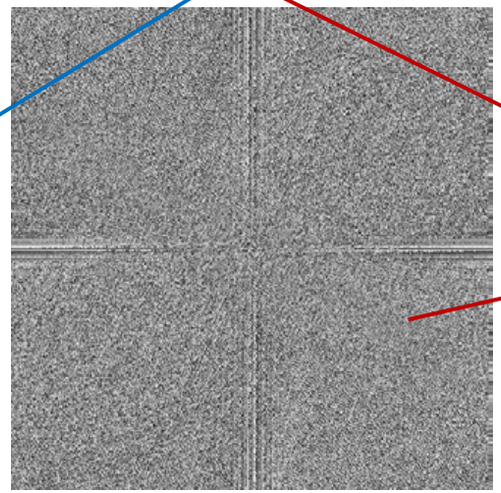
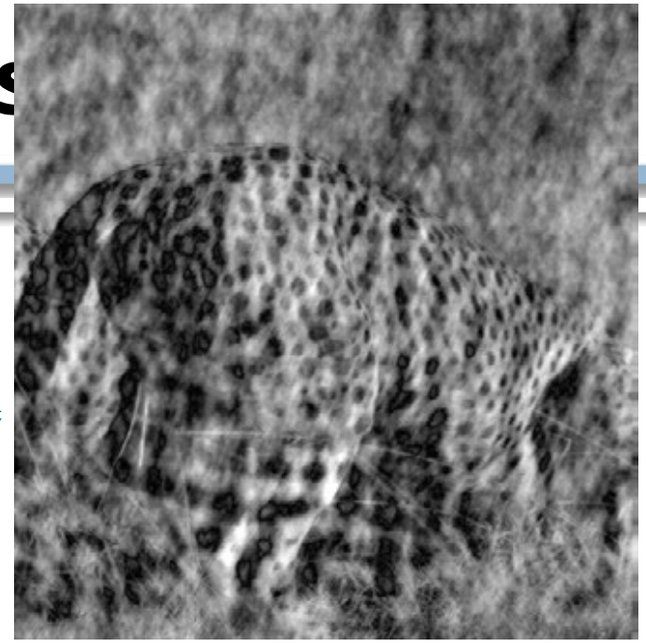
(i.e., magnitude determines the strength of each sinusoidal component)

Reconstructed using phase only

(i.e., phase determines the phase of each sinusoidal component)

Fourier transforms of natural images

amplitude phase reconstruction

2D DFT
Properties

2D DFT – Properties: Separability

- The 2D FT can be implemented as two consecutive 1D FTs: first in the x direction, then in the y direction (or vice versa).

$$F_1(u,y) = \frac{1}{M}\sum_{x=0}^{M-1} f(x,y)e^{-j2\pi(ux/M)}$$

$$F(u,v) = \frac{1}{N}\sum_{y=0}^{M-1} F_1(u,y)e^{-j2\pi(vy/N)}$$

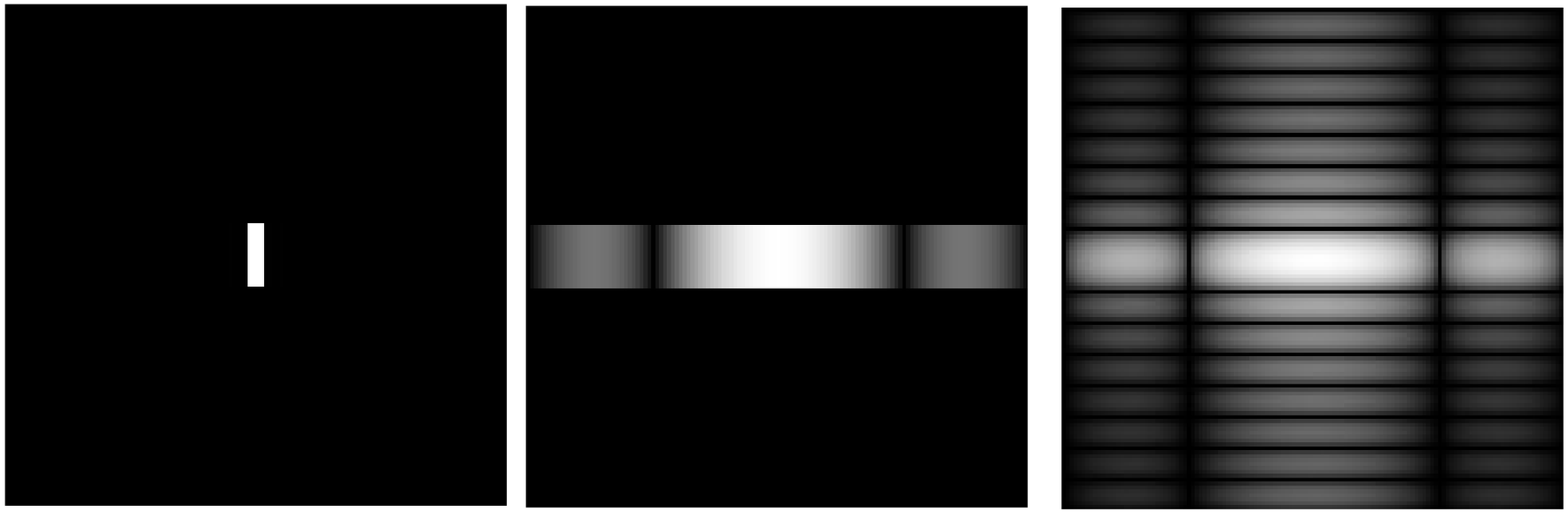
  - Proof: DIY

$$F[u,v] = \sum_{y=0}^{N-1}\left[\sum_{x=0}^{M-1} f[x,y]e^{-j2\pi ux/M}\right]e^{-j2\pi vy/N}$$

2D DFT – Properties: Separability

- **Separable:** The 2D DFT **F(u,v)** can be obtained by

1. taking the 1D DFT of every row of image **f(x,y)**, → **F(u,y)**,
2. taking the 1D DFT of every column of **F(u,y)**, → **F(u,v)**,

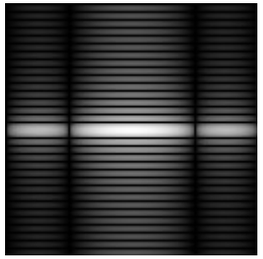


(a)f(x,y) (b)F(u,y) (c)F(u,v)

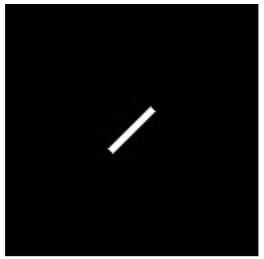
Linearity: proof

A

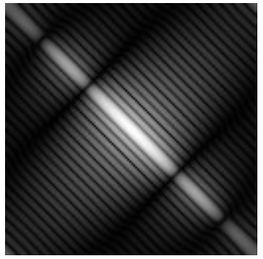
DFT



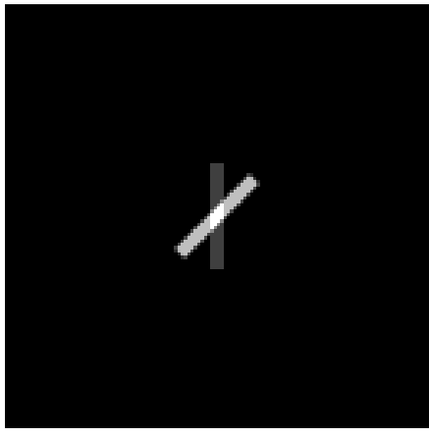
B



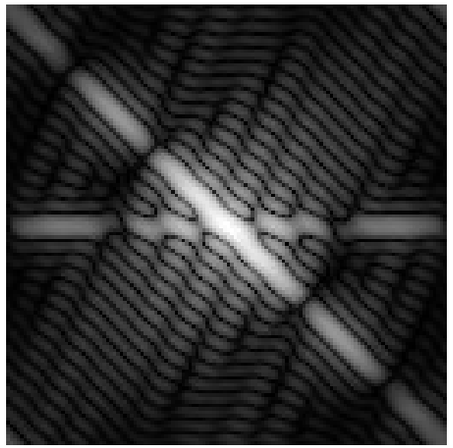
DFT



0.25 * A
+ 0.75 * B



DFT



Linearity: proof

Substitute g[x,y] = a f_1[x,y] + b f_2[x,y].
Distribute the summation over addition.
Constants a and b factor out of the sums.
Therefore, Fourier filtering is a linear operation.

$$\mathcal{F}\{af_1 + bf_2\} = \sum_x \sum_y (af_1[x,y] + bf_2[x,y]) e^{-j2\pi(ux/M + vy/N)}$$
$$= aF_1[u,v] + bF_2[u,v]$$

2D DFT – Properties: Shifting

- The FT of a shifted function is unaltered except for a linearly varying phase factor.

$$f(x - x_0, y - y_0) \Leftrightarrow F(u,v)e^{-j2\pi(ux_0/M + vy_0/N)}$$

Time / spatial shift

  - and

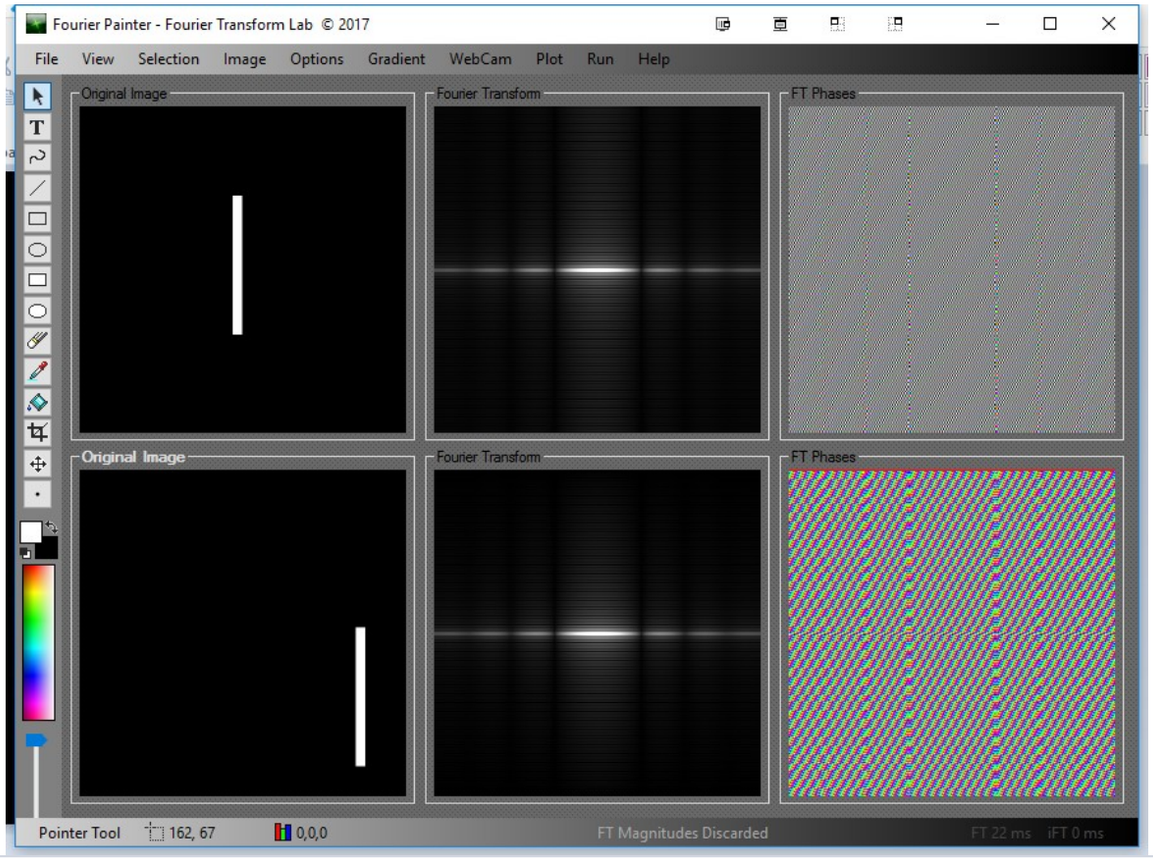
$$f(x,y)\, e^{j2\pi(u_0 x/M + v_0 y/N)} \Leftrightarrow F(u - u_0, v - v_0)$$

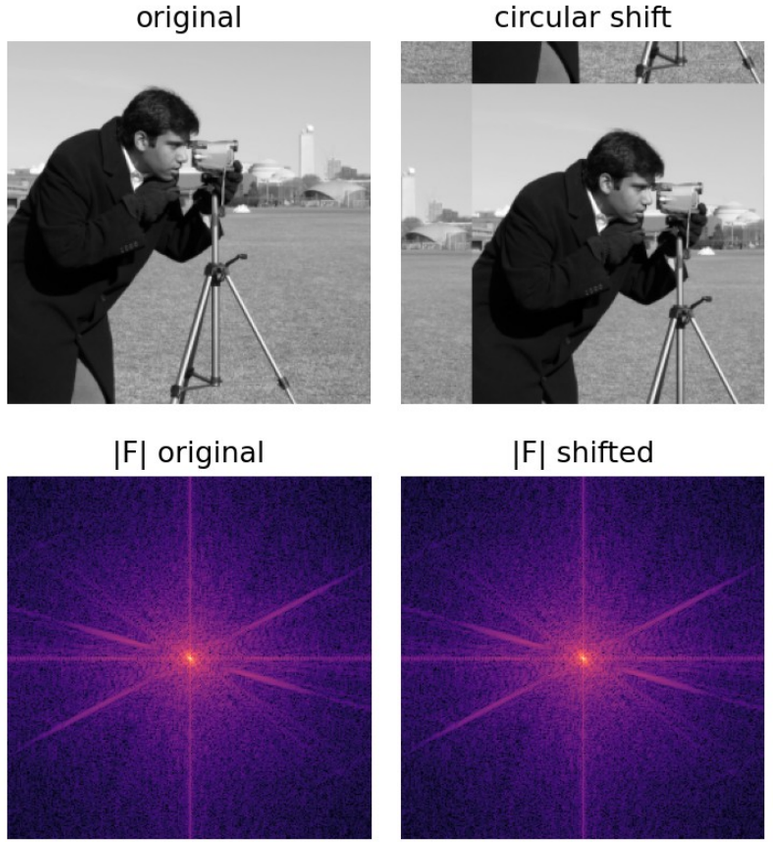
frequency shift

Modulation property

2D DFT – Properties: Shifting

Shifting an image changes only DFT phase.
Magnitude remains unchanged.
Spectrum magnitude ignores absolute position.
Phase stores location information.





2D DFT – Properties: Shifting

- The FT of a shifted function is unaltered except for a linearly varying phase factor.

$$f(x - x_0, y - y_0) \Leftrightarrow F(u,v)e^{-j2\pi(ux_0/M + vy_0/N)}$$

- **Proof:**

$$DFT[f(x - x_0, y - y_0)] = \frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f(x - x_0, y - y_0)e^{-j2\pi(ux/M + vy/N)}$$

$$= e^{-j2\pi[ux_0/M + vy_0/N]} \frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f(x - x_0, y - y_0)e^{-j2\pi[\frac{u(x-x_0)}{M} + \frac{v(y-y_0)}{N}]}$$

$$= e^{-j2\pi[ux_0/M + vy_0/N]}F(u,v)$$

2D DFT – Properties: Shifting

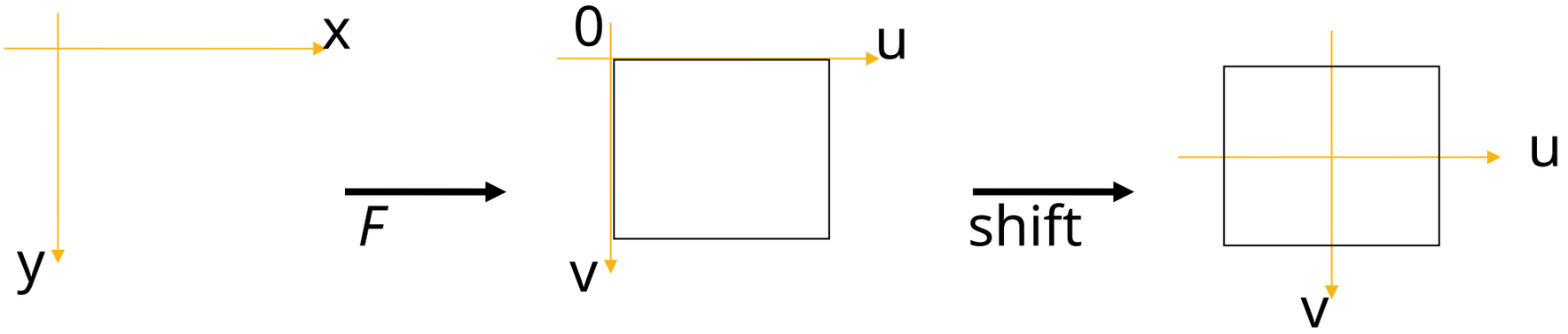
- If, u0 = M/2, v0 = N/2

$$f(x,y)\, e^{j2\pi(u_0 \frac{x}{M} + v_0 \frac{y}{N})} \leftrightarrow F(u - u_0, v - v_0)$$

- Becomes

cos(nπ) = 1 if *n* is even and cos(nπ) = −1 for odd *n*
sin(nπ) = 0 always

$$f(x,y)\,(-1)^{x+y} \leftrightarrow F(u - M/2, v - N/2)$$



2D DFT – Properties: Periodicity

- A [M,N] point DFT is periodic with period [M,N]

$$F(u,v) = F(u+M, v) = F(u, v+N)$$
$$= F(u+M, v+N) = F(u+pM, v+qN)$$

- **Proof:**

$$F(u,v) = \frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f(x,y)e^{-j2\pi(ux/M+vy/N)}$$

$$F(u+M,v+N) = \frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f(x,y)e^{-j2\pi\left(\frac{u+M}{M}x+\frac{v+N}{N}y\right)}$$

$$= \frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f(x,y)e^{-j2\pi\left(\frac{u}{M}x+\frac{v}{N}y\right)}.e^{-j2\pi\left(\frac{M}{M}x+\frac{N}{N}y\right)}$$

$$= e^{-j2\pi(x+y)}F(u,v)$$

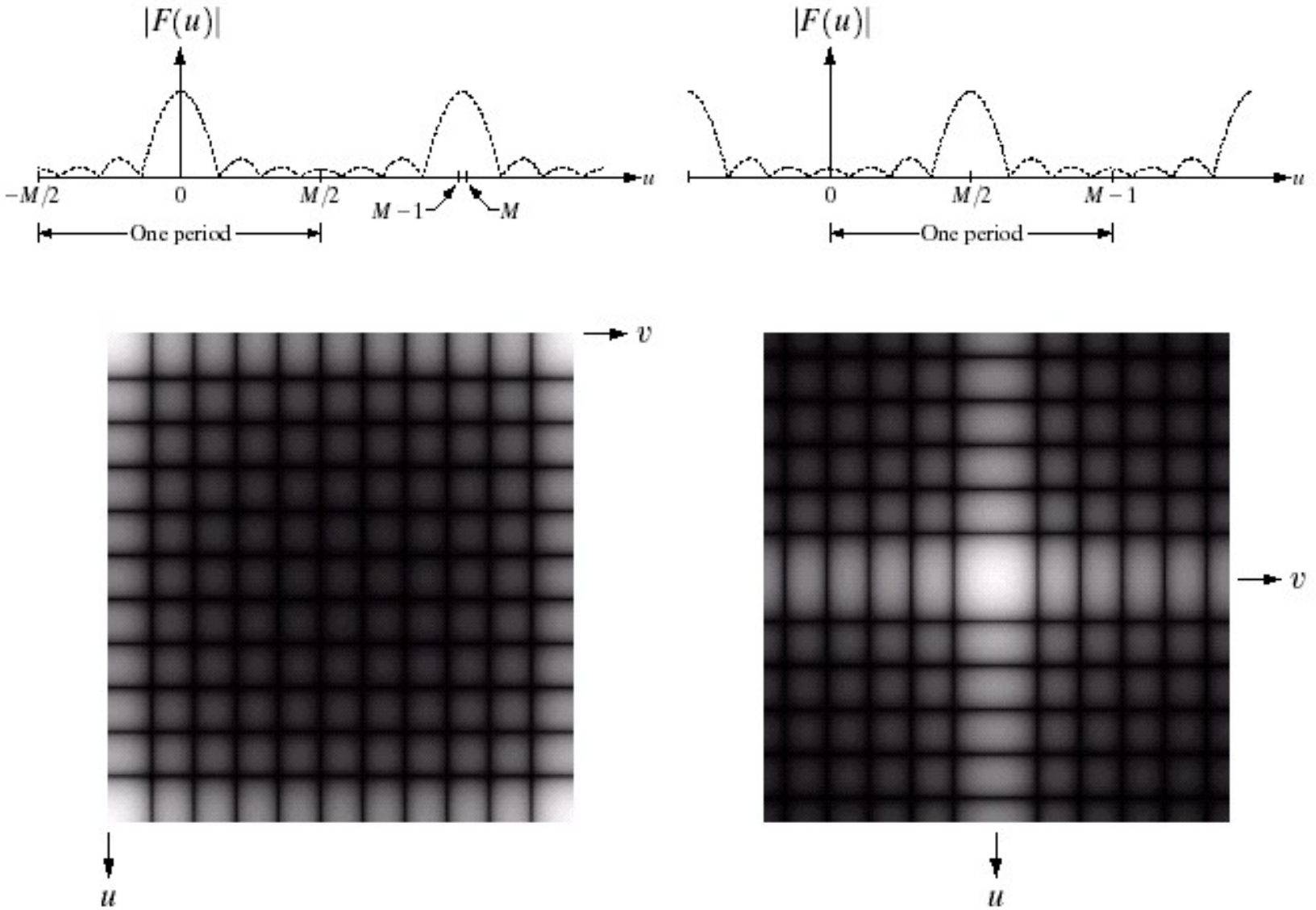
$$= F(u,v)$$

$$\left[\ \cos 2n\pi = 1,\ \sin 2n\pi = 0\right]$$

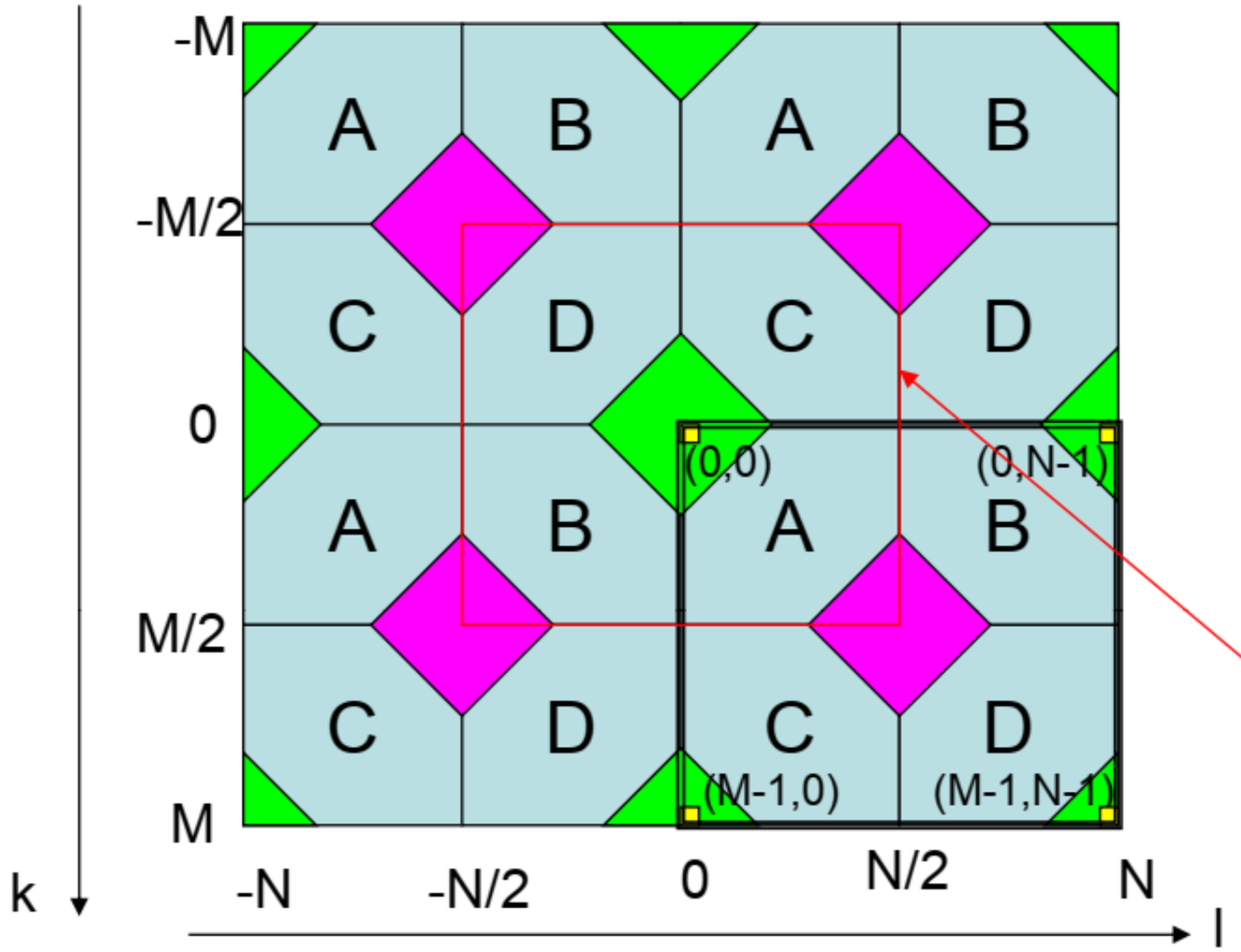
2D DFT – Properties: Periodicity

a b
c d

FIGURE 4.34
(a) Fourier spectrum showing back-to-back half periods in the interval $[0, M-1]$.
(b) Shifted spectrum showing a full period in the same interval.
(c) Fourier spectrum of an image, showing the same back-to-back properties as (a), but in two dimensions.
(d) Centered Fourier spectrum.



2D DFT – Properties: Periodicity



Low Frequency

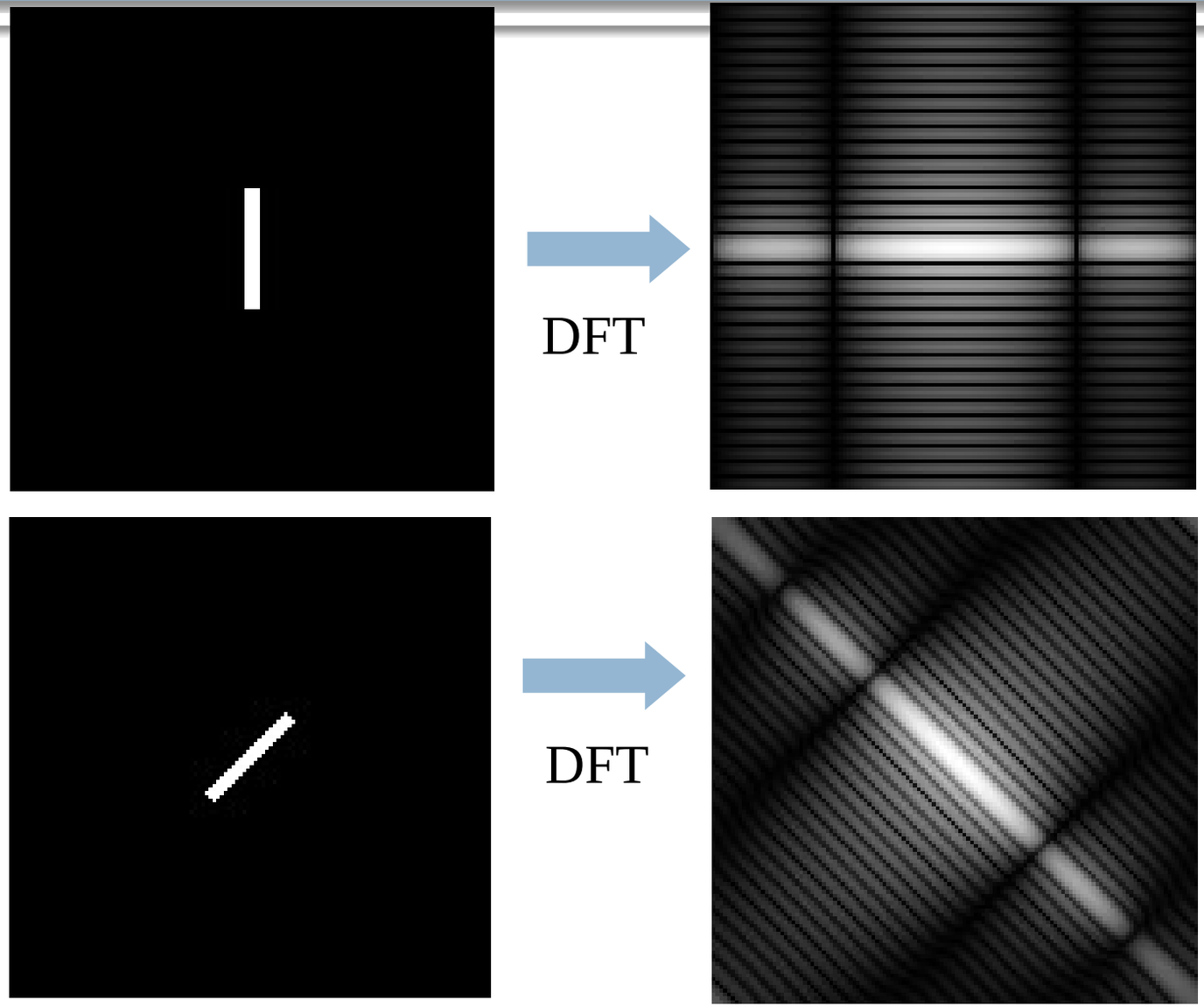
High Frequency

Displayed area using fftshift

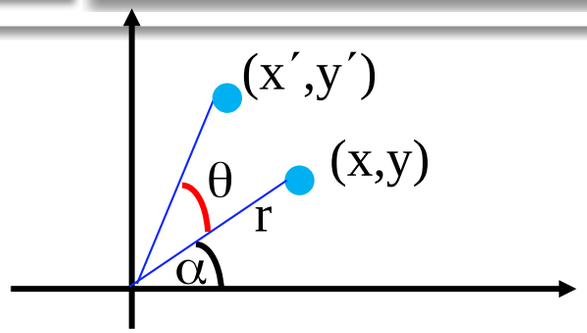
2D DFT – Properties: Rotation

- if a function is rotated, then its Fourier transform rotates an equal amount.

Rotation in spatial domain
⟺ same rotation infrequency domain.



2D DFT – Properties: Rotation



$$x = r\cos\alpha \qquad y = r\sin\alpha$$

Rotate θ

$$x' = r\cos(\alpha+\theta) \;;\; y' = r\sin(\alpha+\theta)$$

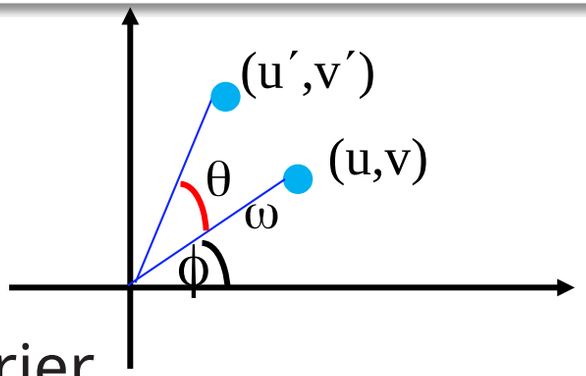
$$x' = r\cos\alpha\cos\theta - r\sin\alpha\sin\theta$$

$$\Rightarrow \quad y' = r\sin\alpha\cos\theta + r\cos\alpha\sin\theta$$

$$x' = x\cos\theta - y\sin\theta$$

$$\Rightarrow \quad y' = x\sin\theta + y\cos\theta$$

Similarly, in Fourier domain



$$u = \omega\cos\phi$$

$$v = \omega\sin\phi$$

$$u' = u\cos\theta - v\sin\theta$$

$$v' = u\sin\theta + v\cos\theta$$

$$u = u'\cos\theta + v'\sin\theta$$

$$v = -u'\sin\theta + v'\cos\theta$$

We can also express (*x*, *y*) in terms of (x′,y′)

$$x = x'\cos\theta + y'\sin\theta \;;\; y = -x'\sin\theta + y'\cos\theta$$

2D DFT – Properties: Rotation

- **Proof:** $DFT\left[f(x,y) \right] = F(u,v)$

$$DFT\left[f(r\cos\alpha, r\sin\alpha) \right] = F(\omega\cos\phi, w\sin\phi)$$

$$DFT\left[f(x',y') \right] = DFT\left[f(r\cos(\alpha+\theta), r\sin(\alpha+\theta)) \right]$$

$$= \sum_{x=-\infty}^{\infty} \sum_{y=-\infty}^{\infty} f(x',y') e^{-j2\pi(xu+yv)}$$

$$= \sum_{x=-\infty}^{\infty} \sum_{y=-\infty}^{\infty} f(x',y') e^{-j2\pi(x'u'+y'v')}$$

$$= F(u',v')$$

$$= F(\omega\cos(\phi+\theta), \omega\sin(\phi+\theta))$$

$$ux + vy$$
$$= u(x'cos\theta + y'sin\theta) + v(-x'sin\theta + y'cos\theta)$$
$$= x'(ucos\theta - vsin\theta) + y'(usin\theta + vcos\theta)$$
$$= u'x' + v'y'$$

2D DFT – Properties

Derivatives and Laplacian

Considering 1-D, it can be shown that

$$\Im\left[\frac{d^n f(x)}{dx^n}\right] = (ju)^n F(u),$$

Implementing the above equation for 2-D Laplacian

$$\Im\left[\frac{\partial^2 f(x,y)}{\partial x^2} + \frac{\partial^2 f(x,y)}{\partial y^2}\right] = (ju)^2 F(u,v) + (jv)^2 F(u,v)$$

$$= -(u^2 + v^2)F(u,v).$$

2D DFT – Properties: Convolution theorem

~ states that convolution in time domain corresponds to multiplication in frequency domain and vice versa

The discrete convolution of two functions $f(x,y)$ and $h(x,y)$ of size MXN is defined as

$$f(x,y) * h(x,y) = \sum_{a=0}^{M-1} \sum_{b=0}^{N-1} f(a,b) h(x-a, y-b)$$

Let $F(u,v)$ and $H(u,v)$ denote the Fourier Transforms of $f(x,y)$ and $h(x,y)$, then

Eq. (4.2-31)

$$f(x,y) * h(x,y) \Leftrightarrow F(u,v)H(u,v)$$

Eq. (4.2-32)

$$f(x,y)h(x,y) \Leftrightarrow F(u,v) * H(u,v)$$

2D DFT – Properties: Convolution theorem

Proof:

$$DFT\left[f(x,y)*h(x,y)\right]=\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}\left[\sum_{a=0}^{M-1}\sum_{b=0}^{N-1}f(a,b)h(x-a,y-b)\right]e^{-j2\pi(ux/M+vy/N)}$$

$$=\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}\sum_{a=0}^{M-1}\sum_{b=0}^{N-1}f(a,b)h(x-a,y-b)e^{-j2\pi\left(\frac{u(x-a+a)}{M}+\frac{v(y-b+b)}{N}\right)}$$

$$=\sum_{a=0}^{M-1}\sum_{b=0}^{N-1}f(a,b)e^{-j2\pi\left(\frac{ua}{M}+\frac{vb}{N}\right)}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}h(x-a,y-b)e^{-j2\pi\left(\frac{u(x-a)}{M}+\frac{v(y-b)}{N}\right)}$$

$$=F(u,v)H(u,v)$$

Convolution using DFT

- Large speedups if S is large
- Example: $M = 512 \times 512, S = 32 \times 32$
- **i) Spatial Domain**
- **Direct Method (2D Convolution):**
  - **Formula:** N^2M^2 real multiplications | $N^2(M^2 - 1)$ real additions
  - **Multiplications:** $512^2 \times 32^2 = 268{,}435{,}456 \approx 268 \times 10^6$
  - **Additions:** $512^2 \times (1024 - 1) = 268{,}173{,}312 \approx 268 \times 10^6$
  - **Total FLOPs:** $\approx 5.37 \times 10^8$
- **Separability (1D Kernels):**
  - **Formula:** $2N^2M$ real multiplications | $2N^2(M - 1)$ real additions
  - **Multiplications:** $2 \times 512^2 \times 32 = 16{,}777{,}216 \approx 16.8 \times 10^6$
  - **Additions:** $2 \times 512^2 \times 31 = 16{,}252{,}928 \approx 16.3 \times 10^6$
  - **Total FLOPs:** $\approx 3.30 \times 10^7$

Convolution using DFT

- Large speedups if S is large
- Example: $M = 512 \times 512, S = 32 \times 32$
- **ii) Fourier Domain using Direct Method (Standard DFT)**
- *(Without precomputed kernel: Requires **3 DFTs** [Image Forward + Kernel Forward + Output Inverse] +* N^2 *pointwise complex multiplications)*
- **Direct Method (2D Non-Separable DFT):**
  - **Formula:** $3N^4 + N^2$ complex mults
  - **Complex Multiplications:** $3(512^4) + 512^2 = \mathbf{206,158,692,352} (\approx 206 \times 10^9)$
- **Separability (Row-Column 1D DFTs):**
  - **Formula:** $6N^3 + N^2$ complex mults
  - **Complex Multiplications:** $6(512^3) + 512^2 = \mathbf{805,568,512} (\approx 806 \times 10^6)$

Convolution using DFT

- Large speedups if S is large
- Example: $M = 512 \times 512, S = 32 \times 32$
- **iii) Fourier Domain using FFT**
- *(Without precomputed kernel: Requires **3 FFTs** + N^2 pointwise complex multiplications; $\log_2(512) = 9$)*
- **Direct FFT (2D Vector-Radix FFT):**
  - **Formula:** $3 \times \left(\frac{3}{4} N^2 \log_2 N\right) + N^2 = \frac{9}{4} N^2 \log_2 N + N^2$ complex mults
  - **Complex Multiplications:** $\frac{9}{4}(512^2 \times 9) + 512^2 = \mathbf{5,570,560} (\approx 5.57 \times 10^6)$
- **Separability (Row-Column 1D Radix-2 FFTs):**
  - **Formula:** $3(N^2 \log_2 N) + N^2$ complex mults
  - **Complex Multiplications:** $3(512^2 \times 9) + 512^2 = \mathbf{7,340,032} (\approx 7.34 \times 10^6)$
- *(Note: Zero-padding to 1024×1024 for exact linear convolution requires $3 \times 1024^2 \log_2(1024) + 1024^2 = \mathbf{32,505,856}$ complex mults).*

2D DFT

- Some other properties of Fourier transform:

$$F(0,0) = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y)$$

  - $F(0,0)$ is the average intensity of an image
    - If $f(x,y)$ is real, its Fourier transform is conjugate symmetric

$$F(u,v) = F^*(-u,-v)$$

      - The spectrum has even symmetry about the origin

$$|F(u,v)| = |F(-u,-v)|$$

      - The phase has odd symmetry about the origin

$$\phi(u,v) = -\phi(-u,-v)$$

Summary

- Fourier analysis represents images using 2D sinusoidal basis functions.
- The 2D DFT maps spatial patterns to frequency coefficients.
- Key properties explain phase, symmetry, separability, convolution, and filtering.

Filtering in Frequency Domain

Prof. Dr. Sk. Md. Masudul Ahsan
Dept. of CSE, KUET

Quick recap

The 2D DFT represents an image as weighted 2D complex exponentials.
Low frequency is near the center after fftshift; high frequency is farther away.
Magnitude shows strength of each frequency; phase preserves spatial structure.
Filtering changes selected coefficients before inverse DFT.

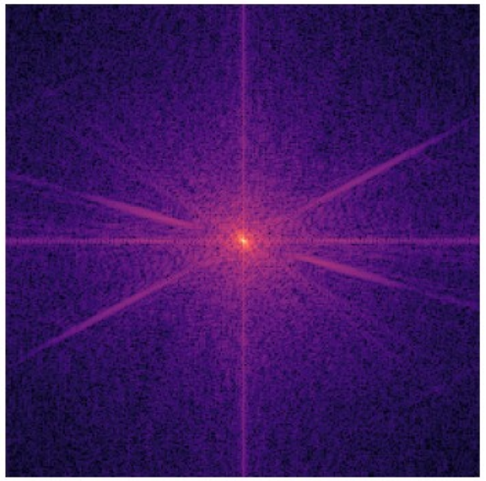
$$F[u,v] = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f[x,y] e^{-j2\pi(ux/M + vy/N)}$$

The filtering question

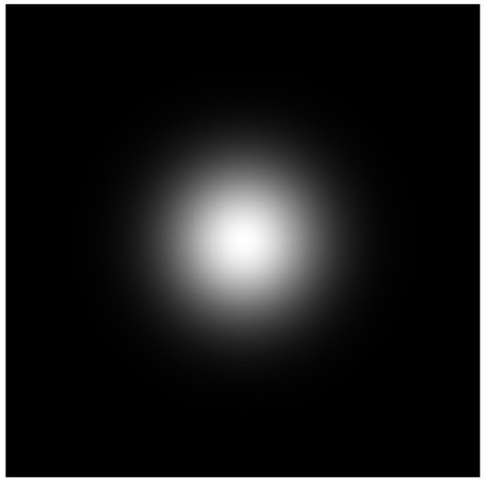
What frequency content should be preserved?
What frequency content should be attenuated?
What frequency content should be amplified?
The answer depends on the image goal: smoothing, sharpening, or artifact removal.

image f(x,y)

log |F(u,v)|



filter H(u,v)



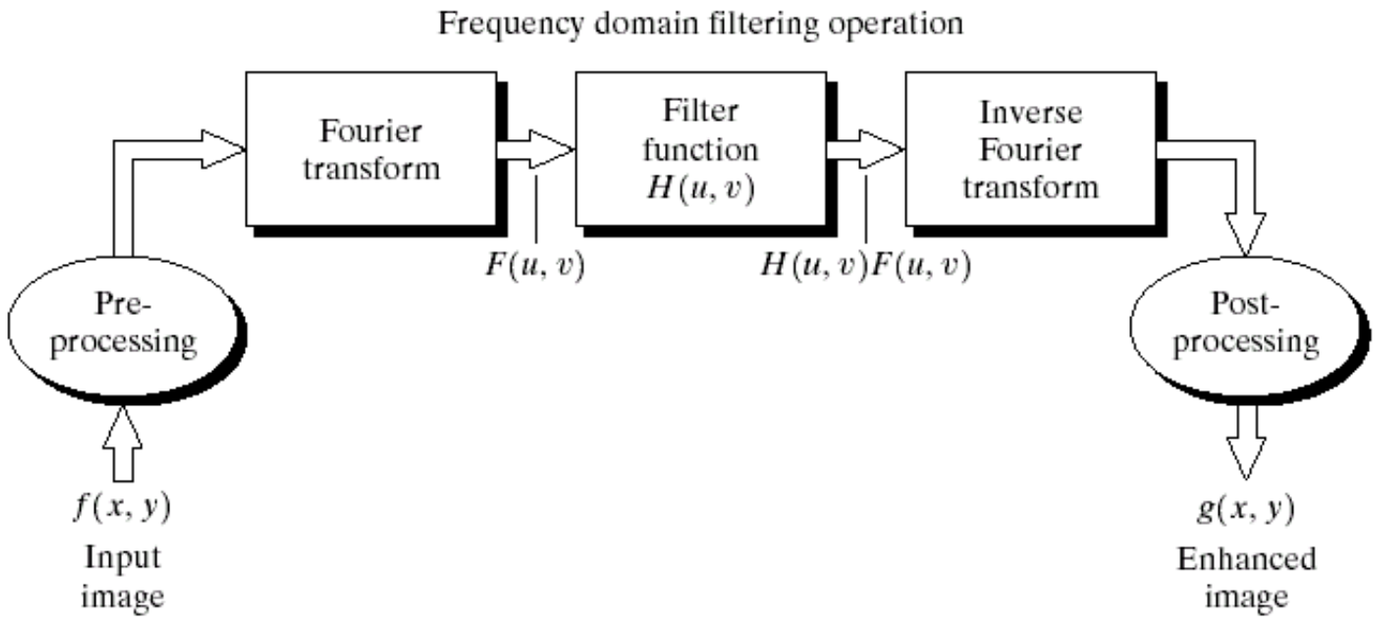
output g(x,y)

Basics of Filtering in the Frequency Domain

To filter an image in the frequency domain:

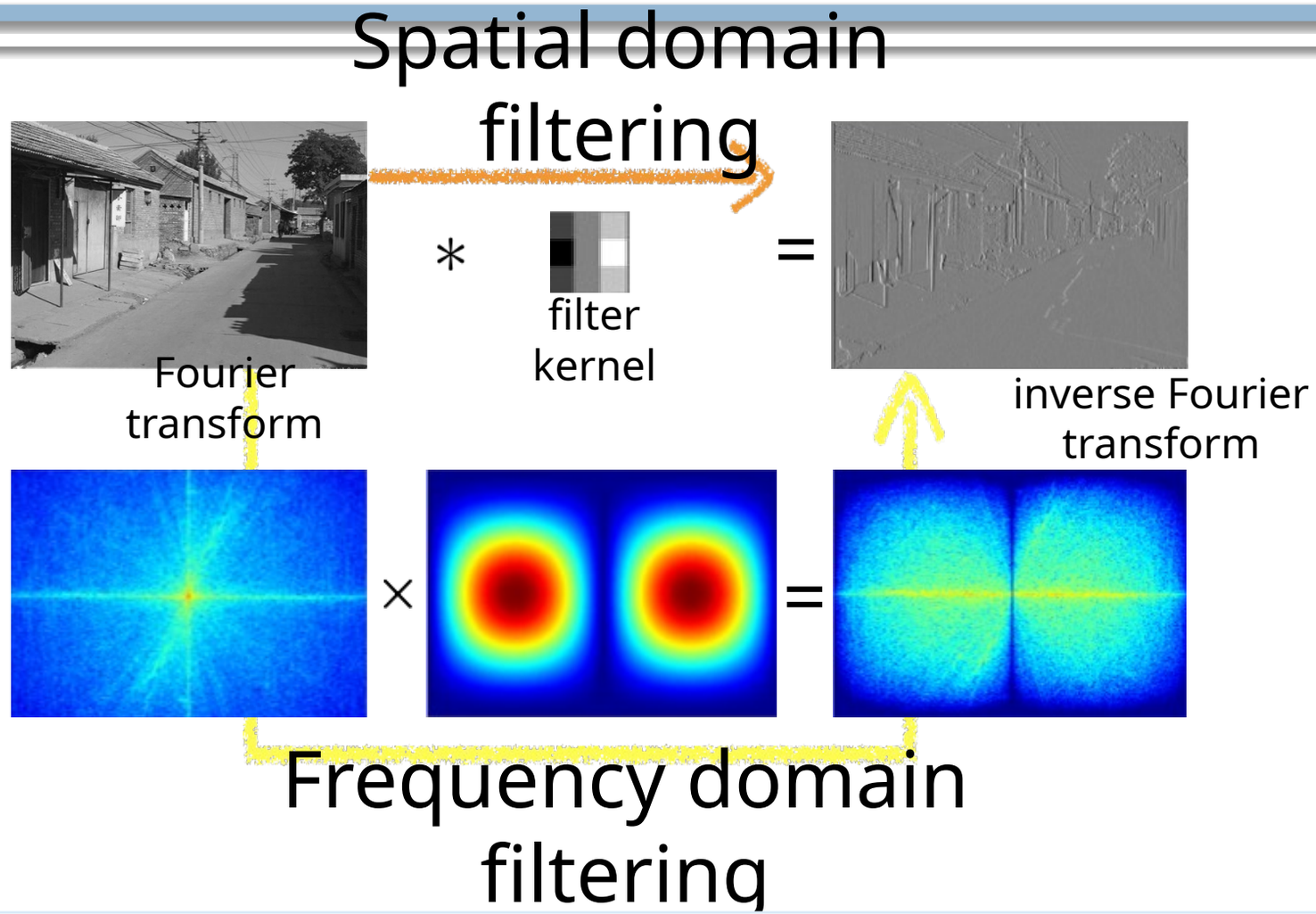
1. Compute $F(u,v)$ the DFT of the image
2. Multiply $F(u,v)$ by a filter function $H(u,v)$
3. Compute the inverse DFT of the result

$$G[u,v] = H[u,v]F[u,v], \qquad g[x,y] = \mathcal{F}^{-1}\{G[u,v]\}$$



If H is real and nonnegative, it changes magnitude. If H has phase, it also shifts or distorts structures.

Spatial vs Frequency domain filtering

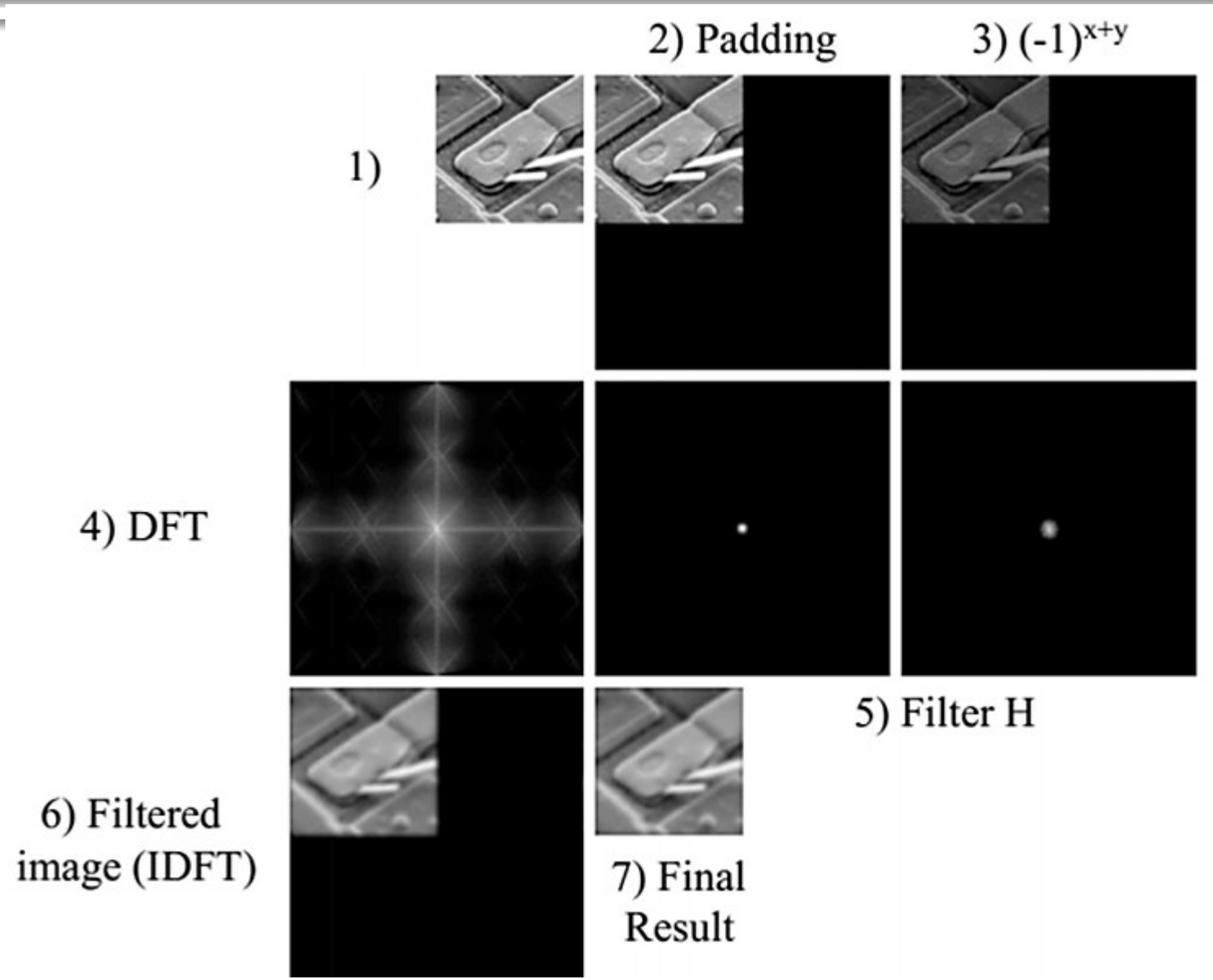


Designing H(u,v) is the main engineering decision.

Steps for Filtering in the Frequency Domain

1. Given an input image $f(x,y)$ of size MxN, obtain padding parameters P and Q. Typically, $P=2M$ and $Q=2N$.
2. Form a padded image $f_p(x,y)$ of size PxQ by appending the necessary number of zeros to f(x,y).
3. Multiply $f_p(x,y)$ by $(-1)^{x+y}$ to centre its transform.
4. Compute the DFT, $F(u,v)$, of the image from step 3.
5. Generate a real, symmetric filter function, $H(u,v)$, of size PxQ with centre at coordinates $(P/2,Q/2)$. Form the product $G(u,v)=H(u,v)F(u,v)$ using array multiplication.
6. Obtain the processed image: $g_p(x,y) = \text{real}\left[IDFT\left[G(u,v)\right]\right](-1)^{x+y}$
7. Obtain the final processed result, $g(x,y)$, by extracting the MxN region from the top, left quadrant of $g_p(x,y)$

Steps for Filtering in the Frequency Domain

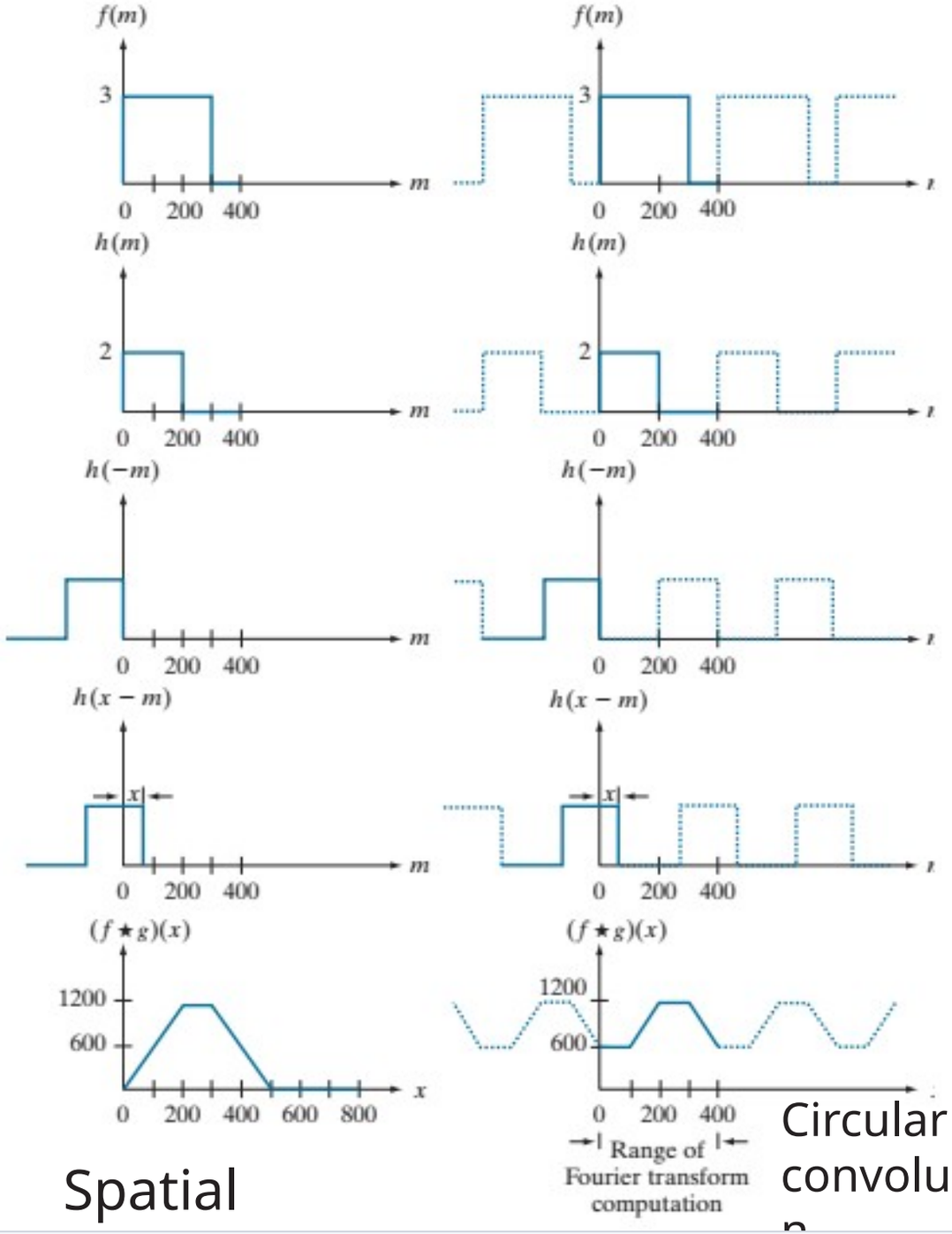


a b c
d e f
g h

FIGURE 4.36
(a) An $M \times N$ image, f.
(b) Padded image, f_p of size $P \times Q$.
(c) Result of multiplying f_p by $(-1)^{x+y}$.
(d) Spectrum of F_p. (e) Centered Gaussian lowpass filter, H, of size $P \times Q$.
(f) Spectrum of the product HF_p.
(g) g_p, the product of $(-1)^{x+y}$ and the real part of the IDFT of HF_p.
(h) Final result, g, obtained by cropping the first M rows and N columns of g_p.

Why – pad?

- The Circular convolution result we would obtain using the DFT
- This erroneous result can be remedied by using zero padding



Filter design axes

Pass region: frequencies that should remain.
Stop region: frequencies that should be suppressed.
Transition width: how quickly the response changes.
Gain: whether frequencies are attenuated or amplified.
Phase: usually keep zero phase for enhancement tasks.

Most enhancement filters use real, symmetric H(u,v), so the output has little phase distortion.

Filters to be Discussed

- Notch Filters
- Low pass filters (smoothing filters)
  - Ideal low pass filters
  - Butterworth low pass filters
  - Gaussian low pass filters
- High pass filters (sharpening filters)
  - Ideal high pass filters
  - Butterworth high pass filters
  - Gaussian high pass filters
  - Laplacian in Frequency domain
  - Unsharp masking, high boost filtering and high frequency emphasis filters
- Homomorphic filters

Filters in FD

- A **low-pass filter** attenuates high frequencies and retains low frequencies unchanged.
  - The result in the spatial domain is equivalent to that of a smoothing filter; as the blocked high frequencies correspond to sharp intensity changes, i.e. to the fine-scale details and noise in the spatial domain image.
- A **high pass filter**, on the other hand, yields edge enhancement or edge detection in the spatial domain, because edges contain many high frequencies. Areas of constant gray level consist of mainly low frequencies and are therefore suppressed.

Filters in FD

- A **bandpass filter** attenuates very low and very high frequencies, but retains a middle range band of frequencies.
  - can be used to enhance edges (suppressing low frequencies) while reducing the noise at the same time (attenuating high frequencies).
  - They are a combination of both lowpass and highpass filters. They attenuate all frequencies smaller than a frequency D_0 and higher than a frequency D_1 , while the frequencies between the two cut-offs remain in the resulting output image.

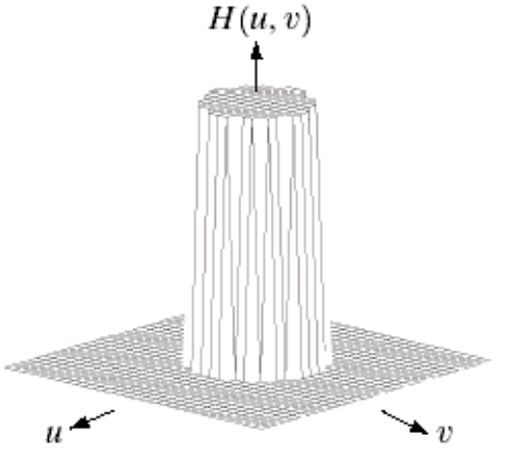
LOW-PASS SMOOTHING

Smoothing keeps slow spatial variation and suppresses high-frequency detail or noise.

Low-pass idea for images

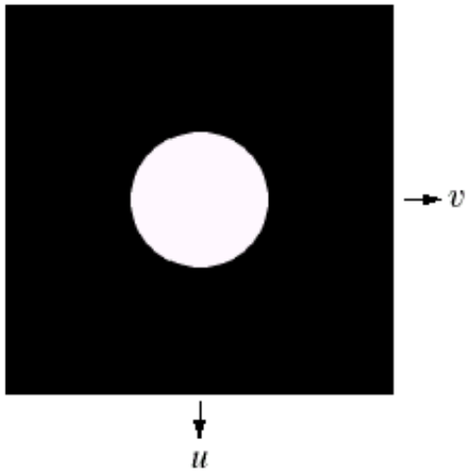
Slow variations represent background, illumination, and broad tissue regions.
Rapid variations represent edges, texture, small structures, and noise.
A low-pass filter keeps the center of the shifted spectrum.
The cutoff controls how much detail is removed.

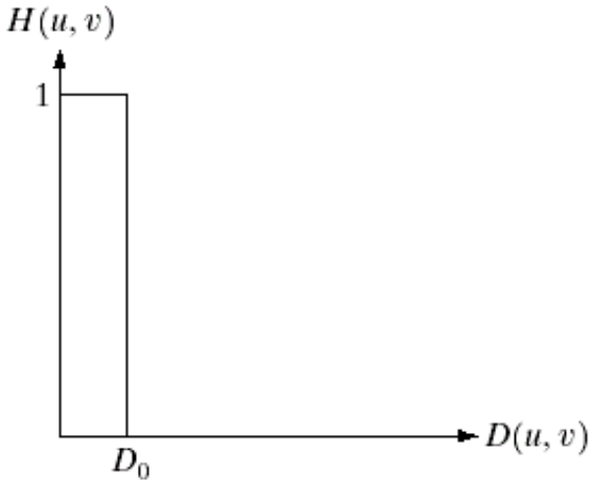
Ideal Low pass Filter (ILPF)



- The ILPF has a hard circular cutoff D_0.
  - creates ringing in the spatial domain
- It keeps all frequencies inside radius D_0.
- It removes all frequencies outside radius D_0.
- The transfer function

$$H(u,v) = \begin{cases} 1 & \text{if } D(u,v) \le D_0 \\ 0 & \text{if } D(u,v) > D_0 \end{cases}$$

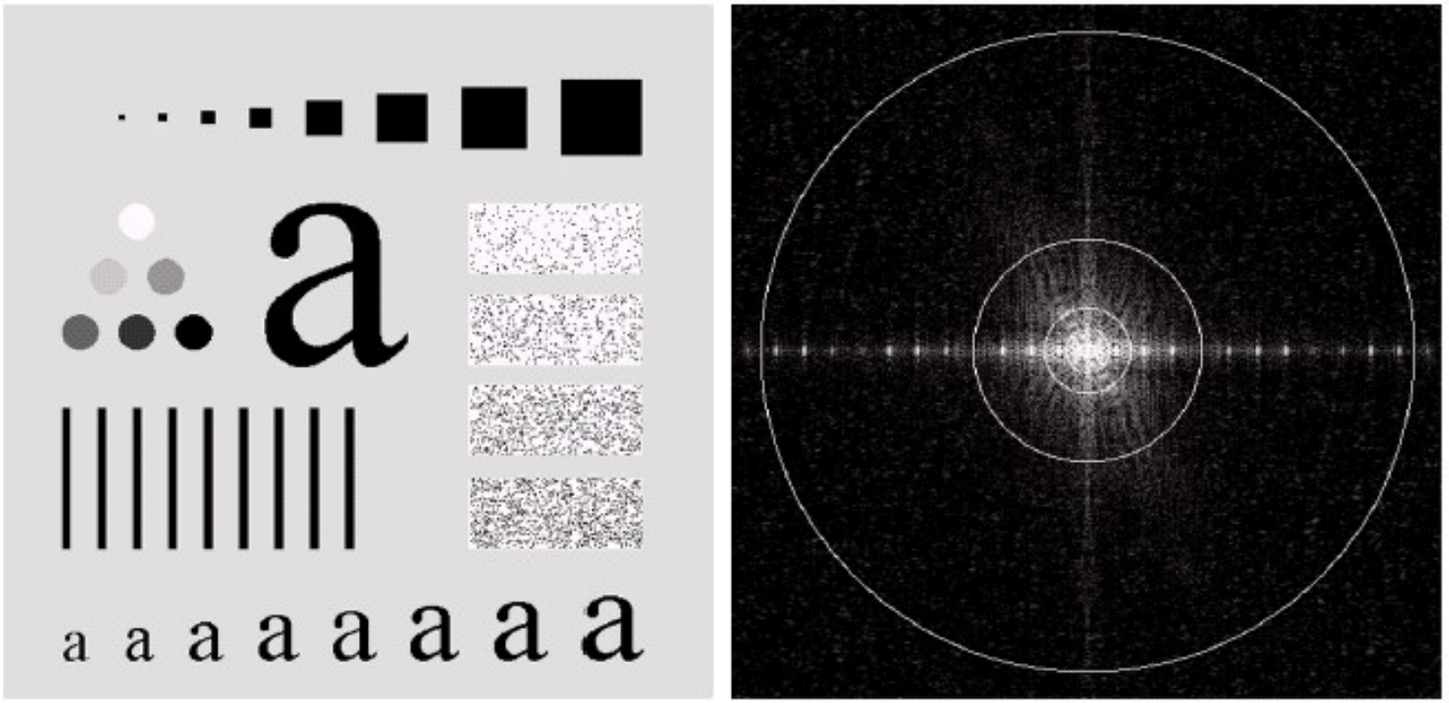




where D(u,v) : the distance from point (u,v) the center of their frequency rectangle

$$D(u,v) = \left[(u - M/2)^2 + (v - N/2)^2\right]^{\frac{1}{2}}$$

ILPF - example



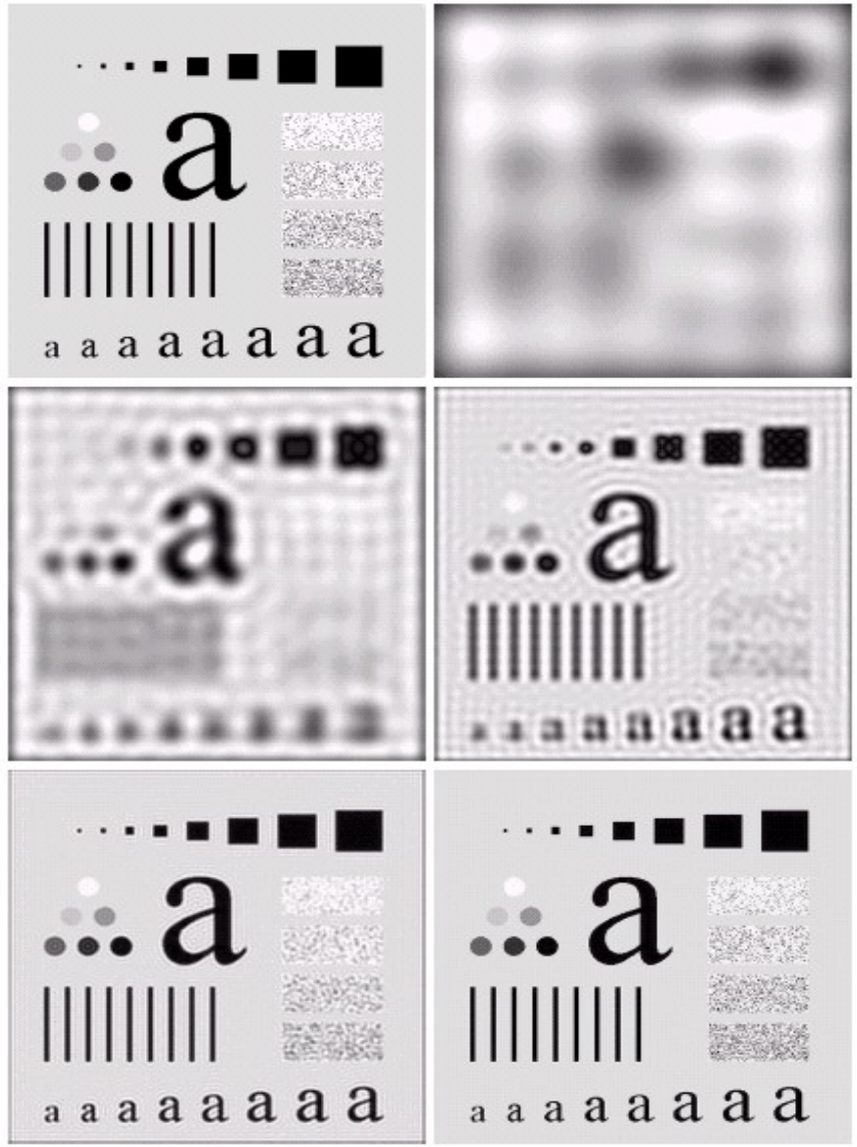
a b

FIGURE 4.11 (a) An image of size 500 × 500 pixels and (b) its Fourier spectrum. The superimposed circles have radii values of 5, 15, 30, 80, and 230, which enclose 92.0, 94.6, 96.4, 98.0, and 99.5% of the image power, respectively.

ILPF - example

- As the filter radius increases, less & less power is removed, resulting in less server blurring
- The ringing effect is a characteristic of ideal filters
- For α=99.95, only slight blurring in the noisy squares are shown
  - the little edge information is contained in the upper 0.5% of the spectrum power in this case

$$\alpha = 100\left[\sum_u \sum_v P(u,v) / P_T\right] \qquad P_T = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} P(u,v)$$



a b
c d
e f

FIGURE 4.12 (a) Original image. (b)–(f) Results of ideal lowpass filtering with cutoff frequencies set at radii values of 5, 15, 30, 80, and 230, as shown in Fig. 4.11(b). The power removed by these filters was 8, 5.4, 3.6, 2, and 0.5% of the total, respectively.

ILPF – ringing effect

- It has a central dominant circular component (providing the blurring)
- It has concentric circular components (rings) giving rise to the ringing effect

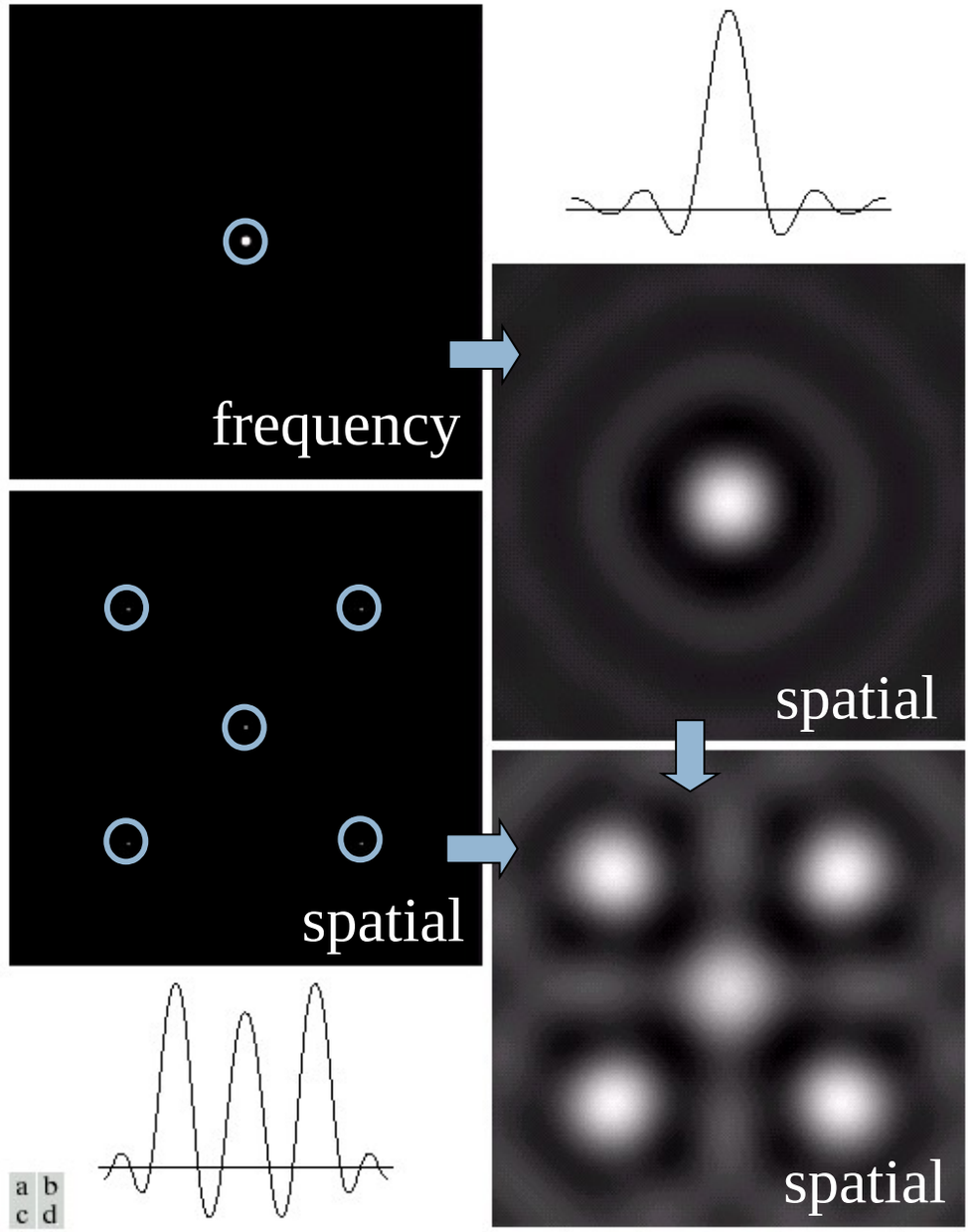


FIGURE 4.13 (a) A frequency-domain ILPF of radius 5. (b) Corresponding spatial filter (note the ringing). (c) Five impulses in the spatial domain, simulating the values of five pixels. (d) Convolution of (b) and (c) in the spatial domain.

Figure 4.13 (a) A frequency-domain ILPF of radius 5. (b) Corresponding spatial filter. (c) Five impulses in the spatial domain, simulating the values of five pixels. (d) Convolution of (b) and (c) in the spatial domain.

$$f(x,y) * h(x,y) \Leftrightarrow F(u,v)H(u,v)$$

Gaussian Lowpass Filters (GLPF)

- The form of these filters in 2-D

$$H(u,v) = e^{-D^2(u,v)/2\sigma^2}$$

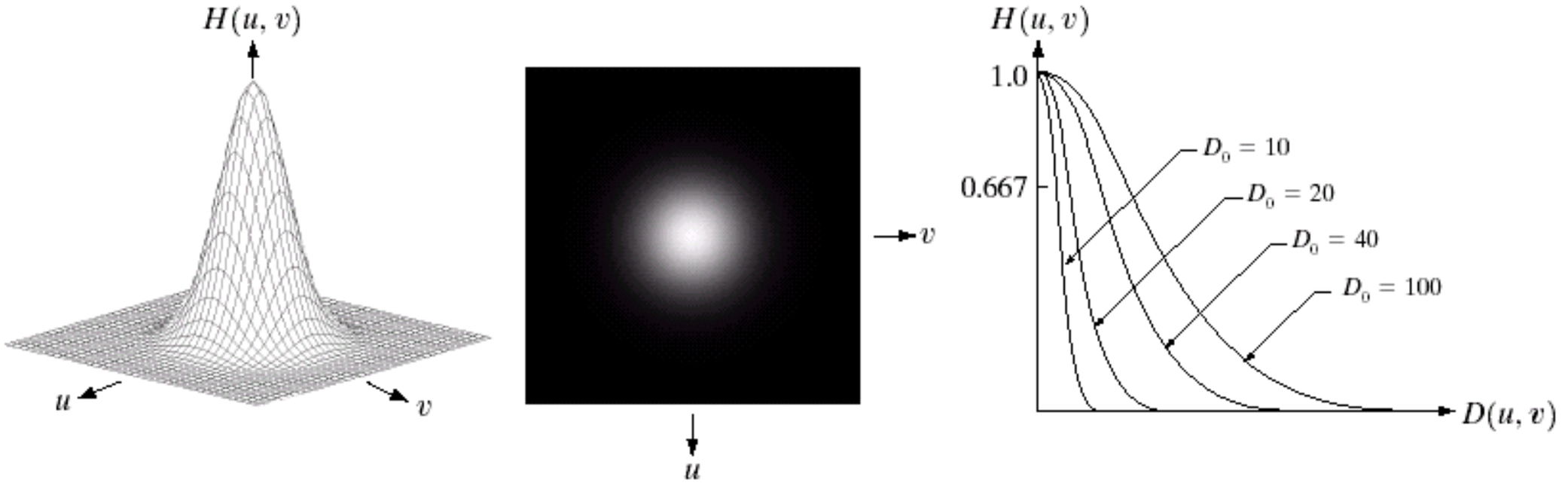
  - D(u,v): the distance from the origin of the Fourier transform
  - σ: a measure of the spread of the Gaussian curve
  - let $\sigma = D_0$

$$H(u,v) = e^{-D^2(u,v)/2D_0^2}$$

    - when $D(u,v) = D_0$, the filter is down to 0.607 of its maximum value

GLPF

- The inverse Fourier transform of the GLPF also is Gaussian
  - This makes Gaussian smoothing mathematically stable and practical
- Gaussian filters attenuate high frequencies smoothly.
- no abrupt cutoff and Guarantees no ringing



GLPF - example

- No ringing
- The GLPF did not achieve as much smoothing as the BLPF of order 2 for the same value of cutoff frequency
- BLPF is a more suitable choice if: tight control of the transition between low & high frequencies about the cutoff frequency are needed
  - the price: ringing

Small cutoff = strong blur. Large cutoff = mild blur.

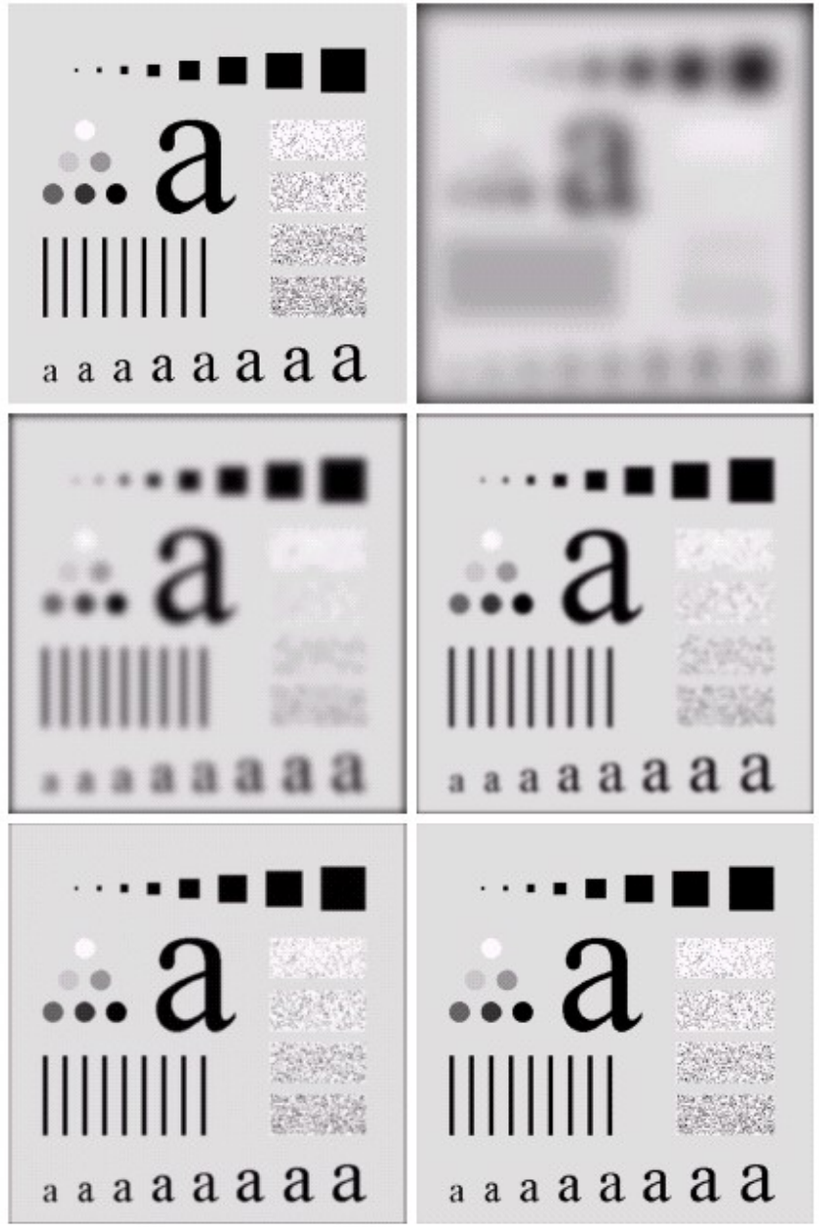


FIGURE 4.18 (a) Original image. (b)–(f) Results of filtering with Gaussian lowpass filters with cutoff frequencies set at radii values of 5, 15, 30, 80, and 230, as shown in Fig. 4.11(b). Compare with Figs. 4.12 and 4.15.

D_0=5,15,30,80,and 230

Correspondence of Gaussian Fn

Let $H(u)$ denote a frequency domain, Gaussian filter function given the equation

$$H(u) = Ae^{-u^2/2\sigma^2}$$

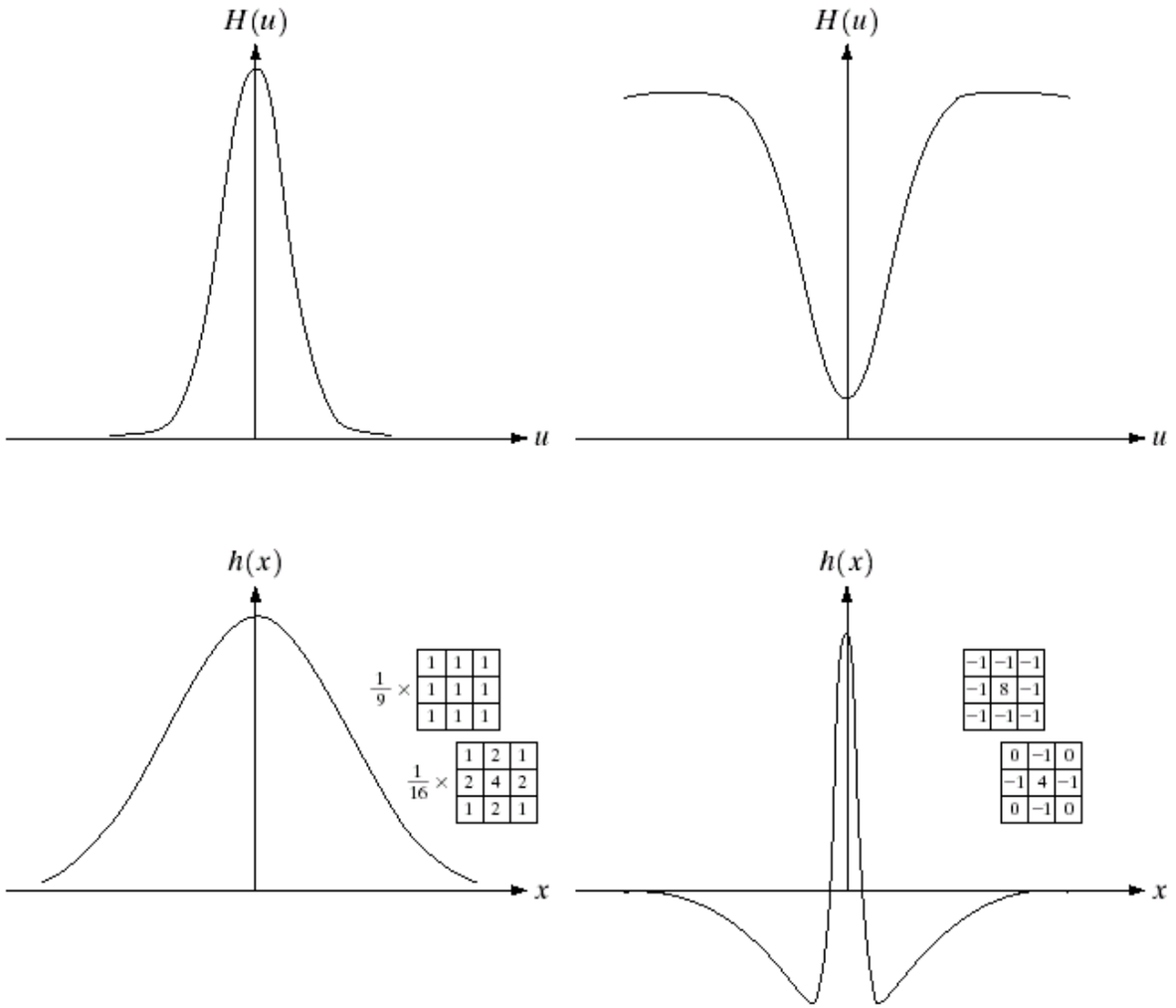
where σ : the standard deviation of the Gaussian curve.

The corresponding filter in the spatial domain is

$$h(x) = \sqrt{2\pi}\sigma Ae^{-2\pi^2\sigma^2x^2}$$

Note: Both the forward and inverse Fourier transforms of a Gaussian function are real Gaussian functions.

Correspondence of Gaussian Fn



a b
c d

FIGURE 4.9
(a) Gaussian frequency domain lowpass filter.
(b) Gaussian frequency domain highpass filter.
(c) Corresponding lowpass spatial filter.
(d) Corresponding highpass spatial filter. The masks shown are used in Chapter 3 for lowpass and highpass filtering.

Butterworth Low pass Filters (BLPFs) With order *n*

- Filter function

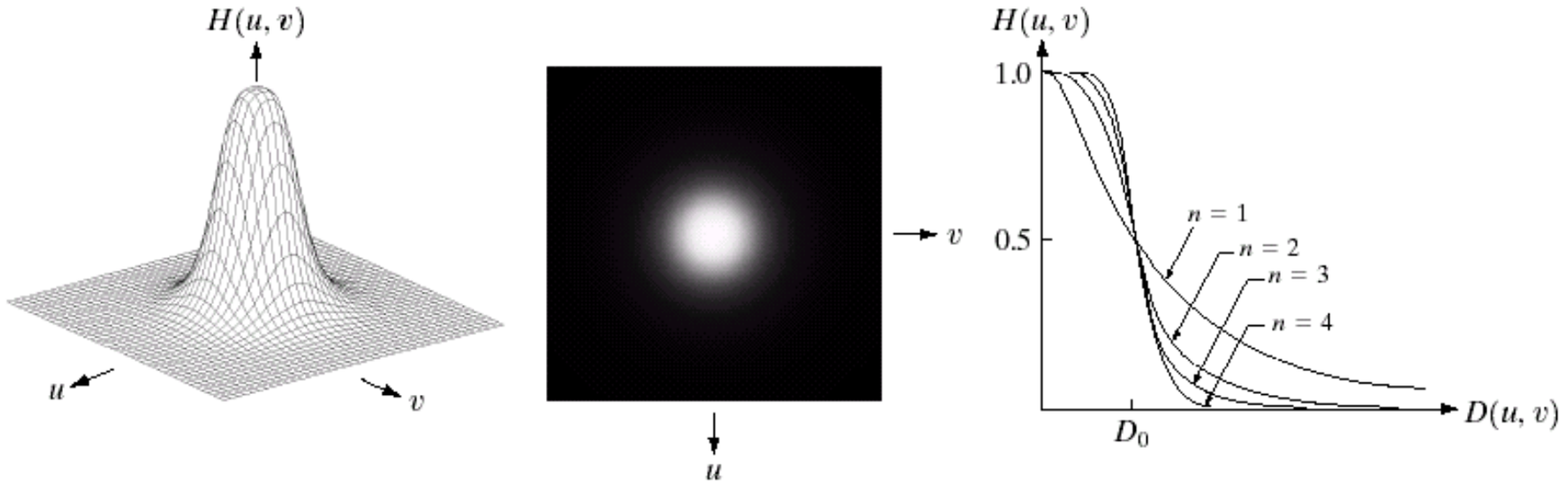
$$H(u,v) = \frac{1}{1 + \left[D(u,v) / D_0 \right]^{2n}}$$

- Butterworth filters provide a controlled transition.
- Order n controls how sharp the cutoff is.
  - for high values of “filter order” *n* it approaches the form of the ideal lowpass filter
  - For low n, it has a smooth form similar to the Gaussian filter

Butterworth is a useful compromise between ideal and Gaussian filters.

BLPF

- BLPF does not have a sharp discontinuity that establishes a clear cutoff between passed and filtered frequencies
- H(u,v)=0.5 (down 50% from its maximum value of 1) when D(u,v)=D0



a b c

FIGURE 4.14 (a) Perspective plot of a Butterworth lowpass filter transfer function. (b) Filter displayed as an image. (c) Filter radial cross sections of orders 1 through 4.

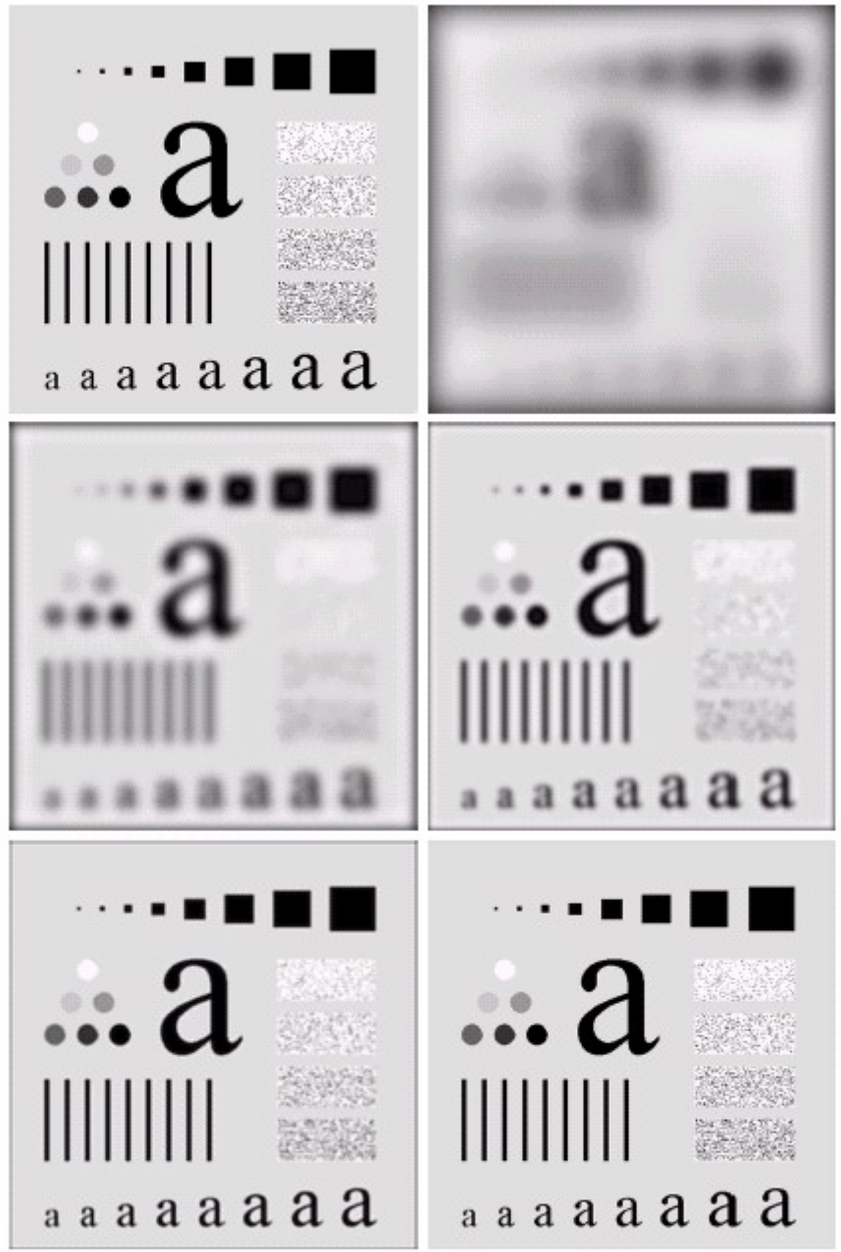
BLPF - example

- A smooth transition in blurring as a function of increasing cutoff frequency

- No ringing is visible due to the smooth transition between low & high frequencies

$$n=2$$

$$D_0=5,15,30,80,\text{and } 230$$



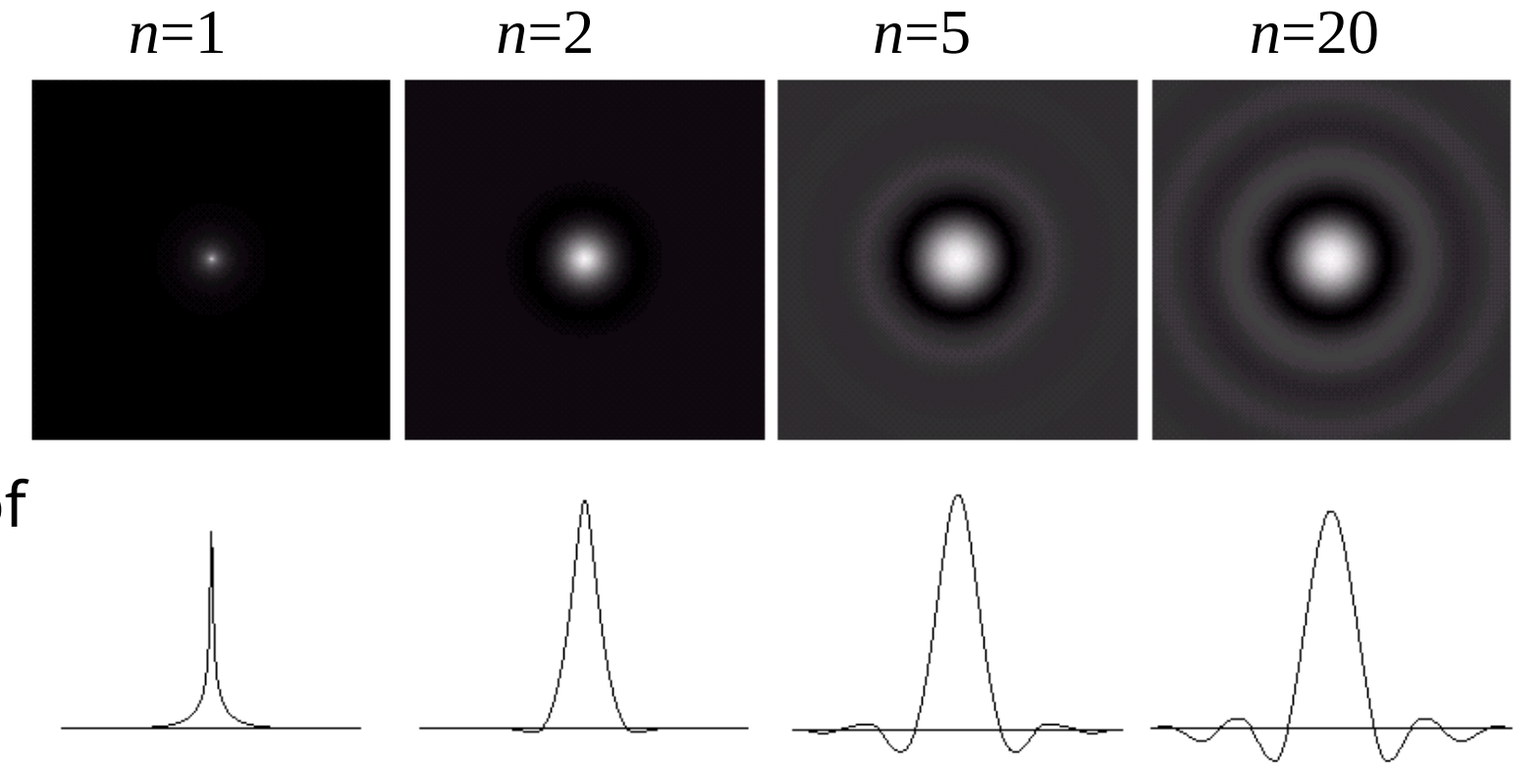
a b
c d
e f

FIGURE 4.15 (a) Original image. (b)–(f) Results of filtering with BLPFs of order 2, with cutoff frequencies at radii of 5, 15, 30, 80, and 230, as shown in Fig. 4.11(b). Compare with Fig. 4.12.

BLPF - Spatial Representation

- Ringing becomes a significant factor in filters of higher order
- Order 2: a good compromise between effective lowpass filtering & acceptable ringing characteristics

Spatial View of BLPF
Cutoff 15



a b c d

FIGURE 4.16 (a)–(d) Spatial representation of BLPFs of order 1, 2, 5, and 20, and corresponding gray-level profiles through the center of the filters (all filters have a cutoff frequency of 5). Note that ringing increases

GLPF vs BLPF

- The GLPF did not achieve as much smoothing as the BLPF of order 2 for the same value of cutoff frequency
- BLPF is a more suitable choice if: tight control of the transition between low & high frequencies about the cutoff frequency are needed
  - the price: ringing

BLPF

Low-pass smoothing trade-off

More smoothing removes more noise but loses detail.
Less smoothing preserves structure but leaves more noise.
Smooth transition reduces ringing but may blur more gradually.
Cutoff should be chosen based on the task, not only visual preference.

For analysis tasks, preserve diagnostically or semantically important edges.

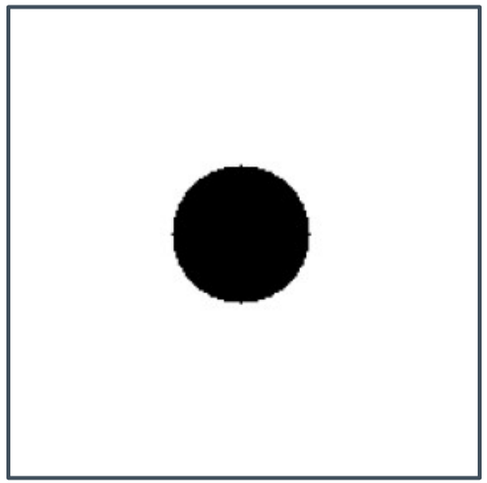
HIGH-PASS SHARPENING

Sharpening emphasizes rapid spatial changes such as edges and fine structures.

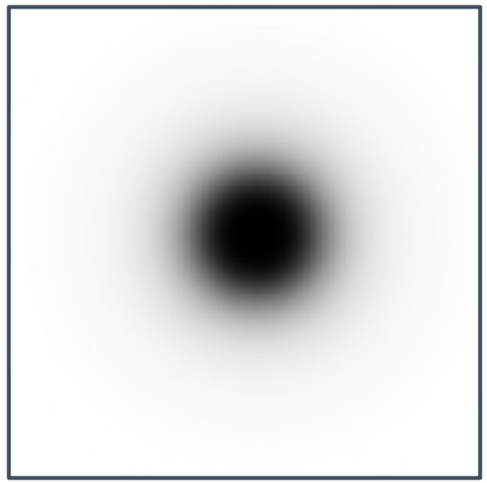
High-pass idea for images

High-pass filtering suppresses slow background variation.
It preserves or emphasizes rapid intensity changes.
Edges and texture become more visible.
Noise is also high-frequency, so sharpening must be controlled.

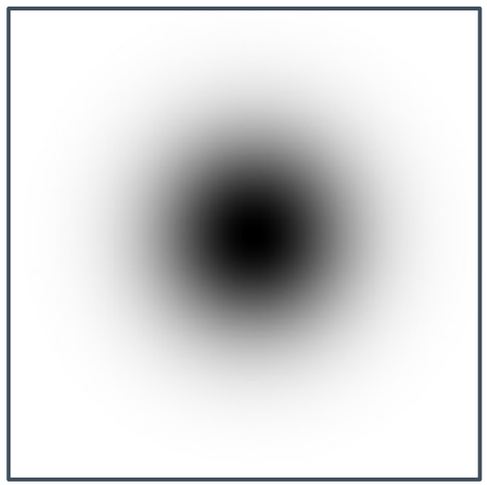
Ideal HP



Butterworth HP



Gaussian HP



Common high-pass filter equations

High-pass filters are often built from low-pass filters.
Ideal HP has a hard rejection near the center.
Gaussian HP transitions smoothly from center to outer frequencies.
Butterworth HP controls transition using order n.

$$H_{HP}(u,v) = 1 - H_{LP}(u,v)$$

This complement relation is used for Gaussian and ideal high-pass design.

Sharpening Frequency Domain

Ideal highpass filter

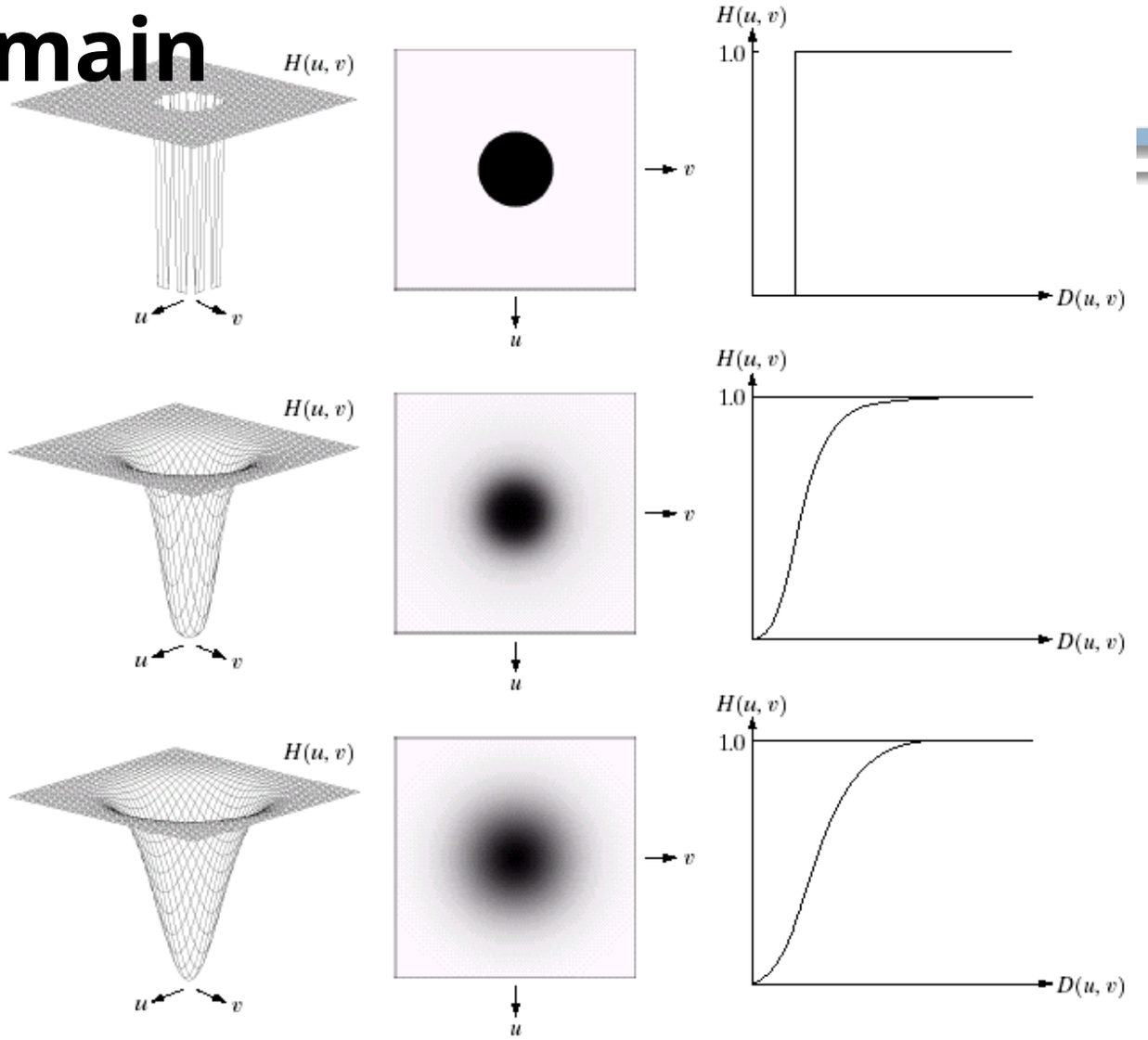
$$H(u,v) = \begin{cases} 0 & \text{if } D(u,v) \le D_0 \\ 1 & \text{if } D(u,v) > D_0 \end{cases}$$

Butterworth highpass filter

$$H(u,v) = \frac{1}{1 + \left[D_0 / D(u,v) \right]^{2n}}$$

Gaussian highpass filter

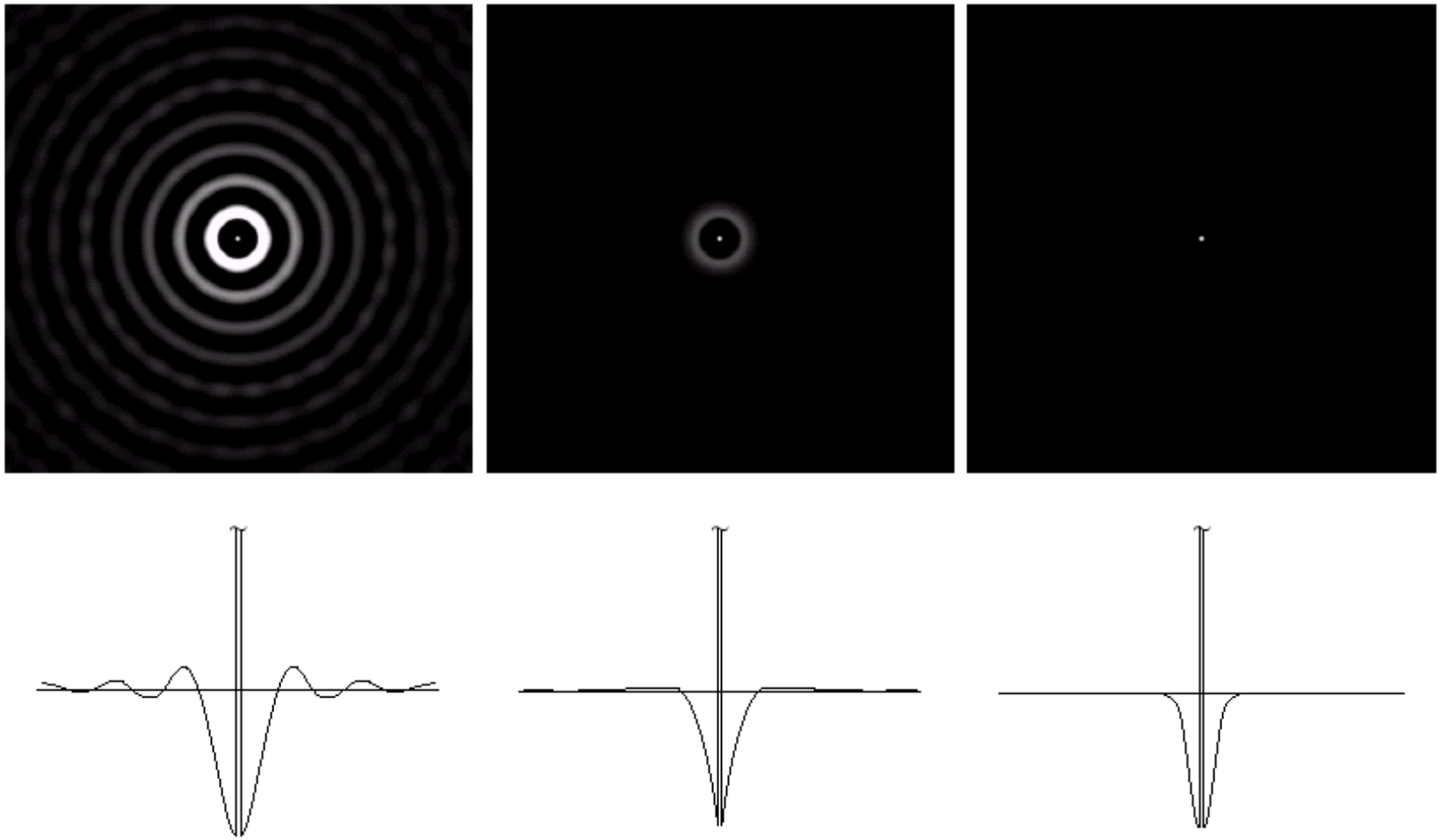
$$H(u,v) = 1 - e^{-D^2(u,v)/2D_0^2}$$



a b c
d e f
g h i

FIGURE 4.22 Top row: Perspective plot, image representation, and cross section of a typical ideal highpass filter. Middle and bottom rows: The same sequence for typical Butterworth and Gaussian highpass filters.

Highpass Filters
Spatial Representations

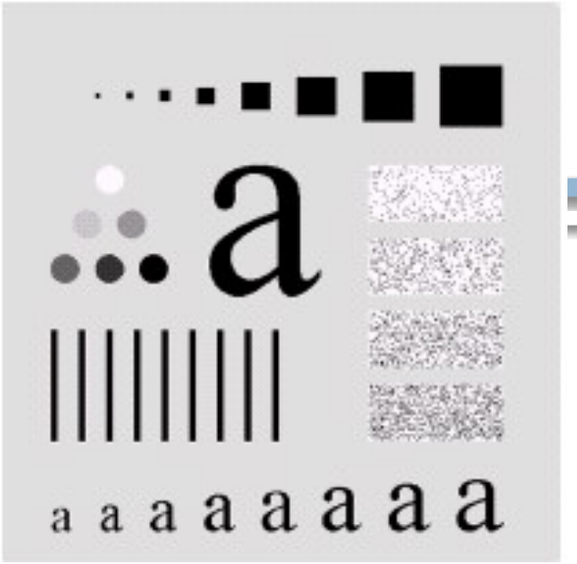


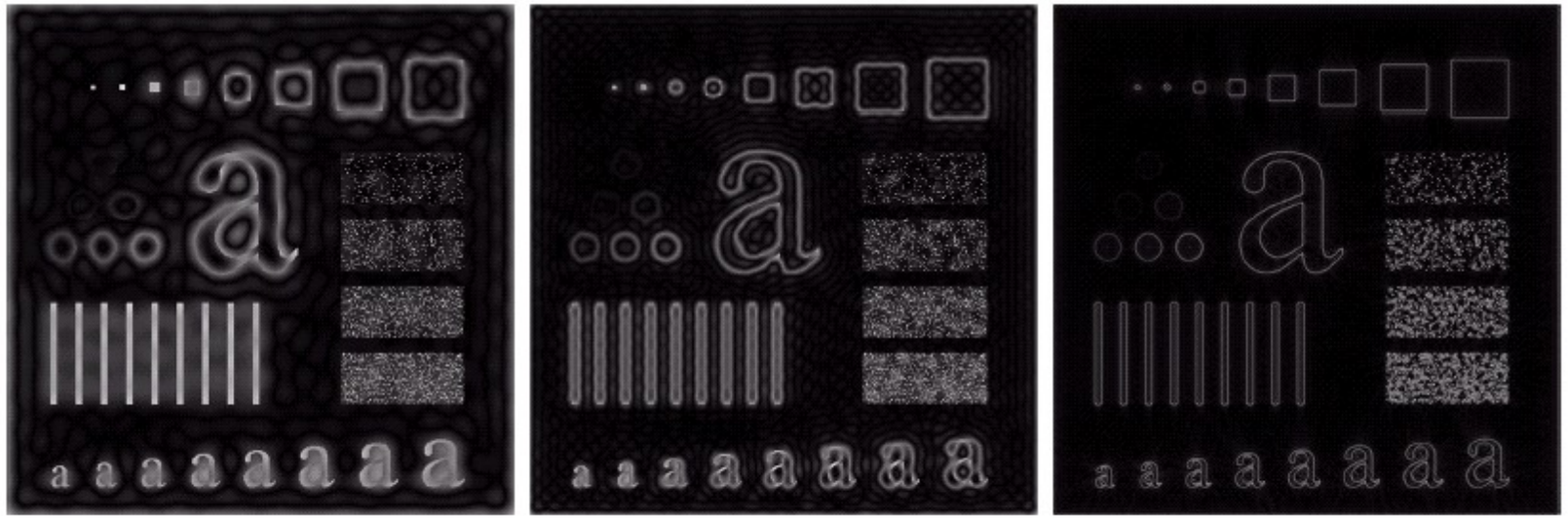
a b c

FIGURE 4.23 Spatial representations of typical (a) ideal, (b) Butterworth, and (c) Gaussian frequency domain highpass filters, and corresponding gray-level profiles.

Ideal Highpass Filters

$$H(u,v) = \begin{cases} 0 & \text{if } D(u,v) \le D_0 \\ 1 & \text{if } D(u,v) > D_0 \end{cases}$$



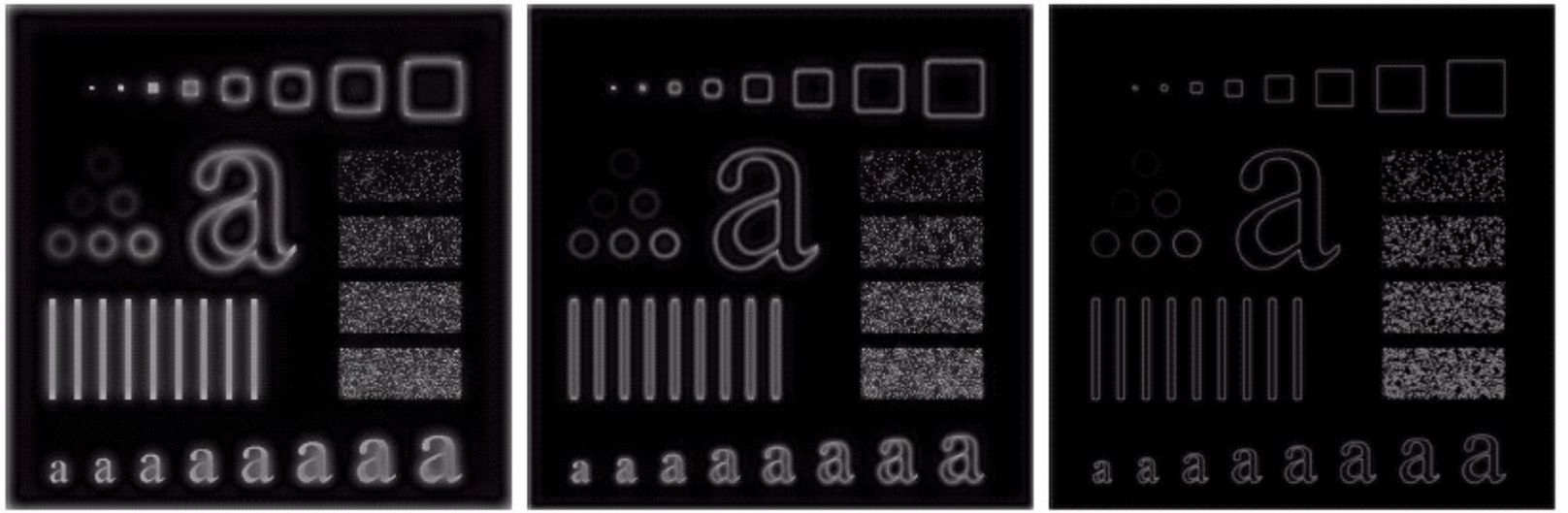


a b c

FIGURE 4.24 Results of ideal highpass filtering the image in Fig. 4.11(a) with $D_0 = 15$, 30, and 80, respectively. Problems with ringing are quite evident in (a) and (b).

Butterworth Highpass Filters

$$H(u,v) = \frac{1}{1 + \left[D_0 / D(u,v) \right]^{2n}}$$

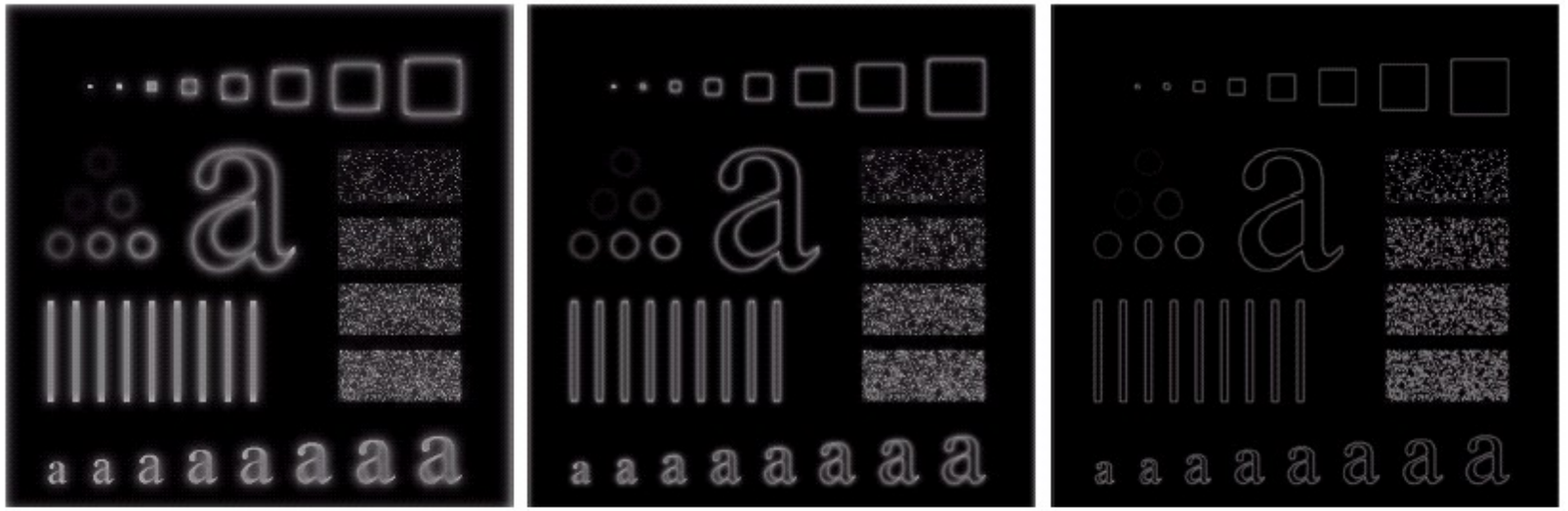


a b c

FIGURE 4.25 Results of highpass filtering the image in Fig. 4.11(a) using a BHPF of order 2 with $D_0 = 15$, 30, and 80, respectively. These results are much smoother than those obtained with an ILPF.

Gaussian Highpass Filters

$$H(u,v) = 1 - e^{-D^2(u,v)/2D_0^2}$$



a b c

FIGURE 4.26 Results of highpass filtering the image of Fig. 4.11(a) using a GHPF of order 2 with $D_0 = 15$, 30, and 80, respectively. Compare with Figs. 4.24 and 4.25.

Laplacian in the Frequency Domain

It was shown in the properties that

$$\Im\left[\frac{\partial^2 f(x,y)}{\partial x^2}+\frac{\partial^2 f(x,y)}{\partial y^2}\right]=(ju)^2F(u,v)+(jv)^2F(u,v)$$

$$=-(u^2+v^2)F(u,v).$$

The above equation shows that Laplacian in spatial domain is equal to applying the Fourier domain filter:

$$H(u,v)=-(u^2+v^2).$$

which is centered and given by:

$$H(u,v)=-[(u-M/2)^2+(v-N/2)^2].$$

therefore

$$\nabla^2 f(x,y)\Leftrightarrow -[(u-M/2)^2+(v-N/2)^2]F(u,v).$$

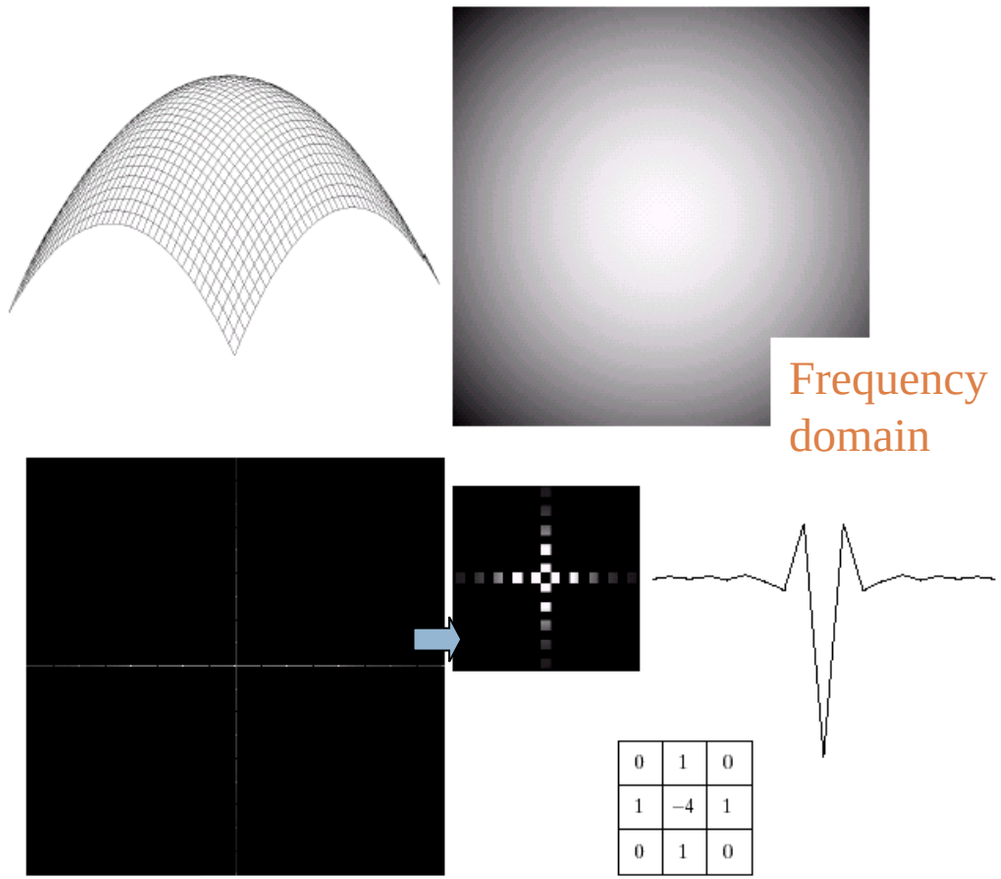
Laplacian in the Frequency Domain

- The Laplacian filter

$$H(u,v) = -(u^2 + v^2)$$

- Shift the center:

$$H(u,v) = -\left[(u - \frac{M}{2})^2 + (v - \frac{N}{2})^2\right]$$



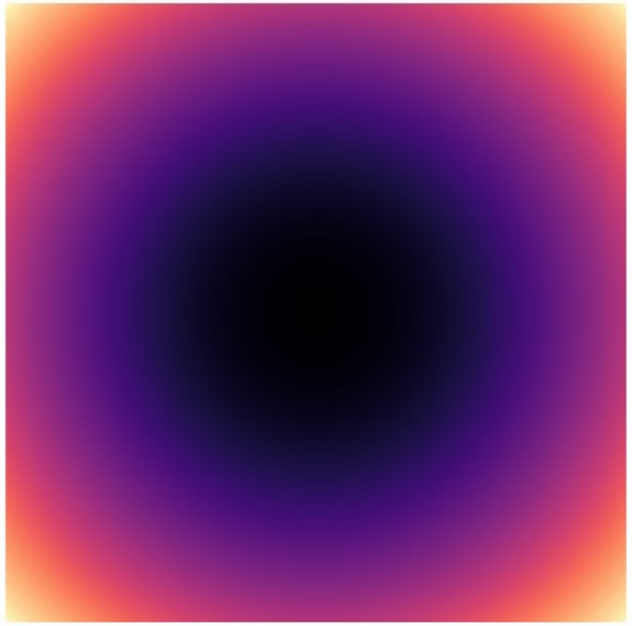
a b
c d e
f

FIGURE 4.27 (a) 3-D plot of Laplacian in the frequency domain. (b) Image representation of (a). (c) Laplacian in the spatial domain obtained from the inverse DFT of (b). (d) Zoomed section of the origin of (c). (e) Gray-level profile through the center of (d). (f) Laplacian mask used in Section 3.7.

Laplacian in the frequency domain

The response grows with squared frequency radius.
Edges become strong because they contain high-frequency components.
Noise can also become strong for the same reason.
Practical sharpening often combines smoothing and Laplacian

Laplacian response ~ D^2



original

frequency Laplacian detail

Homomorphic Filtering

Goal: Simultaneous brightness range compression and contrast enhancement

Illumination reflectance model:

$$f(x,y) = i(x,y).r(x,y)$$

$i(x,y) \Rightarrow$ Illumination, $r(x,y) \Rightarrow$ reflectance

Since Fourier transform is not distributive in multiplication

$$\Im[f(x,y)] \neq \Im[i(x,y)].\Im[r(x,y)]$$

However it is distributive in addition so we define:

$$z(x,y) = \ln[f(x,y)] = \ln[i(x,y)] + \ln[r(x,y)]$$

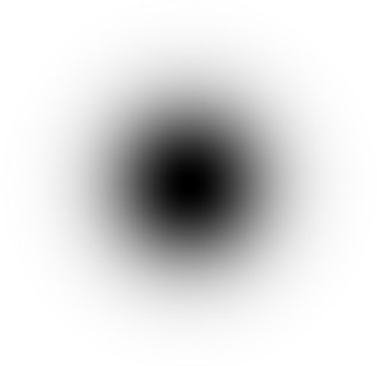
and $$\Im[z(x,y)] = \Im[\ln[f(x,y)]] = \Im[\ln[i(x,y)]] + \Im[\ln[r(x,y)]]$$

High-pass detail and sharpening

The high-pass result often looks like an edge/detail image.
Adding it back to the original image increases local contrast.
The gain controls sharpening strength.
Very strong gain produces halos and noise amplification.

original

Gaussian HP mask



high-frequency image

original + HP detail

Unsharp masking in frequency view

First create a blurred image using a low-pass filter.
Subtract it from the original to get a high-frequency mask.
Add a scaled mask back to the original image.

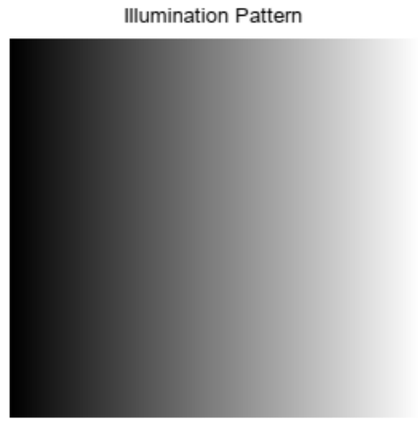
$$g = f + k(f - f_{LP}) = (1 + k)f - kf_{LP}$$

In frequency: $G = [1 + k(1 - H_{LP})]\,F.$

Increasing k strengthens edges but can create halos.

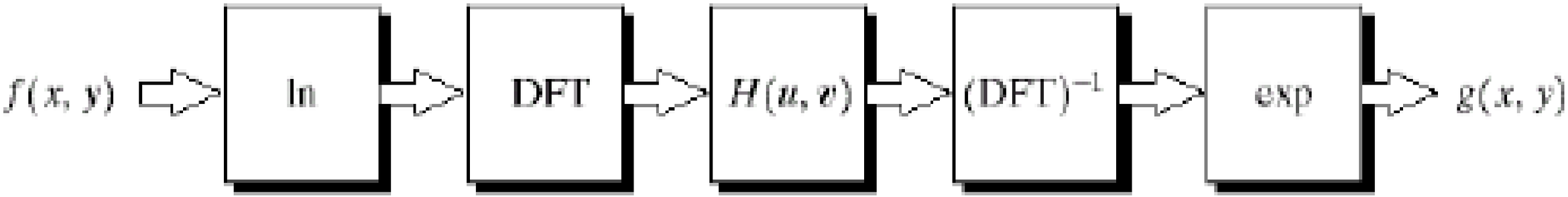
Homomorphic filtering

- Uneven illumination is often low-frequency multiplicative variation.
- Taking the log converts multiplication into addition.
- High-frequency emphasis can reduce illumination variation and enhance contrast.
- This is useful for microscopy, document images, and shaded scenes.



Homomorphic Filtering

The Homomorphic filtering steps are given as



Homomorphic Filtering

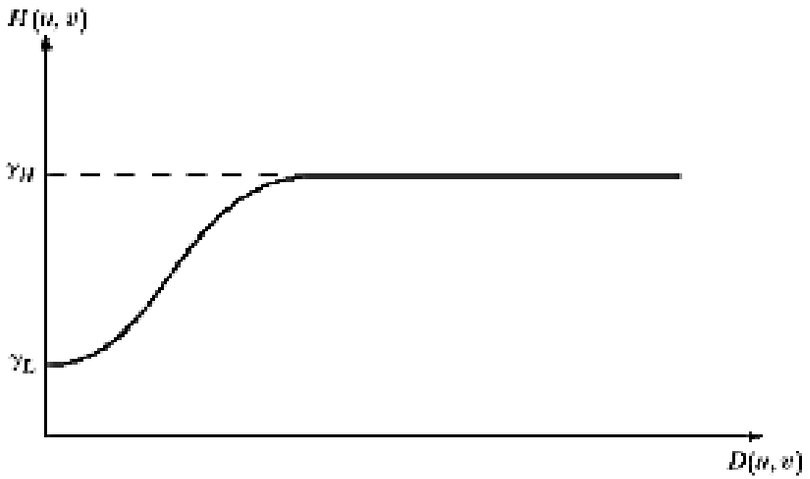


FIGURE 4.32 Cross section of a circularly symmetric filter function. $D(u, v)$ is the distance from the origin of the centered transform.

A filter example which suppresses low frequency components (illumination) and enhances high frequency component (reflectance)

$$H(u,v) = (\gamma_H - \gamma_L)\left[1 - \exp\left(-c\frac{D^2(u,v)}{D_0^2}\right)\right] + \gamma_L$$

- $D(u, v)$: Distance from frequency origin $(u, v)$to the center of the frequency spectrum.
- D_0: Cutoff frequency.
- c: Constant controlling the slope of the transition zone.
- $\gamma_L < 1$: Low-frequency gain; compresses dynamic range to reduce lighting variations.
- $\gamma_H > 1$: High-frequency gain; boosts edge detail and local contrast.

Homomorphic Filtering (Example)

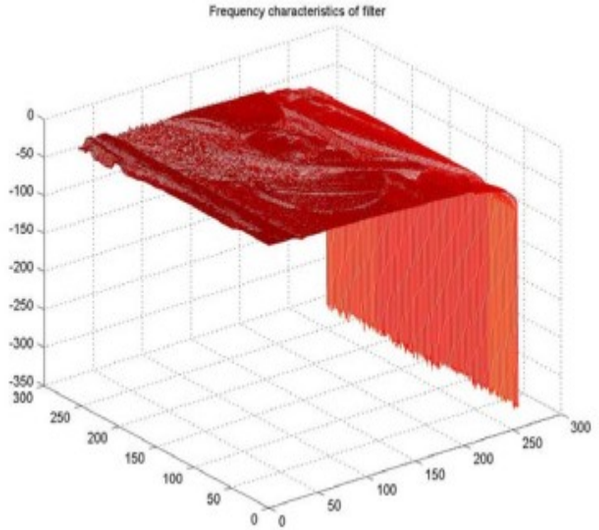
Illumination Pattern

Original Image

Corrupted Image

Image restored after histogram equalization

Frequency characteristics of filter



Restored Image

Summary

Frequency-domain filtering modifies selected 2D Fourier coefficients.
Convolution theorem explains why spatial kernels have frequency responses.
Low-pass filters smooth images by attenuating high frequencies.
High-pass and high-boost filters sharpen by emphasizing high frequencies.
Filter design must balance enhancement, ringing, boundary effects, and noise.

Main message: H(u,v) is simple to write, but its visual behavior depends on transition shape and image content.

Image Restoration

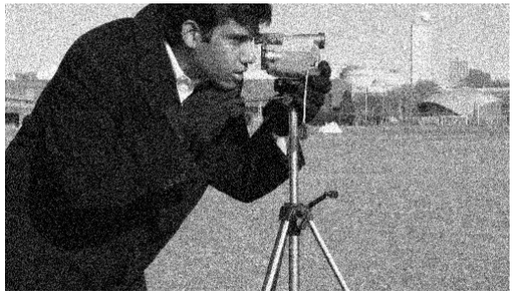
Prof. Dr. Sk. Md. Masudul Ahsan
Dept. of CSE, KUET

Learning outcomes

What students should be able to do after this lecture

- Explain degradation and restoration using a mathematical image model.
- Select suitable quantitative metrics for restored images.
- Identify common noise models and where they appear in real imaging.
- Apply spatial filters: mean, order-statistic, adaptive, and edge-preserving.
- Compare restoration methods based on noise type, edge preservation, and artifacts.

Original camera image

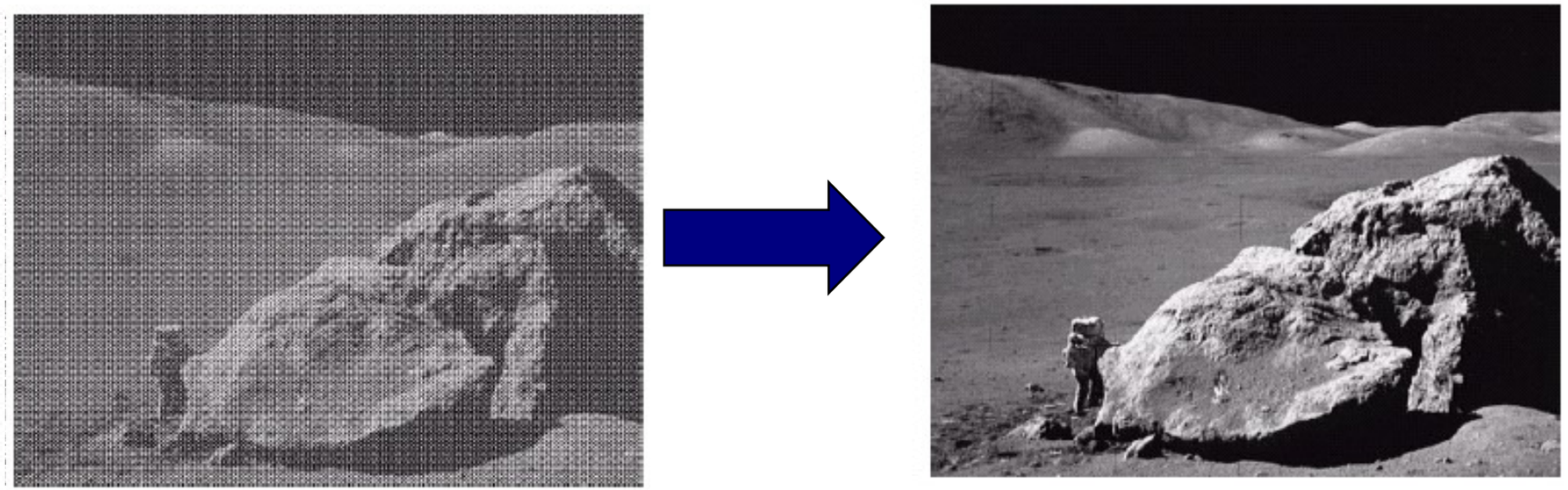


Noisy observation

Restoration = use prior knowledge of degradation to estimate the clean image f from observed image g.

Introduction

- Image restoration attempts to restore images that have been degraded
  - Identify the degradation process and attempt to reverse it
  - Similar to image enhancement, but more objective



Enhancement vs restoration

Image enhancement

Goal: make image visually pleasing or easier to inspect.
Often subjective: contrast stretching, sharpening, color adjustment.
No explicit physical degradation model is required.

Image restoration

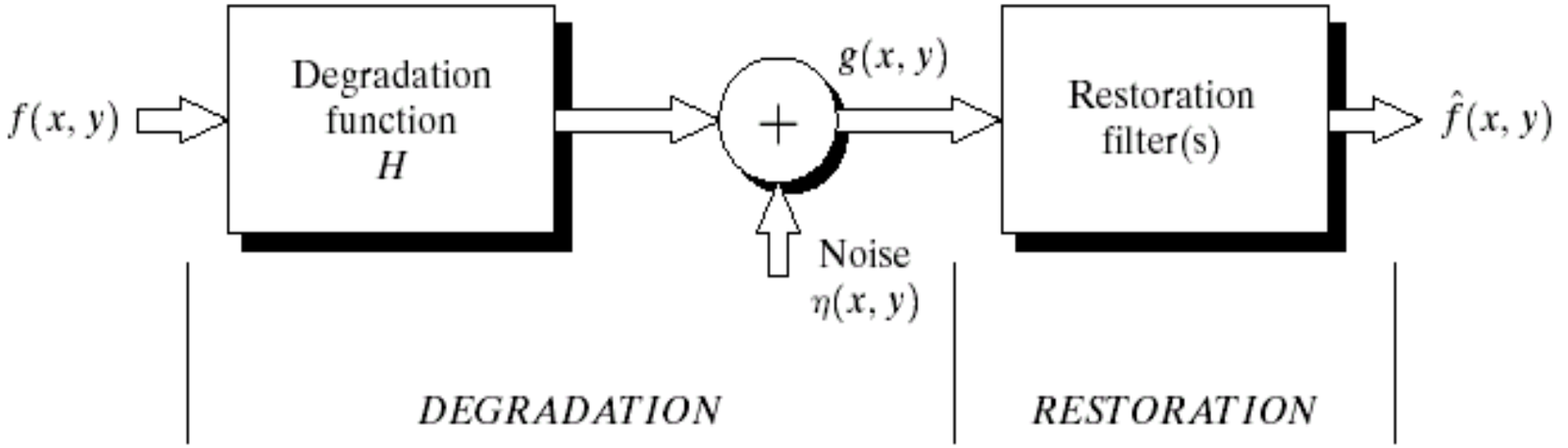
Goal: recover an estimate of the original scene.
Uses noise model, blur model, priors, or optimization.
Can be evaluated objectively if a reference image or task label exists.

Rule of thumb: enhancement asks “Does it look better?” Restoration asks “Is it closer to the original or more useful for the task?”

Common degradations in imaging systems

Cause	Typical PSF/noise model	Example effect
Motion	Line PSF	Directional streaking
Defocus	Disk or Gaussian PSF	Uniform soft blur
Atmosphere	Turbulence OTF	Low-pass fading
Sensor noise	Gaussian / Poisson	Grain, speckle, shot noise
Sampling/readout	Aliasing / periodic noise	Rings, stripes, Moiré

Image degradation / restoration model



Spatial domain

$$g(x,y)=f(x,y)*h(x,y)+\eta(x,y)$$

f: original image
h: point spread function (PSF)
η: additive noise
H: optical transfer function (OTF)

Frequency domain

$$G(u,v)=F(u,v)H(u,v)+N(u,v)$$

Restoration estimates F or f $(\hat{f})$ from G, H, and noise assumptions.

Image degradation / restoration model

$$g = H f + \eta$$

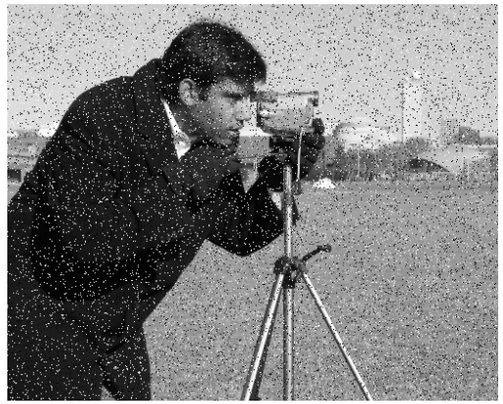
Vector/matrix form. f and g stack all image pixels; H represents blur, sampling, motion, defocus, or sensor response; η represents additive noise.

$$\hat{f} = R(g, H, prior, parameters)$$

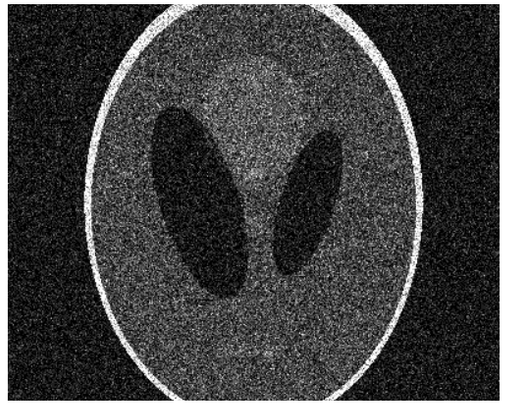
R is the restoration algorithm. It may use a filter window, noise variance, edge weights, or learned prior. f̂ is the restored estimate.

Motion blur

Periodic noise

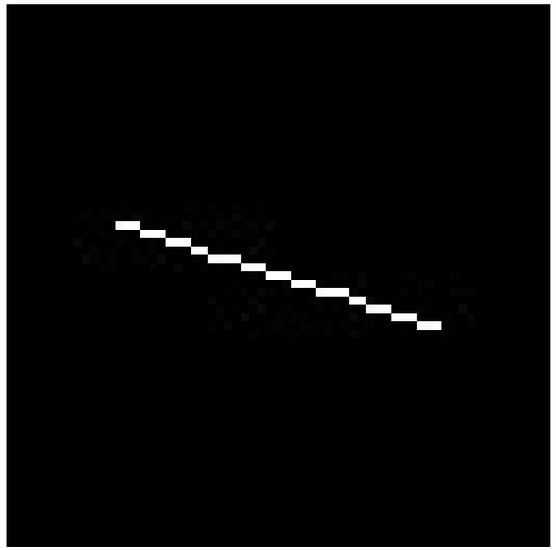


Impulse noise

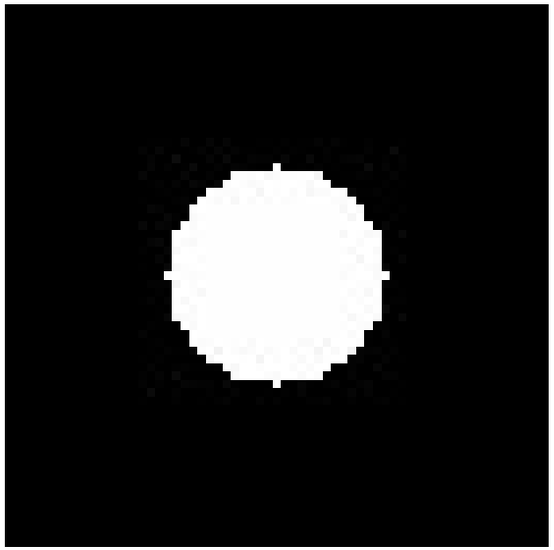


Photon/count noise

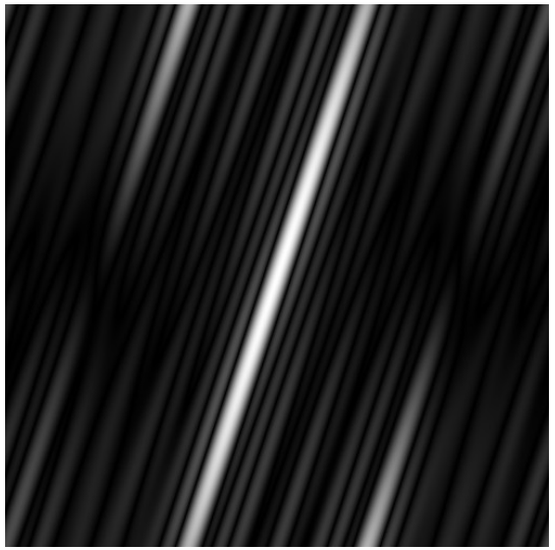
Point spread function: what a point becomes



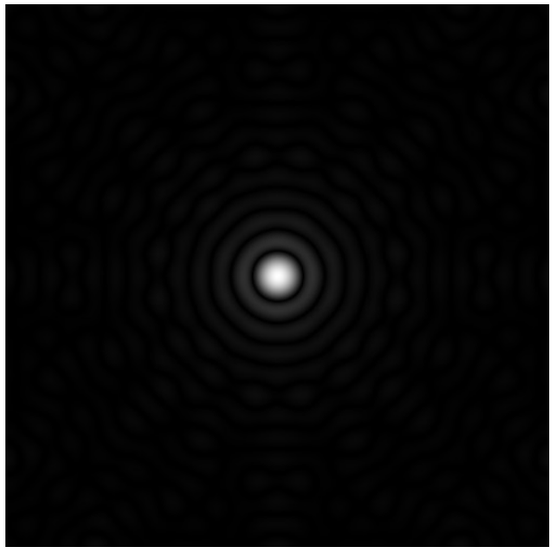
Motion PSF



Defocus PSF



Motion |H|



Defocus |H|

PSF h(x,y) describes blur in the spatial domain.
OTF H(u,v) describes attenuation and phase shift in the frequency domain.

Where restoration fits in an image processing pipeline

Acquisition → degradation/noise → restoration → analysis/vision task

Input: degraded image g(x,y), captured by camera or sensor.
Restoration stage estimates $\hat{f}(x,y)$, an approximation of the true image f(x,y).
Downstream task: segmentation, detection, recognition, measurement, registration, or visualization.

$$g(x,y) = h(x,y) * f(x,y) + \eta(x,y)$$

g: observed image; f: ideal unknown image; h: point spread function (blur/degradation); *: convolution; η: random noise.

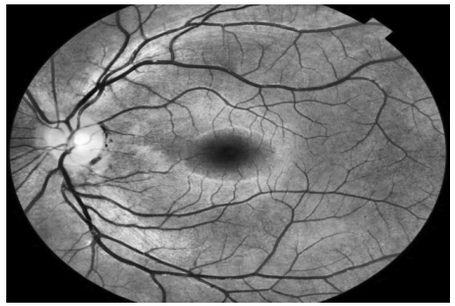
Degradation and restoration example

Evaluation metrics: full-reference case

When a clean reference image is available

$$MSE = \left(\frac{1}{MN}\right) \Sigma x \, \Sigma y \, [\, f(x,y) - \hat{f}(x,y) \,]^2$$

RMSE = √MSE



Clean reference f

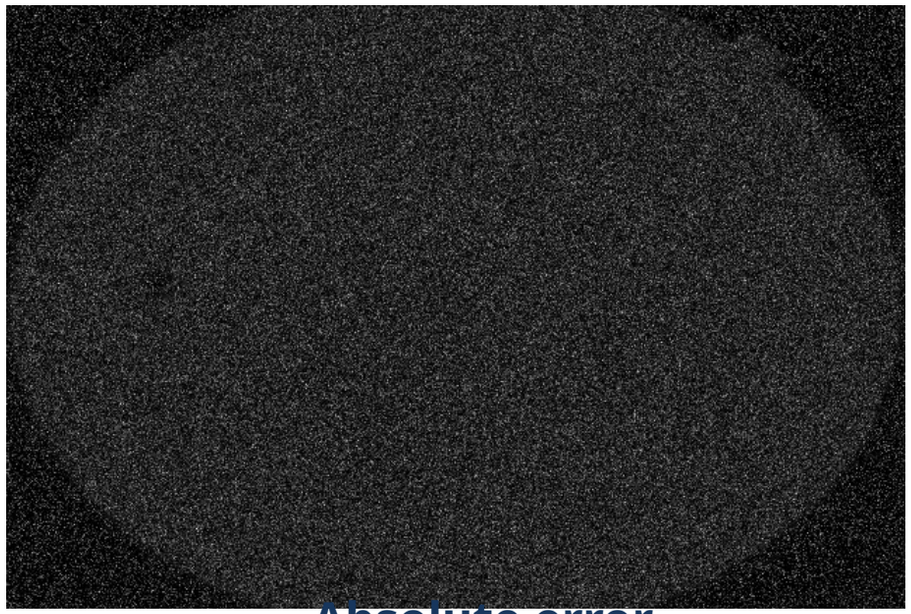
Noisy image

- Lower MSE/RMSE means closer pixel values.
- RMSE has the same intensity unit as image pixels.

Strength: simple, differentiable, common in optimization.

Weakness: treats all pixels equally and ignores structure/perception.

Very sensitive to outliers and small shifts.



Absolute error map

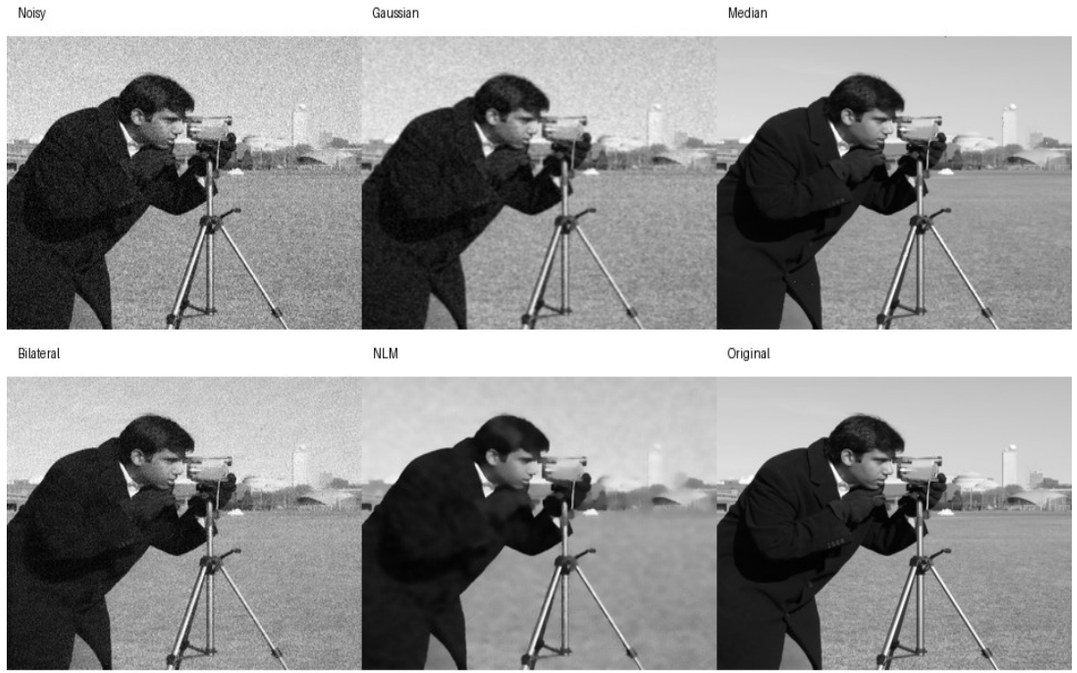
Peak Signal-to-Noise Ratio (PSNR)

$PSNR = 10 \log_{10}(L^2 / MSE) dB$

L: maximum possible pixel value. For 8-bit images L=255; for normalized images L=1. MSE: mean squared error.

Higher PSNR usually indicates less pixel-level distortion.
A 3 dB gain roughly means the MSE is halved.

- PSNR is easy to report, but it may disagree with human perception.



Different filters can have similar PSNR but different texture/edge quality

SSIM: Structural Similarity Index

$$\mathrm{SSIM}(f,\hat{f}) = \frac{\left(2\mu_f\mu_{\hat{f}} + C_1\right)\left(2\sigma_{f\hat{f}} + C_2\right)}{\left(\mu_f^2 + \mu_{\hat{f}}^2 + C_1\right)\left(\sigma_f^2 + \sigma_{\hat{f}}^2 + C_2\right)}$$

SSIM compares local luminance, contrast, and structure. It is usually computed in small windows and averaged over the image.

- Often aligns better with visual quality than MSE.
- Useful for comparing denoising strength at equal noise level.

- $\mu_f, \mu_{\hat{f}}$: local mean intensities → luminance similarity.
- $\sigma_f^2, \sigma_{\hat{f}}^2$: local variances → contrast similarity.
- $\sigma_{f\hat{f}}$: local covariance → structural agreement.
- C_1, C_2: small constants that avoid division by zero and stabilize flat regions.
- Range is commonly near -1 to 1, with 1 meaning perfect structural match.

SSIM emphasizes local structure

Noise models

- We first consider the degradation due to noise only
  - h is an impulse for now (H is a constant = 1)

- **Assuming that noise is**
  - independent of spatial coordinates, and
  - uncorrelated with respect to the image content

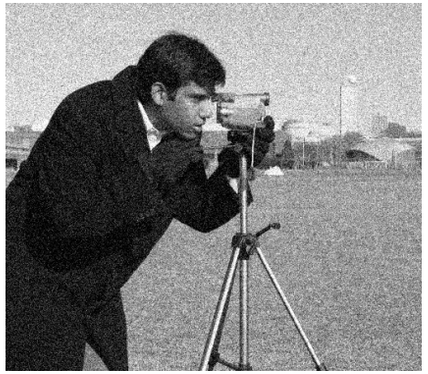
- Noise (image) can be classified according the distribution of the values of pixels (of the noise image) or its (normalized) histogram

Noise model: additive Gaussian noise

$$g(x,y) = f(x,y) + \eta(x,y), \quad \eta \sim N(0, \sigma^2)$$

η is independent random noise. σ^2 is noise variance: larger σ^2 means stronger grain.

Original



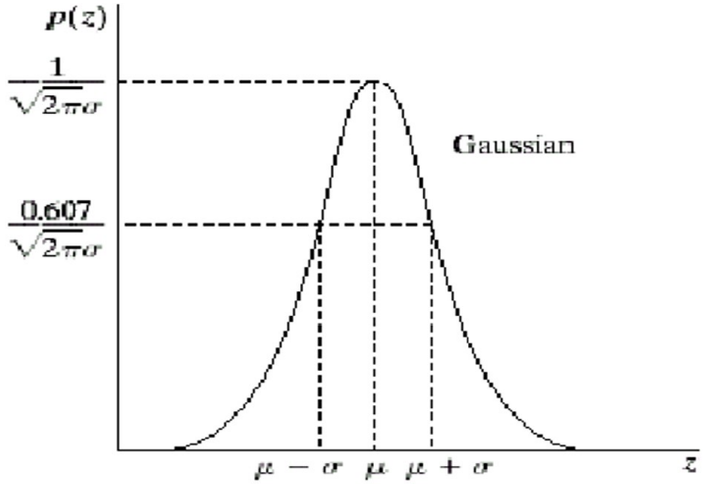
Gaussian noise

Common approximation for electronic sensor noise and many small random effects.

- Mean and Gaussian smoothing can reduce it but blur edges.

Gaussian restored

Mean μ shifts brightness; variance σ^2 controls noise strength.



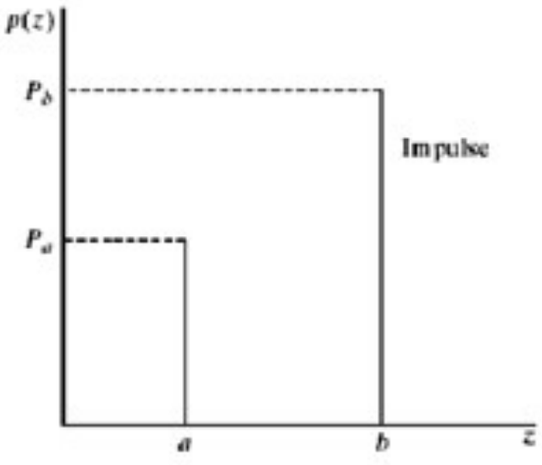
Noise model: salt-and-pepper impulse noise

P(g=a)=Pa, P(g=b)=Pb, otherwise g=f

a,b are low/high impulse values, often 0 and L. Pa and Pb are probabilities of pepper and salt pixels.

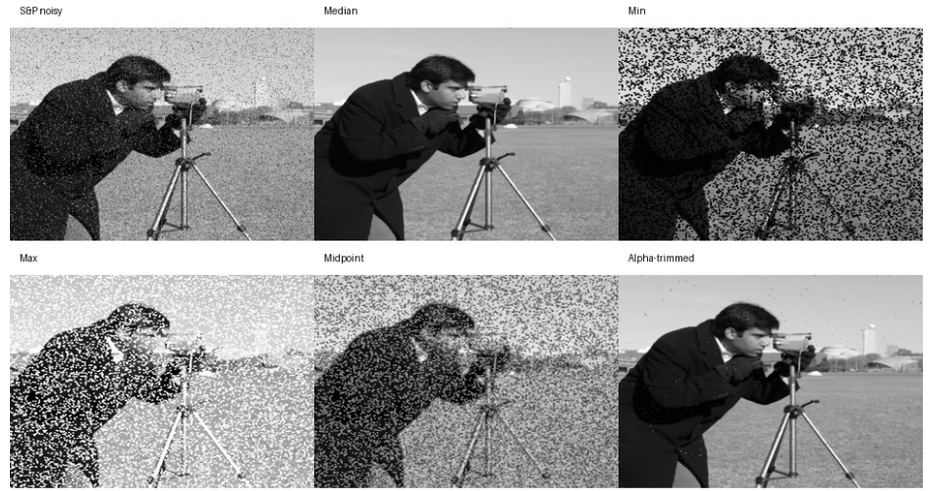
Appears as random black and white dots. Often caused by **transmission errors**, faulty sensor elements, or thresholding artifacts.

- Median and adaptive median filters are usually better than mean



Impulse noise

Median result



Order-statistic comparison

Noise model: Poisson / photon-counting noise

$$P(g = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

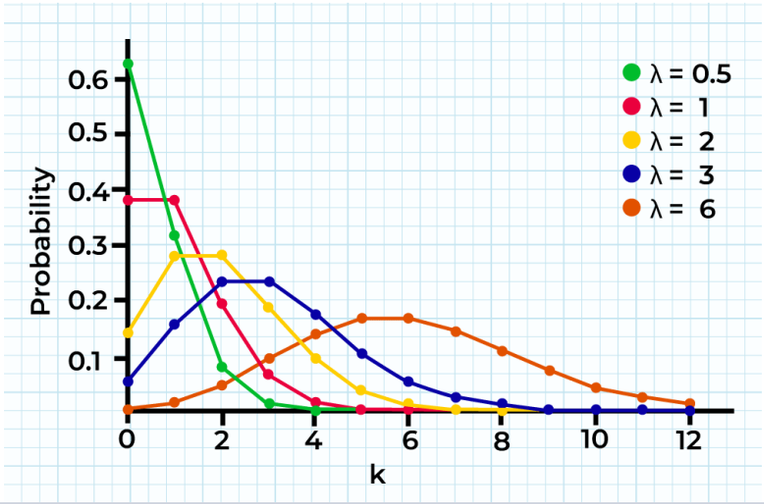
k: observed count. λ: expected count
mean = variance = λ

Phantom reference

Poisson noise

Important in low-light imaging, **microscopy**, astronomy, **CT, PET**, and photon-limited cameras. Noise strength depends on intensity: darker regions have poorer signal-to-noise ratio.

- Variance-stabilizing transforms or Poisson-aware methods can help.



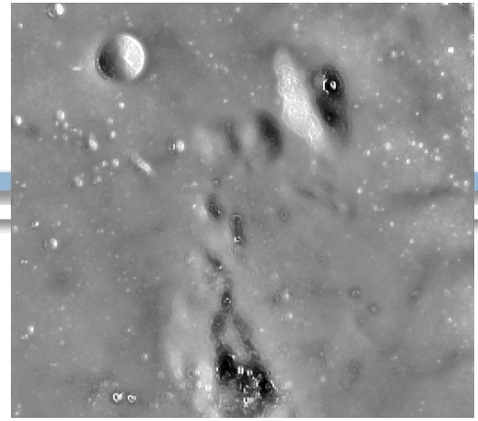
Photon-limited astronomy

Noise model: speckle / multiplicative noise

$$g(x,y) = f(x,y) + f(x,y)\eta(x,y) = f(x,y)[1 + \eta(x,y)]$$

η has mean near 1

Noise multiplies the signal. Bright regions often show stronger absolute fluctuations than dark regions.

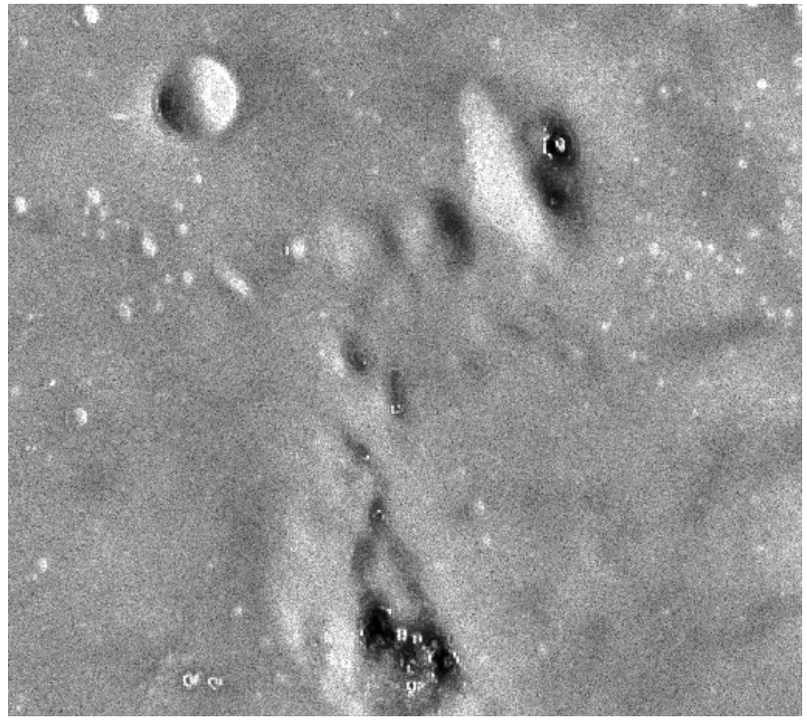


Remote/lunar texture

Appears in coherent imaging: ultrasound, SAR radar, laser imaging, some remote-sensing systems.
Smoothing reduces speckle but may remove fine texture and boundaries.

- Log transform can convert multiplicative noise approximately into additive noise.

$$\log(g) \approx \log(f) + \eta'$$



Speckle noise

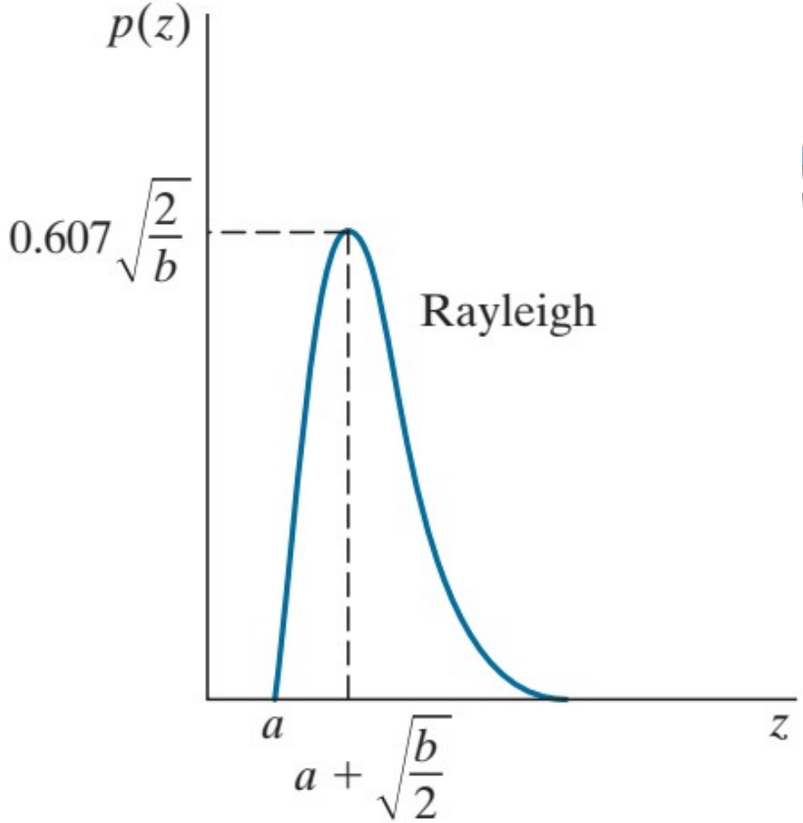
Other common noise models

- Rayleigh Noise
  - Approximate skewed histogram

$$p(z) = \frac{2}{b}(z - a)e^{-(z-a)^2/b} \quad \text{for} \quad z \geq a$$
$$= 0 \quad \text{for} \quad z < a$$

$$\text{mean} : \mu = a + \sqrt{\frac{\pi b}{4}}$$

$$\text{variance} : \sigma^2 = \frac{b(4-\pi)}{4}$$



- a type of non-symmetric, right-skewed random noise
- appears in range imaging and radar or laser-based image acquisitions

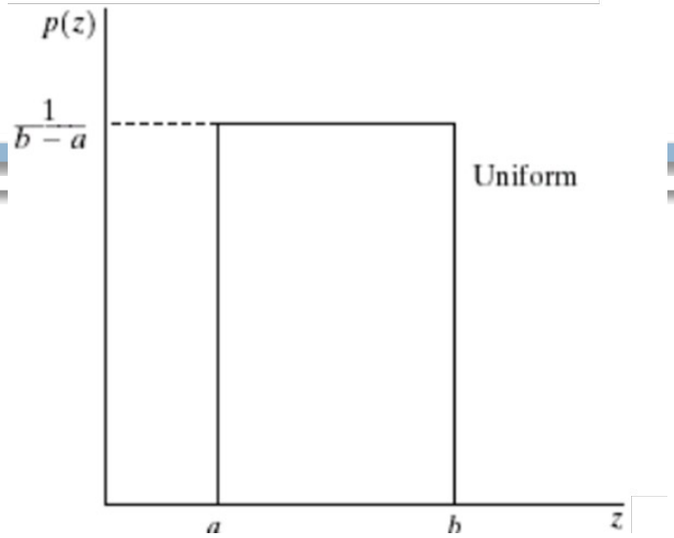
- contraharmonic mean filters or specific Lp mean filters help suppress Rayleigh noise

Other common noise models

- Uniform Nois

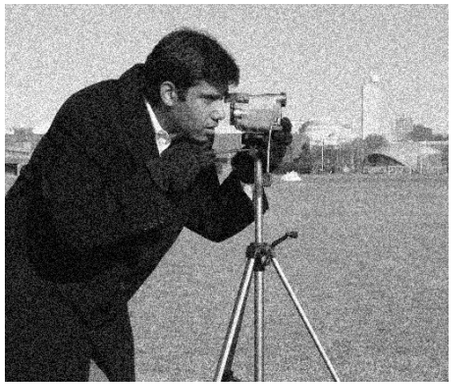
$$p(z) = \begin{cases} \dfrac{1}{b-a}, & \text{if } a \le z \le b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2}, \quad \sigma^2 = \frac{(b-a)^2}{12}$$



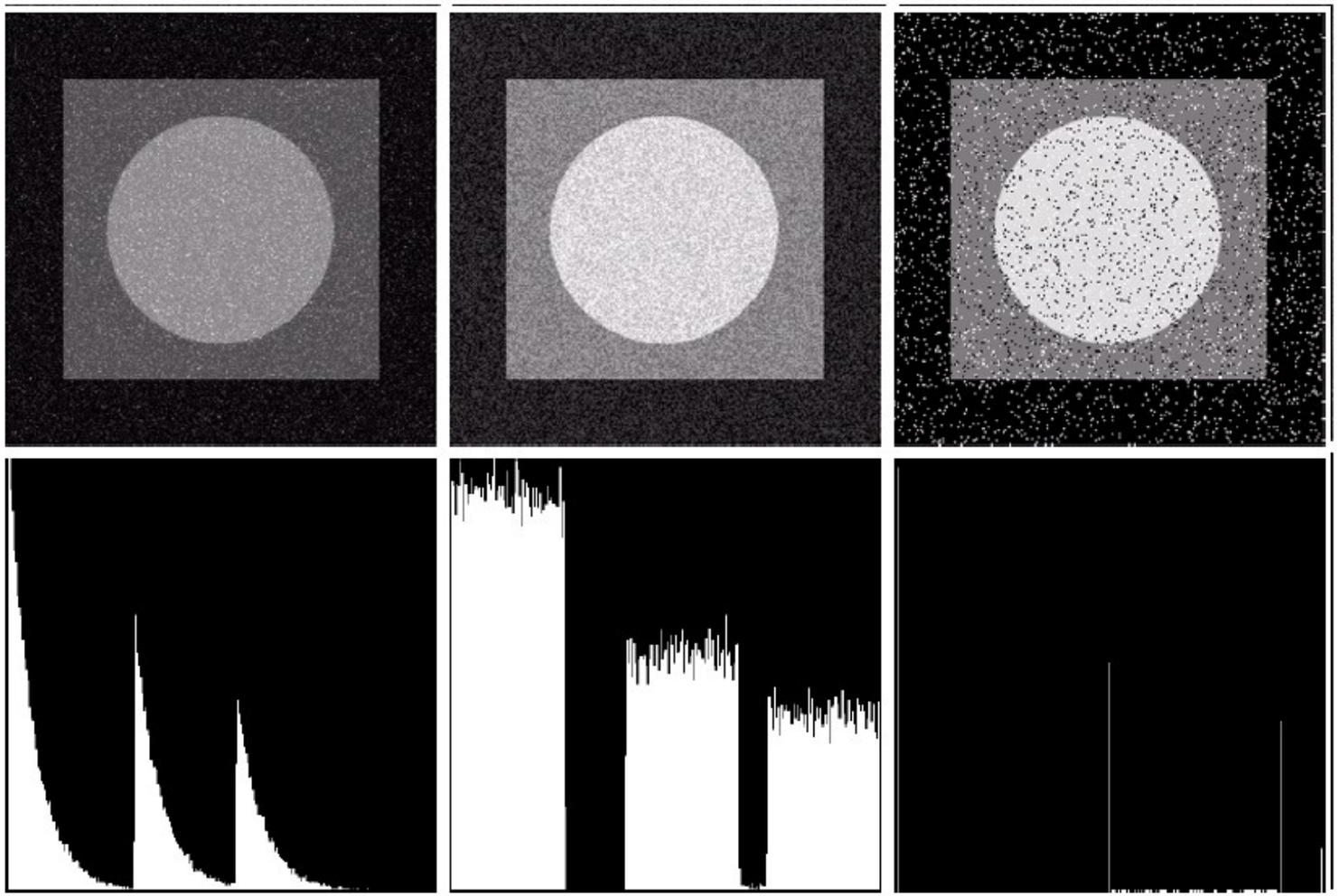
- random disturbance where pixel values are evenly spread across a specific range
- quantization errors when converting continuous analog signals into discrete digital pixel

- Basic linear smoothing filters like the mean filter (averaging filter) work well

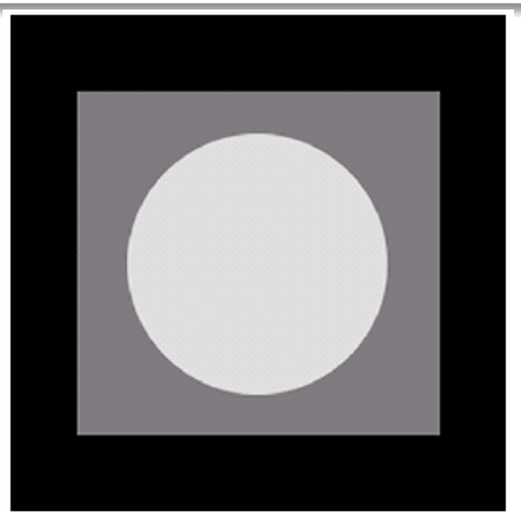


Uniform noise

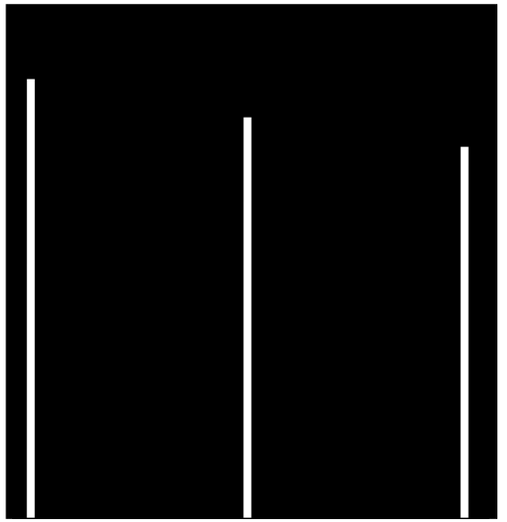
Noise Example



Exponential Uniform Salt & Pepper



Image



Histogram

Noise model: periodic interference

$$g(x,y)=f(x,y)+A\sin(2\pi(u_0x+v_0y))$$

A: interference amplitude; u_0,v_0: spatial frequencies. Periodic noise is often better treated in the frequency domain.

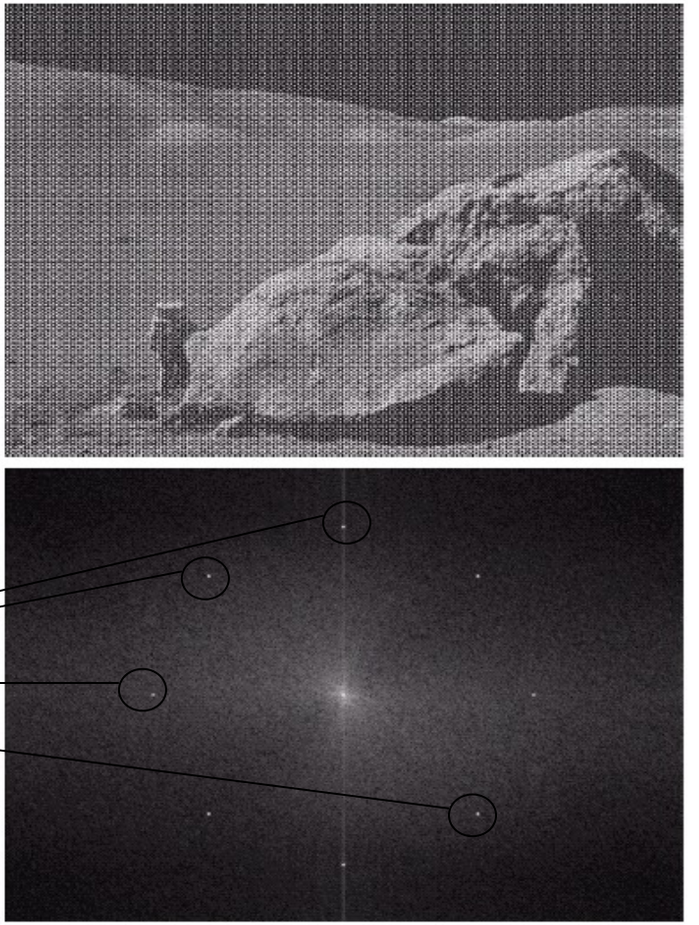
Periodic noise

This lecture focuses on spatial methods. Periodic noise will connect to next lecture

Periodic Noise

a
b

FIGURE 5.5
(a) Image corrupted by sinusoidal noise. (b) Spectrum (each pair of conjugate impulses corresponds to one sine wave). (Original image courtesy of NASA.)



Noise components

Periodic noise can be reduced in via frequency domain

Are generated due to electrical or electromechanical interference during image acquisition

Estimation of noise parameters

- Periodic noise: Fourier spectral components
- Imaging system is available: study the characteristics of system noise by acquiring a set of images of flat environment under uniform illumination (constant background)
- Only images are available: estimate the noise PDF from small patches of reasonably constant gray level

Estimation of noise parameters

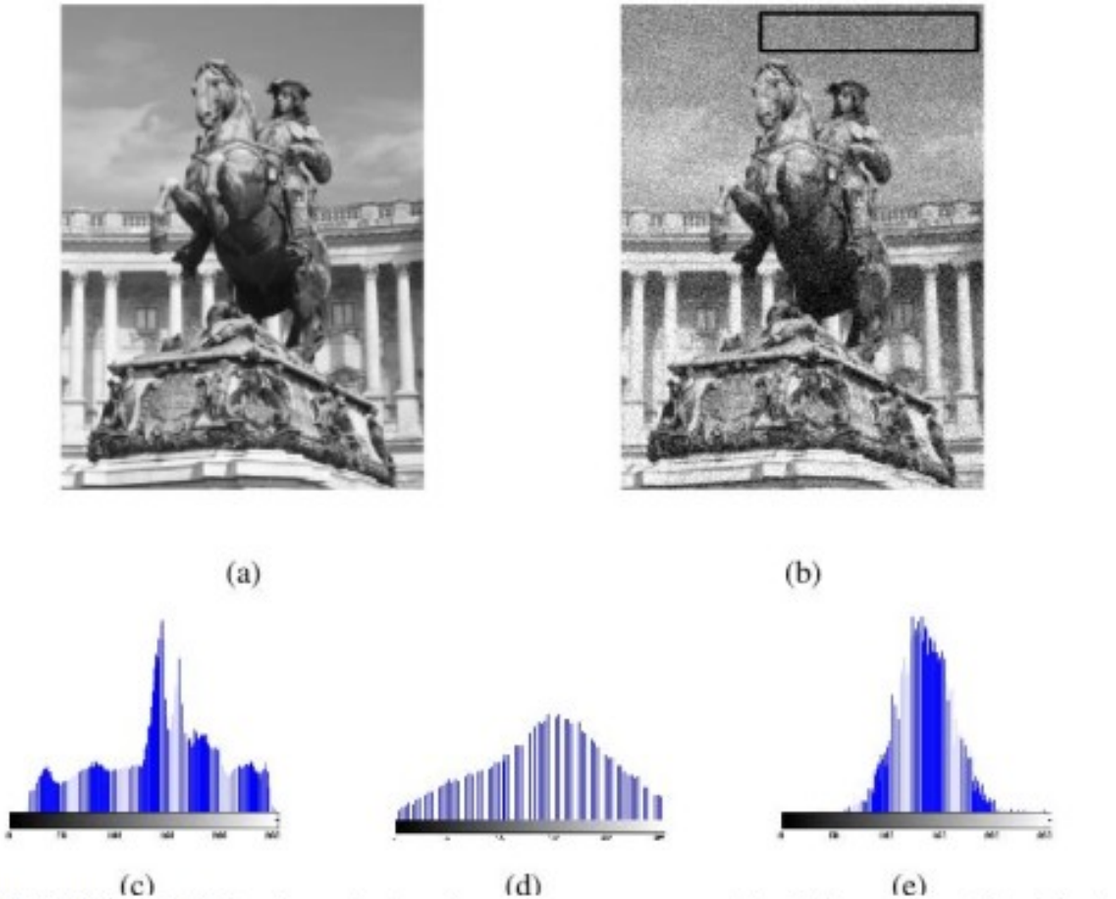
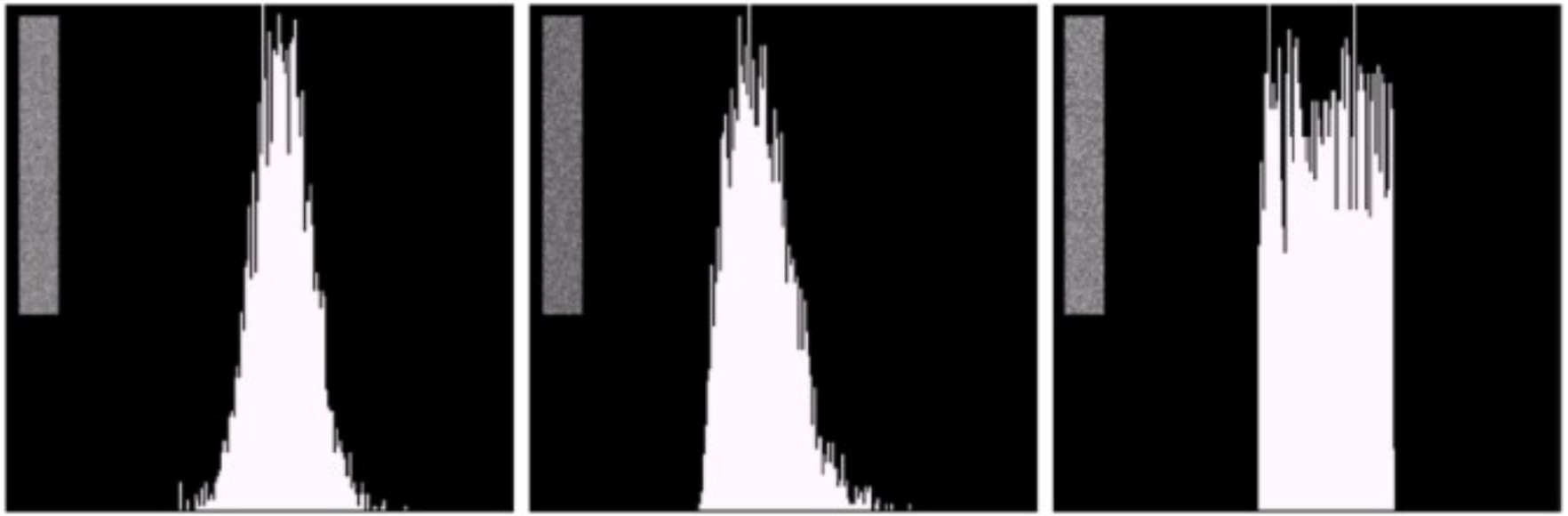


FIGURE 12.5 Estimating noise type from a homogeneous patch within an image: (a) original image; (b) noisy image (where the rectangle indicates a manually selected patch); (c) histogram of the original image; (d) histogram of the noisy image; (e) histogram of selected patch showing clearly that the noise is of Gaussian type in this case.

Estimation of noise parameters



a b c

FIGURE 5.6 Histograms computed using small strips (shown as inserts) from (a) the Gaussian, (b) the Rayleigh, and (c) the uniform noisy images in Fig. 5.4.

Estimation of noise parameters

- Once the PDF model is determined, estimate the model parameters (mean μ, variance σ^2) or (a,b)
- Estimate mean and variance → compute a and b

$$\mu = \sum_{z_i \in S} z_i p(z_i)$$

$$\sigma^2 = \sum_{z_i \in S} (z_i - \mu)^2 p(z_i)$$

Exercise

- Consider that a 3-bit image has the following histogram for a flat region.
  - What is the type of noise that is present in that region?
  - Provide an estimation of the parameters of the noise; that is, provide the parameters of the noise probability density function.

0	0	0	0.28	0.39	0.22	0.09	0.03

RESTORATION BY SPATIAL FILTERING

Spatial Filtering to Remove Noise

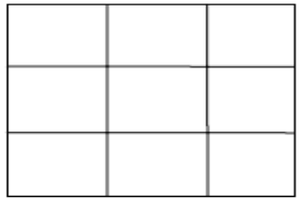
- spatial filters of different kinds used to remove different kinds of noise
- **Assume noise is the only degradation source**

$$g(x,y) = f(x,y) + \eta(x,y)$$

$$G(u,v) = F(u,v) + N(u,v)$$

Spatial filtering is appropriate when only additive noise is present

- **Spatial filtering**
  - a means when only additive noise is present
  - similar to enhancement in spatial domain



3×3 filter

Noise is unknown

- Various mean filters can be used to blur the image
  - Thereby, remove noise

Spatial Filtering to Remove Noise

- Various mean filters can be used to blur the image
  - Thereby, remove noise

Mean filters:

- Arithmetic mean filter
- Geometric mean filter
- Harmonic mean filter
- Contraharmonic mean filter

Order-statistics filters:

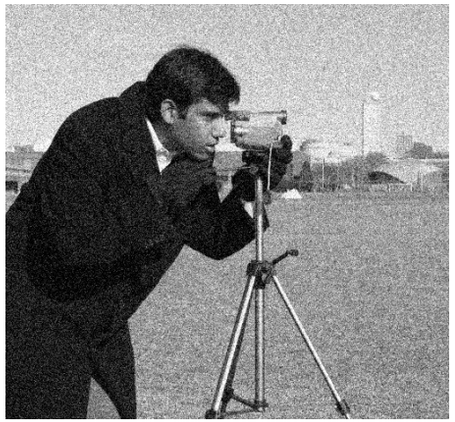
- Median filter
- Max and min filters
- Midpoint filter
- Alpha-trimmed mean filter

Mean Filters

- The ***Arithmetic Mean* Filter (AMF)** is a very simple one and is calculated as follows:

$$\hat{f}(x,y) = \frac{1}{mn} \sum_{(s,t)\in S_{xy}} g(s,t)$$

$^1/_9$	$^1/_9$	$^1/_9$
$^1/_9$	$^1/_9$	$^1/_9$
$^1/_9$	$^1/_9$	$^1/_9$

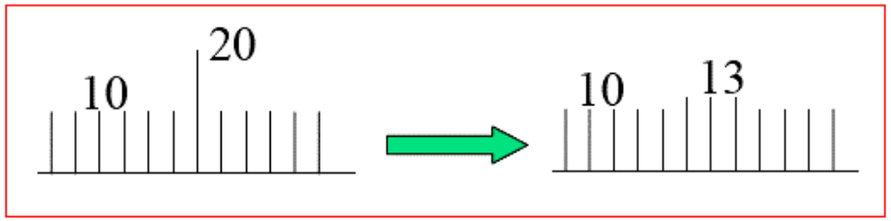


Noisy

Mean filter

S_{xy} is the set of coordinates in a rectangular $m \times n$ window centered at (x,y)

1 x 3 mask



1D illustration

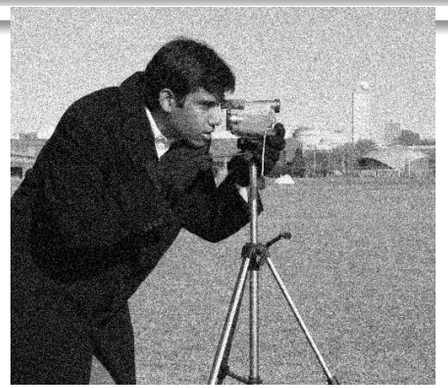
Weighted mean and Gaussian smoothing

$$\hat{f}(x,y) = \Sigma_{(s,t)\in Sxy}\, w(s,t)\, g(s,t), \qquad \Sigma w = 1$$

w(s,t): kernel weights. Gaussian weights give more importance to pixels near the center.

$$G_\sigma(s,t) = \left(\frac{1}{2\pi\sigma^2}\right)\exp\left(-\frac{s^2+t^2}{2\sigma^2}\right)$$

σ controls spatial spread. Small σ preserves more detail; large σ gives stronger smoothing and more blur.



Noisy

Gaussian smoothing

Gaussian filtering is a building block for scale-space, pyramid processing, and many vision algorithms.

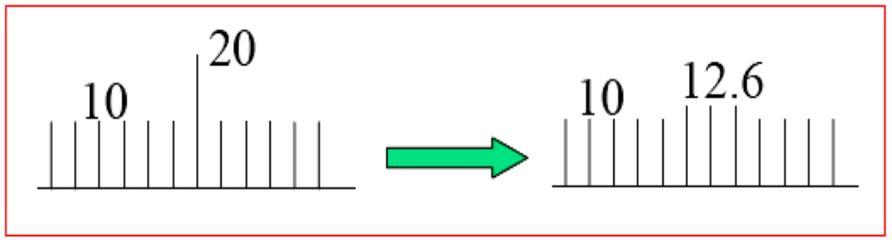
Mean Filters

- **Geometric mean filter**

$$\hat{f}(x,y) = \left[\prod_{(s,t)\in S_{xy}} g(s,t) \right]^{\frac{1}{mn}}$$

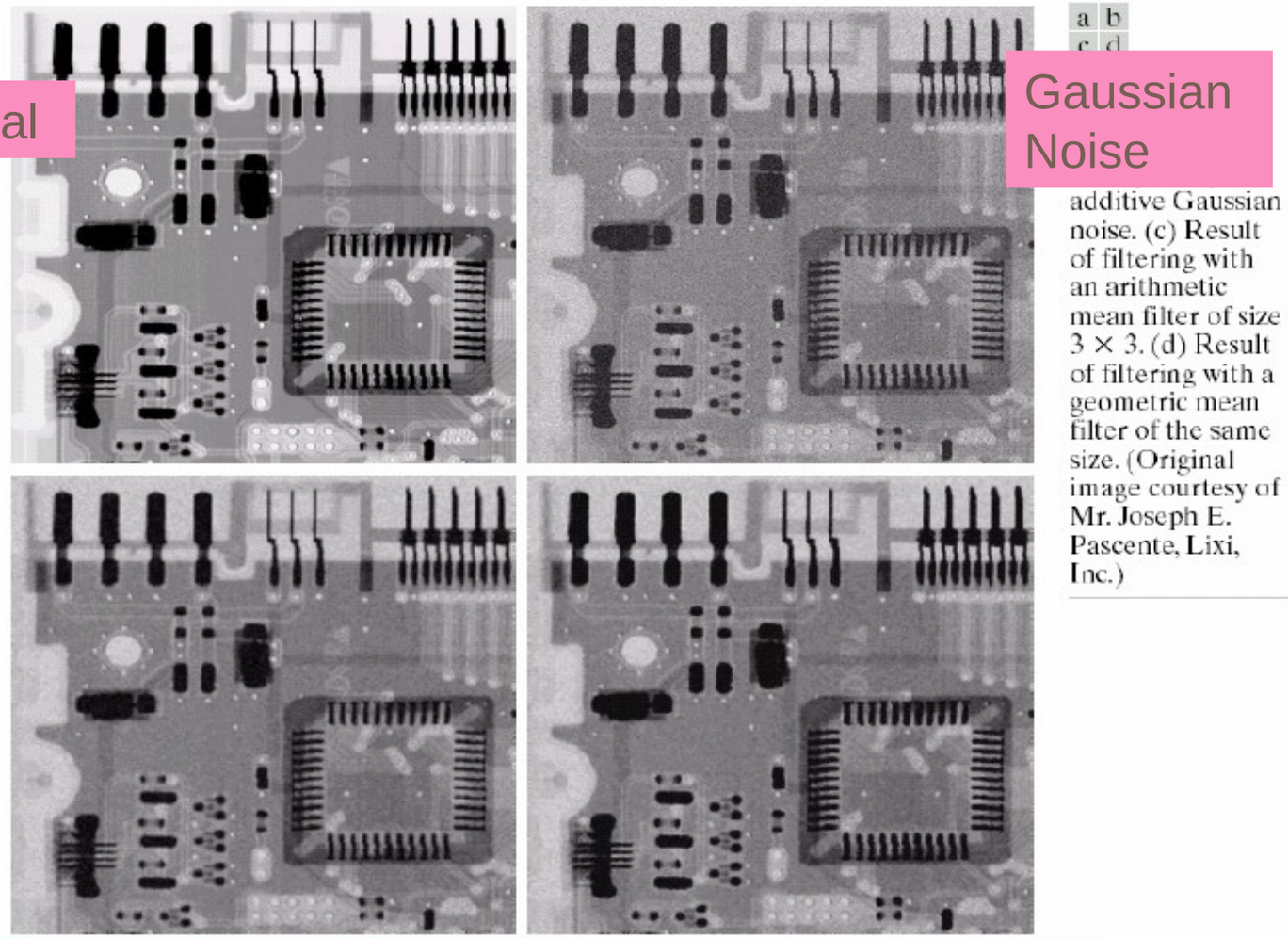
  - Smoothing comparable to arithmetic mean filter
  - Losing less image details

1D illustration



Mean Filters

- **An example**
  - Both attenuate noise
  - Image d is sharper than image c



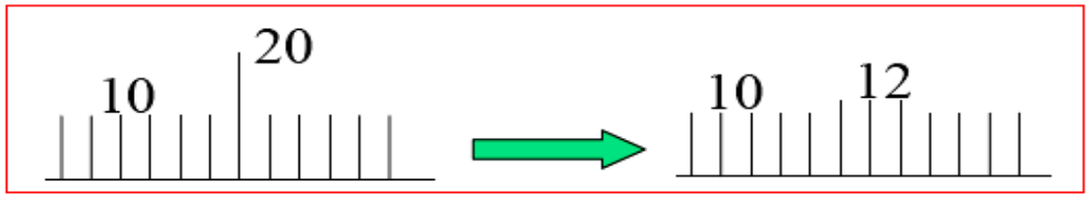
a b
c d

additive Gaussian noise. (c) Result of filtering with an arithmetic mean filter of size 3 × 3. (d) Result of filtering with a geometric mean filter of the same size. (Original image courtesy of Mr. Joseph E. Pascente, Lixi, Inc.)

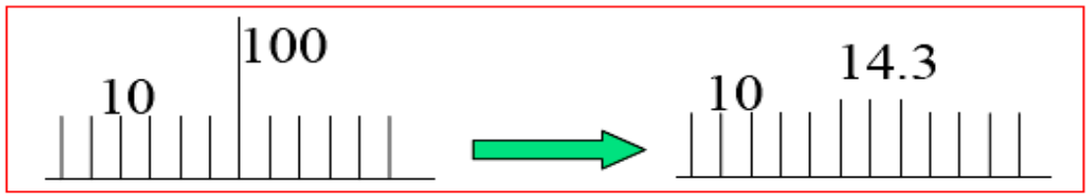
Mean Filters

- Another variation of the arithmetic mean filter
  - **Harmonic mean filter**
    - good for Gaussian noise
    - good for salt noise
    - bad for pepper noise

$$\hat{f}(x,y) = \frac{mn}{\sum_{(s,t)\in S_{xy}} \frac{1}{g(s,t)}}$$

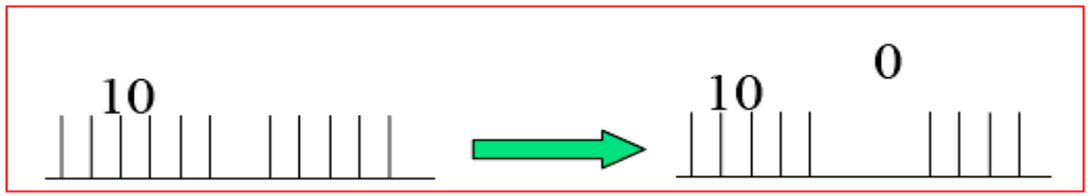


good



good

1 x 3 mask



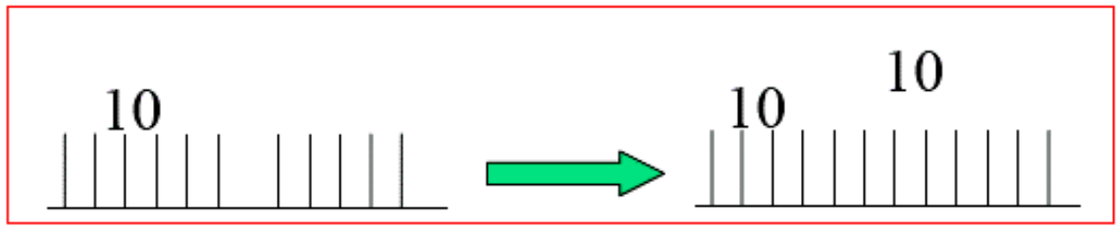
failed

Mean Filters

- **Contraharmonic mean filter**
  - $Q > 0$: eliminating pepper noise
  - $Q < 0$: eliminating salt noise
  - cannot do both simultaneously
  - $Q = 0$: arithmetic mean filter
  - $Q = -1$: harmonic mean filter

$$\hat{f}(x,y) = \frac{\sum_{(s,t)\in S_{xy}} g(s,t)^{Q+1}}{\sum_{(s,t)\in S_{xy}} g(s,t)^{Q}}$$

Q is the order of filter



$Q = 1$

1 x 3 mask

Geometric, harmonic, and contraharmonic mean filters

Geometric

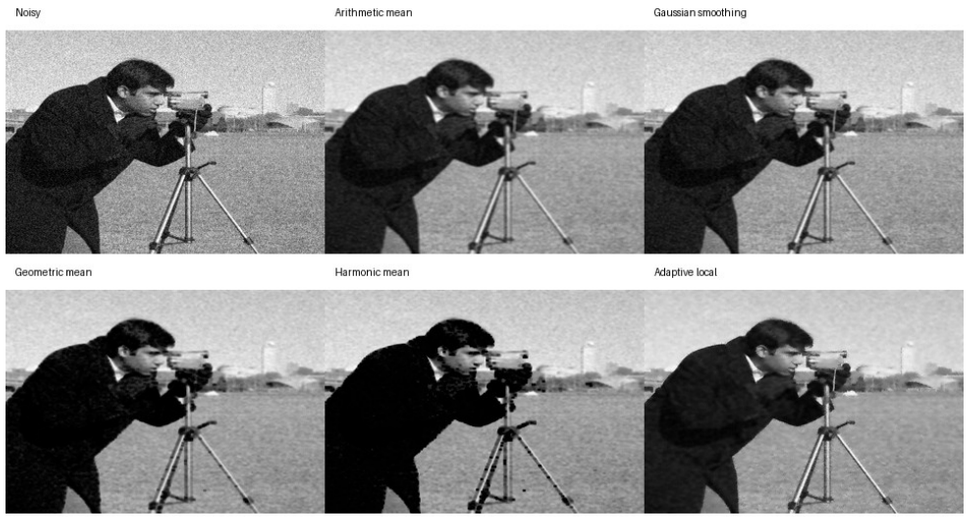
- Can preserve detail better than arithmetic mean for some noise,
- fails when zero pixels are present unless stabilized.

Harmonic

- Works better for salt noise;
- not suitable for pepper noise because near-zero values dominate.

Contraharmonic

- Q controls behavior.
- Positive Q reduces pepper noise; negative Q reduces salt noise.
- Wrong Q can amplify the opposite noise.



Mean-family filters: compare smoothing strength and artifact behavior

Mean Filters

(a)

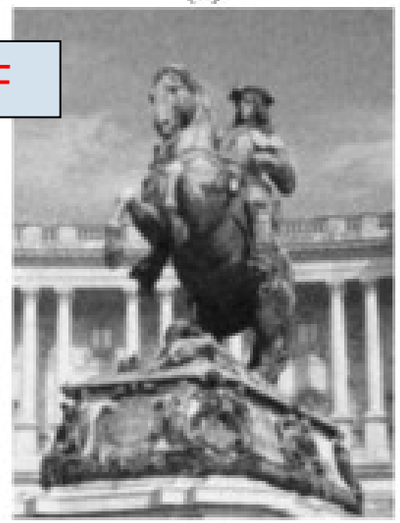
Original

(b)

Gaussian

(c)

3x3 AMF


(d)

5x5 AMF

3x3 GMF

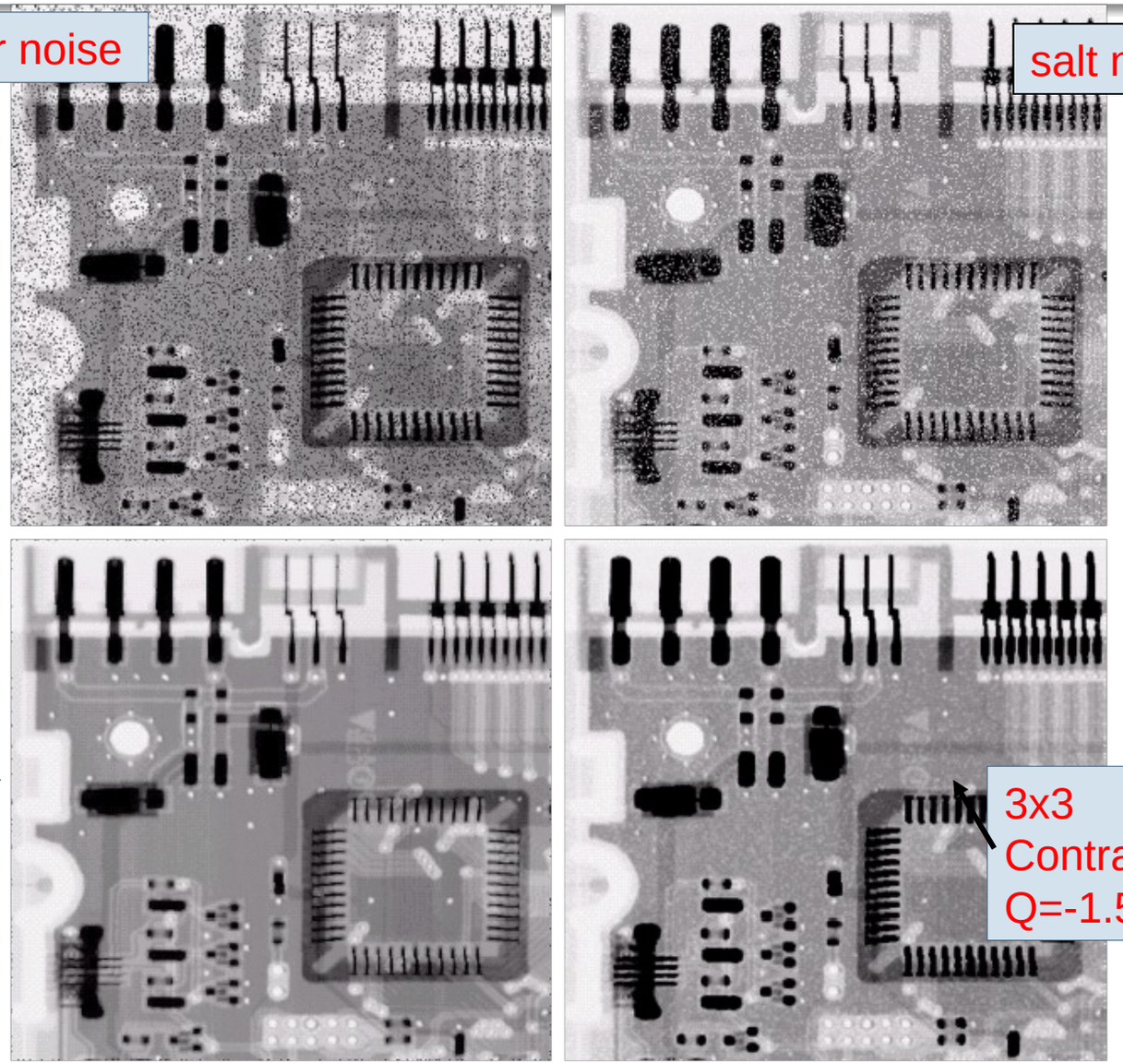
(f)

3x3 HMF

Mean Filters

a b
c d

FIGURE 5.8
(a) Image corrupted by pepper noise with a probability of 0.1. (b) Image corrupted by salt noise with the same probability. (c) Result of filtering (a) with a 3×3 contraharmonic filter of order 1.5. (d) Result of filtering (b) with $Q = -1.5$.



pepper noise

salt noise

3x3 Contraharmonic Q=1.5

3x3 Contraharmonic Q=-1.5

Mean filters - usage

- AMF works best for Gaussian, uniform, or Erlang noise.
- GMF- A variation of the AMF
  - Primarily used on images with Gaussian noise
  - Retains image detail better than the AMF
- *HMF - useful for images with Gaussian or salt noise*
  - *Black pixels (pepper noise) are not filtered*

- Contraharmonic mean filter is well suited for impulse noise
  - Disadvantage: must know pepper noise or salt noise in advance

Order Statistics Filters

- Known as Rank filters, Order filters OR Order Statistics filters
  - Operate on a neighborhood around a reference pixel by ordering (ranking) the pixel values
  - and then perform an operation on those ordered values to obtain the new value for the reference pixel

- They perform very well in the presence of salt and pepper noise but are more computationally expensive as compared to mean filters

Order Statistics Filters

- **Median Filter** - Most popular and useful
  - Selects the middle pixel value from the ordered set of values within the m × n neighborhood (W) around a reference pixel.
  - If mn is even, the average of the two closest to the middle of the ordered set is used instead.
    - Excellent for removing both bipolar and unipolar impulse noise

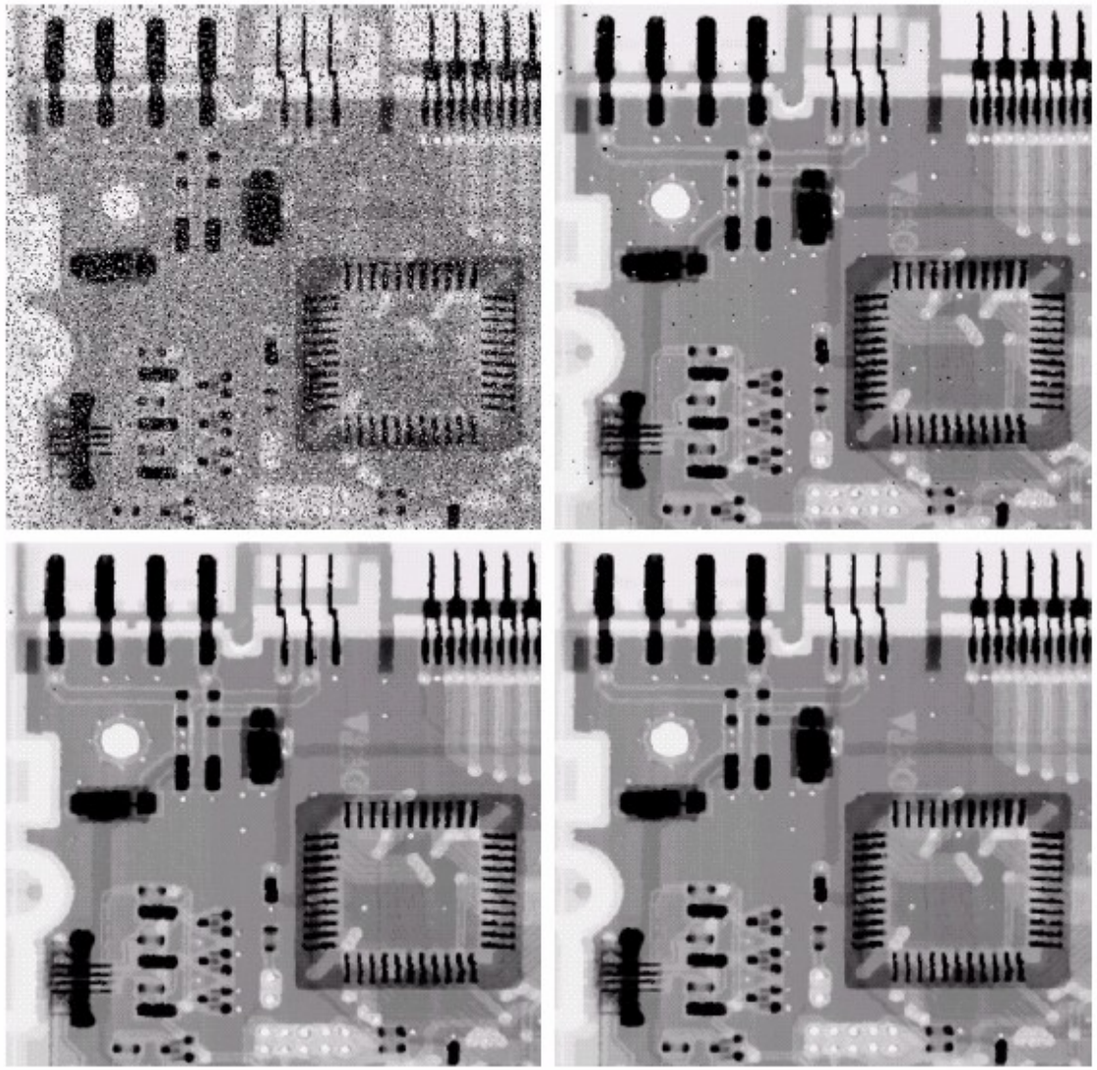
$$\hat{f}(x,y) = \underset{(s,t)\in S_{xy}}{median}\{g(s,t)\}$$

  - Many variants, extensions, and optimized implementations in the literature.

Median Filter

a b
c d

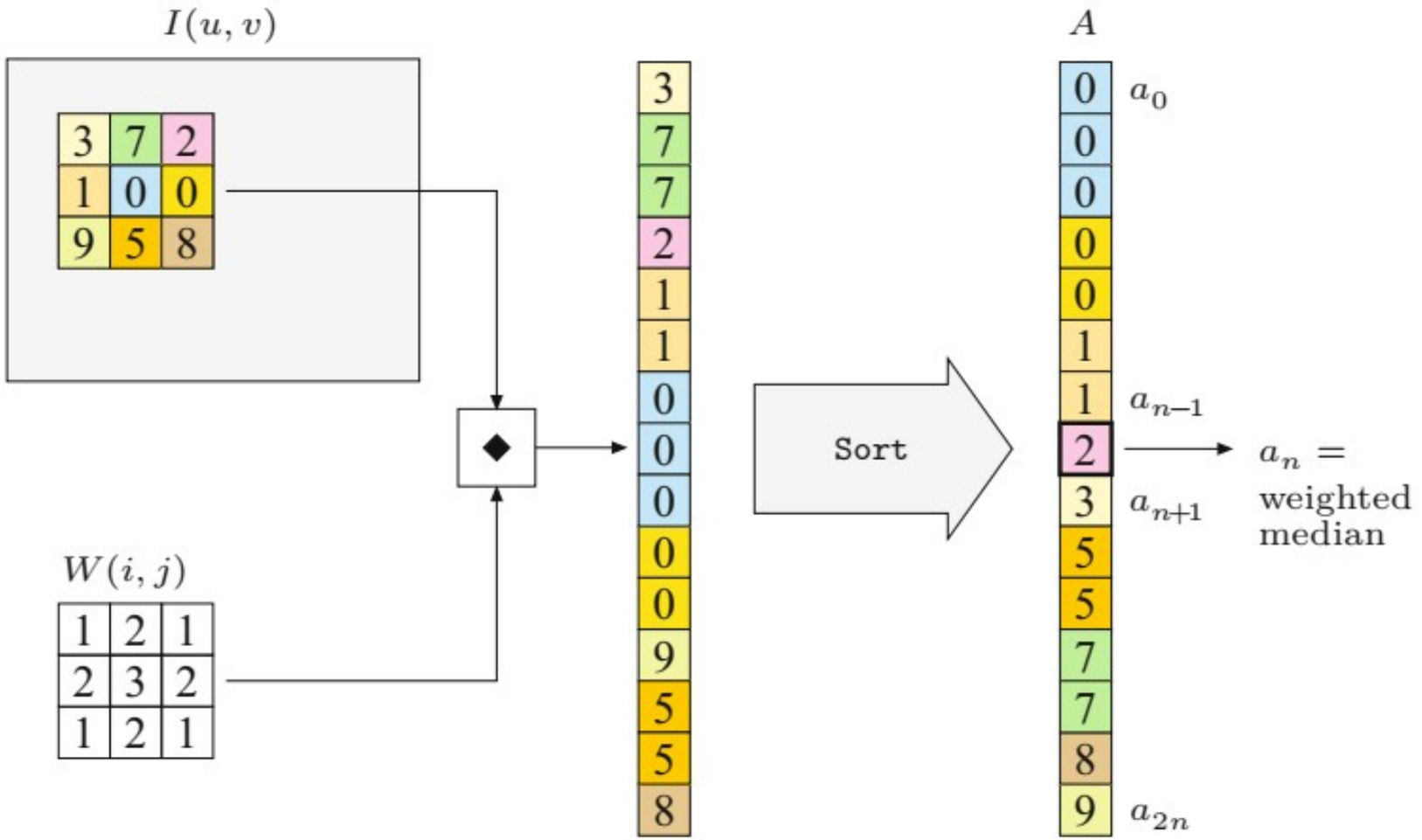
FIGURE 5.10
(a) Image corrupted by salt-and-pepper noise with probabilities $P_a = P_b = 0.1$.
(b) Result of one pass with a median filter of size 3×3.
(c) Result of processing (b) with this filter.
(d) Result of processing (c) with the same filter.



-- Repeated passes of median filter tend to blur the image.
-- Keep the number of passes as low as possible.

Weighted Median Filter

Fig. 5.21
Weighted median example. Each pixel value is inserted into the extended pixel vector multiple times, as specified by the weight matrix W. For example, the value 0 from the center pixel is inserted three times (since $W(0,0) = 3$) and the pixel value 7 twice. The pixel vector is sorted and the center value (2) is taken as the median.



Order Statist

- **Max filter**
  - Replace the reference
  - reducing pepper n

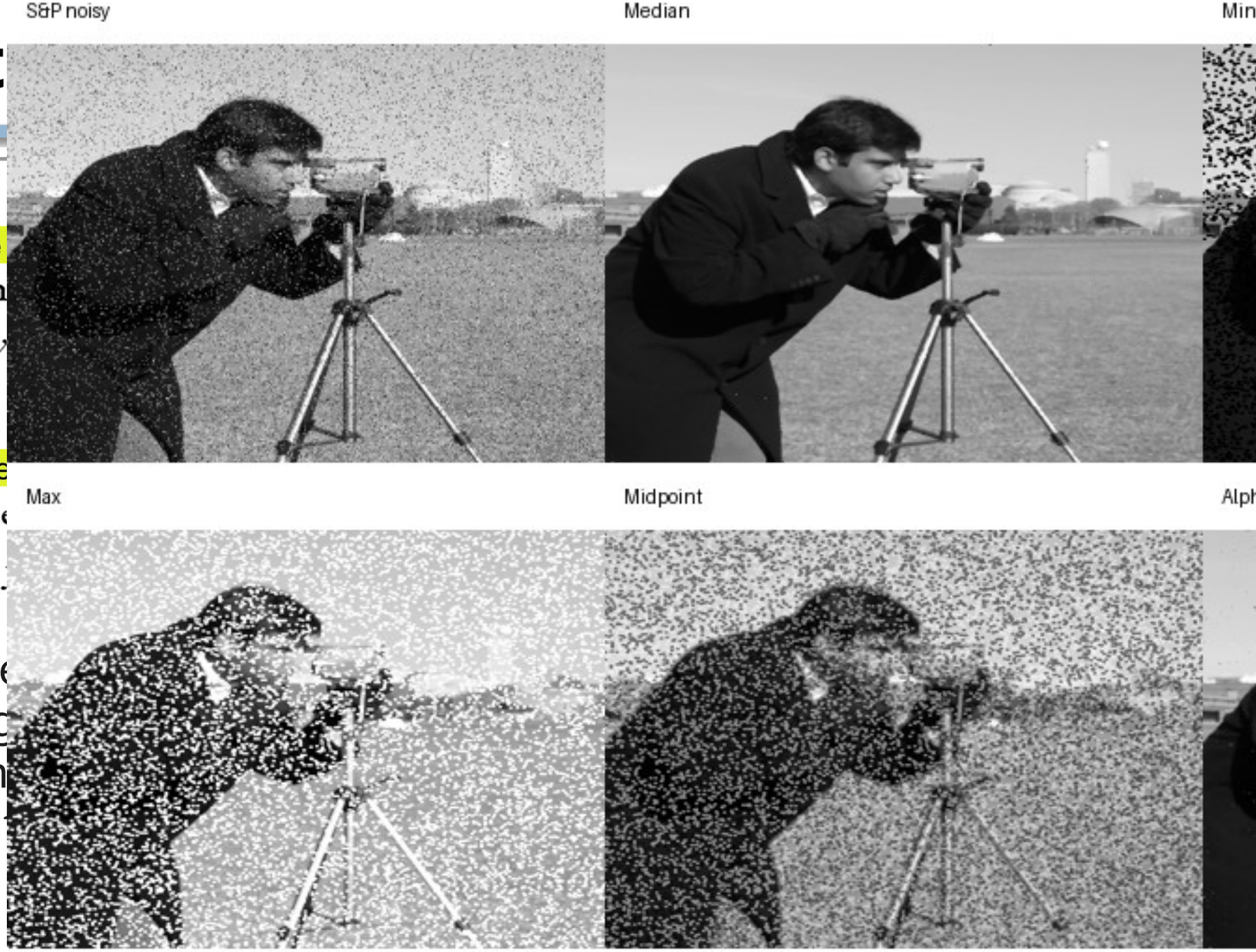
$$\hat{f}(x,y$$

- **Min filter**
  - Replace with the darke
  - reducing salt noise

$$\hat{f}(x,$$

Min filter: remove
expands dark reg

- Max filter: rem
  expands bright

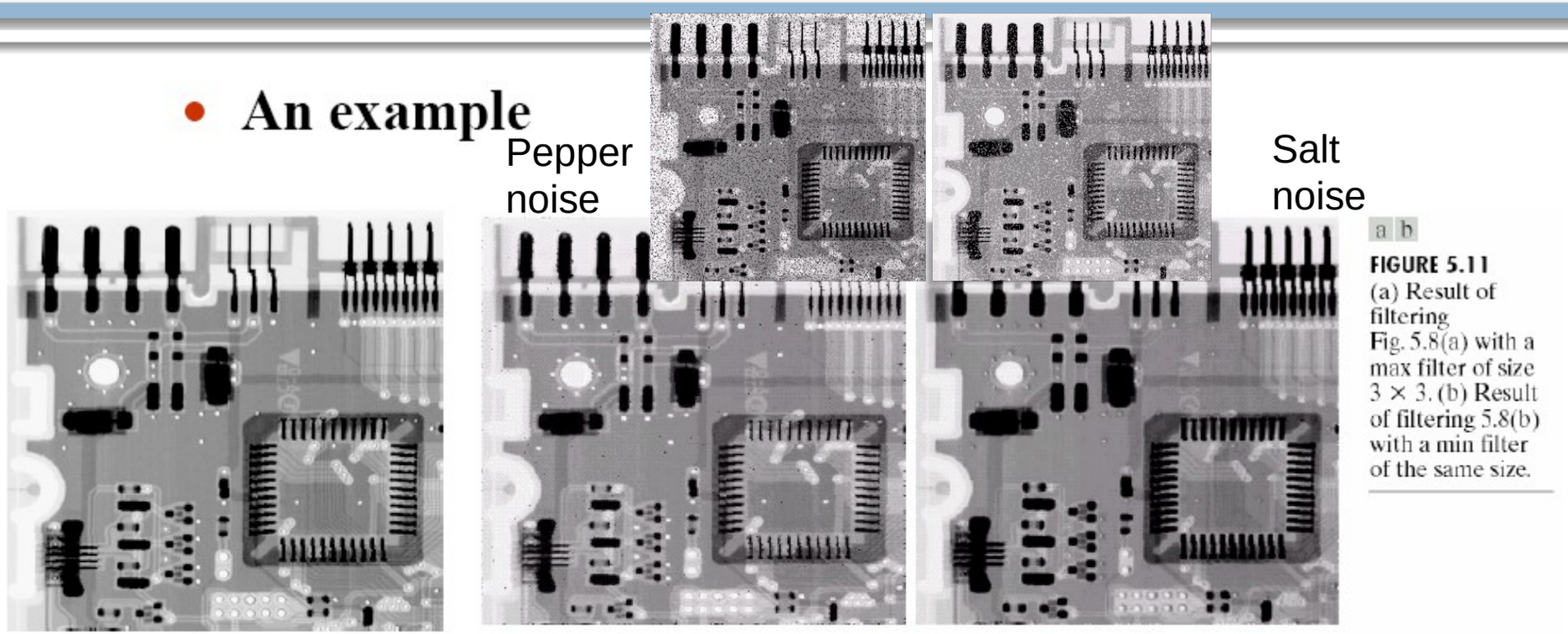


Order Statistics Filters

- **An example**

Pepper noise

Salt noise



a b

FIGURE 5.11 (a) Result of filtering Fig. 5.8(a) with a max filter of size 3 × 3. (b) Result of filtering 5.8(b) with a min filter of the same size.

Original image

Max filtering removes pepper noise

Min filtering remove salt noise

It also removes
It also removes
some dark pixels

It also removes
It also removes
some white pixels

Order Statistics Filters

Midpoint Filter

- Combine order statistics and averaging
- Midpoint filter works best for randomly distributed noise (Gaussian or uniform)

$$\hat{f}(x,y) = \frac{1}{2}\left[\max_{(s,t)\in S_{xy}}\{g(s,t)\} + \min_{(s,t)\in S_{xy}}\{g(s,t)\}\right]$$

- Calculates the average of the highest and lowest pixel values within a window
  - What would it do with salt and pepper noise ?

Order Statistics Filters

- **Midpoint Filt**

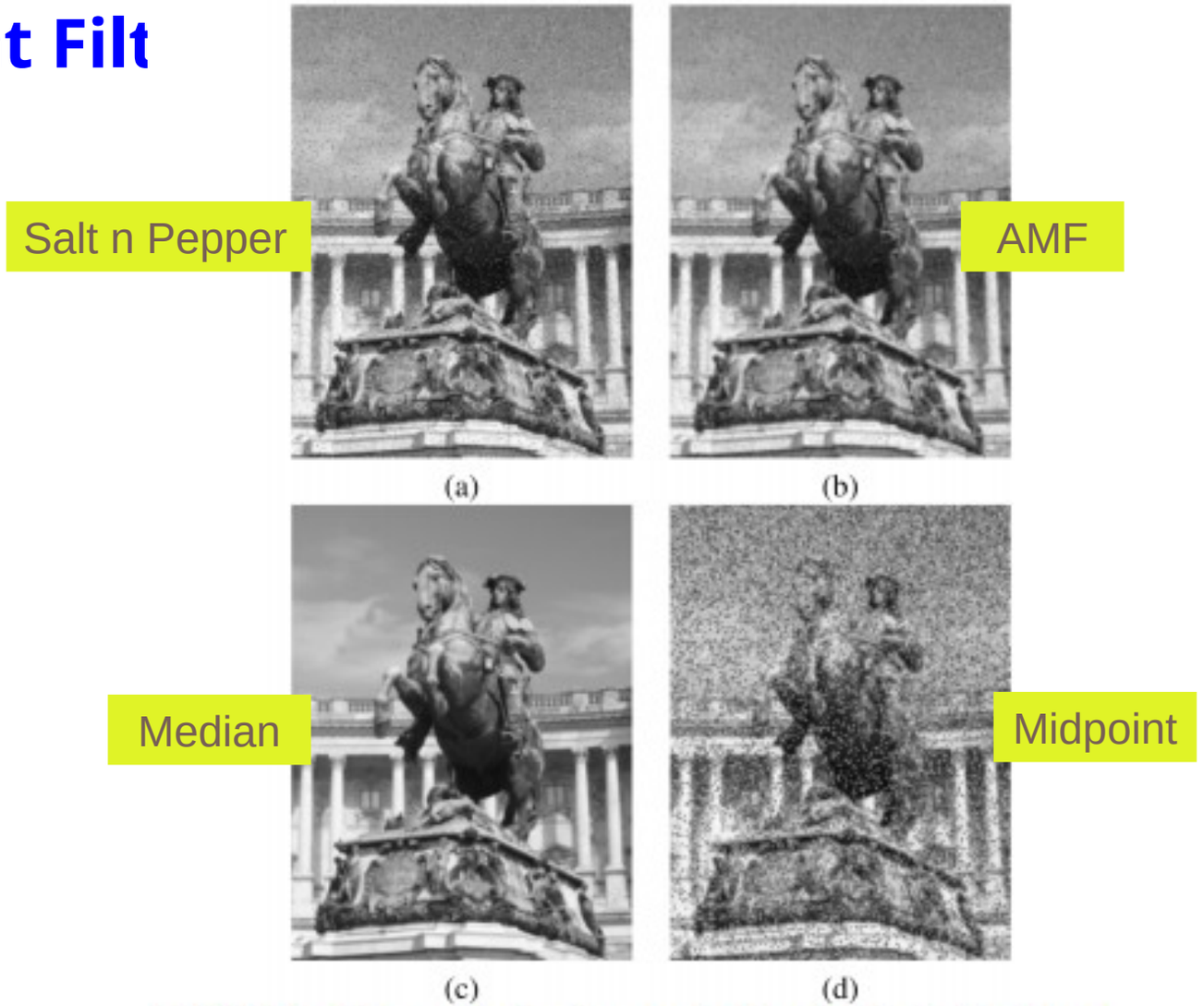


FIGURE 12.8 (a) Image with salt and pepper noise; (b) result of 3×3 arithmetic mean filtering (for comparison); (c) result of 3×3 median filtering; (d) result of 3×3 midpoint

Order Statistics Filters

- **Alpha-trimmed mean filter**
  - $d = 0$: arithmetic mean filter
  - $d = mn - 1$: median filter
  - suitable for the situation involving multiple types of noise

$$\hat{f}(x,y) = \frac{1}{mn-d} \sum_{(s,t)\in S_{xy}} g_r(s,t)$$

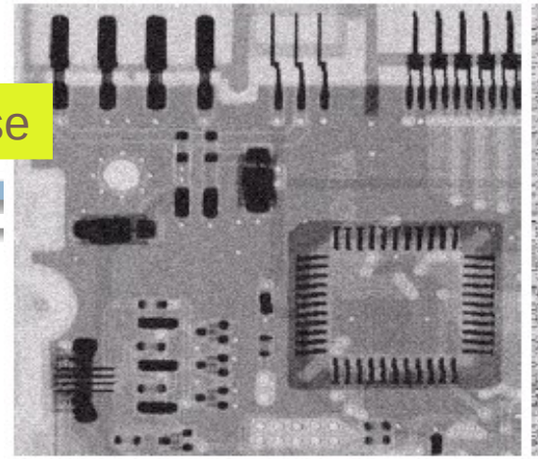
delete d/2 lowest and d/2 highest values first, then average the remaining

Gaussian + Impulse

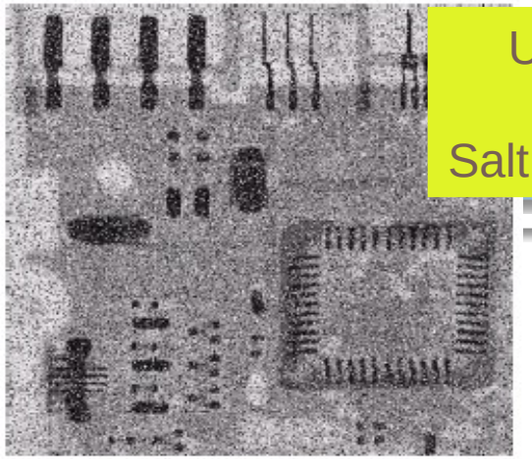
Combines ideas of mean and median filtering.

- d controls how aggressively extremes are removed.

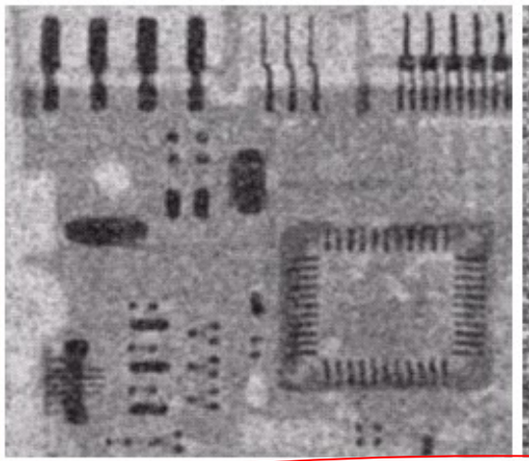
Uniform noise



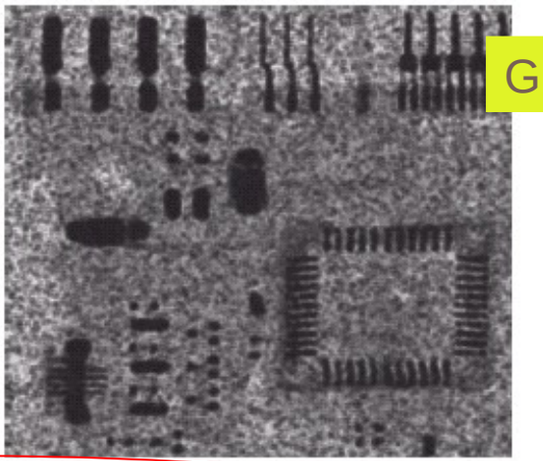
Uniform
+
Salt n Peeper



AMF



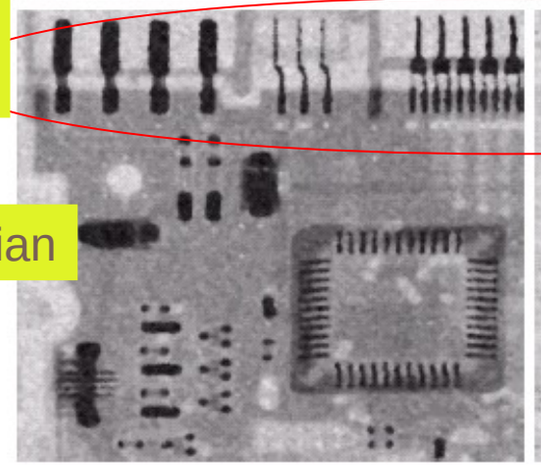
GMF



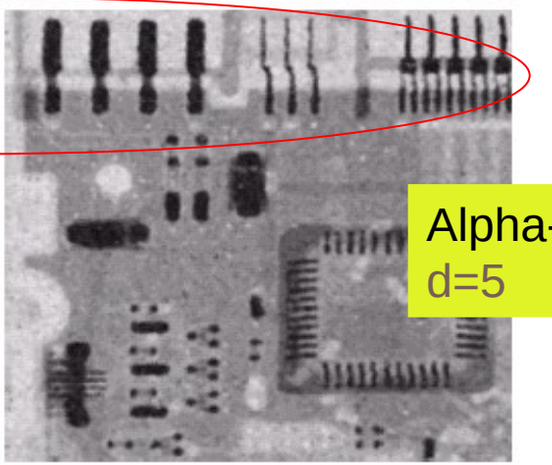
- High level of noise ➔ large filter

- Median and alpha-trimmed filter performed better
- Alpha-trimmed did better than median filter

Median



Alpha-trimmed
d=5



All 5x5 filter

Adaptive Spatial Filters

Restoration by spatial filtering

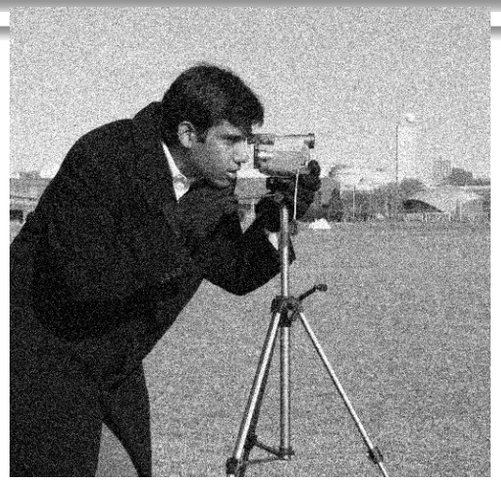
- **Filters discussed so far**
  - Do not consider image characteristics
- **Adaptive filters to be discussed**
  - Behaviors based on statistical characteristics of the subimage under a filter window
  - Better performance, More complicated
  - **Adaptive, local noise reduction filter**
  - **Adaptive median filter**

The behaviour of **adaptive filters** changes depending on the characteristics of the image inside the filter region

Adaptive Local Noise filtering

$$\hat{f}(x,y) = g(x,y) - \frac{\sigma_\eta^2}{\sigma_L^2}[g(x,y) - m_L]$$

- Only noise variance needs to be known or estimated in advance
- when $\sigma_L^2 \geq \sigma_\eta^2$, $\hat{f}(x,y) \geq 0$
- when $\sigma_L^2 < \sigma_\eta^2$, $\hat{f}(x,y)$ may be negative
  - scheme 1: setting $\sigma_L^2 = \sigma_\eta^2$, $\hat{f}(x,y) = m_L$
  - scheme 2: allowing negative values to occur first and rescaling the final image (causing dynamic range reduced)



Noisy

Adaptive

- Flat areas: σ_L^2 is small, so more smoothing is applied.
- Edges/textures: σ_L^2 is large, so less smoothing is applied.
- Better than fixed mean filtering when image content varies across the image.

Adaptive filtering

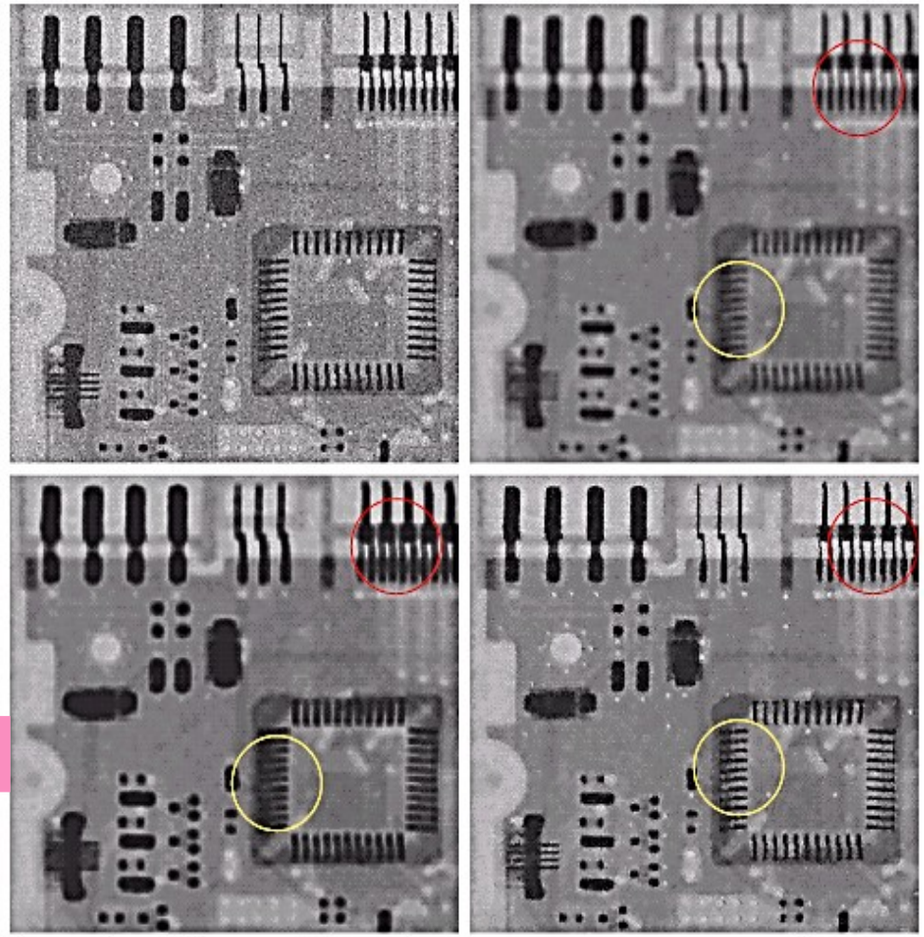
- An example of results obtained by three filters

$\sigma_\eta^2 = 1000$

Filter size
7 x 7

a b
c d

FIGURE 5.13
(a) Image corrupted by additive Gaussian noise of zero mean and variance 1000. (b) Result of arithmetic mean filtering. (c) Result of geometric mean filtering. (d) Result of adaptive noise reduction filtering. All filters were of size 7 × 7.



Noise reduced but significant blurring

AMF

GMF

Adaptive filtering

Noise reduced but significant blurring

Best result

Adaptive Median Filtering

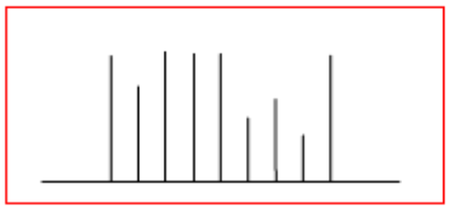
- The median filter performs relatively well on impulse noise as long as the spatial density of the impulse noise is not large
- The adaptive median filter can handle much more spatially **dense** impulse noise, and also performs some smoothing for non-impulse noise
- The key insight in the adaptive median filter is that the filter size changes depending on the characteristics of the image
- Remember that filtering looks at each original pixel image in turn and generates a new filtered pixel

Adaptive filtering

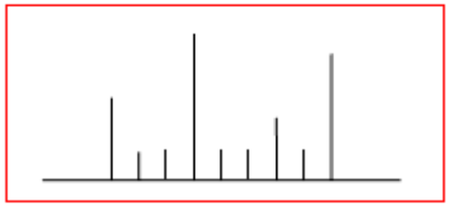
- **Algorithm of adaptive median filtering**
  - Level 0: Set initial window size for the sub image S_{xy} at a new (x,y)
  - Level 1: Calculate Z_{min}, Z_{med}, Z_{max}, for the S_{xy}
  - Level A: If $z_{min} < z_{med} < z_{max}$, Go to Level B
    Else increase the window size
    If window size $\leq S_{max}$, repeat Level 1
    Else output z_{med}
  - Level B: If $z_{min} < z_{xy} < z_{max}$, output z_{xy}
    Else output z_{med}.

Is Z_med impulse?

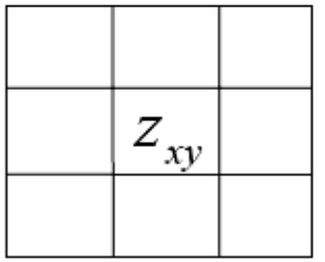
Is Z_xy impulse?



$z_{med} = z_{max}$



$z_{med} = z_{min}$



S_{xy}

Page 28

Adaptive filtering

- An example of results by two filters

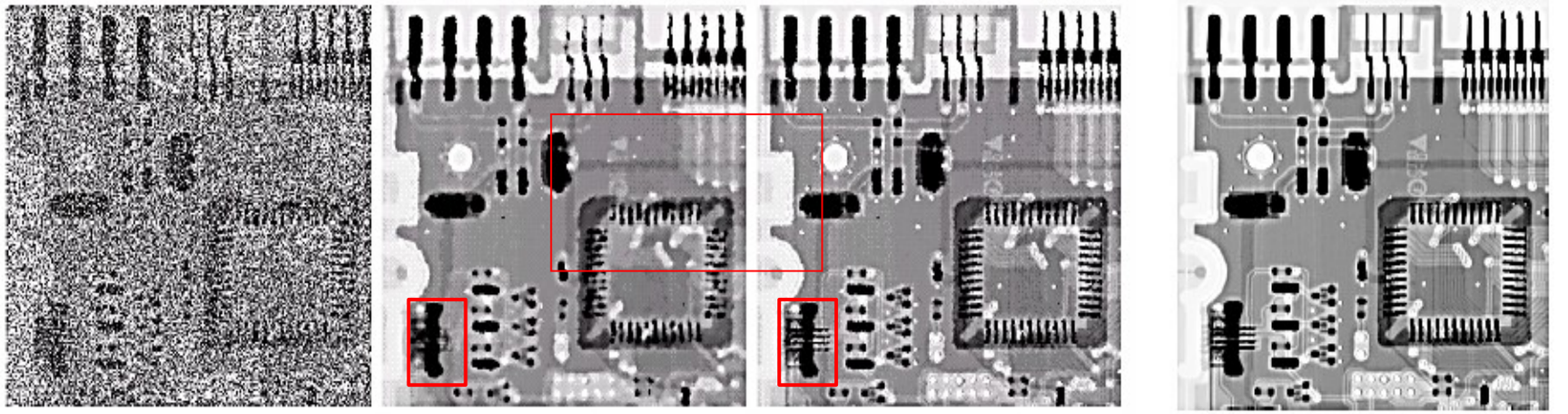


FIGURE 5.14 (a) Image corrupted by salt-and-pepper noise with probabilities $P_a = P_b = 0.25$. (b) Result of filtering with a 7×7 median filter. (c) Result of adaptive median filtering with $S_{max} = 7$.

Noise removed effectively but significant loss of details

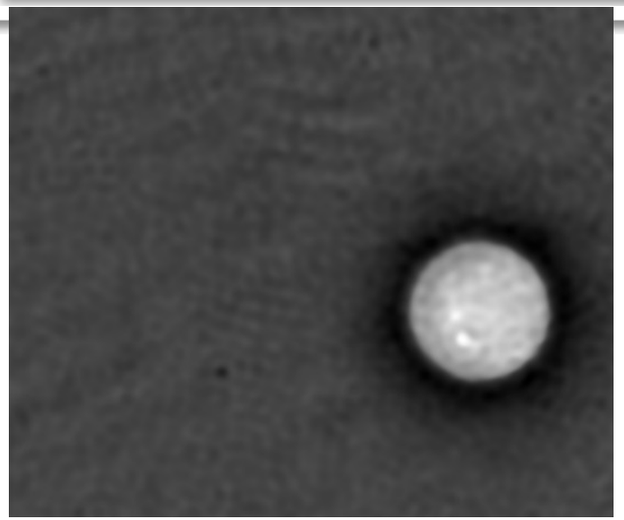
Noise removed effectively and preserving details and sharpness

Choosing a filter from the noise appearance

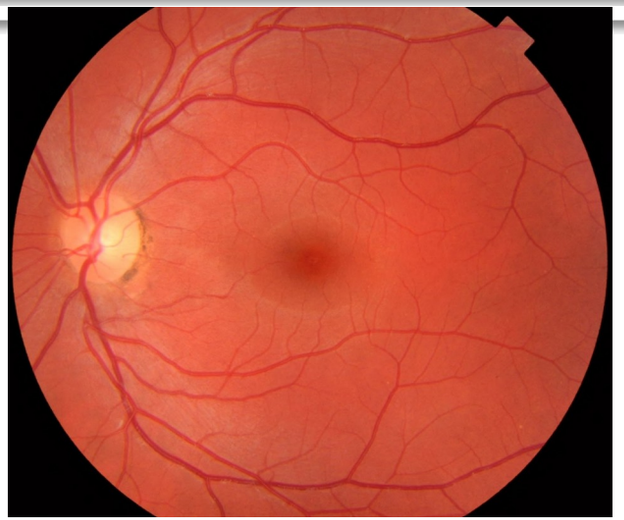
Observed artifact	Likely model	Typical first spatial method
Fine grain everywhere	Gaussian/additive	Mean, Gaussian, adaptive local, NLM
Black/white dots	Impulse	Median, adaptive median, alpha-trimmed
Texture-like granular noise	Speckle/multiplicative	Log + smoothing, bilateral, NLM
Low-count noisy image	Poisson/counting	Variance stabilization, RL, NLM
Repeating stripes	Periodic	Frequency-domain notch / band reject

The table is a starting point. Real images often contain mixed noise, blur, compression, and acquisition artifacts.

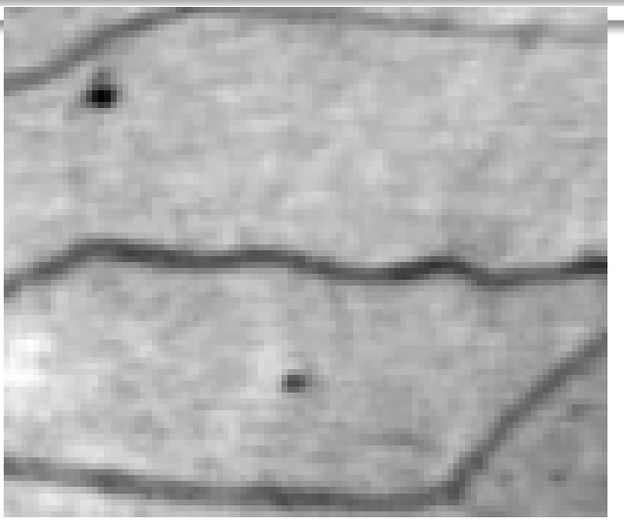
Case study: microscopic and retinal images



Microscopy



Retinal fundus



Tiny lesion example

Restoration can help segmentation of cells, vessels, lesions, or anatomical structures.
But oversmoothing can remove diagnostically important small bright/dark objects.

- Report task-specific sensitivity for small targets, not only image-level metrics.

Limitations of spatial-domain restoration

Local filters cannot remove structured periodic noise efficiently.
Strong blur is hard to reverse without a PSF or frequency-domain model.
Simple smoothing reduces noise but also reduces useful high-frequency information.
Parameter choices are image-dependent.

- Real data often contain mixed degradations: blur + noise + compression + nonuniform illumination.

Periodic noise

Motion blur

These limitations motivate: frequency-domain restoration and model-based deconvolution.

Key takeaways

Image restoration estimates a clean image from a degraded observation using a model.
MSE/PSNR measure pixel error; SSIM measures local structural similarity.
Noise model matters: Gaussian, impulse, Poisson, speckle, uniform, and periodic noise need different approaches.
Spatial filters are intuitive and useful, but they trade noise reduction against edge/detail preservation.

- Frequency-domain restoration becomes necessary for periodic noise and principled deblurring.

Suggested reading / references

R. C. Gonzalez and R. E. Woods, Digital Image Processing, chapters on image restoration and reconstruction.
A. K. Jain, Fundamentals of Digital Image Processing, sections on degradation models and image restoration.
C. Tomasi and R. Manduchi, Bilateral Filtering for Gray and Color Images.

- A. Buades, B. Coll, and J. M. Morel, A Non-Local Algorithm for Image Denoising.

Exercise

- Filter the following image with an adaptive median filter. Assume a maximum window size of 3 ×3.
  - Provide the result only for the two central pixels (with values 5 and 1):

3	4	5	4
0	**5**	**1**	1
0	15	3	3

- Consider the following image.
  - Provide the result of the central pixel after using a harmonic mean filter. The neighborhood size is 3 × 3.
  - Provide the result of the central pixel after using a midpoint filter with a neighborhood size of 5 × 5.

3	4	5	4	3
2	10	4	5	1
0	5	6	2	0
1	6	3	0	5
0	15	3	3	2

Restoration in Frequency Domain

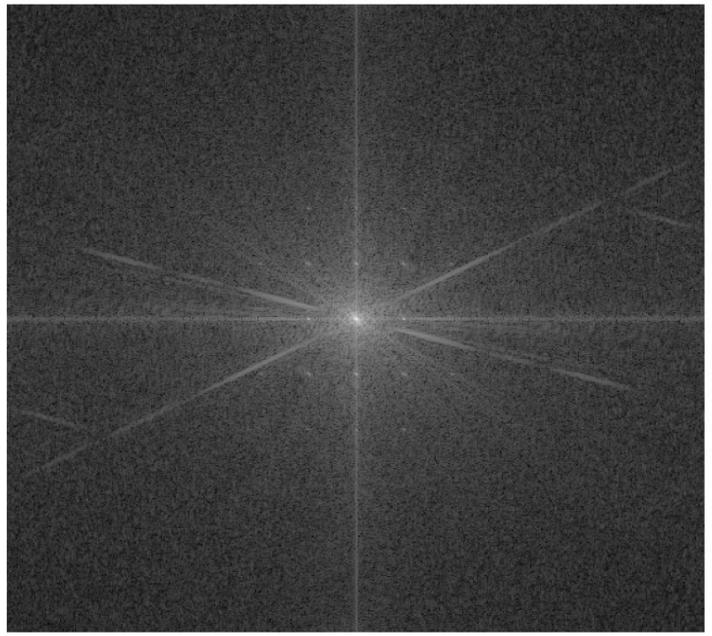
Prof. Dr. Sk. Md. Masudul Ahsan
Dept. of CSE, KUET

Periodic noise: a motivating example

Original camera image

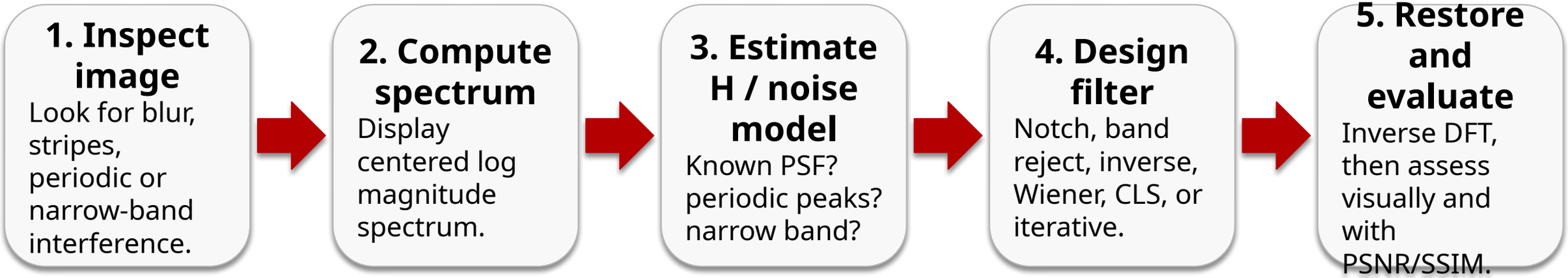
Periodic-noise corruption

Centered log spectrum



- The spatial image shows a structured grid-like artifact, but the centered spectrum reveals distinct bright off-center peaks.
- Each peak location corresponds to one sinusoidal interference component. Because the spectrum is conjugate symmetric, peaks occur in symmetric pairs.

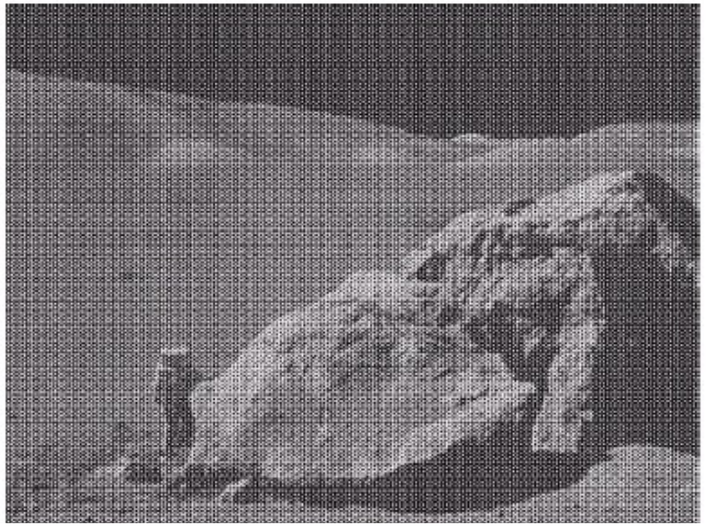
Typical Fourier-domain restoration workflow

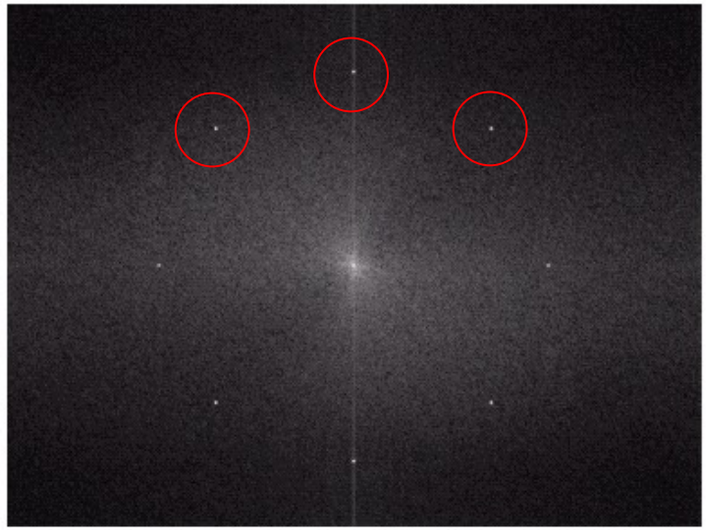


- When peaks are discrete and symmetric, use notch filtering. When interference occupies a narrow band, use band-reject filtering.
- When blur is dominant and the PSF/OTF is known or estimated, use inverse, Wiener, or regularized deconvolution.
- Always inspect boundary artifacts, ringing, over-smoothing, and residual noise before accepting the result.

Periodic Noise

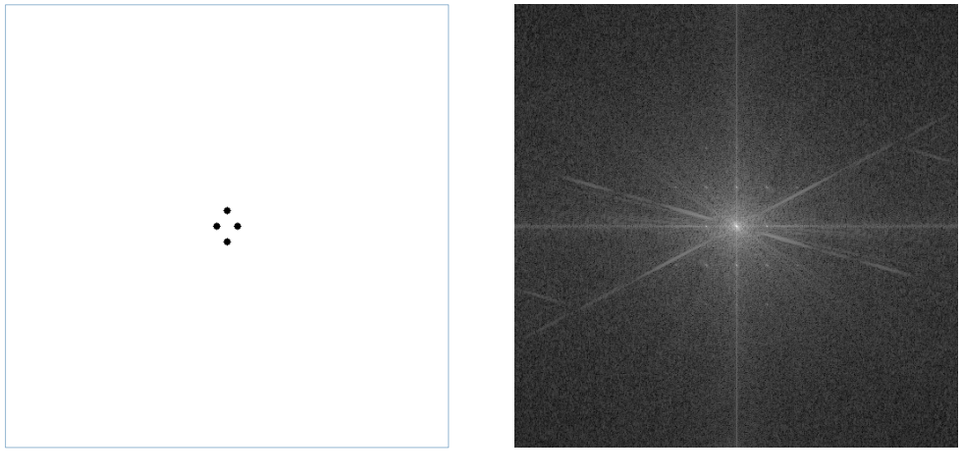
- Typically arises due to electrical or electromagnetic interference
- Gives rise to regular noise patterns in an image
- Frequency domain techniques in the Fourier domain are most effective at removing periodic noise
- Removing periodic noise form an image involves removing a particular range of frequencies from that image
- ***Notch*** *or* ***Band*** *reject* filters can be used for this purpose





Basic notch filtering: idea and geometry

- A notch reject filter suppresses small neighborhoods around the interference frequencies while leaving most other frequencies unchanged.
- If one notch is placed at $+(u_k, v_k)$,a conjugate notch must also be placed at $-(u_k, v_k)$,to preserve a real-valued output image.
- The notch size controls selectivity: too small leaves noise; too large removes useful detail.



Left: example notch response H(u,v). Right: bright interference peaks in the spectrum.

$$\widehat{\boldsymbol{F}}(\boldsymbol{u},\boldsymbol{v}) = \boldsymbol{H_{NR}}(\boldsymbol{u},\boldsymbol{v})\,\boldsymbol{G}(\boldsymbol{u},\boldsymbol{v})$$

H_{NR} : notch-reject filter; G: observed spectrum; $\widehat{\boldsymbol{F}}$: restored spectrum.

Ideal notch-reject filter

Center frequency components

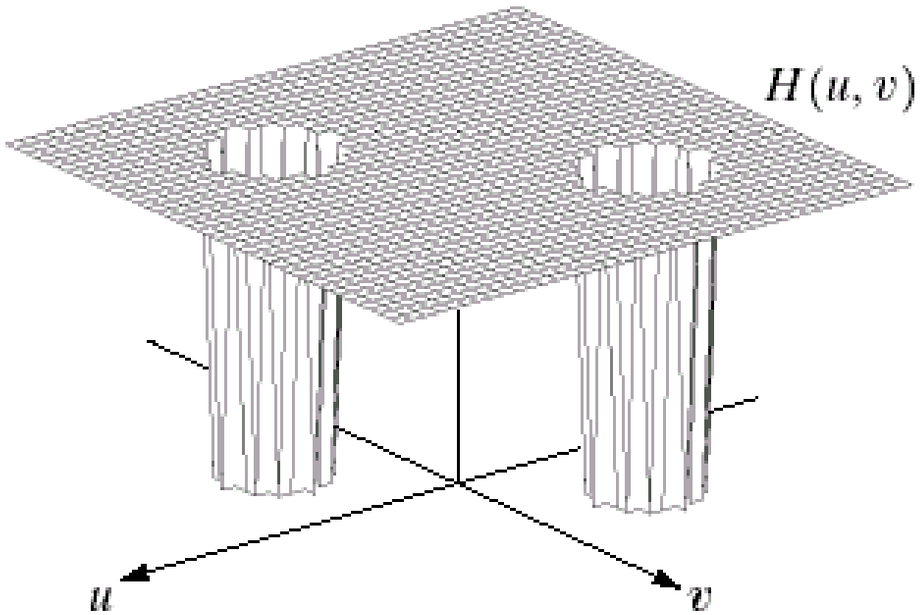
$$H(u,v) = 0 \quad \text{if} \quad D_1(u,v) \le D_0 \quad \text{or} \quad D_2(u,v) \le D_0$$
$$= 1 \quad \text{otherwise}$$

$$D_1(u,v) = \left[(u - M/2 - u_0)^2 + (v - N/2 - v_0)^2 \right]^{1/2}$$

and

$$D_2(u,v) = \left[(u - M/2 + u_0)^2 + (v - N/2 + v_0)^2 \right]^{1/2}$$

Shift with respect to the center



Ideal notch-reject filter

$$H_{NR}(u,v) = \prod_{k=1}^{Q} H_k(u,v) H_{-k}(u,v)$$

$$H_k(u,v) = 0 \; if \; D_k <= D_0, else \; 1 \;\; ;$$
$$H_{-k}(u,v) = 0 \; if \; D_{-k} <= D_0, else \; 1$$

- Q: number of notch pairs;
- $D_k(u,v)$ is the distance from (u,v) to the kth peak center $+(u_k, v_k)$;
- $D_{-k}(u,v)$ is the distance to the symmetric peak; D_0 is the notch radius.

$$D_k(u,v) = \left[(u - M/2 - u_k)^2 + (v - N/2 - v_k)^2\right]^{1/2}$$

$$D_{-k}(u,v) = \left[(u - M/2 + u_k)^2 + (v - N/2 + v_k)^2\right]^{1/2}$$

- The ideal notch has a sharp binary transition. It is simple and effective when peak locations are clear.
- Drawback: abrupt spectral cutoffs create ringing in the restored image, especially near strong edges.

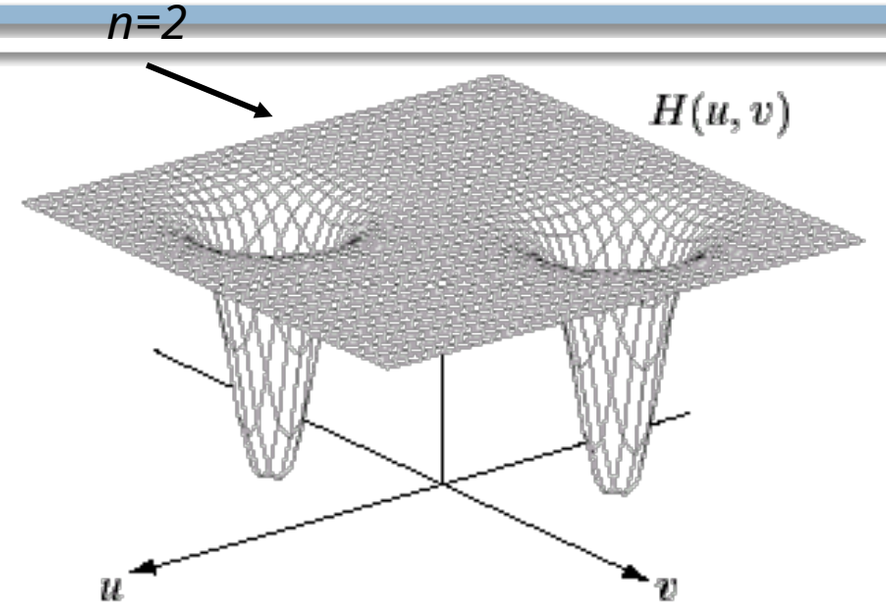
Butterworth and Gaussian notch filters

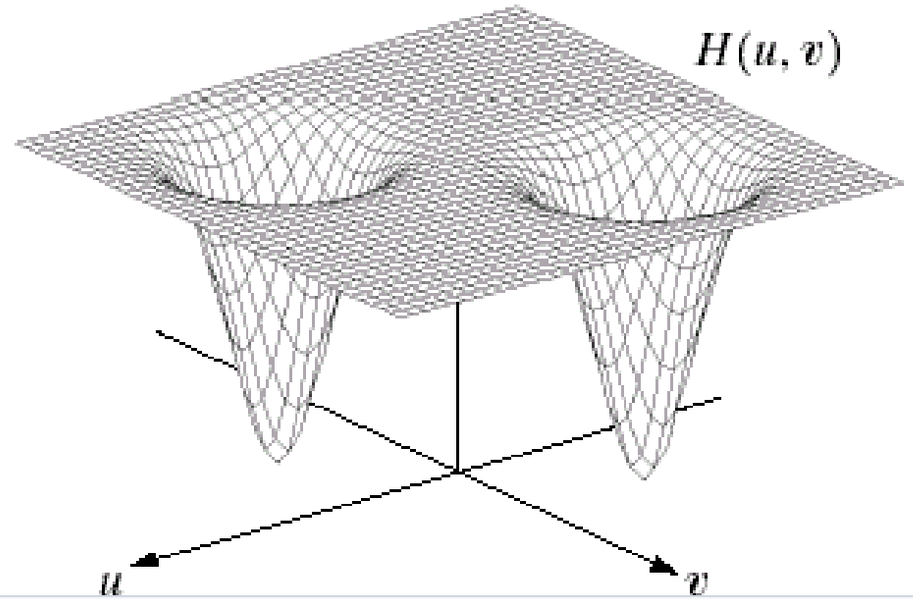
Butterworth notch filter

$$H(u,v) = \frac{1}{1 + \left[\frac{D_0^2}{D_1(u,v) D_2(u,v)} \right]^{2n}}$$

Gaussian notch filter

$$H(u,v) = 1 - e^{-\frac{1}{2}\left[\frac{D_1(u,v) D_2(u,v)}{D_0^2} \right]}$$





Butterworth and Gaussian notch filters

For example, the following is a Butterworth notch reject filter transfer function of order n with three notch pairs:

$$H_{\mathrm{NR}}(u,v) = \prod_{k=1}^{3}\left[\frac{1}{1+\left[D_{0k}/D_k(u,v)\right]^n}\right]\left[\frac{1}{1+\left[D_{0k}/D_{-k}(u,v)\right]^n}\right] \qquad (5\text{-}36)$$

- Butterworth offers a controllable transition width through the order n.
- Gaussian notch filters are smooth and often preserve visual quality better than ideal notches.
- In practice, Butterworth or Gaussian notches are usually preferred to the ideal design.

ideal

Butterworth

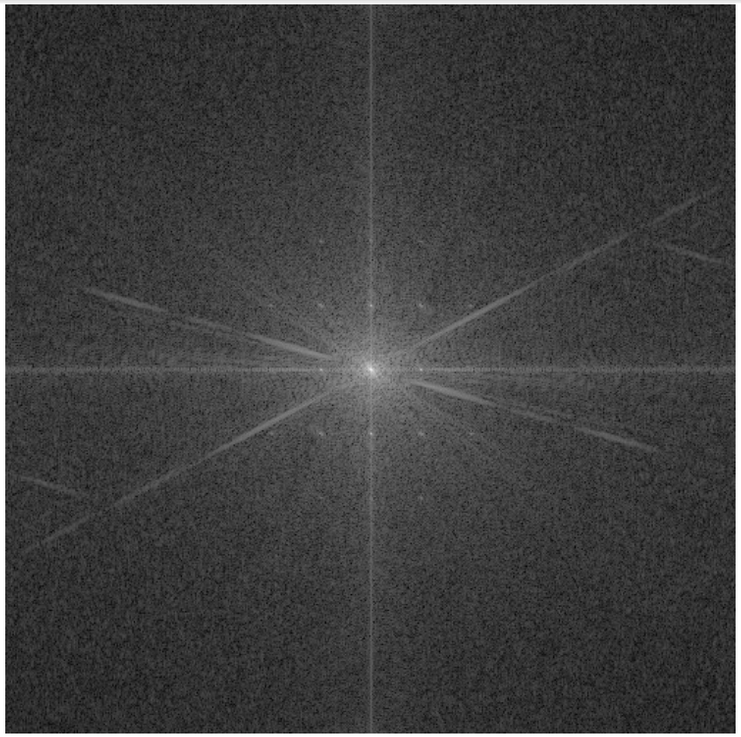
Notch Pass filters

Notch filters that pass, rather than suppress:

$$H_{np}(u,v) = 1 - H_{nr}(u,v)$$

- *NR* filters become highpass filters if $u_0 = v_0 = 0$
- *NP* filters become lowpass filters if $u_0 = v_0 = 0$

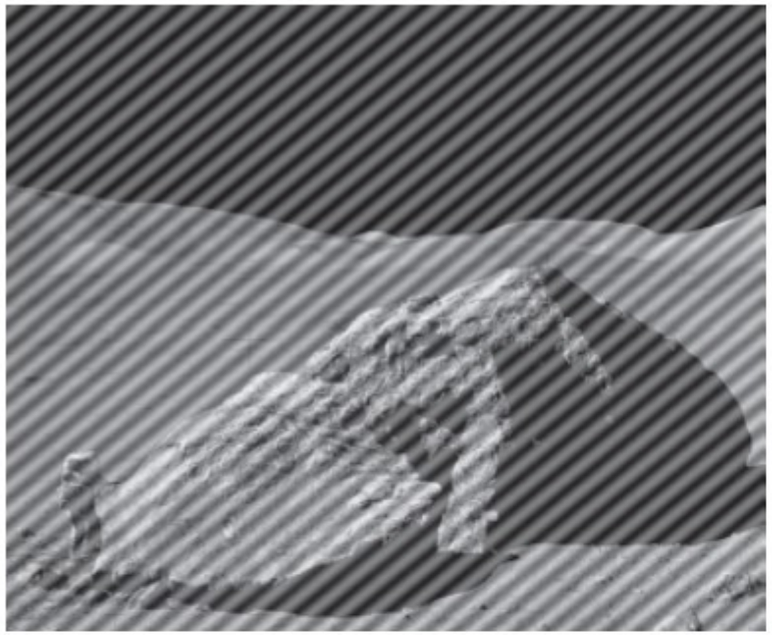
How to find notch locations in practice

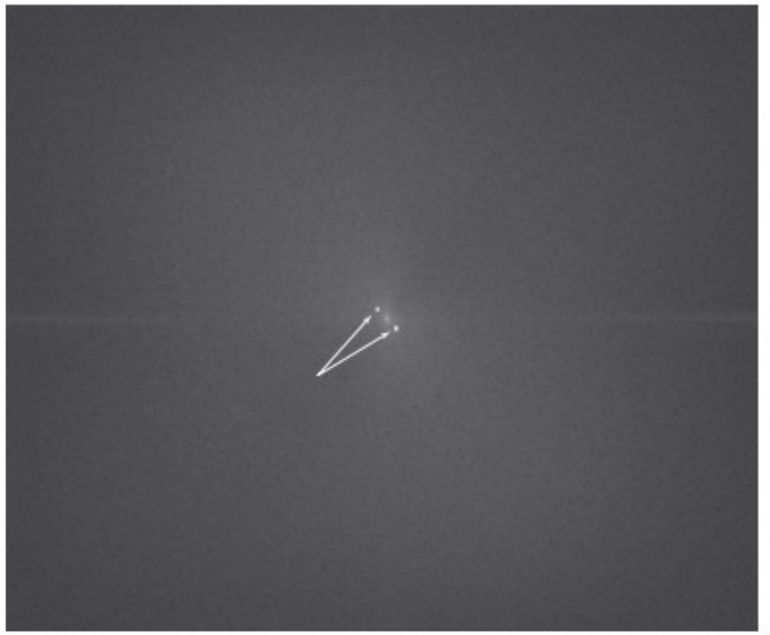


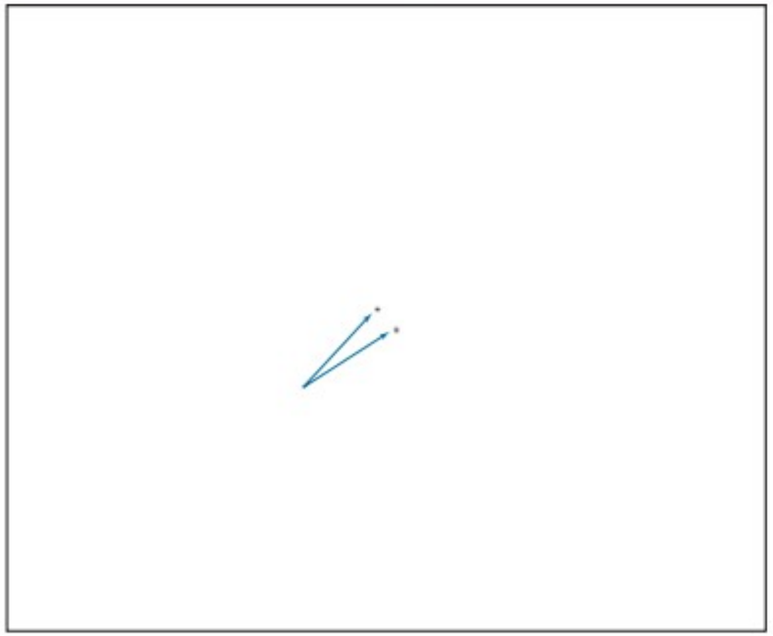
- Display log(1 + |G|) with the DC component centered. Bright off-center points usually indicate periodic contamination.
- **Look for** symmetric **peaks** with respect to the origin; a single physical sinusoid produces a pair of peaks.
- Ignore natural image low-frequency dominance near the center; focus on isolated spikes or unusually bright clusters.

- Peak picking may be **manual**, **threshold-based**, or **automated** after subtracting a smoothed spectral background.

Example



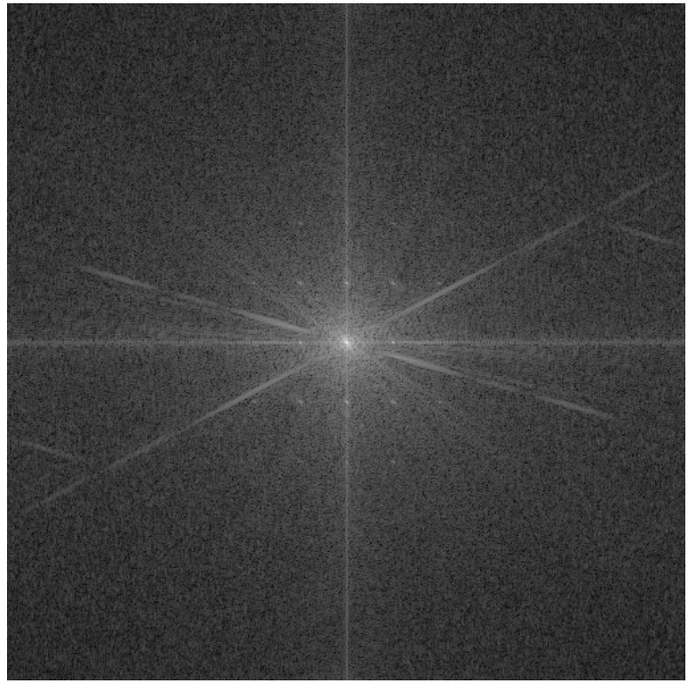




Example

Noisy input

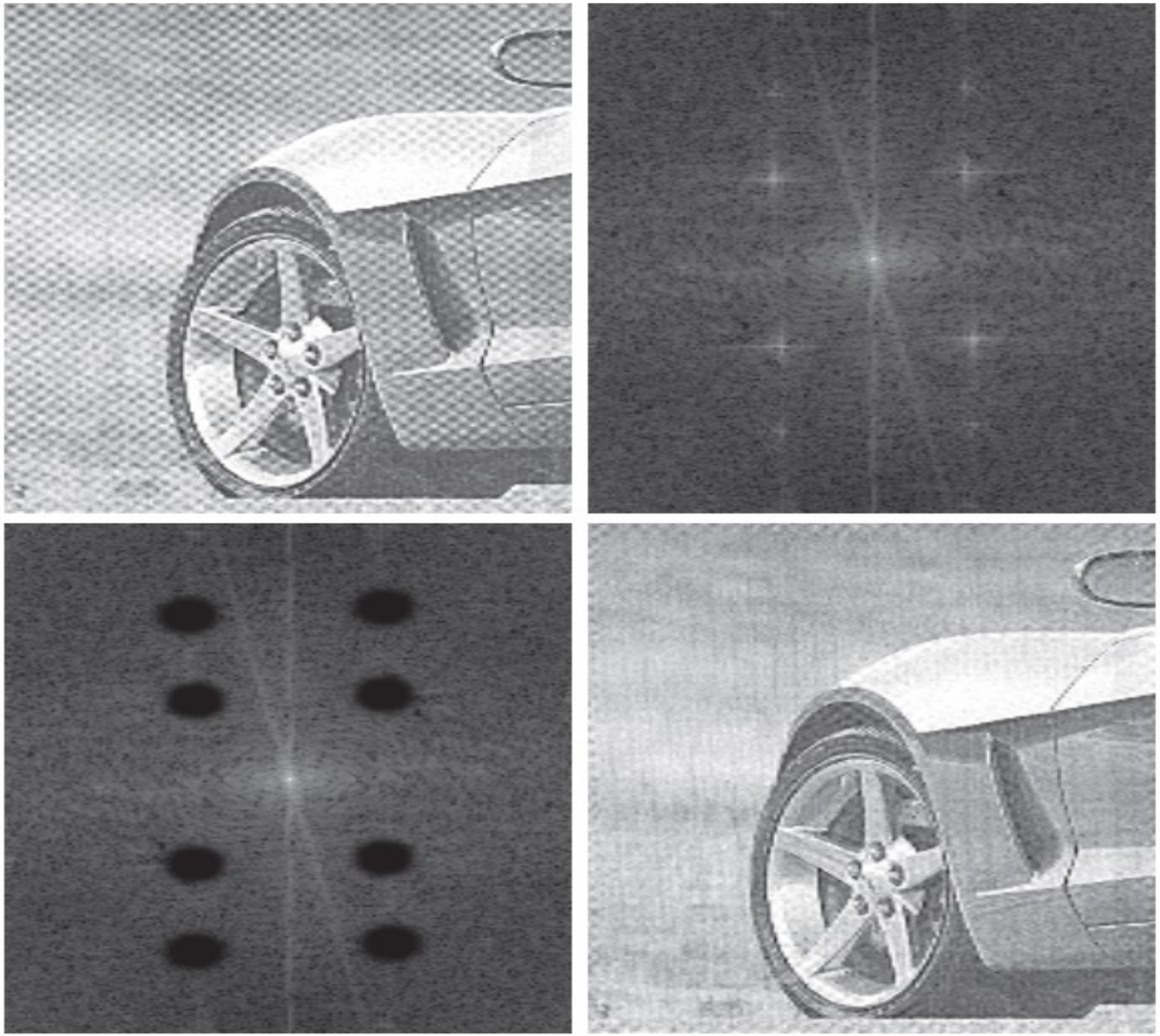
Basic notch result

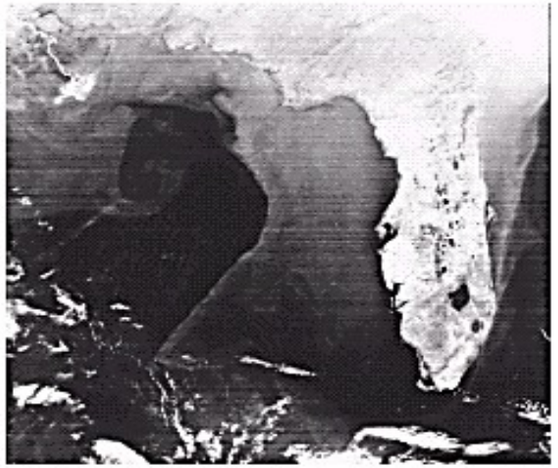


The periodic grid artifact is reduced, but an aggressive ideal notch can still leave residual structure or cause some ringing and contrast loss.

Butterworth notch

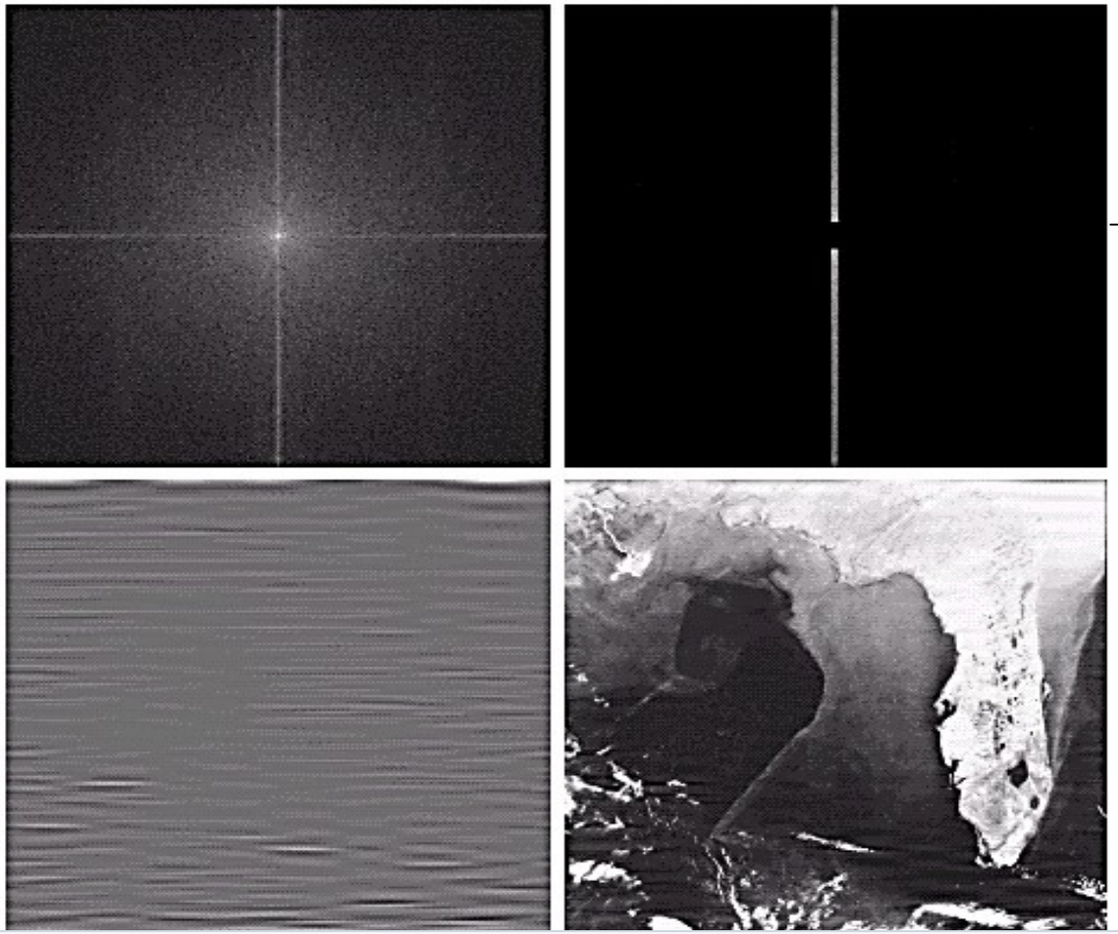
image showing a moiré pattern. (b) Spectrum. (c) Fourier transform multiplied by a Butterworth notch reject filter transfer function. (d) Filtered image.





a
b c
d e

FIGURE 5.19 (a) Satellite image of Florida and the Gulf of Mexico (note horizontal sensor scan lines). (b) Spectrum of (a). (c) Notch pass filter shown superimposed on (b). (d) Inverse Fourier transform of filtered image, showing noise pattern in the spatial domain. (e) Result of notch reject filtering. (Original image courtesy of NOAA.)



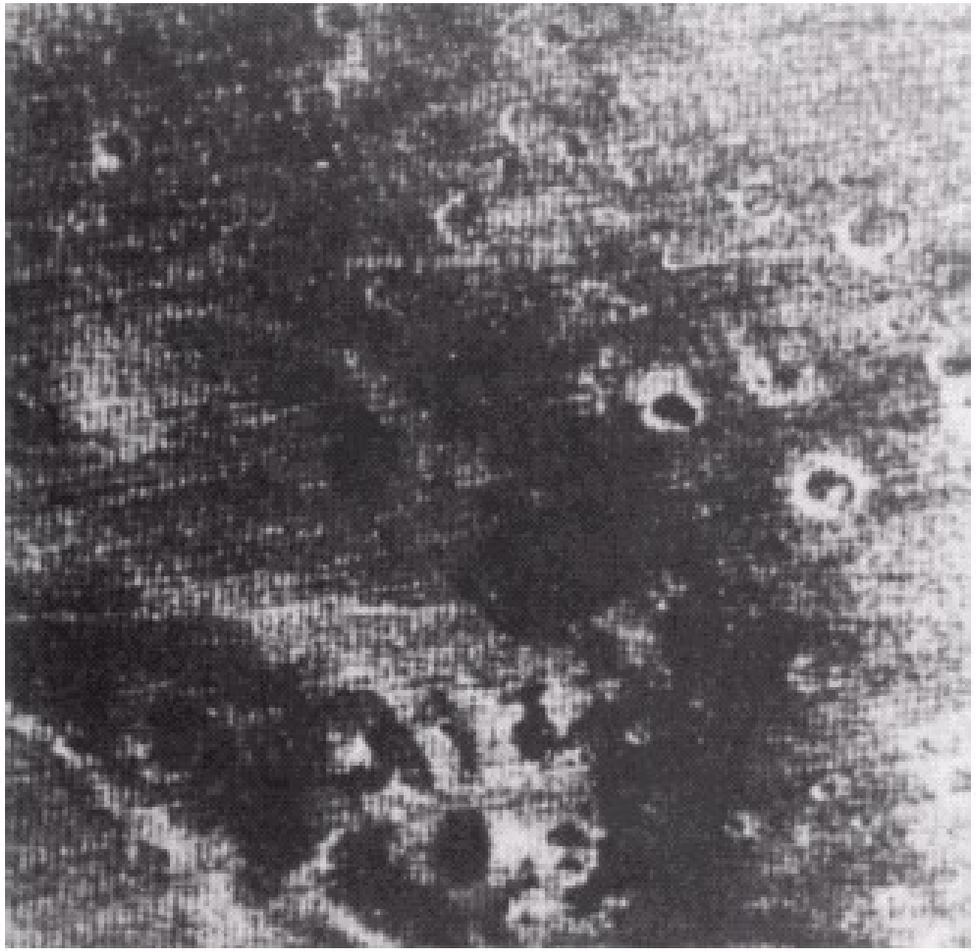
Vertical Notch pass filter

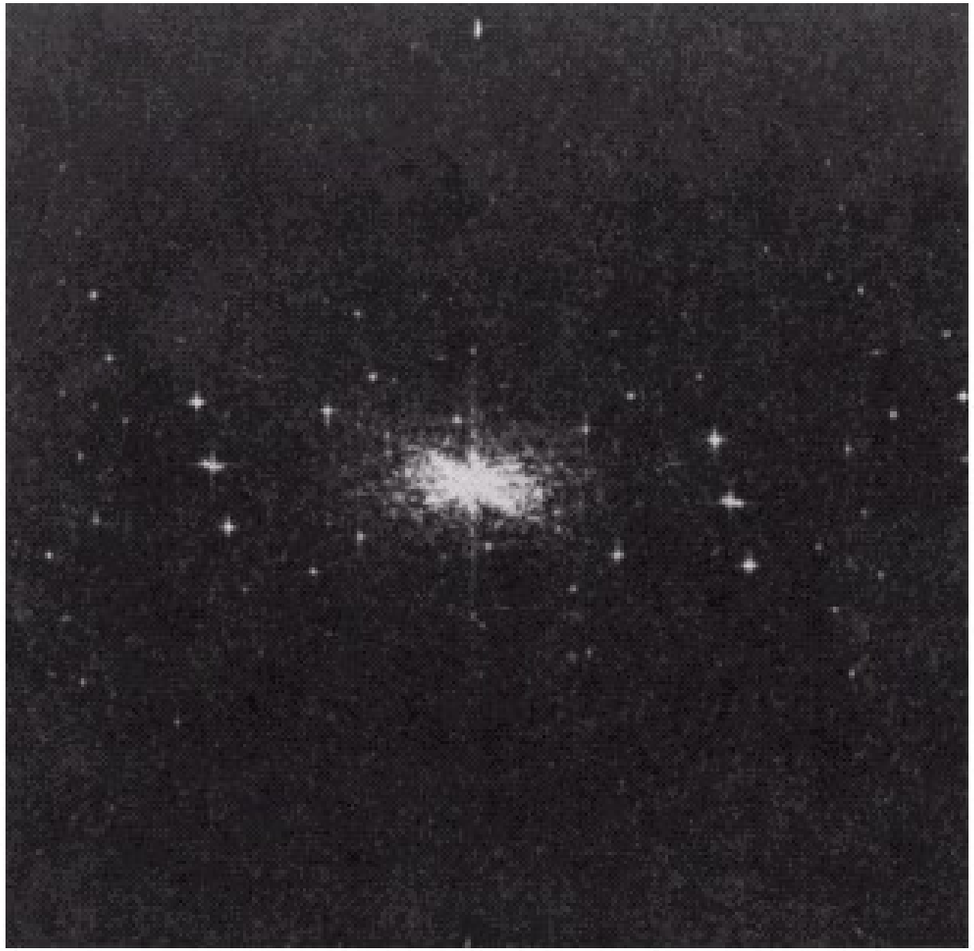
Horizontal lines of the noise pattern I can be seen

Optimum Notch Filtering

a b

FIGURE 5.20
(a) Image of the Martian terrain taken by *Mariner 6*.
(b) Fourier spectrum showing periodic interference.
(Courtesy of NASA.)





semi-periodic interference pattern that is considerably more complex

Why optimum notch filtering?

- Basic notch filters only delete spectral neighborhoods. They do not explicitly estimate the periodic interference component.
- An optimum notch approach estimates the interference in a **local neighborhood** and chooses parameters to minimize mean-square error.
- This often preserves nearby useful frequencies better, especially when the noise peak overlaps important image content.

$$g(x,y) = f(x,y) + \eta(x,y)$$

η(x,y): periodic interference component to be estimated and subtracted.

Optimum notch formulation

$$\widehat{N}(u,v) = H_{NP}(u,v)G(u,v)($$
$$\hat{\eta}(x,y) = \mathcal{F}^{-1}\{\widehat{N}(u,v)\}$$

$$\boxed{\hat{f}(x,y) = g(x,y) - w(x,y)\hat{\eta}(x,y)}$$

Choose w(u,v) to minimize $E\{\,|f(u,v) - \hat{f}(u,v)|^2\,\}$

$\hat{\eta}$: estimated periodic interference; w(u,v): local optimum weighting function; E{.}: expectation.

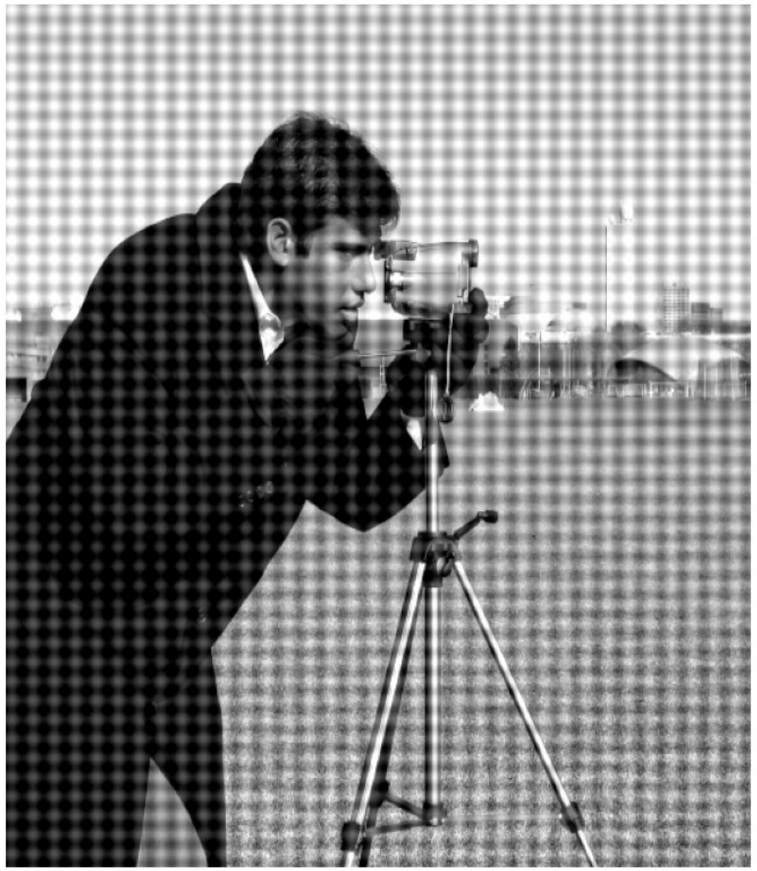
$$\boxed{w(x,y) = \frac{\overline{g\hat{\eta}} - \bar{g}\,\bar{\hat{\eta}}}{\overline{\hat{\eta}^2} - (\bar{\hat{\eta}})^2}}$$

all the means/averages are on a local neiborhood S_{xy}

- Instead of hard deletion, the method adaptively estimates how much of the observed spectrum belongs to noise.
- When the periodic component is modeled as a sinusoid, local least-squares estimation of amplitude and phase can be used.

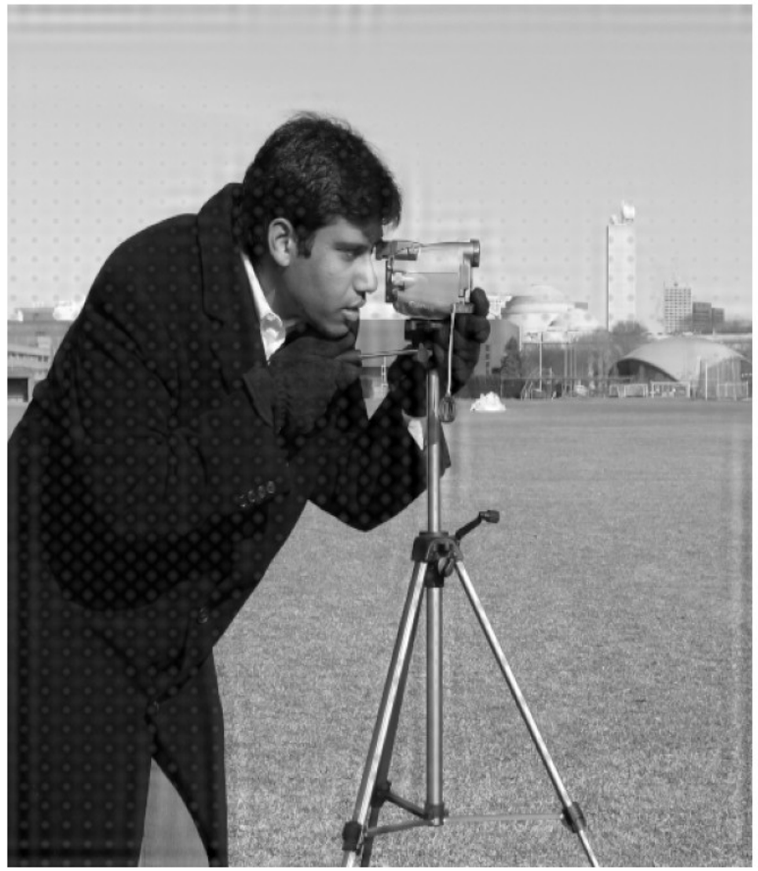
optimum notch filtering

Noisy input

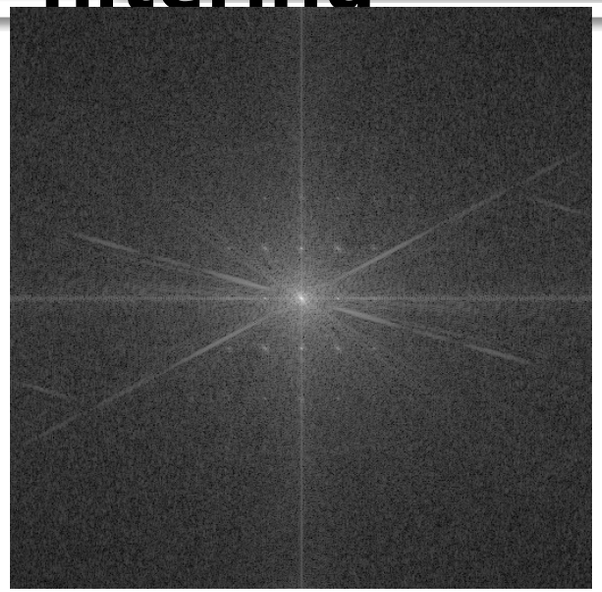


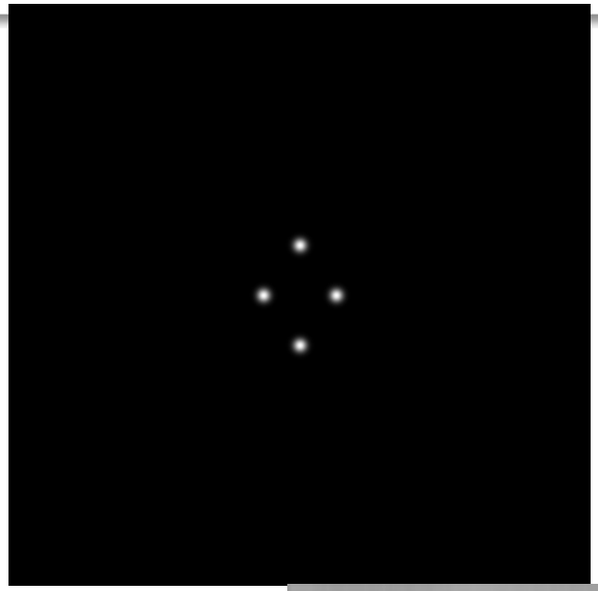
Basic notch result

Optimum/adaptive notch result

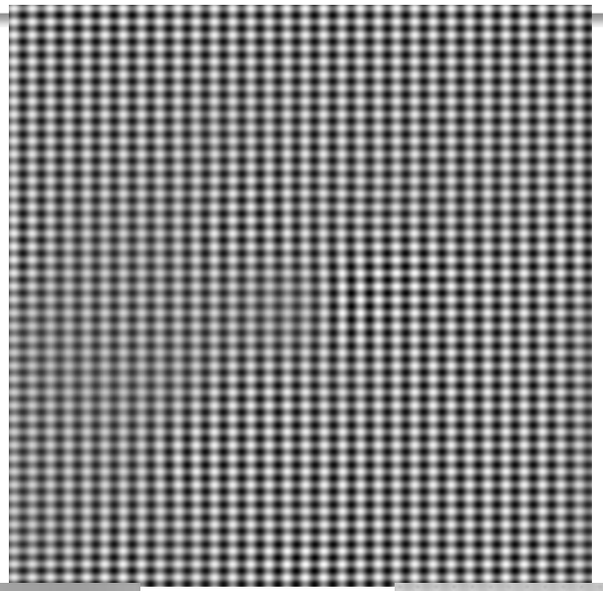


optimum notch filtering



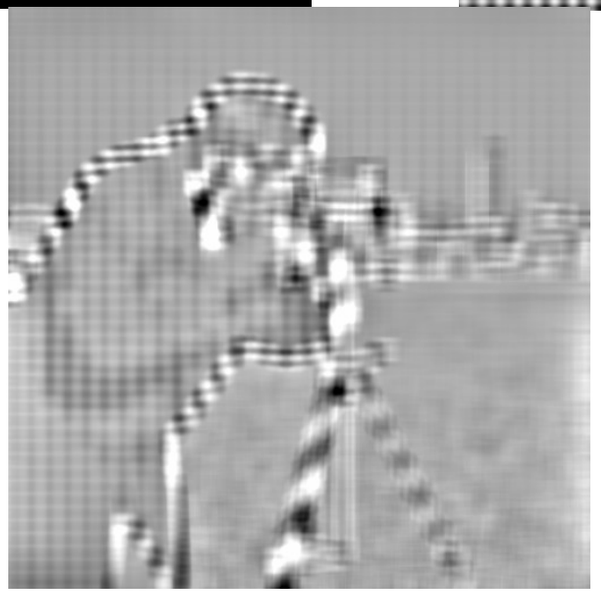


Pass filter



Noise estimate

Weight map



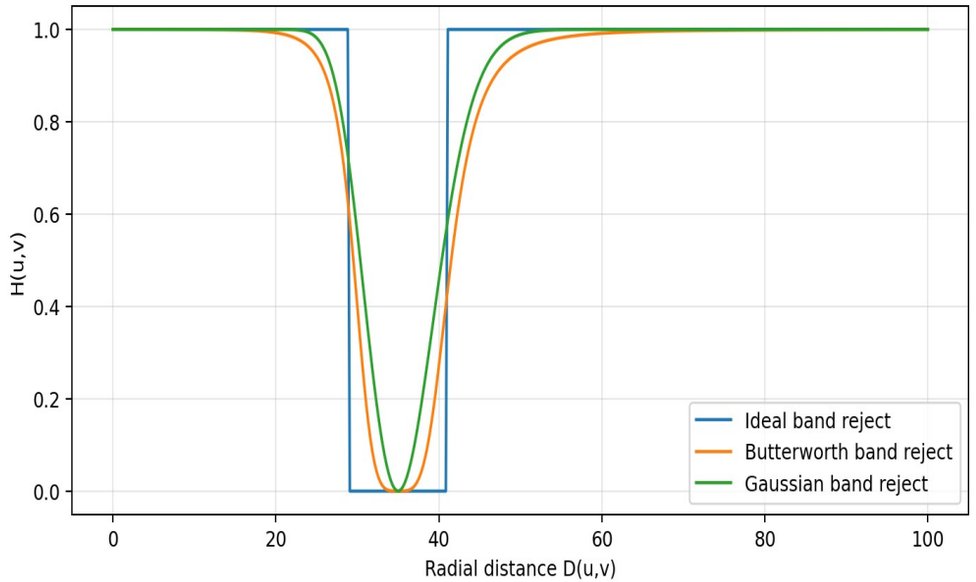
restored

Basic notch vs optimum notch

Aspect	Basic notch	Optimum notch
Main idea	Remove selected frequencies	Estimate + suppress periodic component
Complexity	Low	Moderate
Detail preservation	Can remove useful nearby detail	Usually better around noise peaks
Ringing tendency	Higher for ideal sharp notches	Lower when smooth/local estimation is used
When to prefer	Clear isolated peaks, quick solution	Overlapping peaks or quality-sensitive restoration

Band-reject filter: concept

- A band-reject filter suppresses an annulus or strip of frequencies instead of isolated points.
- It is useful when the interference occupies a narrow range of frequencies rather than a few discrete spikes.
- Examples: scan-line artifacts, repeated sensor stripe patterns, and narrow-band electronic interference.

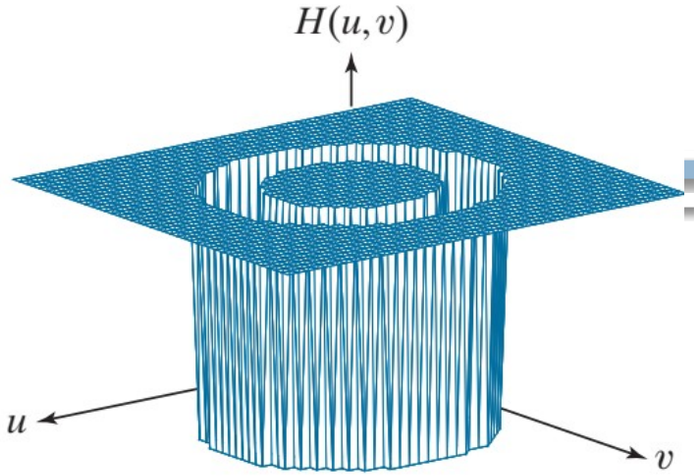


$$D(u,v) = \sqrt{u^2 + v^2}$$

D(u,v) is the radial distance from the frequency origin.

Ideal Band-reject Filter

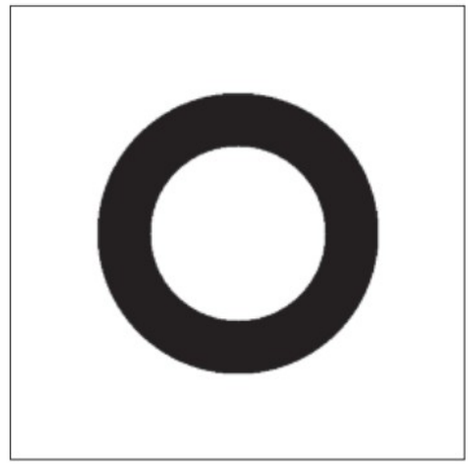
$$H(u,v) = 0 \quad \text{if} \quad D_0 - \frac{W}{2} \le D(u,v) \le D_0 + \frac{W}{2}$$
$$= 1 \quad \text{otherwise}$$



-$D(u,v)$ =distance from the origin of the centered freq. rectangle

-W =width of the band

-D_0=Radial center of the band.



- Rejects a full frequency ring or a narrow radial band.
- Effective when unwanted energy is spread over a band rather than concentrated in discrete peaks.
- Like ideal notch filters, sharp transitions can introduce ringing.

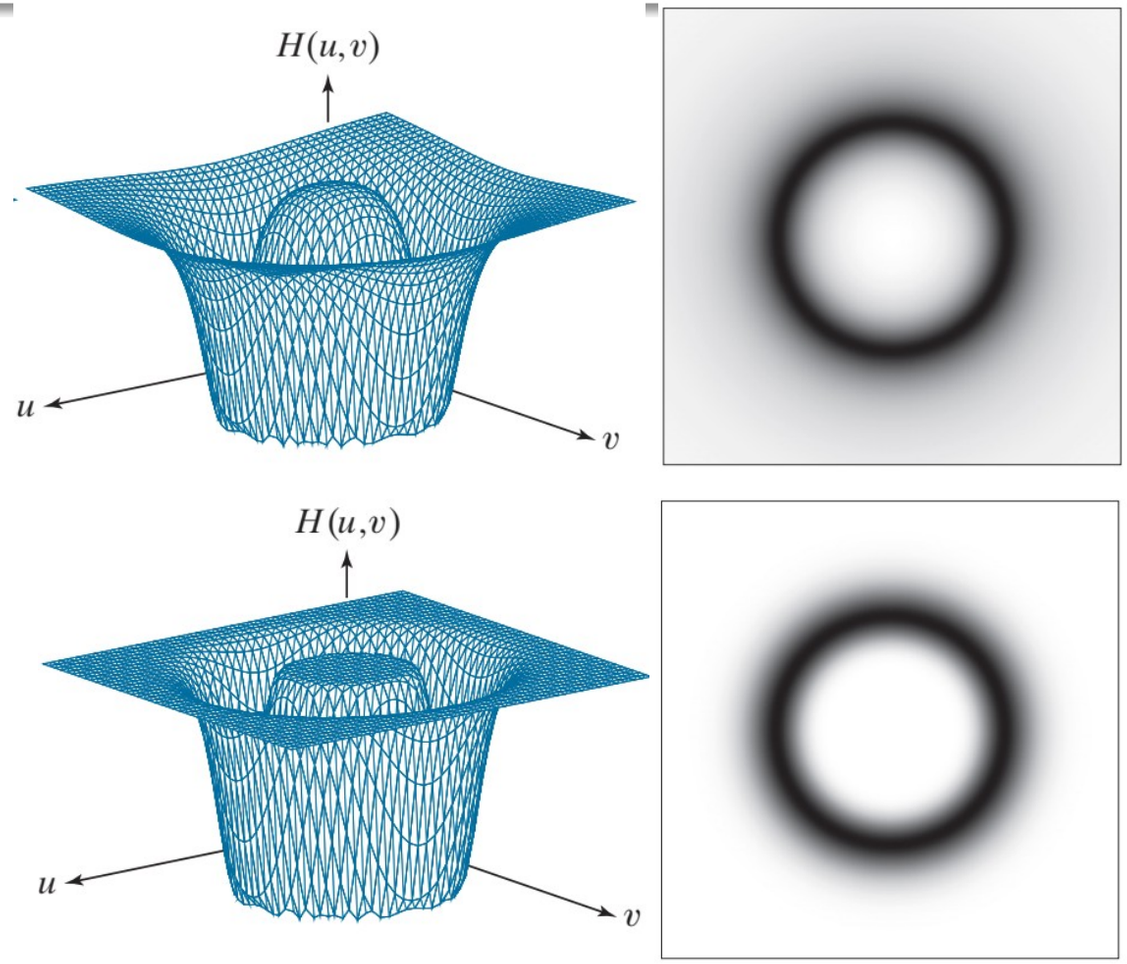
Butterworth and Gaussian band-reject filters

- Butterworth Filter of order *n*

$$H(u,v) = \frac{1}{1 + \left[\frac{D(u,v)W}{D^2(u,v) - D_0^2} \right]^{2n}}$$

- Gaussian Band-Reject Filter

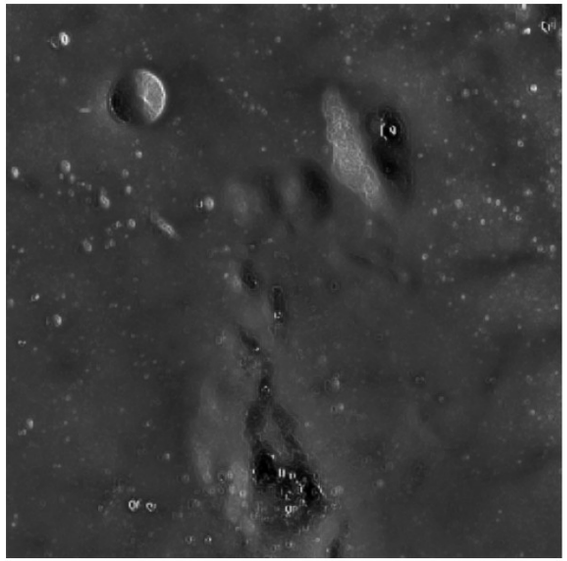
$$H(u,v) = 1 - e^{-\frac{1}{2}\left[\frac{D^2(u,v) - D_0^2}{D(u,v)W} \right]^2}$$



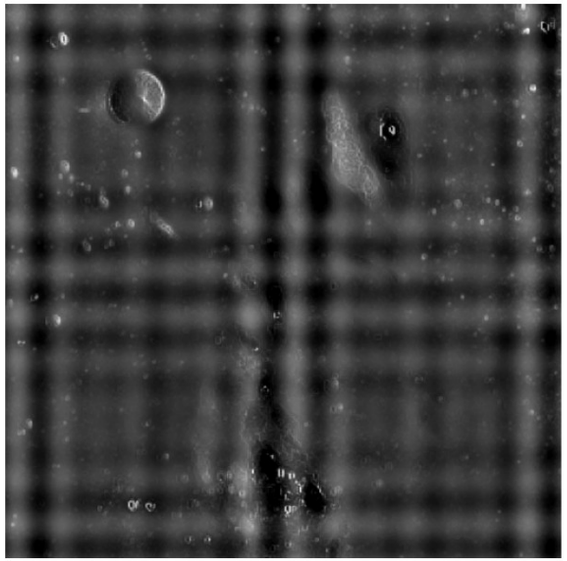
- Butterworth designs provide a tunable slope. Larger n makes the response closer to the ideal filter.
- Gaussian band-reject filters are smooth and often give visually cleaner results with less ringing.

Example: band-reject filtering on a satellite-like image

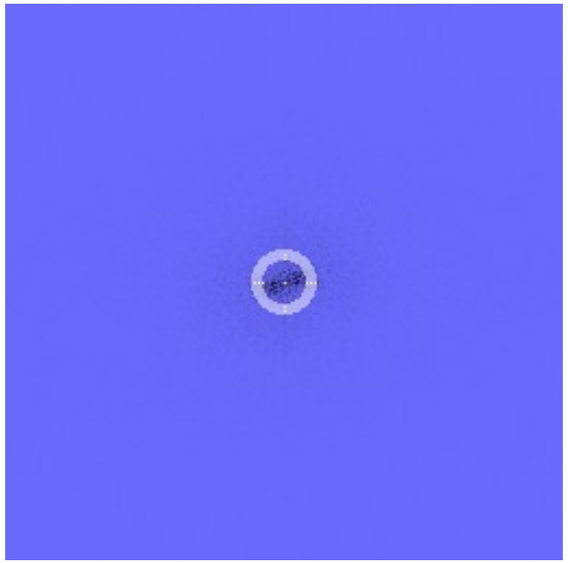
Original satellite-like image



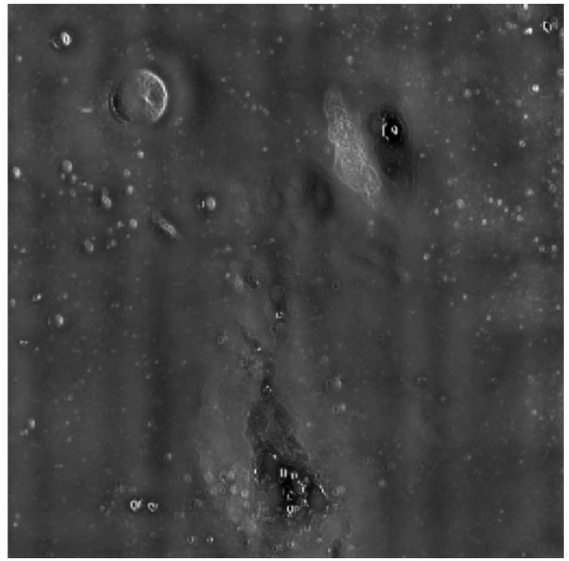
Narrow-band periodic noise



Centered log spectrum

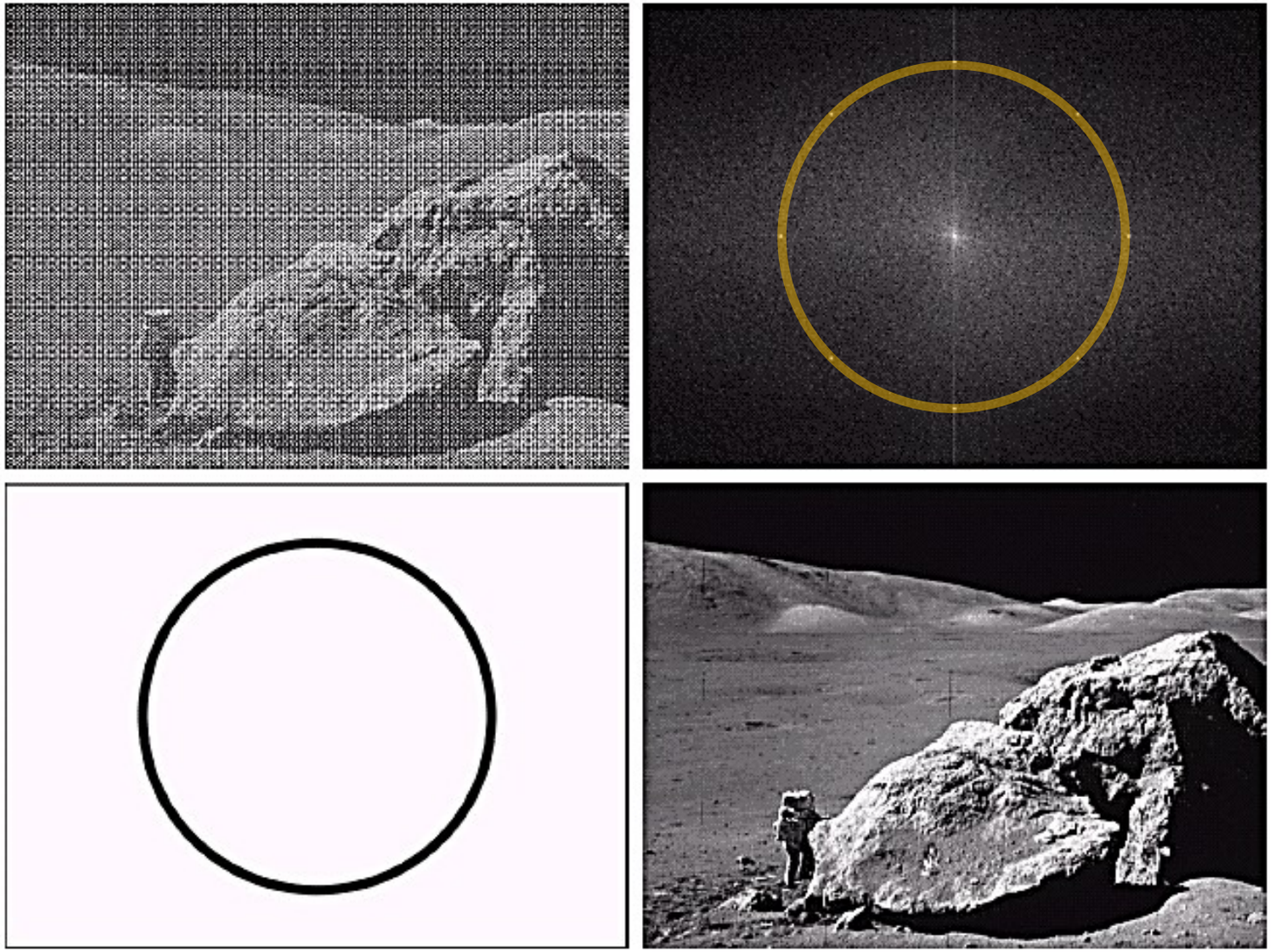


Band-reject restoration



- Narrow-band stripe interference is reduced by rejecting the contaminated frequency band while keeping most background detail intact.

Example: band-reject filtering



a b
c d

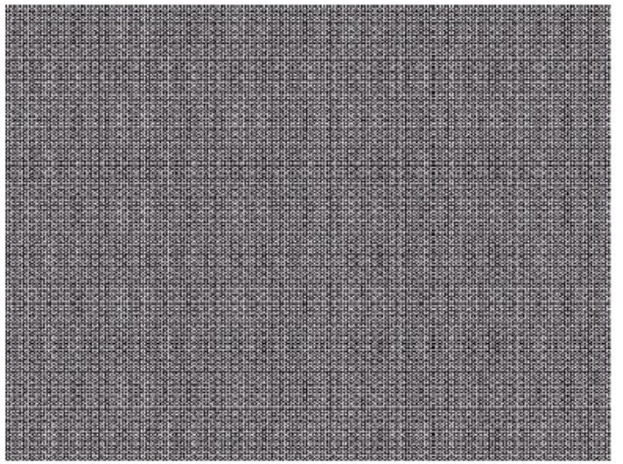
FIGURE 5.16
(a) Image corrupted by sinusoidal noise. (b) Spectrum of (a). (c) Butterworth bandreject filter (white represents 1). (d) Result of filtering. (Original image courtesy of NASA.)

Example: band-pass filtering

Band-Pass Filters

Opposite operation of a band-reject filter

FIGURE 5.17
Noise pattern of the image in Fig. 5.16(a) obtained by bandpass filtering.



$$H_{bp} = 1 - H_{br}(u,v)$$

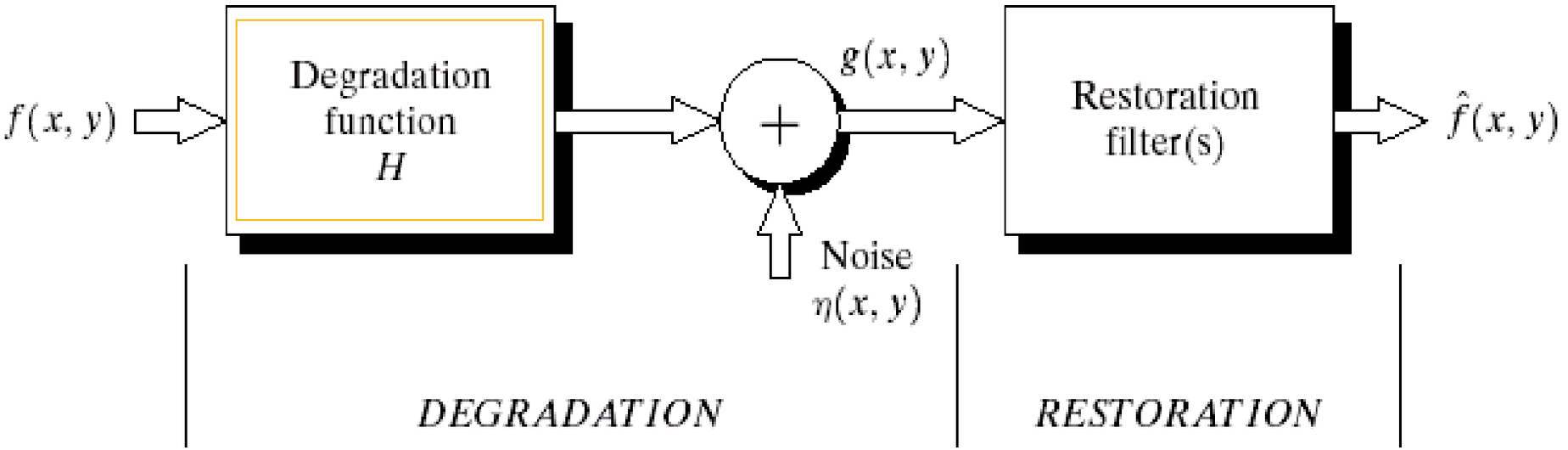
When to use notch filtering vs band-reject filtering

Situation	Better choice	Reason
A few isolated symmetric peaks	Notch reject	High selectivity, removes only those components
A whole narrow frequency interval is contaminated	Band reject	Removes an annulus/band rather than many individual notches
Need to inspect removed component	Notch pass / band pass	Diagnostic isolation of interference
Concern about ringing	Smooth Gaussian / low-order Butterworth	Avoids abrupt frequency cutoffs

Linear, Shift-Invariant Degradation

So far we consider additive noise only

A model of the image degradation /restoration process



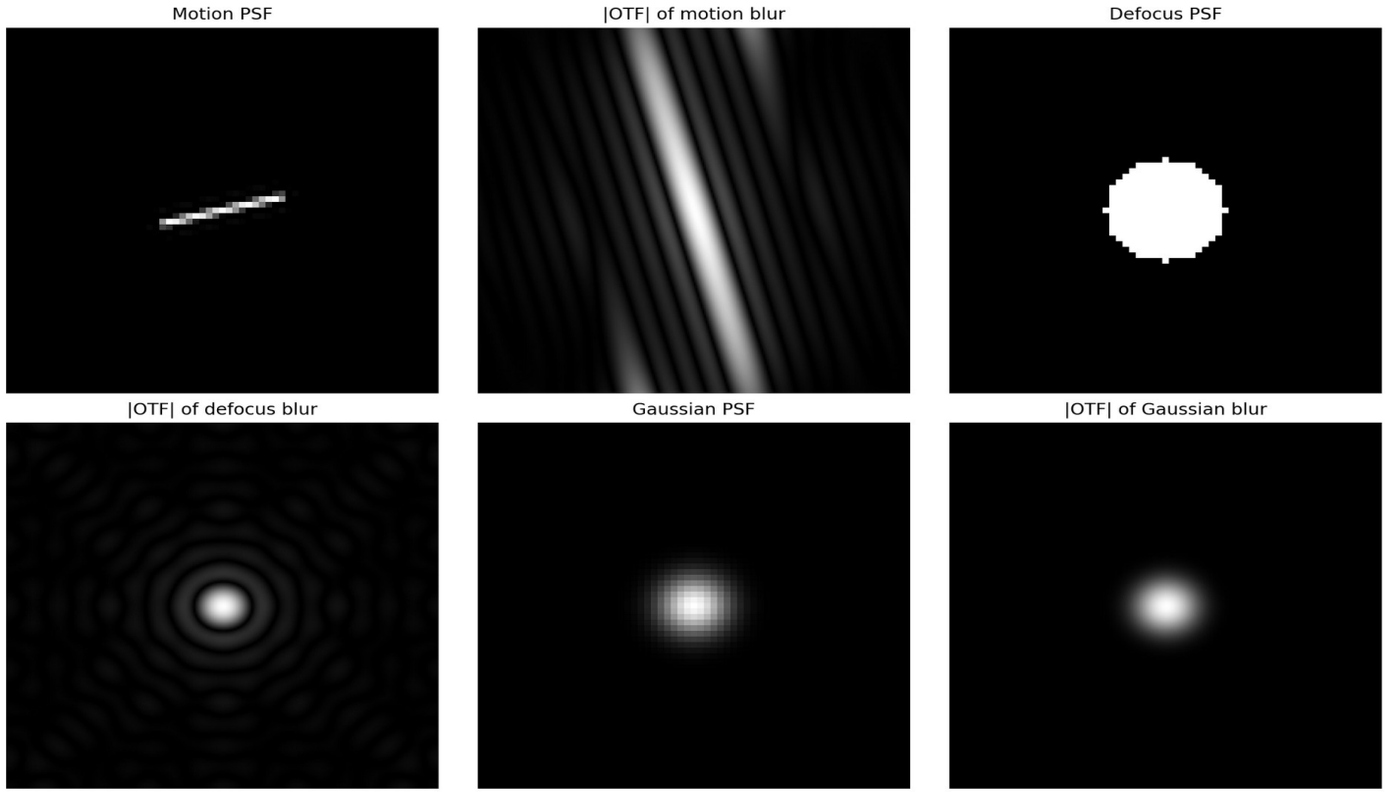
$$g(x,y)=f(x,y)*h(x,y)+\eta(x,y)$$

$$G(u,v)=F(u,v)H(u,v)+N(u,v)$$

Blur restoration fundamentals: PSF, OTF, and MTF

PSF: h(x,y) ; OTF: H(u,v) = FFT{ h(x,y) }
MTF(u,v) = |H(u,v)| ; PTF(u,v) = angle{ H(u,v) }

PSF describes the spread of a point source in space. OTF is the Fourier transform of the PSF. MTF measures how strongly each spatial frequency is transmitted.

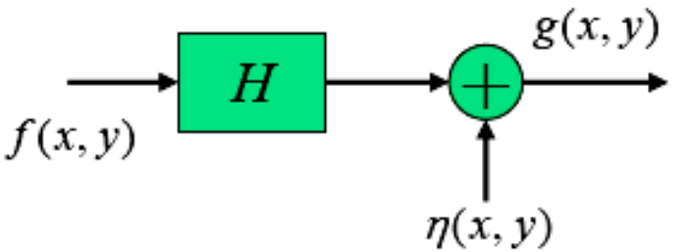


Estimation of degradation functions

- **When degradation function $H(u,v) \neq 1$, it needs to be estimated before image restoration**

$$g(x,y) = h(x,y) * f(x,y) + \eta(x,y)$$

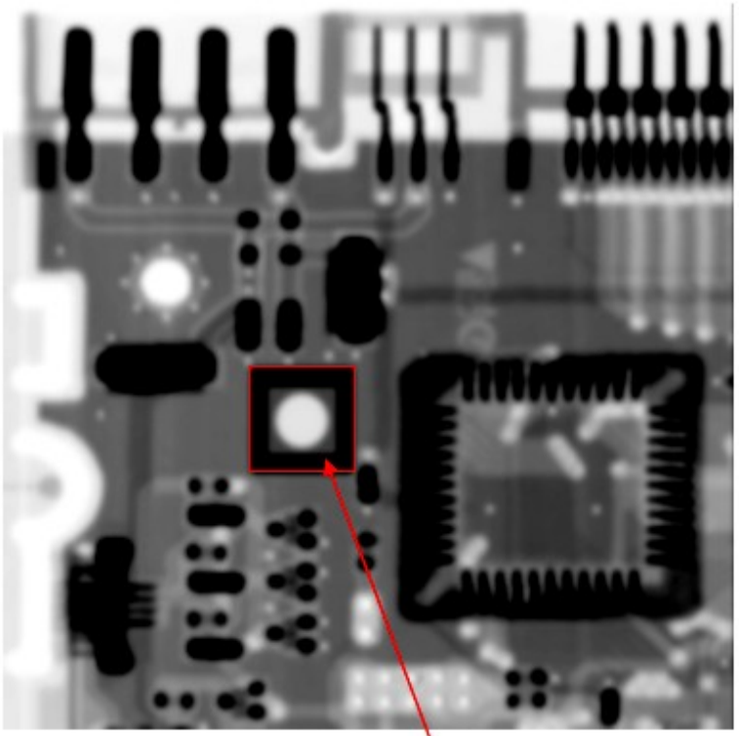
$$G(u,v) = H(u,v)F(u,v) + N(u,v)$$



- **Three ways**
  - Observation
  - Experimentation
  - Mathematical modeling

Estimation of degradation functions

- **Estimation by observation**
  - Given a **degraded image** without knowledge about H, one way to estimate H is to **gather information from the image** itself
  - For example, the given image has been undergone blurring
    - select one part with strong signal content (thus noise is ignored)
    - construct an unblurred image of the same size and characteristics
    - $H_s(u,v) = G_s(u,v) / \hat{F}_s(u,v)$



$g_s(x,y)$

High contrast area

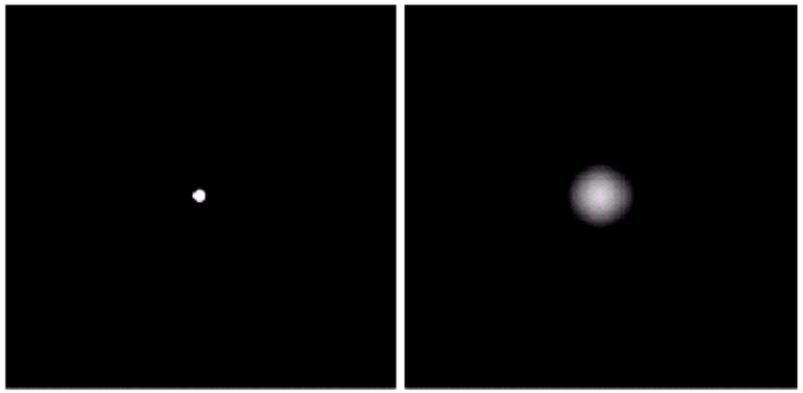
$\hat{f}_s(x,y)$

(unblurred image)

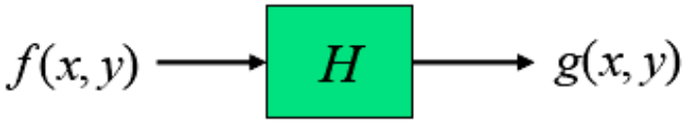
By sharpening filter or even manually

Estimation of degradation functions

- **Estimation by experimentation**
  - If equipment similar to the one used to acquire the degraded image $g(x,y)$ is available, an accurate estimation of H may be obtained
    - Take images similar to the degraded $g(x,y)$
    - Vary the system settings until output images are degraded as closely as possible to $g(x,y)$
    - Obtain the impulse response $g_i(x,y)$ of the system using the same settings
    - $H(u,v) = G_i(u,v)/A$



Impulse of strength A Impulse response $g_i(x,y)$



Estimation of degradation functions

- **Estimation by modeling**
  - Deriving mathematical models from basic principles
  - Based on the physical/environmental characteristics
  - For example, modeling atmospheric turbulence with

$$H(u,v) = e^{-k(u^2+v^2)^{5/6}}$$

Estimation of degradation functions

Estimating by Modeling

Atmospheric turbulence model $H(u,v) = e^{-k(u^2+v^2)^{5/6}}$

origin
al

High
turbulence
k=0.0025

Mid
turbulence
k=0.001

Low
turbulence
k=0.00025

Estimation of degradation functions (model bases)

Model for motion blur

- Mathematical model for uniform linear motion between the image and the sensor during image acquisition
  - Let $x_0(t)$ and $y_0(t)$ denote time-varying components of motion in the x- and y-directions
  - Degradation model

$$g(x,y) = \int_0^T f\left[x - x_0(t), y - y_0(t)\right]dt \quad \text{where} \quad g(x,y) \text{ is the blurred image}$$

$$G(u,v) = F(u,v)\int_0^T \exp^{-j2\pi\left[ux_0(t)+vy_0(t)\right]}\,dt \quad \text{where}$$

$$H(u,v) = \int_0^T e^{-j2\pi\left[ux_0(t)+vy_0(t)\right]}dt \quad \text{is degradation function}$$

Model for motion blur

- If $x_0(t)=\alpha t/T$ and $y_0(t)=0$, then:

$$H(u,v)=\int_0^T e^{-j2\pi(ux_0(t))}dt=\int_0^T e^{-j2\pi u\alpha t/T}dt=$$

$$=\frac{T}{-j2\pi ua}e^{-j2\pi ua t/T}\Big|_0^T=$$

$$\frac{T}{-j2\pi ua}\left[e^{-j2\pi ua}-1\right]=-\frac{T}{j2\pi ua}\left[e^{-j\pi ua}e^{j\pi ua}e^{-j\pi ua}-e^{-j\pi ua}\right]$$

$$=\frac{T}{\pi ua}\underbrace{\frac{1}{2j}\left[e^{j\pi ua}-e^{-j\pi ua}\right]}_{\sin}e^{-j\pi ua}=$$

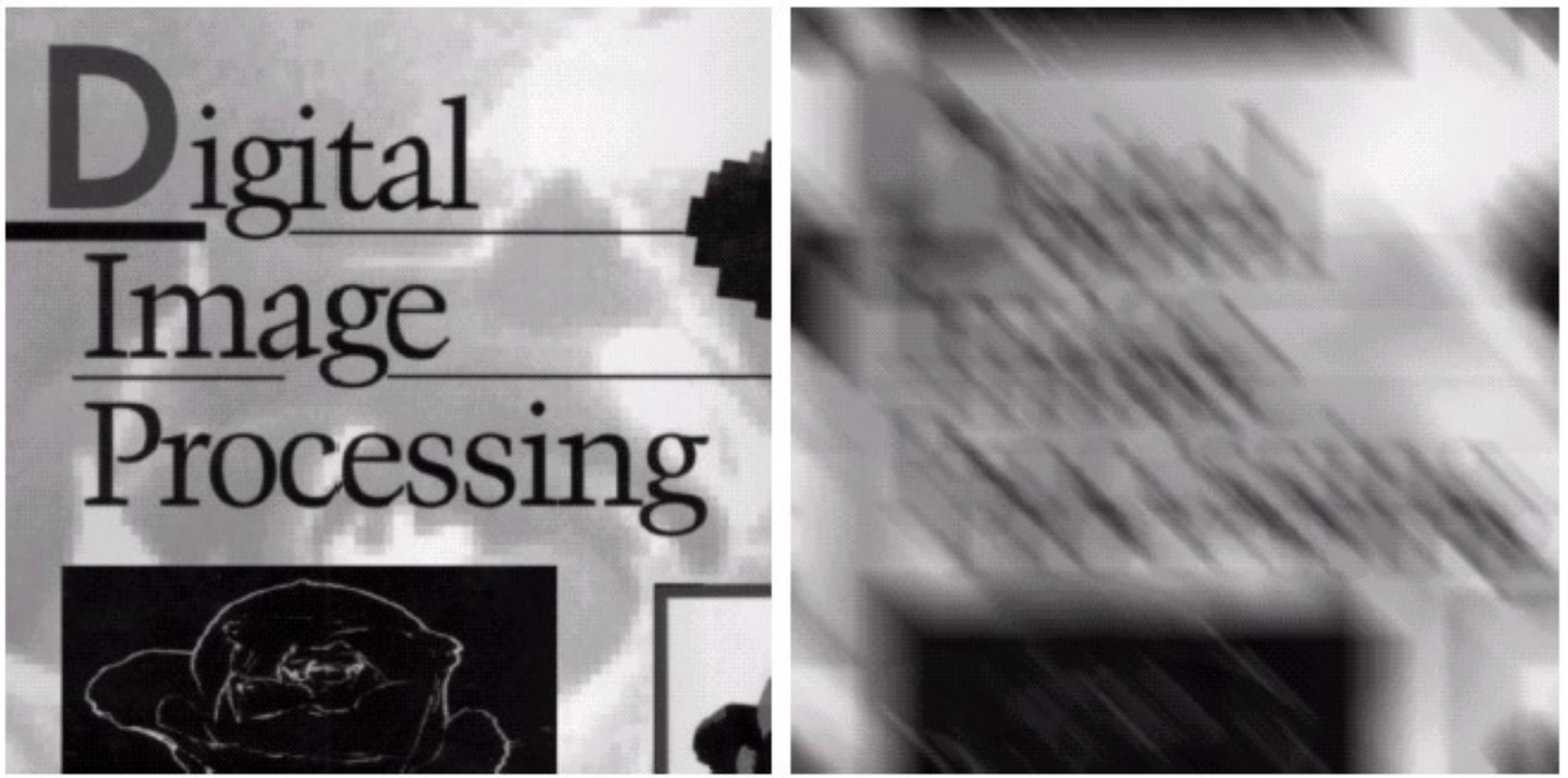
$$=\frac{T}{\pi ua}\sin(\pi ua)\,e^{-j\pi ua}$$

Estimation of degradation functions (model bases)

- Uniform Linear motion in the *x* and y direction $x_0(t)=at/T$ and $y_0(t)=bt/T$
  - where the image has been displaced by a total distance *a* in the x-direction and *b* in the y-direction

$$H(u,v)=\frac{T}{\pi(ua+vb)}\sin\left[\pi(ua+vb)\right]e^{-j\pi(ua+vb)}$$

Estimation of degradation functions (model bases)



a b

FIGURE 5.26 (a) Original image. (b) Result of blurring using the function in Eq. (5.6-11) with $a = b = 0.1$ and $T = 1$.

Estimation of degradation functions

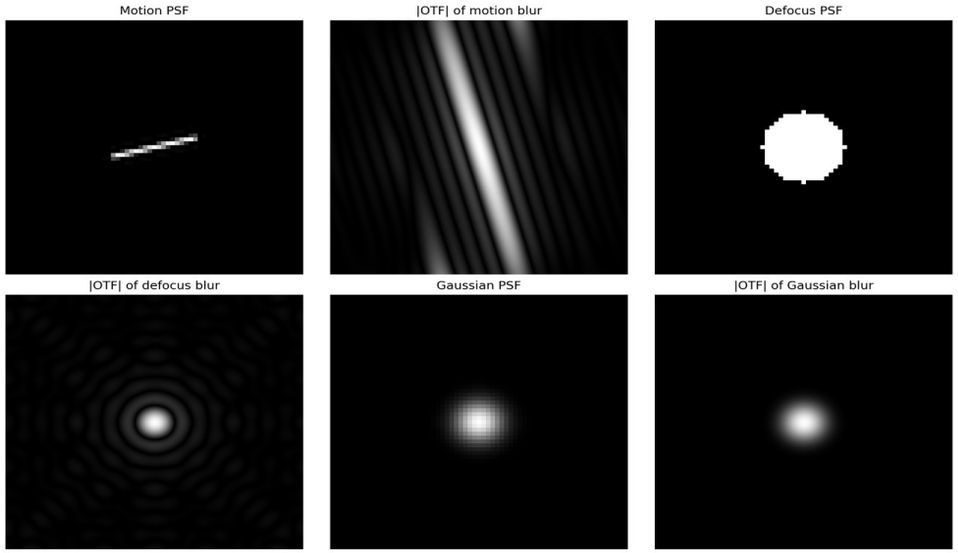
Estimation Category	Best Used When...	Major Limitation
Experimental	You have physical control over the camera/optics	Impractical for real-world photos taken past-tense
Image Observation	High-contrast geometric edges exist in the scene	Highly susceptible to image noise
Physical Modeling	Motion direction, exposure time, or turbulence is known	Simplistic; misses non-linear camera aberrations
Blind Deconvolution	Blur cause is entirely unknown	Computationally heavy; prone to local minima

From PSF to OTF and MTF

1) Zero-pad h(x,y) to the image size
2) Circularly center the PSF at the origin
3) Compute H(u,v) = FFT{ h(x,y) }
4) Compute MTF = |H| and PTF = angle(H)

Proper centering is important. A mis-centered PSF adds unwanted phase ramps and degrades deconvolution results.

- The OTF fully characterizes a linear shift-invariant blur in frequency domain.
- The MTF shows how much contrast each spatial frequency retains after imaging.

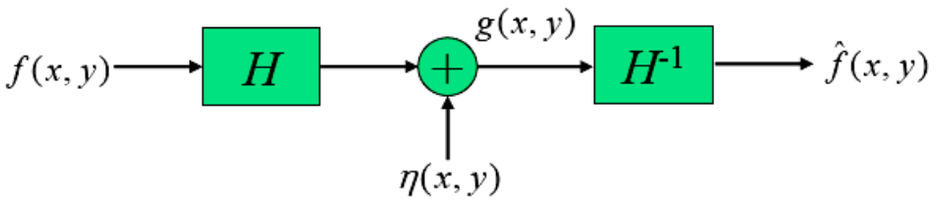


Inverse filtering

- **When $H(u,v)$ is known, the simplest approach to restoration is direct inverse filtering**

$$\hat{F}(u,v) = \frac{G(u,v)}{H(u,v)}$$

$$\hat{F}(u,v) = F(u,v) + \frac{N(u,v)}{H(u,v)}$$



- Inverse filtering tries to undo the blur exactly in the frequency domain.
- **Problem**: if |H(u,v)| is small or zero, the division greatly amplifies noise and numerical errors.

- Therefore pure inverse filtering is mainly a conceptual baseline.

Assumptions: the PSF/OTF is accurately known and **noise is negligible**. In practice these assumptions are often violated.

Inverse Filtering - Practical Issues

(degradation model)

$$G(u,v) = H(u,v)F(u,v) + N(u,v)$$

(inverse filtering estimate)

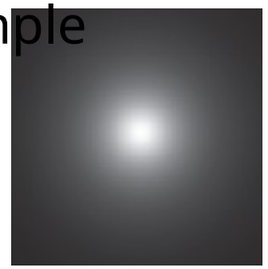
$$\hat{F}(u,v) = G(u,v)/H(u,v)$$
$$= F(u,v) + \frac{N(u,v)}{H(u,v)}$$

(inverse filtering estimate)

$$\hat{F}(u,v) = F(u,v) + \frac{N(u,v)}{H(u,v)}$$

When H(u,v) is close to zero (i.e., at higher frequencies), then **N(u,v)/H(u,v)** will dominate the estimate!

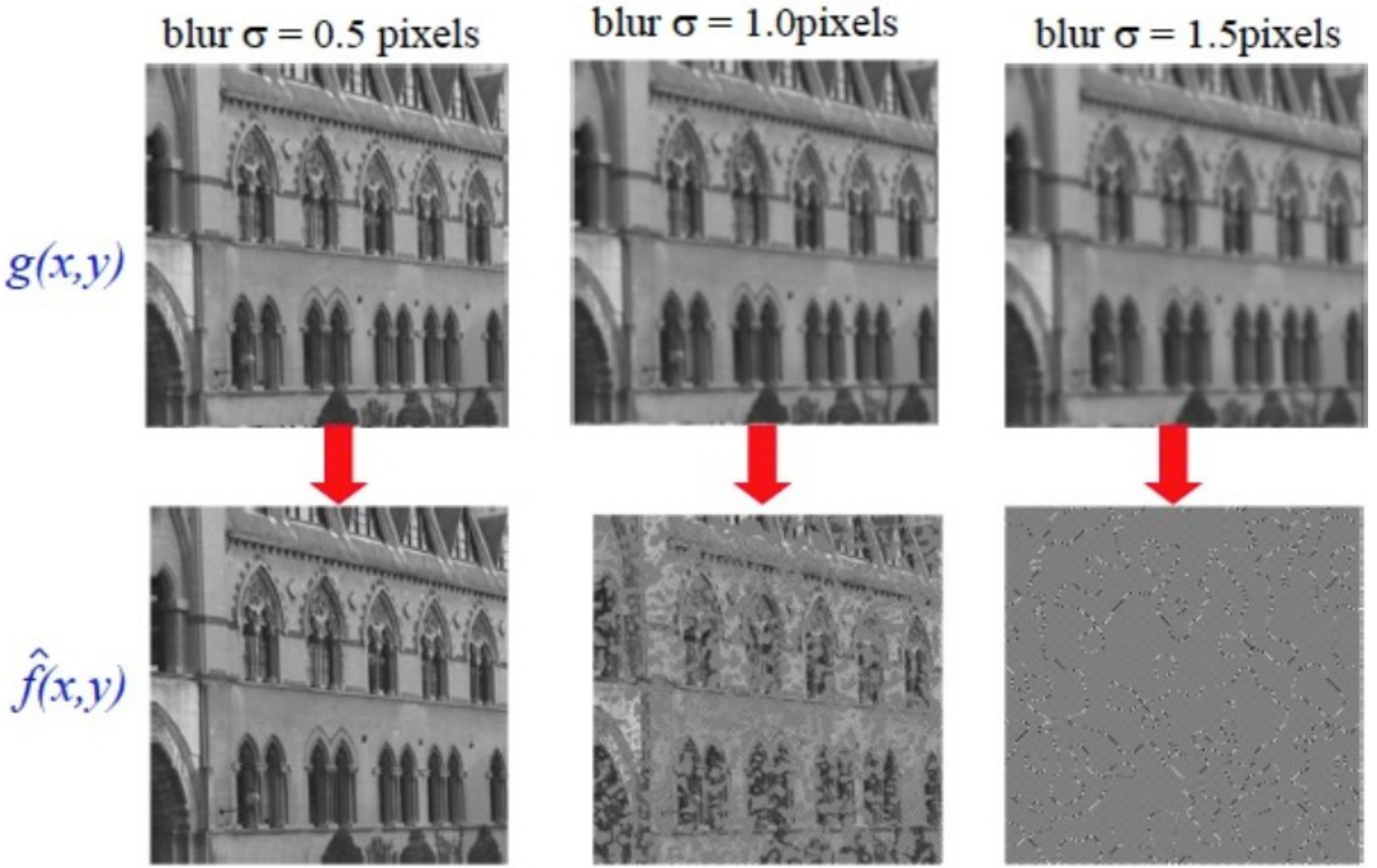
typical H(u,v) - example



Inverse Filtering - Practical Issues

noise σ = 0.3 grey levels

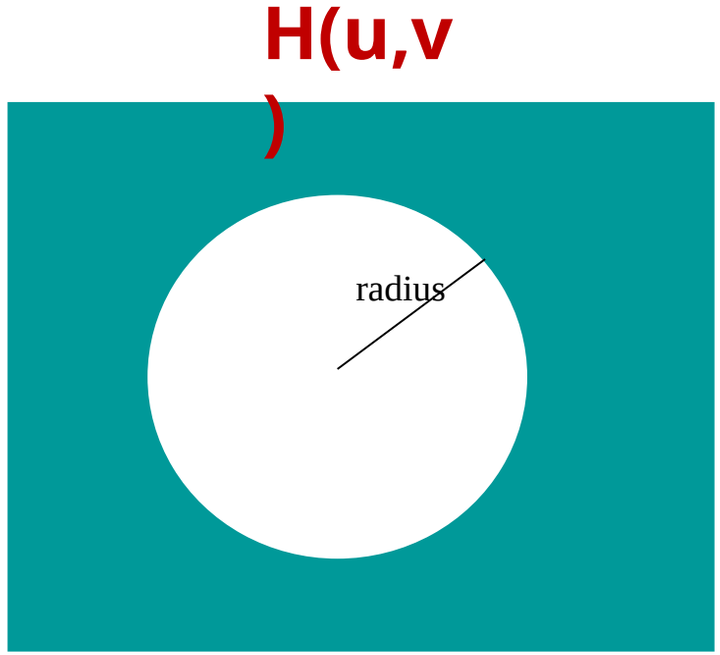
$\hat{F}(u,v) = G(u,v) / H(u,v)$

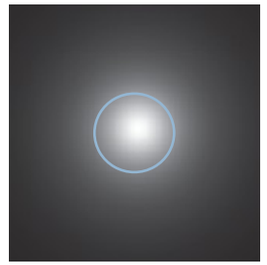


Pseudo-inverse / Radially Limited inverse filtering

Idea: carry out the restoration in a limited neighborhood about the origin to avoid small values of H(u,v).

$$\hat{F}(u,v) = \begin{cases} G(u,v)/H(u,v) & \text{if } \sqrt{u^2+v^2} \le R \\ 0 & \text{otherwise} \end{cases}$$





Inverse filtering

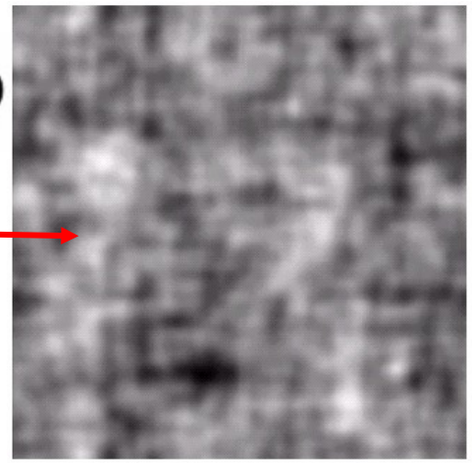
Degraded Image

Degradation function

$$H(u,v) = e^{-k\left[(u-M/2)^2+(v-N/2)^2\right]^{5/6}}$$

$$\frac{G(u,v)}{H(u,v)} = \hat{F}(u,v) \Leftrightarrow \hat{f}(x,y)$$

Full inverse Filtering



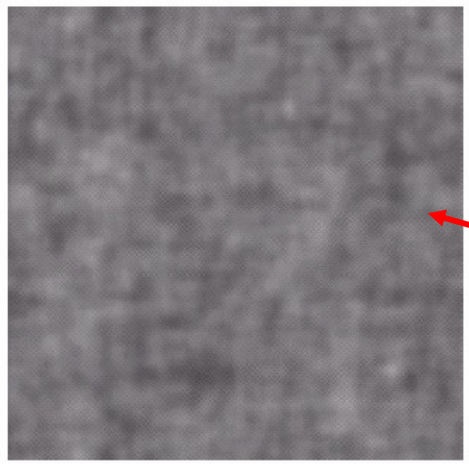
Filtering with H cut off outside a radius of 40

Butterworth filter H_b

$n = 10$

Filtering with H cut off outside a radius of 70

$$\frac{G(u,v)}{H(u,v)} H_b(u,v) = \hat{F}(u,v) \Updownarrow \hat{f}(x,y)$$



Filtering with H cut off outside a radius of 85

Pseudo-inverse / truncated inverse filtering

$$\widehat{F}(u,v) = \left[\frac{H^*(u,v)}{|H(u,v)|^2 + \varepsilon}\right] G(u,v)$$

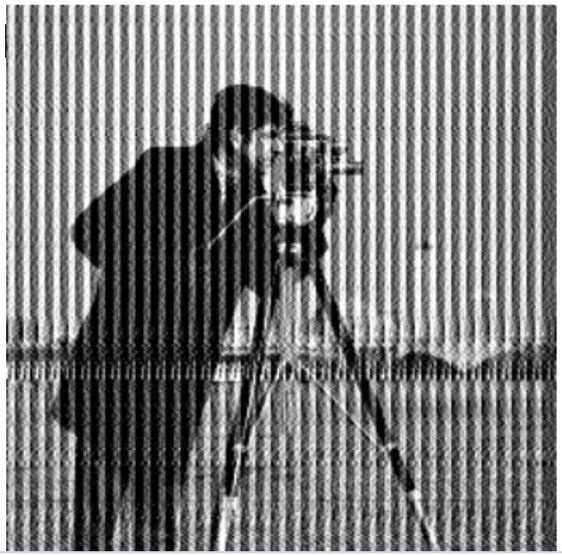
or set $\widehat{F}(u,v) = 0$ whenever $|H(u,v)| < T$

H*: complex conjugate of H; ε : small stabilizer; T: threshold.
These variants reduce catastrophic noise amplification near OTF zeros.

blurred image

- More stable than direct inverse filtering, but still sensitive to noise and PSF mismatch.
- Often used when blur is k ow to moderate.

inverse

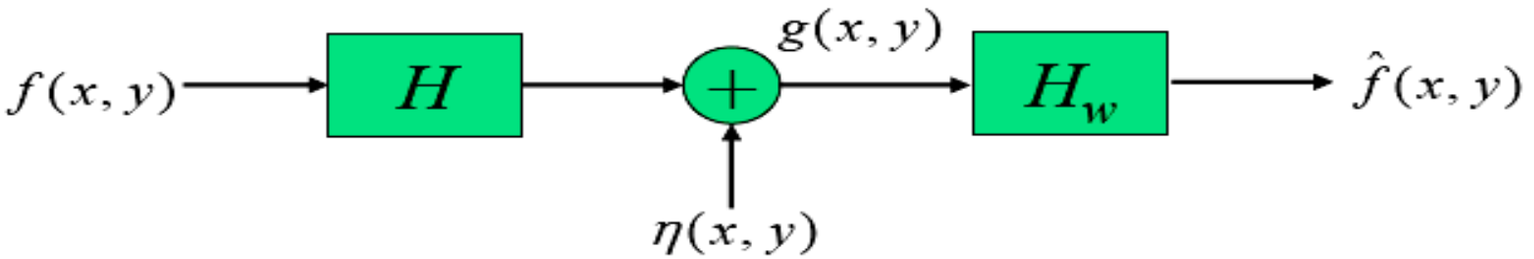


Pseudo-inverse, T = 0.1

Wiener filtering

- **Main limitation of inverse filtering**
  - Very sensitive to noise
- **Wiener filtering (minimum mean square error filtering)**
  - Use **statistic information** about signal and noise to improve the restoration
  - Consider images and noise as **random processes**
  - Objective:

$$e^2 = E\{(f - \hat{f})^2\} \Rightarrow \min$$



Wiener filtering

- **Assumptions**
  - The noise and the image are uncorrelated
  - One or the other has zero mean

  intensity levels in the estimate are a linear function of the levels in the degraded image.
- **Estimated image**

$$\min\{e^2\} \Rightarrow \hat{F}(u,v)$$

$$\hat{F}(u,v) = \left[\frac{1}{H(u,v)}\frac{|H(u,v)|^2}{|H(u,v)|^2 + S_\eta(u,v)/S_f(u,v)}\right]G(u,v)$$

$|H(u,v)|^2 = H^*(u,v)H(u,v)$ $H(u,v)$ is the degradation function

$S_\eta(u,v) = |N(u,v)|^2$: power spectrum of the noise

$S_f(u,v) = |F(u,v)|^2$: power spectrum of the original image

$(\eta(x,y) \Leftrightarrow N(u,v), f(x,y) \Leftrightarrow F(u,v))$

Wiener filtering

- **Wiener filter**

$$H_w(u,v) = \frac{1}{H(u,v)} \frac{|H(u,v)|^2}{|H(u,v)|^2 + S_\eta(u,v)/S_f(u,v)}$$

- **When noise is zero** $(N(u,v) = 0)$

$$H_w(u,v) = \frac{1}{H(u,v)} \Rightarrow \text{the inverse filter}$$

- **When** $S_\eta(u,v)$ **and/or** $S_f(u,v)$ **are unknown**

$$H_w(u,v) \approx \frac{1}{H(u,v)} \frac{|H(u,v)|^2}{|H(u,v)|^2 + K}$$

White noinse

some constant

Very crude approximation but it works reasonably well in practice!

Wiener filtering

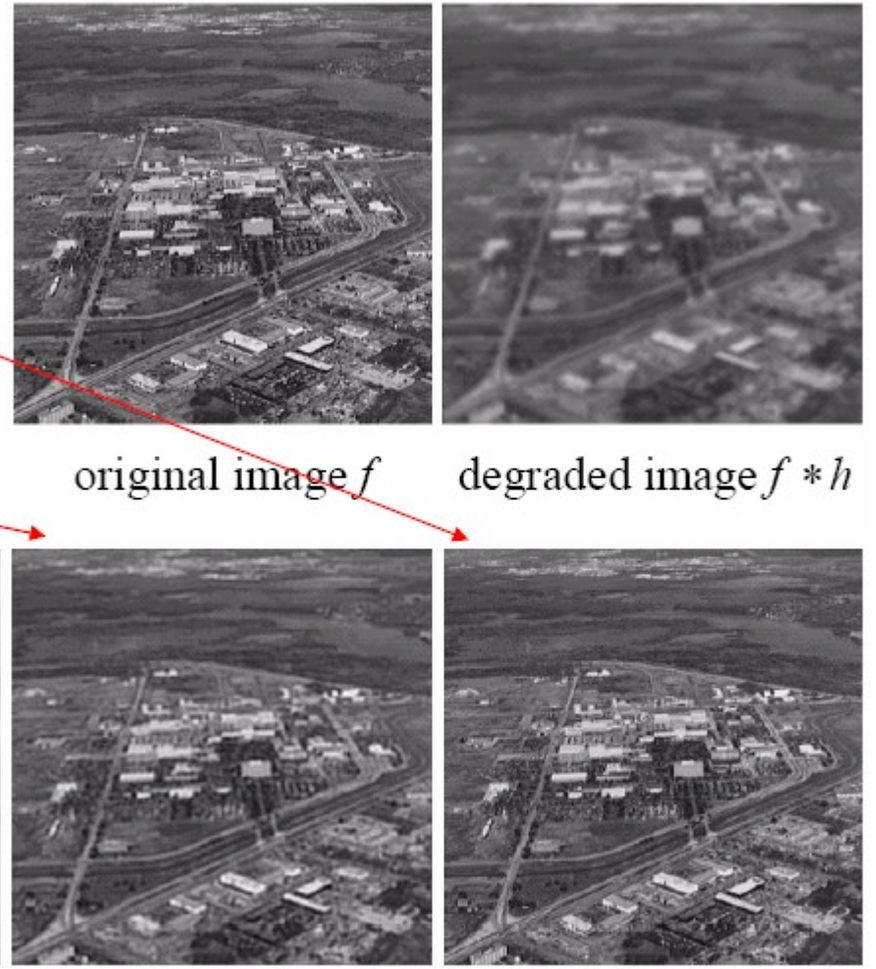
- **An example**

$$\frac{G(u,v)}{H(u,v)}\frac{|H(u,v)|^2}{|H(u,v)|^2+K}=\hat{F}(u,v)\Leftrightarrow\hat{f}(x,y)$$

$$\frac{G(u,v)}{H(u,v)}H_b(u,v)=\hat{F}(u,v)\Leftrightarrow\hat{f}(x,y)$$

$$H(u,v)=e^{-k(u^2+v^2)^{5/6}}$$

$$\hat{f}(x,y)\Leftrightarrow\frac{G(u,v)}{H(u,v)}=\hat{F}(u,v)$$

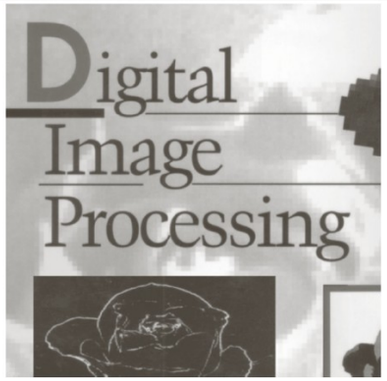


original image f degraded image $f * h$

a b c

FIGURE 5.28 Comparison of inverse- and Wiener filtering. (a) Result of full inverse filtering of Fig. 5.25(b). (b) Radially limited inverse filter result. (c) Wiener filter result.

Linear motion Wiener filter



Reduced noise variance

Motion blur and additive Gaussian noise
μ=0
σ=650

One order of magnitude less noise
μ=0
σ=65

Five order of magnitude less noise

Inverse filtering Wiener filtering

Wiener filtering - Derivation

$$G = HF + N.$$

We want an estimate

$$\hat{F}(u,v) = W(u,v)G(u,v)$$

where $W(u,v)$is the Wiener filter.

minimizes the expected squared error:

$$J = E\left[|\ F - \hat{F}\ |^2\right]$$

$$J = E[|\ F - WG\ |^2].$$

Expanding,

$$J = E[(F - WG)(F^* - W^*G^*)]$$

so

$$J = E[|\ F\ |^2] - W^*E[FG^*] - WE[F^*G] + |\ W\ |^2\ E[|\ G\ |^2].$$

Define the power and cross-power spectra:

$$S_F = E[|\ F\ |^2]$$

$$S_{GG} = E[|\ G\ |^2]$$

$$S_{FG} = E[FG^*].$$

Hence,

$$J = S_F - W^*S_{FG} - WS_{FG}^* + WW^*S_{GG}.$$

For minimum error,

$$\frac{\partial J}{\partial W^*} = 0.$$

$$-S_{FG} + WS_{GG} = 0$$

Hence $\boxed{W = \frac{S_{FG}}{S_{GG}}}$

Wiener filtering - Derivation

Since, $G^* = H^*F^* + N^*$.

$$S_{FG} = E[FG^*] = E[F(H^*F^* + N^*)].$$
$$= H^*E[|\ F\ |^2] + E[FN^*].$$

$$\boxed{S_{FG} = H^*S_F}$$

Assuming, image and noise are **uncorrelated:**
$E[FN^*] = 0$.

$$S_{GG} = E[|\ HF + N\ |^2].$$
$$= |\ H\ |^2\ S_F + S_N$$

because the cross terms disappear when F and N are uncorrelated. where S_N is the noise power spectrum.

Substitute these expressions into $W = \frac{S_{FG}}{S_{GG}}$.

$$\boxed{W(u,v) = \frac{H^*(u,v)S_F(u,v)}{|\ H(u,v)\ |^2\ S_F(u,v) + S_N(u,v)}}$$

$$\boxed{W(u,v) = \frac{H^*(u,v)}{|\ H(u,v)\ |^2 + \frac{S_N(u,v)}{S_F(u,v)}}}$$

Therefore, the restored image is

$$\boxed{\hat{F}(u,v) = \frac{H^*(u,v)}{|\ H(u,v)\ |^2 + \frac{S_N(u,v)}{S_F(u,v)}}G(u,v)}$$

This is the **Wiener restoration filter.**

Exercise

- A degraded image is represented by G(u, v) in the frequency domain and an estimation of its degradation function is

$$H(u,v) = -\sigma\sqrt{2\pi}(u^2+v^2)e^{-j2\pi^2\sigma^2(u^2+v^2)}$$

  - Provide the expression in the frequency domain of the restored image using the Wiener filter.
  - Assume that the power spectrum of the undegraded image is ten times the power spectrum of the noise throughout all the image.

Constrained least squares / Tikhonov restoration

$$\hat{F}(u,v) = \left[\frac{H^*(u,v)}{|\,H(u,v)\,|^2 + \gamma\,|\,C(u,v)\,|^2}\right] G(u,v)$$

gamma: regularization parameter; C(u,v): Fourier transform of the chosen smoothing operator, often the Laplacian. This penalizes undesirable roughness while deblurring.

- CLS incorporates a smoothness prior instead of an explicit signal/noise PSD ratio.
- It is widely used when the PSF is known but accurate power-spectrum estimates are unavailable.
- Too small gamma gives noise amplification; too large gamma over-smooths the result.

Adaptive Tuning: γ is computed iteratively (often using Newton-Raphson root-finding) until the residual energy $\|g - H\hat{f}\|^2$ converges to the noise power estimate $\sigma_n^2 MN$.

CLS - Example

γ was optimized for best results

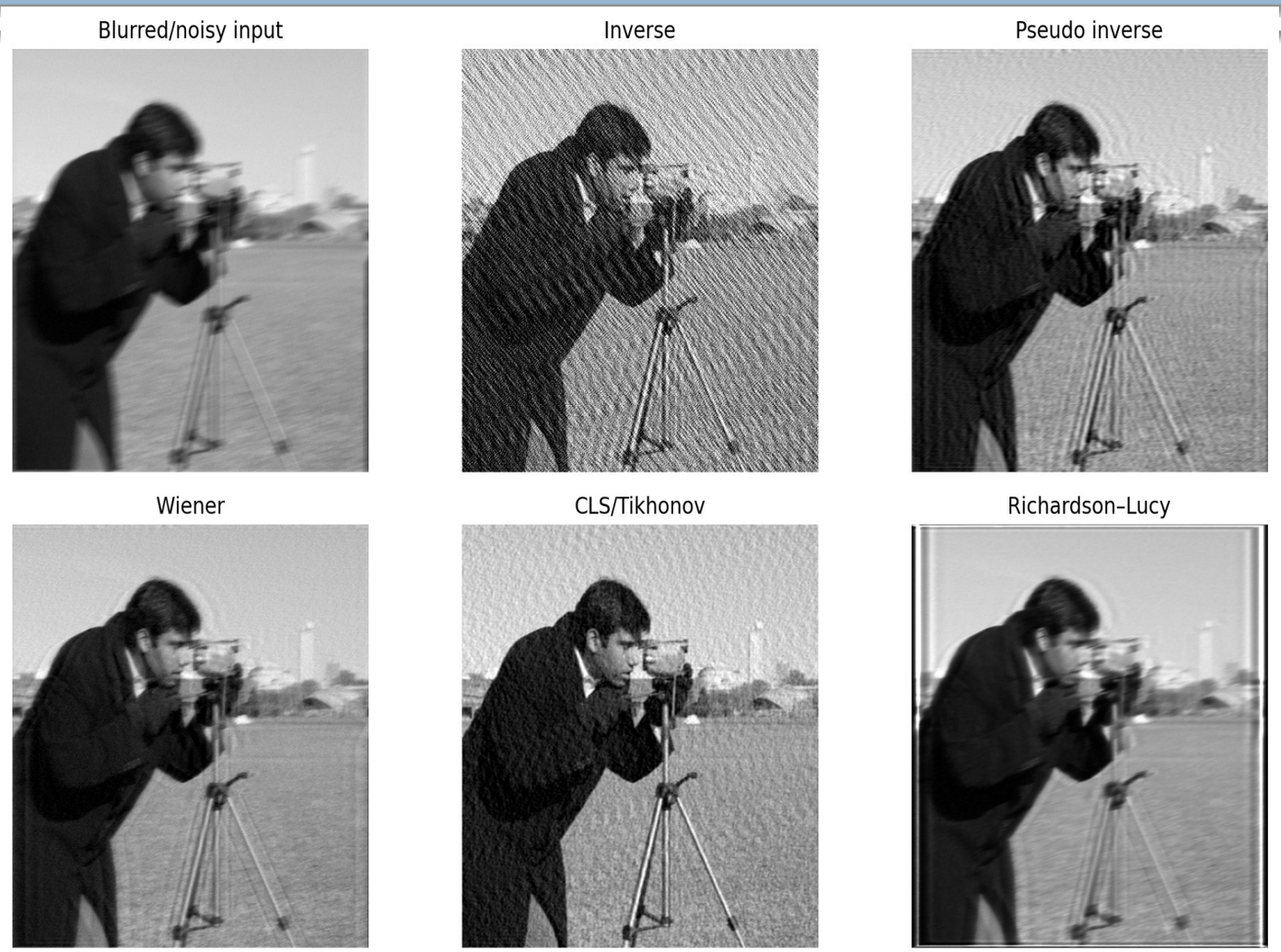
- Low noise: Wiener and CLS generate equal results.
- High noise: CLS outperforms Wiener if γ is properly selected. It is easier to select the scalar value for γ than to approximate the SNR which is seldom constant

Filter Comparison

Filter Type	Required Prior Information	Noise Handling Approach	Output Characteristics
Inverse Filter	System degradation function H	Ignores noise ($N = 0$)	Severely corrupted by amplified noise
Pseudo-Inverse	Degradation H, threshold ϵ	Zeroes out frequencies where $\lvert H\rvert < \epsilon$	Sharp, but subject to ringing artifacts
Wiener Filter	Degradation H, Signal & Noise Power Spectra	Minimizes mean-square error globally	Optimal MSE, but hard to obtain S_f
CLS Filter	Degradation H, Noise variance σ_n^2	Explicitly constrains noise energy	Smooth, sharp restoration without full prior spectra

Example comparison on one motion-blur restoration task

Method	PSNR (dB)	SSIM
CLS	25.25	0.499
Wiener	24.94	0.640
Pseudo inverse	24.54	0.588
Richardson–Lucy	18.20	0.575
Inverse	13.09	0.089



These values come from one simulated motion-blur example and are meant only to illustrate tradeoffs; method ranking changes with blur severity, noise, and PSF accuracy.

Practical workflow for choosing a restoration method

1. Diagnose the artifact: periodic peaks, narrow-band contamination, or blur with/without noise?
2. If periodic/no narrow band: inspect the spectrum and choose notch or band-reject filtering.
3. If blur dominates: estimate the PSF/OTF from system knowledge, a calibration target, or a blind method.
4. Choose restoration: inverse for ideal/noiseless cases, Wiener/CLS for noisy cases, iterative/advanced methods for difficult cases.
5. Evaluate visually and with metrics such as PSNR/SSIM; check for ringing, boundary artifacts, and over-smoothing.

Key takeaways

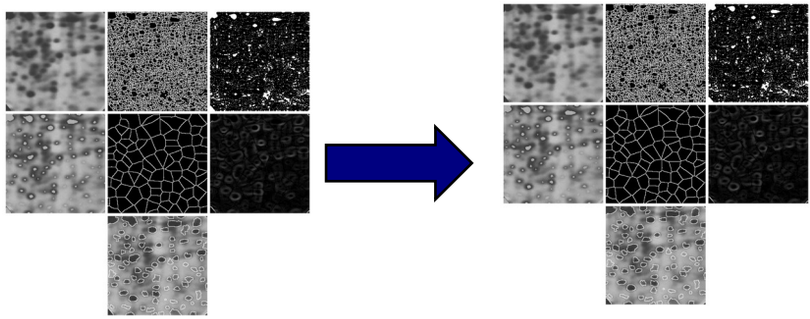
- Fourier-domain restoration is powerful because convolution becomes multiplication and periodic interference becomes spectrally localized.
- Use notch and band-reject filters for periodic or narrow-band noise. Use inverse, pseudo-inverse, Wiener, or CLS for blur restoration when the PSF is known.
- PSF/OTF estimation is central: estimate it from imaging physics, point/edge targets, or blind optimization when necessary.
- Wiener and regularized methods are usually better than pure inverse filtering in realistic noisy conditions.
- Modern methods such as Richardson–Lucy, TV, BM3D, and deep learning extend the classical tools when blur/noise are more challenging.

Image Segmentation

Prof. Dr. Sk. Md. Masudul Ahsan
Dept. of CSE, KUET

What is segmentation?

- Dividing the image into different meaningful regions (or extracts boundaries).
  - Separating objects (anatomical or pathological structures) from background and giving them individual labels(ID numbers)
- Formally: partition image R into sub-regions R_1, R_2, …, R_n such that
  - $UR_i = R$, $R_i \cap R_j = \varnothing$ for $i \neq j$,
  - each R_i is connected / homogeneous w.r.t. some property (intensity, texture).



- Sits between low-level processing (denoising, enhancement) and high-level analysis (measurement, classification).
- Output: a binary mask, a labeled region map, or an explicit boundary/contour.

Targeted Segmentation

- Segmentation is an ill-posed problem
- What is a correct segmentation of this image?
  - ...unless we specify a segmentation target.

"Segment the orange car from the background"

"Segment all road signs from the background"

Segmentation Approaches

- **Thresholding -** Based on pixel intensities (shape of histogram is often used for automation). → Similarity
  - Fast, simple, assumes separable intensity distributions.
- **Edge-based -** Detecting discontinuities in intensities that separate regions from each other. → Discontinuity
  - edges must then be linked into closed contours.
- **Region-based -** Grouping similar pixels based on homogeneity criteria → Similarity
  - e.g. region growing, split & merge.
- **Model-Based -** Uses predefined shape / contour models to locate object boundaries. → Discontinuity
  - Robust to noise and incomplete edges.

Image Thresholding

Classifying pixels by intensity

Thresholding

Original

T=90

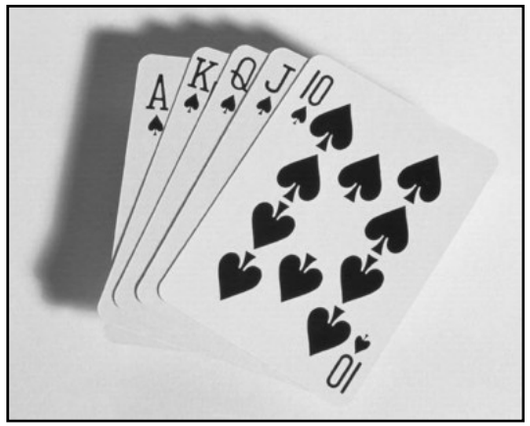
- **Simplest**, **widely used** for image segmentation.
- It is useful in discriminating foreground from the background.
- By selecting an adequate threshold T, the **grayscale** image can be converted to **binary image**.
- mathematically :

$$g(x,y) = \begin{cases} 1 \ if \ \ f(x,y) > T & (\text{object/foreground}) \\ 0 \ if \ \ f(x,y) \leq T & (\text{background}) \end{cases}$$

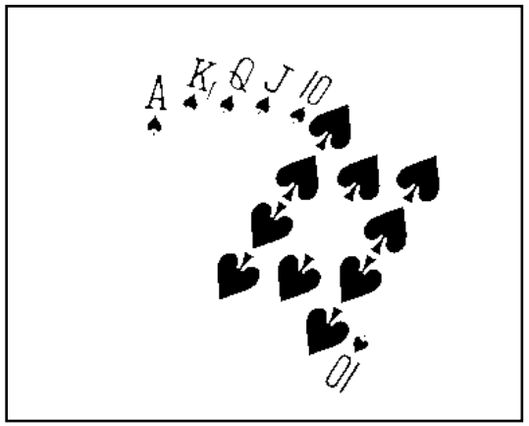
  - T can be a single global constant (global thresholding), or
  - vary spatially, T = T(x,y) (adaptive / local thresholding).
    - depend on local properties, e.g. mean or variance
- Works best when foreground / background intensity distributions are well separated — i.e. the histogram is bimodal.

Thresholding Example - Caution

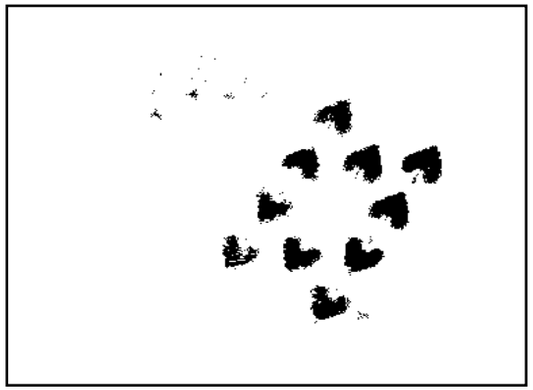
- Imagine a poker playing robot that needs to visually interpret the cards in its hand
- If you get the threshold wrong the results can be disastrous



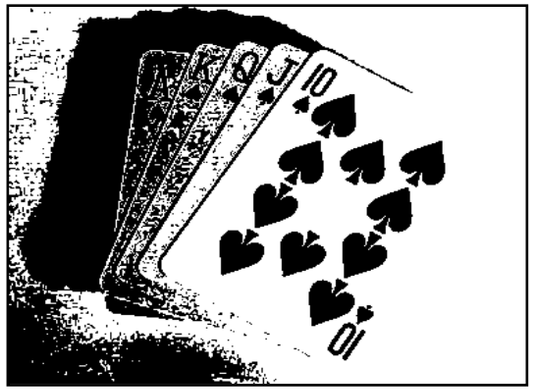
Original Image



Expected Image



Threshold Too Low



Threshold Too High

Thresholding - methods

- **Global threshold -** a single T is applied everywhere
- **Optimal global threshold –** T is **computed** first based on the shape of the current image histogram
  - Searching for valleys, Gaussian distribution etc.
- **Local (or adaptive) threshold -** computed from statistics of a neighborhood around (x,y), e.g. local mean μ(x,y)

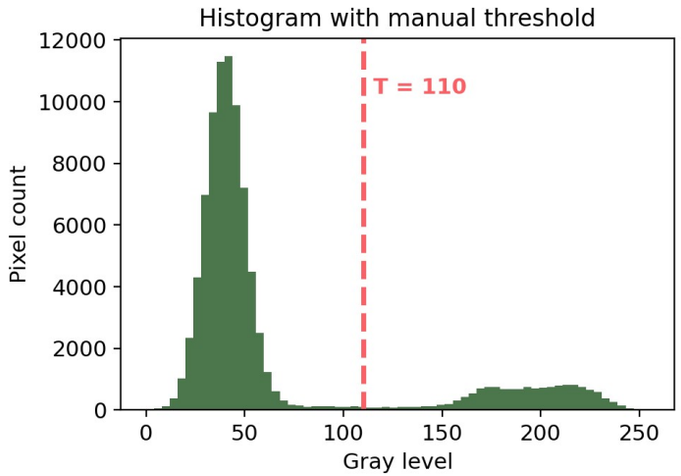
- Global
  - Assumes uniform illumination and a clean bimodal histogram — common in controlled imaging, less common in clinical scans.
- Local
  - Handles uneven illumination, shading artifacts, and noise gradients typical of microscopy and clinical images.
- Trade-off: local methods are more robust but more computationally expensive and sensitive to neighborhood (window) size.

Global Thresholding

Original

Choosing T from the Histogram

- Partition the image histogram using a single global threshold
- Success strongly depends on how well the histogram can be partitioned



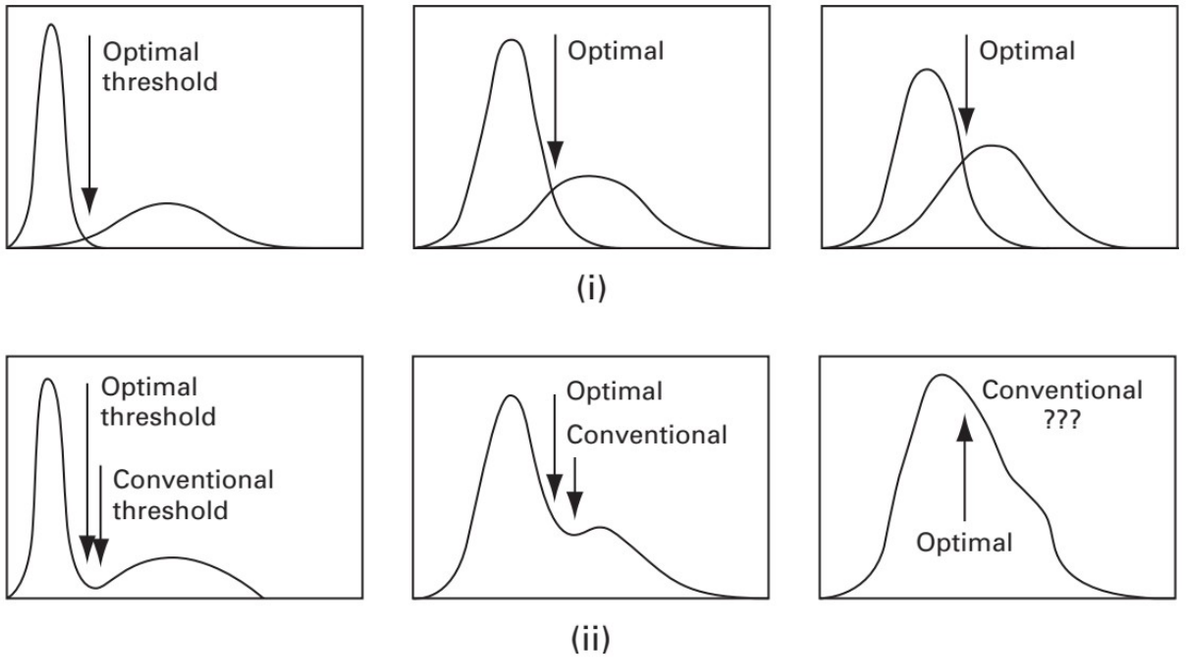
T=90

- If the histogram has two clear peaks (modes) — one for background, one for foreground
  - the threshold T is placed in the valley between them.

T=120

Optimum threshold

- Considers the histogram of an image to be a weighted sum of two (or more) probability densities.
- The threshold is then set as the gray level which results in the smallest number of pixels being misclassifie



(i) Gray-level histograms approximated by two normal distributions, where the intersection represents the optimal threshold. (ii) The resulting optimal and conventional thresholds in the combined histogram. (From SONKA/HLAVAC/BOYLE. *Image Processing, Analysis, and Machine Vision*, 3E. © 2008 Brooks/Cole, a part of Cengage Learning, Inc. Reproduced by permission. www.cengage.com/permissions.)

Optimum Global threshold- Otsu’s Method

- Manually picking T is subjective and not reproducible
  - we need an automatic, optimal threshold.
- Otsu's method (1979) treats thresholding as a discriminant analysis problem: find the T that best separates two classes (background C_0, foreground C_1) of the histogram.

- Based on a very simple idea- Find the threshold that *minimizes the weighted within-class variance*.
  - This turns out to be the same as *maximizing the between-class variance*.
- Operates directly on the gray level histogram [*e.g.* 256 numbers, P(i)], so it’s fast (once the histogram is computed).
- Fully automatic — requires only the image histogram, no parameters to tune. One of the most widely used thresholding methods in medical image analysis.

Otsu's Method

- $\{0,1,2,\ldots,L-1\}$, L means gray level intensity

$$MN = n_0 + n_1 + n_2 + \ldots + n_{L-1}$$

$M*N$ = total number of pixel.

n_i = number of pixels with intensity i

$p_i = n_i / MN$, *probability of* intensity i

- Select a threshold $T(k) = k, 0 < k < L-1$
- use it to classify C_1: intensity in the range [0, k] and C_2: [k+1, L-1]

Class Probability $P_1(k) = \sum_{i=0}^{k} p_i$, $P_2(k) = \sum_{i=k+1}^{L-1} p_i = 1 - P_1(k)$

$$P_1 + P_2 = 1$$

Class mean $m_1(k) = \sum_{i=0}^{k} i \frac{p_i}{P_1} = \frac{1}{P_1} \sum_{i=0}^{k} i p_i$, $m_2(k) = \frac{1}{P_2} \sum_{i=k+1}^{L-1} i p_i$

Otsu's Method

Class variance $\sigma_1^2(k) = \frac{1}{P_1}\sum_{i=1}^{k}[i - m_1]^2 p_i$, $\sigma_2^2(k) = \frac{1}{P_2}\sum_{i=k+1}^{L-1}[i - m_2]^2 p_i$

Minimize: within Class variance $\sigma_w^2(k) = P_1\sigma_1^2 + P_2\sigma_2^2$

Maximize: Between Class *variance* $\sigma_B^2(k) = P_1(m_1 - m_G)^2 + P_2(m_2 - m_G)^2$

Global mean $m_G = \sum_{i=0}^{L-1} ip_i$ $P_1 m_1 + P_2 m_2 = m_G$

global variance $\sigma_G^2 = \sum_{i=0}^{L-1}(i - m_G)^2 p_i$

Otsu's Method

$$\sigma_B^2(k) = P_1(k)P_2(k)[m_1(k) - m_2(k)]^2$$

$$= \frac{(m_G P_1(k) - m(k))^2}{P_1(k)(1 - P_1(k))}$$

where $m = \sum_{i=0}^{k} ip_i = P_1 m_1$

Cumulative mean upto *k*

a measure of *separability between classes*.

$$\eta = \frac{\sigma_B^2(k)}{\sigma_G^2} \qquad 0 \le \eta(k^*) \le 1$$

Threshold, T = $k^* = \arg\max_{0 \le k \le L-1} \sigma_B^2(k)$

$$g(x,y) = \begin{cases} 1 & \text{if } f(x,y) \ge k^* \\ 0 & \text{if } f(x,y) \le k^* \end{cases}$$

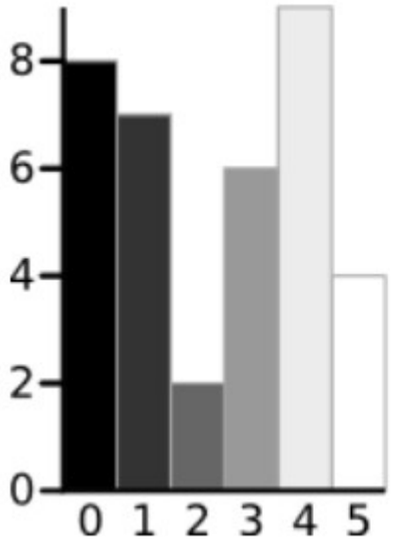
For x = 0,1,2,…,*M*-1
and y = 0,1,2…,*N*-1.

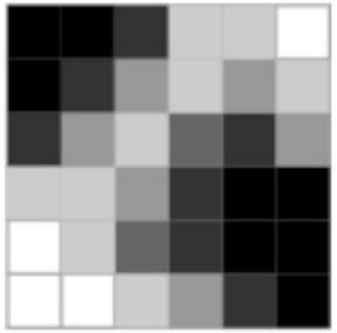
Otsu's Method - Step by Step

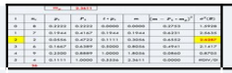
1. Compute the normalized histogram p_i, i = 0, 1,2, ... , L – 1 of the input image.
2. Compute the cumulative sums, $P_1(k)$, and cumulative means, $m(k)$, for each *k*
3. Compute the global intensity mean, m_g,
4. Compute the between-class variance, $\sigma_B^2(k)$, for each k
5. Select the Otsu threshold, k* = value of k for which $\sigma_B^2(k)$is maxi-mum.
   - If the maximum is not unique, obtain k* by averaging the values of k corresponding to the various maxima detected.
   - Measure the separability, η^*, if needed
6. Threshold the image: g(x,y) = 1 if f(x,y) > K*, else 0.

- O(L) after the histogram is built — extremely fast (L = 256 for 8-bit images).
- Generalizes to multilevel Otsu for k thresholds and k+1 classes (slower: O(L^k)).

Otsu - Example





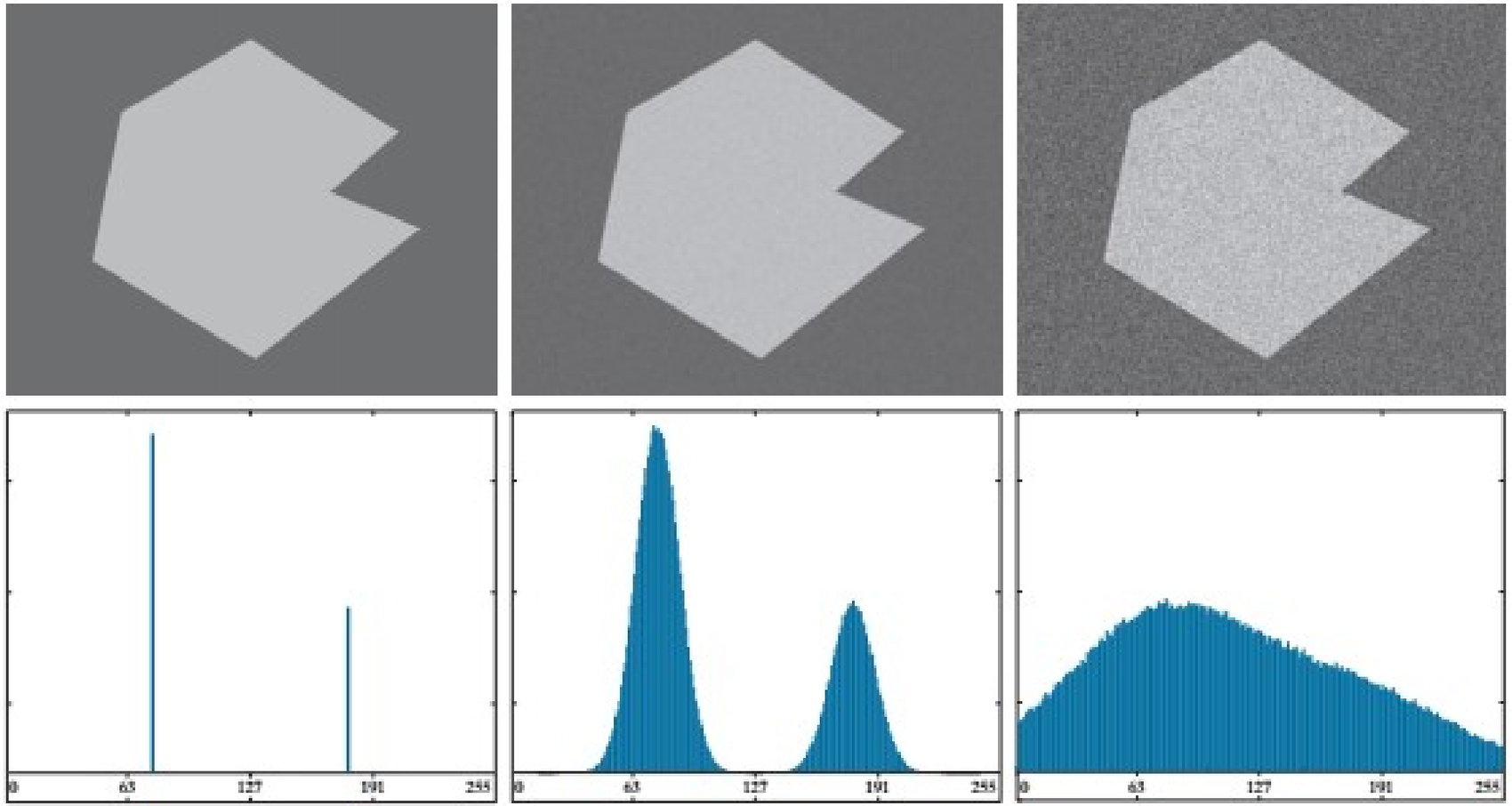


2.3611

T=

i							
0	8	0.2222	0.2222	0.0000	0.0000	0.2753	1.5928
1	7	0.1944	0.4167	0.1944	0.1944	0.6231	2.5635
2	2	0.0556	0.4722	0.1111	0.3056	0.6552	**2.6287**
3	6	0.1667	0.6389	0.5000	0.8056	0.4941	2.1417
4	9	0.2500	0.8889	1.0000	1.8056	0.0860	0.8705
5	4	0.1111	1.0000	0.5556	2.3611	0.0000	#DIV/0!
	36						

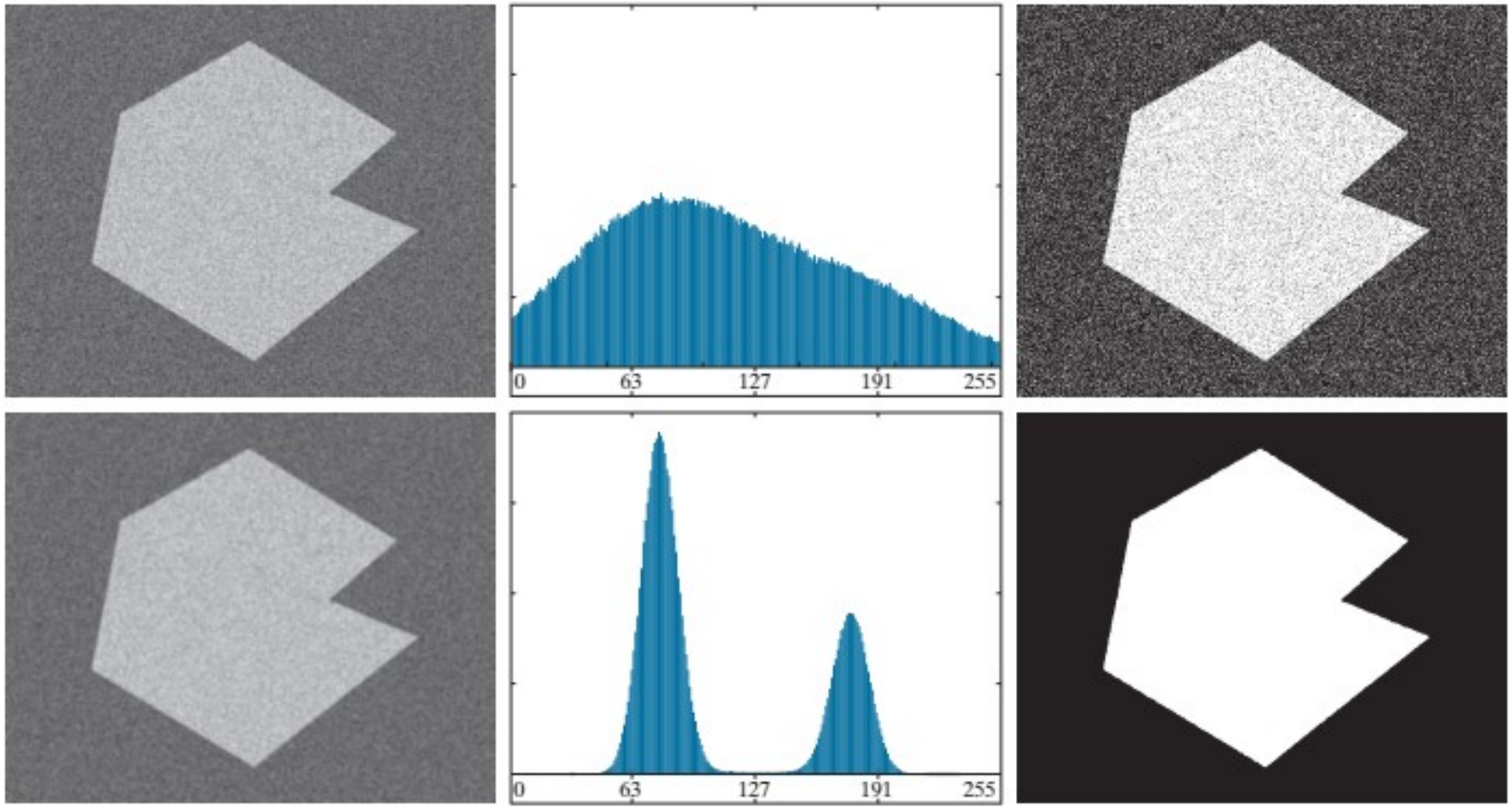
Gaussian noise added



a b c
d e f

FIGURE 10.33 (a) Noiseless 8-bit image. (b) Image with additive Gaussian noise of mean 0 and standard deviation of 10 intensity levels. (c) Image with additive Gaussian noise of mean 0 and standard deviation of 50 intensity levels. (d) through (f) Corresponding histograms.

Otsu's Method – effect of noise and smoothing



a b c
d e f

FIGURE 10.37 (a) Noisy image from Fig. 10.33(c) and (b) its histogram. (c) Result obtained using Otsu's method. (d) Noisy image smoothed using a 5×5 averaging kernel and (e) its histogram. (f) Result of thresholding using Otsu's method.

Isodata method

- (iterative self organizing data analysis technique algorithm)
- The basic global threshold, T, is calculated as follows:

1. Select an initial estimate for T (typically the average grey level in the image)
2. Segment the image using T to produce two groups of pixels:

   G_1 : pixels with grey levels >T

   G_2 : pixels with grey levels ≤ T
3. Compute the average/mean grey level of pixels in G_1 to give μ_1 and G_2 to give μ_2
4. Compute a new threshold value: $T = \frac{\mu_1 + \mu_2}{2}$
5. Repeat steps $2 - 4$ until the difference in T in successive iterations is less than a predefined limit ΔT

Entropy Based thresholding

- Entropy is a measure of the uncertainty of an event taking place, and can be derived from the gray-level histogram.
- Very similar to Otsu's method except rather than maximizing the between-class variance, it
  - Kapur's method **- maximizes the between-class entropy**

$$H_0(T) = -\sum_{\{i=0\}} (p_i/\omega_0)\log(p_i/\omega_0), \qquad H_1(T) = -\sum_{i=T+1}^{L-1} (p_i/\omega_1)\log(p_i/\omega_1)$$

$$T^* = argmax_T\ [\,H_0(T) + H_1(T)\,]$$

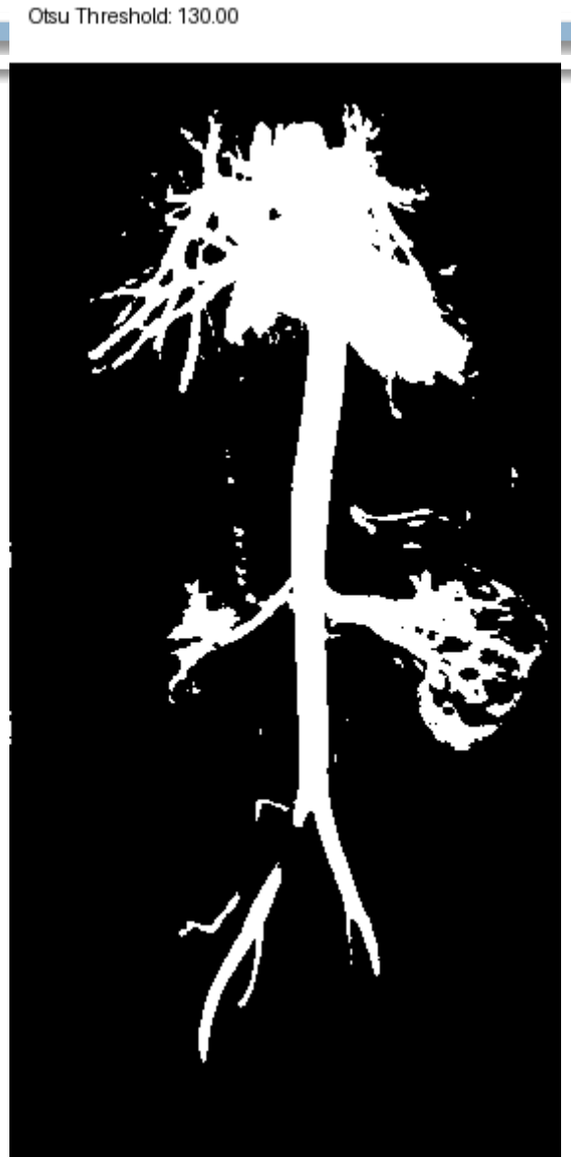
  - Intuition: the optimal threshold maximizes the information content (uncertainty) carried separately by the foreground and background distributions.

Entropy Based thresholding

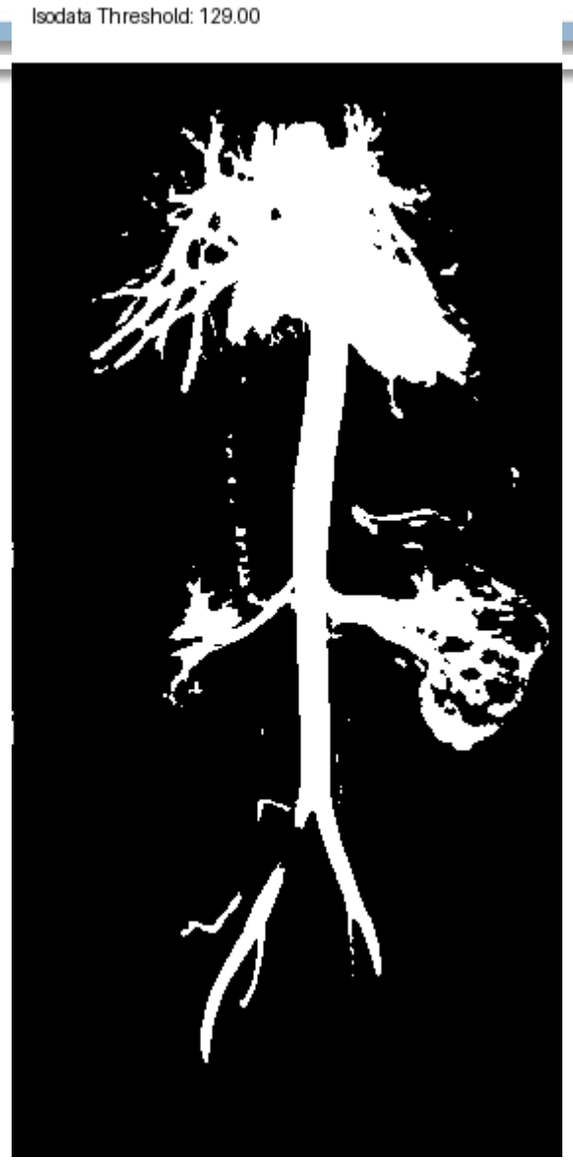
- Yen’s method **- maximizes entropic correlation**.
- Li’s method – **minimize cross entropy**
  - effective in medical image segmentation tasks where the **target object is very small relative to a massive background** (e.g., segmenting small localized infections, skin lesions, or tumors in large radiological fields of view), a scenario where Otsu's method completely fails due to its inherent size bias.

Example

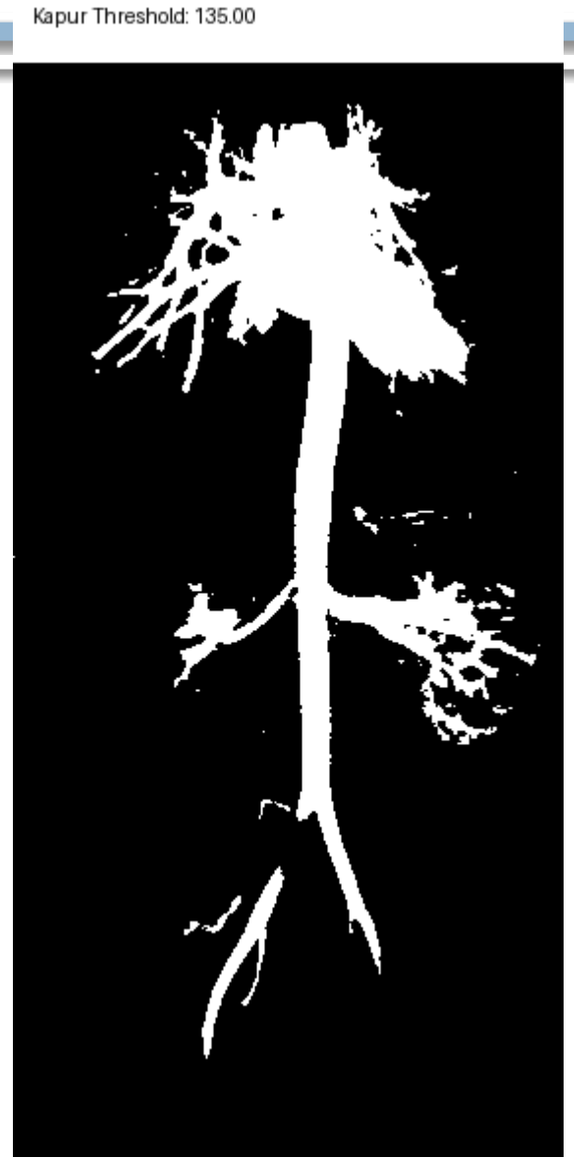
An MR angiography



Otsu



ISODATA

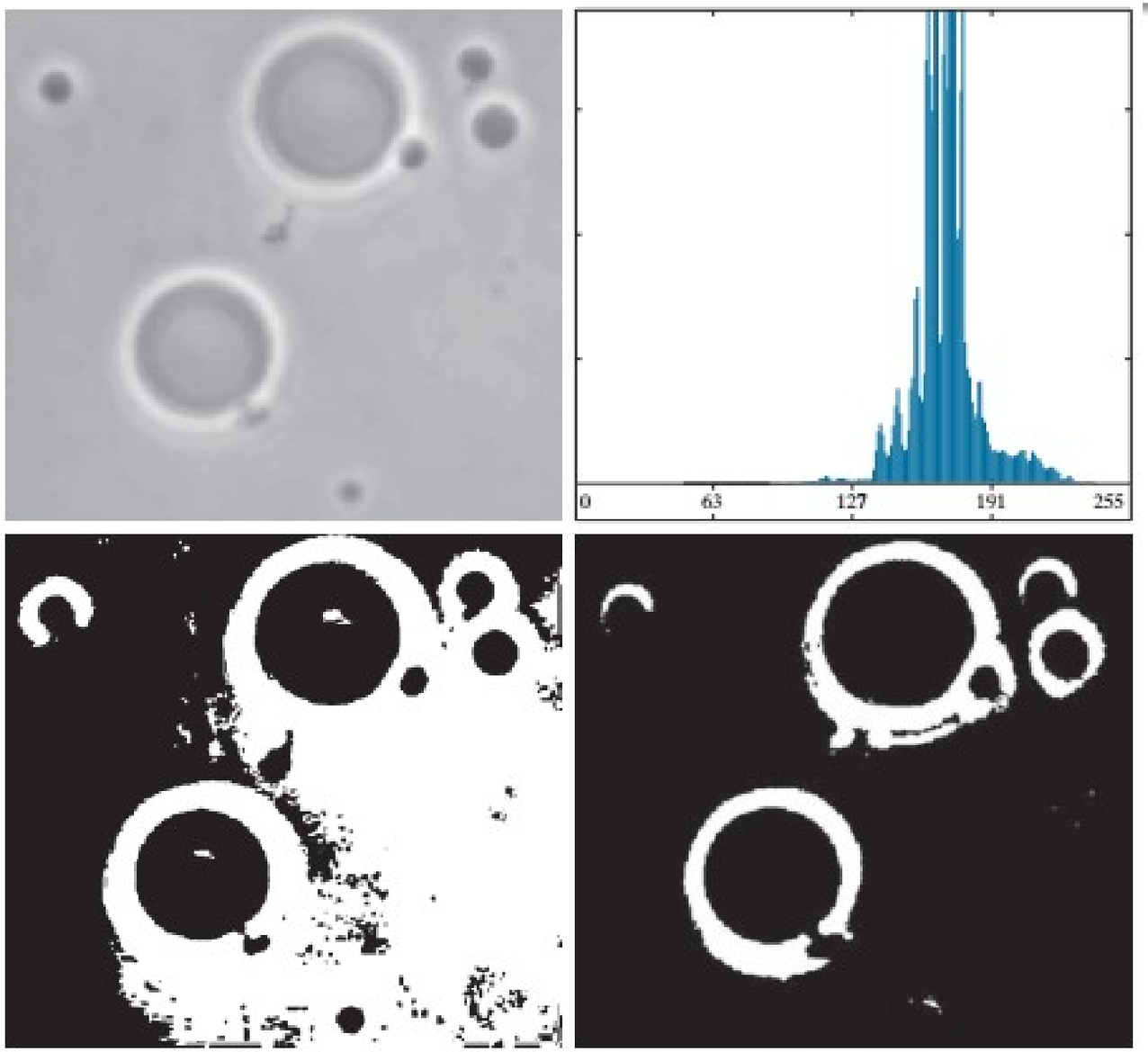


Max Entropy

Example - Caution

a b
c d

FIGURE 10.36
(a) Original image.
(b) Histogram (high peaks were clipped to highlight details in the lower values).
(c) Segmentation result using the basic global algorithm from Section 10.3.
(d) Result using Otsu's method.
(Original image courtesy of Professor Daniel A. Hammer, the University of Pennsylvania.)



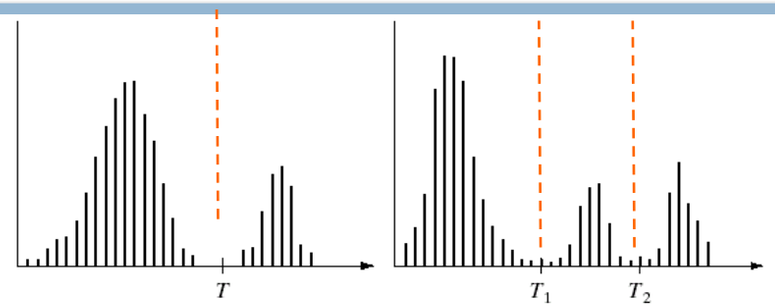
ISODATA

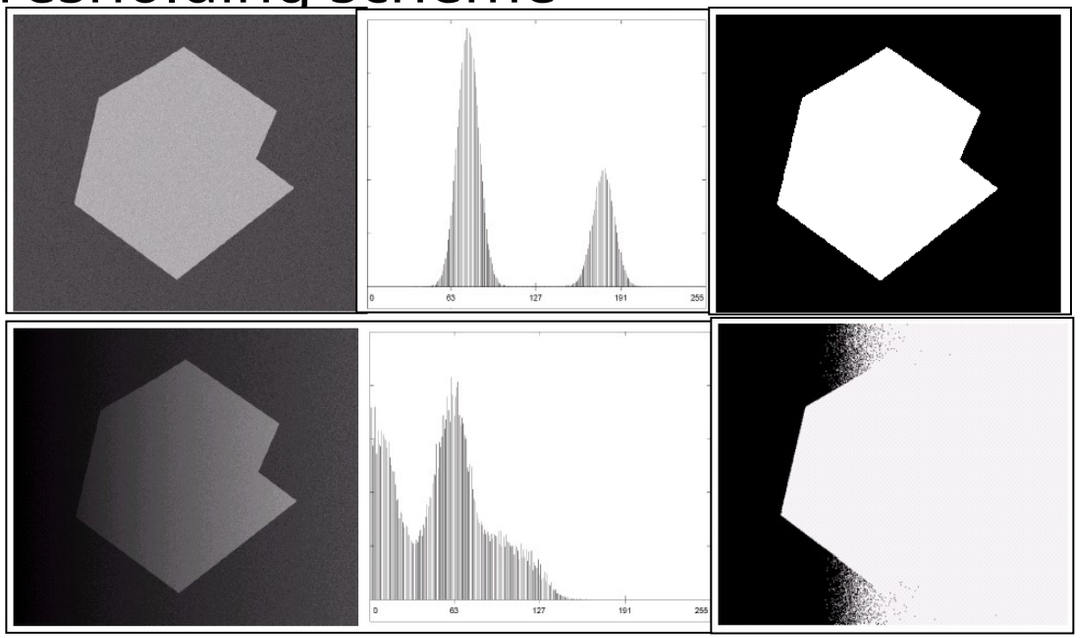
Otsu

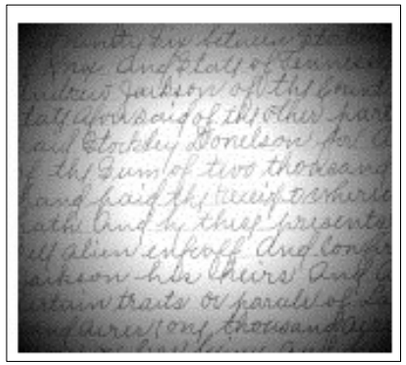
Adaptive thresholding

Limitations of Global Thresholding

- Single value thresholding only works for bimodal histograms
- Images with other kinds of histograms need more than a single threshold
- Uneven illumination can really upset a single valued thresholding scheme







Local Mean Thresholding

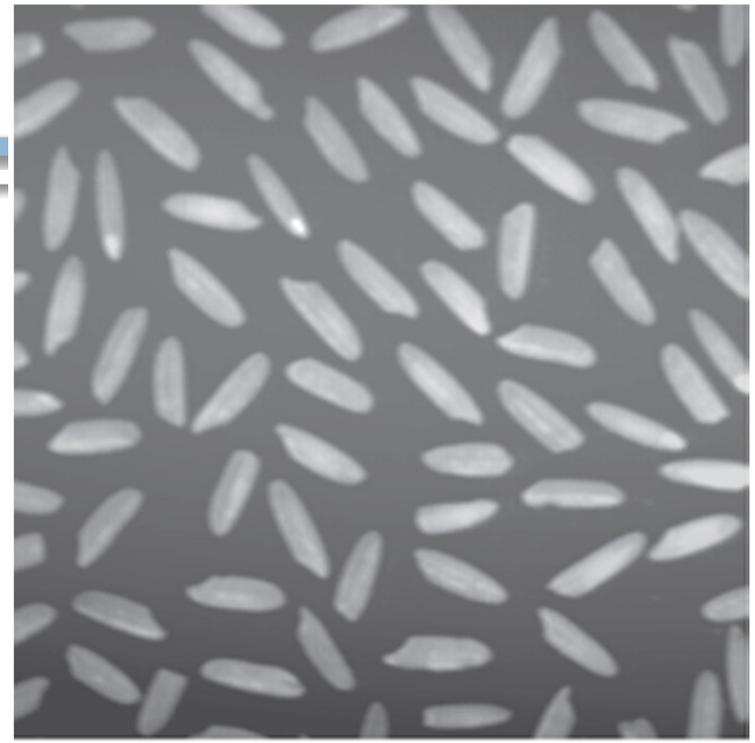
- local threshold, T_L, is chosen from an n × n neighborhood

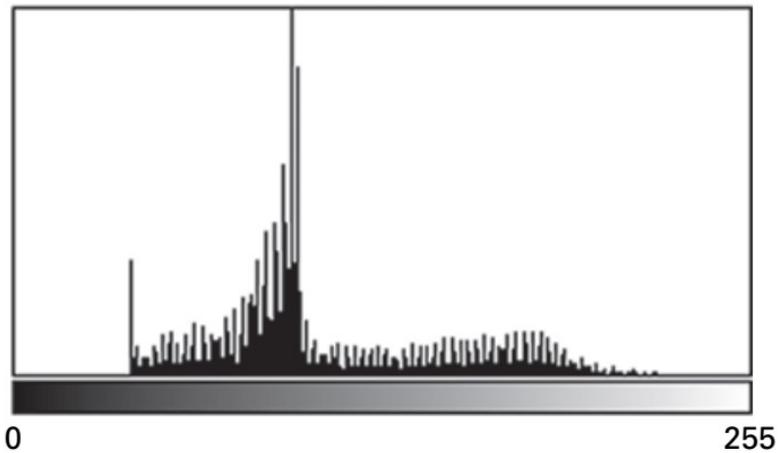
$$T_L(x, y) = mean/median(x, y) - C$$

- where C is a constant

Local Threshold (with n = 45, C = 3)

Otsu





Adaptive Thresholding

- Niblack's Method
  - based on local image properties
  - threshold $Q(u, v)$ is varied across the image as a function of the local intensity average $\mu_R(u, v)$ and standard deviation $\sigma_R(u, v)$

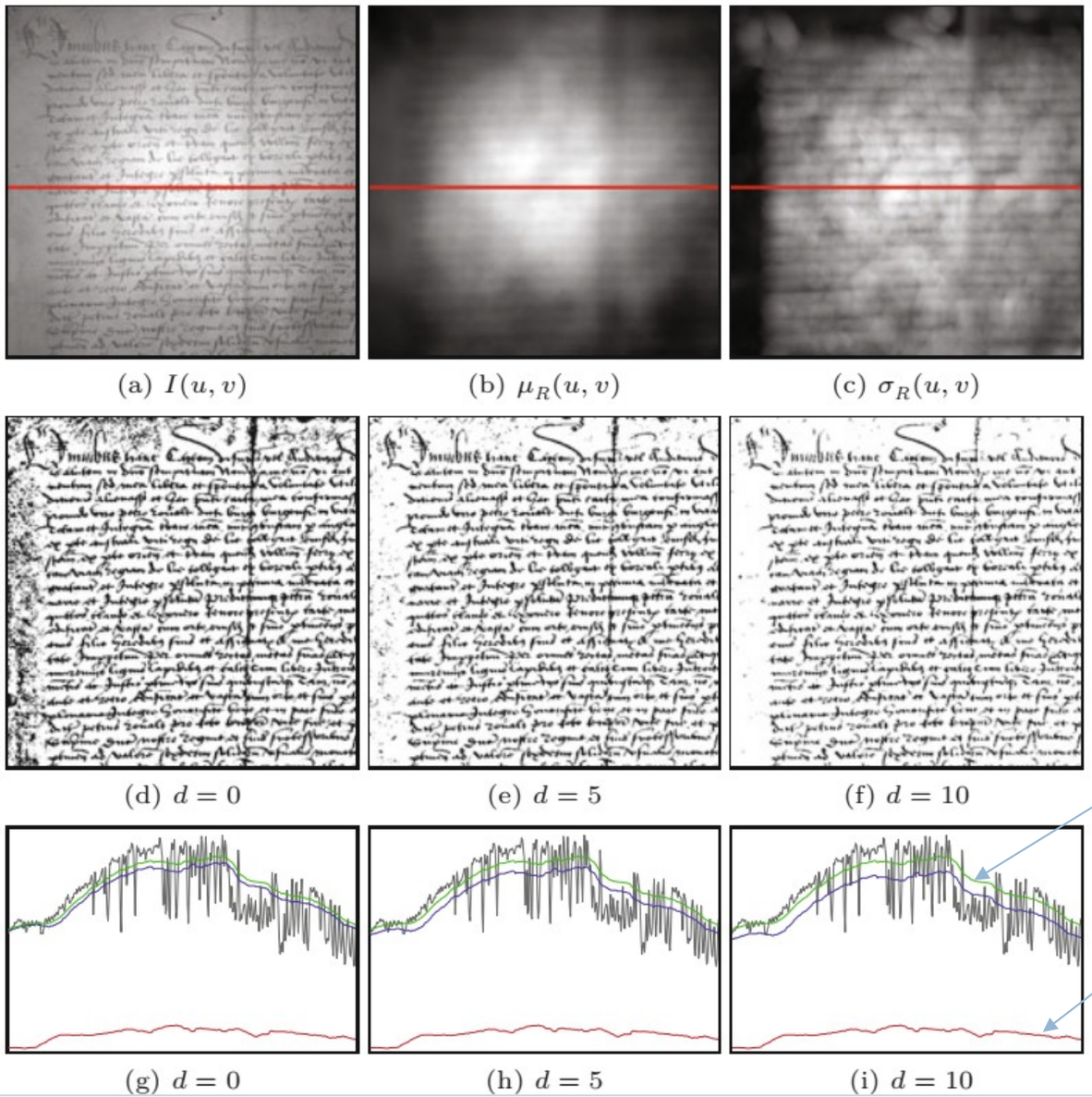
$$Q(u,v) := \mu_R(u,v) + \kappa \cdot \sigma_R(u,v) \; ; \kappa \geq 0$$

- Or, to avoid low-amplitude noise ("ghosting") $Q(u,v) := \mu_R(u,v) + \kappa \cdot \sigma_R(u,v) + d$

- the structures of interest are darker than the background (as, e.g., in typical OCR applications), one could either work with inverted images or modify the calculation of the threshold

$$Q(u,v) := \begin{cases} \mu_R(u,v) + (\kappa \cdot \sigma_R(u,v) + d) & \text{for } \textit{dark} \text{ BG}, \\ \mu_R(u,v) - (\kappa \cdot \sigma_R(u,v) + d) & \text{for } \textit{bright} \text{ BG} \end{cases}$$

Adaptive Thresholding - Niblack's Method



(a) $I(u,v)$ (b) $\mu_R(u,v)$ (c) $\sigma_R(u,v)$

(d) $d = 0$ (e) $d = 5$ (f) $d = 10$

(g) $d = 0$ (h) $d = 5$ (i) $d = 10$

Adaptive thresholding using Niblack's method (with $r = 15$, $\kappa = 0.3$). Original image (a), local mean μ_R (b), and standard deviation σ_R (c). The result for $d = 0$ in (d) corresponds to Niblack's original formulation. Increasing the value of d reduces the amount of clutter in regions with low variance (e, f). The curves in (g–i) show the local intensity (gray), mean (green), variance (red), and the actual threshold (blue) along the horizontal line marked in (a–c).

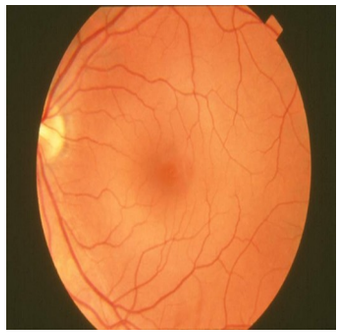
Comparative Analysis

Feature	Global Thresholding (Otsu)	Adaptive Thresholding
Computational Complexity	Extremely Low $O(N)$ where N is pixel count)	Moderate to High (Requires neighborhood window scans)
Parameter Tuning	Minimal (Automated in Otsu)	High (Requires tuning window size W and constant C)
Illumination Robustness	Very Poor (Fails under structural gradients)	High (Excellent at mitigating shading artifacts)
Ideal Medical Target	High contrast structures (e.g., CT Bone, Contrast-enhanced vessels)	Low contrast, noisy, or unevenly illuminated frames (e.g., Ultrasound, Micro-pathology)

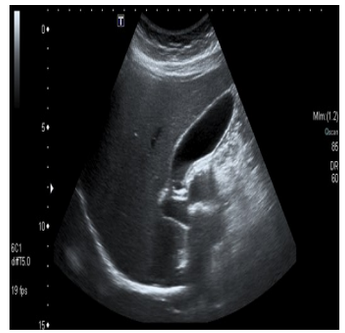
EDGE BASED SEGMENTATION

Edge Detection

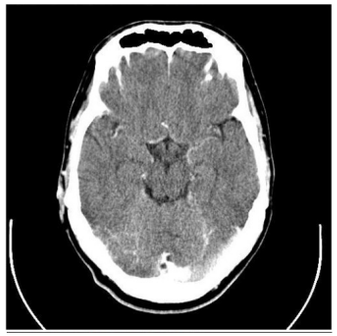
- **Step edge**: abrupt transition from one tissue to another.
- **Ramp edge**: gradual transition caused by blur or partial volume.
- **Roof/ridge edge**: line-like structures such as vessels or thin bones.
- **Noisy edges**: produced by acquisition noise, speckle, or texture.



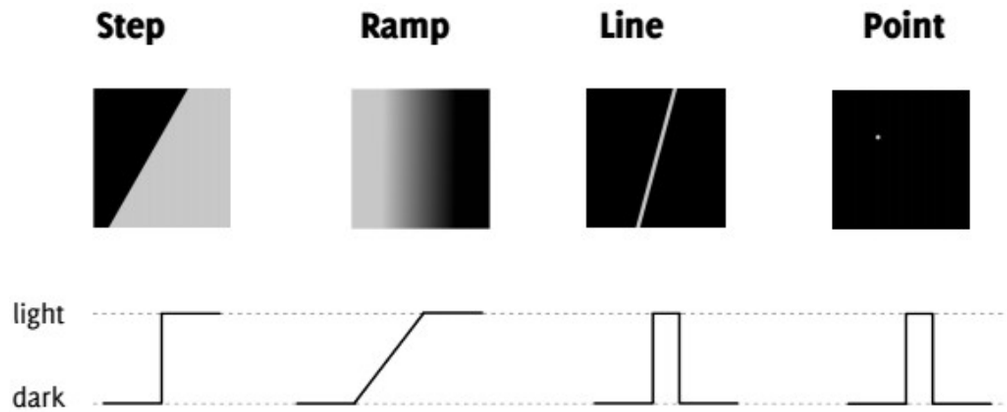
Thin vessel edges



Soft acoustic boundaries

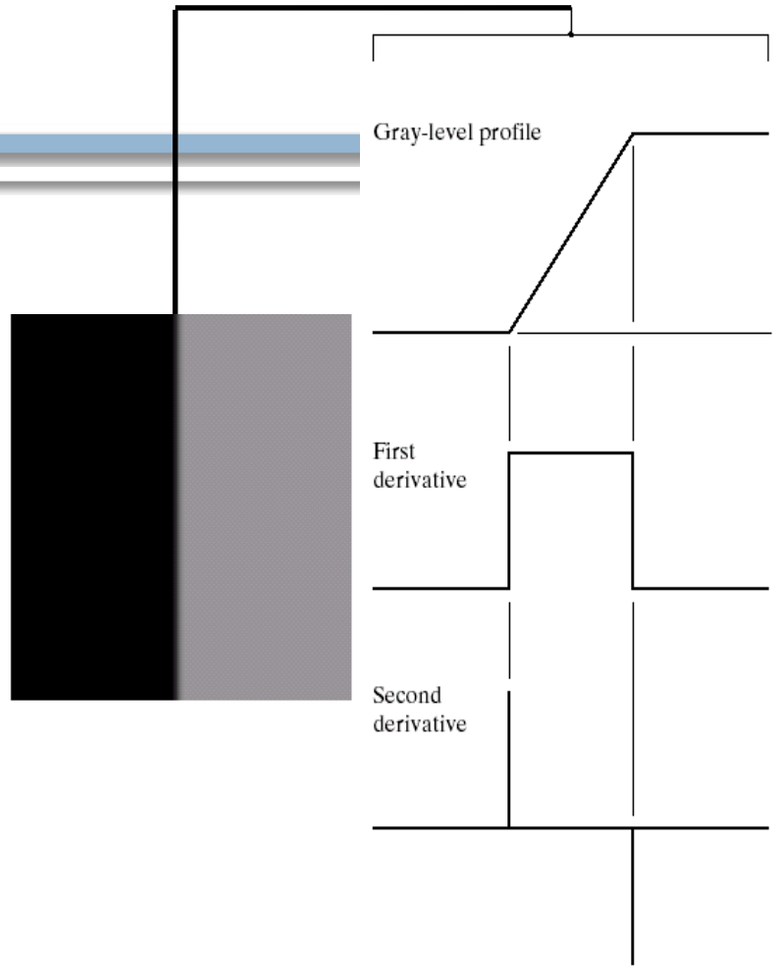


High-contrast skull boundary



Edges & Derivatives

- 1st derivative tells us where an edge is
- 2nd derivative can be used to show edge side



- 1D Discrete derivatives & Filters

		derivate filters
Backward	$\frac{df}{dx} = f(x) - f(x-1) = f'(x)$	[0 1 -1]
Forward	$\frac{df}{dx} = f(x) - f(x+1) = f'(x)$	[-1 1 0]
Central	$\frac{df}{dx} = f(x+1) - f(x-1) = f'(x)$	[1 0 -1]

Gradient mathematics

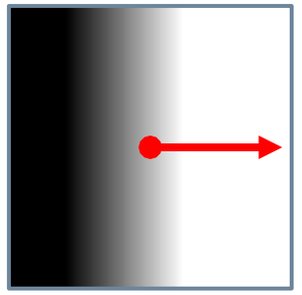
- Discrete derivate in 2D

Given function $f(x,y)$

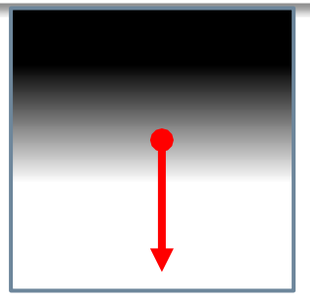
Gradient vector $\nabla f(x,y) = \begin{bmatrix} \frac{\partial f(x,y)}{\partial x} \\ \frac{\partial f(x,y)}{\partial y} \end{bmatrix} = \begin{bmatrix} f_x \\ f_y \end{bmatrix}$

Gradient magnitude $|\nabla f(x,y)| = \sqrt{f_x^2 + f_y^2}$

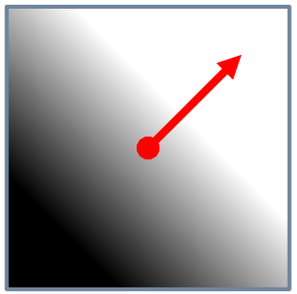
Gradient direction $\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$



$\nabla f = [\frac{\partial f}{\partial x}, 0]$



$\nabla f = [0, \frac{\partial f}{\partial y}]$



$\nabla f = [\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}]$

- Magnitude M measures edge strength.
- Direction θ points toward maximum intensity increase.
- The edge direction is perpendicular to the gradient direction.
- Digital derivatives are approximated by convolution masks.

2D Discrete derivative masks

Roberts Gx

+1	0	
0	−1	

Prewitt Gx

−1	0	+1
−1	0	+1
−1	0	+1

Sobel Gx

−1	0	+1
−2	0	+2
−1	0	+1

Sobel Gy

−1	−2	−1
0	0	0
+1	+2	+1

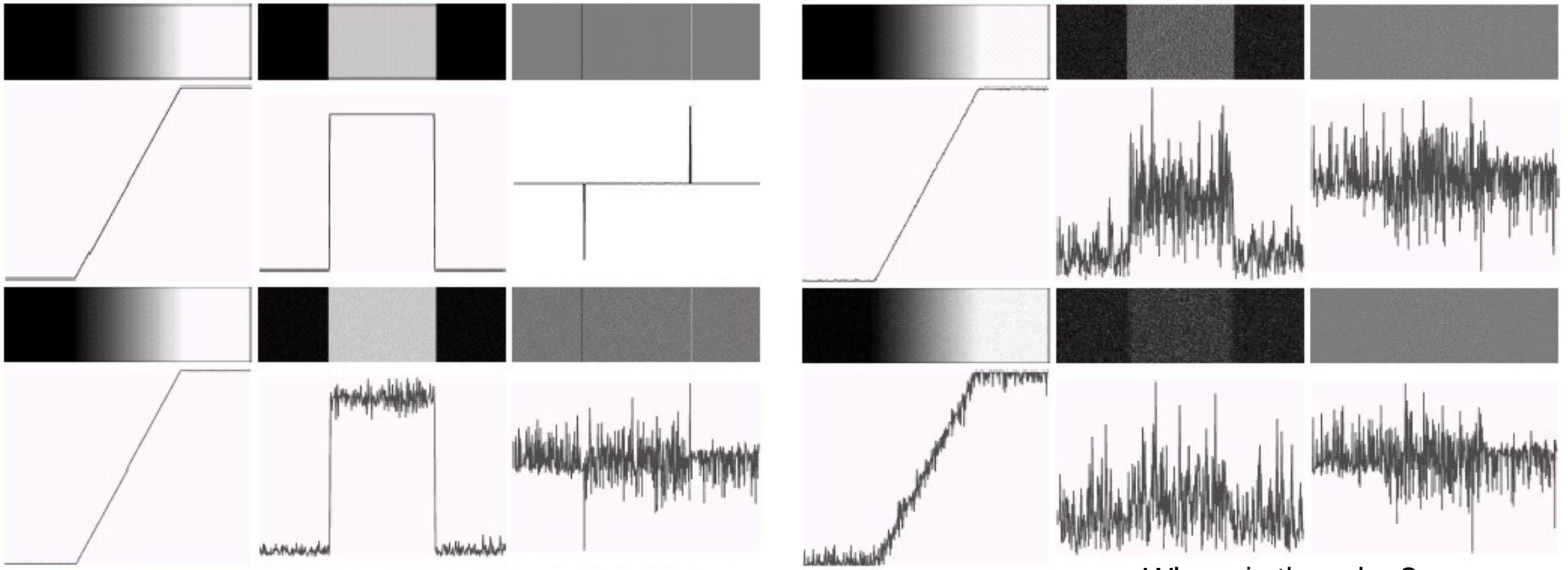
Laplacian

0	−1	0
−1	4	−1
0	−1	0

- Roberts: very small support; sensitive to noise.
- Prewitt: estimates derivative with simple averaging.
- **Sobel**: derivative plus smoothing; common default for quick exploration.
- Laplacian: second derivative; responds to rapid intensity change in all directions.

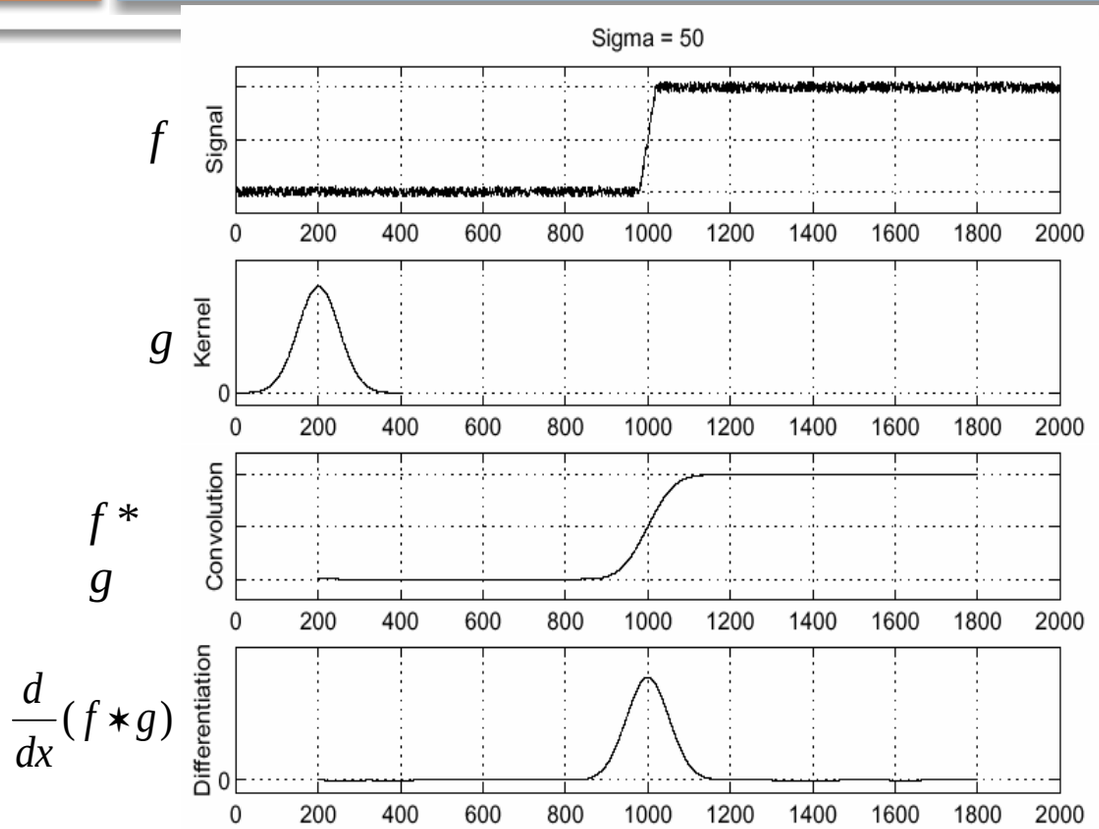
Derivatives & Noise

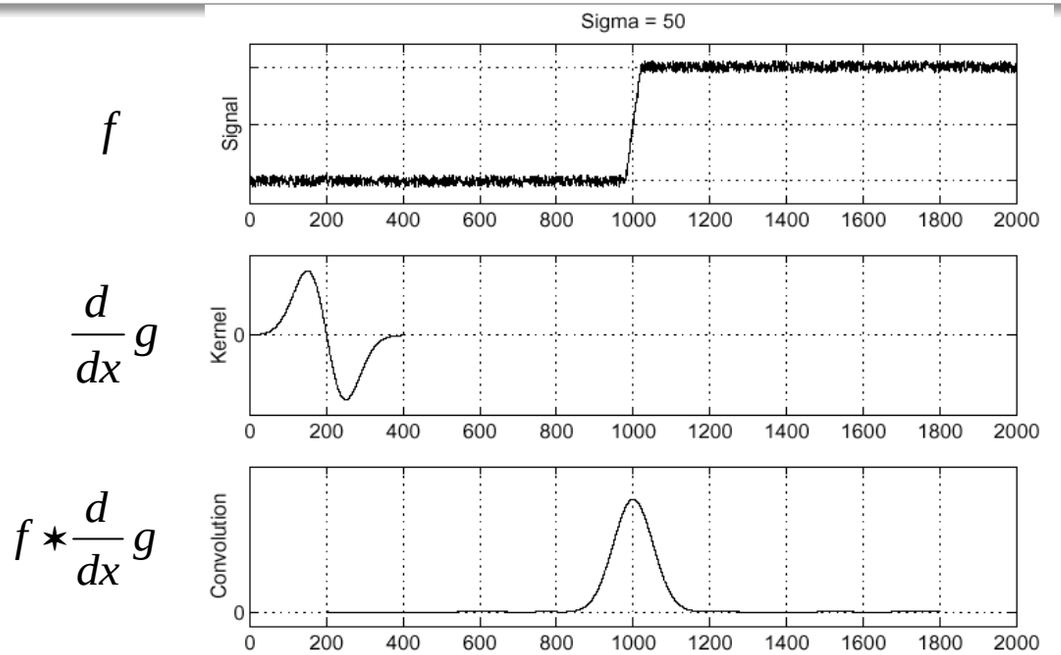
- Derivatives are extremely sensitive to noise
- We need to keep this in mind



Where is the edge?

Solution: smooth first





This saves us one operation

- To find edges, look for peaks in $\frac{d}{dx}(f * g)$

Source: S. Seitz

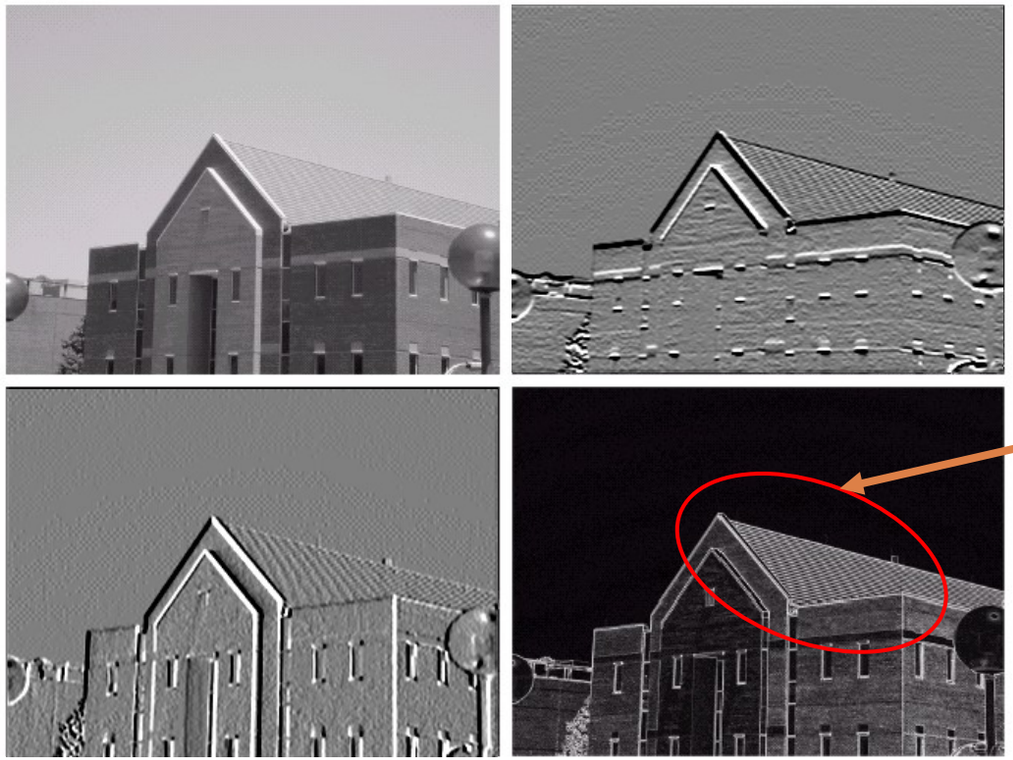
Gradient based Edge Detector

Gradient based Edge Detector

- Compute gradient magnitude
  - Use Sobel or any other edge operator
  - Optionally smooth before computing gradient
- Apply thresholding

$$\mathbf{G} = \sqrt{\mathbf{G}_x{}^2 + \mathbf{G}_y{}^2}$$

Gy

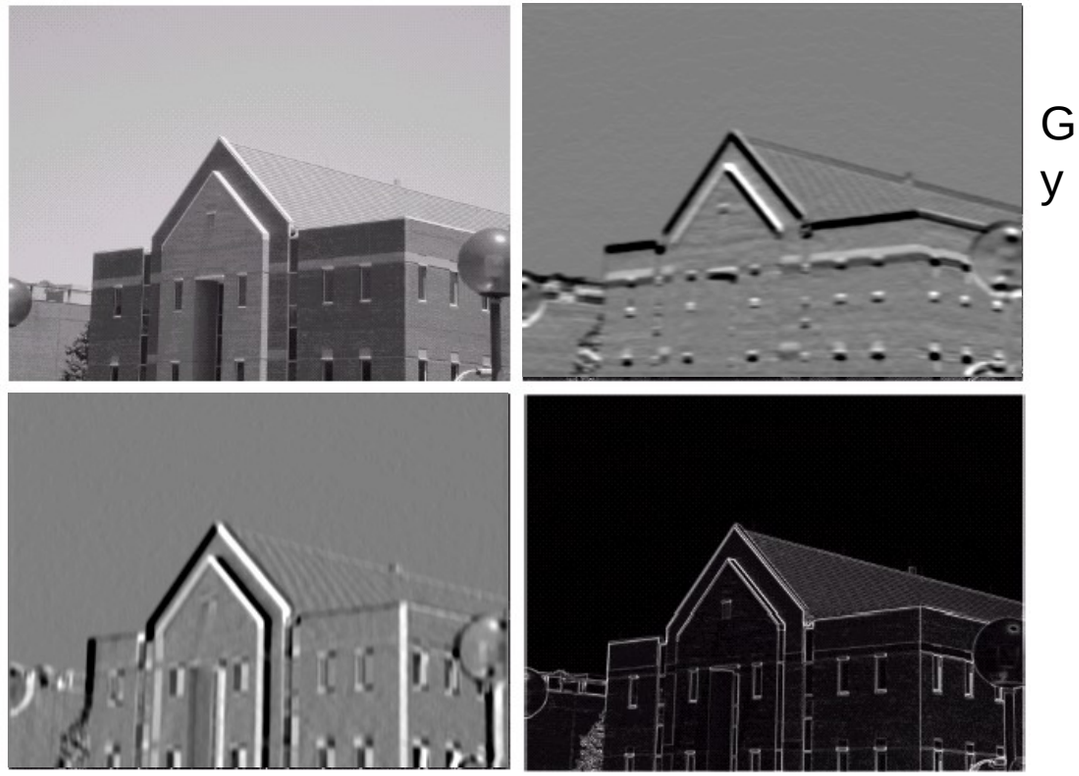


too much detail

Gx

Magnitude

With Smoothing - 5x5 average filter



G

y

G

Magnit

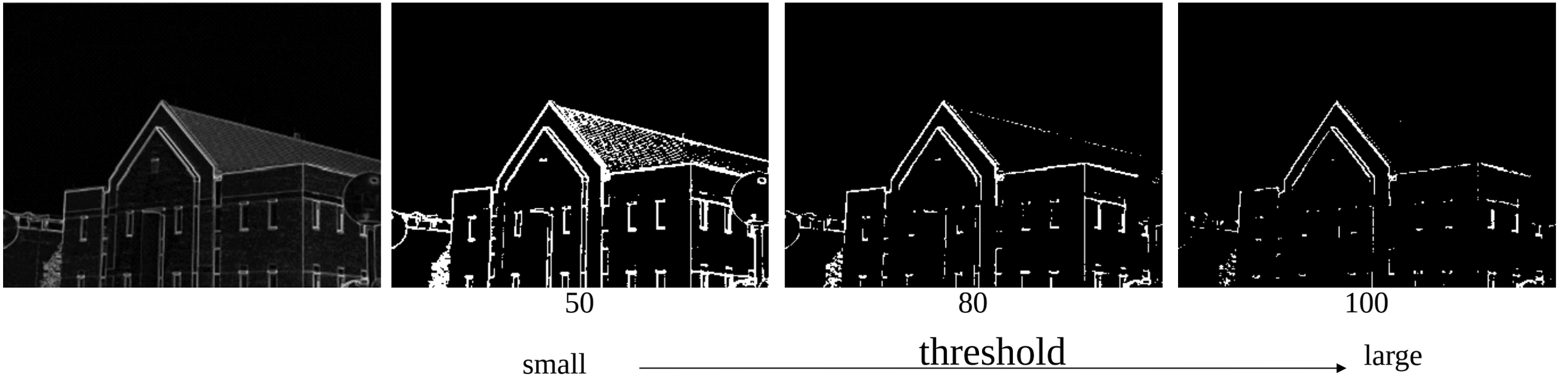
Gradient based Edge Detector

Low threshold:

- many weak edges; higher false positives.

High threshold:

- cleaner map; may miss weak boundaries.

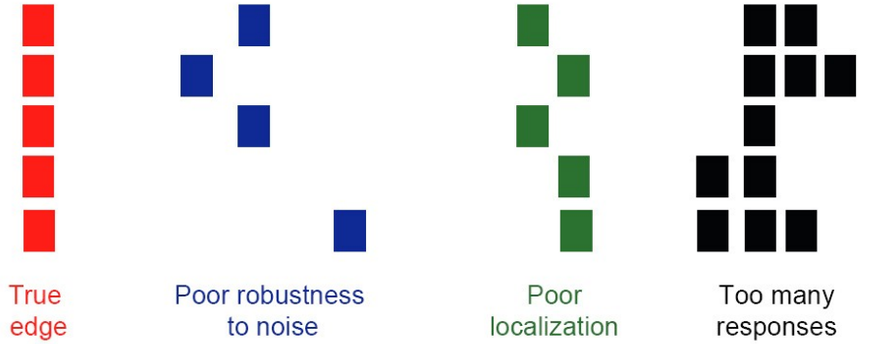


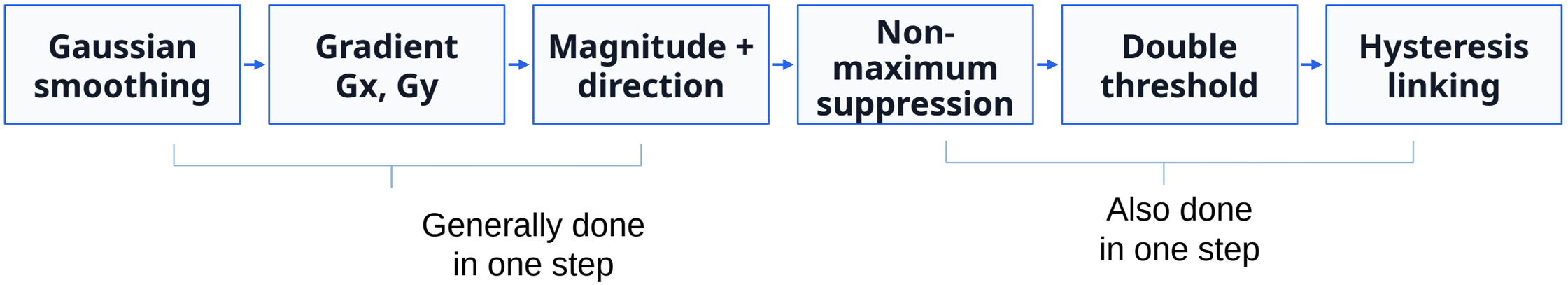
Canny edge detector

Design goals

- **Good detection:** low probability of missing true edges and low false positives.
- **Good localization:** detected edges should be close to true boundaries.
- **Single response:** one detected edge per true edge.

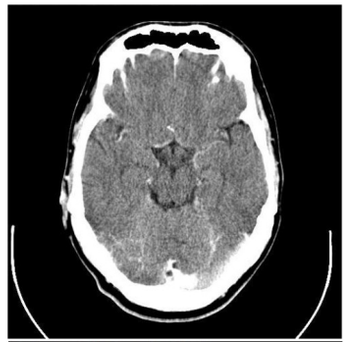
- Canny uses smoothing, gradient magnitude, non-maximum suppression, and hysteresis thresholding.



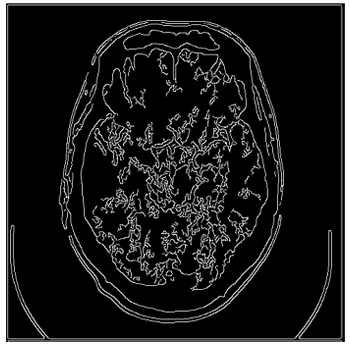


Canny edge detection: algorithm

- T_h controls high-confidence edges.
- T_l controls continuity by allowing connected weak edges.
- Large σ gives smoother but less localized edges.
- For medical images, tune σ and thresholds on validation cases, not on one image.



CT input

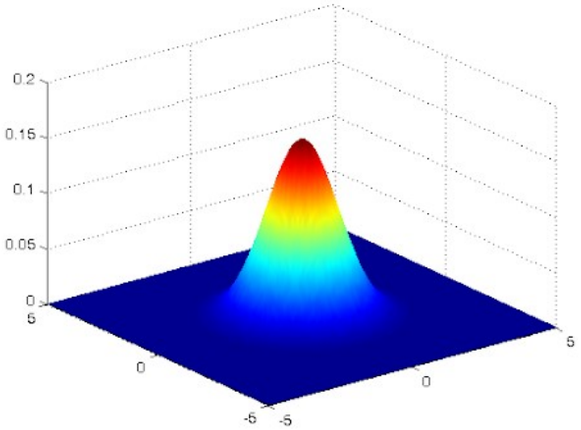


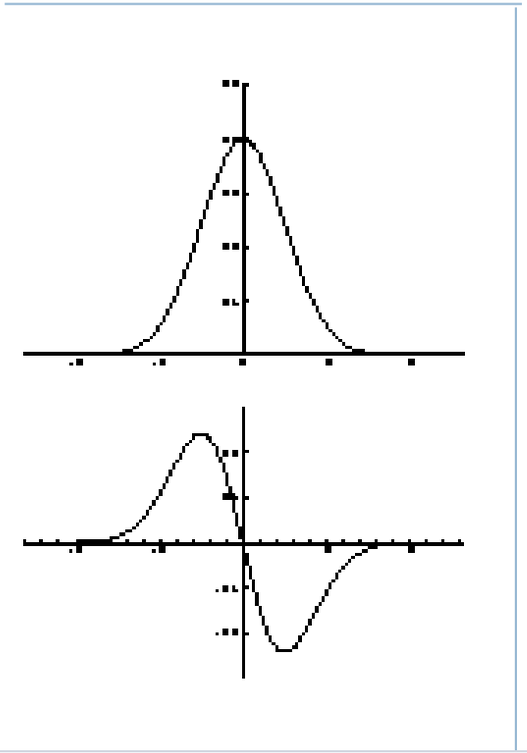
Canny edges

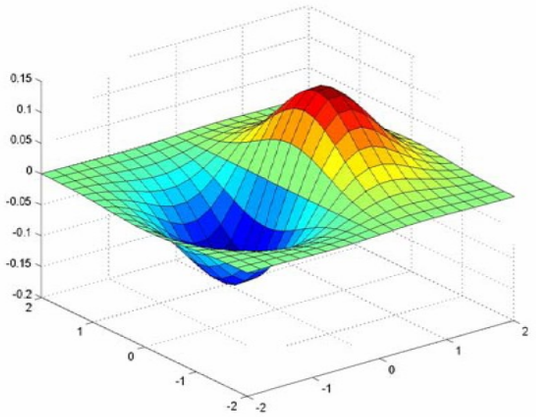
Derivative of Gaussian filter

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{\frac{-(x^2+y^2)}{2\sigma^2}} = \frac{1}{2\pi\sigma^2} e^{\frac{-r^2}{2\sigma^2}} = G(r)$$

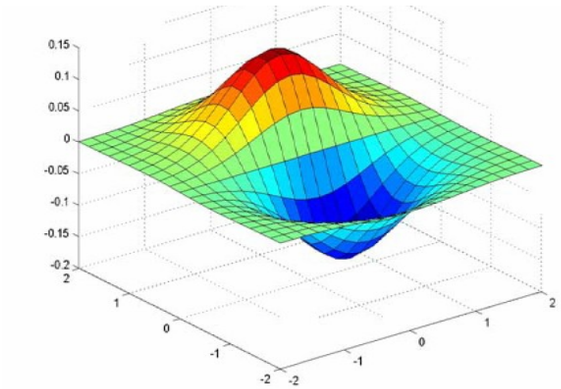
$$G'(r) = \frac{-r}{\sigma^2} G(r)$$



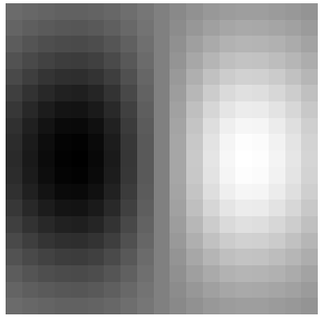


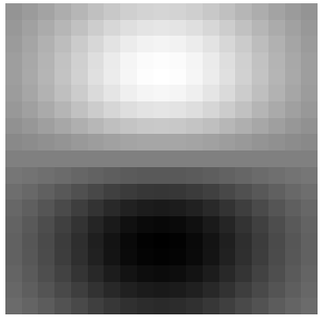


x-direction

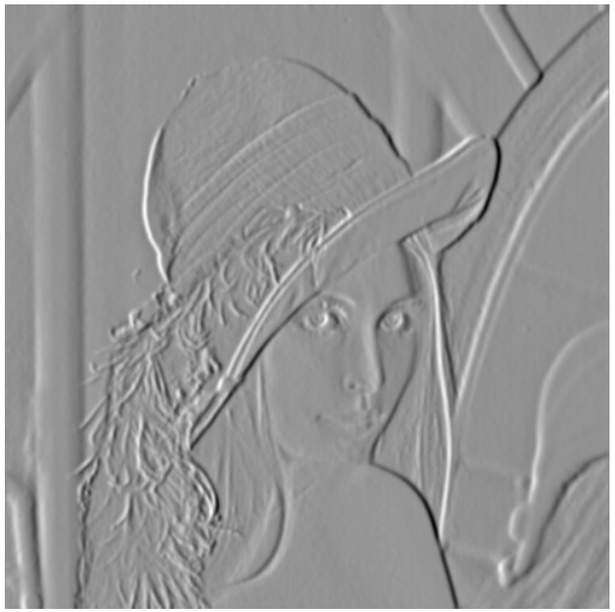


y-direction





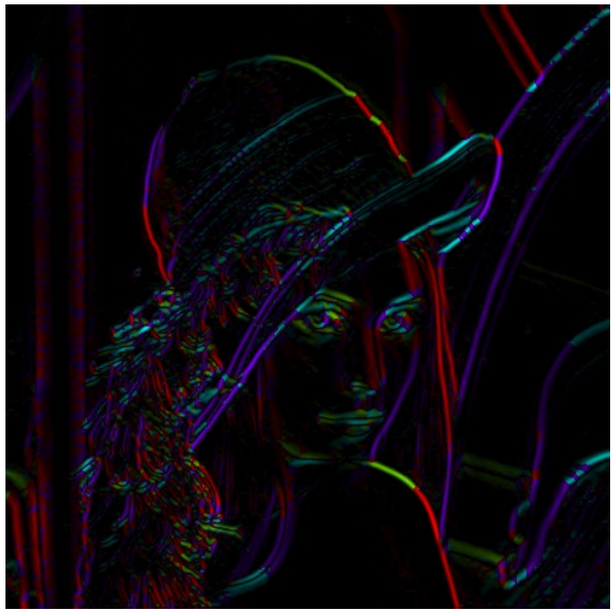
Compute Gradients



X-Derivative of Gaussian

Y-Derivative of Gaussian

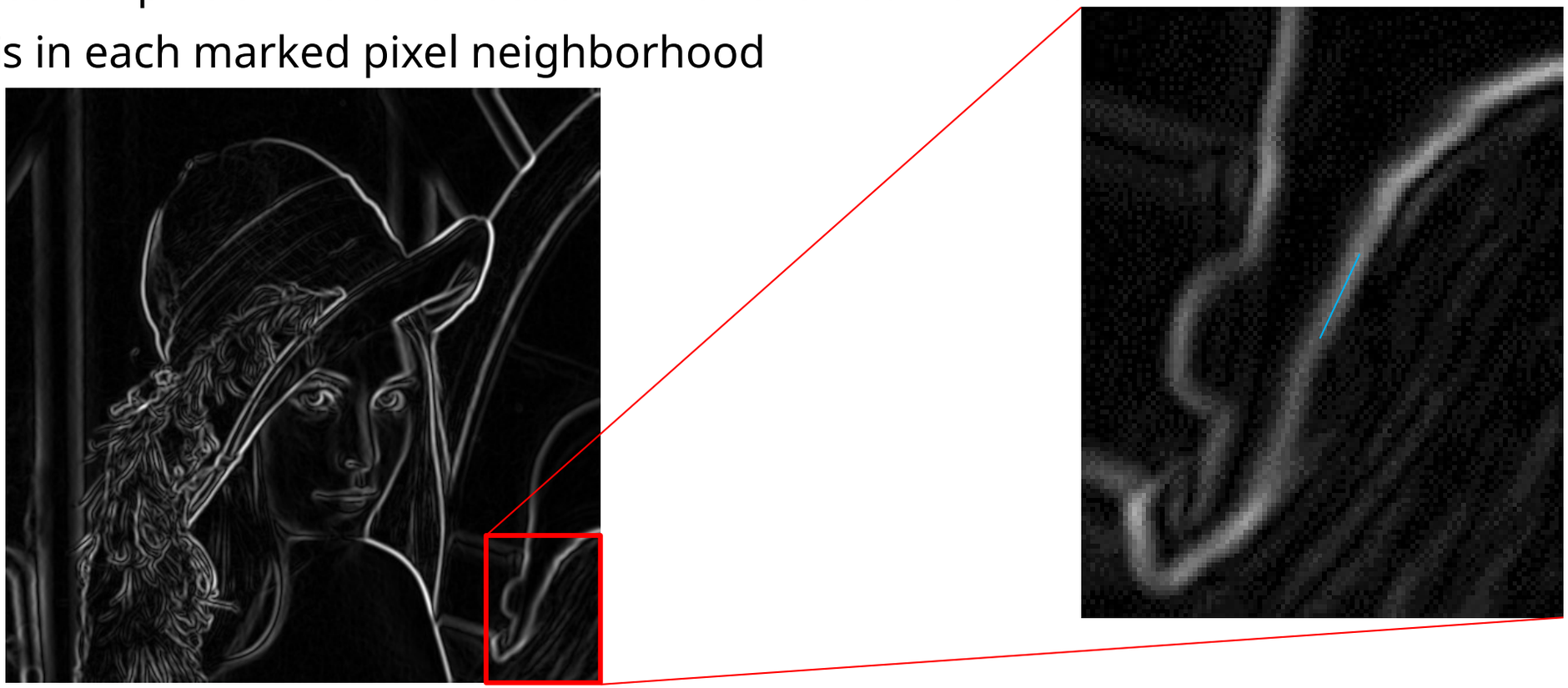
Gradient Magnitude



theta = atan2(Gy, Gx)

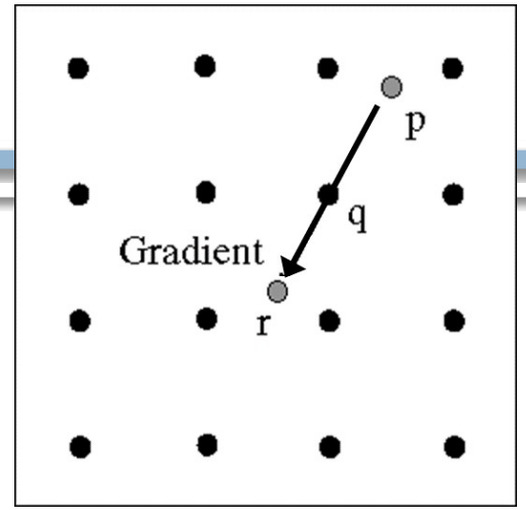
Non-maximum suppression

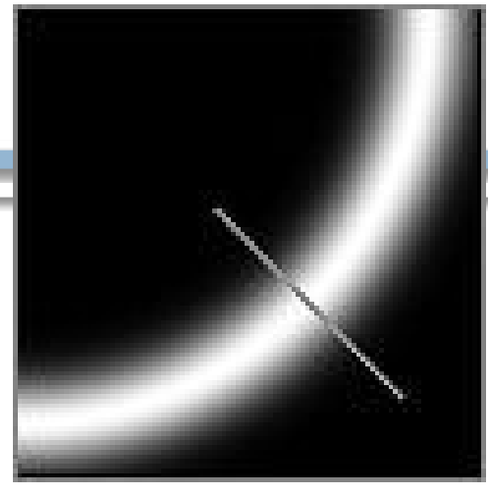
- Edge occurs where gradient reaches a maxima
- Suppress non-maxima gradient even if it passes threshold
- Only eight angle directions possible
  - Suppress all pixels in each direction which are not maxima
  - Do this in each marked pixel neighborhood

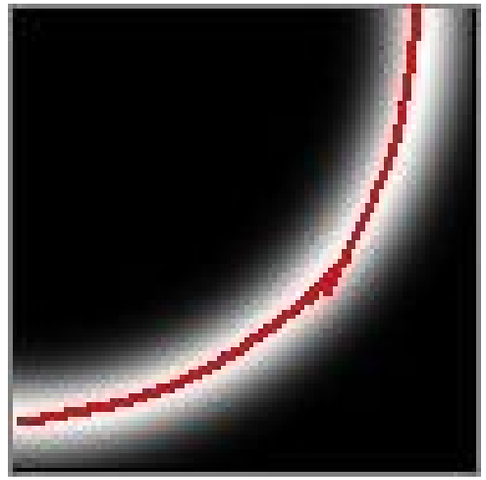


NMS - How

$$M(q) = \begin{cases} |\nabla f(q)| & \text{if } |\nabla f(q)| > |\nabla f(p)| \text{ and } |\nabla f(q)| > |\nabla f(r)| \\ 0 & \text{otherwise} \end{cases}$$





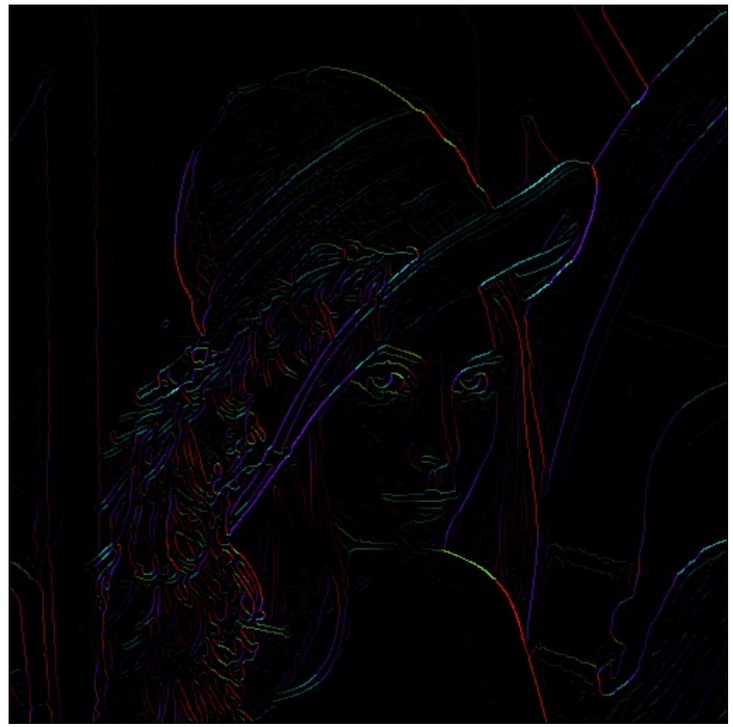


q = current pixel

p, r = neighbor positions along gradient direction

Interpolate (if req.) to get magnitude of these values.

After NMS



Before NMS

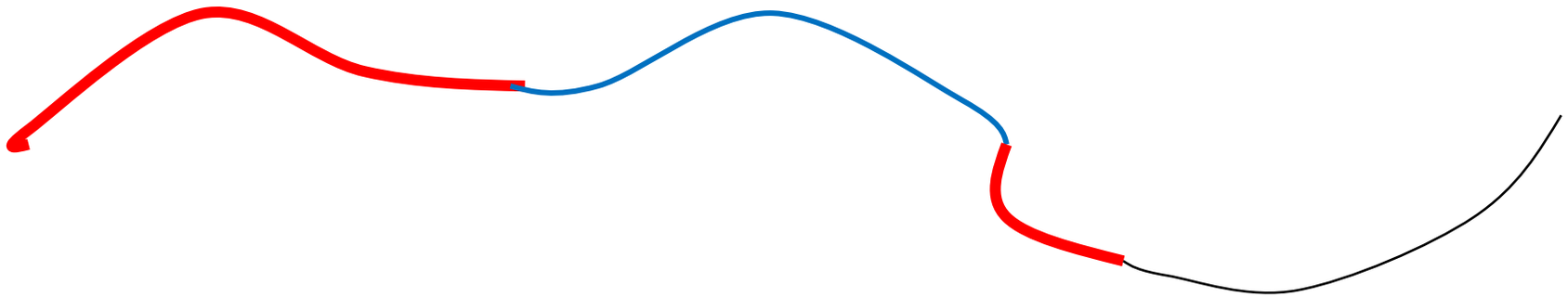
After NMS

Hysteresis thresholding

- Threshold at low/high levels to get weak/strong edge pixels
- Do connected components, starting from strong edge pixels

Hysteresis Threshold

- Canny uses hysteresis threshold to fill up the narrow gap along the edge.
- Uses two threshold - high threshold T_h and low threshold T_l.
- Pixels with gray values greater than T_h is set as the initial edge pixels
- Pixels are set as background if their values are lower than T_l.

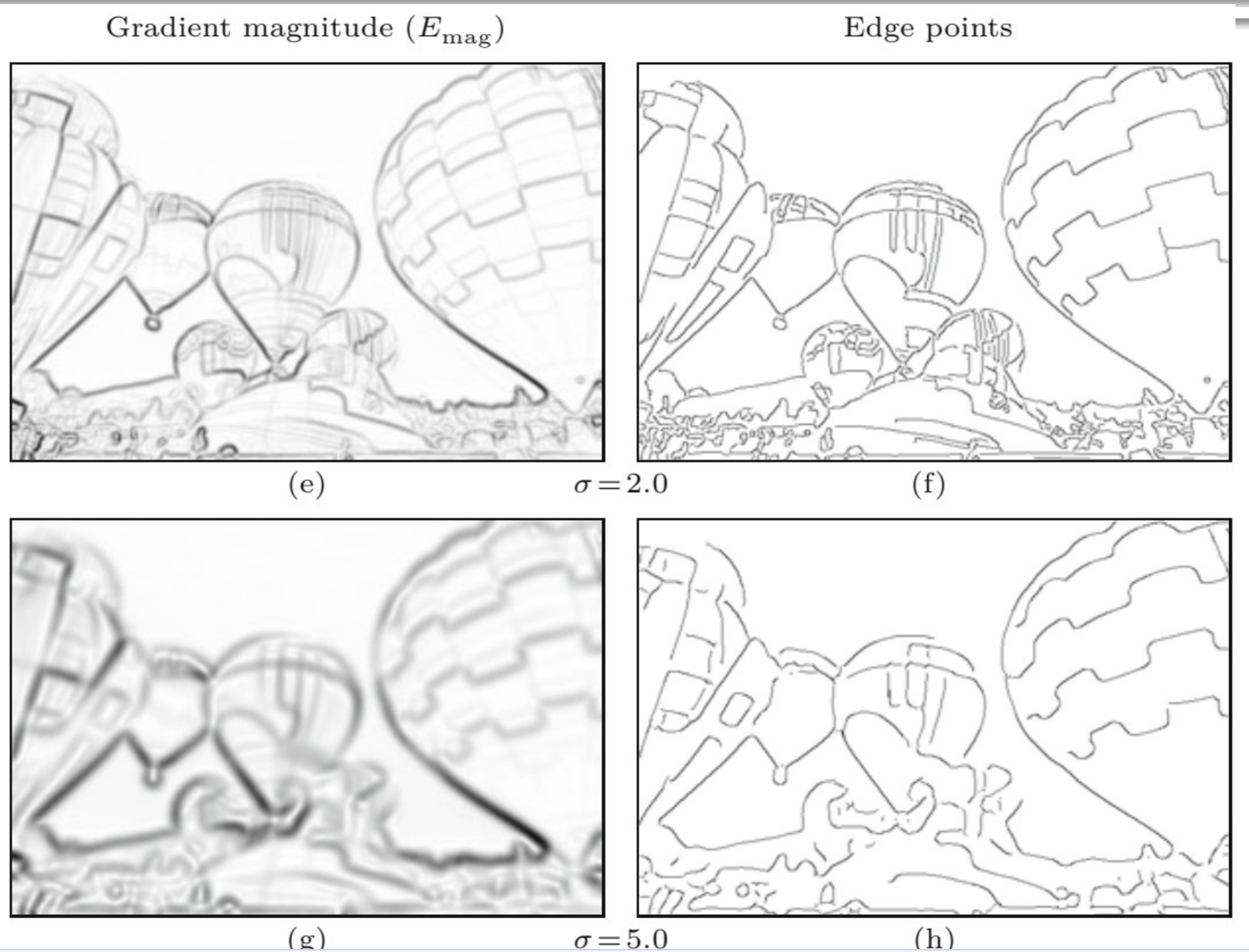


Hysteresis thresholding

Single Threshold (T_{high})

Hysteresis threshold

Canny – effect of sigma



Limitations of edge-based segmentation

Typical failure modes

- **Broken** boundaries due to weak contrast.
- False edges from noise, texture, ribs, vessels, folds, and ultrasound speckle.
- **Double edges** around thick structures.
- Closed contours are not guaranteed.
- Parameter sensitivity across modalities and scanners.

How to improve

- Use **smoothing** and multi-scale edge detection.
- Link edges into contours and impose closure.
- Use region homogeneity and shape priors.
- **Combine** edge maps with thresholding, morphology, active contours, or graph-based optimization.
- Evaluate with clinically meaningful metrics.

Summary

- In this lecture we have begun looking at segmentation, and in particular thresholding
- **Thresholding** is the foundational bedrock of structural image segmentation based on intensity partitioning.
- We saw the basic global thresholding algorithm and its shortcomings
- **Otsu's Method** optimizes global segmentation by maximizing between-class variance, providing an elegant mathematical framework for automated binary conversion on bimodal images.
- We also saw a simple way to overcome some of these limitations using adaptive thresholding
- **Adaptive Thresholding** calculates localized thresholds for individual pixels, making it the superior clinical choice when dealing with non-uniform illumination, noise, and complex anatomy.

Summary

- Edges represent important anatomical transitions but are affected by blur, noise and low contrast.
- Sobel/Prewitt provide simple gradient maps; LoG improves Laplacian robustness; Canny produces thin, connected edges.
- Medical use requires careful smoothing, thresholding, contour linking and validation with expert annotations.
- The final goal is not only an edge map, but a reliable region boundary for measurement or diagnosis support.

Medical image examples: Wikimedia Commons and scikit-image sample data